v修改学习

FANUC 机器人系统
R-30iA 和 R-30iB 控制器
KAREL 参考手册

MARRC75KR07091E Rev D
适用于7.50及更高版本
© 2012 FANUC Robotics America Corporation

关于本手册

本手册可与标有R-30iA或R-J3iC的控制器一起使用。 如果您有一个标有R-J3iC的控制器，则在本手册中您应将R-30iA读作R-J3iC。

版权和商标

此新出版物包含FANUC Robotics America Corporation的专有信息，仅供客户使用。未经FANUC Robotics America Corporation明确书面许可，不得授权其他用途。

FANUC Robotics America Corporation

3900 W. Hamlin Road

罗切斯特山，MI 48309-3253

本手册中包含的说明和规格在本手册获得批准时生效。 FANUC Robotics America Corporation，以下简称FANUC Robotics，保留随时终止型号或更改规格或设计的权利，恕不另行通知，且不承担任何责任。

FANUC机器人手册介绍了FANUC Robotics产品和/或系统的安装，拆卸，连接，操作和编程的说明，规格，图纸，原理图，材料清单，零件，连接和/或程序。这些系统包括机器人，扩展轴，机器人控制器，应用软件，KAREL®编程语言，INSIGHT®视觉设备和特殊工具。

FANUC Robotics建议只允许接受过一个或多个经批准的FANUC机器人培训课程培训的人员安装，操作，使用，执行程序，修理和/或维护FANUC Robotics的产品和/或系统。各自的组成部分。批准的培训需要选择的课程与在客户现场安装的系统类型和应用程序相关。

警告

本设备会产生，使用并辐射射频能量，如果不按照说明手册进行安装和使用，可能会对无线电通信造成干扰。 如法规暂时允许，尚未根据FCC规则第15部分J部分对其符合A类计算设备的限制进行测试，该部分旨在提供针对此类干扰的合理保护。 在住宅区中操作设备可能会造成干扰，在这种情况下，用户需要自费采取任何可能需要的措施来纠正干扰。

FANUC Robotics在其位于密歇根州罗切斯特山的总部定期开展系统和产品课程。 有关其他信息，请联系

FANUC Robotics America Corporation

3900 W. Hamlin Road

罗切斯特山，MI 48309-3253

www.fanucrobotics.com

如需客户帮助，包括技术支持，服务，零件和零件维修以及市场推广请求，请致电1-800-47-ROBOT（1-800-477-6268），全天24小时联系客户资源中心。 国际客户应致电 011-1-248-377-7159。

将您对本手册的意见和建议发送至：

product.documentation@fanucrobotics.com

未经事先书面同意，不得复制，复制，下载，翻译成其他语言，分发或以任何物理或电子格式（包括互联网）发布或以任何方式传播的全部或部分信息。 FANUC Robotics America，Inc。的简介

AccuStat®, ArcTool®, *i*RVision®, KAREL®, PaintTool®,PalletTool®, SOCKETS®, SpotTool®, SpotWorks®, and TorchMate®are Registered Trademarks of FANUC Robotics.

FANUC Robotics保留所有专有权利，包括但不限于商标和商号权利，名称如下：

AccuAir™, AccuCal™, AccuChop™, AccuFlow™, AccuPath™, AccuSeal™, ARC Mate™, ARC Mate Sr.™, ARC Mate System 1™, ARC Mate System 2™, ARC Mate System 3™, ARC Mate System 4™, ARC Mate System 5™, ARCWorks Pro™, AssistTool™, AutoNormal™, AutoTCP™, BellTool™, BODYWorks™, Cal Mate™, Cell Finder™, Center Finder™, Clean Wall™, DualARM™, LR Tool™, MIG Eye™, MotionParts™, MultiARM™, NoBots™, Paint Stick™, PaintPro™, PaintTool 100™, PAINTWorks™, PAINTWorks II™, PAINTWorks III™, PalletMate™, PalletMate PC™, PalletTool PC™, PayloadID™, RecipTool™, RemovalTool™, Robo Chop™, Robo Spray™, S-420i™, S-430i™, ShapeGen™, SoftFloat™, SOFT PARTS™, SpotTool+™, SR Mate™, SR ShotTool™, SureWeld™, SYSTEM R-J2 Controller™, SYSTEM R-J3 Controller™, SYSTEM R-J3*i*B Controller™, SYSTEM R-J3*i*C Controller™, SYSTEM R-30*i*A Controller™, SYSTEM R-30*i*B Controller™, TCP Mate™, TorchMate™, TripleARM™, TurboMove™, visLOC™, visPRO-3D™, visTRAC™, WebServer™, WebTP™, and YagTool™.

专利

以下一项或多项美国专利可能与本手册中描述的FANUC机器人产品有关。

FRA Patent List

4,630,567 4,639,878 4,707,647 4,708,175 4,708,580 4,942,539 4,984,745 5,238,029 5,239,739 5,272,805 5,293,107 5,293,911 5,331,264 5,367,944 5,373,221 5,421,218 5,434,489 5,644,898 5,670,202 5,696,687 5,737,218 5,823,389 5,853,027 5,887,800 5,941,679 5,959,425 5,987,726 6,059,092 6,064,168 6,070,109 6,086,294 6,122,062 6,147,323 6,204,620 6,243,621 6,253,799 6,285,920 6,313,595 6,325,302 6,345,818 6,356,807 6,360,143 6,378,190 6,385,508 6,425,177 6,477,913 6,490,369 6,518,980 6,540,104 6,541,757 6,560,513 6,569,258 6,612,449 6,703,079 6,705,361 6,726,773 6,768,078 6,845,295 6,945,483 7,149,606 7,149,606 7,211,978 7,266,422 7,399,363

FANUC LTD Patent List

4,571,694 4,626,756 4,700,118 4,706,001 4,728,872 4,732,526 4,742,207 4,835,362 4,894,596 4,899,095 4,920,248 4,931,617 4,934,504 4,956,594 4,967,125 4,969,109 4,970,370 4,970,448 4,979,127 5,004,968 5,006,035 5,008,834 5,063,281 5,066,847 5,066,902 5,093,552 5,107,716 5,111,019 5,130,515 5,136,223 5,151,608 5,170,109 5,189,351 5,267,483 5,274,360 5,292,066 5,300,868 5,304,906 5,313,563 5,319,443 5,325,467 5,327,057 5,329,469 5,333,242 5,337,148 5,371,452 5,375,480 5,418,441 5,432,316 5,440,213 5,442,155 5,444,612 5,449,875 5,451,850 5,461,478 5,463,297 5,467,003 5,471,312 5,479,078 5,485,389 5,485,552 5,486,679 5,489,758 5,493,192 5,504,766 5,511,007 5,520,062 5,528,013 5,532,924 5,548,194 5,552,687 5,558,196 5,561,742 5,570,187 5,570,190 5,572,103 5,581,167 5,582,750 5,587,635 5,600,759 5,608,299 5,608,618 5,624,588 5,630,955 5,637,969 5,639,204 5,641,415 5,650,078 5,658,121 5,668,628 5,687,295 5,691,615 5,698,121 5,708,342 5,715,375 5,719,479 5,727,132 5,742,138 5,742,144 5,748,854 5,749,058 5,760,560 5,773,950 5,783,922 5,799,135 5,812,408 5,841,257 5,845,053 5,872,894 5,887,122 5,911,892 5,912,540 5,920,678 5,937,143 5,980,082 5,983,744 5,987,591 5,988,850 6,023,044 6,032,086 6,040,554 6,059,169 6,088,628 6,097,169 6,114,824 6,124,693 6,140,788 6,141,863 6,157,155 6,160,324 6,163,124 6,177,650 6,180,898 6,181,096 6,188,194 6,208,105 6,212,444 6,219,583 6,226,181 6,236,011 6,236,896 6,250,174 6,278,902 6,279,413 6,285,921 6,298,283 6,321,139 6,324,443 6,328,523 6,330,493 6,340,875 6,356,671 6,377,869 6,382,012 6,384,371 6,396,030 6,414,711 6,424,883 6,431,018 6,434,448 6,445,979 6,459,958 6,463,358 6,484,067 6,486,629 6,507,165 6,654,666 6,665,588 6,680,461 6,696,810 6,728,417 6,763,284 6,772,493 6,845,296 6,853,881 6,888,089 6,898,486 6,917,837 6,928,337 6,965,091 6,970,802 7,038,165 7,069,808 7,084,900 7,092,791 7,133,747 7,143,100 7,149,602 7,131,848 7,161,321 7,171,041 7,174,234 7,173,213 7,177,722 7,177,439 7,181,294 7,181,313 7,280,687 7,283,661 7,291,806 7,299,713 7,315,650 7,324,873 7,328,083 7,330,777 7,333,879 7,355,725 7,359,817 7,373,220 7,376,488 7,386,367 7,464,623 7,447,615 7,445,260 7,474,939 7,486,816 7,495,192 7,501,778 7,502,504 7,508,155 7,512,459 7,525,273 7,526,121

VersaBell, ServoBell and SpeedDock Patents Pending.

约定

本手册包含对人员，设备，软件和数据安全至关重要的信息。 此信息由文本中的标题和框表示。



警告
警告下出现的信息涉及人员保护。 它以盒装和粗体显示，以区别于其他文本。



警告
小心下出现的信息涉及设备，软件和数据的保护。 它被装箱以将其与其他文本区分开来。

注意NOTE旁边显示的信息涉及相关信息或有用提示。

目录

关于本手册 i

安全 xxvii

Chapter 1 KAREL LANGUAGE OVERVIEW 1–1

1.1 OVERVIEW 1–2

1.2 KAREL PROGRAMMING LANGUAGE 1–2
1.2.1 Overview 1–2
1.2.2 Creating a Program 1–4
1.2.3 Translating a Program 1–4
1.2.4 Loading Program Logic and Data 1–4
1.2.5 Executing a Program 1–5
1.2.6 Execution History 1–5
1.2.7 Program Structure 1–5

1.3 SYSTEM SOFTWARE 1–7
1.3.1 Software Components 1–7
1.3.2 Supported Robots 1–7

1.4 CONTROLLER 1–8
1.4.1 Memory 1–8
1.4.2 Input/Output System 1–10
1.4.3 User Interface Devices 1–10

Chapter 2 LANGUAGE ELEMENTS 2–1

2.1 LANGUAGE COMPONENTS 2–2
2.1.1 Character Set 2–2
2.1.2 Operators 2–5
2.1.3 Reserved Words 2–6
2.1.4 User-Defined Identifiers 2–7
2.1.5 Labels 2–8
2.1.6 Predefined Identifiers 2–8
2.1.7 System Variables 2–10
2.1.8 Comments 2–10

2.2 **TRANSLATOR DIRECTIVES** 2–11

2.3 DATA TYPES 2–13

2.4 USER-DEFINED DATA TYPES AND STRUCTURES 2–14
2.4.1 User-Defined Data Types 2–14
2.4.2 User-Defined Data Structures 2–16

2.5 ARRAYS 2–18
2.5.1 Multi-Dimensional Arrays 2–19
2.5.2 Variable-Sized Arrays 2–20

Chapter 3 USE OF OPERATORS 3–1

3.1 EXPRESSIONS AND ASSIGNMENTS 3–2
3.1.1 Rule for Expressions and Assignments 3–2
3.1.2 Evaluation of Expressions and Assignments 3–2
3.1.3 Variables and Expressions 3–4
3.2 OPERATIONS 3–4
3.2.1 Arithmetic Operations 3–5
3.2.2 Relational Operations 3–6
3.2.3 Boolean Operations 3–7
3.2.4 Special Operations 3–8

Chapter 4 PROGRAM CONTROL 4–1
4.1 OVERVIEW 4–2
4.2 PROGRAM CONTROL STRUCTURES 4–2
4.2.1 Alternation Control Structures 4–2
4.2.2 Looping Control Statements 4–3
4.2.3 Unconditional Branch Statement 4–3
4.2.4 Execution Control Statements 4–3
4.2.5 Condition Handlers 4–4

Chapter 5 ROUTINES 5–1
5.1 ROUTINE EXECUTION 5–2
5.1.1 Declaring Routines 5–2
5.1.2 Invoking Routines 5–5
5.1.3 Returning from Routines 5–7
5.1.4 Scope of Variables 5–8
5.1.5 Parameters and Arguments 5–9
5.1.6 Stack Usage 5–13
5.2 BUILT- IN ROUTINES 5–15

Chapter 6 CONDITION HANDLERS 6–1
6.1 CONDITION HANDLER OPERATIONS 6–3
6.1.1 Global Condition Handlers 6–3
6.2 CONDITIONS 6–6
6.2.1 Port_Id Conditions 6–7
6.2.2 Relational Conditions 6–7
6.2.3 System and Program Event Conditions 6–8
6.3 ACTIONS 6–11
6.3.1 Assignment Actions 6–11
6.3.2 Motion Related Actions 6–13
6.3.3 Routine Call Actions 6–13
6.3.4 Miscellaneous Actions 6–14

Chapter 7 FILE INPUT/OUTPUT OPERATIONS 7–1
7.1 OVERVIEW 7–3
7.2 FILE VARIABLES 7–3
7.3 OPEN FILE STATEMENT 7–4
7.3.1 Setting File and Port Attributes 7–5
7.3.2 File String 7–10
7.3.3 Usage String 7–11
7.4 CLOSE FILE STATEMENT 7–14

7.5 READ STATEMENT 7–14
7.6 WRITE STATEMENT 7–16
7.7 INPUT/OUTPUT BUFFER 7–17
7.8 FORMATTING TEXT (ASCII) INPUT/OUTPUT 7–18
7.8.1 Formatting INTEGER Data Items 7–19
7.8.2 Formatting REAL Data Items 7–22
7.8.3 Formatting BOOLEAN Data Items 7–25
7.8.4 Formatting STRING Data Items 7–27
7.8.5 Formatting VECTOR Data Items 7–31
7.8.6 Formatting Positional Data Items 7–32
7.9 FORMATTING BINARY INPUT/OUTPUT 7–34
7.9.1 Formatting INTEGER Data Items 7–35
7.9.2 Formatting REAL Data Items 7–36
7.9.3 Formatting BOOLEAN Data Items 7–36
7.9.4 Formatting STRING Data Items 7–36
7.9.5 Formatting VECTOR Data Items 7–37
7.9.6 Formatting POSITION Data Items 7–37
7.9.7 Formatting XYZWPR Data Items 7–37
7.9.8 Formatting XYZWPREXT Data Items 7–38
7.9.9 Formatting JOINTPOS Data Items 7–38
7.10 USER INTERFACE TIPS 7–38
7.10.1 USER Menu on the Teach Pendant 7–38
7.10.2 USER Menu on the CRT/KB 7–40

Chapter 8 POSITION DATA 8–1
8.1 OVERVIEW 8–2
8.2 POSITIONAL DATA 8–2
8.3 FRAMES OF REFERENCE 8–3
8.3.1 World Frame 8–4
8.3.2 User Frame (UFRAME) 8–5
8.3.3 Tool Definition (UTOOL) 8–5
8.3.4 Using Frames in the Teach Pendant Editor (TP) 8–5
8.4 JOG COORDINATE SYSTEMS 8–6

Chapter 9 FILE SYSTEM 9–1
9.1 OVERVIEW 9–2
9.2 FILE SPECIFICATION 9–3
9.2.1 Device Name 9–3
9.2.2 File Name 9–4
9.2.3 File Type 9–5
9.3 STORAGE DEVICE ACCESS 9–6
9.3.1 Overview 9–7
9.3.2 Memory File Devices 9–13
9.3.3 Virtual Devices 9–14
9.3.4 File Pipes 9–16
9.4 FILE ACCESS 9–20
9.5 FORMATTING XML INPUT 9–20
9.5.1 Overview 9–20
9.5.2 Installation Sequence 9–21
9.5.3 Example KAREL Program Referencing an XML File 9–22

9.5.4 Parse Errors 9–26
9.6 MEMORY DEVICE 9–27

Chapter 10 DICTIONARIES AND FORMS 10–1
10.1 OVERVIEW 10–3
10.2 CREATING USER DICTIONARIES 10–3
10.2.1 Dictionary Syntax 10–3
10.2.2 Dictionary Element Number 10–4
10.2.3 Dictionary Element Name 10–5
10.2.4 Dictionary Cursor Positioning 10–5
10.2.5 Dictionary Element Text 10–6
10.2.6 Dictionary Reserved Word Commands 10–8
10.2.7 Character Codes 10–10
10.2.8 Nesting Dictionary Elements 10–10
10.2.9 Dictionary Comment 10–11
10.2.10 Generating a KAREL Constant File 10–11
10.2.11 Compressing and Loading Dictionaries on the Controller 10–11
10.2.12 Accessing Dictionary Elements from a KAREL Program 10–12
10.3 CREATING USER FORMS 10–13
10.3.1 Form Syntax 10–14
10.3.2 Form Attributes 10–15
10.3.3 Form Title and Menu Label 10–16
10.3.4 Form Menu Text 10–17
10.3.5 Form Selectable Menu Item 10–18
10.3.6 Edit Data Item 10–19
10.3.7 Dynamic Forms using Tree View 10–26
10.3.8 Non-Selectable Text 10–27
10.3.9 Display Only Data Items 10–27
10.3.10 Cursor Position Attributes 10–27
10.3.11 Form Reserved Words and Character Codes 10–28
10.3.12 Form Function Key Element Name or Number 10–30
10.3.13 Form Function Key Using a Variable 10–31
10.3.14 Form Help Element Name or Number 10–31
10.3.15 Teach Pendant Form Screen 10–32
10.3.16 CRT/KB Form Screen 10–32
10.3.17 Form File Naming Convention 10–33
10.3.18 Compressing and Loading Forms on the Controller 10–34
10.3.19 Displaying a Form 10–36

Chapter 11 SOCKET MESSAGING 11–1
11.1 OVERVIEW 11–2
11.2 SYSTEM REQUIREMENTS 11–2
11.2.1 Overview 11–2
11.2.2 Software Requirements 11–2
11.2.3 Hardware Requirements 11–2
11.3 CONFIGURING THE SOCKET MESSAGING OPTION 11–3
11.3.1 Overview 11–3
11.3.2 Setting up a Server Tag 11–3
11.3.3 Setting up a Client Tag 11–6
11.4 SOCKET MESSAGING AND KAREL 11–9
11.4.1 Overview 11–9
11.4.2 MSG_CONN(***string, integer***) 11–9

11.4.3 MSG_DISCO(***string, integer***) 11–9
11.4.4 MSG_PING(***string, integer***) 11–10
11.4.5 Exchanging Data during a Socket Messaging Connection 11–10
11.5 NETWORK PERFORMANCE 11–10
11.5.1 Overview 11–10
11.5.2 Guidelines for a Good Implementation 11–11
11.6 PROGRAMMING EXAMPLES 11–11
11.6.1 11–11
11.6.2 A KAREL Client Application 11–11
11.6.3 A KAREL Server Application 11–14
11.6.4 ANSI C Loopback Client Example 11–16

Chapter 12 SYSTEM VARIABLES 12–1
12.1 ACCESS RIGHTS 12–2
12.2 STORAGE 12–3

Chapter 13 KAREL COMMAND LANGUAGE (KCL) 13–1
13.1 COMMAND FORMAT 13–2
13.1.1 Default Program 13–2
13.1.2 Variables and Data Types 13–3
13.2 PROGRAM CONTROL COMMANDS 13–3
13.3 ENTERING COMMANDS 13–3
13.3.1 Abbreviations 13–4
13.3.2 Error Messages 13–4
13.3.3 Subdirectories 13–4
13.4 COMMAND PROCEDURES 13–4
13.4.1 Command Procedure Format 13–5
13.4.2 Creating Command Procedures 13–6
13.4.3 Error Processing 13–6
13.4.4 Executing Command Procedures 13–6

Chapter 14 INPUT/OUTPUT SYSTEM 14–1
14.1 USER-DEFINED SIGNALS 14–2
14.1.1 DIN and DOUT Signals 14–2
14.1.2 GIN and GOUT Signals 14–3
14.1.3 AIN and AOUT Signals 14–3
14.1.4 Hand Signals 14–5
14.2 SYSTEM-DEFINED SIGNALS 14–5
14.2.1 Robot Digital Input and Output Signals (RDI/RDO) 14–6
14.2.2 Operator Panel Input and Output Signals (OPIN/OPOUT) 14–6
14.2.3 Teach Pendant Input and Output Signals (TPIN/TPOUT) 14–17
14.3 Serial Input/Output 14–22
14.3.1 Serial Input/Output 14–22

Chapter 15 MULTI-TASKING 15–1
15.1 MULTI-TASKING TERMINOLOGY 15–2
15.2 INTERPRETER ASSIGNMENT 15–3
15.3 MOTION CONTROL 15–3
15.4 TASK SCHEDULING 15–4
15.4.1 Priority Scheduling 15–5

15.4.2 Time Slicing 15–6
15.5 STARTING TASKS 15–6
15.5.1 Running Programs from the User Operator Panel (UOP) PNS Signal 15–7
15.5.2 Child Tasks 15–7
15.6 TASK CONTROL AND MONITORING 15–7
15.6.1 From TPP Programs 15–8
15.6.2 From KAREL Programs 15–8
15.6.3 From KCL 15–8
15.7 USING SEMAPHORES AND TASK SYNCHRONIZATION 15–9
15.8 USING QUEUES FOR TASK COMMUNICATIONS 15–14

Appendix A KAREL LANGUAGE ALPHABETICAL DESCRIPTION A–1
A.1 OVERVIEW A–9
A.2 - A - KAREL LANGUAGE DESCRIPTION A–18
A.2.1 ABORT Action A–18
A.2.2 ABORT Condition A–18
A.2.3 ABORT Statement A–19
A.2.4 ABORT_TASK Built-In Procedure A–19
A.2.5 ABS Built-In Function A–20
A.2.6 ACOS Built-In Function A–20
A.2.7 ACT_SCREEN Built-In Procedure A–21
A.2.8 ACT_TBL Built-In Procedure A–22
A.2.9 ADD_BYNAMEPC Built-In Procedure A–24
A.2.10 ADD_DICT Built-In Procedure A–25
A.2.11 ADD_INTPC Built-In Procedure A–26
A.2.12 ADD_REALPC Built-In Procedure A–27
A.2.13 ADD_STRINGPC Built-In Procedure A–29
A.2.14 %ALPHABETIZE Translator Directive A–30
A.2.15 APPEND_NODE Built-In Procedure A–30
A.2.16 APPEND_QUEUE Built-In Procedure A–31
A.2.17 APPROACH Built-In Function A–32
A.2.18 ARRAY Data Type A–33
A.2.19 ARRAY_LEN Built-In Function A–34
A.2.20 ASIN Built-In Function A–34
A.2.21 Assignment Action A–35
A.2.22 Assignment Statement A–36
A.2.23 ATAN2 Built-In Function A–38
A.2.24 ATTACH Statement A–39
A.2.25 ATT_WINDOW_D Built-In Procedure A–39
A.2.26 ATT_WINDOW_S Built-In Procedure A–40
A.2.27 AVL_POS_NUM Built-In Procedure A–41
A.3 - B - KAREL LANGUAGE DESCRIPTION A–42
A.3.1 BOOLEAN Data Type A–42
A.3.2 BYNAME Built-In Function A–43
A.3.3 BYTE Data Type A–44
A.3.4 BYTES_AHEAD Built-In Procedure A–45
A.3.5 BYTES_LEFT Built-In Function A–46
A.4 - C - KAREL LANGUAGE DESCRIPTION A–47
A.4.1 CALL_PROG Built-In Procedure A–47
A.4.2 CALL_PROGLIN Built-In Procedure A–48
A.4.3 CANCEL Action A–49
A.4.4 CANCEL Statement A–49

A.4.5 CANCEL FILE Statement A–50
A.4.6 CHECK_DICT Built-In Procedure A–51
A.4.7 CHECK_EPOS Built-In Procedure A–52
A.4.8 CHECK_NAME Built-In Procedure A–53
A.4.9 CHR Built-In Function A–53
A.4.10 CLEAR Built-In Procedure A–54
A.4.11 CLEAR_SEMA Built-In Procedure A–55
A.4.12 CLOSE FILE Statement A–55
A.4.13 CLOSE HAND Statement A–56
A.4.14 CLOSE_TPE Built-In Procedure A–56
A.4.15 CLR_IO_STAT Built-In Procedure A–57
A.4.16 CLR_PORT_SIM Built-In Procedure A–57
A.4.17 CLR_POS_REG Built-In Procedure A–58
A.4.18 %CMOSVARS Translator Directive A–59
A.4.19 %CMOS2SHADOW Translator Directive A–59
A.4.20 CNC_DYN_DISB Built-In Procedure A–59
A.4.21 CNC_DYN_DISE Built-In Procedure A–60
A.4.22 CNC_DYN_DISI Built-In Procedure A–61
A.4.23 CNC_DYN_DISP Built-In Procedure A–62
A.4.24 CNC_DYN_DISR Built-In Procedure A–62
A.4.25 CNC_DYN_DISS Built-In Procedure A–63
A.4.26 CNCL_STP_MTN Built-In Procedure A–64
A.4.27 CNV_CNF_STRG Built-In Procedure A–64
A.4.28 CNV_CONF_STR Built-In Procedure A–65
A.4.29 CNV_INT_STR Built-In Procedure A–66
A.4.30 CNV_JPOS_REL Built-In Procedure A–67
A.4.31 CNV_REAL_STR Built-In Procedure A–67
A.4.32 CNV_REL_JPOS Built-In Procedure A–68
A.4.33 CNV_STR_CONF Built-In Procedure A–69
A.4.34 CNV_STR_INT Built-In Procedure A–70
A.4.35 CNV_STR_REAL Built-In Procedure A–71
A.4.36 CNV_STR_TIME Built-In Procedure A–71
A.4.37 CNV_TIME_STR Built-In Procedure A–72
A.4.38 %COMMENT Translator Directive A–73
A.4.39 COMPARE_FILE Built-in Procedure A–73
A.4.40 CONDITION...ENDCONDITION Statement A–75
A.4.41 CONFIG Data Type A–77
A.4.42 CONNECT TIMER Statement A–78
A.4.43 CONTINUE Action A–79
A.4.44 CONTINUE Condition A–79
A.4.45 CONT_TASK Built-In Procedure A–80
A.4.46 COPY_FILE Built-In Procedure A–81
A.4.47 COPY_PATH Built-In Procedure A–82
A.4.48 COPY_QUEUE Built-In Procedure A–83
A.4.49 COPY_TPE Built-In Procedure A–85
A.4.50 COS Built-In Function A–86
A.4.51 CR Input/Output Item A–86
A.4.52 CREATE_TPE Built-In Procedure A–87
A.4.53 CREATE_VAR Built-In Procedure A–88
A.4.54 %CRTDEVICE A–90
A.4.55 CURJPOS Built-In Function A–91
A.4.56 CURPOS Built-In Function A–92
A.4.57 CURR_PROG Built-In Function A–93
A.5 - D - KAREL LANGUAGE DESCRIPTION A–93
A.5.1 DAQ_CHECKP Built-In Procedure A–93

A.5.2 DAQ_REGPIPE Built-In Procedure A–94
A.5.3 DAQ_START Built-In Procedure A–96
A.5.4 DAQ_STOP Built-In Procedure A–98
A.5.5 DAQ_UNREG Built-In Procedure A–99
A.5.6 DAQ_WRITE Built-In Procedure A–100
A.5.7 %DEFGROUP Translator Directive A–102
A.5.8 DEF_SCREEN Built-In Procedure A–102
A.5.9 DEF_WINDOW Built-In Procedure A–103
A.5.10 %DELAY Translator Directive A–104
A.5.11 DELAY Statement A–105
A.5.12 DELETE_FILE Built-In Procedure A–106
A.5.13 DELETE_NODE Built-In Procedure A–107
A.5.14 DELETE_QUEUE Built-In Procedure A–107
A.5.15 DEL_INST_TPE Built-In Procedure A–108
A.5.16 DET_WINDOW Built-In Procedure A–109
A.5.17 DISABLE CONDITION Action A–109
A.5.18 DISABLE CONDITION Statement A–110
A.5.19 DISCONNECT TIMER Statement A–111
A.5.20 DISCTRL_ALPH Built_In Procedure A–112
A.5.21 DISCTRL_FORM Built_In Procedure A–114
A.5.22 DISCTRL_LIST Built-In Procedure A–116
A.5.23 DISCTRL_PLMN Built-In Procedure A–117
A.5.24 DISCTRL_SBMN Built-In Procedure A–119
A.5.25 DISCTRL_TBL Built-In Procedure A–122
A.5.26 DISMOUNT_DEV Built-In Procedure A–125
A.5.27 DISP_DAT_T Data Type A–125
A.5.28 DOSFILE_INF Built-In Procedure A–127

A.6 - E - KAREL LANGUAGE DESCRIPTION A–128
A.6.1 ENABLE CONDITION Action A–128
A.6.2 ENABLE CONDITION Statement A–129
A.6.3 %ENVIRONMENT Translator Directive A–129
A.6.4 ERR_DATA Built-In Procedure A–131
A.6.5 ERROR Condition A–132
A.6.6 EVAL Clause A–133
A.6.7 EVENT Condition A–134
A.6.8 EXP Built-In Function A–134

A.7 - F - KAREL LANGUAGE DESCRIPTION A–135
A.7.1 FILE Data Type A–135
A.7.2 FILE_LIST Built-In Procedure A–135
A.7.3 FOR...ENDFOR Statement A–137
A.7.4 FORCE_SPMENU Built-In Procedure A–138
A.7.5 FORMAT_DEV Built-In Procedure A–141
A.7.6 FRAME Built-In Function A–142
A.7.7 FROM Clause A–144

A.8 - G - KAREL LANGUAGE DESCRIPTION A–145
A.8.1 GET_ATTR_PRG Built-In Procedure A–145
A.8.2 GET_FILE_POS Built-In Function A–147
A.8.3 GET_JPOS_REG Built-In Function A–148
A.8.4 GET_JPOS_TPE Built-In Function A–148
A.8.5 GET_PORT_ASG Built-in Procedure A–149
A.8.6 GET_PORT_ATR Built-In Function A–151
A.8.7 GET_PORT_CMT Built-In Procedure A–153
A.8.8 GET_PORT_MOD Built-In Procedure A–154
A.8.9 GET_PORT_SIM Built-In Procedure A–155

A.8.10 GET_PORT_VAL Built-In Procedure A–156
A.8.11 GET_POS_FRM Built-In Procedure A–157
A.8.12 GET_POS_REG Built-In Function A–158
A.8.13 GET_POS_TPE Built-In Function A–158
A.8.14 GET_POS_TYP Built-In Procedure A–159
A.8.15 GET_PREG_CMT Built-In-Procedure A–160
A.8.16 GET_QUEUE Built-In Procedure A–161
A.8.17 GET_REG Built-In Procedure A–162
A.8.18 GET_REG_CMT A–163
A.8.19 GET_SREG_CMT Builtin Procedure A–163
A.8.20 GET_STR_REG Built-In Procedure A–164
A.8.21 GET_TIME Built-In Procedure A–164
A.8.22 GET_TPE_CMT Built-in Procedure A–165
A.8.23 GET_TPE_PRM Built-in Procedure A–166
A.8.24 GET_TSK_INFO Built-In Procedure A–168
A.8.25 GET_USEC_SUB Built-In Procedure A–170
A.8.26 GET_USEC_TIM Built-In Function A–171
A.8.27 GET_VAR Built-In Procedure A–171
A.8.28 GO TO Statement A–175

A.9 - H - KAREL LANGUAGE DESCRIPTION A–176
A.9.1 HOLD Action A–176
A.9.2 HOLD Statement A–177

A.10 - I - KAREL LANGUAGE DESCRIPTION A–178
A.10.1 IF ... ENDIF Statement A–178
A.10.2 IN Clause A–179
A.10.3 %INCLUDE Translator Directive A–180
A.10.4 INDEX Built-In Function A–181
A.10.5 INI_DYN_DISB Built-In Procedure A–181
A.10.6 INI_DYN_DISE Built-In Procedure A–183
A.10.7 INI_DYN_DISI Built-In Procedure A–184
A.10.8 INI_DYN_DISP Built-In Procedure A–186
A.10.9 INI_DYN_DISR Built-In Procedure A–187
A.10.10 INI_DYN_DISS Built-In Procedure A–188
A.10.11 INIT_QUEUE Built-In Procedure A–189
A.10.12 INIT_TBL Built-In Procedure A–190
A.10.13 IN_RANGE Built-In Function A–201
A.10.14 INSERT_NODE Built-In Procedure A–202
A.10.15 INSERT_QUEUE Built-In Procedure A–203
A.10.16 INTEGER Data Type A–204
A.10.17 INV Built-In Function A–205
A.10.18 IO_MOD_TYPE Built-In Procedure A–206
A.10.19 IO_STATUS Built-In Function A–207

A.11 - J - KAREL LANGUAGE DESCRIPTION A–208
A.11.1 J_IN_RANGE Built-In Function A–208
A.11.2 JOINTPOS Data Type A–209
A.11.3 JOINT2POS Built-In Function A–209

A.12 - K - KAREL LANGUAGE DESCRIPTION A–211
A.12.1 KCL Built-In Procedure A–211
A.12.2 KCL_NO_WAIT Built-In Procedure A–212
A.12.3 KCL_STATUS Built-In Procedure A–213

A.13 - L - KAREL LANGUAGE DESCRIPTION A–213
A.13.1 LN Built-In Function A–213
A.13.2 LOAD Built-In Procedure A–214
A.13.3 LOAD_STATUS Built-In Procedure A–215

A.13.4 LOCK_GROUP Built-In Procedure A–216
A.13.5 %LOCKGROUP Translator Directive A–217
A.14 - M - KAREL LANGUAGE DESCRIPTION A–218
A.14.1 MIRROR Built-In Function A–218
A.14.2 MODIFY_QUEUE Built-In Procedure A–219
A.14.3 MOTION_CTL Built-In Function A–221
A.14.4 MOUNT_DEV Built-In Procedure A–221
A.14.5 MOVE_FILE Built-In Procedure A–222
A.14.6 MSG_CONNECT Built-In Procedure A–223
A.14.7 MSG_DISCO Built-In Procedure A–225
A.14.8 MSG_PING A–225
A.15 - N - KAREL LANGUAGE DESCRIPTION A–226
A.15.1 NOABORT Action A–226
A.15.2 %NOABORT Translator Directive A–227
A.15.3 %NOBUSYLAMP Translator Directive A–227
A.15.4 NODE_SIZE Built-In Function A–227
A.15.5 %NOLOCKGROUP Translator Directive A–229
A.15.6 NOMESSAGE Action A–230
A.15.7 NOPAUSE Action A–230
A.15.8 %NOPAUSE Translator Directive A–231
A.15.9 %NOPAUSESHFT Translator Directive A–231
A.16 - O - KAREL LANGUAGE DESCRIPTION A–232
A.16.1 OPEN FILE Statement A–232
A.16.2 OPEN HAND Statement A–233
A.16.3 OPEN_TPE Built-In Procedure A–233
A.16.4 ORD Built-In Function A–234
A.16.5 ORIENT Built-In Function A–235
A.17 - P - KAREL LANGUAGE DESCRIPTION A–236
A.17.1 PATH Data Type A–236
A.17.2 PATH_LEN Built-In Function A–238
A.17.3 PAUSE Action A–238
A.17.4 PAUSE Condition A–239
A.17.5 PAUSE Statement A–240
A.17.6 PAUSE_TASK Built-In Procedure A–241
A.17.7 PEND_SEMA Built-In Procedure A–242
A.17.8 PIPE_CONFIG Built-In Procedure A–242
A.17.9 POP_KEY_RD Built-In Procedure A–243
A.17.10 Port_Id Action A–244
A.17.11 Port_Id Condition A–245
A.17.12 POS Built-In Function A–246
A.17.13 POS2JOINT Built-In Function A–246
A.17.14 POS_REG_TYPE Built-In Procedure A–248
A.17.15 POSITION Data Type A–249
A.17.16 POST_ERR Built-In Procedure A–250
A.17.17 POST_ERR_L Built-In Procedure A–251
A.17.18 POST_SEMA Built-In Procedure A–253
A.17.19 PRINT_FILE Built-In Procedure A–253
A.17.20 %PRIORITY Translator Directive A–254
A.17.21 PROG_BACKUP Built-In Procedure A–255
A.17.22 PROG_CLEAR Built-In Procedure A–258
A.17.23 PROG_RESTORE Built-In Procedure A–260
A.17.24 PROG_LIST Built-In Procedure A–262
A.17.25 PROGRAM Statement A–263
A.17.26 PULSE Action A–264

A.17.27 PULSE Statement A–265
A.17.28 PURGE CONDITION Statement A–266
A.17.29 PURGE_DEV Built-In Procedure A–267
A.17.30 PUSH_KEY_RD Built-In Procedure A–268
A.18 - Q - KAREL LANGUAGE DESCRIPTION A–269
A.18.1 QUEUE_TYPE Data Type A–269
A.19 - R - KAREL LANGUAGE DESCRIPTION A–269
A.19.1 READ Statement A–269
A.19.2 READ_DICT Built-In Procedure A–271
A.19.3 READ_DICT_V Built-In-Procedure A–272
A.19.4 READ_KB Built-In Procedure A–273
A.19.5 REAL Data Type A–277
A.19.6 Relational Condition A–279
A.19.7 RELAX HAND Statement A–280
A.19.8 RELEASE Statement A–281
A.19.9 REMOVE_DICT Built-In Procedure A–281
A.19.10 RENAME_FILE Built-In Procedure A–282
A.19.11 RENAME_VAR Built-In Procedure A–283
A.19.12 RENAME_VARS Built-In Procedure A–283
A.19.13 REPEAT ... UNTIL Statement A–284
A.19.14 RESET Built-In Procedure A–285
A.19.15 RESUME Action A–286
A.19.16 RESUME Statement A–286
A.19.17 RETURN Statement A–287
A.19.18 ROUND Built-In Function A–288
A.19.19 ROUTINE Statement A–289
A.19.20 RUN_TASK Built-In Procedure A–290
A.20 - S - KAREL LANGUAGE DESCRIPTION A–291
A.20.1 SAVE Built-In Procedure A–291
A.20.2 SAVE_DRAM Built-In Procedure A–292
A.20.3 SELECT ... ENDSELECT Statement A–293
A.20.4 SELECT_TPE Built-In Procedure A–294
A.20.5 SEMA_COUNT Built-In Function A–294
A.20.6 SEMAPHORE Condition A–295
A.20.7 SEND_DATAPC Built-In Procedure A–295
A.20.8 SEND_EVENTPC Built-In Procedure A–296
A.20.9 SET_ATTR_PRG Built-In Procedure A–297
A.20.10 SET_CURSOR Built-In Procedure A–299
A.20.11 SET_EPOS_REG Built-In Procedure A–300
A.20.12 SET_EPOS_TPE Built-In Procedure A–301
A.20.13 SET_FILE_ATR Built-In Procedure A–302
A.20.14 SET_FILE_POS Built-In Procedure A–303
A.20.15 SET_INT_REG Built-In Procedure A–304
A.20.16 SET_JPOS_REG Built-In Procedure A–304
A.20.17 SET_JPOS_TPE Built-In Procedure A–305
A.20.18 SET_LANG Built-In Procedure A–306
A.20.19 SET_PERCH Built-In Procedure A–307
A.20.20 SET_PORT_ASG Built-In Procedure A–308
A.20.21 SET_PORT_ATR Built-In Function A–309
A.20.22 SET_PORT_CMT Built-In Procedure A–311
A.20.23 SET_PORT_MOD Built-In Procedure A–312
A.20.24 SET_PORT_SIM Built-In Procedure A–313
A.20.25 SET_PORT_VAL Built-In Procedure A–314
A.20.26 SET_POS_REG Built-In Procedure A–315

A.20.27 SET_POS_TPE Built-In Procedure A–316
A.20.28 SET_PREG_CMT Built-In-Procedure A–317
A.20.29 SET_REAL_REG Built-In Procedure A–317
A.20.30 SET_REG_CMT Built-In-Procedure A–317
A.20.31 SET_SREG_CMT Built-in Procedure A–318
A.20.32 SET_STR_REG Built-in Procedure A–319
A.20.33 SET_TIME Built-In Procedure A–319
A.20.34 SET_TPE_CMT Built-In Procedure A–321
A.20.35 SET_TRNS_TPE Built-In Procedure A–321
A.20.36 SET_TSK_ATTR Built-In Procedure A–322
A.20.37 SET_TSK_NAME Built-In Procedure A–323
A.20.38 SET_VAR Built-In Procedure A–324
A.20.39 %SHADOWVARS Translator Directive A–327
A.20.40 SHORT Data Type A–327
A.20.41 SIGNAL EVENT Action A–328
A.20.42 SIGNAL EVENT Statement A–328
A.20.43 SIGNAL SEMAPHORE Action A–329
A.20.44 SIN Built-In Function A–329
A.20.45 SQRT Built-In Function A–330
A.20.46 %STACKSIZE Translator Directive A–330
A.20.47 STD_PTH_NODE Data Type A–331
A.20.48 STOP Action A–331
A.20.49 STOP Statement A–332
A.20.50 STRING Data Type A–333
A.20.51 STR_LEN Built-In Function A–334
A.20.52 STRUCTURE Data Type A–334
A.20.53 SUB_STR Built-In Function A–335

A.21 - T - KAREL LANGUAGE DESCRIPTION A–336
A.21.1 TAN Built-In Function A–336
A.21.2 %TIMESLICE Translator Directive A–337
A.21.3 %TPMOTION Translator Directive A–337
A.21.4 TRANSLATE Built-In Procedure A–337
A.21.5 TRUNC Built-In Function A–339

A.22 - U - KAREL LANGUAGE DESCRIPTION A–340
A.22.1 UNHOLD Action A–340
A.22.2 UNHOLD Statement A–340
A.22.3 UNINIT Built-In Function A–341
A.22.4 %UNINITVARS Translator Directive A–341
A.22.5 UNLOCK_GROUP Built-In Procedure A–342
A.22.6 UNPAUSE Action A–343
A.22.7 UNPOS Built-In Procedure A–344
A.22.8 USING ... ENDUSING Statement A–344

A.23 - V - KAREL LANGUAGE DESCRIPTION A–345
A.23.1 V_CAM_CALIB *i*RVision Built-In Procedure A–345
A.23.2 V_GET_OFFSET *i*RVision Built-In Procedure A–347
A.23.3 V_GET_PASSFL *i*RVision Built-In Procedure A–348
A.23.4 V_GET_QUEUE *i*RVision Built-in Procedure A–350
A.23.5 V_INIT_QUEUE *i*RVision Built-in Procedure A–350
A.23.6 V_RALC_QUEUE *i*RVision Built-in Procedure A–351
A.23.7 V_RUN_FIND *i*RVision Built-In Procedure A–351
A.23.8 V_SET_REF *i*RVision Built-In Procedure A–353
A.23.9 V_START_VTRK *i*RVision Built-in Procedure A–354
A.23.10 V_STOP_VTRK *i*RVision Built-in Procedure A–354
A.23.11 VAR_INFO Built-In Procedure A–355

A.23.12 VAR_LIST Built-In Procedure A–357
A.23.13 VECTOR Data Type A–360
A.23.14 VOL_SPACE Built-In Procedure A–361
A.23.15 VREG_FND_POS *i*RVision Built-in Procedure A–362
A.23.16 VREG_OFFSET *i*RVision Built-in Procedure A–363
A.24 - W - KAREL LANGUAGE DESCRIPTION A–364
A.24.1 WAIT FOR Statement A–364
A.24.2 WHEN Clause A–364
A.24.3 WHILE...ENDWHILE Statement A–365
A.24.4 WITH Clause A–365
A.24.5 WRITE Statement A–366
A.24.6 WRITE_DICT Built-In Procedure A–367
A.24.7 WRITE_DICT_V Built-In Procedure A–368
A.25 - X - KAREL LANGUAGE DESCRIPTION A–369
A.25.1 XML_ADDTAG Built-In Procedure A–369
A.25.2 XML_GETDATA Built-In Procedure A–370
A.25.3 XML_REMTAG Built-In Procedure A–371
A.25.4 XML_SCAN Built-In Procedure A–371
A.25.5 XML_SETVAR Built-In Procedure A–373
A.25.6 XYZWPR Data Type A–374
A.25.7 XYZWPREXT Data Type A–375
A.26 - Y - KAREL LANGUAGE DESCRIPTION A–375
A.27 - Z - KAREL LANGUAGE DESCRIPTION A–375

Appendix B KAREL EXAMPLE PROGRAMS B–1
B.1 SETTING UP DIGITAL OUTPUT PORTS FOR PROCESS MONITORING B–6
B.2 COPYING PATH VARIABLES B–18
B.3 SAVING DATA TO THE DEFAULT DEVICE B–28
B.4 STANDARD ROUTINES B–31
B.5 USING REGISTER BUILT-INS B–33
B.6 PATH VARIABLES AND CONDITION HANDLERS PROGRAM B–38
B.7 LISTING FILES AND PROGRAMS AND MANIPULATING STRINGS B–44
B.8 GENERATING AND MOVING ALONG A HEXAGON PATH B–49
B.9 USING THE FILE AND DEVICE BUILT-INS B–54
B.10 USING DYNAMIC DISPLAY BUILT-INS B–58
B.11 MANIPULATING VALUES OF DYNAMICALLY DISPLAYED VARIABLES B–68
B.12 DISPLAYING A LIST FROM A DICTIONARY FILE B–70
B.12.1 Dictionary Files B–80
B.13 USING THE DISCTRL_ALPHA BUILT-IN B–81
B.13.1 Dictionary Files B–85
B.14 APPLYING OFFSETS TO A COPIED TEACH PENDANT PROGRAM B–85

Appendix C KCL COMMAND ALPHABETICAL DESCRIPTION C–1
C.1 ABORT command C–6
C.2 APPEND FILE command C–6
C.3 APPEND NODE command C–6
C.4 CHDIR command C–7

C.5 CLEAR ALL command C–7
C.6 CLEAR BREAK CONDITION command C–8
C.7 CLEAR BREAK PROGRAM command C–8
C.8 CLEAR DICT command C–9
C.9 CLEAR PROGRAM command C–9
C.10 CLEAR VARS command C–9
C.11 COMPRESS DICT command C–10
C.12 COMPRESS FORM command C–10
C.13 CONTINUE command C–11
C.14 COPY FILE command C–11
C.15 CREATE VARIABLE command C–12
C.16 DELETE FILE command C–13
C.17 DELETE NODE command C–13
C.18 DELETE VARIABLE command C–14
C.19 DIRECTORY command C–14
C.20 DISABLE BREAK PROGRAM command C–15
C.21 DISABLE CONDITION command C–15
C.22 DISMOUNT command C–15
C.23 EDIT command C–16
C.24 ENABLE BREAK PROGRAM C–16
C.25 ENABLE CONDITION command C–16
C.26 FORMAT command C–17
C.27 HELP command C–17
C.28 HOLD command C–18
C.29 INSERT NODE command C–18
C.30 LOAD ALL command C–18
C.31 LOAD DICT command C–19
C.32 LOAD FORM command C–19
C.33 LOAD MASTER command C–20
C.34 LOAD PROGRAM command C–20
C.35 LOAD SERVO command C–21
C.36 LOAD SYSTEM command C–21
C.37 LOAD TP command C–22
C.38 LOAD VARS command C–23
C.39 LOGOUT command C–24
C.40 MKDIR command C–24
C.41 MOUNT command C–24
C.42 MOVE FILE command C–25
C.43 PAUSE command C–25
C.44 PURGE command C–26
C.45 PRINT command C–26

C.46 RECORD command C–27
C.47 RENAME FILE command C–27
C.48 RENAME VARIABLE command C–28
C.49 RENAME VARS command C–28
C.50 RESET command C–29
C.51 RMDIR command C–29
C.52 RUN command C–30
C.53 RUNCF command C–30
C.54 SAVE MASTER command C–31
C.55 SAVE SERVO command C–31
C.56 SAVE SYSTEM command C–31
C.57 SAVE TP command C–32
C.58 SAVE VARS command C–33
C.59 SET BREAK CONDITION command C–33
C.60 SET BREAK PROGRAM command C–34
C.61 SET CLOCK command C–35
C.62 SET DEFAULT command C–35
C.63 SET GROUP command C–35
C.64 SET LANGUAGE command C–36
C.65 SET LOCAL VARIABLE command C–36
C.66 SET PORT command C–37
C.67 SET TASK command C–37
C.68 SET TRACE command C–38
C.69 SET VARIABLE command C–38
C.70 SET VERIFY command C–39
C.71 SHOW BREAK command C–39
C.72 SHOW BUILTINS command C–40
C.73 SHOW CONDITION command C–40
C.74 SHOW CLOCK command C–40
C.75 SHOW CURPOS command C–41
C.76 SHOW DEFAULT command C–41
C.77 SHOW DEVICE command C–41
C.78 SHOW DICTS command C–41
C.79 SHOW GROUP command C–41
C.80 SHOW HISTORY command C–42
C.81 SHOW LANG command C–42
C.82 SHOW LANGS command C–42
C.83 SHOW LOCAL VARIABLE command C–42
C.84 SHOW LOCAL VARS command C–44
C.85 SHOW MEMORY command C–44
C.86 SHOW PROGRAM command C–44

C.87 SHOW PROGRAMS command C–45
C.88 SHOW SYSTEM command C–45
C.89 SHOW TASK command C–45
C.90 SHOW TASKS command C–46
C.91 SHOW TRACE command C–46
C.92 SHOW TYPES command C–46
C.93 SHOW VARIABLE command C–47
C.94 SHOW VARS command C–47
C.95 SHOW data_type command C–48
C.96 SIMULATE command C–48
C.97 SKIP command C–49
C.98 STEP OFF command C–50
C.99 STEP ON command C–50
C.100 TRANSLATE command C–50
C.101 TYPE command C–51
C.102 UNSIMULATE command C–51
C.103 WAIT command C–52

Appendix D CHARACTER CODES D–1
D.1 CHARACTER CODES D–2

Appendix E SYNTAX DIAGRAMS E–1

Glossary GL–28

Index Index–1

插图目录

Figure 1–1. Controller Memory 1–9
Figure 1–2. R-30*i*A Controller 1–11
Figure 3–1. Determining **w_handle** Relative to WORLD Frame 3–10
Figure 3–2. Determining **b_handle** Relative to BUMPER Frame 3–11
Figure 7–1. "t_sc" Screen 7–39
Figure 7–2. "t_sc" Screen with $TP_USESTAT = TRUE 7–39
Figure 7–3. "c_sc" Screen 7–40
Figure 7–4. "c_sc" Screen with $CRT_USERSTAT = TRUE 7–41
Figure 8–1. Referencing Positions in KAREL 8–4
Figure 9–1. XML File 1 Separated start and end tags 9–22
Figure 9–2. XML File 2 Combined start and end tags 9–22
Figure 9–3. XML File 3 GRID tag not registered or processed 9–22
Figure 9–4. KAREL Program 9–23
Figure 10–1. Dictionary Compressor and User Dictionary File 10–12
Figure 10–2. Teach Pendant Form Screen 10–32
Figure 10–3. CRT/KB Form Screen 10–33
Figure 10–4. Dictionary Compressor and Form Dictionary File 10–34
Figure 10–5. Example of Selectable Menu Items 10–36
Figure 10–6. Example of Edit Data Items 10–38
Figure 10–7. Example of Display Only Data Items 10–42
Figure 14–1. KAREL Logic for Converting Input to a Real Value Representing the Voltage 14–4
Figure 14–2. RSR Timing Diagram 14–16
Figure 14–3. PNS Timing Diagram 14–17
Figure 14–4. Location of Ports on the Controller 14–23
Figure 15–1. Task Synchronization Using a Semaphore 15–10
Figure A–1. Example of COMPARE_FILE built-in Procedure A–74
Figure A–2. FRAME Built-In Function A–143
Figure A–3. GET_USEC_SUB Built-In Function A–171
Figure E–1. ..... E–3
Figure E–2. ..... E–4
Figure E–3. ..... E–5
Figure E–4. ..... E–6
Figure E–5. ..... E–7

Figure E–6. ..... E–8
Figure E–7. ..... E–9
Figure E–8. ..... E–10
Figure E–9. ..... E–11
Figure E–10. ..... E–12
Figure E–11. ..... E–13
Figure E–12. ..... E–14
Figure E–13. ..... E–15
Figure E–14. ..... E–16
Figure E–15. ..... E–17
Figure E–16. ..... E–18
Figure E–17. ..... E–19
Figure E–18. ..... E–20
Figure E–19. ..... E–21
Figure E–20. ..... E–22
Figure E–21. ..... E–23
Figure E–22. ..... E–24
Figure E–23. ..... E–25
Figure E–24. ..... E–26
Figure E–25. ..... E–27

表目录

Table 2–1. ASCII Character Set 2–2
Table 2–2. Multinational Character Set 2–4
Table 2–3. Graphics Character Set 2–5
Table 2–4. KAREL Operators 2–5
Table 2–5. KAREL Operator Precedence 2–6
Table 2–6. Reserved Word List 2–6
Table 2–7. Predefined Identifier and Value Summary 2–8
Table 2–8. Port and File Predefined Identifier Summary 2–9
Table 2–9. Translator Directives 2–11
Table 2–10. Simple and Structured Data Types 2–14
Table 3–1. Summary of Operation Result Types 3–3
Table 3–2. KAREL Operators 3–5
Table 3–3. Arithmetic Operations Using +, -, and * Operators 3–5
Table 3–4. Arithmetic Operations Examples 3–5
Table 3–5. Arithmetic Operations Using Bitwise Operands 3–6
Table 3–6. KAREL Operator Precedence 3–6
Table 3–7. Relational Operation Examples 3–7
Table 3–8. BOOLEAN Operation Summary 3–8
Table 3–9. BOOLEAN Operations Using AND, OR, and NOT Operators 3–8
Table 3–10. Examples of Vector Operations 3–13
Table 5–1. Stack Usage 5–13
Table 5–2. KAREL Built—In Routine Summary 5–16
Table 6–1. Conditions 6–2
Table 6–2. Actions 6–2
Table 6–3. Condition Handler Operations 6–3
Table 6–4. Interval Between Global Condition Handler Scans 6–4
Table 6–5. Port_Id Conditions 6–7
Table 6–6. Relational Conditions 6–8
Table 6–7. System and Program Event Conditions 6–9
Table 6–8. Assignment Actions 6–12
Table 6–9. Motion Related Actions 6–13
Table 6–10. Miscellaneous Actions 6–14
Table 7–1. Predefined File Variables 7–3

Table 7–2. Predefined Attribute Types 7–5
Table 7–3. Attribute Values 7–6
Table 7–4. Usage Specifiers 7–11
Table 7–5. Text (ASCII) Input Format Specifiers 7–18
Table 7–6. Text (ASCII) Output Format Specifiers 7–19
Table 7–7. Examples of INTEGER Input Data Items 7–20
Table 7–8. Examples of INTEGER Output Data Items 7–21
Table 7–9. Examples of REAL Input Data Items 7–23
Table 7–10. Examples of REAL Output Data Items 7–24
Table 7–11. Examples of BOOLEAN Input Data Items 7–26
Table 7–12. Examples of BOOLEAN Output Data Items 7–27
Table 7–13. Examples of STRING Input Data Items 7–28
Table 7–14. Examples of STRING Output Data Items 7–30
Table 7–15. Examples of VECTOR Output Data Items 7–32
Table 7–16. Examples of POSITION Output Data Items (p = POS(2.0,-4.0,8.0,0.0,90.0,0.0,config_var)) 7–33
Table 7–17. Binary Input/Output Format Specifiers 7–35
Table 7–18. Defined Windows for t_sc" 7–38
Table 7–19. Defined Windows for c_sc" 7–40
Table 8–1. Turn Number Definitions 8–2
Table 9–1. File Type Descriptions 9–5
Table 9–2. Virtual Devices 9–14
Table 9–3. System Variable Field Descriptions 9–18
Table 9–4. File Listings for the MD Device 9–27
Table 9–5. Testing Restrictions when Using the MD: Device 9–30
Table 10–1. Conversion Characters 10–7
Table 10–2. Reserved Words 10–9
Table 10–3. Conversion Characters 10–20
Table 10–4. Tree View Format 10–26
Table 10–5. Reserved Words 10–28
Table 10–6. Reserved Words for Scrolling Window 10–29
Table 12–1. Access Rights for System Variables 12–2
Table 14–1. Standard Operator Panel Input Signals 14–7
Table 14–2. Standard Operator Panel Output Signals 14–7
Table 14–3. User Operator Panel Input Signals 14–9
Table 14–4. User Operator Panel Output Signals 14–14
Table 14–5. Teach Pendant Input Signal Assignments 14–17
Table 14–6. Ports P1 - P4 14–24
Table 14–7. Default Communications Settings for Devices 14–24
Table 15–1. System Function Priority Table 15–5
Table A–1. Syntax Notation A–9

Table A–2. Actions A–11
Table A–3. Clauses A–11
Table A–4. Conditions A–12
Table A–5. Data Types A–12
Table A–6. Directives A–13
Table A–7. KAREL Built—In Routine Summary A–13
Table A–8. Items A–16
Table A–9. Statements A–17
Table A–10. Valid and Invalid BOOLEAN Values A–42
Table A–11. INTEGER Representation of Current Time A–165
Table A–12. Conversion Characters A–185
Table A–13. Conversion Characters A–193
Table A–14. Valid and Invalid INTEGER Literals A–204
Table A–15. IO_STATUS Errors A–207
Table A–16. Group_mask Setting A–216
Table A–17. Group_mask Setting A–221
Table A–18. Valid and Invalid REAL operators A–278
Table A–19. Group_mask setting A–291
Table A–20. Attribute Values A–310
Table A–21. 32–Bit INTEGER Format of Time A–320
Table A–22. Example STRING Literals A–333
Table A–23. Group_mask Settings A–342
Table A–24. Function Code Values A–346
Table A–25. Valid Data Types A–356
Table A–26. Valid Data Types A–358
Table B–1. KAREL Example Programs B–2
Table D–1. ASCII Character Codes D–2
Table D–2. Special ASCII Character Codes D–3
Table D–3. Multinational Character Codes D–4
Table D–4. Graphics Character Codes (not available in R-30iB) D–6
Table D–5. Teach Pendant Input Codes D–7
Table D–6. European Character Codes D–8
Table D–7. Graphics Characters D–10

安全

FANUC Robotics不是也不代表您自己是安全系统，安全设备或贵公司和/或其员工的特定安全方面的专家。所有者，雇主或用户有责任采取一切必要措施来保证工作场所所有人员的安全。安全系统专业人员可以最好地确定适合您的应用和安装的安全级别。因此，FANUC Robotics建议每位客户咨询此类专业人员，以便提供允许FANUC机器人系统安全应用，使用和操作的工作场所。

根据行业标准ANSI / RIA R15-06，建议业主或用户查阅标准，以确保符合其对机器人系统设计，可用性，操作，维护和服务的要求。此外，作为机器人系统的所有者，雇主或用户，您有责任安排机器人系统操作员的培训，以识别和响应与您的机器人系统相关的已知危险并了解建议的特定应用程序和机器人安装的操作过程。

确保使用的机器人适合应用程序。在分类（危险）场所使用的机器人必须经过认证才能使用。因此，FANUC Robotics建议所有打算操作，编程，维修或以其他方式使用机器人系统的人员必须接受经过批准的FANUC机器人培训课程的培训，并熟悉系统的正常运行。负责系统编程的人员 - 包括应用程序的设计，实现和调试 - 必须熟悉应用程序和机器人安装的推荐编程过程。提供以下指南以强调工作场所安全的重要性。

考虑到机器人安装的安全性

无论何时使用机器人，安全都至关重要。 请记住以下有关安全的因素：

- 人员和设备的安全
- 使用安全增强装置
- 机器人安全教学和手动操作的技巧
- 机器人安全自动操作的技术
- 定期检查机器人和工作单元
- 正确维护机器人

保证人的安全

在任何情况下，人的安全始终是最重要的。 在对机器人系统应用安全措施时，请考虑以下事项：

- External devices
- Robot(s)
- Tooling
- Workpiece

使用安全增强设备

始终注意机器人周围的工作区域。 通过安装以下部分或全部设备可以提高工作区域的安全性：

- Safety fences, barriers, or chains
- Light curtains
- Interlocks
- Pressure mats
- Floor markings
- Warning lights
- Mechanical stops
- EMERGENCY STOP buttons
- DEADMAN switches

设置安全的工作站

安全的工作站对于保护人员和设备至关重要。 请遵守以下准则以确保安全地设置工作站。 这些建议旨在补充而不是取代现有的联邦，州和地方有关安全的法律，法规和指南。

- 赞助您的人员接受与您的申请相关的经批准的FANUC机器人培训课程的培训。 绝不允许未经培训的人员操作机器人。
- 安装使用访问代码的锁定设备，以防止未经授权的人员操作机器人。
- 使用反束缚逻辑可防止操作员绕过安全措施。
- 安排工作站，使操作员面向工作站，并可以看到站内部发生了什么。

- 清楚地识别系统中每个机器人的工作范围，包括楼层标记，标志和特殊障碍。工作范围是由机器人的最大运动范围定义的区域，包括连接到腕部法兰的任何延伸此范围的工具。
- 将所有控制器置于机器人工作范围之外。
- 绝不依赖基于软件或固件的控制器作为主要安全元件，除非它们符合适用的当前机器人安全标准。
- 安装足够数量的急停按钮或开关，方便操作员和工作单元内外的关键点。
- 安装闪光灯和/或声音警告装置，在机器人运行时激活，即伺服驱动系统通电时。声音警告装置应超过最终用途应用中的环境噪音水平。
- 尽可能安装安全围栏，以防止人员未经授权进入工作区域。
- 安装特殊防护装置，防止操作员进入工作范围的受限区域。
- 使用联动装置。
- 使用存在或接近感应设备，如光幕，垫子，电容和视觉系统，以提高安全性。
- 定期检查可选择安装在机器人手腕法兰和工具之间的安全关节或安全离合器。如果工具撞击物体，这些设备会移开，从系统中移除电源，并有助于最大限度地减少对工具和机器人的损坏。
- 确保所有外部设备均已正确过滤，接地，屏蔽和抑制，以防止由于电磁干扰（EMI），射频干扰（RFI）和静电放电（ESD）的影响而产生危险运动。
- 在控制器上设置电源锁定/挂牌。
- 消除夹点。夹点是人员可能被困在移动机器人和其他设备之间的区域。
- 在工作单元内提供足够的空间，以允许人员教授机器人并安全地进行维护。
- 编程机器人以安全地装载和卸载材料。
- 如果存在高压静电，请务必提供适当的联锁，警告和信标。
- 如果在危险的高压下使用材料，请提供电气联锁装置，以锁定物料流和压力。

在示教或手动操作机器人时保持安全

建议所有必须示教机器人或以其他方式手动操作机器人的人员遵守以下规则：

- 切勿佩戴可能被移动机械夹住的手表，戒指，领带，围巾或宽松的衣服。
- 如果您在危险环境中工作，请了解您是否使用本安型示教器。
- 在教学之前，目视检查机器人和工作范围，确保不存在潜在的危险情况。工作范围是由机器人的最大运动范围定义的区域。这些包括连接到腕部法兰的工具，扩展了该范围。
- 机器人附近区域必须清洁，无油，水或碎屑。立即向主管或安全部门报告不安全的工作条件。
- FANUC Robotics建议除了机器人教学操作外，没有人进入机器人的工作范围。但是，如果必须输入工作范围，请确保所有安全措施都已到位，检查示教器DEADMAN开关是否正常运行，并将机器人置于示教模式。随身携带示教器，打开它，并准备释放DEADMAN开关。只有带示教器的人才应该在工作范围内。

> **警告**
>
> **切勿绕过，绑扎或以其他方式停用安全装置，例如限位开关，以确保操作方便。 已知停用安全装置会导致严重的伤害和死亡。**

- 了解可用于逃离移动机器人的路径; 确保永远不会阻止转义路径。
- 将机器人与所有可在播放数据时引起运动的遥控信号隔离。
- 以下列方式测试第一次运行的任何程序：

> **警告**
>
> **无论何时运行程序，都要留在机器人工作区域外。 否则可能导致受伤。**

— 使用低运动速度，将程序单步执行至少一个完整循环。

— 使用低运动速度，测试程序连续运行至少一个完整周期。

— 使用编程的速度，测试程序连续运行至少一个完整循环。

- 在运行生产之前，确保所有人员都在工作范围之外。

在自动操作期间保持安全

建议在生产过程中操作机器人的所有人员遵守以下规则：

- 确保所有安全措施都存在且有效。
- 了解整个工作单元区域。工作单元包括机器人及其工作范围，以及机器人与之交互的所有外部设备和其他设备所占用的区域。
- 了解机器人在启动自动操作之前执行的完整任务。
- 在操作机器人之前，确保所有人员都在工作范围之外。
- 在机器人自动操作期间，切勿进入或允许其他人进入工作范围。
- 了解可能导致机器人移动的所有开关，传感器和控制信号的位置和状态。
- 了解机器人控制和外部控制设备上的紧急停止按钮的位置。准备好在紧急情况下按下这些按钮。
- 如果机器人不移动，切勿假设程序已完成。机器人可能正在等待允许其继续活动的输入信号。
- 如果机器人以某种模式运行，请不要认为它将继续以相同的模式运行。
- 切勿尝试使用您的身体停止机器人或使其运动。立即停止机器人运动的唯一方法是按下位于控制器面板上的紧急停止按钮，示教器或工作单元周围的紧急停止站。

检查期间保持安全

检查机器人时，请务必

- 关闭控制器的电源。
- 根据工厂的政策锁定并标记控制器上的电源。
- 关闭压缩空气源并释放空气压力。
- 如果检查电路不需要机器人运动，请按操作面板上的EMERGENCY STOP按钮。
- 切勿佩戴可能被移动机械夹住的手表，戒指，领带，围巾或宽松的衣服。
- 如果需要通电来检查机器人运动或电路，请准备好在紧急情况下按下急停按钮。
- 请注意，当您卸下伺服电机或制动器时，如果不支撑或停在硬停止位置，相关的机器人手臂将会掉落。 在松开制动器之前，将支撑臂支撑在坚固的支撑上。

Staying Safe During Maintenance

When performing maintenance on your robot system, observe the following rules:

- Never enter the work envelope while the robot or a program is in operation.
- Before entering the work envelope, visually inspect the workcell to make sure no potentially hazardous conditions exist.
- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- Consider all or any overlapping work envelopes of adjoining robots when standing in a work envelope.
- Test the teach pendant for proper operation before entering the work envelope.
- If it is necessary for you to enter the robot work envelope while power is turned on, you must be sure that you are in control of the robot. Be sure to take the teach pendant with you, press the DEADMAN switch, and turn the teach pendant on. Be prepared to release the DEADMAN switch to turn off servo power to the robot immediately.
- Whenever possible, perform maintenance with the power turned off. Before you open the controller front panel or enter the work envelope, turn off and lock out the 3-phase power source at the controller.
- Be aware that an applicator bell cup can continue to spin at a very high speed even if the robot is idle. Use protective gloves or disable bearing air and turbine air before servicing these items.
- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.



Warning

Lethal voltage is present in the controller WHENEVER IT IS CONNECTED to a power source. Be extremely careful to avoid electrical shock. HIGH VOLTAGE IS PRESENT at the input side whenever the controller is connected to a power source. Turning the disconnect or circuit breaker to the OFF position removes power from the output side of the device only.

- Release or block all stored energy. Before working on the pneumatic system, shut off the system air supply and purge the air lines.
- Isolate the robot from all remote control signals. If maintenance must be done when the power is on, make sure the person inside the work envelope has sole control of the robot. The teach pendant must be held by this person.

- Make sure personnel cannot get trapped between the moving robot and other equipment. Know the path that can be used to escape from a moving robot. Make sure the escape route is never blocked.
- Use blocks, mechanical stops, and pins to prevent hazardous movement by the robot. Make sure that such devices do not create pinch points that could trap personnel.

> **Warning**
>
> **Do not try to remove any mechanical component from the robot before thoroughly reading and understanding the procedures in the appropriate manual. Doing so can result in serious personal injury and component destruction.**

- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.
- When replacing or installing components, make sure dirt and debris do not enter the system.
- Use only specified parts for replacement. To avoid fires and damage to parts in the controller, never use nonspecified fuses.
- Before restarting a robot, make sure no one is inside the work envelope; be sure that the robot and all external devices are operating normally.

KEEPING MACHINE TOOLS AND EXTERNAL DEVICES SAFE

Certain programming and mechanical measures are useful in keeping the machine tools and other external devices safe. Some of these measures are outlined below. Make sure you know all associated measures for safe use of such devices.

Programming Safety Precautions

Implement the following programming safety measures to prevent damage to machine tools and other external devices.

- Back-check limit switches in the workcell to make sure they do not fail.
- Implement “failure routines” in programs that will provide appropriate robot actions if an external device or another robot in the workcell fails.
- Use *handshaking* protocol to synchronize robot and external device operations.
- Program the robot to check the condition of all external devices during an operating cycle.

Mechanical Safety Precautions

Implement the following mechanical safety measures to prevent damage to machine tools and other external devices.

- Make sure the workcell is clean and free of oil, water, and debris.
- Use DCS (Dual Check Safety), software limits, limit switches, and mechanical hardstops to prevent undesired movement of the robot into the work area of machine tools and external devices.

KEEPING THE ROBOT SAFE

Observe the following operating and programming guidelines to prevent damage to the robot.

Operating Safety Precautions

The following measures are designed to prevent damage to the robot during operation.

- Use a low override speed to increase your control over the robot when jogging the robot.
- Visualize the movement the robot will make before you press the jog keys on the teach pendant.
- Make sure the work envelope is clean and free of oil, water, or debris.
- Use circuit breakers to guard against electrical overload.

Programming Safety Precautions

The following safety measures are designed to prevent damage to the robot during programming:

- Establish *interference zones* to prevent collisions when two or more robots share a work area.
- Make sure that the program ends with the robot near or at the home position.
- Be aware of signals or other operations that could trigger operation of tooling resulting in personal injury or equipment damage.
- In dispensing applications, be aware of all safety guidelines with respect to the dispensing materials.

Note Any deviation from the methods and safety practices described in this manual must conform to the approved standards of your company. If you have questions, see your supervisor.

ADDITIONAL SAFETY CONSIDERATIONS FOR PAINT ROBOT INSTALLATIONS

Process technicians are sometimes required to enter the paint booth, for example, during daily or routine calibration or while teaching new paths to a robot. Maintenance personal also must work inside the paint booth periodically.

Whenever personnel are working inside the paint booth, ventilation equipment must be used. Instruction on the proper use of ventilating equipment usually is provided by the paint shop supervisor.

Although paint booth hazards have been minimized, potential dangers still exist. Therefore, today's highly automated paint booth requires that process and maintenance personnel have full awareness of the system and its capabilities. They must understand the interaction that occurs between the vehicle moving along the conveyor and the robot(s), hood/deck and door opening devices, and high-voltage electrostatic tools.



Caution

Ensure that all ground cables remain connected. Never operate the paint robot with ground provisions disconnected. Otherwise, you could injure personnel or damage equipment.

Paint robots are operated in three modes:

- Teach or manual mode
- Automatic mode, including automatic and exercise operation
- Diagnostic mode

During both teach and automatic modes, the robots in the paint booth will follow a predetermined pattern of movements. In teach mode, the process technician teaches (programs) paint paths using the teach pendant.

In automatic mode, robot operation is initiated at the System Operator Console (SOC) or Manual Control Panel (MCP), if available, and can be monitored from outside the paint booth. All personnel must remain outside of the booth or in a designated safe area within the booth whenever automatic mode is initiated at the SOC or MCP.

In automatic mode, the robots will execute the path movements they were taught during teach mode, but generally at production speeds.

When process and maintenance personnel run diagnostic routines that require them to remain in the paint booth, they must stay in a designated safe area.

Paint System Safety Features

Process technicians and maintenance personnel must become totally familiar with the equipment and its capabilities. To minimize the risk of injury when working near robots and related equipment, personnel must comply strictly with the procedures in the manuals.

This section provides information about the safety features that are included in the paint system and also explains the way the robot interacts with other equipment in the system.

The paint system includes the following safety features:

- Most paint booths have red warning beacons that illuminate when the robots are armed and ready to paint. Your booth might have other kinds of indicators. Learn what these are.
- Some paint booths have a blue beacon that, when illuminated, indicates that the electrostatic devices are enabled. Your booth might have other kinds of indicators. Learn what these are.
- EMERGENCY STOP buttons are located on the robot controller and teach pendant. Become familiar with the locations of all E-STOP buttons.
- An intrinsically safe teach pendant is used when teaching in hazardous paint atmospheres.
- A DEADMAN switch is located on each teach pendant. When this switch is held in, and the teach pendant is on, power is applied to the robot servo system. If the engaged DEADMAN switch is released during robot operation, power is removed from the servo system, all axis brakes are applied, and the robot comes to an EMERGENCY STOP. Safety interlocks within the system might also E-STOP other robots.



Warning

An EMERGENCY STOP will occur if the DEADMAN switch is released on a bypassed robot.

- Overtravel by robot axes is prevented by software limits. All of the major and minor axes are governed by software limits. DCS (Dual Check Safety), limit switches and hardstops also limit travel by the major axes.
- EMERGENCY STOP limit switches and photoelectric eyes might be part of your system. Limit switches, located on the entrance/exit doors of each booth, will EMERGENCY STOP all equipment in the booth if a door is opened while the system is operating in automatic or manual mode. For some systems, signals to these switches are inactive when the switch on the SOC is

in teach mode.When present, photoelectric eyes are sometimes used to monitor unauthorized intrusion through the entrance/exit silhouette openings.

- System status is monitored by computer. Severe conditions result in automatic system shutdown.

Staying Safe While Operating the Paint Robot

When you work in or near the paint booth, observe the following rules, in addition to all rules for safe operation that apply to all robot systems.



Warning

Observe all safety rules and guidelines to avoid injury.



Warning

Never bypass, strap, or otherwise deactivate a safety device, such as a limit switch, for any operational convenience. Deactivating a safety device is known to have resulted in serious injury and death.



Warning

Enclosures shall not be opened unless the area is known to be nonhazardous or all power has been removed from devices within the enclosure. Power shall not be restored after the enclosure has been opened until all combustible dusts have been removed from the interior of the enclosure and the enclosure purged. Refer to the Purge chapter for the required purge time.

- Know the work area of the entire paint station (workcell).
- Know the work envelope of the robot and hood/deck and door opening devices.
- Be aware of overlapping work envelopes of adjacent robots.
- Know where all red, mushroom-shaped EMERGENCY STOP buttons are located.
- Know the location and status of all switches, sensors, and/or control signals that might cause the robot, conveyor, and opening devices to move.
- Make sure that the work area near the robot is clean and free of water, oil, and debris. Report unsafe conditions to your supervisor.
- Become familiar with the complete task the robot will perform BEFORE starting automatic mode.

- Make sure all personnel are outside the paint booth before you turn on power to the robot servo system.
- Never enter the work envelope or paint booth before you turn off power to the robot servo system.
- Never enter the work envelope during automatic operation unless a safe area has been designated.
- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- Remove all metallic objects, such as rings, watches, and belts, before entering a booth when the electrostatic devices are enabled.
- Stay out of areas where you might get trapped between a moving robot, conveyor, or opening device and another object.
- Be aware of signals and/or operations that could result in the triggering of guns or bells.
- Be aware of all safety precautions when dispensing of paint is required.
- Follow the procedures described in this manual.

Special Precautions for Combustible Dusts (powder paint)

When the robot is used in a location where combustible dusts are found, such as the application of powder paint, the following special precautions are required to insure that there are no combustible dusts inside the robot.

- Purge maintenance air should be maintained at all times, even when the robot power is off. This will insure that dust can not enter the robot.
- A purge cycle will not remove accumulated dusts. Therefore, if the robot is exposed to dust when maintenance air is not present, it will be necessary to remove the covers and clean out any accumulated dust. Do not energize the robot until you have performed the following steps.

1. Before covers are removed, the exterior of the robot should be cleaned to remove accumulated dust.
2. When cleaning and removing accumulated dust, either on the outside or inside of the robot, be sure to use methods appropriate for the type of dust that exists. Usually lint free rags dampened with water are acceptable. Do not use a vacuum cleaner to remove dust as it can generate static electricity and cause an explosion unless special precautions are taken.
3. Thoroughly clean the interior of the robot with a lint free rag to remove any accumulated dust.
4. When the dust has been removed, the covers must be replaced immediately.
5. Immediately after the covers are replaced, run a complete purge cycle. The robot can now be energized.

Staying Safe While Operating Paint Application Equipment

When you work with paint application equipment, observe the following rules, in addition to all rules for safe operation that apply to all robot systems.



Warning

When working with electrostatic paint equipment, follow all national and local codes as well as all safety guidelines within your organization. Also reference the following standards: *NFPA 33 Standards for Spray Application Using Flammable or Combustible Materials* , and *NFPA 70 National Electrical Code* .

- **Grounding**: All electrically conductive objects in the spray area must be grounded. This includes the spray booth, robots, conveyors, workstations, part carriers, hooks, paint pressure pots, as well as solvent containers. Grounding is defined as the object or objects shall be electrically connected to ground with a resistance of not more than 1 megohms.
- **High Voltage**: High voltage should only be on during actual spray operations. Voltage should be off when the painting process is completed. Never leave high voltage on during a cap cleaning process.
- Avoid any accumulation of combustible vapors or coating matter.
- Follow all manufacturer recommended cleaning procedures.
- Make sure all interlocks are operational.
- No smoking.
- Post all warning signs regarding the electrostatic equipment and operation of electrostatic equipment according to NFPA 33 Standard for Spray Application Using Flammable or Combustible Material.
- Disable all air and paint pressure to bell.
- Verify that the lines are not under pressure.

Staying Safe During Maintenance

When you perform maintenance on the painter system, observe the following rules, and all other maintenance safety rules that apply to all robot installations. Only qualified, trained service or maintenance personnel should perform repair work on a robot.

- Paint robots operate in a potentially explosive environment. Use caution when working with electric tools.
- When a maintenance technician is repairing or adjusting a robot, the work area is under the control of that technician. All personnel not participating in the maintenance must stay out of the area.

- For some maintenance procedures, station a second person at the control panel within reach of the EMERGENCY STOP button. This person must understand the robot and associated potential hazards.
- Be sure all covers and inspection plates are in good repair and in place.
- Always return the robot to the ‘‘home’’ position before you disarm it.
- Never use machine power to aid in removing any component from the robot.
- During robot operations, be aware of the robot’s movements. Excess vibration, unusual sounds, and so forth, can alert you to potential problems.
- Whenever possible, turn off the main electrical disconnect before you clean the robot.
- When using vinyl resin observe the following:
  - — Wear eye protection and protective gloves during application and removal
  - — Adequate ventilation is required. Overexposure could cause drowsiness or skin and eye irritation.
  - — If there is contact with the skin, wash with water.
  - — Follow the Original Equipment Manufacturer’s Material Safety Data Sheets.
- When using paint remover observe the following:
  - — Eye protection, protective rubber gloves, boots, and apron are required during booth cleaning.
  - — Adequate ventilation is required. Overexposure could cause drowsiness.
  - — If there is contact with the skin or eyes, rinse with water for at least 15 minutes. Then, seek medical attention as soon as possible.
  - — Follow the Original Equipment Manufacturer’s Material Safety Data Sheets.

第1章

KAREL 语言概述

目录

第1章 KAREL 语言概述 1–1
1.1 目录 1–2
1.2 KAREL 编程语言 1–2
1.2.1 概述 1–2
1.2.2 创建程序 1–4
1.2.3 编译程序 1–4
1.2.4 加载程序逻辑和数据 1–4
1.2.5 执行程序 1–5
1.2.6 执行历史 1–5
1.2.7 程序结构 1–5
1.3 系统软件 1–7
1.3.1 软件组件 1–7
1.3.2 支持的机器人 1–7
1.4 控制器 1–8
1.4.1 内存 1–8
1.4.2 输入/输出系统 1–10
1.4.3 用户界面设备 1–10

1.1 概述

FANUC 机器人系统的 KAREL 系统由机器人，控制器和系统软件组成。它使用用 KAREL 编程语言编写的程序来完成工业任务。 KAREL 可以操作数据，控制和与相关设备进行通信并与操作员进行交互。
配备 KAREL 的系统 R-30iA 控制器可与各种机器人模型配合使用，以处理各种应用。这意味着常见的操作，编程和故障排除程序以及更少的备件。 KAREL 系统扩展到包括全系列支持产品，如整体视觉，离线编程和特定于应用的软件包。
KAREL 编程语言实际上融合了像 Pascal和PL / 1 这样的高级语言的逻辑，类似英语的功能，以及机器控制语言经验证的工厂车间效率。 KAREL 结合了高级语言通用的结构和惯例以及专门为机器人应用开发的功能。这些 KAREL 功能包括

- **简单和结构化的数据类型**
- **算术，关系和布尔运算符**
- **控制循环和选择的结构**
- **条件处理程序**
- **程序和功能例程**
- **输入和输出操作**
- **多编程支持**

本章总结了 KAREL 编程语言，并描述了 KAREL 系统软件和控制器。

以下说明适用于 R-30iB控制器：

注意为了加载 KAREL 程序，必须在机器人控制器上安装 KAREL 选件。

1.2 KAREL 编程语言

1.2.1 概述

KAREL程序由存储在源代码文件中的声明和可执行语句组成。 与程序关联的变量数据值存储在变量文件中。

使用 **OLPC PRO** 或其他编辑器（如Word Pad）创建和编辑 KAREL程序。

KAREL 语言编译器将源代码转换为称为p代码的内部格式并生成p代码文件。编译器提供 **OLPC PRO**。编译完成后，可以使用 KAREL 命令语言（KCL）或FILE菜单将产生的p代码程序加载到控制器上。

在加载期间，系统将创建任何不在RAM中的所需变量并将其设置为未初始化。当你运行程序时，KAREL 解释器执行加载的p代码指令。

KAREL 程序由程序逻辑和程序数据组成。程序逻辑定义了执行特定任务所需执行的一系列步骤。程序数据是程序逻辑使用的任务相关信息。在KAREL中，程序逻辑与程序数据是分开的。

程序逻辑由 KAREL 程序中BEGIN和END语句之间的 KAREL 可执行语句定义。程序数据包括通过RAM中的名称，数据类型和存储区域在 KAREL 程序的VAR声明部分中标识的变量。

程序数据的值可以通过示教器点动机器人，通过程序计算，从数据文件读取，从CRT / KB内设置或者教导式菜单结构，或者在执行过程中作为输入接受。数据值可以从一次执行改变到下一次，但是使用相同的程序逻辑来操纵数据。

由于以下原因，程序逻辑和程序数据在 KAREL 程序中是分开的：

- **允许在同一个程序中的多个地方引用数据**
- **允许多个程序引用或共享相同的数据**
- **允许程序使用替代数据**
- **促进离线计算机辅助设计（CAD）系统建立数据文件**

程序的可执行部分包含数据操作语句，I / O语句和例程调用。

下面的列表简要介绍程序开发周期. 第1.2.2节 - 以下第1.2.6节提供了每个阶段的详细信息。

- **创建一个程序源代码文件**
- **编译程序文件。**
- **加载程序逻辑和数据。**
- **执行程序。**
- **维护程序的执行历史。**

已执行的程序的日志或历史记录由控制器维护并可查看。

1.2.2 创建一个程序

您可以使用离线编辑器（如**OLPC PRO**）或任何文本编辑器（如写字板）创建 KAREL 程序。生成的文件称为源文件或源代码。

1.2.3 编译程序

在执行之前，KAREL源文件必须编译成内部代码，称为p代码。 KAREL语言编译器执行此功能并检查源代码中的错误。

KAREL语言编译器从源代码的第一行开始并持续到遇到错误或成功编译程序。如果遇到错误，转换程序会尝试继续检查程序，但不会生成p代码。

您可以从 **OLPC PRO** 调用编译器，并且您正在编辑的源代码将被编译。成功编译后，编译器将显示成功的编译消息并创建一个p代码文件。 p代码文件将使用源代码文件名和.pc文件类型。该文件包含源代码的内部表示以及系统将程序链接到变量数据和例程所需的信息。

如果编译器检测到任何错误，它将显示正在编译的错误消息和源代码行。纠正错误后，您可以再次编译程序。

1.2.4 加载程序逻辑和数据

以下说明适用于R-30iB控制器：

注意为了加载KAREL程序，必须在机器人控制器上安装KAREL选件。

一个程序的p代码被加载到一个可以执行的控制器上。加载程序时，将在RAM中创建一个包含程序中所有静态变量的变量数据表。变量数据表包含程序标识符，所有变量标识符以及变量所在RAM中的存储区域的名称。

加载程序还会建立语句和变量之间的链接。最初，变量数据表中的值将被初始化。如果成功加载变量文件（.vr），则任何变量的值都将存储在可变数据存储区

（CMOS，DRAM，SHADOW）中。

通常使用多个程序来将大型应用程序或问题分解为可以单独开发和测试的较小块。 KAREL 系统允许加载多个程序。 每个加载的程序都有自己的p代码结构。
可变数据可以在多个程序之间共享。 在这种情况下，必须在VAR声明中指定 KAREL 语言的FROM子句，以便系统可以在程序加载时执行链接。 这节省了包含数据的多个副本所需的存储空间。
以下限制适用于可以加载的KAREL程序的数量和大小：

- **程序数量限制为2704或可用RAM。**
- **每个程序的变量数量限制为2704或可用RAM。**

1.2.5 执行程序

从程序列表中选择一个程序并将p代码和变量文件加载到RAM后，测试和调试程序以确保其按预期运行。
程序执行从第一个可执行文件行开始。 除非您指定堆栈大小，否则将分配300个字的堆栈。
堆栈从可用的用户RAM分配。 堆栈使用情况在5.1.6节中描述。

1.2.6 执行历史

每次执行程序时，可以使用SHOW HISTORY命令从KCL显示已执行的嵌套例程和行号的日志。
这在程序暂停或意外中止时非常有用。 执行历史显示导致中断的事件序列。

1.2.7 程序结构

KAREL程序由声明和由KAREL语言声明组成的可执行部分组成，如KAREL程序的结构所示。

一个 KAREL 程序的结构

```
PROGRAM  prog_name
     Translator Directives
     CONST, TYPE, and/or VAR Declarations
     ROUTINE Declarations
  BEGIN
```

```
   Executable Statements
END  prog_name
   ROUTINE Declarations
```

在KAREL计划的结构中，用大写字母表示的词语是具有专用意义的KAREL保留字。 PROGRAM，CONST，TYPE，VAR和ROUTINE表示程序的声明部分。 BEGIN和END标记可执行部分。 保留字在2.1.3节中描述。

确定程序的PROGRAM语句必须是任何KAREL程序中的第一条语句。 PROGRAM语句由保留字PROGRAM和您选择的标识符（KAREL程序结构中的prog_name）组成。标识符在第2.1.4节中描述。

注意您的程序必须位于文件中。该文件可以但不一定与程序具有相同的名称。这种区别很重要，因为您调用翻译器并使用包含程序的文件的名称加载程序，但是您会启动程序的执行并使用程序名称清除程序。

例如，如果名为移动器的程序包含在名为transfer的文件中，则可以通过传输引用该文件进行翻译，但会使用程序名移动器来执行该程序。如果程序和文件都被命名为移动器，则可以使用移动器翻译文件并执行该程序。

创建任务来执行程序，任务名称是您启动的程序的名称。该程序可以在另一个程序中调用例程，但任务名称不会更改。

用于命名程序的标识符不能在程序中用于任何其他目的，例如标识变量或常量。

CONST（常量），TYPE（类型）和VAR（变量）声明节在PROGRAM语句之后。 一个程序可以包含任意数量的CONST，TYPE和VAR部分。 每个部分还可以包含任意数量的单个声明语句。 此外，多个CONST，TYPE和VAR部分可以按任意顺序出现。 CONST，TYPE和VAR段以及声明语句的数量仅受可用内存量的限制。

例程声明可以遵循CONST，TYPE和VAR部分。 每个例程都以保留字ROUTINE开头，在语法上与程序类似。 在END语句后，ROUTINE声明也可以跟在主程序的可执行部分之后。

可执行部分必须在开始和结束时用BEGIN标记，最后用程序标识符（在KAREL程序的结构中为prog_name）标记。 END语句中必须使用与PROGRAM语句中相同的程序标识符。 可执行部分可以包含任意数量的可执行语句，仅受可用内存量的限制。

另请参阅：第2章语言要素，第3章运算符的用法和第5章例程。

1.3 系统软件

R-30iA系统包括一个机器人和控制器电子器件。 硬件接口和系统软件支持编程，日常操作，维护和故障排除。

本节概述了支持的系统软件和机器人模型。

有关机器人和控制器型号的维护手册中详细介绍了硬件主题。

1.3.1 软件组件

R-30iA系统软件是由FANUC Robotics提供的软件，由控制器CPU执行，允许您操作R-30iA系统。 您使用系统软件来运行程序，以及执行日常操作，维护和故障排除。

系统软件的组件包括：

- **Motion Control** - 将工具中心点（TCP）从初始位置移动到所需的目标位置
- **File System** - 将数据存储在RAM磁盘或外围存储设备上
- **System Variables** - 作为KAREL系统软件一部分声明的永久定义的变量
- **CRT/KB or Teach Pendant Screens** - 便于操作KAREL系统的屏幕
- **KCL** - KAREL 命令语言
- **KAREL Interpreter** - 执行 KAREL 程序

另请参阅：特定应用FANUC机器人设置和操作手册，了解使用 CRT / KB 和示教器屏幕的详细操作步骤。

1.3.2 支持的机器人

机器人使用适当的工具执行由系统软件和控制器执行的应用任务。 R-30iA 系统支持各种机器人，每种机器人都为特定类型的应用而设计。

有关支持的机器人型号的最新列表，请咨询您的 FANUC Robotics 技术代表。

另请参阅：您的特定机器人类型的维护手册，以获取有关机器人的更多信息。

1.4 控制器

R-30iA 控制器包含操作R-30iA 系统所需的电子电路和存储器。 由系统软件支持的电子电路指导机器人的操作和动作，并允许与外围设备进行通信。

控制器电子设备包括中央处理单元（CPU），多种类型的存储器，输入/输出（I / O）系统和用户界面设备。 一个机柜内装有控制器电子元件和远程用户界面设备和其他外围设备连接的端口。

1.4.1 内存

有三种控制器内存：

- **动态随机存取存储器（DRAM）**
- **有限的电池备份静态/随机访问存储器（SRAM）**
- **闪存可编程只读存储器（FROM）**

另外，控制器能够在外部存储信息。

DRAM

DRAM内存不稳定。存储器内容在断电时不保留其存储的值。 DRAM存储器也被称为临时存储器（TEMP）。系统软件在DRAM内存中执行。 KAREL程序和大多数KAREL变量都被加载到DRAM中并从此处执行。

注：即使DRAM变量处于易失性存储器中，也可以在启动时控制其值。任何时候加载程序.VR或.PC文件时，该程序在DRAM中的值都将设置为.VR文件中的值。这意味着在每次启动时都不需要重新加载VR文件来设置初始值。如果该变量的值在正常操作期间发生变化，它将恢复为上次加载.VR或.PC文件时所设置的值。

如果您希望启动时未初始化DRAM变量，则可以使用IN UNINIT_DRAM子句对要确保在启动时未初始化的任何变量。您可以使用%UNINITDRAM指令来指定程序中的所有变量在启动时将被初始化。

如果在同一个KAREL程序中有SHADOW变量和DRAM变量，则可能会在不加载.PC / .VR文件的情况下更改DRAM变量的加电设置。在这种情况下，程序员必须特别注意KAREL软件在启动时对特定DRAM变量设置的依赖。具体而言，DRAM启动值将始终保留它们在受控启动结束时的值。如果SHADOW内存已满，则可以在正常系统操作期间设置DRAM启动值。

SRAM

SRAM存储器是非易失性的。 存储器内容在断电时保留其存储的值。 SRAM存储器也被称为CMOS或永久存储器（PERM）。

TPP内存池（用于示教程序）从PERM分配。 卡尔程序可以指定要存储在CMOS中的变量。 一部分SRAM存储器可以被定义为称为RAM盘（RD :)的用户存储设备。

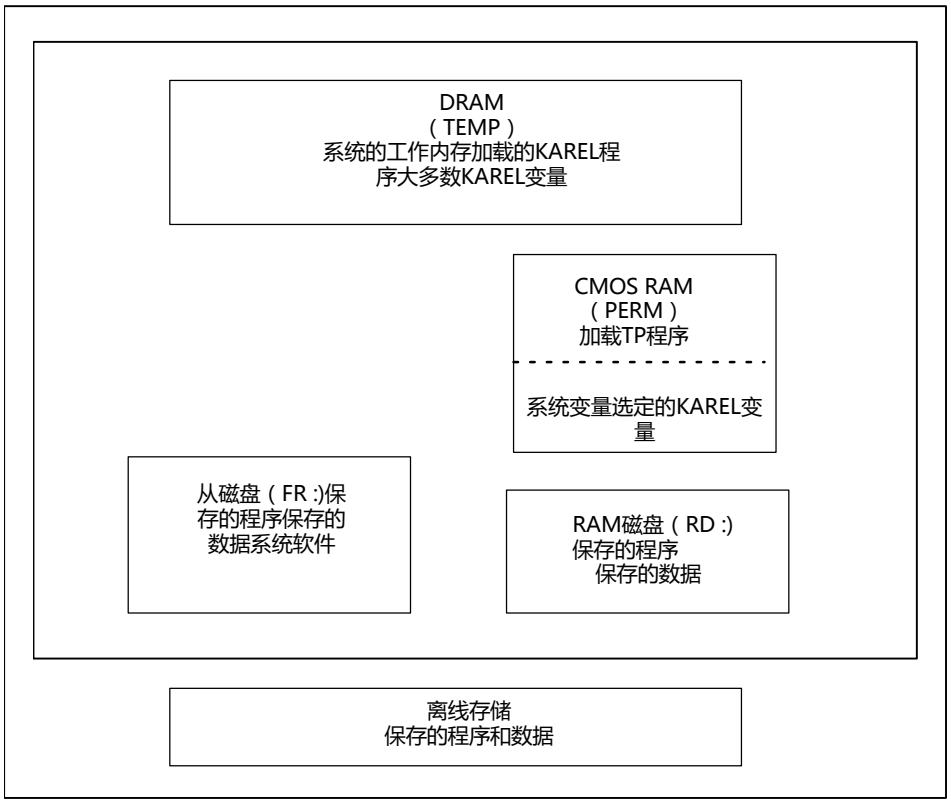
闪存（FROM）

FROM内存是非易失性的。 存储器内容在断电时保留其存储的值。 FROM用于永久存储系统软件。 FROM也可作为FROM设备（FR :)用于用户存储。

SHADOW

SHADOW存储器提供与SRAM相同的功能。 在SHADOW中设置的任何值都是非易失性的，并且会在电源循环中保持其状态。 SHADOW记忆旨在用于倾向于静态的数据。 将动态变量存储在SHADOW内存中，例如FOR循环索引或其他快速变化的数据，效率不高。

图 1–1. 控制器内存



外置储存

您可以在外部设备上备份和存储文件。 您可以使用以下设备：

- **存储卡**
- **通过FTP的以太网**
- **USB存储棒**

1.4.2 输入/输出系统

该控制器可以支持模块化I / O结构，允许您按照您的应用程序的要求添加I / O板。 数字和模拟输入和输出模块均受支持。 另外，您还可以为其他I / O添加可选的过程I / O板。 您拥有的I / O信号的类型和数量取决于您的应用程序的要求。

另见：第14章输入/输出系统，了解更多信息。

1.4.3 用户界面设备

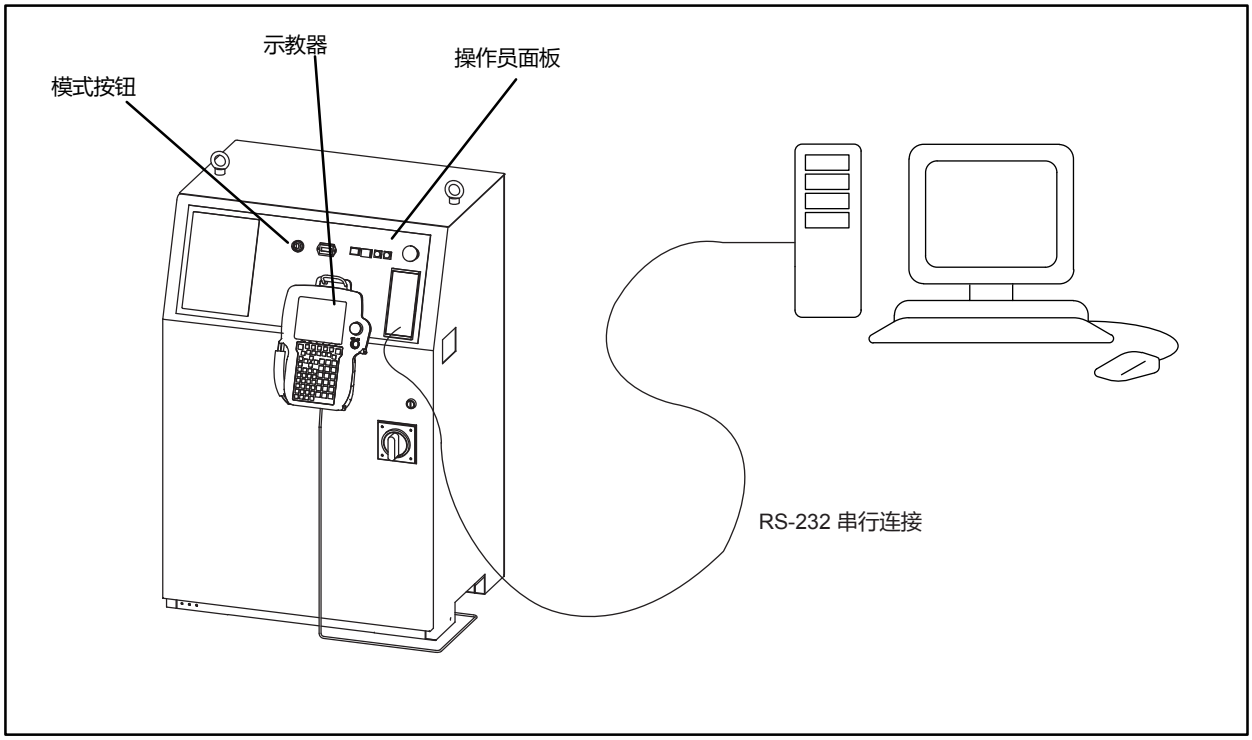
用户界面设备使您能够编程和操作KAREL系统。 KAREL支持的通用用户界面设备包括操作面板，示教器或CRT / KB。

图1-2 显示了这些用户界面设备。 所有型号的操作面板和示教器都具有相同的基本功能; 但是，也可以使用不同的配置。

位于控制器机柜正面的操作面板提供用于执行日常操作（如开机，运行程序和关机）的按钮。操作面板上的指示灯表示操作条件，例如电源打开和机器人处于循环状态。

该系统还支持用户操作面板（UOP）的I / O信号，这是用户提供的设备，如自定义控制面板，可编程控制器或主机。 请参阅第14章输入/输出系统。

图 1–2. R-30*i*A 控制器



CRT / KB是控制器上的一个软件选项，允许运行TelNet的PC等外部终端显示菜单系统，该菜单系统与教师相同。

示教器由LCD显示屏，菜单驱动的功能键，小键盘按键和状态LED组成。 它通过电缆连接到控制器机柜，允许您远离控制器执行操作。

示教器内部连接到控制器的主CPU板。 它用于慢跑机器人，教程序数据，测试和调试程序以及调整变量。 它还可用于控制I / O，控制臂端工具，并显示机器人当前位置或应用程序状态等信息。

应用程序特定的FANUC机器人设置和操作手册提供了每个用户界面设备的说明以及操作每个设备的步骤。

第2章

语言元素

目录

第2章 语言元素 2–1
2.1 语言组件 2–2
2.1.1 字符集 2–2
2.1.2 Operators（运算符函数） 2–5
2.1.3 保留字 2–6
2.1.4 用户定义的标识符 2–7
2.1.5 标签 2–8
2.1.6 预定义标识符 2–8
2.1.7 系统变量 2–10
2.1.8 注释 2–10
2.2 翻译指令 2–11
2.3 数据类型 2–13
2.4 用户定义的数据类型和结构 2–14
2.4.1 用户定义的数据类型 2–14
2.4.2 用户定义的数据结构 2–16
2.5 阵列 2–18
2.5.1 多维数组 2–19
2.5.2 可变大小的数组 2–20

KAREL语言为编写有效的机器人应用程序提供了必要的元素。 本章列出并描述了KAREL语言的每个组件，可用的翻译器指令和可用的数据类型。

2.1 语言组件

本节介绍KAREL语言的以下基本组件：

- 字符集
- Operators
- 保留字
- 用户定义的标识符
- 标签
- 预定义标识符
- 系统变量
- 注释

2.1.1 字符集

ASCII 字符集以KAREL语言提供。 表2-1列出了ASCII字符集中的元素。 KAREL语言提供三种字符集：

- ASCII 字符集
- 多国文字字符集
- 图形字符集（不适用于R-30iB）

表2-1，表2-2 和 表2-3 列出了所有由KAREL语言识别的字符。 默认字符集是ASCII。 多国和图形字符集只允许在文字，数据和评论中使用。

另见：CHR内置程序，附录A.

表 2–1. ASCII字符集

字母	a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
数字	0 1 2 3 4 5 6 7 8 9

表 2–1. ASCII字符集（续）

符号	@ < > = / * + - _ , ; : . # $ ' [ ] ( ) & % { }
特殊字符	空白或空间 换页（作为新行处理）选项卡 （视为空白处理）

以下规则适用于ASCII字符集：

- **空白或空格是：**

  - 需要分开保留字和标识符。 例如，语句 **PROGRAM**

  PROG_NAME 必须在 **PROGRAM** 和 **prog_name** 之间包含一个空格。

  - 允许但符号运算符和它们之间的表达式不需要

  操作数。 例如，语句 a = bis 相当于 a = b。

  - 用于缩进程序中的行。

- 回车或分号（;）分开陈述。 回车也可以出现在其他地方。
- 在BEGIN声明之后需要回车或分号。
- 一行在252个字符后被截断。 可以使用连接字符＆继续下一行。

表 2–2. 多国文字字符集

符号	¡	¢	£	¥	§	¤
	©	a	«	O	±	2
	3	µ	¶	•	1	o
	»	¼	½	¿		
特殊字符	À	Á	Â	Ã	Ä	Å
	Æ	Ç	È	É	Ê	Ë
	Ì	Í	Î	Ï	Ñ	Ò
	Ó	Ô	Õ	Ö	Œ	Ø
	Ù	Ú	Û	Ü	Y	ß
	à	á	â	ã	ä	å
	æ	ç	è	é	ê	ë
	ì	í	î	ï	ç	ñ
	ò	ó	ô	õ	ö	œ
	ø	ù	ú	û	ü	ÿ

表 2–3. 图形字符集

字母	A B C D E F G H I J K L MN O P Q R S T U V W X Y Z													
数字	0 1 2 3 4 5 6 7 8 9													
符号	@ < > = / * + - , ; : . # $ ’ \ [ ] ( ) & % ! " ^													
特殊符号	♦		H T	F F	C R	L F	O	±	N L	V T	┘	┐	┌	└
	+	-	-	-	-	_	├	┤	┴	┬	|	≤	≥	⊓
	≠	£	.											

另请参阅：附录D 中每个字符集的字符代码列表。

2.1.2 Operators

KAREL为标准算术运算，关系运算和布尔（逻辑）运算提供了Operators。 KAREL还包括特殊的Operators，可以将位置和VECTOR数据类型用作操作数。

表2-4列出了所有可用于KAREL的AA。

表 2–4. KAREL Operators

运算	+	-	*	/	DIV	MOD
大小关系	<	< =	=	< >	> =	>
布尔	AND	OR	NOT			
特殊符号	> = <	:	#	@		

这些Operators的优先规则如下：

- 首先评估括号内的表达式。
- 在给定水平的括号内，操作从最高优先级开始执行，并进行到最低优先级的操作。
- 在括号和Operators优先级的相同级别内，操作从左到右执行。

表2-5 列出了KAREL operators 的优先级别。

表 2–5. KAREL Operator 优先级

运算符	优先级
NOT	High
:, @, #	↓
*, /, AND, DIV, MOD	↓
Unary + and -, OR, +, -	↓
<, >, =, < >, < =, > =, > = <	Low

另见：第3章Operator的用法，用于描述Operator执行的功能

2.1.3 保留字

保留字在KAREL中具有专用含义。它们只能用于所描述的上下文中。 表2-6列出了所有KAREL保留字。

表 2–6. 保留字列表

ABORT	CONST	GET_VAR	NOPAUSE	STOP
ABOUT	CONTINUE	GO	NOT	STRING
ABS	COORDINATED	GOTO	NOWAIT	STRUCTURE
AFTER	CR	GROUP	OF	THEN
ALONG	DELAY	GROUP_ASSOC	OPEN	TIME
ALSO	DISABLE	HAND	OR	TIMER
AND	DISCONNECT	HOLD	PATH	TO
ARRAY	DIV	IF	PATHHEADER	TPENABLE
ARRAY_LEN	DO	IN	PAUSE	TYPE
AT	DOWNTO	INDEPENDENT	POSITION	UNHOLD
ATTACH	DRAM	INTEGER	POWERUP	UNINIT
AWAY	ELSE	JOINTPOS	PROGRAM	UNPAUSE
AXIS	ENABLE	JOINTPOS1	PULSE	UNTIL
BEFORE	END	JOINTPOS2	PURGE	USING
BEGIN	ENDCONDITION	JOINTPOS3	READ	VAR
BOOLEAN	ENDFOR	JOINTPOS4	REAL	VECTOR
BY	ENDIF	JOINTPOS5	RELATIVE	VIA

表 2–6. 保留字列表（续）

BYNAME	ENDMOVE	JOINTPOS6	RELAX	VIS_PROCESS
BYTE	ENDSELECT	JOINTPOS7	RELEASE	WAIT
CAM_SETUP	ENDSTRUCTURE	JOINTPOS8	REPEAT	WHEN
CANCEL	ENDUSING	JOINTPOS9	RESTORE	WHILE
CASE	ENDWHILE	MOD	RESUME	WITH
CLOSE	ERROR	MODEL	RETURN	WRITE
CMOS	EVAL	MOVE	ROUTINE	XYZWPR
COMMAND	EVENT	NEAR	SELECT	XYZWPREXT
COMMON_ASSOC	END	NOABORT	SEMAPHORE	
CONDITION	FILE	NODE	SET_VAR	
CONFIG	FOR	NODEDATA	SHORT	
CONNECT	FROM	NOMESSAGE	SIGNAL	

另请参见：参考 KAREL保留字 描述的索引

2.1.4 用户定义的标识符

用户定义的标识符表示常量，数据类型，语句标签，变量，例程名称和程序名称。 身份标识

- 从字母开始
- 可以包括字母，数字和下划线
- 最多可以有12个字符
- 在特定范围内只能有一个含义。 参考5.1.4节。
- 不能成为保留字
- 必须在可以使用之前进行定义。

例如，声明标识符中的程序节选显示了如何声明程序变量和变量标识符。

声明标识符

```
PROGRAM mover  --program identifier (mover)
    VAR
      original     :  POSITION  --variable identifier (original)
    CONST
```

```
no_of_parts = 10  --constant identifier (no_of_parts)
```

2.1.5 标签

标签是特殊标识符，用于标记可以使用GOTO语句传送程序控制的程序中的位置。

- 紧接着是两个冒号（ ：:)。 可执行语句允许在两个冒号后面的同一行和后续行中。
- 不能用于将控制转入或转出例程。

在使用标签中，焊接::表示零件焊接程序的部分。 当陈述去焊接时，程序控制转移到焊接部分。

使用标签

```
weld::  --label
   .     --additional program statements
   .
   .
GOTO weld
```

2.1.6 预定义标识符

KAREL语言中的预定义标识符具有预定义的含义。 这些可以是常量，类型，变量或内置例程名称。 表2-7和表2-8列出了预定义的标识符及其相应的值。 标识符或值可以在程序语句中指定。 例如，$MOTYPE = 7 与 $MOTYPE = LINEAR 相同。 但是，预定义标识符MININT是此规则的例外。 该标识符必须始终用于代替其值-2147483648。 值或数字本身不能使用。

表 2–7. 预定义标识符和值摘要

预定义标识符	类型	值
TRUE FALSE	BOOLEAN	ON OFF
ON OFF	BOOLEAN	ON OFF
MAXINT MININT	INTEGER	+2147483647 -2147483648

表 2–7. 预定义的标识符和值摘要（续）

预定义标识符	类型	值
RSWORLD AESWORLD WRISTJOINT	定位类型： $ORIENT_TYPE	1 2 3
JOINT LINEAR (or STRAIGHT) CIRCULAR	运动类型： $MOTYPE	6 7 8
FINE COARSE NOSETTLE NODECEL VARDECEL	终止类型： $TERMTYPE 和 $SEGTERMTYPE	1 2 3 4 5

表 2–8. 端口和文件预定义标识符摘要

预定义标识符	类型
DIN（数字输入） DOUT（数字输出）	布尔端口数组
GIN（组输入） GOUT（组输出） AIN（模拟输入） AOUT（模拟输出）	整数端口数组
TPIN（示教器输入） TPOUT（示教器输出） RDI（机器人数字输入） RDO（机器人数字输出） OPIN（操作面板输入） OPOUT（操作面板输出） WDI（焊接输入） WDOUT（焊接输出） UIN（用户操作面板输入） UOUT（用户操作面板输出） LDI（激光数字输入） LDO（激光数字输出） FLG（Flag旗标） MRK（标记）	布尔端口数组

表 2-8。 端口和文件预定义标识符摘要（续）

预定义标识符	类型
LAI（激光模拟输入） LAO（激光模拟输出）	整数端口数组
TPDISPLAY（示教器上KAREL显示）* TPERROR（示教器上信息） TPPROMPT（示教器上功能键行）* TPFUNC（示教器上功能键行）* TPSTATUS（示教器上状态行）* INPUT（CRT / KB KAREL键盘）* OUTPUT（CRT / KB KAREL屏幕）* CRTERROR（CRT / KB信息行） CRTFUNC（CRT功能键行）* CRTSTATUS（CRT状态行）* CRTPROMPT（CRT提示行）* VIS_MONITOR（视觉监视器屏幕）	文件

*输入和输出发生在示教器或CRT / KB的USER菜单上。

2.1.7 系统变量

系统变量是作为KAREL系统软件的一部分声明的变量。 他们已经永久定义了以美元符号（$）开头的变量名称。 许多机器人都是特定的，这意味着它们的值取决于连接到系统的机器人的类型。

KAREL程序无法访问某些系统变量。 访问权限控制KAREL程序是否可以读取或写入系统变量。

另请参阅：FANUC Robotics软件参考手册，了解所有可用系统变量的完整列表和说明。

2.1.8 注释

注释是用于使程序更易于您或其他程序员理解的程序中的文本行。 例如，程序中的注释包含来自KAREL程序中的%INCLUDE指令和KAREL程序的包含文件mover_decs中的一些注释。

程序中的注释

```
--This program, called mover, picks up 10 objects
```

```
--from an original POSITION and puts them down
--at a destination POSITION.
original :  POSITION   --POSITION of objects
destination :  POSITION   --Destination of objects
count  :  INTEGER    --Number of objects moved
```

评论用一对连续的连字符（ - - ）标记。 在程序行中，这些连字符右侧的任何内容均被视为评论。
注释可以自行插入，也可以在包含任何程序语句的行末尾插入。 他们被翻译忽略，对正在运行的程序完全没有影响。

2.2 翻译指令

译者指令提供了指导KAREL程序翻译的机制。 译者指令是在KAREL程序中使用的特殊语句

- 在翻译时将其他文件包含到程序中
- 指定程序和任务属性

除 **%INCLUDE** 之外的所有指令都必须在程序语句之后，但在任何其他语句之前。 **表2-9**列出并简要描述了每个译码器指令。 请参阅**附录A**，了解每个译员指令的完整说明。

表 2–9. 译者指令

指令	描述
%ALPHABETIZE	指定在加载p代码时将按字母顺序创建变量。
%CMOSVARS	指定KAREL变量的默认存储区是CMOS RAM。
%CMOS2SHADOW	指示翻译器将所有CMOS变量放入SHADOW内存中。
%COMMENT = 'comment'	指定最多16个字符的注释。 在加载时间内，注释作为程序属性存储，并可显示在示教器或CRT / KB的SELECT屏幕上。
%CRTDEVICE	指定CRT和KB用户窗口将成为READ和WRITE语句中的默认值，而不是TPDISPLAY窗口。
%DEFGROUP = n	指定由翻译者员使用的默认运动组。
%DELAY	指定程序每250毫秒延迟的时间量。

表 2–9. 译者指令（续）

指令	描述
%ENVIRONMENT filename	off-line转换器用于指定应加载特定的环境文件。
%INCLUDE filename	指定要在翻译时插入到程序中的文件。
%LOCKGROUP =n,n	指定由此任务锁定的运动组。
%NOABORT = option	指定一组将禁止中止程序的条件。
%NOBUSYLAMP	指定执行期间busy指示灯将熄灭。
%NOLOCKGROUP	指定没有运动组将被此任务锁定。
%NOPAUSE = option	指定一组将禁止暂停程序的条件。
%NOPAUSESHFT	如果释放示教器切换键，则指定任务未暂停。
%PRIORITY = n	指定任务优先级。
%SHADOWVARS	指定默认情况下所有变量均在 SHADOW 中创建。
%STACKSIZE = n	以长字指定堆栈大小。
%TIMESLICE = n	支持具有相同优先级的任务的循环类型时间分片。
%TPMOTION	指定仅在示教器启用时启用任务动作。
%UNINITVARS	指定所有变量在默认情况下都是未初始化的。

KAREL程序中的%INCLUDE指令说明了%INCLUDE指令。 包含用于**KAREL程序的mover_decs文件**显示包含的文件。

KAREL 程序中的 %INCLUDE 指令

```
PROGRAM mover
-- This program, called mover, picks up 10 objects
-- from an original position and puts them down
-- at a destination position.
%INCLUDE mover_decs
-- Uses %INCLUDE directive to include the file
-- called mover_decs containing declarations
BEGIN
   OPEN HAND gripper
-- Loop to move total number of objects
   FOR count = 1 TO num_of_parts DO
   -- Put position in Position Register 1
      SET_POS_REG(1,original,status)
   -- Call TP program to move to Position Register
      move_to_pr
      CLOSE HAND gripper
```

```
      SET_POS_REG(1,destination,status)
      move_to_pr
      OPEN HAND gripper
   ENDFOR    -- End of loop
END mover
The TP program move_to_pr is a one line program
to do the move:
  1:J PR[1] 100% FINE
```

包含用于KAREL程序的文件mover_decs

```
-- Declarations for program mover in file mover_decs
VAR
   original  :  XYZWPR     --POSITION of objects
   destination :  XYZWPR     --Destination of objects
   count  :  INTEGER      --Number of objects moved
CONST
   gripper = 1    -- Hand number 1
   num_of_parts = 10    -- Number of objects to move
```

2.3 数据类型

KAREL 提供了三种数据类型来定义程序中的数据项：

- 简单类型的数据项
  - 可以在KAREL程序中分配常量或变量
  - 可以在KAREL程序中分配实际（文字）值
  - 只能假设单个值
- 结构化类型数据项
  - 被定义为允许或需要多个值的数据项 - 简单数据和结构化数据的组合
- 用户定义的类型数据项
  - 根据现有数据类型（包括其他用户定义的类型）进行定义 - 可以定义为由多个KAREL变量数据类型组成的结构 - 不能包含自身

表2-10列出了 KAREL 中可用的简单和结构化数据类型。 用户定义的数据类型在2.4节中描述。

Table 2–10. Simple and Structured Data Types

Simple	Structured	
BOOLEAN	ARRAY OF BYTE	JOINTPOS8
FILE	CAM_SETUP	JOINTPOS9
INTEGER	CONFIG	MODEL
REAL	JOINTPOS	PATH
STRING	JOINTPOS1	POSITION
	JOINTPOS2	QUEUE_TYPE
	JOINTPOS3	ARRAY OF SHORT
	JOINTPOS4	VECTOR
	JOINTPOS5	VIS_PROCESS
	JOINTPOS6	XYZWPR
	JOINTPOS7	XYZWPREXT

另请参阅：附录A中每种数据类型的详细说明

2.4 用户定义的数据类型和结构

用户定义的数据类型是您根据现有数据类型定义的数据类型。 用户定义的数据结构是数据结构，您可以在其中将新数据类型定义为由多个KAREL变量数据类型（包括之前定义的用户数据类型）组成的结构。

2.4.1 用户定义的数据类型

用户定义的数据类型是您根据现有数据类型定义的数据类型。 用户定义的数据类型，可以

- 将他们的声明包含在KAREL程序的TYPE部分中。
- 根据其他数据类型定义KAREL名称以表示新的数据类型。
- 可以使用特定应用所需的预定义数据类型。

用户定义的数据类型可以定义为结构，由几个KAREL变量数据类型组成。

接续字符“&”可用于在新行上继续声明。

用户定义的数据类型示例显示了用户定义的数据类型用法和连续字符用法的示例。

用户定义的数据类型示例

```
CONST
   n_pages = 20
   n_lines = 40
   std_str_lng = 8
TYPE
   std_string_t = STRING [std_str_lng]
   std_table_t = ARRAY [n_pages]&         --continuation character
    OF ARRAY [n_lines] OF std_string_t
   path_hdr_t FROM main_prog = STRUCTURE   --user defined data type
     ph_uframe:  POSITION
     ph_utool:  POSITION
   ENDSTRUCTURE
   node_data_t FROM main_prog = STRUCTURE
     gun_on:  BOOLEAN
     air_flow:  INTEGER
   ENDSTRUCTURE
std_path_t FROM main_prog =
  PATH PATHDATA = path_hdr_t NODEDATA = node_data_t
VAR
   msg_table_1:  std_table_t
   msg_table_2:  std_table_t
   temp_string:  std_string_t
   seam_1_path:  std_path_t
```

用法

用户定义的类型数据可以是

- 分配给相同类型的其他变量
- 作为参数传递
- 作为函数返回

不同用户定义的数据类型的变量之间的分配，即使是相同的声明，也是不允许的。 此外，系统还可以加载和保存用户定义数据类型的变量，并在加载期间用当前数据类型的声明检查一致性。

限制

用户定义的数据类型不能

- 包括自己
- 包括任何直接或间接包含它的类型
- 在例程中声明

2.4.2 用户定义的数据结构

结构用于存储通常一起使用的信息集合。 用户定义的数据结构是数据结构，您可以在其中将新数据类型定义为由多个 KAREL 变量数据类型组成的结构。

当加载包含用户定义数据类型变量的程序时，将根据先前创建的定义检查这些类型的定义。如果以前创建的定义不存在，则会创建一个新定义。

用户定义的数据结构，可以

- 将数据类型定义为由组件字段列表组成的结构，每个组件字段可以是标准数据类型或另一个以前定义的用户数据类型。 请参阅将数据类型定义为用户定义的结构。

将数据类型定义为用户定义的结构

```
new_type_name = STRUCTURE
  field_name_1:  type_name_1
  field_name_2:  type_name_2
  ..
ENDSTRUCTURE
```

- 访问在KAREL程序中定义为结构的数据类型的元素。 继续性符号“&”可以用来继续访问结构元素。 请参阅访问KAREL程序中用户定义结构的元素。

在KAREL程序中访问用户定义结构的元素

```
var_name = new_type_name.field_nam_1
new_type_name.field_name_2 = expression
outer_struct_name.inner_struct_name&
  .field_name = expression
```

- 访问从CRT / KB和示教器定义为结构的数据类型的元素。
- 定义一系列可执行语句，其中可以访问STRUCTURE类型变量的字段而不必重复变量的名称。 请参阅定义一系列可执行语句。

定义一系列可执行语句

```
USING struct_var, struct_var2 DO
  statements
  ..
ENDUSING
```

在上面的例子中，struct_var 和 struct_var2 是结构类型变量的名称。

注意如果相同的名称是双字段名称和可变名称，则假定字段名称。 如果同一个字段名称出现在多个变量中，则使用 USING 语句中最右侧的变量。

限制

用户定义的数据结构有以下限制：

- 以下数据类型作为数据结构的一部分无效：
  - 结构定义; 允许声明结构的类型。 请参阅有效的结构定义。

有效的结构定义

```
The following is valid:
  TYPE
   sub_struct = STRUCTURE
      subs_field_1:  INTEGER
      subs_field_2:  BOOLEAN
   ENDSTRUCTURE
   big_struct = STRUCTURE
      bigs_field_1:  INTEGER
      bigs_field_2:  sub_struct
   ENDSTRUCTURE
The following is not valid:
    big_struct = STRUCTURE
      bigs_field_1:  INTEGER
      bigs_field_2:  STRUCTURE
      subs_field_1:  INTEGER
      subs_field_2:  BOOLEAN
    ENDSTRUCTURE
ENDSTRUCTURE
```

— PATH 类型

- 文件类型
- 视觉类型
- 可变长度数组
- 数据结构本身或包含它的任何类型，直接或间接 - 任何先前未定义的结构。

- 可以不定义结构的结构，但可以定义为以前定义为结构的类型。 请参阅将变量定义为以前定义为结构的类型。

将变量定义为以前定义为结构的类型

```
The following is valid:
  TYPE
    struct_t = STRUCTURE
      st_1:  BOOLEAN
      st_2:  REAL
    ENDSTRUCTURE
  VAR
    var_name:  struct_t
The following is not valid:
  VAR
    var_name:  STRUCTURE
      vn_1:  BOOLEAN
      vn_2:  REAL
    ENDSTRUCTURE
```

2.5 阵列

您可以声明除 PATH 以外的任何数据类型的数组。

您可以通过 KAREL 程序从 CRT / KB 和示教器访问这些阵列的元素。

另外，你可以定义两种类型的数组：

- **多维数组**
- **可变大小的数组**

2.5.1 多维数组

多维数组是具有二维或三维元素的数组。 这些数组允许您使用两个或三个下标标识一个元素。
多维数组允许您这样做

- 将变量声明为具有两个或三个（但不是更多）维度的数组。 **请参阅将变量声明为二维或三维数组。**

将变量声明为二维或三维数组

```
VAR
  name:  ARRAY [size_1] OF ARRAY [size_2] .., OF element_type
  OR
VAR
  name:  ARRAY [size_1, size_2,...] OF element_type
```

- 在KAREL语句中访问这些数组的元素。 **请参阅在KAREL语句中访问多维数组的元素。**

在KAREL声明中访问多维数组的元素

```
name [subscript_1, subscript_2,...] = value
value = name [subscript_1, subscript_2,...]
```

- 将例程参数声明为多维数组。 **请参阅将例程参数声明为多维数组。**

将例程参数声明为多维数组

```
Routine expects 2-dimensional array of INTEGER.
   ROUTINE array_user (array_param:ARRAY [*,*] OF INTEGER)
The following are equivalent:
  ROUTINE rtn_name(array_param:  ARRAY[*] OF INTEGER)
  and
  ROUTINE rtn_name(array_param:  ARRAY OF INTEGER)
```

- 使用KCL命令和示教器访问元素。
- 在变量文件中保存和加载多维数组。

限制

以下限制适用于多维数组：

- 一个子数组可以作为参数传递，或者只有当它被定义为一个单独的类型时，才会通过省略一个或多个最右下角来分配给另一个数组。 请参阅使用子数组。

使用子数组

```
TYPE
   array_30 = ARRAY[30] OF INTEGER
   array_20_30 = ARRAY[20] OF array_30
VAR
   array_1:  array_30
   array_2:  array_20_30
   array_3:  ARRAY[10] OF array_20_30
ROUTINE array_user(array_data:  ARRAY OF INTEGER
   FROM other-prog
BEGIN
array_2 = array_3[10]          -- 分配元素 array_3[10,1,1]
                                  -- 通过 array_3[10,20,30] 来
array_2
array_2[2] = array_1           -- 通过分配元素 array_1[1]
                                  -- array_1 [30] 到元素 array_2[2,1]
                                  -- 通过 array_2[2,30]
array_user(array_3[5,3])       -- 传递元素 array_3[5,3,1]
                                  -- 通过 array_3[5,3,30] 到 array_user
```

- 元素类型不能是以下任何一种：

  - 数组（但它可以是一个用户定义的数组类型）

  - 路径

2.5.2 可变大小的数组

可变大小的数组是指其实际大小未知的数组，并且从程序到另一个程序的使用有所不同。 可变大小的数组允许您在不建立数组变量的维度的情况下编写KAREL程序。 在所有情况下，变量的维度必须在加载 **.PC** 文件之前建立。

可变大小的数组允许您这样做

- 声明一个数组大小为"**To-Be-Determined**"（*）。 请参阅表明数组的大小是“**To-Be-Determined**”。

指示数组的大小是“To-Be-Determined'”

```
VAR
   one_d_array:  ARRAY[*] OF type
   two_d_array:  ARRAY[*,*] OF type
```

- 根据变量文件或KCL CREATE VAR命令而不是从KAREL源代码中确定数组大小。

一个变量大小的数组的实际大小将取决于数组的实际大小（如果它已经存在），变量文件中的数组大小（如果它首先被加载）或者执行的KCL CREATE VAR命令中指定的大小 在程序加载之前。 程序中明确指定的尺寸必须与**.VR**文件中指定的尺寸或KCL CREATE VAR命令中指定的尺寸一致。

限制

可变大小的数组有以下限制：

- 变量必须在内存中加载或创建（在**.VR**文件中或使用 KCL），并且具有已知长度，才能使用。
- 当加载 **.PC** 文件时，它使用已建立的维度，否则使用0。
- 变量大小的数组只允许在 VAR 部分而不是程序的 TYPE 部分。
- 变量大小的数组只允许用于静态变量。

第3章

使用运算符号

目录

第3章 使用运算符号 3–1
3.1 表达式和赋值 3–2
3.1.1 表达式和赋值规则 3–2
3.1.2 评估表达式和赋值 3–2
3.1.3 变量和表达式 3–4
3.2 运算 3–4
3.2.1 算术运算 3–5
3.2.2 关系运算 3–6
3.2.3 布尔运算 3–7
3.2.4 特殊运算 3–8

本章介绍操作符如何与其他语言元素一起使用以在KAREL应用程序中执行操作。 首先解释表达式和赋值，它们是包含运算符和操作数的程序语句。 接下来，讨论可以使用每个可用的KAREL算子执行的操作种类。

3.1 表达式和赋值

表达式是由一系列操作数定义的值，由操作符连接并引起所需的计算。 例如，4 + 8是其中4和8是操作数并且该符号（ + ）是操作符的表达式。
赋值是将变量值设置为评估表达式结果的语句。

3.1.1 表达式和赋值规则

以下规则适用于表达式和赋值：

- 表达式的每个操作数都有一个由操作符的性质决定的数据类型。
- 每个 KAREL 操作符都需要一个特定的操作数类型，并导致产生特定结果类型的计算。
- 表达式中的两个操作数必须具有相同的数据类型。 例如，AND运算符要求它的两个操作数都是 INTEGER 值或两者都是BOOLEAN值。 表达式 i AND b，其中 i 是 INTEGER，b 是 BOOLEAN，是无效的。
- 可以混合操作数的五种特殊情况为此规则提供了一个例外。 这五种情况包括以下内容：
  - INTEGER 和 REAL 操作数产生一个 REAL 结果
  - INTEGER 和 REAL 操作数产生 BOOLEAN 结果
  - INTEGER 和 VECTOR 操作数产生一个 VECTOR
  - REAL 和 VECTOR 操作数生成一个 VECTOR
  - POSITION 和 VECTOR 操作数生成一个 VECTOR
- 任何位置数据类型都可以替代 POSITION 数据类型。

3.1.2 评估表达式和赋值

表3-1总结了对包含KAREL运算符和操作数的表达式进行评估所得结果的各种类型。

表 3–1. 操作结果类型摘要

Operator（运算符）	+	-	*	/	DIV MOD	< >,>= <=, <, >, =	> =<	AND OR NOT	#	@	:
Operator（运算符）类型											
INTEGER（整数）	I	I	I	R	I	B	–	I	–	–	–
REAL（实数）	R	R	R	R	–	B	–	–	–	–	–
Mixed** INTEGER-REAL	R	R	R	R	–	B	–	–	–	–	–
BOOLEAN（布尔量）	–	–	–	–	–	B	–	B	–	–	–
STRING（字符串）	S	–	–	–	–	B	–	–	–	–	–
Mixed** INTEGER-VECTOR	–	–	V	V	–	–	–	–	–	–	–
Mixed** REAL-VECTOR	–	–	V	V	–	–	–	–	–	–	–
VECTOR（向量）	V	V	–	–	–	B***	–	–	V	R	–
POSITION（定位元素）	–	–	–	–	–	–	B	–	–	–	P
Mixed** POSITION-VECTOR	–	–	–	–	–	–	–	–	–	–	V

**Mixed混合意味着每种类型的一个操作数

***只能使用= <>来比较VECTOR值 - 不允许操作

I INTEGER

R REAL

B BOOLEAN

V VECTOR

P POSITION

3.1.3 变量和表达式

赋值语句包含变量和表达式。 变量可以是任何用户定义的变量，具有写入权限的系统变量或具有写入权限的输出端口阵列。 该表达式可以是任何有效的 KAREL 表达式。 以下示例是可接受的分配：
$ SPEED = 200.00 - 为系统变量分配一个 REAL 值
count = count + 1 - 将 INTEGER 值赋给 INTEGER 变量
变量和表达式的数据类型必须与三个例外相匹配：

- INTEGER变量可以分配给REAL变量。 在这种情况下，INTEGER在评估表达式时被视为REAL编号。 但是，在需要INTEGER值的情况下，不能使用REAL编号。
- 如果需要，可以使用ROUND或TRUNC内置函数将实数转换为INTEGER。
- INTEGER，BYTE和SHORT类型可以相互分配，但如果分配的值超出范围，则会发生运行时错误。
- 任何位置类型都可以分配给任何其他位置类型。 如果来自无运动组的JOINTPOS分配给XYZWPR，将导致运行时错误。

另请参见：关系运算，ROUND和TRUNC内置函数，附录A，''KAREL语言字母描述''

3.2 运算

操作包括操作变量，常量和文字以使用可用的KAREL运算符来计算值。 讨论以下操作：

- 算术运算
- 关系运算
- 布尔运算
- 特殊运算

表3-2列出了可用于 KAREL 的所有运算符。

表 3–2. KAREL Operators

操作	Operator					
算术运算	+	-	*	/	DIV	MOD
关系操作	<	< =	=	< >	> =	>
布尔运算	AND	OR	NOT			
特殊操作	> = <	:	#	@		

3.2.1 算术运算

加法（ + ），减法（ - ）和乘法（ * ）运算符以及DIV和MOD运算符可用于计算算术表达式中的值。 **请参阅表3-3。**

表 3–3. 使用+， - 和*运算符的算术运算

表达式	结果
3 + 2	5
3 - 2	1
3 * 2	6

- DIV和MOD操作符用于执行INTEGER除法。 请参阅表3-4。

表 3–4. 算术运算示例

表达式	结果
11 DIV 2	5
11 MOD 2	1

- 如果 DIV 运算符不是整数，则 DIV 运算符会截断方程的结果。
- MOD 运算符返回由除以得到的等式的余数左侧运算数由右侧运算数组成。
- 如果 MOD 方程的右侧运算数为负数，则结果也为负。
- 如果 DIV 方程中的除数或 MOD 方程的右侧运算数为零，则 KAREL 程序被“零除”错误中止。

- INTEGER 按位运算符 AND，OR 和 NOT 在两个 INTEGER 值上产生二进制 AND，OR 或 NOT 运算的结果。 请参阅表3-5。

表 3–5. 使用位操作数的算术运算

表达式	等效二进制数	结果
5 AND 8 5 OR 8	0101 AND 1000 0101 OR 1000	0000 = 0 1101 = 13
-4 AND 8 -4 OR 8	1100 AND 1000 1100 OR 1000	1000 = 8 1100 = -4
NOT 5 NOT -15	NOT 0101 NOT 110001	1010 = -6* 1110 = 14*

- *因为负的 INTEGER 值以二进制补码形式表示，所以 NOT i 与 -i 不一样。
- 如果 INTEGER 或 REAL 等式导致超出 INTEGER 或 REAL 变量限制的值，则程序中止并出现错误。 如果结果太小而无法表示，则将其设置为零。
- **表3-6**列出了KAREL运算符的优先级别。

表 3–6. KAREL 运算符优先级

操作符	**优先级**
NOT	High
:, @, #	↓
*, /, AND, DIV, MOD	↓
Unary + and -, OR, +, -	↓
<, >, =, < >, < =, > =, > = <	Low

3.2.2 关系运算

关系运算符（ <> ，= ，> ，< ，<= ，> =）产生一个 BOOLEAN（TRUE / FALSE）结果，对应于运算数的值是否在指定的关系中。 在关系表达式中，两个运算数必须是相同的简单数据类型。 这条规则有两个例外：

- INTEGER 运算数转换为 REAL 编号时，REAL 和 INTEGER 表达式可以混合使用。
- 例如，在表达式 1> .56 中，数字 1 被转换为 1.0，结果为 TRUE。
- VECTOR 运算数是结构化数据类型，可以在关系表达式中进行比较，但只能使用等式（ = ）或不等式（ <> ）运算符进行比较。

关系运算符使用 INTEGER 和 REAL 操作数来评估标准数学方程。 请参阅表3-7。

注意执行REAL值之间的等式（ = ）或不等式（ <> ）测试可能不会产生您期望的结果。 由于REAL值被存储和操作的方式，看起来相等的两个值可能不完全相等。 对于由 REAL 值组成的VECTOR值也是如此。 使用 > = 或 <= 在合适的地方代替 =。

关系运算符也可以将 STRING 值作为操作数。 按字符从左到右逐字比较 STRING 值，直到出现以下情况之一。 请参阅表3-7。

- 一个字符串中字符的字符代码大于另一个字符串中对应字符的字符代码。 这种情况下的结果是第一个字符串更大。 例如，A 的ASCII码为 65，a 为 97. 因此，a> A = TRUE。
- 一个字符串用尽，而另一个字符串保留在另一个字符串中。 结果是第一个 STRING 小于另一个 STRING。
- 两个 STRING 表达式都用尽了，但没有发现不匹配。 结果是 STRING 是平等的。

表 3–7. 关系操作示例

表达式	结果
'A' < 'AA'	TRUE
'A' = 'a'	FALSE
4 > 2	TRUE
17.3< > 5.6	TRUE
(3 *4) < > (4* 3)	FALSE

对于 BOOLEAN 操作数，TRUE> FALSE 被定义为一个真正的语句。 因此表达式 FALSE> = TRUE 是一个错误的陈述。 语句 FALSE> = FALSE 和 TRUE> = FALSE 也是真实的语句。

3.2.3 布尔运算符

具有BOOLEAN操作数的布尔运算符 AND，OR 和 NOT 可用于执行标准的数学评估。

表3-8总结了评估布尔表达式的结果，表3-9列出了一些示例。

表 3–8. 布尔运算概要

操作符	操作数1	操作数2	结果
NOT	TRUE	–	FALSE
	FALSE	–	TRUE
OR	TRUE	TRUE	TRUE
		FALSE	
	FALSE	TRUE	
		FALSE	FALSE
AND	TRUE	TRUE	TRUE
		FALSE	FALSE
	FALSE	TRUE	
		FALSE	

表 3–9. 布尔运算使用AND，OR和NOT运算符

表达式	结果
DIN[1] AND DIN[2]	TRUE if DIN[1] and DIN[2] are both TRUE; otherwise FALSE
DIN[1] AND NOT DIN[2]	TRUE if DIN[1] is TRUE and DIN[2] is FALSE; otherwise FALSE
(x < y) OR (y > z)	TRUE if x < y or if y > z; otherwise FALSE
(i = 2) OR (i = 753)	TRUE if i = 2 or if i = 753; otherwise FALSE

3.2.4 特殊运算

KAREL 语言提供了特殊的运算符来执行功能，例如测试大约相等的 POSITION 变量，相对POSITION 变量，VECTOR 变量和 STRING 变量的值。 本节介绍他们的操作，并举例说明他们的用法。
以下规则适用于大致相同的运算：

- 关系运算符（ > = < ）确定两个 POSITION 运算数是否近似相等并生成 BOOLEAN 结果。 比较与相等（ = ）关系相似，只是比较的运算数不必相同。 不考虑扩展轴的值。
- 必须将大致相等的操作与系统变量一起使用，
- $ LOCTOL，$ ORIENTTOL 和 $ CHECKCONFIG 来确定两个头寸必须有多接近。 有关这些变量的说明，请参阅 FANUC Robotics软件参考手册。

- 关系运算符（ > = < ）仅在正常程序使用中被允许，并且不能在条件处理程序语句中用作条件。

在以下示例中，关系运算符（ > = < ）用于确定当前机器人位置（通过使用 CURPOS 内置过程确定）是否接近指定的位置：

关系运算符

```
IF perch >=< CURPOS (0,0) THEN
  -- Call move to perch program
  move_to_perch
ELSE
  ABORT
ENDIF
```

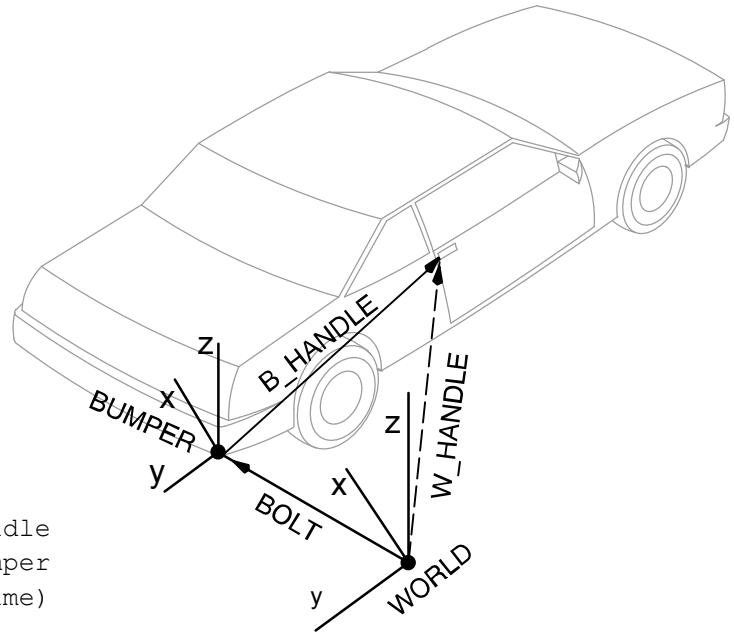
相对位置操作

要在空间中定位一个位置，必须将其引用到特定的坐标系。 在 KAREL 中，参考框架具有 POSITION 数据类型。 相对位置运算符（:)允许您相对于另一个位置（即以另一位置为原点的坐标系）的坐标系参考位置或向量。

相对位置运算符（:)用于将位置从一个参考帧转换为另一个帧。

在图3-1 所示的示例中，使用视觉系统来定位汽车上的目标，例如保险杠上的螺栓头。 相对位置运算符用于根据汽车图纸的数据计算门把手的位置。 图3-1中显示的公式用于计算 WORLD 框架中 w_handle 的位置。

图 3–1. 确定相对于WORLD坐标系的w_handle




```
w_handle = bolt : b_handle
(world     (world    (bumper
 frame)     frame)    frame)
```

where:

`bolt` 是在 WORLD frame 中引用的 BUMPER frame 原点的位置。

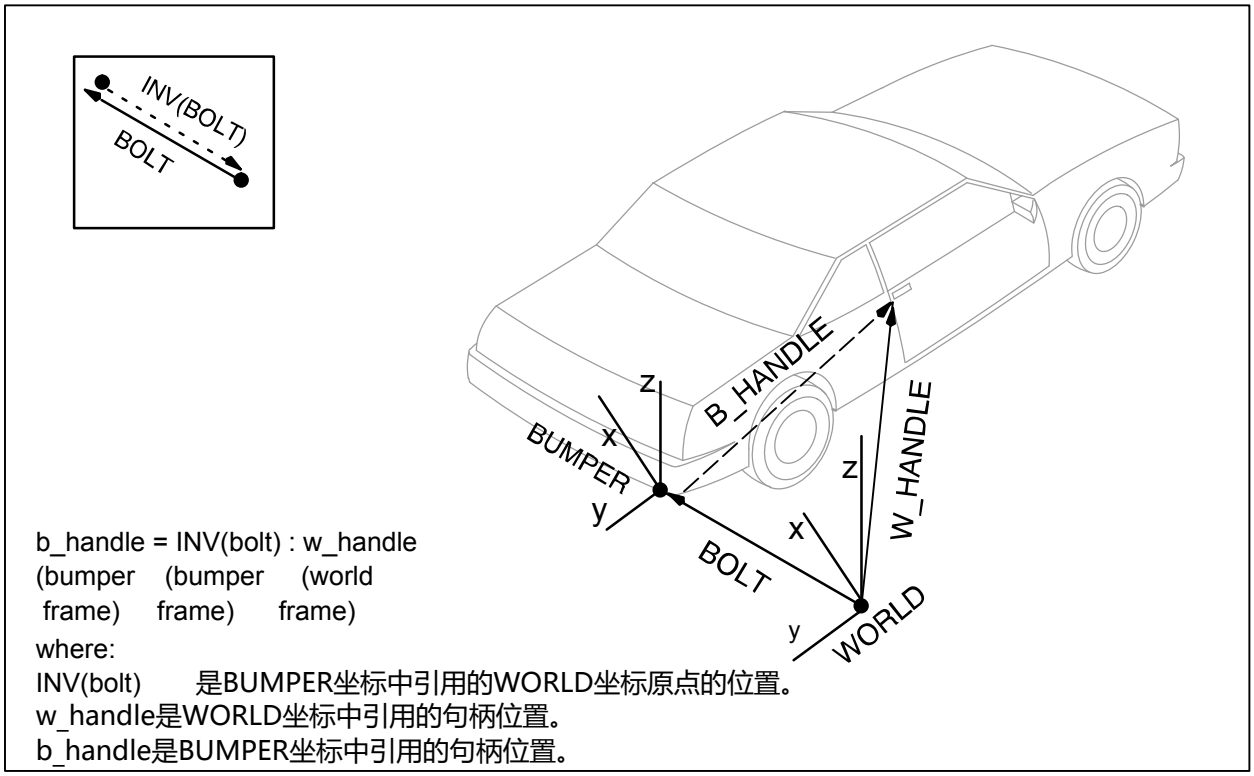
`w_handle` 是 WORLD frame 中引用的手柄位置。

`b_handle` 是 BUMPER frame 中引用的手柄位置。

KAREL INV 内置功能反转参考方向。

例如，要确定门把手目标（b_handle）相对于螺栓位置的位置，请使用 图3-2 中所示的公式。

图 3–2. 确定相对于BUMPER坐标的b_handle



b_handle = INV(bolt) : w_handle
(bumper (bumper (world
frame) frame) frame)
where:
INV(bolt) 是BUMPER坐标中引用的WORLD坐标原点的位置。
w_handle是WORLD坐标中引用的句柄位置。
b_handle是BUMPER坐标中引用的句柄位置。

注意相对运算符(:)的顺序很重要。其中：b_handle = bolt：w_handle与b_handle = w_handle不同：bolt

另请参见：第8章位置数据，INV内置函数，附录A.

矢量运算

以下规则适用于VECTOR操作：

- VECTOR表达式可以在VECTOR操作数上执行加法（+）和减法（ - ）方程。 结果是VECTOR，其组件是操作数的相应组件的总和或差异。 例如，由于以下程序语句，VECTOR vect_3的组件将等于（5,10,9）：

矢量运算

```
vect_1.x = 4; vect_1.y = 8; vect_1.z = 5
vect_2.x = 1; vect_2.y = 2; vect_2.z = 4
vect_3 = vect_1 + vect_2
```

- 乘法（*）和除法（/）运算符可以与其中任何一个一起使用

- 一个 **VECTOR** 和一个 **INTEGER** 操作数
- 一个 **VECTOR** 和一个 **REAL** 操作数

VECTOR 和 **INTEGER** 或 **VECTOR** 和 **REAL** 的乘积是 **VECTOR** 的缩放版本。

VECTOR 的每个分量乘以 **INTEGER**（视为**REAL**数）或 **REAL**。

例如，**VECTOR**（8,16,10）是由以下操作产生的：

（4,8,5）* 2

VECTOR 组件可位于操作员的左侧或右侧。

- 由INTEGER或REAL分割的VECTOR会使VECTOR的每个分量被INTEGER（视为REAL数字）或REAL分割。例如，（4,8,5）/ 2导致（2,4,2.5）。
  如果除数为零，则程序将以“除以零”错误中止。
- 由VECTOR分割的INTEGER或REAL会使INTEGER（视为REAL数）或REAL乘以VECTOR的每个元素的倒数，从而产生新的VECTOR。例如，3.5 / VEC（7.0,8.0,9.0）导致（0.5,0.4375,0.38889）。
  如果VECTOR的任何元素为零，则程序将以“除以零”错误中止。
- 交叉乘积运算符（#）产生一个VECTOR，它与右手规则指示的方向上的两个操作数垂直，其幅度等于两个矢量和SIN（Θ）的大小的乘积，其中θ是两个向量之间的角度。例如，VEC（3.0,4.0,5.0）#VEC（6.0,7.0,8.0）导致（-3.0,6.0，-3.0）。
  如果任一矢量为零，或矢量完全平行，则会发生错误。
- 内积运算符（@）生成一个REAL数，它是两个向量的相应元素的乘积之和。例如，VEC（3.0,4.0,5.0）@VEC（6.0,7.0,8.0）导致86.0。
- 如果上述任何操作的结果是VECTOR的一个分量，其幅度对于KAREL REAL数字而言太大，则程序将因“实际溢出”错误而中止。

表3-10列出了向量运算的其他示例。

表3-10。 矢量运算的例子

表达式	结果
VEC(3.0,7.0,6.0) + VEC(12.6,3.2,7.5)	(15.6,10.2,13.5)
VEC(7.6,9.0,7.0) - VEC(14.0,3.5,17.0)	(-6.4,5.5,-10)
4.5 * VEC(3.2,7.6,4.0)	(14.4,34.2,18.0)
VEC(12.7,2.0,8.3) * 7.6	(96.52,15.2,63.08)
VEC(17.3,1.5,0.23) /2	(8.65,0.75,0.115)

字符串操作

以下规则适用于STRING操作：

- 您可以指定KAREL例程返回STRING作为其值。 请参阅指定KAREL例程以返回STRING值。

指定KAREL例程以返回STRING值

```
ROUTINE name(parameter_list):  STRING
    declares name as returning a STRING value
```

- 可以在字符串之间使用运算符来指示字符串的串联。 请参阅使用运算符连接字符串。

使用运算符连接字符串

```
string_1 = string_2 + string_3 + 'ABC' + 'DEF'
```

- STRING表达式可以在WRITE语句中使用。 请参阅在WRITE语句中使用STRING表达式。

在WRITE语句中使用STRING表达式

```
WRITE(CHR(13) + string_1 + string_2)
    writes a single string consisting of a return
    character followed by string_1 and string_2
```

- 在STRING分配期间，如果目标字符串不足以容纳相同的字符串，则字符串将被截断。 “您可以比较或提取字符串中的字符。 例如，如果string_1 = “ABCDE”？ 你的输出将是“D”。参见比较。

字符串比较

```
IF SUB_STR(string_1, 4, 1) = 'D' THEN
```

- 您可以从另一个字符串构建一个字符串。 请参阅从另一个字符串构建字符串。

从另一个字符串构建字符串

```
ROUTINE toupper(p_char: INTEGER): STRING
 BEGIN
   IF (p_char > 96) AND (p_char < 123) THEN
      p_char = p_char - 32
   ENDIF
   RETURN (CHR(p_char))
 END toupper
 BEGIN
    WRITE OUTPUT ('Enter string: ')
    READ INPUT (string_1)
    string_2 = ''
    FOR idx = 1 TO STR_LEN(string_1) DO
     string_2 = string_2 + toupper(ORD(string_1, idx))
    ENDFOR
```

第4章

程序控制

目录

第4章 程序控制 4–1
4.1 概述 4–2
4.2 程序控制结构 4–2
4.2.1 交替控制结构 4–2
4.2.2 循环控制声明 4–3
4.2.3 无条件分支声明 4–3
4.2.4 执行控制声明 4–3
4.2.5 条件处理程序 4–4

4.1 概述

程序控制结构定义程序或例程中的执行流程，包括交替，循环和无条件分支以及执行控制。

4.2 程序控制结构

程序控制结构可用于定义程序或例程中的执行流程。 默认情况下，执行从BEGIN语句后面的第一个语句开始，并按顺序继续，直到遇到END语句（或RETURN语句）。 KAREL提供以下控制结构：

- 交替
- 循环
- 无条件分支
- 执行控制
- 条件处理程序

有关每种控制结构的详细信息，请参阅附录A“KAREL语言字母说明”。

4.2.1 交替控制结构

交替控制结构允许您在程序或例程中包含替代语句序列。 每个备选方案都可以包含多个语句。
在程序执行期间，基于一个或多个数据项的值选择备选方案。 程序执行然后继续执行选定的语句序列。
可以使用两种类型的交替控制结构：

- **IF**语句 - 提供基于**BOOLEAN**表达式的值指定两个备选方案之一的方法。
- **SELECT**语句 - 在多个备选方案之间进行选择时使用。 根据指定的**INTEGER**表达式的值选择替代方案。

另请参阅：IF ... THEN声明，附录A，SELECT声明，附录A.

4.2.2 循环控制声明

循环控制结构允许您根据程序中数据项的值指定一组语句重复任意次数。 KAREL支持三种循环控制结构：

- **FOR** 语句 - 在一组语句执行指定次数时使用。 次数由FOR语句中的INTEGER数据项确定。 在FOR循环开始时，范围中的初始值将分配给INTEGER计数器变量。 每次重复循环时，都会重新评估计数器。
- **REPEAT** 语句 - 只要某个BOOLEAN表达式保持为FALSE，就允许继续执行一系列语句。 REPEAT语句中的可执行语句序列将始终执行一次。
- **WHILE** 语句 - 只要BOOLEAN表达式保持为TRUE，就会在执行操作时使用。 布尔表达式在每次迭代开始时进行测试，因此可以将操作执行零次。

另请参阅：FOR声明，附录A，REPEAT声明，附录A，WHILE声明，附录A.

4.2.3 无条件分支声明

无条件分支允许您使用 **GO TO** 语句将控制从程序中的一个位置转移到程序的另一个区域中的指定标签，而不依赖于条件或 **BOOLEAN** 表达式。

警告

切勿在 GO 循环中包含 GO TO 语句。 程序可能会因“运行时堆栈溢出”错误而中止。

另见：转到声明，附录A.

4.2.4 执行控制声明

KAREL 语言提供以下程序控制语句，用于终止或暂停程序执行：

- **ABORT** - 导致程序的执行，包括正在进行的任何动作，被终止。 中止后程序无法继续。
- **DELAY** - 导致执行暂停指定的时间，以毫秒为单位。

- **PAUSE** - 导致执行暂停，直到执行 **CONTINUE** 操作。
- **WAIT FOR** - 导致执行暂停，直到满足指定条件或条件列表。

另请参阅：ABORT语句，DELAY语句，PAUSE语句，WAIT FOR语句，全部见附录A，第6章条件处理程序

4.2.5 条件处理程序

条件处理程序定义了在满足指定条件时要执行的一系列操作。 一旦定义，条件处理程序可以是 **ENABLED** 或 **DISABLED**。

第5章

ROUTINES

目录

第5章 ROUTINES 5–1
5.1 ROUTINE 执行 5–2
5.1.1 ROUTINE 声明 5–2
5.1.2 ROUTINE 调用 5–5
5.1.3 从 ROUTINE 返回 5–7
5.1.4 变量范围 5–8
5.1.5 Parameters 和 Arguments 5–9
5.1.6 堆栈使用 5–13
5.2 内置程序 5–15

ROUTINE与程序类似，提供了一种模块化KAREL程序的方法。 ROUTINE可以包括VAR和/或CONST声明和可执行语句。 但是，与程序不同，ROUTINE必须在大写程序中声明，并且不能包含其他ROUTINE声明。
KAREL支持两种类型的ROUTINE：

- 过程ROUTINE - 不返回值
- 函数ROUTINE - 返回值

KAREL ROUTINE 可以是称为内置ROUTINE的预定义例程，也可以是用户定义的。 以下规则适用于所有KAREL ROUTINE：

- 参数可以包含在ROUTINE的声明中。 这允许您在调用时将数据传递到ROUTINE，并将结果返回给调用程序。
- 可以调用或调用ROUTINE：
  - 通过宣布他们的计划
  - 通过该程序中包含的任何ROUTINE
  - 通过其他程序的声明，请参阅第5.1.1节

5.1 ROUTINE 执行

本节介绍过程和功能例程的执行过程：

- 声明例程
- 调用例程
- 从例程中返回
- 变量的范围
- 参数和参数

5.1.1 ROUTINE 声明

以下规则适用于例行声明：

- 例程不能在另一个例程中声明。
- ROUTINE语句用于声明过程和函数例程。
- 必须在调用过程和函数例程之前声明它们。

- 程序本地的例程在程序中完全定义。声明
- 当地的例程包括：
  - ROUTINE声明
  - 例程的任何VAR和/或CONST声明
  - 例程的可执行语句
- 虽然例程中的VAR和CONST部分的语法与程序中的相同，但以下限制适用：
  - 无法指定PATH，FILE和视觉数据类型。
  - 不允许使用FROM子句。
  - IN子句是不允许的。
- 如果使用具有相同程序名的FROM子句声明例程，则可以在可执行部分之后定义程序本地的例程。 参数只能定义一次。 请参阅使用FROM子句定义本地例程。

使用FROM子句定义本地例程

```
PROGRAM funct_lib
  ROUTINE done_yet(x: REAL; s1, s2: STRING): BOOLEAN FROM funct_lib
BEGIN
IF done_yet(3.2, 'T', '')
--
END funct_lib
ROUTINE done_yet
BEGIN
--
END done_yet
```

- 程序外部的例程在一个程序中声明，但在另一个程序中定义。
  - 外部例程的声明只包括ROUTINE语句和一个FROM子句。
  - FROM子句标识定义程序的程序的名称。
  - 例程必须在FROM子句中指定的程序的本地定义。
- 您可以在例程的声明中包含参数列表。 参数列表是ROUTINE语句的可选部分。
- 如果例程在程序外部，则参数列表中的名称没有意义，但必须包含这些名称以指定参数。如果没有参数，则用于包含列表的括号必须在外部和本地例程中省略。

本地和外部过程声明中的示例说明了本地和外部过程例程声明。

本地和外部程序声明

```
PROGRAM procs_lib
ROUTINE wait_a_bit
 --本地程序，没有参数
   BEGIN
     DELAY 20
END wait_a_bit
ROUTINE toggle_out(i: INTEGER)
 --本地程序，一个参数
   BEGIN
     DOUT[i] = ON     --参考参数i
      DELAY 1000
     DOUT[i] = OFF
   END toggle_out
ROUTINE calc_dist(p1,p2: POSITION; dist: REAL)& FROM math_lib
 --math_lib.kL 中定义的外部过程
BEGIN
END procs_lib
```

函数声明中的示例说明了本地和外部函数例程声明。

函数声明

```
PROGRAM funct_lib
ROUTINE done_yet(x: REAL; s1, s2 :STRING): BOOLEAN&     FROM bool_lib
 --在bool_lib.kl中定义的外部函数例程
  --返回一个BOOLEAN值
ROUTINE xy_dist(x1,y1,x2,y2: REAL): REAL
 --本地函数，返回一个REAL值
  VAR
    sum_square: REAL  --动态局部变量
     dx,dy: REAL       --动态局部变量
     BEGIN
     dx = x2-x1   --references parameters x2 and x1
     dy = y2-y1   --references parameters y2 and y1
     sum_square = dx * dx + dy * dy
     RETURN(SQRT(sum_square))  --SQRT is a built-in
  END xy_dist
BEGIN
END funct_lib
```

只课 > FROM子句，附录A，常规声明，附录A.

5.1.2 ROUTINE 调用

在程序中声明的例程可以在程序的可执行部分中调用，也可以在程序中包含的任何例程的可执行部分内调用。 调用例程会导致调用例程。 根据以下过程调用例程：

- 调用例程时，执行控制权将传递给例程。
- 完成过程的执行后，控制将返回到调用过程的点之后的下一个语句。
- 完成函数的执行后，控制返回到调用函数的赋值语句。

调用过程和函数例程时，以下规则适用：

- 过程和函数例程都使用例程名称调用，后跟为例程声明的每个参数的参数。
- 参数列表括在括号中。
- 不带参数的例程仅使用例程名称调用。
- 调用过程就像它是一个语句一样。 因此，过程调用构成完整的可执行语句。

过程调用显示两个过程的声明，然后是过程调用以调用它们。

程序调用

```
ROUTINE wait_a_bit FROM proc_lib
   --没有参数的外部程序
    ROUTINE calc_dist(p1,p2: POSITION; dist:
REAL)& FROM math_lib
   --外部程序有三个参数
    BEGIN
 ...
 wait_a_bit      --调用wait_a_bit程序
    calc_dist (start_pos, end_pos, distance)
   --使用三个参数来调用calc_dist
   --三个声明的参数
```

- 因为函数返回一个值，所以函数调用必须作为表达式的一部分或全部出现。
- 当控制返回到调用程序或例程时，使用返回的值恢复包含函数调用的语句的执行。

函数调用显示两个函数的声明，后跟函数调用以调用它们。

函数调用

```
ROUTINE error_check : BOOLEAN FROM error_prog
 --没有参数的外部函数返回BOOLEAN值
  ROUTINE distance(p1, p2: POSITION) : REAL &
FROM funct_lib
 --带有两个参数的外部函数返回REAL值
  BEGIN  --主程序
 --调用函数error_check并返回BOOLEAN
 --在IF语句中的表达式
   IF error_check THEN
   ...
   ENDIF
   travel_time = distance(prev_pos, next_pos)/current_spd
 --函数距离作为表达式的一部分被调用
 --任务说明
```

- 只要在包含初始例程的程序中声明了其他例程，例程就可以调用其他例程。 例如，如果名为 master_prog 的程序包含名为 call_proc 的例程，则该例程可以调用程序 master_prog 中声明的任何例程。
- 调用自身的例程被认为是递归的，并且在KAREL中被允许。 例如，递归函数中显示的例程因子将自身调用以计算因子值。

递归函数

```
ROUTINE factorial(n: INTEGER) : INTEGER
 --计算整数n的阶乘值
  BEGIN
  IF n = 0  THEN RETURN (1)
 ELSE RETURN (n * factorial(n-1))
 --递归调用阶乘
   ENDIF
 END factorial
```

- 对例程调用深度的唯一约束是使用KAREL堆栈，用于存储临时和局部变量以及参数。 例程调用导致信息被放置在堆栈的内存中。 在例程中执行RETURN或END语句时，此信息将从堆栈中删除。 如果在没有从堆栈中删除此信息的情况下进行太多例程调用，则程序将耗尽堆栈空间。

另请参见：第5.1.6节，了解有关例行调用的堆栈使用了多少空间的信息

5.1.3 从ROUTINE返回

RETURN语句在例程中用于将执行控制从例程恢复到调用例程或程序。

从例程返回时适用以下规则：

- 在过程中，RETURN语句不能包含值。
- 如果没有执行RETURN语句，END语句将恢复对调用程序或例程的控制。

过程RETURN语句说明了在过程中使用RETURN语句的一些示例。

过程RETURN语句

```
ROUTINE gun_on (error_flag: INTEGER)
 --打开"枪"时执行某些操作
 --根据不同的陈述回复,
 --如果发生任何错误。
VAR gun: INTEGER
BEGIN
IF error_flag = 1 THEN RETURN
--从常规退出, 退回之前
--执行WHILE循环
 ENDIF
  WHILE DIN[gun] DO
--继续, 直到枪关闭
  ...
IF error_flag = 2 THEN RETURN
--从常规退出, 从中返回
--with WHILE循环
  ENDIF
ENDWHILE  --枪是关闭的
END gun_on  --正常退出常规
```

- 在函数中，RETURN 语句必须指定在控制时要传回的值
- 恢复到调用例程或程序。
- 函数例程可以返回除外的任何数据类型

  -文件

  -路径

  - 视觉类型
- 如果返回类型是ARRAY，则无法指定大小。 这允许函数返回任何长度的ARRAY。 从ARRAY值函数返回的ARRAY可以

仅用于直接赋值语句。 ARRAY值函数不能用作其他例程的参数。 有关在两个函数例程之间传递的ARRAY的示例，请参阅ARRAY的正确传递。

- 如果函数的RETURN语句中未提供任何值，则会生成转换程序错误。
- 如果函数中没有执行RETURN语句，则在到达END语句时，函数的执行将终止。 没有值可以传递回调用例程或程序，因此程序会因错误而中止。

函数RETURN语句说明了在函数例程中使用RETURN语句的一些示例。

函数RETURN语句

```
ROUTINE index_value (table: ARRAY of INTEGER;
   table_size: INTEGER): INTEGER
 --返回FOR循环 (i) 的索引值取决于
 --IF声明的条件。 在其中的情况下返回0
 --不满足-IF条件。
 VAR i: INTEGER
 BEGIN
  FOR i = 1 TO table_size DO
    IF table[i] = 0 THEN RETURN (i)  --返回索引
     ENDIF
    ENDFOR
    RETURN (0)  --returns 0
 END index_value
 ROUTINE compare (test_var_1: INTEGER;
    test_var_2: INTEGER): BOOLEAN
 --在IF测试的情况下，返回TRUE值
 --满足。否则，返回FALSE值。
    BEGIN
  IF test_var_1 = test_var_2 THEN
    RETURN (TRUE)   --returns TRUE
  ELSE
    RETURN (FALSE)  --returns FALSE
  ENDIF
 END compare
```

另见：ROUTINE声明，附录A.

5.1.4 变量范围

变量声明的范围可以是

- Global

- Local

全局声明和定义

以下规则适用于全局声明和定义：

- 全局声明在整个计划中得到认可。
- 全局声明称为静态，因为它们被赋予一个在程序执行期间不会更改的内存位置，即使程序被清除或重新加载（除非变量本身被清除）。
- 主程序中的声明以及预定义的标识符是全局的。
- 预定义和用户定义的例程，类型，变量，常量和标签的范围规则如下：
  - 在整个程序中识别所有预定义标识符。
  - 在程序的声明部分中声明的例程，类型，变量和常量在整个程序中被识别，包括程序中的例程。

本地声明和定义

以下规则适用于本地声明和定义：

- 本地声明仅在声明它们的例程中被识别。
- 调用例程时会创建本地数据。 当例程完成执行并返回时，本地数据被销毁。
- 预定义和用户定义的例程，变量，常量和标签的范围规则如下：
  - 在例程的声明部分中声明的变量和常量，以及在例程参数列表中声明的参数仅在该例程中被识别。
  - 程序中定义的标签（不在程序的例程中）是程序主体的本地标签，在程序的任何例程中都不会被识别。
  - 例程中定义的标签是例程的本地标签，仅在该例程中被识别。
- 类型不能在例程中声明，因此从不是本地的。

5.1.5 Parameters（函数定义中参数）和 Arguments（函数调用时的实际参数）

在例程声明的参数列表中使用的标识符称为参数。 在整个例程中可以引用例程中声明的参数。 参数用于在调用程序和例程之间传递数据。 调用中提供的数据（称为参数）可能会影响例程的执行方式。

以下规则适用于例程调用的参数列表：

- 作为例程调用的一部分，您必须为例程声明中的每个参数提供一个数据项（称为参数）。
- 参数可以是变量，常量或表达式。 每个参数必须有一个参数。
- 参数必须与它们对应的参数具有相同的数据类型，但有三个例外：

  - INTEGER参数可以传递给REAL参数。 在这种情况下，INTEGER值被视为REAL类型，INTEGER的REAL等效值通过值传递给例程。

  - BYTE或SHORT参数可以通过值传递给INTEGER或REAL参数.-任何位置类型都可以传递给任何其他位置类型。 如果将它们传递给用户定义的例程，则转换参数位置类型并将其按值传递给参数类型。

  - 任何长度的ARRAY或STRING参数都可以传递给相同数据类型的参数。

对应的参数和参数显示了例程声明的示例和对该例程的三次调用。

Parameters（函数定义中参数）和 Arguments（函数调用时的实际参数）

```
PROGRAM params
  VAR
    long_string: STRING[10]; short_string: STRING[5]
    exact_dist: REAL; rough_dist: INTEGER
  ROUTINE label_dist (strg: STRING; dist: REAL) &
    FROM procs_lib
  BEGIN
  ...
    label_dist(long_string, exact_dist)
      --long_string corresponds to strg;
      --exact_dist corresponds to dist
    label_dist(short_string, rough_dist)
      --short_string, of a different length,
      --corresponds to strg; rough_dist, an
      --INTEGER, corresponds to REAL dist
    label_dist('new distance', (exact_dist * .75))
      --literal constant and REAL expression
      --arguments correspond to the parameters
END params
```

- 调用例程时，例程调用中使用的参数将传递给相应的参数。 两个方法用于将参数传递给参数：

  - 通过引用传递参数

如果通过引用传递参数，则相应的参数与参数共享相同的内存位置。 因此，更改参数的值会更改相应参数的值。

-通过值传递参数

如果参数按值传递，则参数的临时副本将传递给例程。 相应的参数使用此临时副本。 更改参数不会影响原始参数。

- 常量和表达式参数始终按值传递给例程。 变量通常通过引用传递。 但是，以下变量参数按值传递： - 端口数组变量
  - 传递给REAL参数的INTEGER变量
  - 将BYTE和SHORT参数传递给INTEGER或REAL参数
  - 具有只读（RO）访问权限的系统变量
  - 需要转换的位置参数
- 虽然变量参数通常通过引用传递，但您可以通过将变量标识符括在括号中来按值传递它们。 实际上，括号将变量转换为表达式。
- PATH，FILE和视觉变量不能按值传递。 ARRAY元素（ARRAY变量的索引形式）可以通过值传递，但整个ARRAY变量不能。

传递变量参数显示一个例程，它根据传递变量的参数，以不同方式影响传递给它的参数。

传递变量参数

```
PROGRAM reference
  VAR arg :  INTEGER
  ROUTINE test(param : INTEGER)
  BEGIN
    param = param * 3
    WRITE ('value of param:', param, CR)
  END test
  BEGIN
    arg = 5
    test((arg))    --arg passed to param by value
    WRITE('value of arg:', arg, CR)
    test(arg)      --arg passed to param by reference
    WRITE('value of arg:', arg, CR)
  END reference
```

传递变量参数中程序的输出如下：

```
value of param: 15
```

```
value of arg: 5
value of param: 15
value of arg: 15
```

如果以相反顺序进行传递变量参数的例程调用，首先使用“test（arg）”通过引用传递arg，然后使用“test（（arg））”按值传递，输出将受到如下影响：

```
value of param: 15
value of arg: 15
value of param: 45
value of arg: 15
```

- 要将变量作为参数传递给KAREL例程，可以使用以下两种方法之一： - 可以在参数列表中指定变量的名称。例如，other_rtn（param_var）将变量param_var传递给例程other_rtn。要编写此语句，您必须知道要传递的变量的名称。
  - 您可以使用BYNAME.BYNAME功能允许程序作为参数将名称包含在字符串中的变量作为参数传递给例程。例如，如果字符串变量prog_name和var_name包含运算符输入的程序和变量的名称，则使用以下语法将此变量传递给例程：
- other_rtn（BYNAME（prog_name，var_name，entry））
- 有关BYNAME的更多信息，请参阅附录A.
- 如果函数例程返回ARRAY，则对此函数的调用不能用作另一个例程的参数。如果尝试传递错误，则会检测到转换错误。
- ARRAY的正确通道显示正确使用在两个函数例程之间传递的ARRAY。

ARRAY 的正确通道

```
PROGRAM correct
     VAR a : ARRAY[8] of INTEGER
       ROUTINE rtn_ary : ARRAY of INTEGER  FROM util_prog
       ROUTINE print_ary(arg : ARRAY of INTEGER)
         VAR i : INTEGER
       BEGIN
         FOR i = 1 to ARRAY_LEN(arg) DO
           WRITE(arg[i],cr)
         ENDFOR
       END print_ary
     BEGIN
       a = rtn_ary
       print_ary(a)
```

```
    END correct
```

ARRAY的错误传递表明在两个函数例程之间传递的ARRAY使用不正确。

ARRAY 的错误传递

```
PROGRAM wrong
  ROUTINE rtn_ary : ARRAY of INTEGER  FROM util_prog
  ROUTINE print_ary(arg : ARRAY of INTEGER)
    VAR i : INTEGER
    BEGIN
      FOR i = 1 to ARRAY_LEN(arg) DO
        WRITE(arg[i],cr)
      ENDFOR
    END print_ary
  BEGIN
    print_ary(rtn_ary)
  END wrong
```

另请参阅：ARRAY_LEN内置函数，附录A，STR_LEN内置函数，附录A，附录E， “语法图

5.1.6 堆栈使用

执行程序时，除非指定堆栈大小，否则将分配300个字的堆栈。 堆栈是从可用的用户RAM分配的。

堆栈使用情况可按如下方式计算：

- 每个调用（或函数引用）至少使用五个字的堆栈。
- 此外，对于例程中的每个参数和局部变量，将使用堆栈上的额外空间，具体取决于变量或参数类型，如表5-1中所示。

Table 5–1. 堆栈使用

类型	参考传递的参数	参数传递的值	局部变量
BOOLEAN ARRAY OF BOOLEAN	1	2 not allowed	1 1 + array size
ARRAY OF BYTE	1	not allowed	1 + (array size)/4

Table 5–1. 堆栈使用（续）

类型	参考传递的参数	参数传递的值	局部变量
CAM_SETUP ARRAY OF CAM_SETUP	1	not allowed not allowed	not allowed not allowed
CONFIG ARRAY OF CONFIG	1	2 not allowed	1 1 + array size
INTEGER ARRAY OF INTEGER	1	2 not allowed	1 1 + array size
FILE ARRAY OF FILE	1	not allowed not allowed	not allowed not allowed
JOINTPOS ARRAY OF JOINTPOS	2 1	12 not allowed	10 1 + 10 * array size
JOINTPOS1 ARRAY OF JOINTPOS1	2 1	4 not allowed	2 1 + 2 * array size
JOINTPOS2 ARRAY OF JOINTPOS2	2 1	5 not allowed	3 1 + 3 * array size
JOINTPOS3 ARRAY OF JOINTPOS3	2 1	6 not allowed	4 1 + 4 * array size
JOINTPOS4 ARRAY OF JOINTPOS4	2 1	7 not allowed	5 1 + 5 * array size
JOINTPOS5 ARRAY OF JOINTPOS5	2 1	8 not allowed	6 1 + 6 * array size
JOINTPOS6 ARRAY OF JOINTPOS6	2 1	9 not allowed	7 1 + 7 * array size
JOINTPOS7 ARRAY OF JOINTPOS7	2 1	10 not allowed	8 1 + 8 * array size
JOINTPOS8 ARRAY OF JOINTPOS8	2 1	11 not allowed	9 1 + 9 * array size
JOINTPOS9 ARRAY OF JOINTPOS9	2 1	12 not allowed	10 1 + 10 * array size
MODEL ARRAY OF MODEL	1 1	not allowed not allowed	not allowed not allowed
PATH	2	not allowed	not allowed
POSITION ARRAY OF POSITION	2 1	16 not allowed	14 1 + 14 * array size

Table 5–1. 堆栈使用（续）

类型	参考传递的参数	参数传递的值	局部变量
REAL ARRAY OF REAL	1 1	2 not allowed	1 1 + array size
ARRAY OF SHORT	1	not allowed	1 + (array size)/2
STRING ARRAY OF STRING	2 1	2 + (string length+2)/4not allowed	(string length+2)/4 1+((string length+2) *array size)/4
VECTOR ARRAY OF VECTOR	1 1	4 not allowed	3 1 + 3 * array size
VIS_PROCESS ARRAY OF VIS_PROCESS	1 1	not allowed not allowed	not allowed not allowed
XYZWPR ARRAY OF XYZWPR	2 1	10 not allowed	8 1 + 8 * array size
XYZWPREXT ARRAY OF XYZWPREX	2 1	13 not allowed	11 1 + 11 * array size
ARRAY [m,n] OF some_type	1	not allowed	m(ele size/4 * n + 1)+1
ARRAY [l,m,n] OF some_type	1	not allowed	l(m(ele size/4 * n + 1)+1)+1

5.2 内置程序

KAREL语言包括称为KAREL内置例程或内置函数的预定义例程。 预定义的例程可以是过程或函数内置函数。 它们作为编程方便提供，并执行常用的服务。

许多内置函数返回一个表示错误的状态参数（如果不等于0）。返回的错误可以是特定于应用程序的FANUC机器人设置和操作手册中定义的任何错误代码。 通过使用返回的status参数调用POST_ERR内置例程，可以将这些错误发布到错误日志并显示在错误行中。

表A-7是KAREL语言中包含的所有预定义内置例程的摘要列表。 附录A中提供了所有KAREL内置例程的详细说明。

另请参阅：附录A，其中列出了可选的KAREL内置函数以及它们的文档。

Table 5–2. KAREL内置常规摘要（续）

类别	标识符		
Byname	CALL_PROG CALL_PROGLIN	CURR_PROG FILE_LIST	PROG_LIST VAR_INFO VAR_LIST
Data Acquisition	DAQ_CHECKP DAQ_REGPIPE	DAQ_START DAQ_STOP	DAQ_UNREG DAQ_WRITE
Error Code Handling	ERR_DATA	POST_ERR	POST_ERR_L
File and Device Operation	CHECK_NAME COMPARE_FILE COPY_FILE DELETE_FILE DISMOUNT_DEV DOSFILE_INF FORMAT_DEV	MOUNT_DEV MOVE_FILE PRINT_FILE PURGE_DEV RENAME_FILE	XML_ADDTAG XML_GETDATA XML_REMTAG XML_SCAN XML_SETVAR
Serial I/O, File Usage	BYTES_AHEAD BYTES_LEFT CLR_IO_STAT GET_FILE_POS GET_PORT_ATR	IO_STATUS MSG_CONNECT MSG_DISCO MSG_PING PIPE_CONFIG	SET_FILE_ATR SET_FILE_POS SET_PORT_ATR VOL_SPACE
Process I/O Setup	CLR_PORT_SIM GET_PORT_ASG GET_PORT_CMT GET_PORT_MOD	GET_PORT_SIM GET_PORT_VAL IO_MOD_TYPE SET_PORT_ASG	SET_PORT_CMT SET_PORT_MOD SET_PORT_SIM SET_PORT_VAL
KCL Operation	KCL	KCL_NO_WAIT	KCL_STATUS
Memory Operation	CLEAR CREATE_VAR LOAD LOAD_STATUS	PROG_BACKUP PROG_CLEAR PROG_RESTORE RENAME_VAR	RENAME_VARS SAVE SAVE_DRAM
Mirror	MIRROR		
Motion and Program Control	CNCL_STP_MTN	MOTION_CTL	RESET
Multi-programming	ABORT_TASK CLEAR_SEMA CONT_TASK GET_TSK_INFO LOCK_GROUP	PAUSE_TASK PEND_SEMA POST_SEMA RUN_TASK	SEMA_COUNT SET_TSK_ATTR SET_TSK_NAME UNLOCK_GROUP
Path Operation	APPEND_NODE COPY_PATH	DELETE_NODE INSERT_NODE	NODE_SIZE PATH_LEN

Table 5–2. KAREL内置常规摘要（续）

类别	标识符		
个人电脑通讯	ADD_BYNAMEPC ADD_INTPC	ADD_REALPC ADD_STRINGPC	SEND_DATAPC SEND_EVENTPC
位置	CHECK_EPOS CNV_JPOS_REL CNV_REL_JPOS CURPOS CURJPOS	FRAME IN_RANGE J_IN_RANGE JOINT2POS	POS POS2JOINT SET_PERCH UNPOS
队列管理器	APPEND_QUEUE COPY_QUEUE DELETE_QUEUE	GET_QUEUE INIT_QUEUE	INSERT_QUEUE MODIFY_QUEUE
寄存器操作	CLR_POS_REG GET_JPOS_REG GET_POS_REG GET_PREG_CMT GET_REG GET_REG_CMT	GET_SREG_CMT GET_STR_REG POS_REG_TYPE SET_EPOS_REG SET_INT_REG SET_JPOS_REG	SET_POS_REG SET_PREG_CMT SET_REAL_REG SET_REG_CMT SET_SREG_CMT SET_STR_REG
字符串操作	CNV_CNF_STRG CNV_CONF_STR CNV_INT_STR CNV_REAL_STR	CNV_STR_CONF CNV_STR_INT CNV_STR_REAL	STR_LEN SUB_STR
系统	ABS ACOS ARRAY_LEN ASIN ATAN2 BYNAME CHR	COS EXP GET_VAR INDEX INV LN ORD	ROUND SET_VAR SIN SQRT TAN TRUNC UNINIT
时间操作	CNV_STR_TIME CNV_TIME_STR	GET_TIME GET_USEC_SUB	GET_USEC_TIM SET_TIME
TPE 程序	AVL_POS_NUM CLOSE_TPE COPY_TPE CREATE_TPE DEL_INST_TPE GET_ATTR_PRG GET_JPOS_TPE	GET_POS_FRM GET_POS_TPE GET_POS_TYP GET_TPE_CMT GET_TPE_PRM OPEN_TPE SELECT_TPE	SET_ATTR_PRG SET_EPOS_TPE SET_JPOS_TPE SET_POS_TPE SET_TPE_CMT SET_TRNS_TPE
翻译	TRANSLATE		

Table 5–2. KAREL内置常规摘要（续）

类别	标识符		
用户界面	ACT_SCREEN ADD_DICT ATT_WINDOW_D ATT_WINDOW_S CHECK_DICT CNC_DYN_DISB CNC_DYN_DISE CNC_DYN_DISI CNC_DYN_DISP CNC_DYN_DISR CNC_DYN_DISS DEF_SCREEN DEF_WINDOW	DET_WINDOW DISCTRL_ALPH DISCTRL_FORM DISCTRL_LIST DISCTRL_PLMN DISCTRL_SBMN DISCTRL_TBL FORCE_SPMENU INI_DYN_DISB INI_DYN_DISE INI_DYN_DISI INI_DYN_DISP INI_DYN_DISR	INI_DYN_DISS INIT_TBL POP_KEY_RD PUSH_KEY_RD READ_DICT READ_DICT_V READ_KB REMOVE_DICT SET_CURSOR SET_LANG WRITE_DICT WRITE_DICT_V
向量	APPROACH	ORIENT	
视觉操作	V_CAM_CALIB V_GET_OFFSET V_GET_PASSFL V_GET_QUEUE	V_INIT_QUEUE V_RALC_QUEUE V_RUN_FIND V_SET_REF	V_START_VTRK V_STOP_VTRK VREG_FND_POS VREG_OFFSET

第6章

条件处理器

目录

第6章 条件处理器 6–1
6.1 条件处理器操作 6–3
6.1.1 全局条件处理程序 6–3
6.2 条件 6–6
6.2.1 Port_Id 条件 6–7
6.2.2 关系条件 6–7
6.2.3 系统和程序事件条件 6–8
6.3 操作 6–11
6.3.1 分配操作 6–11
6.3.2 动作相关操作 6–13
6.3.3 Routine 调用操作 6–13
6.3.4 杂项操作 6–14

KAREL语言的条件处理程序功能允许程序比传统程序控制结构允许的更有效地响应外部条件。

这些条件处理程序（也称为Global条件处理程序）允许在正常程序执行的同时对指定条件进行并行监视，并在出现条件时对相应的操作进行响应。

对于要监视的条件处理程序，它必须先被定义然后被启用。 禁用条件处理程序会将其从正在扫描的组中删除。 清除条件处理程序删除它们的定义。

表6-1列出了条件处理程序可以监视的条件。

表 6–1. 条件

port_id[n]	ERROR[n]
NOT port_id[n]	EVENT[n]
port_id[n]+	ABORT
port_id[n]-	PAUSE
operand = operand	CONTINUE
operand <> operand	SEMAPHORE[n]
operand < operand	
operand <= operand	
operand > operand	
operand >= operand	

表6-2列出了可以采取的操作。

表 6–2. 操作

variable = expression	NOABORT
port_id[n] = expression	NOMESSAGE
STOP	NOPAUSE
CANCEL	ENABLE CONDITION[n]
RESUME	DISABLE CONDITION[n]
HOLD	PULSE DOUT[n] FOR t
UNHOLD	UNPAUSE
routine_name	ABORT
SIGNAL EVENT[n]	CONTINUE

表6-2。 操作（续）

	PAUSE
	SIGNAL SEMAPHORE[n]

6.1 条件处理器操作

表6-3总结了条件处理程序操作。

表6-3。 条件处理程序操作

操作	全局条件处理
Define	CONDITION[n]:<WITH $SCAN_TIME = n>. WHEN conditions DO actions ENDCONDITION
Enable	ENABLE CONDITION[n] (陈述或行动)
Disable	DISABLE CONDITION[n] (声明或行动）或条件满足
Purge	PURGE CONDITION[n] (声明），程序终止

6.1.1 全局条件处理程序

通过在程序的可执行部分中执行CONDITION语句来定义全局条件处理程序。 该定义指定条件/动作对。 以下规则适用于全局条件处理程序。

- 每个全局条件处理程序在整个程序中被一个指定数字（从1到1000）引用。如果之前已经定义了一个具有指定数字的条件处理程序，则它必须在被新程序替换之前进行清除。
- CONDITION 语句的 WHEN 子句中指定了全局条件处理程序的条件/操作对。 条件处理程序的 All WHEN 子句一起启用，禁用和清除。
- 条件列表表示条件处理程序扫描时要监视的条件列表。
- 默认情况下，每个全局条件处理程序都以基于值的速率进行扫描
- $ SCR.$ cond_time。 如果在CONDITION语句中使用''WITH $ SCAN_TIME = n''子句，则大约每隔'n'毫秒扫描一次条件。 扫描间的实际时间间隔如表6-4所示。

表6-4. 全局条件处理程序扫描的间隔

"n"	扫描间隔
n <= $COND_TIME	$COND_TIME
$COND_TIME < n <= (2 * $COND_TIME)	(2 * $COND_TIME)
(2 * $COND_TIME) < n <= (4 * $COND_TIME)	(4 * $COND_TIME)
(4 * $COND_TIME) < n <= (8 * $COND_TIME)	(8 * $COND_TIME)
(8 * $COND_TIME) < n <= (16 * $COND_TIME)	(16 * $COND_TIME)
(16 * $COND_TIME) < n <= (32 * $COND_TIME)	(32 * $COND_TIME)
(32 * $COND_TIME) < n <= (64 * $COND_TIME)	(64 * $COND_TIME)
(64 * $COND_TIME) < n <= (128 * $COND_TIME)	(128 * $COND_TIME)
(128 * $COND_TIME) < n <= (256 * $COND_TIME)	(256 * $COND_TIME)
(256 * $COND_TIME) < n	(512 * $COND_TIME)

- 必须由AND运算符或OR运算符分隔多个条件。 AND和OR的混合是不允许的。
- 如果使用AND，则必须同时满足单个WHEN子句的所有条件，才能触发条件处理程序。
- 如果使用OR，则在任何条件为TRUE时触发操作。
- 动作列表表示当同时满足WHEN子句的相应条件时要采取的动作列表。
- 必须用逗号或新行分隔多个动作。

全局条件处理程序定义显示了定义全局条件处理程序的三个示例。

另请参阅：$ SCR.$ cond_time系统变量，FANUC Robotics软件参考手册

$ SCAN_TIME条件处理程序限定符，FANUC机器人软件参考手册

全局条件处理程序定义

```
CONDITION[1]:     --定义条件处理程序编号 1
1
   WHEN DIN[1] DO DOUT[1] = TRUE       --如果有的话，触发
   WHEN DIN[2] DO DOUT[2] = TRUE        --of the WHEN clauses
   WHEN DIN[3] DO DOUT[3] = TRUE        --满足条件
ENDCONDITION
CONDITION[2]:    --定义条件处理程序编号 2
   WHEN PAUSE DO                    --一个条件触发
     AOUT[speed_out] = 0              --多重行动
     DOUT[pause_light] = TRUE
     ENABLE CONDITION [2]           --启用此条件
```

```
ENDCONDITION                                --再次处理
CONDITION[3]:
   WHEN DIN[1] AND DIN[2] AND DIN[3] DO --multiple
    DOUT[1] = TRUE       --由AND分隔的条件;
    DOUT[2] = TRUE         --这三个条件必须是
    DOUT[3] = TRUE         --同时满足
ENDCONDITION
```

- 您可以在整个程序中根据需要启用，禁用和清除全局条件处理程序。 无论何时触发条件处理程序，它都会自动禁用，除非在操作列表中包含ENABLE操作。 （请参阅全局条件处理程序定义中的条件处理程序2。）
  - ENABLE语句或动作启用指定的条件处理程序。 条件处理程序将在下一次扫描操作期间进行扫描，并将继续扫描，直到它被禁用。

  - DISABLE语句或操作将从扫描的条件处理程序组中删除指定的条件处理程序。 条件处理程序保持定义状态，并可以使用ENABLE语句或操作重新启用。

  - PURGE语句删除指定条件处理程序的定义。
- 如果未定义指定的条件处理程序，则ENABLE，DISABLE和PURGE不起作用。 如果指定的条件处理程序已启用，则ENABLE无效; 如果它已被禁用，DISABLE无效。

使用全局条件处理程序显示启用，禁用和清除全局条件处理程序的示例。

使用全局条件处理程序

```
CONDITION[1]:    --定义条件处理程序编号1
   WHEN line_stop = TRUE DO DOUT[1] = FALSE
 ENDCONDITION
 CONDITION[2]:    --定义条件处理程序编号2
   WHEN line_go = TRUE DO
    DOUT[1] = TRUE, ENABLE  CONDITION [1]
 ENDCONDITION
 ENABLE CONDITION[2]  --条件处理程序2已启用
 . . .
 IF ready THEN line_go = TRUE; ENDIF
 - 如果就绪，则为TRUE条件处理程序2被触发（和--disabled）并启用条件处理程序1。
 - 否则，条件处理程序2未被触发（并且--still已启用），条件处理程序1尚未启用，
 - 接下来的两个语句将不起作用。
 DISABLE CONDITION[1]
 ENABLE CONDITION[2]
```

```
. . .
ENABLE CONDITION[1] --条件处理程序1已启用
. . .
line_stop = TRUE  --触发（和禁用）条件处理程序1
. . .
PURGE CONDITION[2]  --条件处理程序2的定义已删除
ENABLE CONDITION[2] --不再有任何影响
line_go = TRUE      --不再是受监控的情况
```

6.2 条件

在WHEN或UNTIL子句的条件列表中指定一个或多个条件，定义条件/操作对的条件部分。 条件可以

- **States** - 只要state存在，它们仍然满足。 状态的示例是DIN [1]和（VAR1> VAR2）。
- **Events** - 仅在事件发生时才满足。 事件的示例是ERROR [n]，DIN [n] +和PAUSE。

以下规则适用于系统和程序事件条件：

- 启用条件处理程序后，将监视指定的条件。
  - 如果同时满足AND，WHEN或UNTIL子句的所有条件，则触发条件处理程序并执行相应的操作。
  - 如果满足OR，WHEN或UNTIL子句的所有条件，则会触发条件处理程序并执行相应的操作。
- 事件情况很少同时发生。 因此，您不应在单个WHEN或UNTIL子句中的两个事件条件之间使用AND，因为这两个条件不会同时满足。
- 虽然许多条件在形式上与KAREL中的BOOLEAN表达式类似，并且含义相似，但只允许使用本节中列出的表单，而不是一般的BOOLEAN表达式。
- EVAL子句中允许使用表达式。 通过将表达式包含在EVAL子句中，可以在比较条件的右侧使用更一般的表达式：EVAL（表达式）。 但是，在定义条件处理程序时，将评估EVAL子句中的表达式。 他们不会动态评估。
- EVAL子句表达式的值必须是INTEGER，REAL或BOOLEAN。

另见：EVAL条款，附录A.

6.2.1 Port_Id 条件

Port_id 条件用于监视数字端口信号。 Port_id必须是预定义的BOOLEAN端口阵列标识符之一（DIN，DOUT，OPIN，OPOUT，TPIN，TPOUT，RDI，RDO，WDI或WDO）。 n的值指定要监视的端口阵列信号。 表6-5列出了可用的port_id条件。

Table_6–5. Port_Id 条件

条件	满足（TRUE）时
port_id[n]	Digital port n is TRUE. (state)
NOT port_id[n]	Digital port n is FALSE. (state)
port_id[n]+	Digital port n changes from FALSE to TRUE. (event)
port_id[n]-	Digital port n changes from TRUE to FALSE. (event)

- 对于状态条件 **port_id [n]** 和 **NOT port_id [n]**，在每次扫描期间都会测试端口。 如果在扫描期间 **DIN [1]** 为 **TRUE** 且 **DIN [2]** 为 **FALSE**，则满足以下条件：

  ```
  WHEN DIN[1] AND NOT DIN[2] DO . . .
  ```

  请注意，输入信号应保持 ON 或 OFF 最短扫描时间，以确保检测到其状态。

- 对于事件条件 **port_id [n] +**，在启用条件处理程序时测试初始端口值。 每次扫描都会测试信号中指定的变化。 必须在启用条件处理程序时进行更改。
  如果在启用条件处理程序时，**DIN [1]** 自上次扫描后从 **TRUE** 更改为 **FALSE**，则仅满足以下条件。

  ```
  WHEN DIN[1]- DO . . .
  ```

6.2.2 关系条件

关系条件用于测试两个操作数之间的关系。 当指定的关系为TRUE时，它们会满足。 关系条件是状态条件，意味着在每次扫描期间测试关系。 表6-6列出了关系条件。

表 6–6. 关系条件

条件	满足（TRUE）时
operand = operand	指定的关系为 TRUE。 左侧的操作数可以是端口数组元素，引用为 port_id [n] 或变量。 右侧的操作数可以是变量，常量或 EVAL 子句。 （state）
operand < > operand	
operand < operand	
operand < = operand	
operand > operand	
operand > = operand	

以下规则适用于关系条件：

- •两个操作数必须具有相同的数据类型，并且只能是INTEGER，REAL或BOOLEAN类型。 （与其他情况一样，INTEGER常量可用于需要REAL值的地方，并将被视为REAL值。）
- •条件左侧的操作数可以是任何端口数组信号，用户定义的变量，静态变量或可由KAREL程序读取的系统变量。
- •条件右侧的操作数可以是用户定义的变量，静态变量，可由KAREL程序读取的系统变量，任何常量或EVAL子句。 例如：

```
WHEN DIN[1] = ON DO . . .       --port_id和常量
WHEN flag = TRUE DO . . .       --变量和常数
WHEN AIN[1] >= temp DO . . .    --port_id和变量
WHEN flag_1 <> flag_2 DO . . . --variable and variable
WHEN AIN[1] <= EVAL(temp * scale) DO . . .
   --port_id and EVAL clause
WHEN dif > EVAL(max_count - count) DO . . .
   --变量和EVAL条款
```

- **EVAL** 子句允许您在关系条件中包含表达式。 但是，仅在定义条件处理程序时才会对其进行求值。 **EVAL** 子句中的表达式不能包含任何例程调用。

另见：EVAL条款，附录A.

6.2.3 系统和程序事件条件

系统和程序事件条件用于监视系统和程序生成的事件。 仅当启用条件处理程序时发生事件时，才满足指定的条件。

只有在发生指定类型的事件时，才会扫描包含
ERROR，EVENT，PAUSE，ABORT，POWERUP或CONTINUE条件的已启用条件处理程序。 例如，只有在发生错误时才会扫描包含ERROR条件的启用条件处理程序。 表6-7列出了可用的系统和程序事件条件。

表6-7. 系统和程序事件条件

条件	满足（TRUE）WHEN
ERROR [n]	达到由n指定的错误，或者如果n = *，则发生任何错误。（事件）
EVENT[n]	由n指定的事件被发信号通知。（事件）
ABORT	该程序被中止。（事件）
PAUSE	该程序暂停。（事件）
CONTINUE	该程序继续进行。（事件）
POWERUP	该程序继续进行。（事件）
SEMAPHORE[n]	由n指定的信号量的值被传递。

以下规则适用于这些条件：

ERROR 条件

- 通过指定该错误的错误代码，可以使用ERROR条件来监视特定错误的发生。 例如，错误[15018]监视由错误代码15018表示的错误的发生。
  错误代码按以下格式列出：
  ffccc（十进制）

  这里
  ff代表错误的设施代码
  ccc表示指定设施内的错误代码
  例如，15018是MOTN-018，它是“位置不可达”。 设施代码为15，错误代码为018.
  请参阅FANUC机器人错误代码手册以获取错误代码的完整列表。

- ERROR条件还可用于通过指定星号（*）（通配符）代替特定错误代码来监视任何错误的发生。 例如，ERROR [*]监视任何错误的发生。
- 错误条件仅在检测到错误时执行扫描时才满足。 在随后的扫描中不会记住该错误。

EVENT 条件

- EVENT条件监视指定程序事件的发生。 程序中的SIGNAL语句或操作表示发生了事件。
- EVENT条件仅适用于事件发出信号时执行的扫描。 在随后的扫描中不会记住该事件。

ABORT 条件

- ABORT条件监视程序执行的中止。 如果发生ABORT，则执行相应的动作。 但是，如果其中一个操作是例程调用，则不会执行例程，因为程序执行已中止。
  如果在条件处理程序中使用ABORT条件，即使程序已中止，也将执行除例程调用之外的所有操作。

PAUSE 条件

- 暂停状态监视暂停程序执行。 如果其中一个相应的操作是例程调用，则还需要指定NOPAUSE或UNPAUSE操作。

CONTINUE 条件

- CONTINUE条件监视程序执行的恢复。 如果程序执行暂停，则CONTINUE操作，KCL> CONTINUE命令，来自操作面板的CYCLE START或示教器FWD键将继续执行程序并满足CONTINUE条件。

POWERUP 条件

- POWERUP状态监视电源故障恢复后程序执行的恢复。 在恢复程序之前，控制器必须能够从电源故障中成功恢复。

SEMAPHORE 条件

- SEMAPHORE条件监视指定的信号量。 CLEAR_SEMA内置可用于将信号量值设置为0. POST_SEMA内置或SIGNAL SEMAPHORE操作可用于递增信号量值并满足SEMAPHORE条件。

另见：附录A中：

ABORT 条件

CONTINUE 条件

ERROR 条件

EVENT 条件

PAUSE 条件

POWERUP 条件

SEMAPHORE 条件

针对错误代码的特定应用FANUC机器人设置和操作手册。 FANUC Robotics错误代码手册。

6.3 操作

操作在WHEN子句的操作列表中指定。 行动可以

- 与程序并行执行的特定KAREL操作
- 例程调用，将中断程序执行

当满足条件处理程序的条件时，将触发条件处理程序。 除了例行调用之外，对应于满足条件的动作按它们出现在条件处理程序定义中的顺序执行。 在执行所有其他操作后执行例程。

请注意，尽管许多操作在形式上与KAREL语句类似，并且效果类似于相应的 KAREL 语句，但操作不是可执行语句。 只允许使用本节中指明的表格。

另见：行动和声明，附录A.

6.3.1 分配操作

可用的分配操作在表6-8中给出。

表 6–8. 分配操作

操作	结果
variable = expression	表达式的值分配给变量。 表达式可以是变量，常量，端口数组元素或EVAL子句。
port_id[n] = expression	表达式的值分配给n引用的端口数组元素。 表达式可以是变量，常量或EVAL子句。

以下规则适用于分配操作：

- 赋值操作，“ariable = expression”和“ort_id [n] =表达式”可用于为变量和端口数组元素赋值。
  “变量必须是用户定义的变量，静态变量或没有最小/最大范围的系统变量，并且可以由KAREL程序写入。
  - 如果在左侧，端口数组必须是可以由KAREL程序设置的输出端口数组.-表达式可以是用户定义的变量，也可以是静态变量。 系统变量，可由KAREL程序，任何常量或EVAL子句读取。

- 如果变量位于赋值的左侧，则表达式也可以是元素。 但是，您无法直接将端口数组元素分配给端口数组元素。 例如，显示的第一个分配无效，但接下来的两个分配是有效的：

```
DOUT[1] = DOUT[2]    --无效的行动
   port_var = DOUT[2]  --有效的操作，其中port_var是一个变量
   DOUT[1] = port_var  --另一个有效的动作，如果执行的话
                       - 在port_var = DOUT [2]之后，将生效
                       - 将DOUT [2]分配给DOUT [1]
```

- 如果表达式是变量，则它必须是全局变量。 使用的值是采取操作时变量的当前值，而不是在定义条件处理程序时。 如果表达式是EVAL子句，则在定义条件处理程序时评估它，并在执行操作时分配该值。
- 赋值操作的两端必须具有相同的数据类型。 在赋值的右侧允许使用INTEGER或EVAL子句，左侧是INTEGER，REAL或BOOLEAN。

6.3.2 动作相关操作

与运动相关的操作会影响当前运动并可能影响后续运动。 它们在表6-9中给出。

表 6–9. 动作相关操作

ACTION	RESULT
STOP	Current motion is stopped.
RESUME	The last stopped motion is resumed.
CANCEL	Current motion is canceled.
HOLD	Current motion is held. Subsequent motions are not started.
UNHOLD	Held motion is released.

以下规则适用于与动作相关的操作：

- 如果发出STOP，则将当前运动和变化的动作按在刚刚运动的运动堆栈上进行。 如果没有动作正在进行，则在堆栈上按下空条目。
- 如果发出RESUME，则停止运动堆栈上设置的最新停止运动将排队等待执行。
- 如果发出取消，则取消当前正在进行的动作。 在当前议案背后排队等同组的任何动议也会被取消。 如果没有动作正在进行，则该动作无效。
- 如果发出保持，则保持当前动作，并防止后续动作开始。 UNHOLD的行动发布动议。

6.3.3 Routine 调用操作

例程调用操作或中断例程由。指定

<WITH $ PRIORITY = n> routine_name

以下限制适用于例行调用操作或中断例程：

- 中断例程不能有参数，必须是过程（不是函数）。
- 如果被中断的程序使用READ语句，则中断例程无法从同一文件变量中读取。 如果正在读取被中断的程序并且中断例程尝试从同一文件变量读取，则程序将被中止。
- 启动中断例程时，中断的KAREL程序将暂停，直到例程返回。

- 中断例程（如KAREL程序）可能会被其他例程中断。 最大中断深度仅受堆栈内存大小的限制。
- 例程以它们出现在条件处理程序定义中的顺序启动，但由于它们相互中断，它们实际上将以相反的顺序执行。
- 可以对中断进行优先级排序，以便某些中断例程不会被其他中断例程中断。 $ PRIORITY条件处理程序限定符可用于设置指示的例程操作的执行优先级。 $ PRIORITY值必须为0-255，其中较低的值表示较低的优先级。 如果在运行优先级较高的例程时调用低优先级例程，则只有在优先级较高的例程完成时才会执行该例程。 如果未指定$ PRIORITY，则例程的优先级将默认为$ PRIORITY系统变量的当前值。

  另请参阅：WITH子句，附录A，''KAREL语言字母说明， “有关$ PRIORITY的更多信息

6.3.4 杂项操作

表6-10介绍了其他允许的操作。

表6-10. 杂项操作

动作	结果
SIGNAL EVENT[n]	由n指定的事件被发信号通知。
NOMESSAGE	否则将生成的错误消息不会显示或记录。
NOPAUSE	如果程序暂停或阻止暂停，则程序执行恢复。
NOABORT	如果程序被中止，或者被阻止中止，则程序执行恢复。
ABORT	程序执行被中止。
CONTINUE	程序继续执行。
PAUSE	程序执行暂停。
SIGNAL SEMAPHORE[n]	指定的信号量表示。
ENABLE CONDITION[n]	条件处理程序n已启用。
DISABLE CONDITION[n]	条件处理程序n被禁用。
PULSE DOUT[n] FOR t	指定的端口n为时间间隔t(以毫秒为单位)的脉冲。
UNPAUSE	如果将routine_name指定为操作，但暂停程序执行，则仅在例程的持续时间内恢复执行，然后再次暂停。

另请参阅：附录A，了解有关每个杂项操作的更多信息。

第7章

文件输入/输出操作

目录

第7章 文件输入/输出操作 7–1
7.1 概述 7–3
7.2 文件变量 7–3
7.3 OPEN FILE 声明 7–4
7.3.1 设置文件和端口属性 7–5
7.3.2 文件字符串 7–10
7.3.3 用法字符串 7–11
7.4 CLOSE FILE 声明 7–14
7.5 READ 声明 7–14
7.6 WRITE 声明 7–16
7.7 输入/输出缓冲器 7–17
7.8 映柏，ASCII - 逾全/逾击楚弓 7–18
7.8.1 INTEGER 数据项格式 7–19
7.8.2 REAL 数据项格式 7–22
7.8.3 BOOLEAN 数据项格式 7–25
7.8.4 STRING 数据项格式 7–27
7.8.5 VECTOR 数据项格式 7–31
7.8.6 Positional 明摩麻楚弓 7–32
7.9 于都刷逾全/逾击楚弓 7–34
7.9.1 INTEGER 明摩麻楚弓 7–35
7.9.2 REAL 明摩麻楚弓 7–36
7.9.3 BOOLEAN 明摩麻楚弓 7–36
7.9.4 STRING 明摩麻楚弓 7–36
7.9.5 VECTOR 明摩麻楚弓 7–37
7.9.6 POSITION 明摩麻楚弓 7–37
7.9.7 XYZWPR 明摩麻楚弓 7–37
7.9.8 XYZWPREXT 明摩麻楚弓 7–38
7.9.9 JOINTPOS 明摩麻楚弓 7–38

7.10 USER INTERFACE TIPS 7–38
7.10.1 USER Menu on the Teach Pendant 7–38
7.10.2 USER Menu on the CRT/KB 7–40

7.1 概述

KAREL语言工具允许您执行以下串行输入/输出（I / O）操作：

- 使用 **OPEN FILE** 语句打开数据文件和串行通信端口
- 使用 **CLOSE FILE** 语句关闭数据文件和串行通信端口
- 使用 **READ** 语句从文件，通信端口和用户界面设备读取
- 使用 **WRITE** 语句写入文件，通信端口和用户界面设备
- 取消读取或写入操作

文件变量用于指示要在其上执行串行I / O操作的文件，通信端口或设备。

缓冲区用于保存尚未传输的数据。 READ和WRITE语句中的数据项及其格式说明符的使用取决于数据是文本（ASCII）还是二进制，以及数据类型。

7.2 文件变量

KAREL程序可以对以下内容执行串行I / O操作：

- KAREL文件系统中的数据文件
- 与KAREL控制器上的连接器关联的串行通信端口
- 用户界面设备，包括CRT / KB和示教器

文件变量用于指示要在其上执行特定串行I / O操作的文件，通信端口或设备。

表7-1列出了用户界面设备的预定义文件变量。 这些文件变量已经打开，可以在READ或WRITE语句中使用。

表7-1. 预定义的文件变量

标识符	设备	操作
TPFUNC*	Teach pendant function key line	Both
TPDISPLAY*	Teach pendant KAREL display	Both
TPPROMPT*	Teach pendant prompt line	Both
TPERROR	Teach pendant message line	Write
TPSTATUS*	Teach pendant status line	Write

表7-1. 预定义文件变量（续）

CRTFUNC*	CRT/KB function key line	Both
INPUT	CRT/KB keyboard	Read
OUTPUT*	CRT/KB KAREL screen	Write
CRTPROMPT*	CRT/KB prompt line	Both
CRTERROR	CRT/KB message line	Write
CRTSTATUS*	CRT/KB status line	Write

*仅在示教器或CRT位于用户菜单中时显示。

可以在KAREL语句中将文件变量指定为FILE变量。 在KAREL程序中使用FILE显示了一个声明FILE变量和在程序的可执行部分中使用FILE的示例。

在KAREL程序中使用FILE

```
PROGRAM lun_prog
  VAR
    curnt_file : FILE
  ROUTINE input_data(file_spec:FILE) FROM util_prog
  BEGIN
    OPEN FILE curnt_file ('RW','text.dt')  --变量FILE
    input_data(curnt_file)  --文件变量参数
     WRITE TPERROR ('Error has occurred')
  END lun_prog
```

只要单个任务正在执行程序，就允许在程序之间共享FILE变量。 不允许在任务之间共享文件变量。

7.3 OPEN FILE 声明

OPEN FILE语句将文件变量与特定数据文件或通信端口相关联。

在文件关闭之前，关联仍然有效，可以通过CLOSE FILE语句显式关闭，也可以在程序执行终止或中止时隐式关闭。

OPEN FILE语句指定文件的使用方式（用法字符串），以及使用的文件或端口（文件字符串）。

7.3.1 设置文件和端口属性

属性指定串行端口或 KAREL FILE 变量的操作细节。 SET_PORT_ATR 和 SET_FILE_ATR 内置函数用于设置这些属性。必须在打开FILE之前调用 SET_FILE_ATR。 可以在打开使用串行端口的FILE之前或之后调用 SET_PORT_ATR。

表7-2列出了每种属性类型，其功能以及该属性是否适用于示教器和 CRT / KB 设备，串行端口，数据文件或管道。 有关更多信息，请参阅附录A.

Table 7–2. 预定义的属性类型

属性类型	功能	SET_PORT_ATR OR SET_FILE_ATR	TP/ CRT	串口	DATA FILES	PIPES	SOCKET MESSAGING
ATR_BAUD	Baud rate	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_DBITS	Data length	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_EOL	End of line	SET_FILE_ATR	not used	valid	not used	valid	valid
ATR_FIELD	Field	SET_FILE_ATR	valid	valid	valid	valid	valid
ATR_IA	Interactively write	SET_FILE_ATR	valid	valid	valid	valid	valid
ATR_MODEM	Modem line	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_PARITY	Parity	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_PASSALL	Passall	SET_FILE_ATR	valid	valid	not used	valid	valid
ATR_READAHD	Read ahead buffer	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_REVERSE	Reverse transfer	SET_FILE_ATR	not used	valid	valid	valid	valid
ATR_SBITS	Stop bits	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_TIMEOUT	Timeout	SET_FILE_ATR	valid	valid	not used	valid	valid
ATR_UF	Unformatted transfer	SET_FILE_ATR	not used	valid	valid	valid	valid

表7-2. 预定义的属性类型（续）

属性类型	功能	SET_PORT_ATR OR SET_FILE_ATR	TP/ CRT	SERIAL PORTS	DATA FILES	PIPES	SOCKET MESSAGING
ATR_XONOFF	XON/XOFF	SET_PORT_ATR	not used	valid	not used	not used	not used
ATR_PIPOVADV	Pipe Overflow	SET_FILE_ATR	not used	not used	not used	valid	valid
ATR_PIPWAIT	Wait for data	SET_FILE_ATR	not used	not used	not used	valid	valid

表7-3包含每个属性的详细说明。

表7-3. 属性值

属性类型	描述	有效设备	使用模式	有效值	默认值
ATR_BAUD Baud rate	串口的波特率可以更改为其中一个有效的属性值。	PORT	Read/ Write	BAUD_9600: 9600 baud BAUD_4800: 4800 baud BAUD_2400: 2400 baud BAUD_1200: 1200 baud	BAUD_9600
ATR_DBITS Data length	如果指定，则串行端口的数据长度将更改为指定的属性值。	PORT	Read/ Write	DBITS_5: 5 bits DBITS_6: 6 bits DBITS_7: 7 bits DBITS_8: 8 bits	DBITS_8
ATR_EOL End of line	如果指定，则将串行端口更改为在指定的属性值时终止读取。 有关有效属性值的说明，请参阅附录D.	PORT	Read/ Write	Any ASCII character code	13 (carriage return)
ATR_FIELD Field	如果指定，则读取的数据量取决于READ语句中的格式说明符或正在读取的数据类型的默认值。 如果未指定，则读取数据，直到出现终止符（EOL）。	TP/CRT, PORT, FILE	Read only	Ignored	Read data until terminator character (EOL) appears

表7-3. 属性值（续）

属性类型	描述	有效设备	使用模式	有效值	默认值
ATR_IA Interactively write	如果指定，则在对缓冲区的每个写操作完成时输出缓冲区的内容。（交互式） 如果未指定，则仅在缓冲区已满或指定CR时才输出缓冲区的内容。 输出缓冲区的大小为256字节。（不互动）	TP/CRT, PORT, FILE	Write only	Ignored	TP/CRT is interactive, PORT, FILE are not interactive
ATR_MODEM Modem line	有关信息，请参阅后面的“调制解调器线路”部分。				
ATR_PARITY Parity	串行端口的奇偶校验可以更改为有效的属性值。	PORT	Read/ Write	PARITY_NONE: No parity PARITY_ODD: Odd parity PARITY_EVEN: Even parity	PARITY_NONE
ATR_PASSALL Passall	如果指定，则读取输入而不进行解释或交易。 由于终结符（EOL）不会终止读取，因此字段属性会自动采用“字段”选项。	TP/CRT, PORT	Read only	Ignored	只读可显示键，直到按下回车键
ATR_PIPOVADV	发生溢出时配置读取的行为。 默认情况下，行为是在发生溢出时发出文件结束信号（EOF）。	PIPE	Read	该值必须介于0和管道中的总字节数之间。 该值将向上舍入到最近的二进制记录。	value参数是OVF_EOF（设置默认行为）或发生溢出时要前进的字节数。

表7-3. 属性值（续）

属性类型	描述	有效设备	使用模式	有效值	默认值
ATR_PIPWAIT	读操作等待数据到达管道。	PIPE	Read	WAIT_USED or WAIT_NOTUSED	默认为快照，这意味着系统在读取管道中的所有数据时返回EOF。
ATR_READAHD Read Ahead Buffer	属性值以128字节为单位指定，并分配指定大小的预读缓冲区。	PORT	Read/ Write	any positive integer 1=128 bytes 2=256 bytes 0=disable bytes	1 (128 byte buffer)
ATR_REVERSE Reverse transfer	字节将被交换。	PORT, FILE	Read/ Write	Ignored	Not reverse transfer
ATR_SBITS Stop bits	这指定了串行端口的停止位数。	PORT	Read/ Write	SBITS_1:1 bit SBITS_15: 1.5 bits SBITS_2:2 bits	SBITS_1
ATR_TIMEOUT Timeout	如果指定，如果读取的时间超过指定的属性值，则IO_STATUS将返回错误。	TP/CRT, PORT	Read only	任何整数值（单位为毫秒）	0 (external)
ATR_UF Unformatted transfer	如果指定，则执行二进制传输。 对于读操作，终结符（EOL）不会终止读取，因此会自动采用“字段”选项。 如果未指定，则执行ASCII传输。	PORT, FILE	Read/ Write	Ignored	ASCII transfer
ATR_XONOFF XON/XOFF	如果指定，则串行端口的XON / XOFF将更改为指定的属性值。	PORT	Read/ Write	XF_NOT_USED: Not used XF_USED: Used	XF_USED

Modem line

Valid device : PORT

Usage mode : Read/Write

Default value : MD_NOT_USED: DSR, DTR, and RTS not used

Valid attribute values : MD_NOT_USED: DSR, DTR, and RTS not used

MD_USE_DSR: DSR used

MD_NOUSE_DSR: DSR not used

MD_USE_DTR: DTR used

MD_NOUSE_DTR: DTR not used

MD_USE_RTS: RTS used

MD_NOUSE_RTS: RTS not used

- 此属性控制调制解调器线路的操作。 该控件基于以下二进制掩码，其中标志位用于指示您正在更改的位值。

RTS value	DSR value	DTR value	RTS flag	DSR flag	DTR flag

  - RTS（请求发送）和DTR（数据终端就绪）都是输出。
  - DSR（数据集就绪）是输入。

- 通过执行以下操作设置调制解调器线属性。
  - 表示使用RTS（HIGH / ON）： status = SET_PORT_ATR（port_name，ATR_MODEM，MD_USE_RTS）
  - 表示未使用RTS（LOW / OFF）： status = SET_PORT_ATR（port_name，
- ATR_MODEM，MD_NOUSE_RTS）
  - 表示使用RTS（HIGH / ON）且未使用DTR（LOW / OFF）： status = SET_PORT_ATR（port_name，ATR_MODEM，MD_USE_RTS或MD_NOUSE_DTR）
- 以下示例演示了如何使用内置GET_PORT_ATR的返回属性值。
- status = GET_PORT_ATR（port，ATR_MODEM，atr_value）
  - 确定是否使用DTR：

```
IF ((atr_value AND MD_USE_DTR) = MD_USE_DTR) THEN
```

```
            write ('DTR is in use',cr)
        ENDIF
```

— 确定是否未使用DTR（LOW / OFF）

```
IF (atr_value AND MD_USE_DTR) = MD_NOUSE_DTR) THEN
            write ('DTR is not in use', cr)
        ENDIF
```

有关 GET_PORT_ATR 内置的更多信息，请参阅附录A.

7.3.2 文件字符串

OPEN FILE语句中的文件字符串指定数据文件名和类型，或通信端口。

- OPEN FILE语句将文件字符串指定的数据文件或端口与文件变量相关联。例如，OPEN FILE file_var（'RO'，'data_file.dt'）将名为'data_file.dt'的数据文件与文件file_var相关联。
- 如果文件字符串用单引号括起来，则将其视为文字。否则，它被视为STRING变量或常量标识符。
- 指定数据文件时，必须同时包含文件名和有效的KAREL文件类型（任何1,2或3个字符的文件扩展名）。
- 以下STRING值可用于将文件变量与KAREL控制器上的串行通信端口相关联。默认值为：
  - 'P2：' - 操作面板外侧的调试控制台连接器
  - 'P3：' - 主CPU板上的RS-232-C，JD17连接器（CRT / KB）
  - 'P4：' - 主CPU板上的RS-422，JD17连接器
  - 'KB：tp kb' - 从示教器上的数字键盘输入。 TPDISPLAY或TPPROMPT通常使用，因此不需要OPEN FILE。
  - 'KB：cr kb' - 来自CRT / KB的输入。通常使用INPUT或CRTPROMPT，因此OPEN不需要文件。
  - 'WD：window_name' - 写入窗口。
  - 'WD：window_name </ keyboard_name>'，其中keyboard_name为'tpkb'或'crkb' - 写入指定的窗口。输入来自TP键盘（tpkb）或CRT键盘（crkb）。输入将在指定的窗口中回显。

另请参阅：第9章文件系统，以获取文件名和文件类型的说明。

7.3.3 用法字符串

OPEN FILE 语句中的用法字符串表示文件的使用方式。

- 它由一个用法说明符组成。
- 它仅适用于OPEN FILE语句指定的文件，对其他文件没有影响。
- 如果表示为文字，则必须用单引号括起来。
- 它可以表示为变量或常量。

表7-4列出了每个使用说明符，其功能以及预期的设备或端口。

- ''TP / CRT''表示示教器和CRT / KB。
- ''Ports''表示串行端口。
- ''Files''表示数据文件。
- ''Pipes''表示管道设备。
- '' Valid''表示允许使用。
- ''No use''表示允许使用，可能会有不可预知的副作用。

表7-4. 使用说明

说明符	功能	TP/CRT	端口	文件	管道
RO	- 仅允许读取操作 - 将文件位置设置为文件的开头 - 文件必须已存在	valid	valid	valid	valid

Table 7–4. Usage Specifiers (Cont'd)

RW	- 重写文件中的现有数据，删除现有数据 - 允许读写操作 - 将文件位置设置为文件的开头 - 如果文件不存在，将创建文件	valid	valid	valid No use on FRx:	valid
AP	- 附加到现有数据的末尾 - 允许读写（第一次操作必须是写入。） - 将文件位置设置为文件结尾 - 如果文件不存在，将创建文件	no use	valid	valid -RAM disk* no use on FRx:	valid
UD	- 从现有数据的开头更新。（要写入的字符数必须等于要替换的字符数。） - 使用新数据覆盖现有数据 - 允许读写 - 将文件位置设置为现有文件的开头	no use	valid	valid -RAM disk* no use on FRx:	no use

* AP和UD说明符只能与RAM磁盘上的未压缩文件一起使用。 有关RAM磁盘和管道设备的更多信息，请参阅第9章文件系统。

文件字符串示例显示了一个程序，其中包含OPEN FILE语句中各种文件字符串的示例。 包含CONST和VAR部分以说明如何声明文件和端口字符串。

文件字符串示例

```
PROGRAM open_luns
  CONST
    part_file_c ='parts.dt' --数据文件STRING常量
    comm_port = 'P3:'       --端口STRING常量
  VAR
    file_var1 : FILE
    file_var2 : FILE
    file_var3 : FILE
    file_var4 : FILE
    file_var5 : FILE
    file_var12 : FILE
    temp_file : STRING[19]
     --字符串大小为19，可容纳4个字符的设备名称，
     --12个字符文件名，句点和2个字符，
     --文件类型。
     port_var : STRING[3]
  BEGIN
     --文字文件名和类型
     OPEN FILE file_var1 ('RO','log_file.dt')
     --常量指定parts.dt
     OPEN FILE file_var2 ('RW', part_file_c)
     --指定new_file_dt的变量
     temp_file = 'RD:new_file.dt'
     OPEN FILE file_var3 ('AP', temp_file)
     --文字通信端口
     OPEN FILE file_var4 ('RW', 'P2:')
     --常量指定C0:
     OPEN FILE file_var5 ('RW', comm_port)
     --变量指定C3:
     port_var = 'C3:'
     OPEN FILE file_var12 ('RW', port_var)
END open_luns
```

另请参阅：第9章文件系统，有关可用存储设备的更多信息，请参见第14章输入/输出系统，有关C0：和C3：端口的更多信息

7.4 CLOSE FILE 声明

CLOSE FILE语句用于中断指定文件变量与其数据文件或通信端口之间的关联。 它实现了两个目标：

- 任何缓冲的数据都将写入文件或端口。
- 文件变量被释放以供其他用途。

CLOSE FILE示例显示了一个程序，其中包含在FOR循环中使用CLOSE FILE语句的示例，其中打开，读取并关闭多个文件。 每个文件使用相同的文件变量。

CLOSE FILE 示例

```
PROGRAM read_files
   VAR
     file_var    : FILE
     file_names  : ARRAY[10] OF STRING[15]
     loop_count  : INTEGER
     loop_file   : STRING[15]
ROUTINE read_ops(file_spec:FILE) FROM util_prog
--执行一些读操作
ROUTINE get_names(names:ARRAY OF STRING) FROM util_prog
--获取文件名和类型
BEGIN
   get_names(file_names)
   FOR loop_count = 1 TO 10 DO
     loop_file = file_names[loop_count]
     OPEN FILE file_var ('RO', loop_file)
     read_ops(file_var) --调用例程进行读操作
     CLOSE FILE file_var
   ENDFOR END read_files
```

另见：CLOSEFILEStatement，附录A.

IO_STATUS内置函数，附录A，用于描述错误。

7.5 READ 声明

READ语句用于从指示的设备读取一个或多个指定的数据项。 数据项作为READ语句的一部分列出。 以下规则适用于READ语句：

- 必须使用OPEN FILE语句将文件变量与语句中打开的文件相关联，然后才能执行任何读取操作，除非使用其中一个预定义文件（请参阅表7-1）。
- 如果READ语句中省略了文件变量，则TPDISPLAY用作默认值。
- 使用%CRTDEVICE指令将默认值更改为INPUT（CRT输入窗口）。
- 格式说明符可用于控制为每个数据项读取的数据量。格式说明符的效果取决于正在读取的项的数据类型以及数据是文本（ASCII）还是二进制（未格式化）形式。
- 执行READ语句时（对于ASCII文件），将从下一个非空白输入字符开始读取数据，并以除STRING之外的所有输入类型的下一个空白，行尾或文件结尾之前的最后一个字符结束。
- 对于STRING值，输入字段以下一个字符开头，并继续到文件的行或结尾。如果从非字符串字段后面的同一行读取STRING，则STRING中包含任何分隔空格。
- 必须逐个元素地读取ARRAY变量;它们不能以未订阅的形式阅读。通常，它们在FOR循环中使用READ语句读取。
- PATH变量可以在READ语句中指定如下，其中''path_name''是PATH变量而''n''和'm''是PATH节点索引：

  - path_name：指定要读取整个路径，从头开始并包括所有节点及其相关数据。标题由路径长度和写入PATH时生效的相关数据描述组成。

- path_name [0]：指定仅读取标头。 路径头由路径长度和写入PATH时生效的相关数据描述组成。 删除或创建节点以使路径长度正确，并且所有新节点都设置为未初始化。
- path_name [n]：指定将数据从当前文件位置读入node [n]。
n的值必须在0到PATH长度的范围内。
- path_name [n .. m]：指定将数据读入节点n到m。 n的值必须在0到PATH长度的范围内，并且可以小于，等于或大于m的值。 m的值必须在1到PATH长度的范围内。

如果在读取节点n（其中n大于0）时发生错误，则按如下方式处理：
如果n>原始路径长度（在读取操作之前），则从n到新路径长度的节点被设置为未初始化。
如果n <=原始路径长度，则从n到原始路径长度的节点保持与读取操作之前一样，并且任何新节点（大于原始路径长度）都设置为未初始化。

- 如果从PATH读取的相关数据描述与当前用户相关数据不一致，则终止读取操作，并且路径将保持与读取操作之前一样。 如果发生这种情况，IO_STATUS内置函数将返回错误。

- 必须以二进制（无格式）形式读取PATH数据。

READ语句示例显示了使用各种文件变量和数据列表的READ语句的几个示例。

READ 语句示例

```
READ (next_part_no)     --使用默认的TPDISPLAY
  OPEN FILE file_var ('RO','data_file.dt')
  READ file_var (color, style, option)
  READ host_line (color, style, option, CR)
  FOR i = 1 TO array_size DO
    READ data (data_array[i])
  ENDFOR
```

如果在输入期间发生任何错误，则除非文件变量对窗口设备开放，否则正在读取的变量以及数据列表中直到CR的所有后续变量都是未初始化的。

如果从窗口设备读取，则会显示一条错误消息，指示错误的data_item，并提示您输入无效data_item的替换项并重新输入所有后续项目。

内置函数IO_STATUS可用于确定aADAD操作的成功或失败（以及失败的原因）。

另见：READ声明，附录A.

IO_STATUS内置函数，附录A中的I / O错误消息列表%CRTDEVICE转换器指令，附录A.

7.6 WRITE 声明

WRITE语句用于将一个或多个指定的数据项写入指示的设备。 数据项作为WRITE语句的一部分列出。 以下规则适用于WRITE声明：

- 必须使用OPEN FILE语句将文件变量与语句中打开的文件相关联，然后才能执行任何写入操作，除非使用其中一个预定义文件（请参阅表7-1）。
- 如果WRITE语句中省略了文件变量，则TPDISPLAY用作默认值。
- 使用%CRTDEVICE指令将默认值更改为OUTPUT（CRT输出窗口）。

- 格式说明符可用于控制为每个data_item编写的数据格式。 格式说明符的效果取决于正在写入的项的数据类型以及数据是以文本（ASCII）还是二进制（未格式化）形式。
- ARRAY变量必须逐个元素写入; 它们不能以未订阅的形式书写。 通常，它们是在FOR循环中使用WRITE语句编写的。
- PATH变量可以在WRITE语句中指定如下，其中''path_name''是PATH变量而''n''和''m''是PATH节点索引：

  - path_name：指定要写入整个路径，从提供路径长度和关联数据表的标头开始，然后是所有节点，包括其关联数据。

  - path_name [0]：指定仅写入标头。 路径头由路径长度和关联数据表的副本组成。

  - path_name [n]：指定要写入node [n]。

  - path_name [n .. m]：指定要写入节点n到m。 n的值必须在0到PATH长度的范围内，并且可以小于，等于或大于m的值。 m的值必须在1到PATH长度的范围内。
- PATH数据必须以二进制（无格式）形式写入。

WRITE语句示例显示了使用各种文件变量和数据列表的WRITE语句的几个示例。

WRITE 语句示例

```
WRITE TPPROMPT('Press T.P. key "GO" when ready')
  WRITE TPFUNC ('  GO     RECD    QUIT    BACK1    FWD-1')
  WRITE log_file (part_no:5, good_count:5, bad_count:5, operator:3,
CR)
  WRITE ('This is line 1', CR, 'This is line 2', CR)
  --使用默认的 TPDISPLAY
  FOR i = 1 TO array_size DO
    WRITE data (data_array[i])
  ENDFOR
```

另见：WRITE声明，附录A.

IO_STATUS内置函数，附录A.

7.7 输入/输出缓冲器

RAM区域（称为缓冲区）用于保存在读取或写入操作期间尚未传输的最多256个字节的数据。

READ和WRITE语句使用缓冲区如下：

- 在执行READ语句期间，如果从文件中读取的数据多于READ语句所需的数据，则剩余数据将保留在缓冲区中以供后续读取操作使用。 例如，如果在键盘输入行中输入的数据多于满足READ语句所需的数据，则额外数据将保存在缓冲区中。
- 如果对非交互文件执行WRITE语句且最后一个数据项不是CR，则数据将保留在缓冲区中，直到后续WRITE指定CR或填充缓冲区为止。
- 可在单个READ或WRITE语句中处理的总数据限制为127个字节。

7.8 文本（ASCII）输入/输出楚弓

本节介绍用于读取和写入每种数据类型的ASCII（格式）文本的格式说明符。以下规则适用于格式化数据类型：

- 对于文本文件，READ和WRITE语句中的数据项可以是简单类型（INTEGER，REAL，BOOLEAN和STRING）。
- 无法从文本文件中读取Positional和VECTOR变量，但可以在WRITE语句中使用。
- ARRAY变量无法以未订阅的形式读取或写入。 ARRAY的元素以与ARRAY的数据类型对应的格式读取或写入。
- 无法读取或写入PATH变量。
- 某些格式和数据组合的读取方式与写入时不同，如果使用相同格式读取则无效。

对于READ或WRITE语句中的每个数据项，可以使用零个，一个或两个格式说明符来控制读取或写入的数据量。每个格式说明符（表示为INTEGER文字）前面都有双冒号（：:）。表7-5总结了可以与READ语句中的数据项一起使用的输入格式说明符。第7.8.1节至第7.8.6节介绍了每种数据类型的默认格式以及可影响每种数据类型的格式说明符。

表7-5. 文本（ASCII）输入格式说明符

数据类型	第1种格式规范	第2种格式规范
INTEGER	读取的字符总数	数字基数在2 - 16范围内
REAL	读取的字符总数	忽略

表7-5. 文本（ASCII）输入格式说明符（续）

数据类型	第1种格式规范	第2种格式规范
BOOLEAN	读取的字符总数	忽略
STRING	读取的字符总数	0 - 不带引号的STRING 2 - 引用STRING

表7-6总结了可与WRITE语句中的数据项一起使用的输出格式说明符。 第7.8.1节到第7.8.6节介绍了每种数据类型的默认格式和可影响每种数据类型的格式说明符。

表7-6. 文本（ASCII）输出格式说明符

数据类型	第1种格式规范	第2种格式规范
INTEGER	写入的字符总数	数字基数在2-16范围内
REAL	写入的字符总数	小数点右边的位数 如果否定，则使用科学记数法
BOOLEAN	写入的字符总数	0 - 左对齐 1 - 右对齐
STRING	写入的字符总数	0 - 左对齐 1 - 右对齐 2 - 在引号中左对齐（头空格） 3 - 右对齐n引号（头空白）
VECTOR	对每个组件使用REAL格式	对每个组件使用REAL格式
POSITION	对每个组件使用REAL格式	对每个组件使用REAL格式
XYZWPR	对每个组件使用REAL格式	对每个组件使用REAL格式
XYZWPREXT	对每个组件使用REAL格式	对每个组件使用REAL格式
JOINTPOSn	对每个组件使用REAL格式	对每个组件使用REAL格式

7.8.1 INTEGER 明摩麻楚弓

READ语句中的INTEGER数据项按如下方式处理：

默认值：读作十进制（基数为10）INTEGER，从输入行上的下一个非空白字符开始，一直持续到遇到空白或行尾。 如果读取的字符不形成有效的INTEGER，则读取操作将失败。

第一个格式说明符：表示要读取的字符总数。 输入字段必须完全在当前输入行上，并且可以包括前导空格，但不包括尾随空格。

第二个格式说明符：表示用于输入的数字基数，必须在2（二进制）到16（十六进制）的范围内。

对于大于10的基数，字母A，B，C，D，E和F分别用作具有值10,11,12,13,14和15的数字的输入。 接受小写字母。

表7-7列出了INTEGER输入数据项及其格式说明符的示例。 输入数据和INTEGER数据项的结果值包含在表中。 （符号[eol]表示行尾。）

表7-7. INTEGER输入数据项的示例

数据项	输入数据	结果
int_var	-2[eol]	int_var = -2
int_var	20 30 ...	int_var = 20
int_var::3	10000	int_var = 100
int_var::5::2	10101 (base 2 input)	int_var = 21 (base 10 value)
int_var	1.00	format error (invalid INTEGER)
int_var::5	100[eol]	format error (too few digits)

WRITE语句中的INTEGER数据项的格式如下：

默认值：使用所需的位数和一个前导空白写成十进制（基数10）INTEGER。如果INTEGER为负值，则在数字前面加一个减号。

第一个格式说明符：表示要写入的字符总数，包括空格和减号。如果格式说明符大于数据所需的格式说明符，则会添加前导空格。如果它小于要求，则根据需要扩展该字段。

对于文件，说明符必须在1到127的范围内，对于其他输出设备，说明符必须在1到126的范围内。

第二个格式说明符：表示用于输出的数字基数，必须在2（二进制）到16（十六进制）的范围内。

如果指定了10（十进制）以外的数字基数，则写入第一个格式说明符中指定的字符数（减去前导空格的一个），并在需要时添加前导零。

对于大于10的基数，字母A，B，C，D，E和F分别用作具有值10,11,12,13,14和15的数字的输入。

表7-8列出了INTEGER输出数据项及其格式说明符的示例。 INTEGER数据项的输出值也包含在表中。 表中使用双引号作为分隔符以显示前导空格; 但是，双引号不是由KAREL程序编写的。

表7-8. INTEGER输出数据项的示例

数据项	输出	注解
123	" 123 "	头空白
-5	" -5 "	头空白
123::6	" 123 "	右对齐（头空白）
-123::2	" -123 "	按需要展开
1024::0::16	" 400 "	十六进制输出

表7-8. INTEGER输出数据项的示例（续）

数据项	输出	注解
5::6::2	" 00101 "	二进制输出（头零）
-1::9::16	" FFFFFFFF "	十六进制输出

7.8.2 REAL 明摩麻楚弓

READ语句中的REAL数据项按如下方式处理：

默认值：从输入行上的下一个非空白字符开始读取并继续，直到遇到空白或行尾。

数据可以提供或不提供小数部分。 用于科学记数法的E可以是大写或小写。 如果字符不形成有效的REAL，则读取操作失败。

第一个格式说明符：表示要读取的字符总数。 输入字段必须完全在当前输入行上，并且可以包括前导空格，但不包括尾随空格。

第二个格式说明符：对于REAL数据项忽略。

表7-9列出了REAL输入数据项及其格式说明符的示例。 输入数据和REAL数据项的结果值包含在表中。 符号[eol]表示行尾，X表示输入行上的无关数据。

词7-9. REAL逾全明摩麻碍竹侍

明摩麻	逾全明摩	聋桑
real_var	1[eol]	1.0
real_var	1.000[eol]	1.0
real_var	2.5 XX	2.50
real_var	1E5 XX	100000.0
real_var::7	2.5 XX	格式错误（尾随空白）
real_var	1E	格式错误（无指数）
real_var::4	1E 2	格式错误（嵌入式空白）

WRITE语句中的REAL数据项的格式如下：

默认值：以下列形式用科学记数法书写

(blank)(msign)(d).(d)(d)(d)(d)(d)E(esign)(d)(d)

where:

(blank) 是一个空白

如果需要，(msign) 是减号

(d) is a digit

(esign) 是一个加号或减号

第一种格式说明符：表示要写入的字符总数，包括所有数字，空格，符号和小数点。如果格式说明符大于数据所需的格式说明符，则会添加前导空格。如果它小于要求，则根据需要扩展该字段。

在科学记数法的情况下，字符长度应大于（8 +第二格式说明符）以完全写入数据。

对于文件，说明符必须在1到127的范围内，对于其他输出设备，说明符必须在1到126的范围内。

第二种格式说明符：表示要输出小数点右边的位数，无论是否使用科学记数法。

第二格式说明符的绝对值表示要输出到小数点右侧的位数。

如果格式说明符为正数，则数据以固定格式显示（即没有指数）。如果是否定的，则使用科学记数法。

表 7–10 列出REAL输出数据项及其格式说明符的示例。 REAL数据项的输出值也包含在表中。表中使用**双引号**作为分隔符以显示前导空格; 但是，双引号不是由KAREL程序编写的。

词_7_10. REAL逾击明摩麻碍竹侍

明摩麻	逾击	淀谣
123.0	" 1.23000E+02"	科学记数法（默认格式）
123.456789	" 1.23457E+02"	在小数部分中舍入为5位数
.00123	" 1.23000E-03"	负指数
-1.00	" -1.00000E+00"	负值
-123.456::9	" -1.234560E+02"	场指数
123.456::12	"1.234560E+02"	开头的空白添加

词7-10. REAL逾击明摩麻碍竹侍，育 -

明摩麻	逾击	淀谣
123.456::9::2	" 123.46 "	右对齐和四舍五入
123.::12::-3	" 1.230E+02 "	科学计数法

7.8.3 BOOLEAN 数据项格式

READ语句中的BOOLEAN数据项的格式如下：

默认值：从输入行上的下一个非空白字符开始读取并继续，直到遇到空白或行尾。

TRUE的有效输入值包括TRUE，TRU，TR，T和ON。 FALSE的有效输入值包括FALSE，FALS，FAL，FA，F，OFF和OF。 如果读取的字符不形成有效的BOOLEAN，则读取操作将失败。

纯乙中楚弓赶替纬 > 表示要读取的字符总数。 输入字段必须完全在当前输入行上，并且可以包括前导空格，但不包括尾随空格。

纯于中楚弓赶替纬 > 忽略BOOLEAN数据项。

表7-11列出了BOOLEAN输入数据项及其格式说明符的示例。 输入数据和BOOLEAN数据项的结果值包含在表中。 （符号[eol]表示行尾，X表示输入行上的无关数据。）

词7-11. BOOLEAN 逾全明摩麻碍竹侍

明摩麻	逾全明摩	聋桑
bool_var	FALSE[eol]	FALSE
bool_var	FAL 3...	FALSE
bool_var	T[eol]	TRUE
bool_var::1	FXX	FALSE (only reads ‘ F”)
bool_var	O[eol]	format error (ambiguous)
bool_var	1.2[eol]	format error (not BOOLEAN)
bool_var::3	F [eol]	format error (trailing blanks)
bool_var::6	TRUE[eol]	format error (not enough data)

WRITE语句中的BOOLEAN数据项的格式如下：

默认值：写为“TRUE”或“FALSE”。（双引号在表中用作分隔符以显示前导空格;但是，双引号不是由KAREL程序编写的。）

纯乙中楚弓赶替纬 > 表示要写入的字符总数，包括空格（始终包含前导空格）。如果格式说明符大于数据所需的格式说明符，则添加尾随空格。如果它小于要求，则该字段在右侧被截断。

对于文件，说明符必须在1到127的范围内，对于其他输出设备，说明符必须在1到126的范围内。

纯于中楚弓赶替纬 > 指示数据是左对齐还是右对齐。如果格式说明符等于0，则输出字段中的输出字左对齐，其中一个前导空白，并且尾随空白根据需要。如果它等于1，则输出字在输出字段中右对齐，并根据需要使用前导空格。

表7-12列出了BOOLEAN输出数据项及其格式说明符的示例。 BOOLEAN数据项的输出值也包含在表中。表中使用双引号作为分隔符以显示前导空格;但是，双引号不是由KAREL程序编写的。

词7-12. BOOLEAN 逾击明摩麻碍竹侍

明摩麻	逾击	淀谣
FALSE	" **FALSE** "	默认包括前导空格
TRUE	" **TRUE** "	TRUE短于FALSE
FALSE::8	" **FALSE** "	左对齐（默认）
FALSE::8::1	" **FALSE** "	右对齐
TRUE::2	" **T** "	删减

7.8.4 STRING 数据项格式

READ语句中的STRING数据项的格式如下：

矜贝倾 > 从当前位置开始读取并继续到行尾。 如果获得的数据长度长于STRING的声明长度，则数据在右侧被截断。 如果它更短，则STRING的当前长度设置为实际长度。

纯乙中楚弓赶替纬 > 表示输入数据的总字段长度。如果字段长度长于STRING的声明长度，则输入数据将在右侧截断。如果它更短，则STRING的当前长度设置为指定的字段长度。

纯于中楚弓赶替纬 > 指示输入STRING是否用单引号括起来。如果格式说明符等于0，则输入不包含在引号中。

如果它等于2，则输入必须用引号括起来。扫描输入以查找下一个非空白字符。如果字符不是引号，则STRING无效且读取操作失败。

如果字符是引号，则扫描剩余的字符，直到找到另一个引号或行尾。如果未找到另一个引号，则STRING无效且读取操作失败。

如果找到两个引号，则它们之间的所有字符都将读入STRING变量，除非声明的STRING长度太短，在这种情况下数据在右侧被截断。

表7-13列出了STRING输入数据项及其格式说明符的示例，其中str_var已声明为STRING [5]。输入数据和STRING数据项的结果值包含在表中。符号[eol]表示行尾，X表示输入行上的无关数据。

表7-13. STRING输入数据项的示例

明摩麻	**逾全明摩**	**聋桑**
str_var	" **ABC[eol]** "	" **ABC** "
str_var	" **ABCDEFG[eol]** "	" **ABCDE** " （读取FG但STRING截断为5个字符）

词7-13. STRING逾全明摩麻碍竹侍，育 -

明摩麻	逾全明摩	聋桑
str_var	" 'ABC'XX "	" 'AB " （空白和引用被读作数据）
str_var::0::2	" 'ABC'XX "	" 'ABC' " （读取以第二个引用结束）

WRITE语句中的STRING数据项的格式如下：

矛贝倾 > STRING的内容写入时没有尾随或前导空格或引号。

对于文件，STRING的长度不得超过127个字节，对于其他输出设备，STRING的长度不得超过126个字节。否则，程序将因“STRING TOO LONG”错误而中止。

纯乙中楚弓赶替纬 > 表示要写入的字符总数，包括空格。如果格式说明符大于数据所需的格式说明符，则数据左对齐并添加尾随空格。如果格式说明符小于required，则STRING将在右侧截断。

对于文件，说明符必须在1到127的范围内，对于其他输出设备，说明符必须在1到126的范围内。

纯于中楚弓赶替纬 > 指示输出是左对齐还是右对齐以及是否使用以下值将STRING括在引号中：

0左对齐，没有引号

1右对齐，无引号

2左对齐，引号

3右对齐，引用

引用的STRING值，即使左对齐，前面还有一个空格。不带引号的STRING值不会自动以空格开头。

表7-14 列出了STRING输出数据项及其格式说明符的示例。 STRING数据项的输出值也包含在表中。 表中使用双引号作为分隔符以显示前导空格; 但是，双引号不是由KAREL程序编写的。

词7-14. STRING 逾击明摩麻碍竹侍

明摩麻	逾击	淀谣
’ABC’	" **ABC** "	无开头空白
’ABC’::2	" **AB** "	在右边删减
’ABC’::8	" **ABC** "	左对齐
’ABC’::8::0	" **ABC** "	同上
’ABC’::8::1	" **ABC** "	右对齐
’ABC’::8::2	" **’ABC’** "	注开头空白

词7-14. STRING 逾击明摩麻碍竹侍，育 -

明摩麻	逾击	淀谣
'ABC'::8::3	" **'ABC'** "	右对齐
'ABC'::4::2	" **'A'** "	删减

当再次读取值时，STRING数据项的格式说明符可能导致原始STRING值截断或添加尾随空白。
如果必须连续编写和读取STRING值，以下指南将帮助您确保可以相同地读回不同长度的STRING值：

- 读取STRING的变量必须至少与正在读取的实际STRING一样长，或者发生截断。
- 必须做出一些规定，将STRING值与同一数据线上的前一个变量分开。 一种可能性是在STRING和它之前的变量之间写一个''（空白）。
- 如果在读取操作中未使用格式说明符，则在其各自数据行的末尾写入STRING值（即，在CR的输出列表中跟随），因为读取没有格式说明符的STRING变量，直到行的末尾为 到达。
- 将字符串值写入文件并将其读回的最常用方法是使用格式:: 0 :: 2进行读取和写入。

7.8.5 VECTOR 数据项格式

无法从文本（ASCII）文件中读取VECTOR数据项。 但是，您可以读取三个REAL值并将它们分配给VECTOR变量的元素。 WRITE语句中的VECTOR数据项在同一行上被格式化为三个REAL值。
表7-15列出了VECTOR输出数据项及其格式说明符的示例，其中vect.x = 1.0，vect.y = 2.0，vect.z = 3.0。 VECTOR数据项的输出值也包含在表中。

表中使用双引号作为分隔符以显示前导空格; 但是，双引号不是由KAREL程序编写的。

只超又课 > 第7.8.2节 “REAL 数据项格式” ，以获取有关REAL数据项使用的默认输出格式和格式说明符的信息

词7-15. VECTOR 逾击明摩麻碍竹侍

明摩麻	逾击
vect	" 1. 2. 3. "
vect::6::2	" 1.00 2.00 3.00 "
vect::12::-3	" 1.000E+00 2.000E+00 3.000E+00 "

7.8.6 Positional 数据项格式

无法从文本（ASCII）文件中读取位置数据项。 但是，您可以读取六个REAL值和一个STRING值，并将它们分配给XYZWPR变量的元素，或使用POS内置函数来组成POSITION。 内置的CNV_STR_CONF可用于将STRING转换为CONFIG数据类型。

WRITE语句中的POSITION和XYZWPR数据项的格式为三行输出。 第一行包含POSITION的位置（x，y，z），第二行包含方向（w，p，r），第三行包含配置字符串。

位置和方向组件的格式为六个REAL值。 POSITION中REAL值的默认格式是REAL的默认格式。 请参阅第7.8.2节。

配置字符串不以CR终止，这意味着您可以在同一行上使用其他数据。

表7-16列出了POSITION输出数据项及其格式说明符的示例，其中p = POS（2.0，-4.0,8.0,0.0,90.0,0.0，config_var）。 POSITION数据项的输出值也包含在表中。 表中使用双引号作为分隔符以显示前导空格; 但是，双引号不是由KAREL程序编写的。

词7-16. POSITION逾击明摩麻碍竹侍，p = POS，2.0 0 -4.0,8.0,0.0,90.0,0.0 0 config_var - -

明摩麻	逾击
p	" 2. -4. 8. " " 0. 9. 0. " " N, 127, , -1 "
p::7::2	" 2.00-4.00 8.00 " " 0.0090.00 0.00 " " N, 127, , -1 "

WRITE语句中的JOINTPOS数据项的格式与POSITION类型类似，在一行中有三个值。

只超又课 > 纯7.8.2蛙，了解与REAL数据项一起使用的格式说明符的信息

POS内置功能，附录A.

7.9 二进制输入/输击楚弓

本节介绍READ和WRITE语句中用于读取和写入每个数据项的二进制（未格式化）数据的格式说明符。 二进制输入/输出操作有时被称为未格式化，而不是被称为格式化的文本（ASCII）输入/输出操作。

具有ATR_UF属性的内置SET_FILE_ATR用于为二进制操作指定文件变量。 如果未指定，将使用ASCII文本操作。

READ和WRITE语句中的数据项可以是二进制文件的以下任何数据类型：

INTEGER
REAL
BOOLEAN
STRING
VECTOR
POSITION
XYZWPR
XYZWPREXT
JOINTPOS

视觉和数组变量不能以未订阅的形式读取或写入。 ARRAY的元素以与ARRAY的数据类型对应的格式读取或写入。

可以读取或写入整个PATH变量，也可以指定只读取或写入node [0]（包含PATH标头），特定节点或节点范围。 格式说明符具有

对PATH数据没有影响。 PATH数据只能读取或写入文件，而不能读取或写入串行端口，CRT / KB或示教器。

在创建仅由KAREL程序读取的文件时，二进制I / O优先于文本I / O，原因如下：

- 无法直接从文本输入中读取Positional，VECTOR和PATH变量。
- 某些格式和数据组合的读取方式与在文本文件中写入的方式不同，如果以相同格式读取，则无效。
- 二进制数据通常更紧凑，减少了文件大小和I / O时间。
- 从REAL数据转换为ASCII表示并返回时，存在一些不可避免的精度损失。

通常，不需要将格式说明符与二进制I / O一起使用。 如果遵循此规则，则可以完全按照写入之前的所有输入数据进行读取。

但是，如果要写入大量的INTEGER值并且已知它们的值很小，则使用格式说明符写入这些值会减少存储空间和I / O时间。

例如，-128到+127范围内的INTEGER值只需要一个字节的存储空间，而-32768到+32767范围内的INTEGER值需要两个字节的存储空间。 使用第一个格式说明符1和2分别在这些范围内写入INTEGER值可以减少存储空间和I / O时间要求，而不会丢失有效数字。

表7-17总结了可与READ和WRITE语句中的数据项一起使用的输入和输出格式说明符。 还包括每种数据类型的默认格式。 第7.8.1节到第7.8.6节更详细地解释了格式说明符对每种数据类型的影响。

另请参见：SET_FILE_ATR内置例程，附录A.

表7-17. 二进制输入/输出格式说明符

数据类型	默认	第一个格式说明符	第二个格式说明符
INTEGER	读取或写入四个字节	以最大有效字节(1-4)开头的最小有效字节数	忽略
REAL	读取或写入四个字节	忽略	忽略
BOOLEAN	读取或写入四个字节	以最大有效字节(1-4)开头的最小有效字节数	忽略
STRING	字符串的当前长度（1个字节），后跟数据字节	读取或写入的字节数	忽略
VECTOR	读取或写入三个4字节REAL数字	忽略	忽略
POSITION	读取或写入56个字节	忽略	忽略
XYZWPR	读取或写入32个字节	忽略	忽略
XYZWPREXT	读取或写入44个字节	忽略	忽略
JOINTPOSn	读取或写入4 + n * 4个字节	忽略	忽略
PATH	取决于结构的大小	忽略	忽略

7.9.1 INTEGER 数据项格式

READ或WRITE语句中的INTEGER数据项的格式如下：默认值：从最高有效字节开始读取或写入四个字节的数据。

第一个格式说明符：表示要读取或写入的INTEGER的最低有效字节数，其中最重要的是首先读取或写入。 最重要的字节读取的符号扩展为未读字节。 格式说明符必须在1到4的范围内。例如，如果使用格式说明符2写入INTEGER，则将写入字节3和4（其中字节1是最高有效字节）。 在二进制I / O操作中格式化INTEGER值时，不会检查是否存在重要字节丢失。注意格式化INTEGER值可能导致未检测到的高位数丢失。

第二个格式说明符：对于INTEGER数据项忽略。

7.9.2 REAL 数据项格式

READ或WRITE语句中的REAL数据项的格式如下：

默认值：从最高有效字节开始读取或写入四个字节的数据。

第一个格式说明符：忽略REAL数据项。

第二个格式说明符：对于REAL数据项忽略。

7.9.3 BOOLEAN 数据项格式

READ或WRITE语句中的BOOLEAN数据项的格式如下：

默认值：读取或写入四个字节的数据。 在读操作中，从未使用过的单词的其余部分设置为0。

第一个格式说明符：表示要读取或写入的BOOLEAN的最低有效字节数，其中最重要的是这些字节。 格式说明符必须在1到4的范围内。由于BOOLEAN值始终为0或1，因此使用字段宽度1始终是安全的。

第二个格式说明符：忽略BOOLEAN数据项。

7.9.4 STRING 数据项格式

READ或WRITE语句中的STRING数据项的格式如下：

默认值：STRING的当前长度（不是声明的长度）作为单个字节读取或写入，后跟STRING的内容。 没有格式说明符的STRING值将其长度作为输出的一部分，而使用格式说明符写入的STRING值则不会。

同样，如果在没有格式说明符的情况下读取STRING，则在数据中预期长度，而如果使用格式说明符读取STRING，则不期望长度。

这意味着，如果您编写然后读取STRING数据，则必须确保使用格式说明符是一致的。

第一个格式说明符：表示要读取或写入的字节数。

第二个格式说明符：对STRING数据项忽略。

在读取操作中，如果第一个格式说明符大于STRING的声明长度，则数据在右侧被截断。如果它小于STRING的声明长度，则STRING的当前长度设置为读取的字节数。

在写入操作中，如果第一个格式说明符指示的字段短于STRING的当前长度，则STRING数据在右侧被截断。如果它长于STRING的当前长度，则输出将在空白处填充空白。

使用格式说明符编写STRING值会导致在重读时截断原始STRING值或填充STRING值末尾的填充空白。

7.9.5 VECTOR 数据项格式

READ或WRITE语句中的VECTOR数据项的格式如下：

默认值：数据作为三个4字节二进制REAL数读取或写入。

第一个格式说明符：忽略VECTOR数据项。

第二个格式说明符：忽略VECTOR数据项。

7.9.6 POSITION 数据项格式

READ或WRITE语句中的POSITION数据项的格式如下：

默认值：以控制器的内部格式读取或写入，长度为56个字节。

7.9.7 XYZWPR 数据项格式

READ或WRITE语句中的XYZWPR数据项的格式如下：

默认值：以控制器的内部格式读取或写入，长度为32个字节。

7.9.8 XYZWPREXT 数据项格式

READ或WRITE语句中的XYZWPREXT数据项的格式如下：

默认值：以控制器的内部格式读取或写入，长度为44个字节。

7.9.9 JOINTPOS 数据项格式

READ或WRITE语句中的JOINTPOS数据项的格式如下：

默认值：以控制器的内部格式读取或写入，每个轴为4个字节加4个字节。

7.10 用户界面提示

通过在KAREL程序中执行“READ”和“WRITE”语句来完成对示教器或CRT / KB的输入和输出。 如果USER菜单不是当前选择的菜单，则输入将保持挂起状态，直到选择USER菜单。 输出将写入“已保存”窗口，当选择USER菜单时将显示该窗口。 您最多可以保存8个窗口。

7.10.1 示教器上的USER菜单

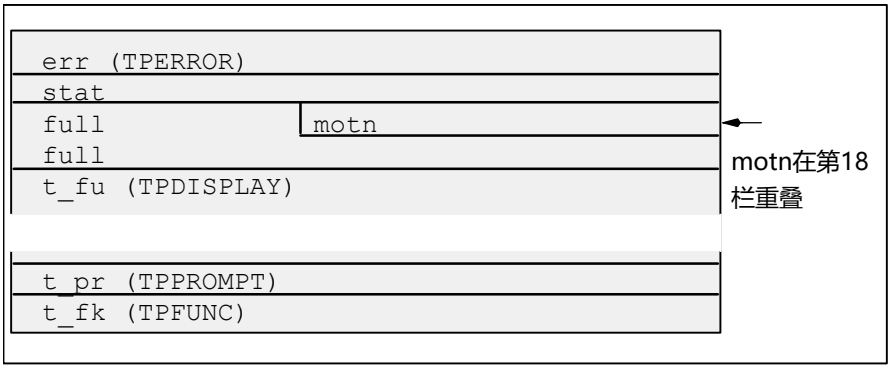
从示教器中选择USER菜单时激活的屏幕名为“t_sc”。 **表7-18**中列出的窗口是为“t_sc”定义的。

表 7–18. 为t_sc定义的窗口"

窗口名称	刘	预定义的文件名	滚动	行
"t_fu"	10	TPDISPLAY	yes	5-14
"t_pr"	1	TPPROMPT	no	15
"t_st"	3	TPSTATUS	no	2-4
"t_fk"	1	TPFUNC	no	16
"err"	1	TPERROR	no	1
"stat"	1		no	2
"full"	2		no	3-4
"motn"	1		no	3

默认情况下，USER菜单会
将“err”，“stat”，“full”，“motn”，“t_fu”，“t_pr”和“t_fk”窗口附加
到“t_sc”屏幕。 见图7-1。

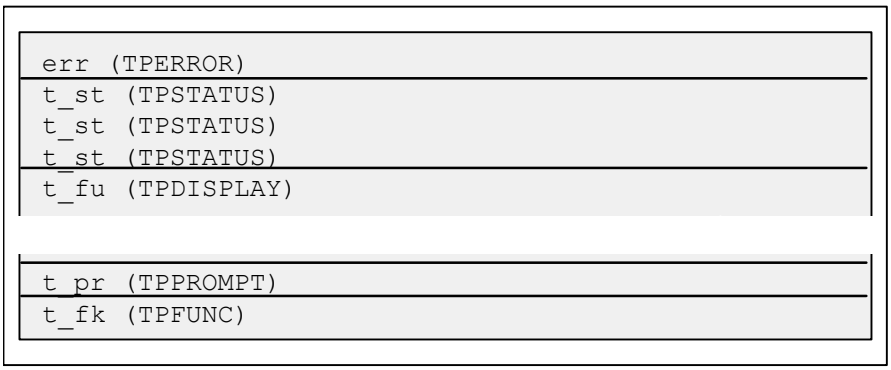
图 7–1. "t_sc" 屏幕



以下系统变量会影响示教器USER菜单：

- **$ TP_DEFPROG**：STRING - 标识示教器默认程序。 从示教器SELECT菜单中选择程序时，会自动设置此项。
- **$ TP_INUSER**：BOOLEAN - 从示教器中选择USER菜单时设置为TRUE。
- **$ TP_LCKUSER**：BOOLEAN - 当$ TP_DEFPROG正在运行且$ TP_LCKUSER为TRUE时，锁定USER菜单中的示教器。
- **$ TP_USESTAT**：BOOLEAN - 当$ TP_USESTAT为TRUE时，将用户状态窗口“t_st”（TPSTATUS）附加到用户屏幕。 附加“t_st”时，“stat”，“motn”和“full”窗口将被分离。 见图7-2。

图 7–2. 带有$ TP_USESTAT = TRUE的“t_sc”屏幕



7.10.2 CRT / KB上的USER菜单

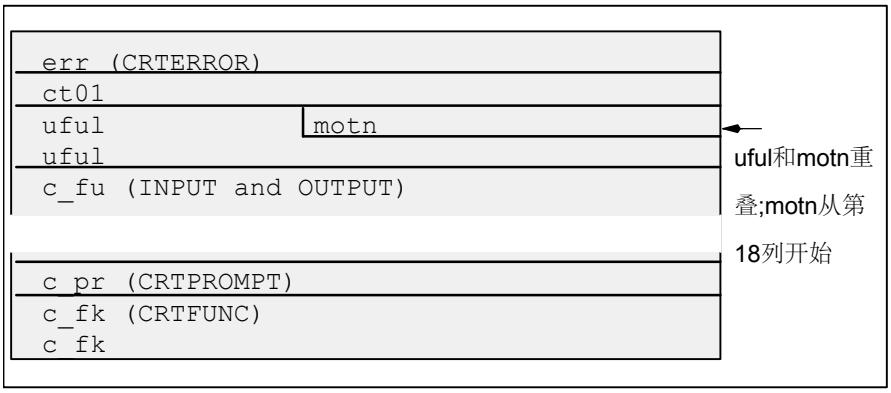
从CRT中选择USER菜单时激活的屏幕名为“c_sc”。 表7-19中列出的窗口是为“c_sc”定义的。

表 7–19. 为c_sc定义的窗口"

窗口名称	列	预定义的文件名	滚动	行
"c_fu"	17	INPUT and OUTPUT	yes	5-21
"c_pr"	1	CRTPROMPT	no	22
"c_st"	3	CRTSTATUS	no	2-4
"c_fk"	2	CRTFUNC	no	23-24
"err"	1	CRTERROR	no	1
"ct01"	1		no	2
"uful"	2		no	3-4
"motn"	1		no	3

默认情况下，USER菜单会将“err”，“ct01”，“uful”，“motn”，“c_fu”，“c_fk”和“uftn”窗口附加到“c_sc”屏幕。 “c_fk”窗口将为功能键标记为F9和F10显示FCTN和MENUS。 见图7-3。

图 7–3. "c_sc" 屏幕

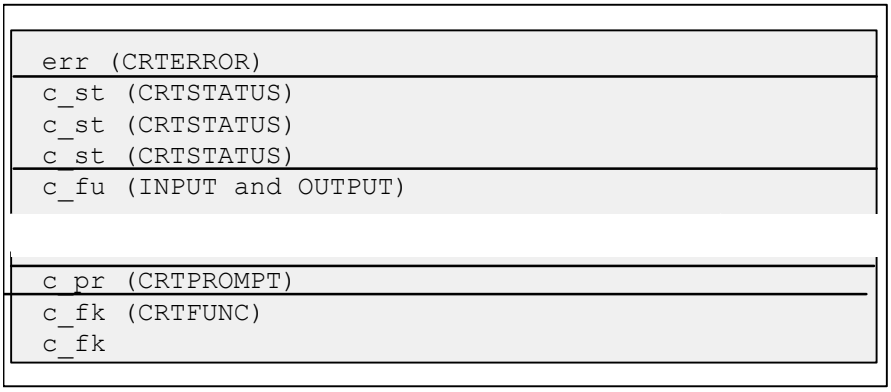


以下系统变量会影响CRT USER菜单：

- $ CRT_DEFPROG：STRING - 此变量标识CRT默认程序。 从CRT SELECT菜单中选择程序时自动设置。
- $ CRT_INUSER：BOOLEAN - 从CRT中选择USER菜单时，此变量设置为TRUE。

- **$ CRT_LCKUSER**：BOOLEAN - 当$ CRT_DEFPROG正在运行且$ CRT_LCKUSER为TRUE时，此变量将锁定USER菜单中的CRT。
- **$ CRT_USERSTAT**：BOOLEAN - 此变量导致用户状态窗口“c_st”（CRTSTATUS）附加到用户屏幕，而$ CRT_USERSTAT为TRUE。 附加“c_st”时，将分离“ct01”，“motn”和“uful”窗口。 请参见**图7-4。**

图 7–4. "c_sc" 屏幕, $CRT_USERSTAT = TRUE



第8章

位置数据

目录

第8章 位置数据 8–1
8.1 概述 8–2
8.2 位置数据 8–2
8.3 坐标系 8–3
8.3.1 World 坐标 8–4
8.3.2 User 坐标系 (UFRAME) 8–5
8.3.3 Tool 定义 (UTOOL) 8–5
8.3.4 在示教器编辑器（TP）中使用坐标系 8–5
8.4 JOG 坐标系统 8–6

8.1 概述

在机器人应用中，单段运动是工具中心点（TCP）从初始位置到期望目标位置的移动。

KAREL系统根据位置（x，y，z），方向（w，p，r）和配置表示位置数据。 相对于笛卡尔坐标系（用户框架）定义位置和方向，使其独立于机器人关节角度。 配置表示特定位置和方向上的唯一关节角度集。

注意KAREL系统提供了一种创建和操作位置数据的方法，但它不支持运动指令。 所有动作必须从示教器程序启动。 说明和内置可用于将KAREL位置数据设置为示教器程序。

8.2 位置数据

KAREL语言使用POSITION，XYZWPR，XYZWPREXT，JOINTPOS和PATH数据类型来表示位置数据。 POSITION数据类型由以下内容组成：

- 三个REAL值表示以毫米为单位表示的x，y，z位置
- 三个REAL值表示以度为单位表示的w，p，r方向
- 一个CONFIG数据类型，由4个布尔值和3个整数组成，代表关节位置和转弯数的配置。在指定配置数据类型之前，请确保它对正在使用的机器人有效。 有效的联合放置值包括：
  - 'R'或'L'（肩膀向右或向左）
  - 'U'或'D'（肘部向上或向下）
  - 'N'或'F'（手腕无翻转或翻转）
  - 'T'或'B'（配置正面或背面）

  转弯数是指多转弯关节超出所需转数到达某个位置的完整转弯次数。 表8-1列出了有效的转弯数定义。

表8-1. 转数字定义

转数	旋转（度）
-8	-2700 to -3059
-7	-2340 to -2699

表8-1. 转数定义（续）

-6	-1980 to -2339
-5	-1620 to -1979
-4	-1260 to -1619
-3	-900 to -1259
-2	-540 to -899
-1	-180 to -539
0	-179 to 179
1	180 to 539
2	540 to 899
3	900 to 1259
4	1260 to 1619
5	1620 to 1979
6	1980 to 2339
7	2340 to 2699

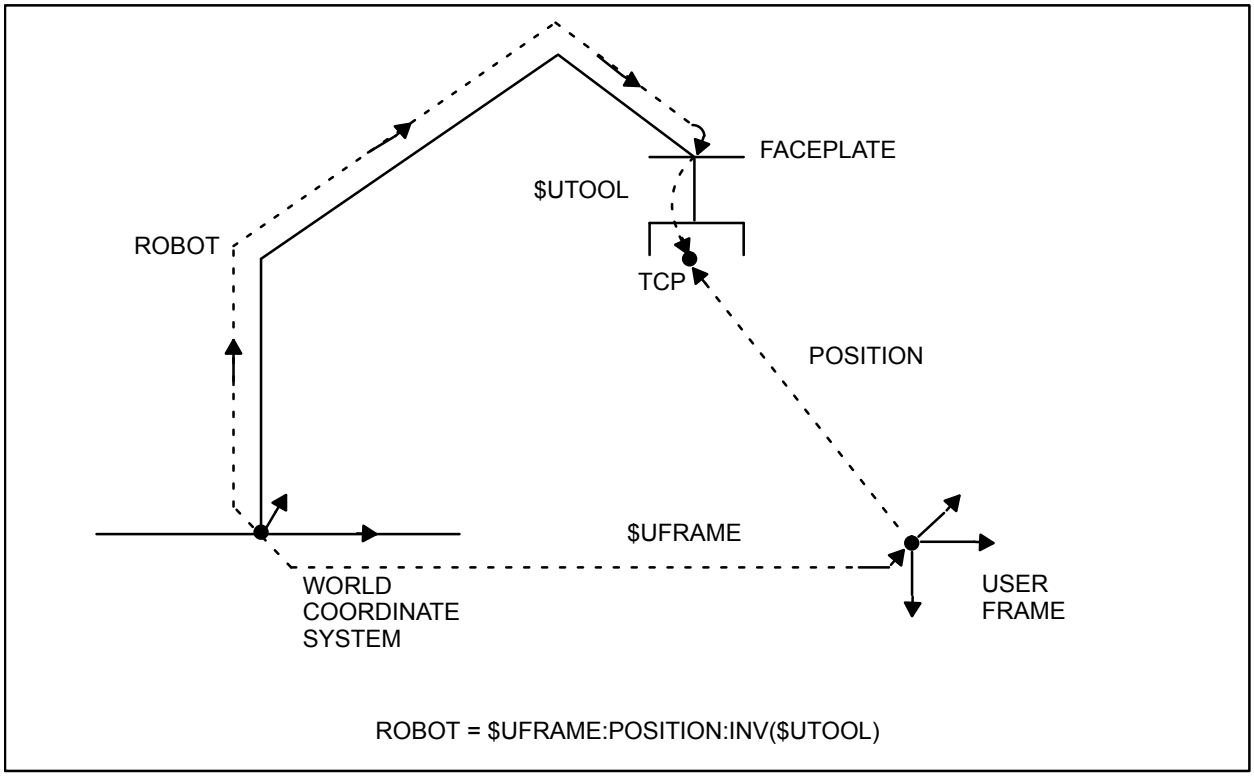
PATH 数据类型由称为路径节点的变长列表元素组成。

另请参阅：适用于特定应用的FANUC机器人设置和操作手册，以获取有关每个支持的机器人型号的配置信息。 POSITION，XYZWPR，XYZWPREXT，JOINTPOS和PATH数据类型，附录A，''KAREL语言按字母顺序描述。''

8.3 参考框架

KAREL系统定义了位置数据相对于用户定义的参考框架（称为用户框架）的位置和方向，如图8-1所示。

图8-1. 引用KAREL中的位置



存在三个参考框架：

- WORLD - 预定义
- UFRAME - 由用户确定
- UTOOL - 由工具定义

使用运动方程，控制器根据已知的世界框架和存储在系统变量$ UFRAME（用于用户框架）和$ UTOOL（用于工具框架）中的数据计算其位置信息。

8.3.1 World 坐标

为每个机器人预定义世界框架。 它用作默认参考框架。 每个机器人模型的世界框架的位置不同。

8.3.2 User 坐标系 (UFRAME)

程序员通过为系统变量$ UFRAME赋值来定义相对于世界框架的用户框架。

警告

无论是示教位置数据还是使用该数据运行程序，都要确保$ UFRAME设置为相同的值，否则可能会损坏工具。

UFRAME 的位置表示沿世界坐标系的x轴，y轴和z轴的距离; 方向表示围绕这些轴的旋转。默认情况下，系统将（0,0,0）位置值和（0,0,0）方向值分配给$ UFRAME，这意味着用户框架与世界坐标系的框架相同。 所有位置都相对于UFRAME记录。

8.3.3 Tool 定义 (UTOOL)

工具中心点（TCP）是UTOOL参照系的原点。 程序员通过为系统变量$ UTOOL分配值来定义TCP相对于机器人面板的位置。 默认情况下，系统为$ UTOOL分配一个（0,0,0）位置和一个（0,0,0）方向，这意味着$ UTOOL与面板坐标系统相同。 UTOOL的正z轴定义了工具的逼近矢量。

警告

确保$ UTOOL正确定义了您正在使用的工具的TCP位置，否则可能会损坏工具。

面板坐标系的原点位于面板表面的中心。 其方向由面板上x轴和y轴的平面定义，正z轴从面板直接指向。

8.3.4 在示教器编辑器（TP）中使用坐标系

系统变量$ USEUFRAME定义在记录或修改位置时，$ MNUFRAMENUM [group_no]的当前值是否将分配给该位置的用户帧。

- 当$ USEUFRAME = FALSE时，无论$ MNUFRAMENUM [group_no]的值如何，位置的初始记录和位置的触摸都是以用户帧编号等于0完成的。
- 当$ USEUFRAME = TRUE时，位置的初始记录是在位置的用户帧等于$ MNUFRAMENUM [group_no]定义的用户帧的情况下完成的。 位置的触摸也必须在位置的用户帧等于$ MNUFRAMENUM [group_no]定义的用户帧的情况下完成。

当在示教器编辑器中记录位置时，位置工具框的值将始终等于记录位置时$ MNUTOOLNUM [group_no]的值。 执行示教器程序时，必须确保位置的用户框架和工具框架等于$ MNUFRAMENUM [group_no]和$ MNUTOOLNUM [group_no]的值; 否则，将发生错误。在记录位置之前，使用示教器编辑器中的UFRAME_NUM = n和UTOOL_NUM = n指令设置$ MNUFRAMENUM [1]和$ MNUTOOLNUM [1]的值，以保证在程序执行期间用户和工具帧编号匹配。

8.4 JOG 坐标系统

KAREL系统提供五种不同的点动坐标系：

- JOINT - 关节坐标系，各个机器人轴移动。运动是联合插值的。
- WORLD - 一种笛卡尔坐标系，其中TCP与预定义的WORLD帧的x，y和z轴平行或旋转。运动是线性插值的。
- TOOLFRAME - 笛卡尔坐标系，其中TCP与当前所选工具框架的x，y和z轴平行或旋转。运动是线性插值的。通常使用“设置框架”菜单选择工具框架。要使用$ GROUP [group_no].$ utool进行点动，请设置$ MNUTOOLNUM [group_no] = 30。
- JOGFRAME - 一个笛卡尔坐标系，其中TCP与$ JOG_GROUP [group_no].$ jogframe系统变量定义的坐标系的x，y和z轴平行或旋转。运动是线性插值的。
- USER FRAME - 笛卡尔坐标系，其中TCP与当前所选用户框架的x，y和z轴平行或旋转。运动是线性插值的。通常使用“设置帧”菜单选择用户帧。要使用$ GROUP [group_no].$ uframe进行点动，请设置$ MNUFRAMENUM [group_no] = 62。

可以在这些点动坐标系中的任何一个中慢跑机器人以到达目的地位置。 但是，一旦达到该位置，就会参考用户框架记录位置数据，如第8.3节所述。

另请参阅：特定于应用程序的FANUC机器人设置和操作手册，逐步说明如何点动和定义框架。

第9章

文件系统

目录

第9章 文件系统 9–1
9.1 概述 9–2
9.2 文件规范 9–3
9.2.1 设备名称 9–3
9.2.2 文件名 9–4
9.2.3 文件类型 9–5
9.3 存储设备访问 9–6
9.3.1 概述 9–7
9.3.2 存储文件设备 9–13
9.3.3 虚拟设备 9–14
9.3.4 文件通道 9–16
9.4 文件访问 9–20
9.5 XML输入格式 9–20
9.5.1 概述 9–20
9.5.2 安装顺序 9–21
9.5.3 示例KAREL程序引用XML文件 9–22
9.5.4 解析错误 9–26
9.6 存储设备 9–27

9.1 概述

文件系统提供了在 **CMOS RAM，FROM** 或外部存储设备上存储数据的方法。 数据被分组为单元，每个单元代表一个文件。 例如，文件可以包含以下内容：

- **KAREL** 程序的源代码语句
- 一系列用于命令过程的 **KCL** 命令
- 程序的可变数据

文件由包含以下内容的文件规范标识：

- 存储文件的设备的名称
- 文件的名称
- 文件中包含的数据类型

KAREL系统包括五种类型的存储设备，可以存储文件：

- **RAM Disk**
- **FROM Disk**
- **IBM PC**
- **Memory Card**
- **USB Memory Stick Device**

RAM Disk 是SRAM（以前称为CMOS RAM）或DRAM存储器的一部分，用作单独的存储设备。 任何文件都可以存储在RAMDisk中。 RAMDiskfile应该被写入磁盘以进行永久存储。
FROM Disk 是FROM内存的一部分，用作单独的存储设备。 任何文件都可以存储在F-ROM磁盘上。 但是，硬件支持有限数量的读写周期。 因此，如果文件需要存储动态变化的数据，则应使用RAM磁盘。
IBM PC 或兼容计算机可用于离线存储文件。 您可以使用OLPC（用于PC的FANUC Robotics离线存储软件）将文件存储在外部存储设备上。 可以通过以下方式访问这些存储设备上的文件：

- 通过示教器上的FILE菜单和CRT / KB
- 通过KAREL程序

Memory Card是指ATA Flash文件存储。 memorycard接口位于MAIN CPU上。
有关存储设备和内存的更多信息，请参见第9.3.1节。

USB Memory Stick Device 支持USB 1.1 接口。 USB组织规定了USB 1.1和2.0的标准。 大多数记忆棒设备符合USB 2.0规范的操作和电气标准。 USB规范定义的USB 2.0设备必须向后兼容USB 1.1设备。

但是，FANUC Robotics不支持USB记忆棒上的任何安全或加密功能。 该控制器支持最广泛使用的USB闪存棒，尺寸从32MB到1GB。 USB接口位于控制器操作面板上。

9.2 文件规范

文件规范标识文件。 规范表明：

- 存储文件的设备名称，请参见第9.2.1节。
- 文件名，请参见第9.2.2节。
- 文件包含的数据类型，请参见第9.2.3节。

文件规范的一般形式是：

device_name：file_name.file_type

9.2.1 设备名称

设备名称至少包含两个字符，用于指示存储文件的设备。 文件可以存储在RAM磁盘，F-ROM磁盘，磁盘驱动器单元，PC上的离线，存储卡或PATH复合设备上。 设备名称始终以冒号（:)结尾。 以下是有效存储设备的列表。

- **RD: (RAM Disk)**

  RD：设备名称是指存储在控制器的RAM磁盘上的文件。 RD：用作默认设备名称。

- **FR: (F-ROM Disk)**

  FR：设备名称是指存储在控制器的F-ROM盘上的文件。

- **MC: (Memory Card Device)**

  存储卡可以格式化并用作MS-DOS文件系统。 它可以在控制器和配备有适当硬件和软件的IBM PC上读取和写入。 如果存储卡用作MS-DOS文件系统，则应仅在控制器上格式化。 有关格式化控制器上存储卡的信息，请参阅特定于应用程序的FANUC机器人设置和操作手册。

- **UD1: (USB Memory Stick Device)**
  USB记忆棒可以格式化并用作MS-DOS文件系统。 它可以在控制器和配备有适当硬件和软件的IBM PC上读取和写入。 如果USB记忆棒用作MS-DOS文件系统，则只应在控制器上格式化。 有关格式化控制器上USB记忆棒的信息，请参阅特定于应用程序的FANUC机器人设置和操作手册。
- **MD: (Memory Device)**
  存储器设备将控制器的程序存储器视为文件设备。 您可以访问控制器中加载的所有示教程序，KAREL程序，KAREL变量，系统变量和错误日志。 更多详细信息，请参见第9.6节。
- **MDB: (Memory Device Backup)**
  内存设备备份设备（MDB :)允许用户复制文件菜单上备份功能提供的相同文件。 这允许用户远程备份控制器。
- **CONS: (Console Device)**
  控制台设备提供对控制台日志文本文件CONSLOG.LS和CONSTAIL.LS的访问。 它用于诊断和调试目的，而不是用作存储设备。
- **MF: (Memory File Device)**
  MF：设备名称是指存储在RAM和F-ROM磁盘上的文件。 由于文件不能同时位于两个磁盘上，因此不会有重复的文件名。
- **PATH: (Composite Device)**
  PATH：设备是一个只读设备，它按顺序搜索F-ROM磁盘（FD :)，存储卡（MC：0和软盘（FLPY :)），用于指定文件.PATH：设备消除 用户需要知道指定文件存在于哪个存储设备上。
- **PIP: (File Pipe Device)**
  PIP：设备提供了一种从一个应用程序写入数据的方法，同时从另一个应用程序读取数据。
  PIP：设备还允许保留写入的最后一组数据用于分析。 PIP：设备允许您访问任意数量的管道文件。 此访问权限是控制器内存中的文件。 这意味着访问这些文件非常有效。 文件大小和文件数受可用控制器内存的限制。 这意味着文件管道的最佳用途是缓冲数据或暂时保存数据。

9.2.2 文件名

文件名是您选择表示文件内容的标识符。 以下规则适用于文件名：

- 文件名限制为36个字符。
- 文件名可以包括字母，数字和下划线。
- 文件名不能包含以下字符：. : *; \ / “'
- 文件名中不允许使用空格。
- 可以谨慎使用其他特殊字符。
- 可以使用子目录。 这些也被称为“路径”。它们以“\”字符开头和结尾。 文件名的规则也适用于路径。 以下是带有设备和路径的文件名示例：

```
MC:\mypath\myfile.txt
```

9.2.3 文件类型

文件类型由两个或三个字符组成，用于指示文件包含的数据类型。 文件类型始终以句点（。）开头。 表9-1是每个可用文件类型及其功能的字母顺序列表。

表9-1. 文件类型说明

文件类型	描述
.BMP	位图文件包含机器人视觉系统中使用的位图图像。
.CF	KCL命令文件是包含命令过程的一系列KCL命令的ASCII文件。
.CH	条件处理程序文件用作条件监视器功能的一部分。
.DF	默认文件是二进制文件，包含每个示教器编程的默认运动指令。
.DG	诊断文件是ASCII文件，提供有关控制器各种功能的状态或诊断信息。
.DT	KAREL数据文件可包含用户所需的任何数据的ASCII或二进制文件。
.IO	包含I / O配置数据的二进制文件 - 在显示I / O屏幕并保存数据时生成。
.KL	KAREL源代码文件是包含KAREL程序的KAREL语言语句的ASCII文件。

表9-1. 文件类型说明（续）

文件类型	描述
.LS	KAREL列表文件是ASCII文件，其中包含KAREL语言程序的列表以及每个KAREL语句的行号。
.MN	KAREL的早期版本支持助记程序文件。
.ML	零件模型文件包含机器人视觉系统中使用的零件模型信息。
.PC	KAREL p代码文件是二进制文件，包含翻译者在翻译.KL文件时产生的p代码。
.SV	系统文件是二进制文件，包含工具和用户帧（SYSFRAME.SV），系统变量（SYSVARS.SV），母版（SYSMAST.SV），伺服参数（SYSSERVO.SV）和宏（SYSMACRO.SV）的数据。
.TP	示教程序文件是二进制文件，其中包含示教程序的说明。
.TX	文本文件是ASCII文件，可以包含系统定义的文本或用户定义的文本。
.VR	程序变量文件是包含KAREL程序的可变数据的二进制文件。
.VA	ASCII变量文件包含具有变量名称，类型和内容的变量文件的列表。
.LS	列表文件是ASCII格式的示教程序，错误日志和描述历史。

9.3 存储设备访问

KAREL系统只能访问已格式化和安装的存储设备。 首次在KAREL系统上安装设备时，将执行这些过程。

访问存储设备时，以下规则适用：

- 格式化设备
  - 删除设备上的所有现有数据。 例如，如果格式化RD2：，您还将重新格式化RD：thru RD7：上存在的任何数据。
  - 记录设备上的目录
  - 记录KAREL系统所需的其他数据

- 为设备分配卷名称

有关格式化设备的更多信息，请参阅附录A中的FORMAT_DEV内置“KAREL语言字母说明”或附录C“KCL命令字母说明”中的FORMAT命令。

9.3.1 概述

以下类型的存储设备可用于存储程序和文件：

- Memory Card (MC:)
- Flash File Storage disk (FR:)
- RAM Disk (RD:) (Not for SpotTool+)
- Ethernet Device (optional)
- Memory Device (MD:)
- Memory Device Binary (MDB:)
- Filtered Memory Device (FMD:)
- USB Memory Stick Device on the controller (UD1:)
- USB Memory Stick Device on the *i*Pendant (UT1:)

本节介绍如何设置存储设备以供使用。 根据存储设备，这可以包括

- 在控制器上设置端口
- 将设备连接到控制器
- 格式化设备

Memory Card (MC:)

控制器支持存储卡。 存储卡支持8MB或更高的各种尺寸。 如果与合适的紧凑型适配器一起使用，也支持Compact Flash PC卡。 存储卡需要存储卡接口，该接口是控制器内主CPU的标准接口。 注意控制器支持从存储卡加载软件。

警告

控制器中存在致命电压，只要它连接到电源。 小心避免电击。 只要控制器连接到电源，输入侧就会出现高电压。 将断路器或断路器转到OFF位置只会断开设备输出侧的电源。

警告

存储卡接口位于控制器机柜的主CPU上。 当电源断开电路断路器关闭时，控制器内部仍然有电。 将存储卡插入存储卡接口之前，请关闭电源断开电路断路器;否则，你可能会伤害到人员。

警告

控制器正在读取或写入存储卡时，请勿取出存储卡。 这样做可能会损坏卡并丢失存储在卡上的所有信息。

存储卡可以在控制器上格式化，可以用作加载设备来安装软件。

注意所有内部文件设备（如FR：，RD：和MD）上的数据应备份到外部文件设备，如ATA Flash PC卡。

注意控制器使用512字节的扇区大小格式化卡。

存储卡可以格式化并用作MS-DOS文件系统。它可以从控制器和配备有适当硬件和软件的IBM PC读取和写入。如果存储卡用作MS-DOS文件系统，则应在控制器上格式化。

控制器可以读取和写入使用FAT（也称为FAT16）或FAT32格式（文件访问表）格式化的存储卡。在Controller上格式化存储卡时，可以将其格式化为FAT或FAT32类型。

FAT32格式（32位FAT）消除了FAT中包含的一些限制。其中一个限制是只能在Root目录中创建512个文件。另一种是FAT格式只支持最大2 GB的存储卡。此功能包含在控制器中，以增加机器人控制器与其他计算机系统的兼容性。

USB Memory Stick Device on the controller (UD1:), and USB Memory Stick Device on the iPendant (UT1:)

如果格式化为FAT16，控制器支持最大2GB的USB闪存棒。 FAT16的访问速度要快得多。不支持具有任何安全或加密功能的USB记忆棒。记忆棒大于2GB无法格式化FAT16，它们必须格式化为FAT32。

注意插入iPendant（UT1 :)上的USB接口时，无法格式化USB记忆棒。必须在PC上格式化或插入控制器上的前面板USB接口。

注意不支持具有任何安全或加密功能的USB记忆棒。

注意大于2GB的记忆棒无法格式化FAT16，它们必须格式化为FAT32。

注意通常，设备的尺寸越大，例如USB记忆棒，访问速度和设备性能就越慢。如果可能，USB记忆棒应格式化为FAT16。格式化为FAT32的较大记忆棒将起作用，但在首次插入时，文件操作和记忆棒启动的速度会较慢。

注意要在iPendant上使用UT：设备，您必须安装iPendant上的USB端口选项（J957）。

注意由于USB是一个完善的规格，因此FANUC Robotics America Corporation不具备与机器人配合使用的特定USB记忆棒的资格。 FANUC Robotics使用SanDisk®制造的USB闪存驱动器（CRUZER Mini和Micro）来验证USB接口的操作。因此，FANUC Robotics建议您使用SanDisk®USB闪存盘，无需安全或加密功能。其他驱动器可能正常工作，但未经FANUC Robotics特别认可。

USB Memory StickDevice需要USB接口，这是控制器的标准配置。

USB记忆棒设备可以在控制器上格式化。



警告

控制器正在读取或写入记忆棒时，请勿取下记忆棒。 这样做可能会损坏记忆棒并丢失存储在记忆棒上的所有信息。

Flash File Storage Disk (FR:)

Flash文件存储磁盘是FROM内存的一部分，用作单独的存储设备。 Flash文件存储磁盘（FR :)不需要备用电池来保留信息。 您可以将以下信息存储在Flash文件存储磁盘上：

- 程序

- 系统变量
- 您可以保存为文件的任何内容

您可以格式化Flash文件存储磁盘。 Flash文件存储磁盘的大小由系统在软件安装时设置。 由于FROM的性质，每次将文件复制或保存到FR时：即使使用相同的文件，可用的FR：内存也会有所下降。

RAM Disk (Not for SpotTool+)

RAM磁盘是静态RAM（SRAM）或DRAM内存的一部分，用作单独的存储设备。 任何文件都可以存储在RAM磁盘中。 应将RAM磁盘文件复制到外部设备以进行永久存储。

RAM磁盘（RD :)的位置和大小取决于系统变量$ FILE_MAXSEC的值。 $ FILE_MAXSEC的默认值取决于安装的选项和工具包。

$ FILE_MAXSEC中的值表示为RD分配的内存大小：512字节扇区。 例如，值为-128表示在DRAM中为RD：分配64K内存。

- 如果$ FILE_MAXSEC> 0，则RAM磁盘被定义为在SRAM的PERM池中。 由于RAM磁盘是SRAM的一部分，因此将所有RAM磁盘文件复制到外部设备以进行永久存储，以防止由于电池电量损失或系统软件加载而丢失信息。
- SRAM是电池供电的易失性存储器。 这意味着SRAM中的所有信息（包括程序）都需要备用电池，以便在控制器关闭然后再打开时保留信息。 编写程序时，示教器程序会自动存储在SRAM的TPP池中。

> 警告
> 如果取出电池或丢失电量，或者控制器上装有新的系统软件，SRAM中的数据可能会丢失。 为防止数据丢失，请将所有文件备份或复制到永久存储设备，如FR：或ATA Flash PC存储卡。

- 如果$ FILE_MAXSEC <0，则RAM磁盘定义为DRAM。

  DRAM是非电池备份的易失性存储器。 这意味着DRAM中的所有信息在电源循环之间消失。 实际上，DRAM是一种临时设备。 关闭控制器时，存储在DRAM中的信息会丢失。

> 警告
> 如果关闭控制器或控制器断电，DRAM中的数据将会丢失。 不要将任何想要保存的内容存储在DRAM中的下一个控制器电源循环之外，否则，您将丢失它。

注意易失性意味着断电时内存会丢失。 非易失性存储器不需要电池电量来保留。

您可以将任何文件存储在RAM磁盘上。 RAM磁盘已经为您格式化。

存储在RAM磁盘上的信息可以存储为压缩或未压缩。 默认情况下，信息是压缩的。 如果希望信息保持未压缩状态，则必须使用RDU：设备名称来指示信息将以未压缩的文件格式保存到该设备。

FTP Ethernet Device

如果安装了FTP选项，FTP以太网设备用于将文件从控制器复制到网络PC或工作站。 显示的客户端设备是已定义和启动的客户端设备。 有关更多信息，请参阅“Internet选项设置和操作手册”。

Memory Device (MD:)

存储器设备（MD :)将控制器的程序存储器视为文件设备。 您可以访问控制器中加载的所有示教程序，KAREL程序和KAREL变量。
内存设备是一组设备（MD：，MDB：和可选的FMD :)，提供以下功能：

- MD：提供对用户设置和程序的ASCII和二进制版本的访问
- MDB：提供对用户设置和程序的二进制版本的访问（类似于示教器文件菜单上的“备份 - 以上所有”）
- FMD：提供对用户设置的ASCII版本的访问，并且过滤的程序仅包括用户可设置的信息（例如，不包括由系统更改的内部计时器或时间系统变量），使这些文件对检测用户更改很有用。

Memory Device Binary (MDB:)

内存设备二进制设备（MDB :)允许您复制文件菜单上备份功能提供的相同文件。 这允许您远程备份控制器，例如从SMON，FTP或KCL备份。 例如，您可以使用MDB：设备将所有示教器文件（包括不可见文件）复制到存储卡（KCL>复制MDB：*。tp TO mc :)。

Filtered Memory Device (FMD:)

Filtered Memory Device选项生成用户程序的所有备份文件的文本版本以及手动更改的变量。 包括系统和KAREL变量，位置和数据寄存器，示教器程序和I / O配置数据。
从远程客户端登录机器人FTP服务器时，您将默认为MD：设备。 您可以使用远程FTP客户端中的更改目录服务导航到其他机械手文件设备（FR：，RD：，MC：，MDB：，FMD :)。 在使用cd命令的命令行中，在此示例中，fmd：是正在使用的设备，这可能如下所示：

```
D:\temp>ftp pderob029
Connected to pderob029.frc.com
220 FTP server ready.  [PaintTool Vx.xxP/01]
User <pderob029.frc.com:<none>>:
230 User logged in [NORM].
ftp>cd fmd:
250 CWD command successful.
ftp>
```

您可以将这些文件与先前版本进行比较，以确定哪些用户或运算符已更改。 在没有用户输入的情况下更改的变量和程序将被过滤掉，并将显示在过滤器排除文件中。
安装该选件后，只要从FMD：设备执行控制器的以太网备份，它就会自动运行。 安装“过滤的内存设备”选项后，FR：设备上可能会显示以下任何过滤器排除文件。



警告

请勿删除这些文件，否则过滤器排除数据将会丢失。

- FR:SVAREEG.DT
- FR:KVAREEG.DT
- FR:POSREEG.DT
- FR:REGEEG.DT
- FR:TPLINEEG.DT

您可以通过KCL查看程序，变量或过滤排除文件。 例如：

```
KCL> DIR FMD:*.*.
```

注意执行定期备份的计算机系统可以修改为使用FMD：设备而不是MD：设备进行某些比较操作。 有关更多信息，请联系FANUC Robotics。

FRA:

控制器F-ROM（FRA :)中有一个自动备份的特殊区域。 您不需要外部设备即可使用自动备份，但也可以使用存储卡。 有关更多信息，请参阅特定于应用程序的“设置和操作手册”。

9.3.2 存储文件设备

RAM和F-ROM磁盘使用块分配文件。 每个块是512个字节。

系统变量$ FILE_MAXSEC指定为RAM磁盘分配的块数。 如果指定的数量小于零，则从DRAM分配RAM磁盘。 如果它大于零，则从CMOS RAM分配RAM磁盘。 要更改要为RAM磁盘分配的块数，请从KCL提示符执行以下步骤：

1. 备份RAM磁盘上的所有文件。 有关如何备份文件的更多信息，请参阅相应的应用程序特定的FANUC机器人设置和操作手册中的第8章“程序和文件操作”。
2. 输入DISMOUNT RD：

```
KCL>DISMOUNT RD:
```

3. Enter SET VAR $FILE_MAXSEC

```
KCL>SET VAR
$FILE_MAXSEC = <new value>
```

4. Enter FORMAT RD:

```
KCL>FORMAT RD:
```

执行格式化时，所有文件都将从RAM磁盘中删除。

5. Enter MOUNT RD:

```
KCL>MOUNT RD:
```

在INIT启动时，RAM磁盘将自动重新格式化。

F-ROM磁盘只能从BootROM格式化，因为系统软件也驻留在F-ROM上。 可用块数由系统设置。 硬件支持有限数量的读写周期，因此虽然F-ROM磁盘的功能类似于RAM磁盘，但它不会擦除已删除或覆盖的文件。

经过一些使用后，F-ROM磁盘将耗尽所有块。 此时，需要清除以擦除不再需要的F-ROM块。 有关清除的更多信息，请参阅附录A中的PURGE_DEV，“KAREL语言按字母顺序描述”或附录C中的PURGE命令，“KCL命令按字母顺序说明”。

有关内存的更多信息，请参见第1.4.1节。

9.3.3 虚拟设备

KAREL虚拟设备类似于DOS子目录。 例如

- 在DOS中，要访问子目录中的文件，您将查看FR：\ FR1：\> test.kl。
- 在KAREL中，要访问虚拟设备中的同一文件，您将查看FR1：test.kl。

控制器支持7个虚拟设备。 标识虚拟设备的编号将附加到设备名称（FR 1 :)。 表9-2显示了一些可用的有效虚拟设备。

表9-2. 虚拟设备

设备名称	实际存储
RD:	RAM disk
FR:	F-ROM disk - compressed and uncompressed files
MF:	Refers to files on both RD: and FR:
RD1: - RD7:	RAM disk - compressed and uncompressed files

表9-2. 虚拟设备（续）

FR1: - FR7:	F-ROM disk - compressed and uncompressed files
MF1: - MF7:	Refers to files on both the RAM disk and F-ROM disk of the respective virtual device

虚拟设备规则

以下规则适用于虚拟设备。

- 虚拟设备上的文件名是唯一的。 RAM或F-ROM磁盘上可能存在文件，但不能同时存在。例如：RD：test.kl和FR：test.kl不能同时存在。
- 可以跨虚拟设备复制文件名。例如：RD：test.kl，RD1：test.kl和FR2：test.kl都可以存在。
- 当实际存储设备未知时，MF：设备名称可用于任何文件操作以查找虚拟设备上的文件。例如：MF：test.kl找到RD：test.kl或FR：test.kl。
- 当您使用MF：设备作为存储设备时，默认情况下，当RD：处于CMOS且$ FILE_MAXSEC> 0时使用RAM磁盘。当RD：位于DRAM和$ FILE_MAXSEC时，默认使用F-ROM磁盘。 <0。例如：KCL> COPY FILE FLPY：test.kl到MF2：文件实际上存在于RD2上：
- 列出MF：设备目录时，将列出RAM和F-ROM磁盘上的所有文件。但是，仅显示指定虚拟设备中的文件。
- 如果指定了RD5：目录而不是MF5：，则仅列出虚拟设备5中RAM磁盘上的那些文件。如果指定了FR3：目录，则仅列出虚拟设备3中F-ROM磁盘上的那些文件。例如：KCL> DIR RD5：
- 可以将文件从一个虚拟设备复制到另一个虚拟设备。如果虚拟设备不同，也可以将文件从RAM磁盘复制到F-ROM磁盘，反之亦然。例如：KCL> COPY RD1：test.kl到FR3：
- 文件只能在虚拟设备中重命名，并且只能在同一设备上重命名。例如：KCL> RENAME FR2：test.kl FR2：example.kl
- 使用与复制不同的特殊命令，可以在虚拟设备中将文件从RAM磁盘移动到F-ROM磁盘，反之亦然。例如：KCL> MOVE MF1：test.kl将test.kl从F-ROM磁盘移动到RAM磁盘。 KCL> COPY FR1：test.kl TO RD1：test.kl还将文件从F-ROM磁盘移动到RAM磁盘。这是因为唯一文件名只能存在于一个设备上。有关移动文件的更多信息，请参阅附录A中的MOVE_FILE内置，“KAREL语言按字母顺序描述”或附录C“KCL命令按字母顺序说明”中的MOVE FILE命令。
- 格式化RAM磁盘RD：或MF：，清除所有虚拟设备上的所有RAM磁盘文件。 F-ROM磁盘上的文件保持不变。例如：**KCL> FORMAT** RD1：重新格式化所有RAM磁盘虚拟设备（RD：到RD7 :)。重新格式化将导致删除现有数据。

- 清除将清除所有虚拟设备不再需要的所有块。 有关清除的更多信息，请参阅附录A中的PURGE_DEV内置“KAREL语言字母说明”或附录C“KCL命令字母说明”中的PURGE命令。

9.3.4 文件通道

PIP：设备允许您访问任意数量的通道文件。此访问权限是控制器内存中的文件。这意味着访问这些文件非常有效。文件大小和文件数受可用控制器内存的限制。这意味着文件通道的最佳用途是缓冲数据或暂时保存数据。

该文件类似于水管，其中数据通过写入任务被注入一端，并且数据从另一端流出到读取任务中。这就是使用的术语是通道的原因。这个概念与在UNIX，Windows和Linux上实现的通道设备非常相似。

通道设备上的文件大小有限，但数据排列在循环缓冲区中。这也称为循环队列。这意味着大小为8kbytes的文件通道（这是默认大小）将包含写入通道的最后8k数据。当用户将第九千字节的数据写入通道时，第一个千字节将被覆盖。

由于通道实际上用于将数据从一个地方传输到另一个地方，因此某些应用程序将从通道中读取数据。在默认模式下，阅读器将等待，直到写入信息。一旦数据在通道中可用，读取将完成。 KAREL应用程序可能使用BYTES_AHEAD来查询通道以获取可读取的数据量。这是默认的读取模式。

提供第二种读取模式，称为“快照”。在此模式下，阅读器将读出通道的当前内容。一旦读取了当前内容，阅读器就会收到文件的结尾。这可以应用于诸如“飞行记录器”之类的应用中。这允许您记录导致事件（例如错误）的信息，然后检索写入通道的最后一组调试信息。快照模式是一个读取属性。它使用内置的SET_FILE_ATTR进行配置。默认情况下，读取操作不处于快照模式。

典型的通道应用程序涉及一个过程将数据写入通道。数据可以调试信息，过程参数或机器人位置。然后，另一个应用程序可以从通道中读取数据。读取应用程序可以是KAREL程序，它正在分析来自通道的数据，或者它可以是KCL或Web服务器读取数据并以ASCII形式显示给用户。

KAREL的例子

以下内容适用于KAREL示例。

- 两个KAREL任务可以通过通道共享数据。 一个KAREL任务可以将数据写入通道，而第二个KAREL任务从通道读取。 在这种情况下，文件属性ATR_PIPWAIT可用于从通道读取的任务。 在这种情况下，读取KAREL任务将等待读取功能，直到写入任务完成数据写入。通道的默认操作是在没有要从通道读取的数据时返回文件的结尾。

- KAREL 应用程序可能正以非常快的数据速率执行条件处理程序。 在这种情况下，条件处理程序例程将数据写入示教器显示屏幕可能是不可行的，因为这会干扰条件处理程序的性能。 在这种情况下，您可以从条件处理程序例程将数据写入PIP：device。 另一个KAREL任务可能会从PIP：设备读取数据并将其显示到示教器。 在这种情况下，示教器显示器不是严格实时的。 在这种情况下，PIP：设备充当缓冲区，以便条件处理程序可以继续执行其主要功能，而无需等待显示完成。 您还可以在应用程序写入KCL的同时从KCL键入文件。

PIP：设备在以下方面与其他设备类似：

- 通道设备类似于RD：设备。 TheRD：设备还包括系统内存中的文件内容。 PIP设备的不同主要是因为通道文件可以同时打开以进行读写。
- 与MC：和FR：设备类似，当您要调试或诊断实时软件时，使用PIP：设备。这允许您输出可以轻松查看的调试信息，而不会干扰编写调试数据的操作。这也允许一个任务写入另一个任务可以读取的信息。
- **PIP：**设备的功能类似于控制器上的所有其他设备。这意味着此设备支持所有文件I / O操作。支持所有I / O功能，除以下内容外，其他功能相同：Chdir，Mkdir和Rmdir。
- **PIP：**设备类似于直接写入存储卡。但是，写入存储卡会延迟写入任务，而PIP：设备的延迟要小得多。这意味着控制器上的任何代码都可以使用此设备。它还具有通过电源循环保留数据的能力。

PIP规则：设备

以下规则适用于PIP：设备：

- PIP：设备可供任何应用程序使用，或者您可以指定相关的常用选项，例如KAREL。
- 设备是可配置的。 您可以配置它使用的内存量以及该内存是CMOS（保留）还是DRAM（未保留）。 您还可以配置数据格式以读取格式化的ASCII类型数据。 该器件通过内置的PIPE_CONFIG进行配置。

安装，设置和操作顺序

通常，PIP：设备像任何其他设备一样运行。 典型的操作顺序包括:

```
OPEN myfile ('PIP:/myfile.dat', 'RW',)
Write myfile ('Data that I am logging', CR)
```

```
Close myfile
```

如果您希望能够从Web服务器访问myfile.dat，请在诊断Web页面上添加指向它的链接。

PIP：设备上的文件是可配置的。 默认情况下，通道配置在$ PIPE_CONFIG系统变量中指定。 表9-3中列出的字段具有以下含义：

表9-3. 系统变量字段描述

字段	**默认**	**定义**
$sectors	8	通道中1024字节扇区的数量。
$filedata		指向实际通道数据的指针（不可访问）。
$recordsize	0	二进制记录大小，零表示未跟踪。
$auxword	0	字典元素，如字典格式或类型校验和。
$memtyp	0	如果非零使用CMOS。
$format	Undefined	格式化模式：未定义，函数，格式字符串或KAREL类型。
$formatter		函数指针，“C”格式说明符指针或类型代码取决于$格式。

每个通道文件都可以通过内置的pipe_config进行配置。 在打开通道文件进行写入之前，将调用内置的pipe_config。 有关更多详细信息，请参见第A.17节“内置pipe_config”。

操作范例

以下示例将数据从一个KAREL例程写入通道，然后从另一个例程读回。 可以从单独的任务调用这些例程，以便一个任务正在编写数据，而另一个任务则可以读取数据。

程序

```
program pipuform
%nolockgroup
var
     pipe, in_file, mcfile, console:file
     record: string[80]
     status: integer
     parm1, parm2: integer
     msg: string[127]
```

```
      cmos_flag: boolean
      n_sectors: integer
      record_size: integer
      form_dict: string[4]
      form_ele: integer
--
--初始化文件属性
 routine file_init
(att_file :FILE) begin
 set_file_atr(att_file, ATR_IA)   --force reads to completion
 set_file_atr(att_file, ATR_FIELD)  --force write to completion
 set_file_atr(att_file, ATR_PASSALL) --ignore cr
 set_file_atr(att_file, ATR_UF)   --binary
end file_init
routine write_pipe
begin
 --file is opened
 file_init (pipe)
 open file pipe ('rw', 'pip:example.dat')
 status = io_status(pipe)
 write console ('Open pipe status:',status,cr)
--  write extra parameters to pipe
 write pipe (msg::8)
 status = io_status(pipe)
end write_pipe
routine read_pipe
var
       record: string[128]
       status: integer
       entry: integer
       num_bytes: integer
begin
 file_init (in_file)
 open file in_file ('ro', 'pip:example.dat')
 BYTES_AHEAD(in_file, entry, status)
 status = 0
 read in_file (parm1::4)
 status = IO_STATUS(in_file)
 write console ('parm1 read',status,cr)
 write console ('parm1',parm1,cr)
 read in_file (parm2::4)
 status = IO_STATUS(in_file)
 write console ('parm2 read',parm2,status,cr)
end read_pipe
begin
       SET_FILE_ATR(console, atr_ia, 0) --ATR_IA is defined in flbt.ke
       OPEN FILE console ('RW','CONS:')
```

```
      if(uninit(msg)) then
        msg = 'Example'
      endif
      if(uninit(n_sectors)) then
      cmos_flag = true
      n_sectors = 16
      record_size = 128
      form_dict = 'test'
      form_ele = 1
     endif
     --           [in] pipe_name: STRING;name of tag
     --           [in] cmos_flag: boolean;
     --           [in] n_sectors: integer;
     --           [in] record_size: integer;
     --           [in] form_dict: string;
     --           [in] form_ele: integer;
     --           [out] status: INTEGER
     pipe_config('pip:example.dat',cmos_flag, n_sectors,
            record_size,form_dict,form_ele,status)
    write_pipe
    read_pipe
    close file pipe
    close file in_file
end pipuform
```

9.4 文件访问

您可以使用CRT / KB上的FILE和SELECT屏幕或示教器或使用KAREL语言语句来访问文件。 在正常操作期间，文件将自动加载到控制器中。 但是，可能需要执行其他功能。

9.5 格式化XML输入

9.5.1 概述

此功能允许KAREL程序通过XML（扩展标记语言）格式的文本文件输入数据。 XML而不是二进制格式允许在PC上轻松操作文件。

XML文件必须遵循最基本的XML语法要求。 这些要求是：

- XML 文件只能包含一个顶级元素。
- 开始标记必须具有匹配的结束标记。
- 空标记可以表示为<tag parameters />
- 标签不能包含特殊字符，例如 *，$ 和 [] 的集合
- 它们不得包含不可打印的字符
- 属性必须是attr = “value” 的形式
- 特殊字符用于以下（标签之外）：
  - <替换为＆lt;
  - <替换为＆gt;
  - ＆替换为＆amp;
  - “可以用” 代替
- 此功能提供 XML 解析器以及 KAREL 和C程序员轻松从 XML 文件中的文本信息中提取二进制数据的方法。 它不需要应用程序对 XML 文件进行任何解析。

注意XML文件只能有一个顶级元素。 例如，

```
<GRID>
  <TPPROG>
  </TPPROG>
</GRID>
```

是合法的。 它有一个顶级元素（GRID）。

```
<GRID>
</GRID>
<TPPROG>
</TPPROG>
```

不合法。 例如，主标签可用于区分GRID文件和密码配置文件。

9.5.2 安装顺序

此功能包含KAREL 内置插件，可为KAREL 用户提供对此库的访问。 环境文件xml.ev必须位于转换程序路径上，以转换引用这些内置函数的KAREL程序。 这些内置函数是XML_ADDTAG，XML_GETDATA，XML_REMTAG，XML_SCAN和XML_SETVAR。 有关这些内置插件的更多信息，请参阅**附录A.**

9.5.3 示例KAREL程序引用XML文件

- 解析xml_name引用的XML文件，并将该文件中的设置返回给xmlstruct。
- 属性名称 - 值对在attrnames和attrvalues中作为字符串返回。它不是 - 需要将XML文件中的数据设置为某些应用程序中的结构 - 名称 - 值对 - 直接使用。
- 最有效的XML实现使用许多名称 - 值对，只使用少量标记。从标签返回一个名称 - 值对需要相同的时间，因为它需要32对。 32个标签需要32倍的时间。
- 支持的最大对数为32。
- 有两种不同类型的XML文件。图1-1和图1-2说明了两种类型的标记结构
- 对于分离的开始和结束标记（图1-1），必须在XML_START返回码上完成标记处理。
- 对于组合的开始和结束标记（图1-2），您无法在标记内提供任何文本。 KAREL XML处理提供了在需要时提取此文本的方法。
- 对于组合的开始和结束标记（图1-2），标记处理必须在XML_END返回代码上完成。
- XML_START返回代码需要设置一个标志，指示标记已被处理。
- XML_END返回代码需要检查是否已在启动代码上完成处理并重置标志。

图9-1. XML文件1分隔的开始和结束标记

```
<?xml version="1.0" ?><!-- This is a comment -- >
<xmlstrct_t first="123456" second="7.8910" third="1" fourth="A string">
Text assocated with xmlstrct_t tag
</xmlstrct_t>
```

图9-2. XML文件2组合的开始和结束标记

```
<?xml version="1.0" ?><!-- This is a comment -- >
<xmlstrct_t first="78910"  second="12.3456" third="0" fourth="A string"/>
```

图9-3. XML文件3未注册或处理GRID标记

```
<?xml version="1.0" ?>-<!—comment
<GRID rows="16" cols="24" scale="80">
  <xmlstrct_t first=”123456” second=”78910” third="1” "fourth=”A String”>
              special characters < > & "
  </xmlstrct_t>
</GRID>
```

GRID标记可以在XML文件中，但不由此示例程序处理。 通常，XML标签可以由不同的软件处理。 信息仅返回给KAREL程序，用于由KAREL程序注册的标签。

图 9–4. KAREL 程序

```
PROGRAM xmlparse
%COMMENT = 'XML Parse'
%NOPAUSESHFT
%NOPAUSE = ERROR + TPENABLE + COMMAND
%NOABORT = ERROR + COMMAND
%NOLOCKGROUP
%NOBUSYLAMP
%ENVIRONMENT xml
%include klerxmlf
CONST
MYXML_CONST = 3
TYPE
  xmlstrct_t = STRUCTURE
    first: INTEGER
    second: REAL
    third: BOOLEAN
    fourth: STRING[20]
  ENDSTRUCTURE
VAR
  xml_name    : string[20]
  tag_name    : string[32]
  text        : array[32] of string[128]
  attrnames   : array[32] of string[32]
  attrvalues  : array[32] of string[64]
  xml_file    : FILE
  status      : INTEGER
  xmlstrct:   xmlstrct_t
  tag_ident:  integer
  func_code:  integer
  text_idx:  integer
  numattr:    integer
  textdone:   BOOLEAN
  done:   BOOLEAN
  console:   FILE
  startdata: BOOLEAN
--------------------------------------------------------------------------------
--
--
-- 有两种类型的XML文件结构。 在一个中，结束标记嵌入在另一个中的开始标记中，结束标记是分开的。 正确的解析器必须处理两个标记结构
```

```
--
-- For the case that the end tag is separate from the start tag
-- (Figure 1.1)the following writes show the sequence of returns:
--Scanned (Rev D) xmlstrct_t   3   100   129015
--Start Tag processing...
--Scanned (Rev D) xmlstrct_t   3   101   129015
--End Tag
--Processed at start tag...
--Scanned (Rev D) xmlstrct_t   0   101   0
--
--For the case where the end tag and start tag are together (Figure 1.2)
--the following writes show the sequences of returns:
--Scanned (Rev D) xmlstrct_t   3   101   129015
--End Tag
--End Tag processing...
--Scanned (Rev D) xmlstrct_t   0   101   0
--
BEGIN
 SET_FILE_ATR(console, ATR_IA, 0) -- ATR_IA is defined in flbt.ke
  OPEN FILE console ('RW', 'CONS:')
  IF UNINIT(xml_name) THEN
    xml_name = 'mc:kl16004.xml'
  ENDIF
  SET_FILE_ATR (xml_file, ATR_XML)       -- XML
  CLR_IO_STAT(xml_file)
  OPEN FILE xml_file ('RO', xml_name)     -- Open does new operation
  status = IO_STATUS(xml_file)
  IF status <> 0 THEN
     POST_ERR(status, '', 0, 0)
     abort
  ENDIF
  xml_addtag(xml_file, 'xmlstrct_t', 32, FALSE, MYXML_CONST, status)
  textdone = TRUE
  done = FALSE
  startdata=FALSE
  WHILE (done = FALSE) DO
    xml_scan(xml_file, tag_name, tag_ident, func_code, status)
    if(status = 0) THEN
      done= TRUE
    ENDIF
   WRITE console ('Scanned (Rev D) ', tag_name,'  ', tag_ident, '  ',
                  func_code, '  ',STATUS,'  ', CR)
    IF (status = XML_FUNCTION) THEN
      status = 0
      SELECT tag_ident OF
      CASE (MYXML_CONST) :
        SELECT func_code OF
```

```
        CASE (XML_START) :
        WRITE console ('Start Tag processing...', CR)
          text_idx = 1
          xml_setvar(xml_file, 'kl16004', 'xmlstrct', status)
        -- Already looked at the attribtues get the text
         xml_getdata(xml_file, numattr, attrnames, attrvalues,
                     text[text_idx], textdone, status)
         startdata = TRUE
        CASE (XML_STEND) :
          -- This tag is never returned
          WRITE console ('StEnd Tag', CR)
        CASE (XML_END) :
        WRITE console ('End Tag', CR)
        if(startdata = TRUE) THEN
          startdata=FALSE
          WRITE console ('Processed at start tag...', CR)
       ELSE
          WRITE console ('End Tag processing...', CR)
          text_idx = 1
          xml_setvar(xml_file, 'kl16004', 'xmlstrct', status)
        -- Already looked at the attribtues get the text
         xml_getdata(xml_file, numattr, attrnames, attrvalues,
                     text[text_idx], textdone, status)
        ENDIF
        CASE (XML_TXCONT) :
        -- Usually the user will do one or the other but not both of
        -- these calls
         text_idx = text_idx + 1
         xml_getdata(xml_file, numattr, attrnames, attrvalues,
                     text[text_idx], textdone, status)
        ELSE:
        ENDSELECT
      ELSE:
      ENDSELECT
    ELSE
      IF(status <> XML_SCANLIM) THEN
       POST_ERR(status, '', 0, 0)
       done = TRUE
      ENDIF
    ENDIF -- Good status from xml_parse
  ENDWHILE
  -- This is not required but allows the user to dynamically remove
  -- and add tags
  xml_remtag(xml_file, 'xmlstrct_t', status)
  CLOSE FILE xml_file
  status = IO_STATUS(xml_file)
  IF status <> 0 THEN
```

```
      POST_ERR(status, '', 0, 0)
  ENDIF
END xmlparse
```

执行此程序将从XML文件中提取第一，第二，第三和第四属性及其值。这些值将在变量xmlstruct中设置，该变量具有第一，第二，第三和第四个字段。字符串变量也将设置为KAREL字符串变量。

9.5.4 解析错误

XML_TAG_SIZE "标记太长"

XML_ATTR_SIZE "属性太长"

XML_NOSLASH "无效使用/字符"

XML_INVTAG "标记中的字符无效"

XML_ATTRMATCH "没有属性值"

XML_TAGMATCH "没有匹配开始的结束标记"

XML_INVATTR "属性中的字符无效"

XML_NOFILE "无法找到文件"

XML_TAGNEST "标记嵌套级别太深"

XML_COMMENT "注解错误"

XML_BADEXCHAR "未知字符＆xxx;”

XML_TAGNFND "标签未找到"

XML_INVEOF "意外的文件结束"

XML_SCANLIM "超出扫描限制"

注意XML_SCANLIM表示文件太长而无法在一个请求中处理。 此错误的补救措施是重新调用XML扫描例程，如示例中所示。

XML_FUNCTION "功能码返回"

9.6 存储设备

存储设备（MD :)将控制器内存程序和可变内存视为文件设备。 示教程序，KAREL程序，程序变量，SYSTEM变量和错误日志被视为单个文件。 这为通信设备以及普通文件设备提供了扩展功能。 例如：

1. FTP可以通过将PC文件复制到MD：设备来加载它。
2. 可以通过从MD：设备复制来远程检索和分析错误日志。
3. 通过从MD：设备复制***.LS，可以获得示教程序的ASCII列表。
4. 可以通过从MD：device复制SYSVARS.VA来获取系统变量的ASCII列表。

有关MD设备上可用文件的列表和说明，请参阅表9-4。

表9-4. MD设备的文件列表

文件名	描述
ACCNTG.DG	此文件显示操作系统任务的系统报告。
ACCOFF.DG	此文件显示系统报告已关闭。
AXIS.DG	该文件显示轴和伺服状态。
CONFIG.DG	此文件显示系统配置的摘要
CONSLOG.DG	此文件是系统控制台日志的ASCII列表。
CONSTAIL.DG	此文件是系统控制台日志最后几行的ASCII列表。
CURPOS.DG	此文件显示当前机器人位置。
*.DF	该文件包含TP编辑器的默认设置。
DIOCFGSV.IO	此文件包含二进制形式的I / O配置信息。
DIOCFGSV.VA	此文件是DIOCFGSV.IO的ASCII列表。
ERRACT.LS	此文件是活动错误的ASCII列表。
ERRALL.LS	此文件是错误日志的ASCII列表。
ERRAPP.LS	此文件是应用程序错误的ASCII列表。
ERRCOMM .LS	此文件显示通信错误。
ERRCURR.LS	此文件是系统配置的ASCII列表。
ERRHIST.LS	此文件是系统配置的ASCII列表。
ERRMOT.LS	此文件是动作错误的ASCII列表。
ERRPWD.LS	此文件是密码错误的ASCII列表。

表9-4. MD设备的文件列表（续）

文件名	描述
ERRSYS.LS	此文件是系统错误的ASCII列表。
ETHERNET	此文件显示以太网配置。
FRAME.DG	此文件显示坐标系分配。
FRAMEVAR.VR	该文件包含二进制形式的系统框架和工具变量信息。
FRAMEVAR.VA	此文件是FRAMEVAR.VR的ASCII列表。
HIST.LS	此文件显示历史记录寄存器转储。
HISTE.LS	此文件是一般故障异常的ASCII列表。
HISTP.LS	此文件是powerfail异常的ASCII列表。
HISTS.LS	该文件是伺服异常的ASCII列表。
IOCONFIG.DG	此文件显示IO配置和分配。
IOSTATE.DG	此文件位于I / O点状态的ASCII列表中。
IOSTATUS.CM	此文件是用于还原I / O的系统文件。
LOG CONSTAIL.DG	此文件是控制台日志的最后一行。
NUMREG.VA	此文件位于NUMREG.VR的ASCII列表中。
NUMREG.VR	该文件包含系统数字寄存器。
MACRO.DG	此文件显示宏分配。
MEMORY.DG	此文件显示当前内存使用情况。
PORT.DG	此文件显示串行端口配置。
POSREG.VA	此文件位于POSREG.VR的ASCII列表中。
POSREG.VR	该文件包含系统位置寄存器信息。
PRGSTATE.DG	该文件是程序状态的ASCII列表。
RIPELOG.DG	此文件包含详细的状态信息，例如机器人开启和脱机的时间以及其他诊断数据。 有关更多信息，请参阅“Internet选项手册”。
RIPESTAT.DG	此文件包含用于确定网络性能的性能数据。 有关更多信息，请参阅“Internet选项手册”。

表9-4. MD设备的文件列表（续）

文件名	描述
SFTYSIG.DG	该文件是安全信号状态的ASCII列表。
STATUS.DG	此文件显示系统状态的摘要
SUMMARY.DG	此文件显示诊断摘要
SYCLDINT.VA	此文件是在冷启动时初始化的系统变量的ASCII列表。
SYMOTN.VA	此文件是运动系统变量的ASCII列表。
SYNOSAVE.VA	此文件是未保存的系统变量的ASCII列表。
SYSFRAME.SV	该文件包含$ MNUTOOL，$ MNUFRAME，$ MNUTOOLNUM和$ MNUFRAMENUM。在V7.20之前的版本中，这些变量位于SYSVARS.SV中。
SYSMACRO.SV	此文件是系统宏定义的列表。
SYSMACRO.VA	该文件是SYSMACRO.SV的ASCII列表。
SYSMAST.SV	该文件是系统主控信息的列表。
SYSMAST.VA	该文件是SYSMAST.SV的ASCII列表。
SYSSERVO.SV	该文件是系统伺服参数的列表。
SYSSERVO.VA	该文件是SYSSERVO.SV的ASCII列表。
SYSTEM.DG	此文件显示系统信息的摘要
SYSTEM.VA	此文件是非运动系统变量的ASCII列表。
SYSVARS.SV	此文件是系统变量的列表。
SYSVARS.VA	该文件是SYSVARS.SV的ASCII列表。
SYS****.SV	此文件包含特定于应用程序的系
SYS****.VA	此文件是SYS ****。VA的ASCII列表。
TASKLIST.DG	此文件显示系统任务信息。
TESTRUN.DG	此文件显示Testrun状态。
TIMERS.DG	此文件显示系统和程序计时器状态。
TPACCN.DG	此文件显示TP授权状态。
VERSION.DG	此文件显示系统，软件和伺服版本信息。
***.PC	该文件是KAREL二进制程序。
***.VA	此文件是KAREL变量的ASCII列表。
***.VR	该文件包含二进制形式的KAREL变量。
***.LS	此文件是示教程序的ASCII列表。
***.TP	该文件是示教二进制程序。

表9-4. MD设备的文件列表（续）

文件名	描述
***.TX	这个文件是一个字典文件。
***.HTM	此文件是HTML网页。
***.STM	此文件是使用iPendant Control或Server Side Include的HTML网页。
***.GIF	该文件是GIF图像文件。
***.JPG	该文件是JPEG图像文件。

有关使用MD：设备时的限制列表，请参阅表9-5。

表9-5. 使用MD：设备时测试限制

文件名或类型	读	写	删除	注释
***.DG	YES	NO	NO	诊断文本文件。
***.PC	NO	YES	YES	
***.VR	YES	YES	YES	没有参考文献的限制。
***.LS	YES	NO	NO	
***.TP	YES	YES	YES	
***.LS	YES	NO	NO	
FFF.DF	YES	YES	NO	
SYS***.SV	YES	YES	NO	仅在CTRL START时写入。
SYS***.VA	YES	NO	NO	
ERR***.LS	YES	NO	NO	
HISTX.LS	YES	NO	NO	
***REG.VR	YES	YES	NO	
***REG.VA	YES	NO	NO	
DIOCFGSV.IO	YES	YES	NO	仅在CTRL START时写入。
DIOCFGSV.VA	YES	NO	NO	

第10章

字典和表单

目录

第10章 字典和表单 10–1
10.1 概述 10–3
10.2 创建用户字典 10–3
10.2.1 字典语法 10–3
10.2.2 字典元素编号 10–4
10.2.3 字典元素名称 10–5
10.2.4 字典光标定位 10–5
10.2.5 字典元素文本 10–6
10.2.6 字典保留字命令 10–8
10.2.7 字符代码 10–10
10.2.8 嵌套字典元素 10–10
10.2.9 字典注释 10–11
10.2.10 生成KAREL常量文件 10–11
10.2.11 在控制器上压缩和加载字典 10–11
10.2.12 从KAREL程序访问字典元素 10–12
10.3 创建用户表单 10–13
10.3.1 表单语法 10–14
10.3.2 表单属性 10–15
10.3.3 表格标题和菜单标签 10–16
10.3.4 表单菜单文本 10–17
10.3.5 表单可选菜单项 10–18
10.3.6 编辑数据项 10–19
10.3.7 使用树视图的动态表单 10–26
10.3.8 不可选文本 10–27
10.3.9 仅显示数据项 10–27
10.3.10 光标位置属性 10–27
10.3.11 形成保留字和字符代码 10–28
10.3.12 表单功能键元素名称或编号 10–30
10.3.13 使用变量的表单功能键 10–31
10.3.14 表单帮助元素名称或编号 10–31

10.3.15 竹早噪词卖展干 10–32
10.3.16 CRT/KB 表单屏幕 10–32
10.3.17 表单文件命名规定 10–33
10.3.18 在控制器上压缩和加载表单 10–34
10.3.19 显示表单 10–36

10.1 概述

字典和表单用于在示教器和CRT / KB屏幕上使用KAREL程序创建操作员界面。

本章包含有关的信息

- 创建用户词典文件，请参见第10.2节。
- 创建和使用表单，请参阅第10.3节。

在这两种情况下，屏幕的文本和格式都存在于KAREL程序之外。 这样可以在不改变KAREL程序的情况下轻松修改屏幕。

10.2 创建用户字典

字典文件提供了一种自定义显示文本的方法，包括文本属性（闪烁，下划线，双宽等）和屏幕上的文本位置，而无需重新翻译程序。
以下是使用词典的步骤。

1. 创建带有.UTX文件扩展名的格式化ASCII字典文本文件。
2. 使用KCL COMPRESS DICT命令压缩字典文件。 这将创建一个带有.TX扩展名的可加载字典文件。 压缩后，可以从系统中删除.UTX文件。 仅加载压缩字典（.TX）文件。
3. 使用KCL LOAD DICT命令或内置的KAREL ADD_DICT加载压缩的字典文件。
4. 使用KAREL词典内置函数显示词典文本。 有关更多信息，请参见第10.2.12节“从KAREL程序访问字典元素”。

当每个机器人上显示的文本略有不同时，字典文件对于在不同机器人上运行相同程序非常有用。 例如，如果程序仅在一个机器人上运行，则可以使用KAREL WRITE语句。 但是，通过允许创建使用相同程序显示文本的多个字典文件，使用字典文件简化了许多机器人上文本的显示。

注意字典文件在多语言程序中很有用。

10.2.1 字典语法

用户词典的语法由一个或多个词典元素组成。 字典元素具有以下特征：

- 字典元素可以包含多行信息，直观的信息屏幕。 用户词典文件具有以下语法：

```
<*comment>
  $n<,ele_name><@cursor_pos><&res_word><#chr_code><"Ele_text"><&res_wor d>
  <#chr_code><+nest_ele>
  <*comment>
  <$n+1>
```

- 括号<>中的项目是可选的。
- * comment是以*开头的任何项目。 行末尾的所有文本都将被忽略。 请参阅第10.2.9节。
- $ n指定元素编号。 n是0或更大的正整数。 请参阅第10.2.2节.-，ele_name指定逗号后跟元素名称。 请参阅第10.2.3节。
- @cursor_pos指定一个光标位置（用逗号分隔的两个整数。）光标位置从@ 1,1开始。 请参阅第10.2.4节。
- &res_word指定字典保留字。 请参阅第10.2.6节。
- “Ele_text”指定要显示的元素文本。 请参阅第10.2.5节。
- + nest_ele指定下一个字典文本。 请参阅第10.2.8节。

- 字典元素不必全部驻留在一行上。 在任何地方插入回车符，但在带引号的文本中。 带引号的文本必须以相同的行开头和结尾。
- 字典元素可以包含文本，位置和显示属性信息。 表10-2列出了字典元素的属性。

10.2.2 字典元素编号

字典元素编号标识字典元素。 字典元素以“$”开头，后跟元素编号。 元素编号具有以下特征：

- 元素编号从0开始并按顺序继续。
- 如果跳过元素编号，字典压缩器将为每个跳过的数字添加5个字节的额外开销。 因此，您不应该跳过大量的数字。
- 如果要使用字典按顺序自动生成元素编号，请使用“-”代替数字。 在以下示例中，“-”等同于元素编号7。

  $1

  $2

$3

$6

$-

10.2.3 字典元素名称

每个字典元素都可以有一个可选的元素名称。 名称通过逗号和零个或多个空格与元素编号分隔。 元素名称区分大小写。 只有前12个字符用于区分元素名称。

以下是元素名称的示例：

$1, KCMN_SH_LANG

$2, KCMN_SH_DICT

字典元素可以通过名称而不是数字来引用其他元素。 此外，元素名称可以是KAREL包含文件中的常量。

10.2.4 字典光标定位

字典元素显示在从光标当前位置开始的指定窗口中。 在大多数情况下，将光标移动到特定位置并开始在那里显示字典元素。

- 光标位置属性“@”用于将光标移动到窗口内的屏幕。
- “@”符号后跟两个用逗号分隔的数字。 第一个数字是窗口行，第二个数字是窗口列。
  例如，在示教器上，“t_fu”窗口从“t_sc”屏幕的第5行开始，高10行，宽40列。
  - 光标位置''@ 1,1''是“t_fu”窗口的左上位置，位于“t_sc”屏幕第5行第1列。
  - “t_fu”窗口的右下位置是“@ 10.40”并且位于“t_sc”屏幕第15行第40列。
- 有关示教器屏幕和窗口的更多信息，请参阅第7.10.1节。
- 例如，在CRT / KB上，“c_fu”窗口从“c_sc”屏幕的第5行开始，高17行，宽80列。

- 光标位置''@ 1,1''是“c_fu”窗口的左上角位置，位于“c_sc”屏幕第5行。
- 窗口的右下位置是“@ 17,80”，位于“c_sc”屏幕第21行，第80列。

有关CRT / KB屏幕和窗口的更多信息，请参见第7.10.2节。 窗口大小定义字典元素的显示限制。

10.2.5 字典元素文本

元素文本或引用文本是您要在屏幕上显示的信息。

- 元素文本必须用双引号字符''''括起来。
- 要在文本中插入反斜杠，请使用\\（双反斜杠）。
- 要在文本中插入双引号，请使用\“（反斜杠，引用。）
- 多个项目文本可以驻留在字典元素中，由保留字分隔。 有关更多信息，请参见第10.2.6节。
- 要在元素文本中包含KAREL变量的值，请使用KAREL内置函数。 WRITE_DICT_V和READ_DICT_V，用于传递变量的值。
- 要标识要插入KAREL变量的位置，请在文本中使用格式说明符。
- 格式说明符是字符“%”，后跟一些可选字段，然后是转换字符。 格式说明符具有以下语法：

```
%<-><+><width><.precision>conversion_character<^argument_numbe r>
```

说明符格式

- <>中包含的项目是可选的。
- 表示左对齐显示的值。
- 如果参数是数字，+符号总是表示符号。
- width字段是一个数字，表示字段应占用的最小字符数。
- .precision就是。 字符后跟一个数字。 它具有特定含义，具体取决于转换字符：
- conversion_characters标识正在传递的参数的数据类型。 它们列于表10-1中。
- ^ argument_number是^（up-caret字符）后跟一个数字。

转换字符

转换字符用于标识传递的KAREL变量的数据类型。 表10-1列出了转换字符：

表 10–1. 转换字符

字符	参数类型:打印为
d	INTEGER; 十进制数。
o	INTEGER; 无符号八进制表示法（没有前导零）。
x, X	INTEGER; 无符号十六进制表示法（没有前导0x或0X），使用abcdef或ABCDEF表示10，...，15。
u	INTEGER; 无符号十进制表示法。
s	STRING; 从字符串打印字符直到字符串结尾或精度给出的字符数。
f	REAL; 形式< - > mmm.dddddd的十进制表示法，其中d's的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
e, E	REAL; 形式< - > mmm.dddddd的十进制表示法，其中d's的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
g, G	REAL; 如果指数小于-4或大于或等于精度，则使用%e或%E; 否则使用%f。 不打印尾随零和尾随小数指针。
%	没有参数被转换; 打印%。

- 字符d，o，x，X和u可与INTEGER，SHORT，BYTE和BOOLEAN数据类型一起使用。 BOALAN数据类型显示为0表示FALSE，1表示TRUE。
- f，e，E，g和G字符可与REAL数据类型一起使用。
- 字符s用于STRING数据类型。

> **警告**
>
> 确保对传递的参数类型使用正确的转换字符。 如果转换字符和参数类型不匹配，则可能会出现意外结果。

宽度和精度

可选的宽度字段用于固定显示的变量占用的最小字符数。 这对于显示数字列很有用。

设置宽度大于最大数字会使数字对齐。

- 如果显示的数字小于宽度，则数字将在空格中填充左侧（如果使用“ - ”字符，则为右侧）。
- 如果空格以“0”开头，则字段用零填充。

对于指定的转换字符，精度具有以下含义

- d，o，x，X和and - 要打印的最小位数。 如果显示的整数小于精度，则填充前导零。 这与在场宽上使用前导零相同。
- s - 要打印的最大字符数。 如果字符串长于精度，则忽略其余字符。
- f，e和E - 小数点后要打印的位数。
- g和G - 有效位数。

参数排序

元素文本字符串可以包含多个格式说明符。 显示字典元素时，第一个格式说明符应用于第一个参数，第二个说明符用于第二个参数，依此类推。 在某些情况下，您可能需要不按顺序应用格式说明符。 如果您开发语言程序，然后将字典翻译成另一种语言，就会发生这种情况。

要重新排列订单说明符，请使用带有“^”字符和参数编号的转换字符。 举个例子，

```
$20, file_message "File %s^2 on device %s^1 not found" &new_line
```

表示使用第一个说明符的第二个参数和第二个说明符的第一个参数。



警告

您无法重新排列这些是SHORT或BYTE类型，因为它们与其他数据类型不同。 重新排列SHORT或BYTE类型参数可能会导致意外结果。

10.2.6 字典保留字命令

保留字以“&”字符开头，用于控制屏幕。 它们会影响文本的显示方式，并在某些情况下显示。 它们提供了一种简单的自我记录方式，可以将控制信息添加到字典中。 有关可用保留字的列表，请参见表10-2。

表10-2。 保留字

保留字	功能
&bg_black	背景颜色黑色
&bg_blue	背景颜色蓝色
&bg_cyan	背景颜色青色
&bg_dflt	背景颜色默认
&bg_green	背景颜色绿色
&bg_magenta	背景颜色洋红色
&bg_red	背景颜色红色
&bg_white	背景颜色白色
&bg_yellow	背景颜色黄色
&fg_black	前景色黑色
&fg_blue	前景色蓝色
&fg_cyan	前景色青色
&fg_dflt	前景色默认
&fg_green	前景色绿色
&fg_magenta	前景色紫红色
&fg_red	前景色红色
&fg_white	前景色白色
&fg_yellow	前景色黄色
&clear_win	清除窗口 (#128)
&clear_2_eol	清除到行尾 (#129)
&clear_2_eow	清除到窗口结束 (#130)
$cr	回车 (#132)
$lf	换行 (#133)
&rev_lf	反向换行 (#134)
&new_line	新行 (#135)
&bs	退格 (#136)
&home	窗口中的主光标 (#137)
&blink	闪烁显示属性 (#138)
&reverse	反转显示属性 (#139)
&bold	粗体显示属性 (#140)

表10-2. 保留字（续）

保留字	功能
&under_line	下划线显示属性 (#141)
&double_wide	宽显示尺寸 (#142) (有关用法，请参阅下面的说明)
&standard	所有属性都正常 (#143)
&graphics_on	打开图形字符 (#146)
&ascii_on	打开ASCII字符 (#147)
&double_high	高显示尺寸 (#148) (有关用法，请参阅下面的说明)
&normal_size	正常显示大小 (#153)
&multi_on	打开多国字符 (#154)

属性&normal_size，&double_high和&double_wide用于清除屏幕上一行的数据。 但是，它们目前有效。 要使用这些属性，请先将光标放在要调整大小的行上。 然后编写属性和文本。

- **对于示教器**，&double_high表示双高和双宽都有效，而double_wide与&normal_size相同。
- **对于CRT / KB**，&double_high表示双高和双宽都有效，而&double_wide表示双宽但正常高度。

10.2.7 字符代码

字符代码是“#”字符，后跟0到255之间的数字。它提供了一种在字典中插入键盘上未显示的特殊可打印字符的方法。 有关ASCII字符代码的列表，请参阅附录D.

10.2.8 嵌套字典元素

加号“+”属性允许字典元素引用同一字典的另一个字典元素，最多五个级别。 这些嵌套元素可以通过元素名称或元素编号引用，并且可以在当前元素之前或之后。 显示嵌套元素时，它们以嵌套顺序显示，就好像它们是一个单独的元素一样。

10.2.9 字典注释

星号字符（*）表示所有文本，即行的结尾，是注释。 压缩字典时，将忽略所有注释。 注释可以放在允许空格的任何位置，除非在元素文本中。

10.2.10 生成KAREL常量文件

KAREL包含KAREL编程文件。 include文件想要包含CONST声明符和每个命名元素的常量声明。

```
 element_name = element_number
```

您的KAREL程序可以使用此文件并按名称而不是数字引用每个字典元素。

要生成KAREL包含文件，请在字典fie的第一行指定“.kl”，后跟文件名。 压缩字典时会自动生成KAREL包含文件。

压缩字典时，以下将创建文件kcmn.kl。

```
.kl kcmn
$-, move_home, "press HOME to move home"
```

kcmn.kl 文件如下所示

```
-- WARNING: This include file generated by dictionary compressor.
--
-- Include File: kcmn.kl
-- Dictionary file: apkcmneg.utx
--CONST
move_home = 0
```

KAREL程序确保使用正确的元素编号。

10.2.11 在控制器上压缩和加载字典

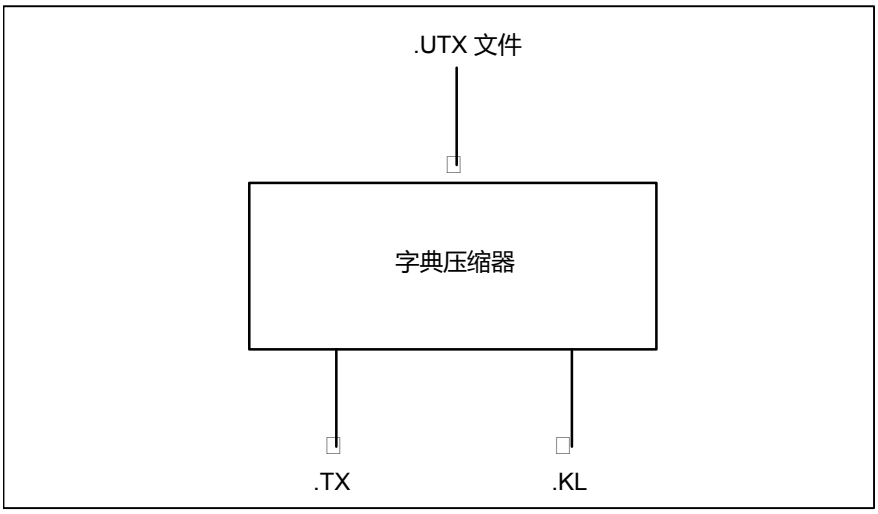
KAREL编辑器可用于创建和修改用户词典。 编辑完文件后，可以从KCL命令提示符对其进行压缩。

```
KCL> COMPRESS DICT filename
```

不要将.UTX文件类型包含在文件名中。 如果压缩器检测到任何错误，则需要做出决定。 在继续之前编辑用户词典并更正问题。

将创建一个名为filename但具有.TX文件类型的可加载字典。 如果使用.kl符号，则将创建KAREL包含文件。 图10-1说明了压缩过程。

图10-1. 字典压缩器和用户字典文件



在KAREL程序可以使用字典之前，必须在控制器中加载字典并给出字典名称。 字典是一个一到四个字符的单词，在加载时分配给字典。 使用KCL LOAD DICT命令加载字典。

```
KCL> LOAD DICT filename dictname <lang_name>
```

可选的lang_name允许使用相同的字典名称加载多个字典。

当前字典旨在由$ LANGUAGE的当前值使用。 该系统变量由KCL SET LANGUAGE命令或SET_LANG KAREL内置。 允许的语言为

ENGLISH，JAPANESE，FRENCH，GERMAN，SPANISH或DEFAULT。

因此，KAREL程序可以加载字典。 KAREL内置ADD_DICT用于将字典加载到指定语言并分配字典名称。

10.2.12 从KAREL程序访问字典元素

您的KAREL程序使用字典名称和元素编号，或元素名称来访问字典元素。 以下KAREL内置函数用于访问字典元素：

- ADD_DICT - 将字典添加到指定的语言。

- REMOVE_DICT - 从指定的语言中删除字典并关闭该文件或释放它所驻留的内存。
- WRITE_DICT - 将字典元素写入窗口。
- WRITE_DICT_V - 将具有KAREL变量的格式说明符的字典元素写入窗口。
- READ_DICT - 将字典元素读入KAREL STRING变量。
- READ_DICT_V - 将具有格式说明符的字典元素读入STRING变量。
- CHECK_DICT - 检查字典元素是否存在。

10.3 创建用户表单

R-30iA菜单界面与R-30iA菜单界面相同。

以下是使用表单的步骤。

1. 使用.FTX文件扩展名创建ASCII表单文本文件。
2. 使用KCL COMPRESS FORM命令压缩表单文件。 这将创建一个带有.TX扩展名和相关变量文件（.VR）的可加载字典文件。
3. 加载表单。
   •从KCL，使用KCL LOAD FORM命令。 这将加载字典文件（.TX）和关联的变量文件（.VR）。
   •从KAREL，使用内置的ADD_DICT加载字典文件（.TX），使用LOAD内置加载关联变量文件（.VR）。
4. 使用内置的KAREL DISCTRL_FORM显示表单文本。 DISCTRL_FORM内置处理所有输入操作，包括光标位置，滚动，分页，输入验证和选择选择。 请参阅DISCTRL_FORM内置附录A“KAREL语言字母说明”。

表单对需要用户输入数据的程序很有用。例如，用户输入数据，程序必须对其进行测试以确保其处于可接受的形式并且必须是有效的选择。如果输入了不正确的值，程序必须通知用户并提示输入新条目。表单提供了一种自动验证输入数据的方法。通过为操作员提供熟悉的界面，表单允许程序看起来好像已集成到系统菜单的其余部分中。

表单必须选择USER2菜单。表单分别使用教学挂件和CRT / KB的“t_sc”和“c_sc”屏幕。系统预测的窗口用于显示表单文本。对于两个屏幕，此窗口高10行，宽40列。这意味着＆double_high和＆double_wide属性在CRT / KB上使用，无法更改。

10.3.1 表单语法

表单定义了示教器或CRT / KB屏幕上显示的操作员界面。 表单是一个特殊的字典元素。 许多表单可以与其他（非表单）字典元素一起驻留在同一个字典中。

注意如果您的程序需要表单字典文件（.FTX），则不必创建用户字典文件（.UTX）。 您可以将用户词典元素放在与表单相同的文件中。

为了将表单与字典中的其他元素区分开来，符号“.form”放置在元素之前，符号“.endform”放置在元素之后。 符号必须位于各自的行上。 压缩字典中省略了表单符号。

以下是表单的语法：

表单语法

```
.form <form_attributes>
$n, form_name<@cursor_pos><&res_word>"Menu_title"<&res_work>&new_line
     <@cursor_pos><&res_word>"Menu_label"<&res_word>&new_line
     <@cursor_pos><&res_word><"-Selectable_item"<&res_word>&new_line>
     <@cursor_pos><&res_word><"-%Edit_item"<&res_word>&new_line>
     <@cursor_pos><&res_word><"Non_selectable_text"<&res_word>&new_line>
     <@cursor_pos><&res_word><"Display_only_item"<&res_word>&new_line>
     <^function_key &new_line>
     <?help_menu &new_line>
.endform
<$n,function_key
     <"Key_name" &new_line>
     <"Key_name" &new_line>
     <"Key_name" &new_line>
     <"Key_name" &new_line>
     <"help_label" &new_line>
     <"Key_name" &new_line>
     <"Key_name" &new_line>
     <"Key_name" &new_line>
     <"Key_name" &new_line>
     "Key_name"
>
<$n,help_menu
     <"Help_text" &new_line>
     <"Help_text" &new_line>
     "Help_text">
```

限制

- 括号<>中的项是可选的。

- 此处未定义的符号是标准用户词典元素符号（$ n,@cursor_pos,&res_word和new_line）。
- form_attributes是关键词unnumber和unclear。
- form_name指定标识表单的元素名称。
- "Menu_title"和"Menu_label"指定填充表单前两行并始终显示的元素文本。
- "- Selectable_item"指定可以被光标和选择的元素文本。
- "-%Editable_item"指定可以被光标和编辑的元素文本。
- "Non_selectable_text"指定在表单中显示且无法被镜像的元素文本。
- "%Display_only_item"使用格式说明符指定元素文本。它不能被改变。
- ^function_key 使用元素名称定义功能键的标签。
- ?help_menu 定义使用元素名称与表单关联的帮助文本页面。
- "Key_name"指定功能键上显示的元素文本。
- "Help_label"是功能键5的特殊标签。它可以是任何标签或特殊字HELP。
- "Help_text"是最多40个字符的元素文本。
- 可以在表单中指定颜色属性。 i Pendant将显示颜色。单色链坠将忽略颜色属性。

10.3.2 表单属性

通常，在光标可以移动到的任何项目前面都会显示一个带有行号的表单。 为了防止表单生成和显示行号，使用符号“.form unnumber”。

为了防止表单在显示之前清除任何窗口，使用符号''.form noclear''。 符号''noclear''和''unnumber''可以按任何顺序使用。

在以下示例中，MH_TOOLDEFN是未编号的形式，不会清除任何窗口。 MH_APPLIO是一种编号形式。

```
.form unnumber noclear
$1, MH_TOOLDEFN

.endform

$2, MH_PORT

$3, MH_PORTFKEY
```

```
.form

$6, MH_APPLIO

.endform
```

10.3.3 表格标题和菜单标签

菜单标题是表单名称后面的文本的第一个元素。 菜单标题后面是菜单标签。 每个都包含非滚动窗口中的一行文本。

- 在示教器上，“完整”窗口的第一行用于菜单标题。 第二行用于菜单标签。
- 在CRT / KB上，“cr05”窗口的第一行用于菜单标题。 第二行用于菜单标签。
- 菜单标题位于第1行第1-21行。
- 菜单标签位于第4行第1-40行。

除非指定了“noclear”表单属性，否则将清除菜单标题和菜单标签。
必须在菜单标题之前指定保留字和home，以确保光标定位正确。 保留字和反向也应在菜单标题之前指定，保留字和标准应直接在菜单标题之后。 这些是确保菜单看起来与R-30iA菜单界面一致的必要条件。 必须在菜单标题和菜单标签之后指定保留字和new_line，以指示文本的结尾。 以下是菜单标题和菜单标签定义的示例。

```
.form

$1, mis_form

&home &reverse "Menu Title" &standard &new_line

"Menu Label" &new_line

.endform
```

如果不需要菜单标签文本，可以在菜单标题后指定＆new_line两次，如下例所示。

```
.form

$1,misc_form

&home &reverse " Menu Title" &standard &new_line &new_line

.endform
```

如果指定了光标位置属性，则不必指定＆new_line保留字。 以下示例将菜单标题的光标位置设置为第1行，第2列，将菜单标签设置为第2行第5列。

```
.form

$1,misc_form

@1,2 &reverse "Menu Title" &standard

@2,5 "Menu Label"

.endform
```

10.3.4 表单菜单文本

表单菜单文本在菜单标题和菜单标签后面。 它由无限数量的行组成，这些行将显示在示教器上名为“fscr”的10行滚动窗口和CRT / KB上的“ct06”中。 该窗口位于第5-14行和第1-40列。 除非指定了''noclear''选项，否则在显示表单之前将清除所有行。

菜单文本可以包含以下内容：

- 可选菜单项
- 编辑以下类型的数据项：

— INTEGER
— INTEGER port
— REAL
— SHORT (32768 to 32766)
— BYTE (0 to 255)
— BOOLEAN
— BOOLEAN port
— STRING
— 程序名称字符串
— 功能键枚举类型
— 子窗口枚举类型
— 使用变量的子窗口枚举类型
— 端口模拟

- 不可选择的文字
- 仅显示具有格式说明符的数据项
- 光标位置属性
- 保留字或ASCII码
- 功能键元素名称或编号
- 帮助元素名称或编号

以下各节介绍了各种菜单文本。

10.3.5 表单可选菜单项

可选菜单项具有以下特征：

- 可选择的菜单项在字典中输入为双引号括起来的字符串。
- 字符串中的第一个字符必须是短划线' - '。 此字符不会打印到屏幕上。 例如，

```
"- Item 1 "
```

- 当可选项是默认值时，整个字符串将突出显示。
- 如果可选项跨越多行，则串联字符“+”应用作字符串中的最后一个字符。 连接字符不会打印到屏幕上。 属性&new_line用于表示新行。 例如，

```
"- Item 1, line 1 +" &new_line
```

```
" Item 1, line 2 "
```

- 自动编号使用前三列，不会移动表单文本。 因此，文本必须通过添加空格或指定光标位置来允许三列。 例如，

```
"- Item 1 " &new_line

"- Item 2 " &new_line

"- Item 3 "
```

或者

```
@3,4"- Item 1 "

@4,4"- Item 2 "

@5,4"- Item 3 "
```

- 滚动窗口中的第一行定义为表单的第3行。
- 无论终止掩码设置如何，在可选菜单项上按Enter键将始终使表单处理器以ky_select的终止字符退出。 将返回所选的项目编号。
- 通过按下示教器上的ITEM硬键选择项目将仅突出显示该项目。 它不会导致退出。
- 虽然可以将快捷编号选择指定为终止掩码，但不会自动处理快捷编号选择。

10.3.6 编辑数据项

您可以编辑具有以下特征的数据项：

- 数据项在字典中输入为双引号括起来的字符串。
- 字符串中的第一个字符必须是短划线' - '。 此字符不会打印到屏幕上。
- 字符串中的第二个字符必须是'%'。 此字符标记格式说明符的开头。
- 每个格式说明符以%开头，以转换字符结尾。 这两个字符之间的所有字符与用户字典元素具有相同的含义。

注意您应该为每个格式说明符提供字段宽度，否则将使用默认值。 此默认值可能会导致您的表单错位。

表10-3列出了可编辑数据项的转换字符。

表10-3. 转换字符

字符	参数类型:打出为
d	INTEGER; 十进制数。
o	INTEGER; 无符号八进制表示法（没有前导零）。
x, X	INTEGER; 无符号十六进制表示法（没有前导0x或0X），使用abcdef或ABCDEF表示10，...，15。
u	INTEGER; 无符号十进制表示法。
pu	INTEGER 端口; 无符号十进制表示法。
px	INTEGER 端口; 无符号十六进制表示法（没有前导0x或0X），使用abcdef或ABCDEF表示10，...，15。
f	REAL; 形式< - > mmm.dddddd的十进制表示法，其中d's的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
e, E	REAL; 形式的十进制表示法< - > m.dddddde + -xx或< - > m.ddddddE + -xx，其中d's的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
g, G	REAL; 如果指数小于-4或大于或等于精度，则使用%e或%E; 否则使用%f。 不打出尾随零和尾随小数指针。
h	SHORT; 短字符。
b	BYTE; 无符号字节。
B	BOOLEAN; 从布尔枚举字符串中打出字符。
P	BOOLEAN 港口; 从布尔端口枚举字符串中打出字符。
S	INTEGER或BOOLEAN 端口; 从端口模拟枚举字符串中打出字符。
k	STRING; 从KAREL字符串打出字符直到字符串结尾或精度给出的字符数。
pk	STRING; 从KAREL字符串打出程序名称，直到字符串结尾或精度给出的字符数。
n	INTEGER; 从函数键枚举字符串中打出字符。 使用字典元素来定义枚举字符串。
w	INTEGER; 从子窗口枚举字符串中打出字符。 使用字典元素来定义枚举字符串。
v	INTEGER; 从子窗口枚举字符串中打出字符。 使用变量来定义枚举字符串。

表 10–3. 转换字符（续）

字符	参数类型:打出为
%	没有参数被转换; 打出%。
t	BOOLEAN; 打出+或 - 用于树视图。

以下是说明符格式的示例：

```
"-%5d" or "-%-10s"
```

表单处理器从输入值数组中检索值并按顺序显示它们。 所有值都会动态更新。
编辑数据项：INTEGER，INTEGER Ports，REAL，SHORT，BYTE

- 您可以通过为每个格式说明符指定允许的最小值和最大值“（min，max）”来指定可接受值的范围。 如果未指定最小值和最大值，则将接受任何整数或浮点值。 例如，

```
"-%3d(1,255)" or "-%10.3f(0.,100000.)"
```

- 选择编辑数据项时，表单处理器调用适当的输入例程。 输入例程读取新值（反向视频处于活动状态）并使用字典元素中指定的最小值和最大值来确定新值是否在有效范围内。
  - 如果新值超出范围，将向提示行写入错误消息，并且不会修改当前值。
  - 如果新值在有效范围内，它将覆盖当前值。

编辑数据项：BOOLEAN

- 格式说明符%B用于KAREL BOOLEAN值，用于显示和选择F4和F5功能键的菜单选项。 包含功能键标签的字典元素的名称括在括号中，并在%B之后指定。 例如，

```
"-%4B(enum_bool)"
```

定义功能键的字典元素应首先定义FALSE值（F5标签），然后定义TRUE值（F4标签）。 例如，

```
$2,enum_bool

" NO" &new_line

" YES"
```



当光标移动到枚举项时，表单处理器将标记功能键。 字段中显示的值与功能键标签相同，但删除了所有前导空格。

编辑数据项：BOOLEAN端口

- 说明符格式%P用于KAREL BOOLEAN端口值，用于显示和选择F4和F5功能键中的菜单选项。 包含功能键标签的字典元素的名称括在括号中，并在%P之后指定。 例如，

```
"-%3P(enum_bool)"
```

定义功能键的字典元素应首先定义0值（F5标签）和第二个值（F4标签）。 例如，

```
$2,enum_bool

" OFF" &new_line

" ON"
```



当光标移动到枚举项时，表单处理器将标记功能键。 字段中显示的值与功能键标签相同，但删除了所有前导空格。

编辑数据项：端口模拟

- 格式说明符%S用于端口模拟，以显示和选择F4和F5功能键中的菜单选项。 包含功能键标签的字典元素的名称括在括号中，并在%S之后指定。 例如，

```
"-%1S(sim_fkey)"
```

定义功能键的字典元素应首先定义0值（F5标签）和第二个值（F4标签）。 例如，

```
$-, sim_fkey
" UNSIM " &new_line * F5键标签，端口将是未经模拟的
"SIMULATE" &new_line * F4键标签，端口将被模拟
```

当光标移动到枚举项时，表单处理器将标记功能键。 字段中显示的值与功能键标签相同，但删除了所有前导空格，并且将截断该值以适合字段宽度。

编辑数据项：STRING

- 您可以选择在编辑之前清除字符串的内容。 为此，请使用STRING格式说明符，并在括号中加上“clear”一词。 如果未指定“（清除）”，则默认为修改现有字符串。 例如，

```
"-%10k(clear)"
```

编辑数据项：程序名称字符串

- 您可以使用%pk格式说明符显示并从子窗口中选择程序名称。 要显示的程序类型括在括号中，并在%pk后指定。 例如，

```
"-%12pk(1)" * 指定TP程序

"-%12pk(2)" * 指定PC程序

"-%12pk(6)" * 指定TP, PC, VR

"-%12pk(16)" * 指定TP和PC
```

子窗口中将显示与指定类型匹配且当前在内存中的所有程序。 选择程序时，字符串值将复制到关联的变量中。

编辑数据项：功能键枚举

- 您可以使用格式说明符%n（对于枚举的整数值）来显示和选择功能键中的选项。 包含有效选项列表的字典元素的名称括在括号中，并在%n之后指定。 例如，

```
"-%6n(enum_fkey)"
```

定义功能键的字典元素应该每行列出一个功能键标签。 如果指定左侧的功能键未激活，则应将其设置为“”。 最多可以使用5个功能键。 例如，

```
$2,enum_fkey

"" &new_line *指定F1不活动

"JOINT" &new_line *指定 F2

"LINEAR" &new_line *指定 F3
```

```
"CIRC" *Specifies F4
```

当选择枚举项时，表单处理器将标记相应的功能键。 选择功能键时，整数中设置的值如下：

```
User presses F1, value = 1

User presses F2, value = 2

User presses F3, value = 3

User presses F4, value = 4

User presses F5, value = 5
```

除了所有前导空格被删除之外，在该字段中显示的值与功能键标签相同。



编辑数据项：子窗口枚举

- 您可以使用格式说明符%w（对于枚举的整数值）来显示子窗口并从中选择选项。 包含有效选项列表的字典元素的名称括在括号中，并在%w后指定。 例如，

```
"-%8w(enum_sub)"
```

需要一个字典元素来定义子窗口中的每个选项。 可以使用35种选择。 如果使用的选项少于35个，则最后一个选项后面应该跟一个包含“\ a”的字典元素。 选项将显示在2列中，每页有7个选项。 如果仅使用4个或更少的选项，则选项将显示在1列中，宽度为36个字符。 例如，

```
$2,enum_sub "Option 1"

$3 "Option 2"

$4 "Option 3"

$5 "\a"
```

当光标移动到枚举项时，表单处理器将F4标记为''[CHOICE]''。 当选择功能键F4，[CHOICE]时，它将创建具有适当显示的子窗口。 选择选项时，整数中设置的值是选定的数字。 除非删除所有前导空格，否则该字段中显示的值与字典标签相同。

编辑数据项：使用变量的子窗口枚举

- 您还可以使用格式说明符%v（对于枚举的整数值）来显示和选择子窗口中的选项。 但是，不是在字典中定义选项，而是在变量中定义它们。 包含程序和变量名称的字典元素的名称括在括号中，并在%v之后指定。 例如，

```
"-%8v(enum_var)"

$-,enum_var

"RUNFORM" &new_line * 变量的程序名称

"CHOICES" &new_line * 包含选项的变量名称
```

[RUNFORM] CHOICES必须定义为KAREL字符串数组。 数组的每个元素都应该在子窗口中定义一个选项。 此格式说明符类似于%w。 但是，第一个元素与值0相关并且从不使用。 值1从第二个元素开始。 最后一个值是数组的结尾或第一个未初始化的值。

```
[RUNFORM] CHOICES:ARRAY[6] OF STRING[12] =

[1] *uninit*

[2] 'Red' <= value 1

[3] 'Blue' <= value 2

[4] 'Green' <= value 3

[5] *uninit*

[6] *uninit*
```

编辑数据项：树视图

- 格式说明符%t用于指定树视图项。 它使用KAREL BOOLEAN值来确定树是展开还是折叠。 例如，

```
"-%t"
```

表单处理器将%t更改为%c。 当BOOLEAN值为FALSE时，将使用格式说明符显示“+”，并且不会显示树视图后面的项目（折叠状态）。 当BOOLEAN值为TRUE时，将使用格式说明符显示“-”，并显示树视图后面的项目（展开状态）。 表10-4显示了一些格式示例。

表10-4. 树视图格式

格式	FALSE 值	TRUE 值
"-%t"	+	–
"-%2t "	+	–
"-%t Weld schedule:"	+ Weld schedule:	– Weld schedule:

- KAREL BOOLEAN值仅确定树视图的初始状态。 它没有受到监控。 当用户选择树视图项并按ENTER时，树视图状态将切换，BOOLEAN值将设置为结果值。 退出并重新输入表单时，将保持此状态。 如果KAREL BOOLEAN值处于CMOS或SHADOW中，则状态在电源周期之间保持不变。
- 在那里刷新视图。 Passingctl_w将根据表单关联的KAREL BOOLEAN值展开或折叠所有树视图项。 将保留默认项目。 如果不再显示默认项，则其树视图项将成为新的默认项。
- 不允许嵌套树视图项。 如果找到另一个%t，则结束当前树视图并启动新的树视图。 当应该结束树视图项而不创建新的树视图项时，可以使用.endtree。 例如：

```
"   "-%t Burnback schedule:" &new_line
"     Wire feed ""-%6.2f(0.0, 9999.0)" &new_line
"     Trim      ""-%6.2f(0.0, 9999.0)" &new_line
.endtree
"   Gas postflow time:" &new_line
```

- 如果通过连接多个表单来构造表单，并且项目将在树视图中，则必须使用.tree。 The.endtree是可选的，仅在其他项不在树视图中时才需要。

```
.form
$-,wl_data_cmd &new_line &new_line
.tree
"    Wire Feed ""-%6.2f(0.0, 9999.0)" &new_line
.endform
```

10.3.7 使用树视图的动态表单

树视图可用于创建动态表单。 它可以用来代替将多个表单连接成一个。 在这种情况下，使用树视图数据项而不使用前导 - 。 例如，

```
"%t" &new_line
```

```
"   PC Share Configuration:" &new_line
"   WINS Server:         " "16k" &new_line
"   Client Caching:      " "-%10B(bool_fkey)" &new_line
"   Broadcast Discovery:" "-%10B(bool_fkey)" &new_line
```

当KAREL BOOLEAN值为FALSE时，树视图后面的项将不可见。 当KAREL BOOLEAN值为TRUE时，树视图后面的项目将可见。 树视图行始终不可见。 其行上的任何其他项目也是不可见的。 KAREL BOOLEAN值仅确定树视图的初始状态。 它没有受到监控。 由于树视图项不可见，用户将永远无法更改状态。

有一种方法可以刷新树视图。 将ctl_w传递给表单将根据其关联的KAREL BOOLEAN值展开或折叠所有树视图项。 将保留默认项目。 如果不再显示默认项目，则上一项将成为新的默认项目。

10.3.8 不可选文本

可以在表单中指定不可选择的文本。 这些项目具有以下特征：

- 不可选择的文本在字典中输入为双引号括起来的字符串。
- 可以在表单中的任何位置定义不可选文本，但不得超过窗口中的最大列数。

10.3.9 仅显示数据项

只显示数据项可以在表单中指定。 这些项目具有以下特征：

- 仅显示数据项在字典中输入为双引号括起来的字符串。
- 字符串中的第一个字符必须是'%'。 此字符标记格式说明符的开头。
- 在上一节中为编辑数据项定义了格式规范。

10.3.10 光标位置属性

游标定位属性可用于定义任何文本的行和列。 始终首先指定行。 如果表单尝试回溯到前一行或列，则字典压缩器将生成错误。 表单标题和标签位于第1行和第2行。滚动窗口从第3行开始。例如，

```
@3,4 "- Item 1"
```

```
@4,4 "- Item 2"

@3,4 "- Item 3" <- backtracking to row 3 not allowed
```

即使滚动窗口只有10行，长形式也可以指定大于12的行位置。表单处理器在滚动期间跟踪当前行。

10.3.11 形成保留字和字符代码

可以使用保留字或字符代码。 有关所有可用保留字的列表，请参见表10-5。 但是，在滚动窗口中只允许不移动光标的保留字。 有关滚动窗口的可用保留字列表，请参阅表10-6。

表 10–5. 保留字

保留字	功能
&bg_black	背景颜色黑色
&bg_blue	背景颜色蓝色
&bg_cyan	背景颜色青色
&bg_dflt	背景颜色默认
&bg_green	背景颜色绿色
&bg_magenta	背景颜色洋红色
&bg_red	背景颜色红色
&bg_white	背景颜色白色
&bg_yellow	背景颜色黄色
&fg_black	前景色黑色
&fg_blue	前景色蓝色
&fg_cyan	前景色青色
&fg_dflt	前景色默认
&fg_green	前景色绿色
&fg_magenta	前景色洋红色
&fg_red	前景色红色
&fg_white	前景色白色
&fg_yellow	前景色黄色
&clear_win	清除窗口 (#128)

表 10–5. 保留字（续）

保留字	功能
&clear_2_eol	清除到行尾 (#129)
&clear_2_eow	清除到窗口结束 (#130)
$cr	回车 (#132)
$lf	换行 (#133)
&rev_lf	反向换行 (#134)
&new_line	新建一行 (#135)
&bs	退格 (#136)
&home	窗口中的主光标 (#137)
&blink	闪烁显示属性 (#138)
&reverse	反转显示属性 (#139)
&bold	粗体显示属性 (#140)
&under_line	下划线显示属性 (#141)
&double_wide	宽显示尺寸（＃142）（有关用法，请参阅下面的说明）
&standard	所有属性都正常 (#143)
&graphics_on	打开图形字符 (#146)
&ascii_on	打开ASCII字符 (#147)
&double_high	高显示尺寸（＃148）（有关用法，请参阅下面的说明）
&normal_size	正常显示大小 (#153)
&multi_on	打开多国字符 (#154)

表10-6 列出了可用于滚动窗口的保留字。

表 10–6. 滚动窗口的保留字

保留字	**功能**
&new_line	新行 (#135)
&blink	闪烁显示属性 (#138)
&reverse	反转显示属性 (#139)
&bold	粗体显示属性 (#140)
&under_line	下划线显示属性 (#141)
&standard	所有属性都正常 (#143)
&graphics_on	打开图形字符 (#146)

表 10–6. 滚动窗口的保留字（续）

保留字	功能
&ascii_on	打开ASCII字符 (#147)
&multi_on	打开多国字符 (#154)

10.3.12 表单功能键元素名称或编号

每个表单可以有一个相关的功能键菜单。 功能键菜单具有以下特征：

- 功能键菜单在字典中用插入符号^指定，紧接着是功能键字典元素的名称或编号。 例如，

```
^misc_fkey
```

- 定义功能键的字典元素应该每行列出一个功能键标签。 如果指定左侧的功能键未激活，则应将它们设置为“”。 最多可以使用10个功能键。 例如，

```
$3,misc_fkey

" F1" &new_line

" F2" &new_line

" F3" &new_line

" F4" &new_line

" HELP >" &new_line

"" &new_line

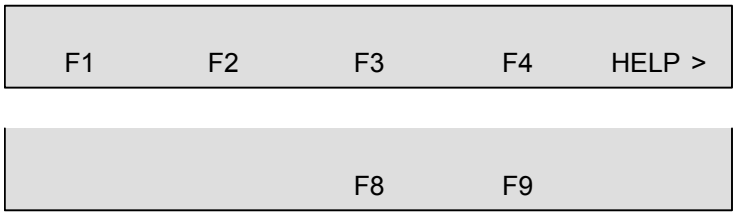
"" &new_line

" F8" &new_line

" F9" &new_line
```

- 如果按下有效键，表单处理器将标记相应的功能键并从例程返回。 终止字符将设置为ky_f1到ky_f10。
- 如果枚举数据项使用相同的功能键，则功能键将暂时无效。

- 如果功能键F5标记为HELP，它将自动调用表单的帮助菜单（如果存在）。



10.3.13 使用变量的表单功能键

功能键菜单也可以在变量中定义。 功能键字典项将包含程序和变量名称，前缀为星号以区别于功能键文本。 例如，

```
* Specify the function keys in a variable
* whose type is an ARRAY[m] of STRING[n].
$3,misc_fkey
  "*RUNFORM"  &new_line * program name of variable
  "*FKEYS"    &new_line * variable name containing function keys
```

[RUNFORM] FKEYS必须定义为KAREL字符串数组。 数组的每个元素都应该定义一个功能键标签。

```
[RUNFORM] FKEYS:ARRAY[10] OF STRING[12] =
[1] '  F1'
[2] '  F2'
[3] '  F3'
[4] '  F4'
[5] ' HELP   >'
[6] ''
[7] ''
[8] '  F8'
[9] '  F9'
[10]'        >'
```

10.3.14 表单帮助元素名称或编号

每个表单都可以有一个相关的帮助菜单。 帮助菜单具有以下特征：

- 在字典中使用问号“？”指定帮助元素名称或编号，紧接着是帮助字典元素的名称或编号。 例如，

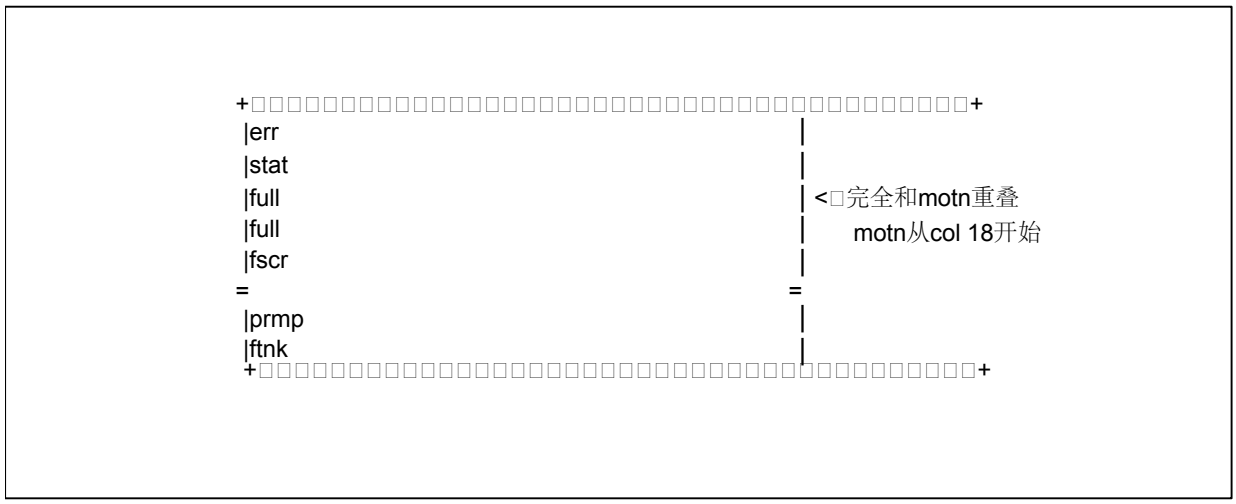
```
?misc_help
```

- 定义帮助菜单的字典元素限制为48行文本。
- 表单处理器将通过在预定义窗口中显示帮助字典元素来响应帮助键。 预定义窗口宽40列，占据行3到14。
- 帮助菜单响应以下输入：
  - 向上或向下箭头向上或向下滚动1行。
  - 向上或向下移动箭头以向上或向下滚动3/4页。
  - 上一步，退出帮助。 帮助菜单在返回之前恢复上一屏幕。

10.3.15 示教器表单屏幕

您可以在显示表单时写入其他活动示教窗口。 屏幕本身被命名为“tpsc”。 图10-2显示了连接到此屏幕的所有窗口。 除非指定了noclear选项，否则在显示表单之前，将清除''full'，'''fscr'，''''，''和''ftnk''窗口。

图 10–2. 示教器表单屏幕

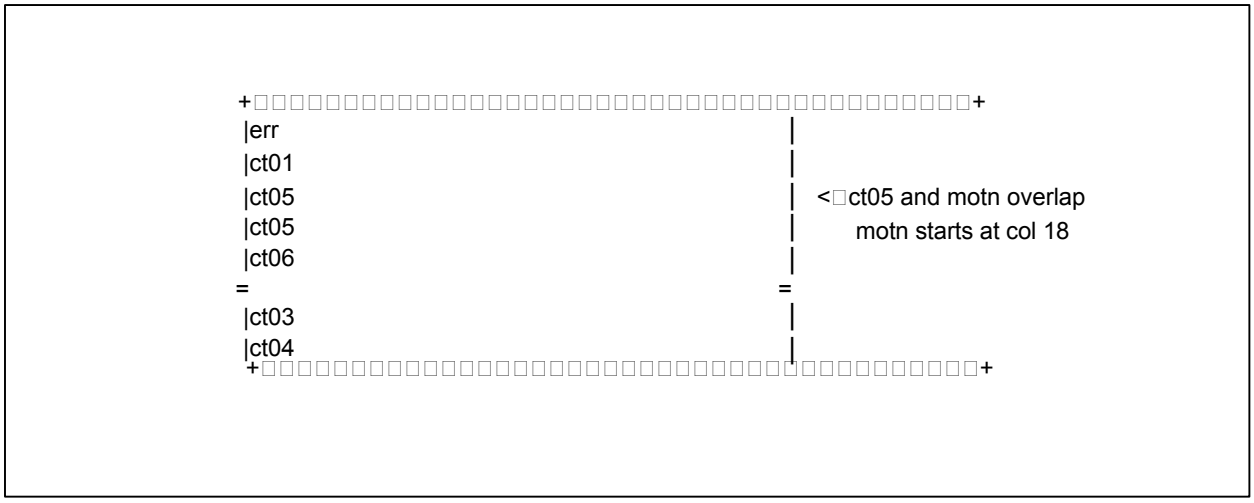


10.3.16 CRT/KB 表单屏幕

您可以在显示表单时写入其他活动的 CRT/KB 窗口。 屏幕本身名为''ctsc。''屏幕中的所有行都设置为双倍高和双倍宽的视频大小。 图10-3

显示连接到此屏幕的所有窗口。 除非指定''noclear''选项，否
则'ct05'，'''ct06，''''ct03，''和''ct04''窗口将在显示表单之前被清除

图 10_3. CRT/KB 表单屏幕



10.3.17 表单文件命名规定

未压缩的表单字典文件必须使用以下文件名规定：

- 字典文件名中的前两个字母可以是应用程序前缀。
- 如果文件名大于四个字符，则表单处理器将在尝试确定字典名称时跳过前两个字母。
- 接下来的四个字母必须是用于加载.TX文件的字典名称，否则表单处理器将无法工作。
- 最后两个字母是可选的，应该用于识别语言;
  - — ‘‘EG’’ 为 ENGLISH
  - — ‘‘JP’’ 为 JAPANESE
  - — ‘‘FR’’ 为 FRENCH
  - — ‘‘GR’’ 为 GERMAN
  - — ‘‘SP’’ 为 SPANISH
- 包含表单文本的字典文件必须具有.FTX文件类型，否则字典压缩器将不起作用。 压缩后，相同的字典文件将具有.TX文件类型。

以下是未压缩的表单字典文件名的示例：

```
MHPALTEG.FTX
```

MH代表物料处理，PALT是用于在控制器上加载字典的字典名称，EG代表英语。

10.3.18 在控制器上压缩和加载表单

表单文件只能压缩在RAM磁盘RD：上。 压缩表单类似于压缩用户词典。 在KCL命令提示符下，输入：

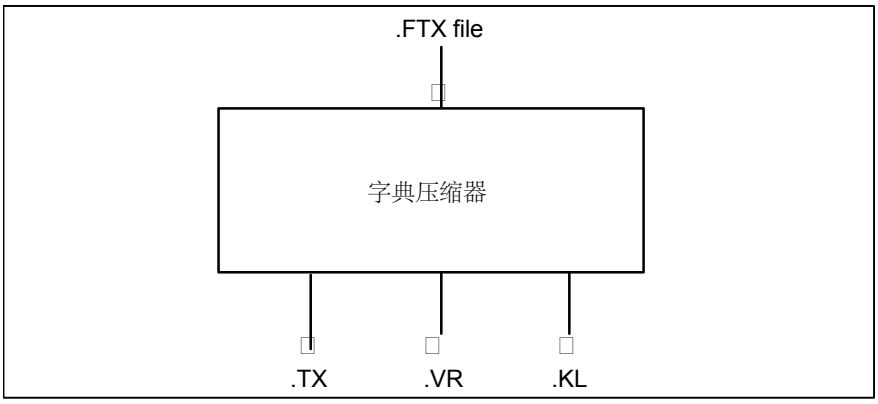
```
KCL> COMPRESS FORM filename
```

不要包含.FTX文件类型。 如果压缩器检测到任何错误，它将指向有问题的单词，并简要说明错误。 在继续之前编辑表单并更正问题。

注意表单文件必须是未压缩的文件，以便将错误指向正确的行。

压缩器将创建两个文件。 一个是可加载的字典文件，其名称为 *filename* 但具有.TX文件类型。 另一个是具有.VR文件类型但具有四个字符字典名称作为文件名的变量文件。 如前所述，从文件名中提取字典名称。 如果您使用'.kl''符号生成KAREL包含文件，也可能会创建第三个文件。 图10-4说明了压缩。

图 10 4. 字典压缩器和表单字典文件



每个表单都会生成三种变量。 表单处理器使用这些变量。 每次重新压缩表单字典时都必须重新加载它们。 变量如下：

1. 项目数组变量 - 变量名称将是四个字符的字典名称，与元素编号连接，与_IT连接。
2. 行数组变量 - 变量名称将是四字符字典名称，与元素编号连接，与_LN连接。

3.其他变量 - 变量名称将是四字符字典名称，与元素编号连接，与_MS连接。

定义表单的数据生成为KAREL变量。 这些变量保存在变量文件中并加载到控制器上。 程序的名称是以星号开头的字典名称。 例如，Dictionary MHPALTEG.FTX包含：

```
.form unnumber

$1, MH_TOOLDEFN

.endform

$2, MH_PORT

$3, MH_PORTFKEY

.form

$6, MH_APPLIO

.endform
```

如文件命名约定部分所述，从文件名中提取的字典名称为“PALT”。 字典元素1和6是表格。

使用程序名称''* PALT生成名为PALT.VR的变量文件。它包含以下变量：

```
PALT1_IT, PALT1_LN, and PALT1_MS

PALT6_IT, PALT6_LN, and PALT6_MS
```

注意 KCL CLEAR ALL不会清除这些变量。 要显示或清除它们，您可以设置VAR VAR $ CRT_DEFPROG ='* PALT'并使用SHOW VARS和CLEAR VARS。

使用KCL LOADFORM命令加载。

```
KCL> LOAD FORM filename
```

name filename是可加载字典文件的名称。 加载此文件后，将从*filename*中提取字典名称，并用于加载变量文件。 这个KCL命令相当于

```
KCL> LOAD DICT filename dict_name DRAM

KCL> LOAD VARS dict_name
```

10.3.19 显示表单

DISCTRL_FORM 内置用于显示和控制示教器或CRT / KB屏幕上的表格。 所有输入键都在DISCTRL_FORM 内处理。 这意味着将暂停执行KAREL程序，直到输入键导致DISCTRL_FORM 退出表单。 您的KAREL程序暂停时，任何条件处理程序都将保持活动状态。

注意如果USER2菜单是所选菜单，DISCTRL_FORM将仅显示表单。 因此，在调用DISCTRL_FORM 强制USER2菜单之前，请使用FORCE_SPMENU（device_stat，SPI_TPUSER2,1）。

以下屏幕显示示教器上显示的FORM.FTX中的第一个模板。 此示例包含四个可选菜单项。

图 10_5. 可选菜单项的示例

```
RUNFORM           LINE 22          RUNNING
Title here                        JOINT 10%
     label here                          1/5
  1  Menu item 1
  2  Menu item 2
  3  Menu item 3
  4  Menu item 4 line 1
     Menu item 4 line 2
  5  Menu item 5
```

FORM.FTX 中的字典元素（如可选菜单项的示例表单字典中所示）用于创建图10-5中所示的表单。

可选菜单项的示例表单词典

```
*     Dictionary Form File: form.ftx
*
*     Generate form.kl which should be included in your KAREL program
.kl form
.form
$-,forml
      &home &reverse "Title here" &standard $new_line
      "    label here " &new_line
      @3,10     "- Menu item 1 "
      @4,10     "- Menu item 2 "
      @5,10     "- Menu item 3 "
      @6,10     "- Menu item 4 line 1 +"
      @7,10     " Menu item 4 line 2 "
      @8,10     "- Menu item 5 "
      * Add as many items as you wish.
```

```
      * The form manager will scroll them.
      ^form1_fkey    * specifies element which contains
                     * function key labels
      ?form1_help    * element which contains help
.endform
$-,form1_fkey    * function key labels
      "     F1" &new_line
      "     F2" &new_line
      "     F3" &new_line
      "     F4" &new_line
      "     HELP >" &new_line * help must be on F5
      "     F6" &new_line
      "     F7" &new_line
      "     F8" &new_line
      "     F9" &new_line
      "     F10 >"
      * you can have a maximum of 10 function keys labeled
$-, form1_help     * help text
      "Help  Line 1" &new_line
      "Help Line 2" &new_line
      "Help Line3" &new_line
      * You can have a maximum of 48 help lines
```

可选菜单项的示例程序中显示的程序用于显示图10-5中所示的表单。

可选菜单项的示例程序

```
PROGRAM runform
%NOLOCKGROUP
%INCLUDE form               -- allows you to access form element numbers
%INCLUDE klevccdf
%INCLUDE klevkeys
%INCLUDE klevkmsk
VAR
    device_stat:  INTEGER                   --tp_panel or crt_panel
    value_array:  ARRAY [1] OF STRING [1]   --dummy variable for DISCTRL_FORM
    inact_array:  ARRAY [1] OF BOOLEAN      --not used
    change_array: ARRAY [1] OF BOOLEAN      --not used
    def_item:           INTEGER
    term_char:          INTEGER
    status:             INTEGER
BEGIN
    device_stat = tp_panel
    FORCE_SPMENU (device_stat, SPI_TPUSER2, 1)--forces the TP USER2
menu
```

```
    def_item = 1 -- start with menu item 1
    --Displays form named FORM1
    DISCTRL_FORM ("FORM", form1, value_array, inact_array,
           change_array, kc_func_key, def_item, term_char, status)
    WRITE TPERROR (CHR(cc_clear_win))        --clear the TP error
window
    IF term_char = ky_select THEN
          WRITE TPERROR ("Menu item", def_item: :1, 'was selected.')
    ELSE
          WRITE TPERROR ('Func key', term_char: :1, ' was selected.')
    ENDIF
END runform
```

图10-6显示了CRT / KB上显示的FORM.FTX中的第二个模板（一次只显示10个编号的行）。 此示例包含所有编辑数据类型。

图 10_6. 编辑数据项的示例

```
RUNFORM        LINE 81              RUNNING
Title here                        JOINT 10%
     label here
  1  Integer:     12345
  2  Integer:         1
  3  Real:        0.000000
  4  Boolean:     TRUE
  5  String:      This is a test
  6  String:      **************
  7  Byte:        10
  8  Short:       30
  9  DIN[1]:      OFF
 10  AIN[1]:      0 S
 11  AOUT[2]:     0 U
 12  Enum Type:   FINE
 13  Enum Type:   Green
 14  Enum Type:   Red
 15  Prog Type:   MAINTEST
 16  Prog Type:   RUNFORM
 17  Prog Type:   PRG1
 18  Prog Type:   MAINTEST
EXIT
   F1       F2       F3       F4       F5

  ITEM     PAGE□    PAGE+     FCTN     MENUS
   F6       F7       F8       F9       F10
```

FORM.FTX中的字典元素（如编辑数据项的示例字典中所示）用于创建图10-6中所示的表单。

编辑数据项的示例字典

```
* Dictionary Form File: form.ftx
*
* Generate form.kl which should be included in your KAREL program
.kl form
.form
$-,form2
&home &reverse " Title here" &standard                          &new_line
"    label here     "                                          &new_line
"    Integer:       "    "-%10d"                               &new_line
"    Integer:       "    "-%10d(1,32767)"                      &new_line
"    Real:          "    "-%12f"                               &new_line
"    Bolean:        "    "-%10B(bool_fkey)"                    &new_line
"    String:        "    "-%-20k"                              &new_line
"    String:        "    "-%12k(clear)"                        &new_line
"    Byte:          "    "-%10b"                               &new_line
"    Short:         "    "-%10h"                               &new_line
"    DIN[1]:        "    "-%10P(dout_fkey)"                    &new_line
"    AIN[1]:        "    " "-%10pu" " " "-%1S(sim_fkey)"       &new_line
"    AOUT[2]:       "    " "-%10px" " " "-%1S(sim_fkey)"       &new_line
"    Enum Type:     "    "-%8n(enum_fkey)"                     &new_line
"    Enum Type:     "    "-%6w(enum_subwin)"                   &new_line
"    Enum Type:     "    "-%6V(ENUM_VAR)"                      &new_line
"    Prog Type:     "    "-%12pk(1)"                           &new_line
"    Prog Type:     "    "-%12pk(2)"                           &new_line
"    Prog Type:     "    "-%12pk(6)"                           &new_line
"    Prog Type:     "    "-%12pk(16)"                          &new_line
     ^form2_fkey
.endform
$-,form2_fkey
       EXIT" &new_line
*Allows you to specify the labels for F4 and F5 function keys
$-,bool_fkey
"FALSE"    &new_line   *   F5 key label, value will be set FALSE
"TRUE"    &new_line    *   F4 key label, value will be set TRUE
* Allows you to specify the labels for F4 and F5 function keys
$-, dout_fkey
"OFF"      &new_line    *   F5 key label, value will be set OFF
"ON"       &new_line    *   F4 key label, value will be set
ON
*Allows you to specify the labels for F4 and F5 function keys
$-, sim_fkey
" UNSIM "  &new_line    *   F5 key label, port will be unsimulated
"SIMULATE" &new_line    *   F4 key label, port will be simulated
*Allows you to specify the labels for 5 function keys
$-, enum_fkey
```

```
"FINE"      &new_line  * F1 key label, value will be set to 1
"COARSE"    &new_line  * F2 key label, value will be set to 2
"NOSETTL"   &new_line  * F3 key label, value will be set to 3
"NODECEL"   &new_line  * F4 key label, value will be set to 4
"VARDECEL"  &new_line  * F5 key label, value will be set to 5
*Allows you to specify a maximum of 35 choices in a subwindow
$-,enum_subwin
"Red"       *      value will be set to 1
$-
"Blue"      *      value will be set to 2
$-
"Green"
$-
"Yellow"
$-
"\a"         *      specifies end of subwindow list
* Allows you to specify the choices for the subwindow in a
* variable whose type is an ARRAY[m] of STRING[n].
$-,enum_var
"RUNFORM"   &new_line     *    program name of variable
"CHOICES"   &new_line     *    Variable name containing choices
```

编辑数据项的示例程序中显示的程序用于显示图10-6。

编辑数据项的示例程序

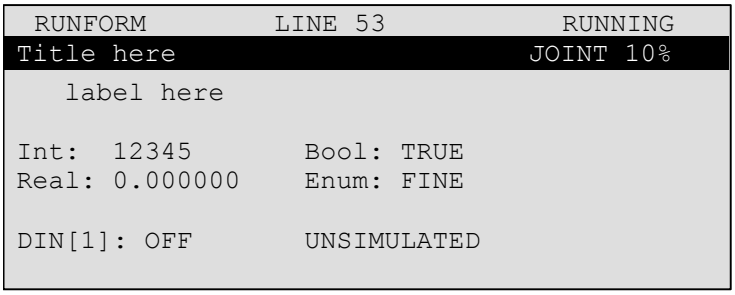
```
PROGRAM runform
%NOLOCKGROUP
%INCLUDE form     -- allows you to access form element numbers
%INCLUDE klevccdf
%INCLUDE klevkeys
%INCLUDE klevkmsk
TYPE
      mystruc = STRUCTURE
      byte_var1: BYTE
      byte_var2: BYTE
      short_var: SHORT
ENDSTRUCTURE
VAR
      device_stat: INTEGER -- tp_panel or crt_panel
      value_array: ARRAY [20] OF STRING [40]
      inact_array: ARRAY [1] OF BOOLEAN
      change_array: ARRAY[1] OF BOOLEAN
      def_item: INTEGER
      term_char: INTEGER
```

```
      status: INTEGER
int_var1: INTEGER
int_var2: INTEGER
real_var: REAL
bool_var: BOOLEAN
str_var1: STRING[20]
str_var2: STRING[12]
struc_var: mystruc
color_sel1: INTEGER
color_sel2: INTEGER
prog_name1: INTEGER[12]
prog_name2: STRING[12]
Prog_name3: STRING[12]
prog_name4: STRING[12]
choices: ARRAY[5] OF STRING[12]
BEGIN
      value_array [1] = 'int_var1'
      value_array [2] = 'int_var2'
      value_array [3] = 'real_var'
      value_array [4] = 'bool_var'
      value_array [5] = 'str_var1'
      value_array [6] = 'str_var2'
      value_array [7] = 'struc_var.byte_var1'
      value_array [8] = 'struc_var.short_var'
      value_array [9] = 'din[1]'
      value_array [10] = 'ain[1]'
      value_array [11] = 'ain[1]'
      value_array [12] = 'aout[2]'
      value_array [13] = 'aout[2]'
      value_array [14] = '[*system*]$group[1].$termtype'
      value_array [15] = 'color_sel1'
      value_array [16] = 'color_sel2'
      value_array [17] = 'prog_name1'
      value_array [18] = 'prog_name2'
      value_array [19] = 'prog_name3'
      value_array [20] = 'prog_name4'
      choices [1] = ''          --不使用
      choices [2] = 'Red'       --对应color_sel12 = 1
      choices [3] = 'Blue'      --对应color_sel12 = 2
      choices [4] = 'Green'     --对应color_sel12 = 3
      choices [5] = 'Yellow'    --对应color_sel12 = 4
-- 初始化变量
int_var1 = 12345
-- 故意保留int_var2未初始化
real_var = 0
bool_var = TRUE
str_var1 = 'This is a test'
```

```
-- str_var = is purposely left uninitialized
struc_var.byte_var1 = 10
struc_var.short_var = 30
color_sel1 = 3                --对应于enum_subwin的第三项
color_sel2 = 1
device_stat = crt_panel       --指定要显示的CRT / KB
form
FORCE_SPMENU(device_stat, SPI_TPUSER2,1)
def_item = 1 -- start with menu item 1
DISCTRL_FORM('FORM', form2, value_array, inact_array,
       change_array, kc_func_key, def_item, term_char, status);
END runform
```

图10-7显示了示教器上显示的FORM.FTX中的第三个模板。 此示例包含仅显示项目。 它显示了如何从KAREL程序自动加载表单字典文件和可变数据文件。

图10-7. 仅显示数据项的示例



FORM.FTX 中的字典元素（在“仅显示数据项的示例字典”中显示）用于创建图10-7中所示的表单。

仅显示数据项的示例字典

```
*       字典表单文件: form.ftx
*
*       生成应包含在KAREL程序中的form.kl
.kl form
.form
$-,form3
      &home &reverse "Title here" &standard    &new_line
      "label here"                             &new_line
                                               &new_line
"Int: " "%-10d" " Bool: " "%-10B(bool_fkey)"   &new_line
"Real: " "%-10f" " Enum: " "%-10n(enum_fkey)"  &new_line
"DIN[""%1d""]: " "%-10P(dout_fkey)" "%-12S(sim2_fkey)"
*您可以拥有任意数量的列，而不超过* 40列。
```

```
*You can specify blank lines too.
.endform
$-,sim2_fkey
  "UNSIMULATED" &new_line  * F5键标签，端口将被取消模拟
  "SIMULATED"   &new_line  * F4键标签，端口将被模拟
```

使用示例程序显示数据项中显示的程序用于显示图10-7中所示的表单。

仅显示数据项的示例程序

```
PROGRAM runform
%NOLOCKGROUP
 %INCLUDE form      -- 允许您访问表单元素编号
 %INCLUDE klevccdf
 %INCLUDE klevkeys
 %INCLUDE klevkmsk
        device_stat: INTEGER -- tp_panel or crt_panel
        value_array: ARRAY [20] OF STRING [40]
        inact_array: ARRAY [1] OF BOOLEAN -- not used
        change_array: ARRAY[1] OF BOOLEAN -- not used
        def_item: INTEGER
        term_char: INTEGER
        status: INTEGER
        loaded: BOOLEAN
        initialized: BOOLEAN
int_var1: INTEGER
int_var2: INTEGER
real_var: REAL
bool_var: BOOLEAN
BEGIN
-- Make sure 'FORM' dictionary is loaded.
CHECK_DICT('FORM', form3, status)
IF status <> 0 THEN
      WRITE TPPROMPT(CR,'Loading form.....')
      KCL ('CD MF2:',status)            -- 使用KCL CD命令
                                        -- 将目录更改为MF2:
      KCL ( 'LOAD FORM', status)        -- 使用KCL加载命令
                                        -- 加载表格
      IF status <> 0 THEN
            WRITE TPPROMPT(CR,'loading from failed, STATUS=',status)
            ABORT      -- 没有字典，这个程序就无法继续。
      ENDIF
ELSE
      WRITE TPPROMPT (CR,'FORM already loaded.')
ENDIF
```

```
      value_array [1] = 'int_var1'
      value_array [2] = 'bool_var'
      value_array [3] = 'real_var'
      value_array [4] = '[*system*]$group[1].$termtype'
      value_array [5] = 'int_var2'
      value_array [6] = 'din[1]'
      value_array [7] = 'din[1]'
int_var1 = 12345
bool_var = TRUE
real_var = 0
int_var2 = 1
device_stat = tp_panel
FORCE_SPMENU(device_stat, SPI_TPUSER2,1)
def_item = 1 -- start with menu item 1
DISCTRL_FORM('FORM', form3, value_array, inact_array,
      change_array, kc_func_key, def_item, term_char, status);
END runform
```

第11章

SOCKET 通讯

目录

第11章 SOCKET 通讯 11–1
11.1 概述 11–2
11.2 系统要求 11–2
11.2.1 概述 11–2
11.2.2 软件要求 11–2
11.2.3 硬件要求 11–2
11.3 配置 SOCKET 通讯选项 11–3
11.3.1 概述 11–3
11.3.2 设置服务器标记 11–3
11.3.3 设置客户端标记 11–6
11.4 SOCKET 通讯和 KAREL 11–9
11.4.1 概述 11–9
11.4.2 MSG_CONN(*string, integer*) 11–9
11.4.3 MSG_DISCO(*string, integer*) 11–9
11.4.4 MSG_PING(*string, integer*) 11–10
11.4.5 在Socket通讯传递连接期间交换数据 11–10
11.5 网络性能 11–10
11.5.1 概述 11–10
11.5.2 良好实施指南 11–11
11.6 编程示例 11–11
11.6.1 11–11
11.6.2 KAREL客户端应用程序 11–11
11.6.3 KAREL服务器应用程序 11–14
11.6.4 ANSI C环回客户端示例 11–16

11.1 概述

用户Socket 通讯传递选项为您提供了使用KAREL的TCP / IP Socket消息传递的好处。
Socket 通讯可以使用LINUX或UNIX工作站在网络机器人和远程PC之间进行数据交换。
Socket 通讯的典型应用可能是运行KAREL程序的机器人，该程序将进程信息发送到远程PC上的监视程序。建议将机器人上的PC接口选项与PC上的PC-Developers Kit结合使用，以便在机器人和基于Windows的PC之间进行数据交换。
Socket 通讯使用TCP / IP协议在网络上传输原始数据或原始未格式化数据。 Socket 通讯用于传输数据的命令和方法是TCP / IP协议的一部分。由于Socket 通讯支持客户端和服务器标记，因此需要超时，检测信号或数据格式化命令的应用程序可以在Socket 通讯传递连接的客户端和服务器（应用程序）端提供这些附加语义。

11.2 系统要求

11.2.1 概述

本节包含有关Socket 通讯传递与某些典型网络软件，传输协议和接口硬件的兼容性的信息。

11.2.2 软件要求

Socket 通讯与所有其他Internet选项兼容，包括DNS，FTP，Web服务器和Telnet。
注意客户端和服务器标记在Socket 通讯和其他支持的协议（如FTP）之间共享。 例如，可以为FTP操作或SM（Socket 通讯）操作设置标记。

11.2.3 硬件要求

Socket 通讯传递与使用TCP / IP网络协议的所有网络硬件配置兼容。 其中一些网络硬件配置包括以太网，串行PPP连接和PPP调制解调器连接。

11.3 配置 SOCKET 通讯选项

11.3.1 概述

要使用Socket 通讯，您需要配置以下网络硬件和软件参数：

- 在服务器上
  - 要用于Socket 通讯传递的端口
- 在客户端，
  - 服务器的IP地址或名称
  - 要用于Socket 通讯传递的服务器上的端口。

使用过程11-1设置Socket 通讯 Server标记。 使用过程11-2设置Socket 通讯 Client标记。

注意服务器侦听的服务器端口应与客户端尝试连接的端口匹配。

11.3.2 设置服务器标记

您需要配置要用于Socket 通讯传递的服务器标记。 使用过程11-1设置服务器标签。

注意如果要使用的服务器标签正由TCP / IP以外的网络协议使用，则需要先取消定义标记，然后才能将其用于Socket 通讯传递。 确保要使用的标记对网络的其他组件不重要后，必须取消定义标记。

过程**11-1**设置服务器标记

条件

- 您要设置的标记未配置为由网络上的其他设备使用。

步骤

1. 冷启动控制器。
   a. 在示教器上，按住SHIFT和RESET键。 或者，在操作面板上，按住RESET。
   b. 在仍然按下示教器上的SHIFT和RESET（或操作面板上的RESET）的同时，打开电源断开电路断路器。
   c. 释放所有键

2. 在示教器上，按MENUS。
3. 选择“设置”。
4. 按F1，[TYPE]。
5. 选择Host Comm。
6. 按F4，[显示]。
7. 选择服务器。
8. 将光标移动到要为Socket Messaging设置的标记，然后按F3，DETAIL。 您将看到类似于以下内容的屏幕。

```
SETUP Tags

  Tag S3:

  Comment:            ****************
  Protocol Name:      ********
  Current State:      UNDEFINED
  Startup State:
  Server IP/Hostname:*****************
  Remote Path/Share: *****************
  Port:               *****
  Inactivity Timeout:  15 min
  Username:                    anonymous
  Password                    **********
```

9. **将光标移动到协议名称，然后按F4，[CHOICE]。**
10. **选择SM。**
11. **将光标移动到Startup State，然后按F4，[CHOICE]。**
12. **选择START。**
13. **按F2，[动作]。**
14. **选择DEFINE。**
15. **按F2，[动作]。**
16. **选择START。**
17. **设置系统变量：**
    a. 按下菜单
    b. 选择NEXT.
    c. 选择SYSTEM，按F1 [TYPE]。
    d. 选择变量。

e. 将光标移动到$ HOSTS_CFG，然后按ENTER。

f. 将光标移动到与步骤8中选择的标记对应的结构。 例如，如果要设置标记S3，请移动光标结构元素[3]，如下面的屏幕所示。

```
   系统变量
$HOSTS_CFG
   1    [1]            HOST_CFG_T
   2    [2]            HOST_CFG_T
   3    [3]            HOST_CFG_T
   4    [4]            HOST_CFG_T
   5    [5]            HOST_CFG_T
   6    [6]            HOST_CFG_T
   7    [7]            HOST_CFG_T
   8    [8]            HOST_CFG_T
```

g. 按ENTER键。 您将看到类似于以下内容的屏幕。

```
   系统变量
$HOSTS_CFG[3]
  1 $COMMENT         *uninit*
  2 $PROTOCOL        'SM'
  3 $PORT            *uninit*
  4 $OPER            3
  5 $STATE           3
  6 $MODE            *uninit*
  7 $REMOTE          *uninit*
  8 $REPERRS         FALSE
  9 $TIMEOUT         15
 10 $PATH            *uninit*
 11 $STRT_PATH       *uninit*
 12 $STRT_REMOTE     *uninit*
 13 $USERNAME        *uninit*
 14 $PWRD_TIMOUT     0
 15 $SERVER_PORT     0
```

h. 将光标移动到$ SERVER_PORT。 键入要用于socket 通讯传递的TCP / IP端口的名称。服务器标签现在可以从KAREL程序中使用了。

11.3.3 设置客户端标记

您需要配置要用于socket 通讯传递的客户端标记。 使用过程11-2设置服务器标记。 您还可以使用过程11-2取消定义标记。
注意如果要使用的客户端标记正由TCP / IP以外的网络协议使用，则需要先取消定义标记，然后才能将其用于socket 通讯传递。

过程11-2设置客户端标记

条件

- 您要设置的标记未配置为由网络上的其他设备使用。

步骤

1. 冷启动控制器。
   a. 在示教器上，按住SHIFT和RESET键。 或者，在操作面板上，按住RESET。
   b. 在然后按下示教器上的SHIFT和RESET（或操作面板上的RESET）的同时，打开电源断开电路断路器。
   c. 释放所有键。
2. 在示教器上，按菜单。
3. 选择“设置”。
4. 按F1，[TYPE]。
5. 选择Host Comm。
6. 按F4，[显示]。
7. 选择客户端。
8. 将光标移动到要为Socket 通讯设置的标记，然后按F3，DETAIL。 您将看到类似于以下内容的屏幕。

```
SETUP Tags

  Tag C3:

  Comment:            ****************
  Protocol Name:      ********
  Current State:      UNDEFINED
  Startup State:
  Server IP/Hostname:****************
  Remote Path/Share: ****************
  Port:               *****
  Inactivity Timeout:  15 min
  Username:                  anonymous
  Password                  **********
```

9. 将光标移动到Protocol Name项，然后按F4，[CHOICE]。
10. 选择SM。
11. 将光标移动到Startup State项，按F4，[CHOICE]，然后选择DEFINE。
12. 将光标移动到“服务器IP /主机名”项，然后按Enter。
13. 键入要用于Socket 通讯传递的远程主机服务器的主机名或IP地址。

    注意：如果未使用DNS，则必须将远程主机及其IP地址添加到主机条目表中。
14. 按F2，[ACTION]，然后选择DEFINE。
15. **设置系统变量：**
    a. 按菜单。
    b. 选择下一步。
    c. 选择SYSTEM，然后按F1，[TYPE]。
    d. 选择变量。
    e. 将光标移动到$ HOSTC_CFG，然后按ENTER键。
    f. 将光标移动到与步骤8中选择的标记对应的结构。例如，如果要设置标记C3，请移动光标结构元素[3]，如下面的屏幕所示。

```
SYSTEM Variables
$HOSTC_CFG
  1    [1]           HOST_CFG_T
  2    [2]           HOST_CFG_T
  3    [3]           HOST_CFG_T
  4    [4]           HOST_CFG_T
  5    [5]           HOST_CFG_T
  6    [6]           HOST_CFG_T
  7    [7]           HOST_CFG_T
  8    [8]           HOST_CFG_T
```

g. 按ENTER键。 您将看到类似于以下内容的屏幕。

```
SYSTEM Variables
$HOSTC_CFG[3]
  1 $COMMENT         *uninit*
  2 $PROTOCOL        'SM'
  3 $PORT            *uninit*
  4 $OPER            3
  5 $STATE           3
  6 $MODE            *uninit*
  7 $REMOTE          *uninit*
  8 $REPERRS         FALSE
  9 $TIMEOUT         15
 10 $PATH            *uninit*
 11 $STRT_PATH       *uninit*
 12 $STRT_REMOTE     *uninit*
 13 $USERNAME        *uninit*
 14 $PWRD_TIMOUT     0
 15 $SERVER_PORT     0
```

将光标移动到$ SERVER_PORT。 键入要用于Socket 通讯传递的TCP / IP服务器端口的名称。 客户端标记现在可以使用KAREL程序了。

11.4 SOCKET 通讯和 KAREL

11.4.1 概述

SOCKET 通讯传递是KAREL的集成组件。 当您使用KAREL程序中的SOCKET 通讯传递函数和实用程序时，语法类似于其他文件读取和写入操作，但您在使用SOCKET 通讯传递函数和实用程序时需要建立网络连接。

以下KAREL SOCKET 通讯传递功能和实用程序使服务器能够与网络上的远程主机建立连接。本节中有几个KAREL程序示例提供了这些函数和实用程序如何与KAREL文件读写函数和实用程序一起使用来编写完整的SOCKET 通讯 KAREL客户机或服务器程序或应用程序的示例。

环境flbt语句需要使用任何列出的内置函数（%ENVIRONMENT flbt）。

11.4.2 MSG_CONN(*string, integer*)

在任何标记可用于SOCKET 通讯传递之前，需要调用MSG_CONN。

此命令的第一个参数包含标记名称（例如“S1：”），第二个参数是包含操作状态的整数。

如果使用此命令连接到服务器标记，则只有在远程客户端设备与此服务器标记建立连接后，此命令才会返回状态值。

如果使用此命令连接到客户端标记，则仅当远程服务器尝试接受连接时，此命令才会返回状态值。如果连接成功，该命令将返回一个值，表明已成功建立连接。如果连接不成功，该命令将返回一个值，指示发生了连接错误。

在SOCKET 通讯传递会话期间，必须使用MSG_DISCO关闭与客户端或服务器标记的SOCKET连接，然后才能使用MSG_CONN进行任何后续尝试连接到同一客户端或服务器标记。

11.4.3 MSG_DISCO(*string, integer*)

MSG_DISCO用于关闭SOCKET 通讯传递连接。 如果连接丢失，可能是因为远程服务器终止SOCKET 通讯传递连接时发生READ或WRITE错误，您将需要使用MSG_DISCO来关闭与远程服务器的连接。 在这种情况下，必须使用MSG_DISCO来关闭客户端的连接，然后才能使用MSG_CONN建立与远程服务器的另一个连接。

此命令的第一个参数包含标记名称（例如“S1：”），第二个参数是一个整数值，指示在客户端关闭连接的状态。

11.4.4 MSG_PING(*string, integer*)

MSG_PING是一个实用程序命令，用于检查与远程主机的网络连接，以便您可以确定它当前是否已连接到网络。 MSG_PING命令将ping数据包发送到远程主机并等待回复。 （Ping数据包是在网络上的主机之间传输的数据块。）如果远程主机没有回复，这通常意味着您将无法使用其他网络协议，如FTP，TELNET或Socket 通讯 连接到该主机。 如果您未成功尝试使用Socket 通讯连接到远程主机，则MSG-PING实用程序是尝试诊断问题的好地方。
此命令的第一个参数包含要ping的远程主机的名称。 如果未在网络上使用DNS，则远程主机的主机名和IP地址必须存在于“主机条目”表中。

11.4.5 在Socket通讯传递连接期间交换数据

成功建立Socket通讯传递连接后，可以使用KAREL命令在连接的设备之间交换数据。
KAREL有几个可用于数据交换操作的命令：

- OPEN FILE
- WRITE
- READ
- BYTES_AHEAD

有关详细信息，请参阅KAREL参考手册。

11.5 网络性能

11.5.1 概述

网络上Socket通讯的性能将根据连接到网络的设备数量，控制器上运行的应用程序数量，网络布线配置以及消息到达目的地所需的跃点数而有所不同 设备。

注意Hops是一个术语，表示源主机和目标主机之间的路由器数量。 通常，数据从路由器到路由器的跳数越少，源和目标主机之间的数据传输速度就越快。 同一网络上的主机之间的数据传输速度最快。

11.5.2 良好实施指南

使用Socket通讯传递实现任何应用程序的解决方案时，请遵循以下准则。

- 您不能单独传输小数据，而是收集数据并作为更大的数据包传输。 这适用于读写。
- 您必须了解数据速率不会泛滥远程端。
- 您必须了解以太网上还有其他节点，因此无法保证性能。
- 您必须了解机器人上的其他应用程序也使用TCP / IP，性能可能会受到影响。

11.6 编程示例

11.6.1

本节包含KAREL Socket通讯传递客户端和KAREL Socket通讯传递服务器的编程示例。 还有一个基于UNIX的ANSI C示例用于环回客户端应用程序，它假定您可以访问与UNIX兼容的ANSI C编译器，以及使用ANSI C语言编程的基本知识。

注意KAREL示例假定已使用过程11-1和过程11-2为Socket通讯设置了相应的标记（客户端为C2，服务器为S3）。

11.6.2 KAREL 客户端应用程序

KAREL客户端应用程序为基本KAREL客户端应用程序提供代码，该应用程序可用于建立与远程主机的Socket通讯传递连接，远程主机可以是KAREL服务器应用程序中显示的KAREL服务器Socket通讯传递应用程序。

KAREL客户端应用程序

```
--      该素材是 Fanuc Robotics Corporation 和 FANUC LTD Japan 的共同财产
```

```
--
--    必须立即返回Fanuc Robotics Corporation或FANUC LTD Japan。 未经Fanuc
--
--    Robotics和FANUC事先书面同意，不得以任何方式复制，复制，使用或传播本资料及其中所示或
--
--    包含的信息的全部或部分内容。
--
--
--              All Rights Reserved
--              Copyright (C)   2000
--              Fanuc Robotics Corporation
--              FANUC LTD Japan
--
--              Karel is a registered trademark of
--              Fanuc Robotics Corporation
--    +
--    Program: loopcl.kl - Program for TCP Messaging
--
--    Description:
--
--    This program serves as an example on how to use TCP messaging
--    and write a client Karel program.
--
--    Authors: Fanuc Robotics Corporation
--              3900 West Hamlin
--              Rochester Hills, MI 48309
--
--    Modification history:
--
-------------------------------------------------------------------------------
PROGRAM loopcl
%STACKSIZE = 4000
%NOLOCKGROUP
%NOPAUSE=ERROR+COMMAND+TPENABLE
%ENVIRONMENT uif
%ENVIRONMENT sysdef
%ENVIRONMENT memo
%ENVIRONMENT kclop
%ENVIRONMENT bynam
%ENVIRONMENT fdev
%ENVIRONMENT flbt
%INCLUDE klevccdf
%INCLUDE klevkeys
%INCLUDE klevkmsk
-------------------------------------------------------------------------------
VAR
   file_var : FILE
   tmp_int  : INTEGER
   tmp_str  : STRING[128]
```

```
   status   : INTEGER
   entry    : INTEGER
   loop1    : BOOLEAN
-------------------------------------------------------




BEGIN
   SET_FILE_ATR(file_var, ATR_IA)
   SET_VAR(entry, '*SYSTEM*','$HOSTC_CFG[2].$SERVER_PORT',59002,status)
   -- Connect the tag
   WRITE('Connecting..',cr)
   MSG_CONNECT('C2:',status)
   WRITE(' Connect Status = ',status,cr)
   loop1 = TRUE
   IF status = 0 THEN
      WHILE loop1 = TRUE DO
         WRITE('Opening File..',cr)
         OPEN FILE file_var('rw','C2:')
         status = IO_STATUS(file_var)
         IF status = 0 THEN
            FOR tmp_int = 1 TO 100 DO
               tmp_str = '0123456789012345'
               WRITE file_var(tmp_str::10)
               WRITE('Wrote 126 Bytes',cr)
               IF status <> 0 THEN
                  WRITE('Loop Test Fails',cr)
                  loop1 = FALSE
                  tmp_int = 100
               ELSE
                  WRITE('Read 126 Bytes',cr)
                  READ file_var(tmp_str::10)
                  status = IO_STATUS(file_var)
                  WRITE('Read Status ',status,cr)
               ENDIF
            ENDFOR
            WRITE('Closed File',cr)
            CLOSE FILE file_var
         ELSE
            WRITE('Error Opening File',cr)
            loop1 = FALSE
         ENDIF
      ENDWHILE
      WRITE('Disconnecting..',cr)
      MSG_DISCO('C2:',status)
      WRITE('Done.',cr)
```

```
   ENDIF
END loopcl
```

11.6.3 KAREL服务器应用程序

KAREL Server Application为基本KAREL服务器应用程序提供代码，该应用程序可用于承载由远程客户端（可能是A KAREL客户端应用程序中显示的KAREL客户端Socket通讯传递应用程序）创建的Socket通讯传递连接。

KAREL服务器应用程序

```
--    This material is the joint property of Fanuc Robotics Corporation and
--    FANUC LTD Japan, and must be returned to either Fanuc Robotics
--    Corporation or FANUC LTD Japan immediately upon request.  This material
--    and the information illustrated or contained herein may not be
--    reproduced, copied, used, or transmitted in whole or in part in any way
--    without the prior written consent of both Fanuc Robotics and FANUC.
--
--    All Rights Reserved
--    Copyright (C) 2000
--    Fanuc Robotics Corporation
--    FANUC LTD Japan
--             Karel is a registered trademark of
--             Fanuc Robotics Corporation
--    +
--    Program: tcpserv3.kl - Program for TCP Messaging
--
--    Description:
--
--    This program serves as an example on how to use TCP messaging and write
--    a server Karel program.
--
--    Authors: Fanuc Robotics Corporation
--             3900 West Hamlin
--             Rochester Hills, MI 48309
--
--    Modification history:
--
--------------------------------------------------------------------------------
PROGRAM tcpserv3
%STACKSIZE = 4000
%NOLOCKGROUP
%NOPAUSE=ERROR+COMMAND+TPENABLE
%ENVIRONMENT uif
```

```
%ENVIRONMENT sysdef
%ENVIRONMENT memo
%ENVIRONMENT kclop
%ENVIRONMENT bynam
%ENVIRONMENT fdev
%ENVIRONMENT flbt
%INCLUDE klevccdf
%INCLUDE klevkeys
%INCLUDE klevkmsk
--------------------------------------------------------------------------------
VAR
   file_var : FILE
   tmp_int  : INTEGER
   tmp_int1 : INTEGER
   tmp_str  : STRING[128]
   tmp_str1 : STRING[128]
   status   : INTEGER
   entry    : INTEGER
--------------------------------------------------------------------------------



BEGIN
   SET_FILE_ATR(file_var, ATR_IA)
   -- set the server port before doing a connect
   SET_VAR(entry, '*SYSTEM*','$HOSTS_CFG[3].$SERVER_PORT',59002,status)
   WRITE('Connecting..',cr)
   MSG_CONNECT('S3:',status)
   WRITE(' Connect Status = ',status,cr)
   IF status = 0 THEN
      -- Open S3:
      WRITE ('Opening',cr)
      FOR tmp_int1 = 1 TO 20 DO
         OPEN FILE file_var ('rw','S3:')
         status = IO_STATUS(file_var)
         WRITE (status,cr)
         IF status = 0 THEN
            -- write an integer
            FOR tmp_int = 1 TO 1000 DO
               WRITE('Reading',cr)
               -- Read 10 bytes
               BYTES_AHEAD(file_var, entry, status)
               WRITE(entry, status, cr)
               READ file_var (tmp_str::10)
               status = IO_STATUS(file_var)
               WRITE (status, cr)
```

```
                -- Write 10 bytes
                WRITE (tmp_str::10, cr)
                status = IO_STATUS(file_var)
                WRITE (status, cr)
             ENDFOR
             CLOSE FILE file_var
          ENDIF
       ENDFOR
       WRITE('Disconnecting..',cr)
       MSG_DISCO('S3:',status)
       WRITE('Done.',cr)
    ENDIF
END tcpserv3
```

11.6.4 ANSI C环回客户端示例

ANSI C基于UNIX的环回客户端示例提供了一个基于UNIX的环回客户端的示例，该客户端可用于与远程主机建立连接。

ANSI C基于UNIX的环回客户端示例

```
/* BSD Standard Socket Programming Example - UNIX */
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#define SERV_TCP_PORT 59002
#define SERV_HOST_ADDR "199.5.148.56"
#define MAXLINE 512
int written(int fd, char *ptr, int nbytes);
int readline(int fd, char *ptr, int maxlen);
void str_cli(int sockfd);
char *pname;
int main(int argc, char *argv[])
{
  int sockfd;
  struct sockaddr_in serv_addr;

  pname = argv[0];

  bzero((char *) &serv_addr, sizeof(serv_addr));
  serv_addr.sin_family = AF_INET;
  serv_addr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);
```

```
  serv_addr.sin_port = htons(SERV_TCP_PORT);

  if((sockfd = socket(AF_INET, SOCK_STREAM,0)) < 0){
    printf("Client: Can't Open Stream Socket\n");
  }

  printf("Client: Connecting...\n");

  if(connect(sockfd,(struct sockaddr *) &serv_addr, sizeof(serv_addr))<0){
    printf("Client: Can't Connect to the server\n");
  }
  else{
    str_cli(sockfd);
  }
  exit(0);
}



void str_cli (int sockfd)
{
  int n, i;
  char sendline[MAXLINE], recvline[MAXLINE + 1];
  while(1)
  {
    memset (sendline, 2, 128);
    if(written(sockfd, sendline, 126)!=126){
       printf("strcli:written error on sock\n");
      }
      i = readline(sockfd, recvline, 126);
  }
}
int readline(int fd, char *ptr, int maxlen)
{
  int n, rc;
  char c;
  for(n = 0; n < maxlen; n++){
     if((rc = read(fd, &c, 1)) == 1){
        *ptr++ = c;
        if(c=='\n'){
          break;
         }
         else if(rc== 0) {
            if(n== 0) {
              return (0);
             }
```

```
            else{
              break;
           }
       }
    }
    else{
      return (-1);
    }
   }
*ptr = 0;
return (n);
 }
int written(int fd, char *ptr, int nbytes)
{
  int nleft, nwritten;
  nleft = nbytes;
  while(nleft > 0) {
    nwritten = write(fd, ptr, nleft);
    if(nwritten <= 0) {
       return(nwritten);
     }
     nleft -= nwritten;
     ptr += nwritten;
}
return(nbytes - nleft);
}
```

第12章

系统变量

目录

第12章 系统变量 12–1
12.1 访问权限 12–2
12.2 存储 12–3

系统变量是声明为KAREL系统软件一部分的变量。 它们具有永久定义的以美元符号（$）开头的变量名称。 许多系统变量是结构变量，在这种情况下，每个字段也以美元符号（$）开头。 许多是机器人特定的，这意味着它们的值取决于连接到系统的机器人的类型。

系统变量具有以下特征：

- •它们具有预定义的数据类型，可以是任何一种有效的KAREL数据类型。
- •系统变量的初始值是内部默认值或存储在默认系统变量文件SYSDEF.SV中的变量。
- •从FILE屏幕或KCL加载和保存系统变量时，系统变量文件名默认为SYSVARS.SV。
- •访问权限控制您是否可以检查或更改系统变量。
- •可以保存已修改的系统变量以反映系统的当前状态。

另请参阅：第2章语言元素，以获取有关KAREL中可用数据类型的更多信息

12.1 访问权限

以下规则适用于系统变量：

- •如果系统变量允许KAREL程序读取其值，则可以在与使用KAREL程序中的程序变量值或常量值相同的上下文中使用该值。
- 例如，这些系统变量可以在赋值语句的右侧使用，也可以在控制语句中用作测试条件。
- •如果系统变量允许KAREL程序写入其值，则可以在为KAREL程序中的变量赋值的任何上下文中使用该系统变量。

表12-1列出了程序访问权限的符号。 这些符号是针对FANUC Robotics SYSTEM软件参考手册中的每个系统变量给出的。

表12-1。 系统变量的访问权限

权限	**意义**
NO	没有访问
RO	只读
RW	读和写
FP	现场保护; 如果它是结构变量，则前三个访问权限之一将适用。

另请参阅：FANUC机器人软件参考手册，了解系统变量

12.2 存储

系统变量在加电时根据基准分配初始值

- 内部默认值
- 存储在默认系统变量文件SYSDEF.SV中的值

第13章

KAREL 命令语言 (KCL)

目录

第13章 KAREL 命令语言（KCL） 13–1
13.1 命令格式 13–2
13.1.1 默认程序 13–2
13.1.2 变量和数据类型 13–3
13.2 程序控制命令 13–3
13.3 输入命令 13–3
13.3.1 缩写 13–4
13.3.2 错误消息 13–4
13.3.3 子目录 13–4
13.4 命令程序 13–4
13.4.1 命令过程格式 13–5
13.4.2 创建命令过程 13–6
13.4.3 错误处理 13–6
13.4.4 执行命令过程 13–6

KAREL命令语言（KCL）环境包含一组可用于指导KAREL系统的命令。 KCL命令允许您开发和执行程序，处理文件，获取有关系统的信息，以及执行许多其他日常操作。

通过按菜单（F10）并从菜单中选择KCL，可以在CRT / KB上显示KCL环境。

除了直接在KCL提示符下输入命令外，还可以从命令文件执行KCL命令。

13.1 命令格式

命令条目由命令关键字以及与该命令关联的任何参数或参数组成。 某些命令还需要指定命令对象的标识符。

- KCL命令关键字是动作词，例如LOAD，EDIT和RUN。 命令参数或参数有助于定义关键字应该作用于哪个对象。
- 许多KCL命令都有与之关联的默认参数。 对于这些命令，您只需输入关键字，系统将提供默认参数。
- KCL支持使用星号（*）作为通配符，允许您将一组对象指定为以下KCL命令的命令参数：
  - COPY
  - DELETE FILE
  - DIRECTORY
- KCL标识符遵循与KAREL编程语言中的标识符相同的规则。
- KCL支持KAREL编程语言支持的所有数据类型。 因此，您可以在KCL中创建和设置变量。

另请参阅：第2章语言元素和第9章文件系统，

13.1.1 默认程序

将程序名称设置为程序名称参数和文件名参数的缺省值，可以在不键入名称的情况下发出 KCL命令。

可以通过执行以下操作之一来设置KCL默认程序：

- 使用 SET DEFAULT KCL 命令
- 在 CRT / KB 的 SELECT 菜单中选择程序名称

13.1.2 变量和数据类型

KCL> CREATE VARIABLE 命令允许您声明变量。 KCL> SET VARIABLE 命令允许您为声明的变量赋值。 分配的值可以是 INTEGER，REAL，BOOLEAN 和 STRING 数据类型。 可以将值分配给特定的 ARRAY 元素或指定的 PATH 节点。 VECTOR 变量被指定为三个REAL 值，POSITION 变量被指定为六个 REAL 值。

另请参阅：附录C“KCL命令按字母顺序说明”中的 CREATE VARIABLE 和 SET VARIABLE KCL命令

13.2 程序控制命令

KCL命令可用于运行程序。 在某些情况下，这些程序可能会导致运动，例如运行示教程序或运行调用示教程序的KAREL程序时。 发出KCL命令的设备必须具有运动控制才能执行此操作。 有关将运动控制分配给远程设备的详细信息，请参阅“FANUC机器人软件参考手册”中的$ RMT_MASTER说明。

程序控制命令：

- 可立即引起机器人和/或辅助轴运动，或有可能引起运动
- 只有在满足许多条件时才能执行

以下命令是程序控制命令：

- CONTINUE
- RUN

> **警告**
>
> **在发出程序控制命令或启动在加电时自动执行程序的机器人之前，请确保机器人工作范围没有人员。 否则，您可能会伤害人员或损坏设备。**

13.3 输入命令

您只能从 CRT / KB 输入 KCL 命令。

要输入 KCL 命令：

1. 按 CRT / KB 上的 MENUS（F10）。
2. 选择 KCL。
3. 在 KCL 提示符下输入命令。

通过输入需要多个关键字的KCL命令的第一个关键字，然后按ENTER键，将显示所有其他KCL关键字的列表。

例如，在KCL提示符下输入DELETE将显示以下可能的命令列表：“FILE，NODE或VARIABLE”。

注意向上箭头键可用于调用输入的最后十个命令中的任何一个。

13.3.1 缩写

只要缩写在KCL中是唯一的，任何KCL命令都可以缩写。 例如，TRAN对TRANSLATE和ED是唯一的EDIT。

13.3.2 错误消息

如果您错误地输入KCL命令，KCL将显示相应的错误消息并返回KCL>提示，允许您重新输入命令。 向上箭头（^）表示违规字符或违规字的开头。

13.3.3 子目录

存储卡设备上提供了子目录。 子目录允许在任何MS-DOS文件系统上格式化存储卡和闪存卡。 您可以在子目录上执行所有与KCL文件相关的命令。 您可以将子目录嵌套到多个级别。但是，FANUC Robotics不建议嵌套超过八个级别的子目录。

13.4 命令程序

命令过程是一系列KCL命令，存储在命令文件（.CF文件类型）中，可以按顺序自动执行。

- 命令过程允许您使用一系列KCL命令而无需反复输入。
- 使用RUNCF命令执行命令过程。

13.4.1 命令过程格式

除RUNCF之外的所有KCL命令都可以在命令过程中使用。 对于需要确认的命令，您可以在一行输入命令和确认，否则KCL将提示输入行上的确认。 命令过程中的确认显示CLEAR ALL作为KCL命令，YES显示为确认。

命令过程中的确认

在一行输入命令和确认：

```
CLEAR ALL YES
```

嵌套命令程序

嵌套命令过程时，请遵循以下准则：

- 可以通过在命令过程中使用 %INCLUDE filename 来嵌套命令过程。
- 命令程序的嵌套仅限于四个级别。

  如果尝试嵌套超过四个命令过程，KCL将检测错误并根据系统变量$ STOP_ON_ERR采取适当的操作。 有关$ STOP_ON_ERR的更多信息，请参见第13.4.3节。

另请参见：第13.4.3节 “错误处理”

延续性

KCL延续字符＆符号（＆）允许您在命令过程中跨多行继续命令输入。

您可以在关键字之间或特殊字符之间分解KCL命令。

例如，使用＆符号（＆）继续跨两行执行命令：

```
CREATE VAR [TESTING_PROG.]PICK_UP_PNT &
      :POSITION
```

评论

注释行可用于记录命令过程。 以下规则适用于在命令过程中使用注释：

- 带有两个连续连字符（ - ）的注释。
- 注释可以单独放置在一行上，也可以放在命令行的末尾。

13.4.2 创建命令过程

可以通过在命令文件中键入命令并保存文件来创建命令过程。 这可以使用全屏编辑器完成。
另请参阅：编辑KCL命令，附录C， “KCL命令按字母顺序说明”

13.4.3 错误处理

如果系统在执行命令过程时检测到KCL错误，系统将以两种方式之一处理错误，具体取决于系统变量$ STOP_ON_ERR的值：

- 如果检测到KCL错误时$ STOP_ON_ERR为TRUE，则命令过程终止，并返回KCL>提示符。
- 如果$ STOP_ON_ERR为FALSE，系统将忽略KCL错误，并且命令过程将运行至完成。

13.4.4 执行命令过程

除非使用 SET VERIFY OFF 命令，否则在执行命令过程中的每个命令都会显示。 每个命令前面都有命令文件中的行号。 但是，如果文件不在RD：设备上，则在执行之前将整个命令文件读入内存，并且将从显示中省略行号。
可以使用 KCL RUNCF 命令执行命令过程。

第14章

输入/输出系统

目录

第14章 输入/输出系统 14–1
14.1 用户定义的信号 14–2
14.1.1 DIN和DOUT信号 14–2
14.1.2 GIN和GOUT信号 14–3
14.1.3 AIN和AOUT信号 14–3
14.1.4 Hand 信号 14–5
14.2 系统定义的信号 14–5
14.2.1 机器人数字输入和输出信号（RDI / RDO） 14–6
14.2.2 操作面板输入和输出信号（OPIN / OPOUT） 14–6
14.2.3 TP示教器链输入和输出信号（TPIN / TPOUT） 14–17
14.3 串行输入/输出 14–22
14.3.1 串行输入/输出 14–22

输入/输出（I / O）系统通过KAREL为用户提供用户定义的I / O信号，系统定义的I / O信号和通信端口。 用户定义的I / O信号在KAREL程序中控制，允许您与外围设备和机器人手臂末端工具进行通信。 系统定义的I / O信号是KAREL系统为特定目的指定的信号。 还存在标准和可选的通信端口配置。

用户定义的I / O信号的数量取决于控制器硬件以及所选模块的类型和数量。

14.1 用户定义的信号

用户定义的信号是那些其含义由KAREL程序定义的输入和输出信号。 您可以通过以下预定义端口阵列访问用户定义的信号：

- DIN（数字输入）和 DOUT（数字输出）
- GIN（组输入）和 GOUT（组输出）
- AIN（模拟输入）和 AOUT（模拟输出）

除端口阵列外，您还可以通过 KAREL OPEN 和 CLOSE HAND 语句访问机器人手控信号。

14.1.1 DIN和DOUT信号

DIN和DOUT信号可以访问KAREL程序中单个输入或输出线路上的数据。

该程序将数据视为BOOLEAN数据类型。 该值为ON（活动）或OFF（不活动）。 您可以将信号的极性定义为高电平有效（施加电压时为ON）或低电平有效（未施加电压时为ON）。

输入信号在KAREL程序中通过名称DIN [n]访问，其中“n”是信号编号。

评估DIN信号使系统执行输入端口的读取操作。 除非已模拟DIN信号，否则为DIN信号分配值是无效操作。 除非已模拟DIN信号，否则永远不能在KAREL程序中设置这些。

评估DOUT信号会导致系统从指定的输出信号返回当前输出值。 为DOUT信号分配值会导致系统将输出信号设置为ON或OFF。

打开DOUT：

```
DOUT[n] = TRUE or
```

```
DOUT[n] = ON
```

To turn off a DOUT:

```
DOUT[n] = FALSE or
DOUT[n] = OFF
```

使用示教器 I / O 菜单或 KAREL 内置例程 SET_PORT_ASG 将数字信号分配给 I / O 设备上的端口。

14.1.2 GIN和GOUT信号

GIN和GOUT信号在KAREL程序中提供对DIN和DOUT的访问，作为一组输入或输出信号。组的大小可以是1到16位，每个位对应一个输入或输出信号。您可以定义组大小以及与特定组关联的DIN或DOUT。第一个（编号最小的）端口是组值的最低有效位。

该程序将数据视为INTEGER数据类型。未使用的位被解释为零。

输入信号在KAREL程序中以名称GIN [n]访问，其中“n”是组号。

评估GIN信号使系统执行输入端口的读操作。除非已模拟GIN信号，否则为GIN信号分配值是无效操作。除非已经模拟了GIN信号，否则永远不能在KAREL程序中设置它们。

设置GOUT信号会导致系统从指定的输出端口返回当前输出值。为GOUT信号分配值会导致系统执行输出操作。

要控制组输出，使用等于所需二进制输出的整数值。例如，命令GOUT [n] = 25将具有以下二进制结果“0000000000011001”，其中1 =输出打开，0 =输出关闭，最低有效位（LSB）是右侧的第一位。

您可以使用示教器I / O菜单或KAREL内置程序分配组信号

SET_PORT_ASG。

14.1.3 AIN和AOUT信号

AIN和AOUT信号可以在KAREL程序中访问模拟电信号。 对于输入信号，模拟数据由系统数字化并作为16位二进制数传递给KAREL程序，其中14位，12位或8位是重要的，具体取决于模拟模块。 该程序将数据视为INTEGER数据类型。 对于输出信号，输出对应于编程的INTEGER值的模拟电压。

输入信号在KAREL程序中以名称AIN [n]访问，其中“n”是信号编号。

评估AIN信号使系统执行输入端口的读操作。除非已模拟AIN信号，否则在示教器上设置AIN信号是无效操作。除非模拟了AIN信号，否则永远不能在KAREL程序中设置它们。

TP上显示的值或程序从模拟输入端口读取的值取决于提供给端口的电压和模数转换提供的有效数据的位数。对于正输入电压，读取的值将在0到2 **（N-1）-1的范围内，其中N是有效数据的位数。对于12位设备（大多数FANUC模块），这是2 ** 11-1或2047。

对于负输入电压，当电压从最小可检测负电压变化到器件处理的最大负电压时，该值将在2 ** N-1到2 **（N-1）的范围内。对于12位器件，这是从4095到2048。

用于将此输入转换为表示电压的实际值的KAREL逻辑示例，其中设备是处理+ 10v至-10v范围的12位设备，如下所示：

图14-1。 用于将输入转换为表示电压的实际值的KAREL逻辑

```
V: REAL
AINP: INTEGER
AINP = AIN[1]
IF (AINP <= 2047) THEN
V = AINP * 10.0 /2047.0
ELSE
V = (AINP - 4096) * 10.0 / 2047
ENDIF
```

在TPP中，将使用以下逻辑：

```
R[1] = AI[1]
IF (R[1] > 2047) JMP LBL[1]
R[2] = R[1] * 10
R[2] = R[2] / 2047
JMP LBL[2]
LBL[1]:
R[2] = R[1] - 4096
R[2] = R[2] * 10
R[2] = R[2] / 2047
LBL[2]
```

R [2]具有所需的电压。

评估AOUT信号会导致系统从指定的输出信号返回当前输出值。 为AOUT信号分配值会导致系统执行输出操作。

可以在KAREL程序中打开AOUT，其中AOUT [n] =（整数值）。 结果将是指定整数值的AOUT信号线[n]上的输出电压。 例如，AOUT [1] = 1000将在模拟输出线1上输出+5 V信号（使用具有12个有效位的输出模块）。

您可以使用示教器I / O菜单或KAREL内置程序分配模拟信号SET_PORT_ASG。

14.1.4 Hand 信号

您可以访问一组特殊的机器人HAND控制信号，用于通过KAREL语言HAND语句控制机械臂末端工具，而不是通过端口阵列。 HAND信号为KAREL程序提供了对两个输出信号的访问，这两个输出信号以协调的方式工作以控制工具。 信号被指定为开放线和闭合线。 系统最多可支持两个HAND信号。

HAND [1]使用与RDO [1]和RDO [2]相同的物理输出。

HAND [2]使用与RDO [3]和RDO [4]相同的物理输出。

提供以下KAREL语言语句用于控制信号，其中“n”是信号编号。

OPEN HAND n激活开放线，并关闭关闭线

CLOSE HAND n停用开放线，并激活关闭线

RELAX HAND n停用两条线

14.2 系统定义的信号

系统定义的I / O信号是控制器软件为特定目的指定的信号。 除了某些UOP信号，系统定义的I / O不能重新分配。

您可以通过以下端口阵列访问系统定义的I / O信号：

- 机器人数字输入（RDI）和机器人数字输出（RDO）
- 操作面板输入（OPIN）和操作面板输出（OPOUT）
- 示教器输入（TPIN）和示教器输出（TPOUT）

14.2.1 机器人数字输入和输出信号（RDI / RDO）

机器人I / O是控制器和机器人之间的输入和输出信号。 这些信号被发送到位于机器人上的EE（末端执行器）连接器。 机器人输入和输出信号（RDI和RDO）的数量取决于系统中的轴数。 有关配置Robot I / O的更多信息，请参阅相应的应用程序特定的FANUC Robotics设置和操作手册。

RDI [1]到RDI [8]可用于刀具输入。 根据机器人型号，可以使用所有或部分信号。 有关详细信息，请参阅特定于机器人型号的维护手册。

RDO [1]到RDO [8]可用于刀具控制。 根据机器人型号，可以使用所有或部分信号。 有关详细信息，请参阅特定于机器人型号的维护手册。

RDO [1]到RDO [4]是使用OPEN，CLOSE和RELAX手设置的相同信号。 见14.1.4节。

14.2.2 操作面板输入和输出信号（OPIN / OPOUT）

操作面板输入和输出信号是标准操作面板（SOP）和用户操作面板（UOP）的输入和输出信号。

操作面板输入信号分配如下：

- 前16个信号OPIN [0] - OPIN [15]被分配给标准操作面板。
- 接下来的18个信号OPIN [16] - OPIN [33]被分配给用户操作面板（UOP）。 如果您有过程I / O板，则这18个UOP信号将映射到过程I / O板上的前18个输入端口。

操作面板输出信号分配如下：

- 前16个信号OPOUT [0] - OPOUT [15]被分配给标准操作面板。
- 接下来的20个信号OPOUT [16] - OPOUT [35]被分配给用户操作面板（UOP）。 如果您有过程I / O板，则这20个UOP信号将映射到过程I / O板上的前20个输出端口。

标准操作面板输入和输出信号

标准操作面板输入和输出信号由KAREL系统识别为OPIN [0] -OPIN [15]和OPOUT [0] - OPOUT [15]，并通过示教器屏幕显示为SI [0] - SI [15]和SO [0] - SO [15]。 表14-1列出了每个标准操作面板输入信号。 表14-2列出了每个标准操作面板输出信号。

表14-1. 标准操作面板输入信号

OPIN[n]	SI[n]	功能	描述
OPIN[0]	SI[0]	NOT USED	-
OPIN[1]	SI[1]	FAULT RESET	此信号通常为OFF，表示未按下FAULT RESET按钮。
OPIN[2]	SI[2]	REMOTE	此信号通常OFF，表示控制器未设置为远程。
OPIN[3]	SI[3]	HOLD	该信号通常为ON，表示没有按下HOLD按钮。
OPIN[6]	SI[6]	CYCLE START	该信号通常OFF，表示未按下CYCLE START按钮。
OPIN[7] - OPIN[15]	SI[7], SI[10] - SI[15]	NOT USED	-
	SI[8] SI[9]	CE/CR Select b0 CE/CR Select b1	该信号为2位，表示模式选择开关的状态。

表14-2. 标准操作面板输出信号

OPOUT[n]	SOI[n]	功能	描述
OPOUT[0]	SO[0]	REMOTE LED	此信号表示控制器已设置为远程。
OPOUT[1]	SO[1]	CYCLE START	该信号表示已按下CYCLE START按钮或程序正在运行。
OPOUT[2]	SO[2]	HOLD	该信号表示已按下HOLD按钮或存在保持条件。
OPOUT[3]	SO[3]	FAULT LED	该信号表示发生了故障。
OPOUT[4]	SO[4]	BATTERY ALARM	该信号表示CMOS电池电压低。
OPOUT[5]	SO[5]	USER LED#1 (PURGE COMPLETE for P-series robots)	该信号是用户可定义的。
OPOUT[6]	SO[6]	USER LED#2	该信号是用户可定义的。
OPOUT[7]	SO[7]	TEACH PENDANT ENABLED	此信号表示示教器已启用。
OPOUT[8] - OPOUT[15]	SO[8] - SO[15]	NOT USED	-

用户操作面板输入和输出信号

用户操作面板输入和输出信号由 KAREL 系统识别为 OPIN [16] -OPIN [33] 和 OPOUT [16] -OPOUT [35]，以及示教器上的屏幕为 UI [1] -UI [18] 和 UO [1] -UO [20]。 在过程I / O板上，UOP输入信号映射到前18个数字输入信号，UOP输出信号映射到前20个数字输出信号。表14-3列出并描述了每个用户操作面板输入信号。 表14-4列出了每个用户操作员面板输出信号。 图14-2和图14-3说明了UOP信号的时序。

表14-3. 用户操作面板输入信号

OPIN[n]	UI[n]	Process I/O Number	功能	描述
OPIN[16]	UI[1]	1	*IMSTP Always active	* IMSTP是即时停止软件信号。 * IMSTP是一个常开的OFF信号保持ON，当设置为OFF时 • 如果正在运行，则暂停程序。 • 关闭伺服电源。 • 立即停止机器人并应用机器人制动器。 错误代码SRVO-037 *当此信号丢失时，将显示IMSTP输入（组：i）。 此信号始终有效。 **警告** **** IMSTP是软件控制的输入，不能用于安全目的。 使用** **** IMSTP与EMG1，EMG2和EMGCOM使用此信号与硬件控制器紧急停止。 有关EMG1，EMG2和EMGCOM的连接信息，请参阅特定于您的机器人型号的维护手册。**
OPIN[17]	UI[2]	2	*HOLD Always active	*HOLD是外部保持信号。 * HOLD是常开OFF信号，当设为OFF时 • 暂停程序执行。 • 慢动作到受控制的停止和保持。 • 可选的制动器保持在机器人停止后关闭伺服电源。

表14-3. 用户操作面板输入信号（续）

OPIN[n]	UI[n]	Process I/O Number	功能	描述
OPIN[18]	UI[3]	3	*SFSPD Always active	* SFSPD是安全速度输入信号。 该信号通常连接到安全栅栏。* SFSPD是一个常开的OFF信号，当设置为OFF时 • 暂停程序执行。 • 将速度覆盖值降低到系统变量中定义的值。 * SFSPD为OFF时，不能增加该值。 • 显示错误代码消息MF-0004 Fence Open。 • 不允许REMOTE启动条件。 当UOP或SOP的启动输入被禁用时 * SFSPD设置为OFF。

表14-3. 用户操作面板输入信号（续）

OPIN[n]	UI[n]	Process I/O Number	功能	描述
OPIN[19]	UI[4]	4	CSTOPI Always active	CSTOPI是循环停止输入。 当CSTOPI变为TRUE时，系统变量$ CSTOP设置为TRUE。 此外，如果系统变量$ SHELL_CONFIG.$ shell_name不是TRUE或在加电时未初始化，则CSTOPI的功能如下，具体取决于系统变量$ SHELL_CFG.$ USE_ABORT。 如果系统变量$ SHELL_CFG.$ USE_ABORT设置为FALSE，则为CSTOPI输入 • 清除RSR信号发送的要执行的程序队列。 **警告** **当$ SHELL_CFG.USE_ABORT为FALSE时，CSTOPI不会立即停止自动程序执行。** • 当前程序执行完毕后，将停止自动执行。 **如果系统变量$ SHELL_CFG.$ USE_ABORT设置为TRUE，则为CSTOPI输入** • 清除RSR信号发送的要执行的程序队列。 • 立即中止当前正在执行的程序，以便发送由RSR或PNS执行的程序。

表14-3. 用户操作面板输入信号（续）

OPIN[n]	UI[n]	Process I/O Number	功能	描述
OPIN[20]	UI[5]	5	FAULT_RESET Always active	FAULT_RESET是外部故障复位信号。 收到此信号时 • **错误状态被清除。** • **伺服电源打开。** • **暂停的节目将不会恢复。**
OPIN[21]	UI[6]	6	START 机器人处于远程状态时有效 (CMDENBL = ON)	START是远程启动输入。 该信号的工作方式取决于系统变量$ SHELL_CFG.$ CONT_ONLY。 如果系统变量$ SHELL_CFG.$ CONT_ONLY设置为FALSE，则START输入信号将为 • 恢复暂停的程序。 • 如果程序未暂停，则当前所选程序从光标位置开始。 如果系统变量$ SHELL_CFG.$ CONT_ONLY设置为TRUE，则START输入信号将为 • 仅恢复暂停的程序。 PROD_START输入必须从头开始使用。
OPIN[22]	UI[7]	7	HOME 机器人处于远程状态时有效	HOME是原位输出。 当接收到该信号时，机器人移动到定义的原点位置。
OPIN[23]	UI[8]	8	ENBL Always active	ENBL是启用输入。 该信号必须为ON才能具有运动控制能力。 当此信号为OFF时，可以进行机器人动作。 当ENBL为ON且操作面板上的REMOTE开关处于REMOTE位置时，机器人处于远程操作状态。

表14-3. 用户操作面板输入信号（续）

OPIN[n]	UI[n]	Process I/O Number	功能	描述
OPIN[24]-OPIN[31]	UI[9]-UI[16]	9- 126	RSR1/PNS1, RSR2/PNS2, RSR3/PNS3, RSR4/PNS4,RSR5/PNS5, ,RSR6/PNS6, ,RSR7/PNS7, ,RSR8/PNS8 Active when the robot is in a remote condition (CMDENBL = ON)	**RSR1-8**是机器人服务请求输入信号。 当接收到这些信号之一时，相应的RSR程序正在执行，或者当前正在运行的程序存储在队列中以供稍后执行。 RSR信号用于生产操作，并且可以在ACK输出脉冲时接收。见图14-2。 **PNS 1-8**是程序号选择输入信号。 PNS选择要执行的程序，**但不执行程序。 根据系统变量$SHELL_CFG的值，使用START输入或PROD_START输入执行PNS选择的程序。**$CONT_ONLY。通过脉冲SNO信号（选择的数字输出）和SNACK信号输出PNS编号（ 选定的号码确认）。 见图14-3。
OPIN[32]	UI[17]	17	PNSTROBE Active when the robot is in a remote condition (CMDENBL = ON)	PNSTROBE输入是程序编号选择选通输入信号。 见图14-3。
OPIN[33]	UI[18]	18	PROD_START Active when the robot is in a remote condition (CMDENBL = ON)	PROD_START输入与PNS一起使用时，将从PNS线开始执行所选程序。 在没有PNS的情况下使用时，PROD_START从当前光标位置执行所选程序。见图14-3。

表14-4. 用户操作面板输出信号

OPOUT[n]	UO[n]	Process I/O Number	功能	描述
OPOUT[16]	UO[1]	1	CMDENBL	CMDENBL是命令启用输出。 此输出表示机器人处于远程状态。 当REMOTE开关转到ON或接收到ENBL输入时，此信号变为ON。 此输出仅在机器人未处于故障状态时保持打开状态。 请参见图14-2和图14-3。
OPOUT[17]	UO[2]	2	SYSRDY	SYSRDY是系统就绪输出。 此输出表示伺服器已打开。
OPOUT[18]	UO[3]	3	PROGRUN	PROGRUN是程序运行输出。 程序运行时，此输出将打开。 见图14-3。
OPOUT[19]	UO[4]	4	PAUSED	PAUSED是暂停的程序输出。程序暂停时此输出打开。
OPOUT[20]	UO[5]	5	HELD	HELD是保持输出。 按下SOP HOLD按钮或UOP * HOLD输入为OFF时，此输出打开。
OPOUT[21]	UO[6]	6	FAULT	FAULT是错误输出。当程序处于错误状态时，此输出将打开。
OPOUT[22]	UO[7]	7	ATPERCH	不支持。 请参阅相应的特定于应用的FANUC机器人设置和操作手册，“参考位置实用程序”部分。
OPOUT[23]	UO[8]	8	TPENBL	TPENBL是示教器启用输出。当示教器开启时，此输出打开。
OPOUT[24]	UO[9]	9	BATALM	BATALM是电池警报输出。 当CMOS RAM电池电压低于3.6伏时，此输出打开。
OPOUT[25]	UO[10]	10	BUSY	BUSY是处理器忙输出。 当机器人执行程序或处理器忙时，此信号将打开。

表14-4. 用户操作面板输出信号（续）

OPOUT[n]	UO[n]	Process I/O Number	功能	描述
OPOUT[26] OPOUT[33]	UO[11]- UO[18]	11- 18	ACK1/SNO1, ACK2/SNO2, ACK3/SNO3, ACK4/SNO4, ACK5/SNO5, ACK6/SNO6, ACK7/SNO7, ACK8/SNO8	**ACK 1-8**是确认信号输出1到4.当接收到相应的RSR信号时，这些信号接通。 参见图14-2 **SNO 1-8**是信号编号输出。 这些信号携带相应PNS所选节目号的8位表示。 如果程序不能用8位数表示，则信号设置为全零或关闭。 见图14-3。
OPOUT[34]	UO[19]	19	SNACK	SNACK是信号编号确认输出。如果通过PNS输入选择程序，则此输出是脉冲输出。见图14-3。
OPOUT[35]	UO[20]	20	RESERVED	-
OPOUT[36]	UO[21]	21	UNCAL (option)	UNCAL是未经校准的输出。 机器人未校准时，此输出打开。 当控制器丢失来自一个或所有电机的反馈信号时，机器人未校准。 设置$ OPWORK.$ OPT_OUT = 1以使用此信号。
OPOUT[37]	UO[22]	22	UPENBL (option)	UPENBL是用户面板启用输出。 此输出表示机器人处于远程状态。 当遥控开关转到ON或接收到ENBL输入时，此信号变为ON。 即使机器人处于故障状态，此输出也将保持打开状态。 设置$ OPWORK.$ OPT_OUT = 1以使用此信号。
OPOUT[38]	UO[23]	23	LOCKED (option)	-
OPOUT[39]	UO[24]	24	CSTOPO (option)	CSTOPO是循环停止输出。 收到CSTOPI输入时，此输出打开。 设置$ OPWORK.$ OPT_OUT = 1以使用此信号。

图 14–2. RSR时序图

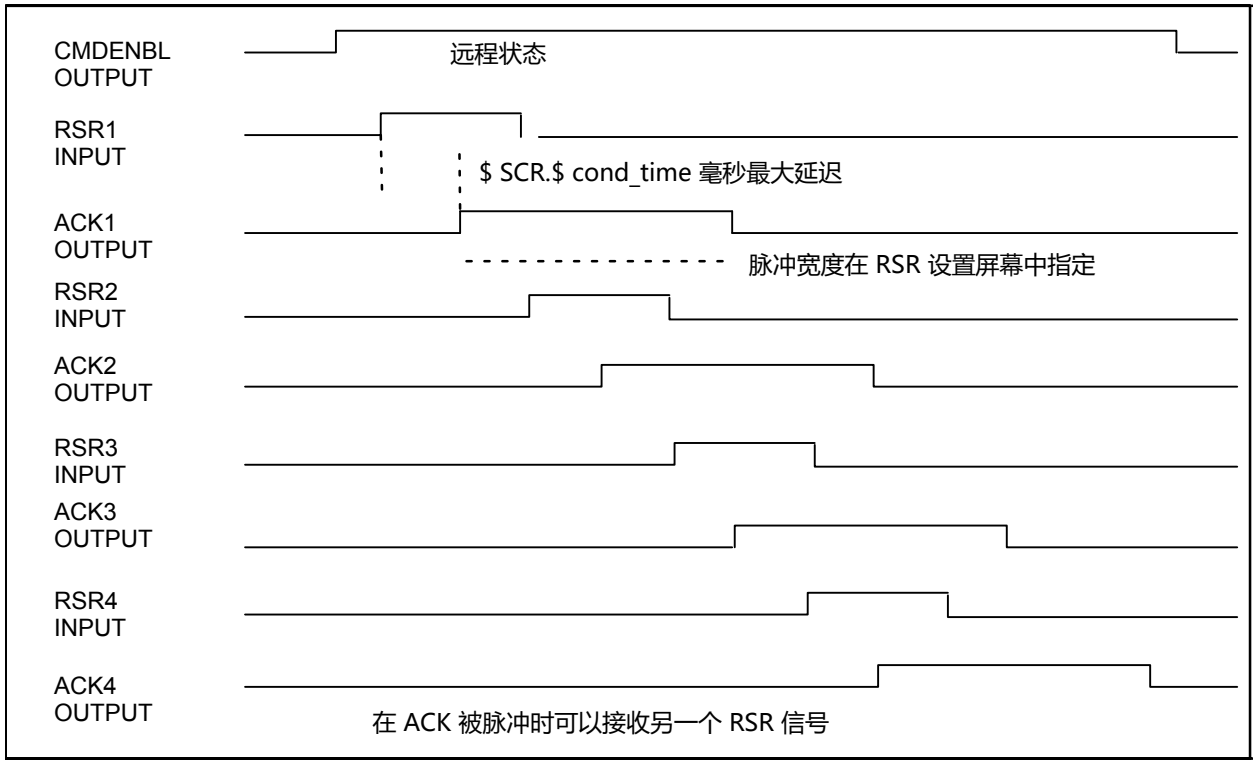
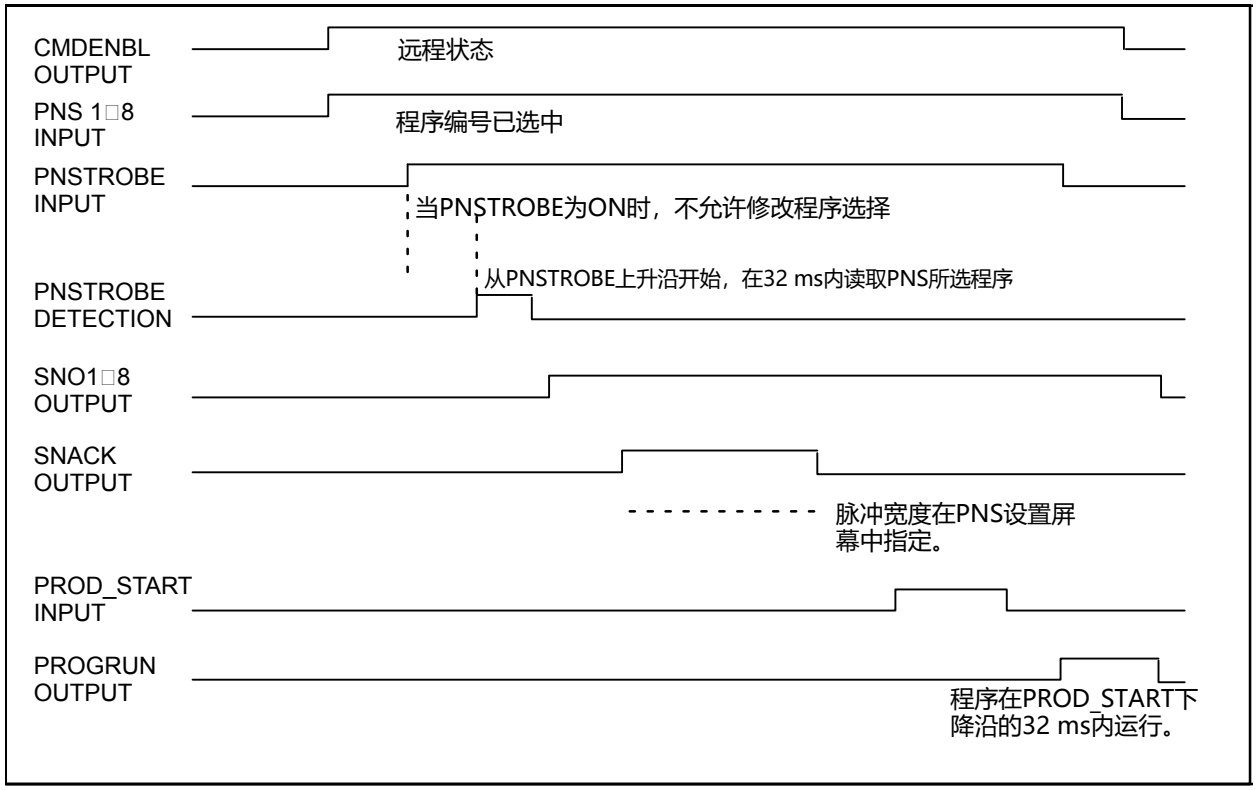

CMDENBL
OUTPUT
远程状态
RSR1
INPUT
$ SCR.$ cond_time 毫秒最大延迟
ACK1
OUTPUT
脉冲宽度在 RSR 设置屏幕中指定
RSR2
INPUT
ACK2
OUTPUT
RSR3
INPUT
ACK3
OUTPUT
RSR4
INPUT
ACK4
OUTPUT
在 ACK 被脉冲时可以接收另一个 RSR 信号

图 14–3. PNS时序图



14.2.3 TP示教器输入和输出信号（TPIN / TPOUT）

示教器输入信号（TPIN）提供对示教器键产生的输入信号的读访问。 可以通过TPIN端口阵列访问示教器输入。 KAREL程序将示教器输入数据视为BOOLEAN数据类型。 该值为ON（激活 - 按下该键）或OFF（非激活 - 未按下该键）。 TPIN信号在KAREL程序中通过名称TPIN [n]访问，其中“n”是信号编号，在内部分配。 有关示教器输入信号分配，请参见表14-5。

表14-5. 示教器输入信号分配

TPIN[n]	示教器按键
EMERGENCY STOP AND DEADMAN	
TPIN[250] TPIN[249] TPIN[247] TPIN[248]	紧急停止 ON/OFF switch Right DEADMAN switch Left DEADMAN switch
Arrow Keys	

表14-5. 示教器输入信号分配（续）

TPIN[n]	示教器按键
TPIN[212]	向上箭头
TPIN[213]	向下箭头
TPIN[208]	右箭头
TPIN[209]	左箭头
TPIN[0]	左移和/或右移
TPIN[204]	向上移动箭头
TPIN[205]	向下移动箭头
TPIN[206]	右移箭头
TPIN[207]	向左移动箭头
键盘键（移位或未移位）	
TPIN[13]	ENTER
TPIN[8]	BACK SPACE
TPIN[48]	0
TPIN[49]	1
TPIN[50]	2
TPIN[51]	3
TPIN[52]	4
TPIN[53]	5
TPIN[54]	6
TPIN[55]	7
TPIN[56]	8
TPIN[57]	9
功能键	
TPIN[128]	PREV
TPIN[129]	F1
TPIN[131]	F2
TPIN[132]	F3
TPIN[133]	F4
TPIN[134]	F5
TPIN[135]	NEXT
TPIN[136]	Shifted PREV
TPIN[137]	Shifted F1
TPIN[138]	Shifted F2
TPIN[139]	Shifted F3
TPIN[140]	Shifted F4
TPIN[141]	Shifted F5
TPIN[142]	Shifted NEXT
Menu Keys	

表14-5. 示教器输入信号分配（续）

TPIN[n]	示教器按键
TPIN[143]	SELECT
TPIN[144]	MENU
TPIN[145]	EDIT
TPIN[146]	DATA
TPIN[147]	FCTN
TPIN[148]	ITEM
TPIN[149]	+%
TPIN[150]	-%
TPIN[151]	HOLD
TPIN[152]	STEP
TPIN[153]	RESET
TPIN[240]	DISP
TPIN[203]	HELP
TPIN[154]	Shifted ITEM
TPIN[155]	Shifted +%
TPIN[156]	Shifted -%
TPIN[157]	Shifted STEP
TPIN[158]	Shifted HOLD
TPIN[159]	Shifted RESET
TPIN[227]	Shifted DISP
TPIN[239]	Shifted HELP
User Function Keys	
TPIN[173]	USER KEY 1
TPIN[174]	USER KEY 2
TPIN[175]	USER KEY 3
TPIN[176]	USER KEY 4
TPIN[177]	USER KEY 5
TPIN[178]	USER KEY 6
TPIN[210]	USER KEY 7
TPIN[179]	Shifted USER KEY 1
TPIN[180]	Shifted USER KEY 2
TPIN[181]	Shifted USER KEY 3
TPIN[182]	Shifted USER KEY 4
TPIN[183]	Shifted USER KEY 5
TPIN[184]	Shifted USER KEY 6
TPIN[211]	Shifted USER KEY 7
Motion Keys	

表14-5. 示教器输入信号分配（续）

TPIN[n]	示教器按键
TPIN[185]	FWD
TPIN[186]	BWD
TPIN[187]	COORD
TPIN[28]	GROUP
TPIN[188]	+X
TPIN[189]	+Y
TPIN[190]	+Z
TPIN[191]	+X rotation
TPIN[192]	+Y rotation
TPIN[193]	+Z rotation
TPIN[12290]	+J7
TPIN[12294]	+J8
TPIN[194]	-X
TPIN[195]	-Y
TPIN[196]	-Z
TPIN[197]	-X rotation
TPIN[198]	-Y rotation
TPIN[199]	-Z rotation
TPIN[12292]	-J7
TPIN[12296]	-J8
TPIN[226]	Shifted FWD
TPIN[207]	Shifted BWD
TPIN[202]	Shifted COORD
Motion Keys Cont'd	
TPIN[214]	Shifted +X
TPIN[215]	Shifted +Y
TPIN[216]	Shifted +Z
TPIN[217]	Shifted +X rotation
TPIN[218]	Shifted +Y rotation
TPIN[219]	Shifted +Z rotation
TPIN[12291]	Shifted +J7
TPIN[12295]	Shifted +J8
TPIN[220]	Shifted -X
TPIN[221]	Shifted -Y
TPIN[222]	Shifted -Z
TPIN[223]	Shifted -X rotation
TPIN[224]	Shifted -Y rotation
TPIN[225]	Shifted -Z rotation
TPIN[12293]	Shifted -J7
TPIN[12297]	Shifted -J8
***i* Keys**	

表14-5. 示教器输入信号分配（续）

TPIN[n]	示教器按键
TPIN[12288]	*i* key
TPIN[12298]	*i* MENU, Top Menu
TPIN[12299]	*i* SELECT, 4D Select Node Map
TPIN[12300]	*i* EDIT, 4D Position Register
TPIN[12301]	*i* DATA, 4D Edit Node Map
TPIN[12302]	*i* FCTN, Related Views
TPIN[12303]	*i* STEP
TPIN[12304]	*i* COORD
TPIN[12305]	*i* GROUP
TPIN[12306]	*i* +%
TPIN[12307]	*i* -%
TPIN[12308]	*i* Up arrow
TPIN[12309]	*i* Down arrow
TPIN[12310]	*i* Right arrow
TPIN[12311]	*i* Left arrow
TPIN[12312]	*i* DISP, Focus window
TPIN[12313]	*i* HELP
TPIN[12338]	*i* USER KEY 7, 4D Display
TPIN[12339]	*i* USER KEY 6
TPIN[12340]	*i* USER KEY 5
TPIN[12341]	*i* ENTER, space
***i* Keys Cont'd**	
TPIN[12314]	*i* +X
TPIN[12315]	*i* +Y
TPIN[12316]	*i* +Z
TPIN[12317]	*i* +X rotation
TPIN[12318]	*i* +Y rotation
TPIN[12319]	*i* +Z rotation
TPIN[12320]	*i* +J7 rotation
TPIN[12321]	*i* +J8 rotation
TPIN[12322]	*i* -X
TPIN[12323]	*i* -Y
TPIN[12324]	*i* -Z
TPIN[12325]	*i* -X rotation
TPIN[12326]	*i* -Y rotation
TPIN[12327]	*i* -Z rotation
TPIN[12328]	*i* -J7 rotation
TPIN[12329]	*i* -J8 rotation

三个示教器输出信号可供使用：

- TPOUT [6] - 控制示教器用户LED＃1

- TPOUT [7] - 控制示教器用户LED # 2
- TPOUT [8] - 控制示教器用户LED # 3

14.3 串行输入/输出

14.3.1 串行输入/输出

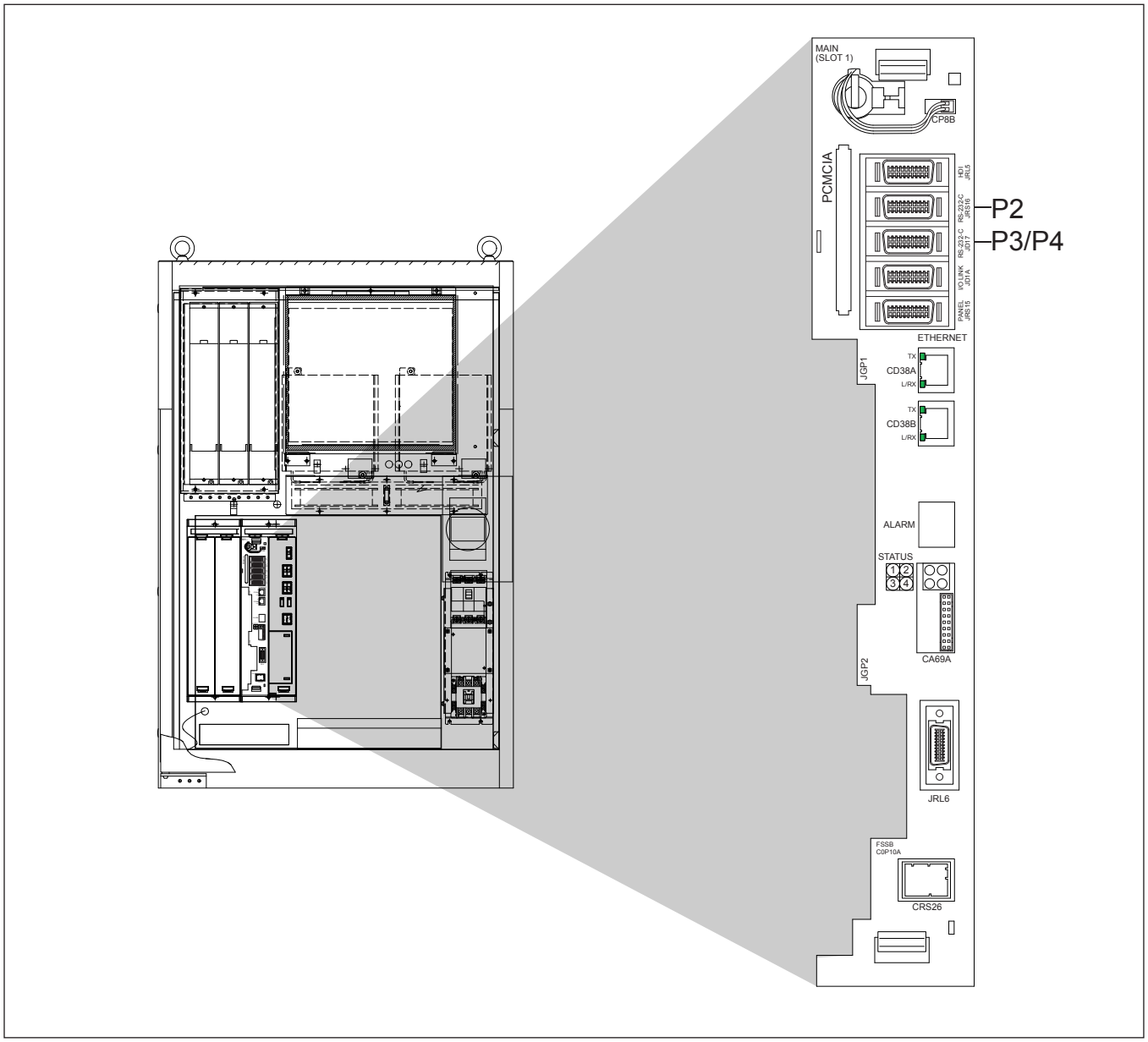
串行I / O系统允许您与连接到KAREL系统的外围串行设备进行通信。 例如，您可以使用串行I / O通过连接到控制器的电缆将消息从其中一个通信端口写入远程终端。

要使用串行I / O，您必须提供串行设备和相应的电缆。 有关电气规格，请参阅特定于您的机器人型号的维护手册。

用于读取和写入串行数据的通信端口在系统软件中定义。 每个软件端口都与您连接通信电缆的控制器上的物理连接器相关联。

图14-4显示了控制器上端口的位置。

图14-4. 控制器上端口的位置



端口

设置端口意味着初始化控制器串行端口以使用特定设备，例如CRT / KB。 初始化端口涉及根据将连接到端口的设备类型为端口设置特定信息。 这是在示教器PORT INIT屏幕上完成的。控制器最多支持四个串行端口。 可以将几种不同类型的设备连接到这些端口。

最多可提供四个端口，P1-P4。 表14-6列出了端口。 如果有，可以设置端口P2到P4，但不能设置示教器端口P1。

表 14–6. 端口 P1 - P4

端口	屏幕上的项目名称	端口的种类	Use	默认设备
P2	JRS16 RS-232–C	RS-232-C	任何设备	Maintenance Console
P3	JD17 RS-232–C 在主CPU卡上	RS-232-C		KCL
P4	主CPU卡上的JD17。 如果$ RS232_NPORT = 4，此端口将显示在示教器上。	RS-422		No use

设备

您可以修改除端口1之外的每个端口的默认通信设置，端口1专用于示教器（TP）。 表14-7列出了可以连接到端口的每种设备的默认设置。

表14-7。 设备的默认通信设置

设备	速度（波特率）	奇偶校验位	停止位	超时值（秒）
Sensor*	4800	Odd	1 bit	0
Host Comm.*	4800	Odd	1 bit	0
KCL/CRT	9600	None	1 bit	0
Maintenance Console	9600	None	1 bit	0
Factory Terminal	9600	None	1 bit	0
TP Demo Device	9600	None	1 bit	0
No Use	9600	None	1 bit	0
Current Position (for use with the Current Position option)	9600	None	1 bit	0
PMC Programmer	9600	None	2 bit	0
Modem/PPP	有关支持的调制解调器的信息，请参阅FANUC Robotics Internet选项设置和操作手册。			
HMI Device	19200	Odd	1 bit	0

*您可以调整这些设置; 但是，如果这样做，它们可能无法按预期运行，因为它们连接到外部设备。

连接硬件并配置相应的端口并连接外部端口后，可以使用KAREL语言OPEN FILE，READ和WRITE语句与外围设备进行通信。

支持更高级别的通信协议作为可选功能。

另请参阅：附录A有关KAREL中可用的语句和内置插件的更多信息有关设置端口的详细信息，请参阅特定于应用程序的FANUC机器人设置和操作手册。

第15章

多任务

目录

第15章 多任务 15–1
15.1 多任务术语 15–2
15.2 解释器分配 15–3
15.3 运动控制 15–3
15.4 任务调度 15–4
15.4.1 优先级调度 15–5
15.4.2 时间分割 15–6
15.5 开始任务 15–6
15.5.1 从用户操作面板（UOP）PNS信号中运行程序 15–7
15.5.2 子任务 15–7
15.6 任务控制和监控 15–7
15.6.1 来自 TPP 程序 15–8
15.6.2 来自 KAREL 程序 15–8
15.6.3 来自 KCL 15–8
15.7 使用信号量和任务同步 15–9
15.8 使用队列进行任务通信 15–14

多任务处理允许多个程序在时间共享的基础上在控制器上运行，以便多个程序同时运行。 当您执行几个通常可以彼此独立操作的操作序列时，多任务特别有用，即使它们之间存在一些交互作用。 例如：

- 监视输入信号和设置输出信号的过程。
- 生成日志信息并将其传输到单元控制器并从单元控制器接收命令或其他输入数据的过程。

重要的是要意识到，尽管多个任务似乎同时运行，但它们共享同一个处理器的使用，因此在任何时候只有一个任务正在执行。 除了可中断语句之外，一旦语句执行开始后，必须先完成另一个任务的语句才能执行。 以下声明是可中断的：

- READ
- DELAY
- WAIT
- WAIT FOR

有关系统如何决定首先执行哪个任务的信息，请参见第15.4节“任务计划”。

15.1 多任务术语

本章使用以下术语和表达式。

- **任务或用户任务**

  任务或用户任务是正在运行或暂停的用户程序。 任务由“解释器”执行。 当程序启动时创建一个任务，当分配给它的解释器被删除时，任务被分配给另一个任务。

- **翻译员**

  解释器是执行用户程序的系统组件。 在冷启动或受控启动时，会创建（$ MAXNUMTASKS + 2）个解释器。 这些解释器能够同时执行任务。

- **任务名称**

  任务名称是创建任务时指定的程序名称。 创建任务时，指定要作为任务名称执行的程序的名称。

注意：任务创建后，任务名称不会更改。 因此，当外部例程正在执行时，当前正在执行的程序名称与任务名称不同。 当您向任务发送任何请求时，请使用任务名称，而不是当前的程序名称。

- **运动控制**
  运动控制由位掩码定义，该位掩码指定任务控制的运动组。 一次只有一个任务可以控制一个运动组。 但是，不同的任务可以同时控制不同的运动组。 有关更多信息，请参见第15.3节“运动控制”。

15.2 解释器分配

当任务开始时，它被分配给解释器。 它分配给的解释器（1，2，3，...）确定其任务编号。 任务编号用于暂停程序，中止程序和继续程序条件处理程序操作。 任务的任务编号可以使用内置的 GET_TSK_INFO 来确定。

以下是将任务分配给解释器的规则：

- 如果任务已经分配给解释器，则它使用相同的解释器。
- 将任务分配给当前没有分配任务的第一个可用解释器。
- 如果所有解释器都分配给任务，则将为具有中止任务的第一个解释器分配一个新任务。
- 如果以上都不能完成，则任务无法启动。

15.3 运动控制

多任务处理中的一个重要限制是控制各种运动组。 只有一个任务可以控制或使用一组轴。

在下列情况下，任务需要对组进行控制：

- 任务开始时，如果未使用控制器指令%NOLOCKGROUP。 如果%LOCKGROUP指令未使用，该任务默认情况下需要控制所有组。 如果使用%LOCKGROUP，则需要控制指定的组。
  对于示教器程序，程序启动时需要运动控制，除非选择屏幕上的详细信息页面用于将组掩码设置为[*，*，*，*，*]。
- 当任务执行内置的LOCK_GROUP时，它需要由组掩码指定的组。

- 当任务调用ROUTINE或示教器程序时，它需要控制这些组。 ROUTINE或TPP +程序所需的组是通过控制器指令或示教器详细设置指定或暗示的组。

假设：一个任务将被控制所需的组，

- 没有其他任务可以控制组。
- 示教器未启用，但可以在示教器上使用shift-FWD启动运动控制时，或者如果它具有％TPMOTION指令，则可以将运动控制赋予程序。
- 没有紧急停止活动。
- 舵机已准备就绪。
- UOP信号IMSTP未被声明。

如果任务无法控制所需的组，则会暂停该任务。 任务获得组的控制权
后，它会一直保留到下列之一：

- 任务结束（中止）。
- 该任务执行内置的UNLOCK_GROUP。
- 该任务在内置的RUN_TASK中传递组的控制权。
- 常规程序或示教程序返回，并且组是由程序或教导程序需要的，但不需要调用程序。

15.4 任务调度

当前正在运行（未中止或暂停）的任务将执行语句，直到执行以下操作之一：

- 保持条件发生。
- 一个更高优先级的程序已准备好运行。
- 任务时间片到期。
- 程序中止或暂停。

以下是保持条件的示例：

- 等待读取操作完成。
- 等待动作完成。
- 等待WAIT，WAIT或DELAY语句完成。

当任务处于运行状态且没有保持条件时，即可运行任务。 一次只能执行一项任务。 有两个规则可以确定在多个任务准备好运行时将执行哪个任务：

- 优先级 - 如果两个或更多不同优先级的任务已准备好运行，则首先执行具有较高优先级的任务。 有关更多信息，请参阅第15.4.1节“优先级调度”。
- 时间切片 - 如果两个具有相同优先级的任务准备运行，则任务的执行是时间切片的。 有关更多信息，请参阅第15.4.2节“时间片”。

15.4.1 优先级调度

如果两个或多个具有不同优先级的任务已准备好运行，则具有最高优先级的任务将首先运行。任务的优先级由其优先级编号决定。 优先级编号必须在-8到143之间。优先级编号越低，任务优先级越高。
例如：如果 TASK_A 的 优先级数为50，并且TASK_B的优先级数为 60，并且都准备好运行，那么 TASK_A 将首先执行，只要它已准备好运行即可。
任务优先级可以通过以下方式之一设置：

- 默认情况下，每个用户任务分配的优先级为50。
- KAREL程序可能包含 %PRIORITY 译码器指令。
- 内置的 SET_TSK_ATTR 可用于设置任何任务的当前优先级。

除了影响其他用户任务之外，任务优先级还会影响执行它的解释器的优先级，这与其他系统功能相比的优先级。 如果用户任务具有比系统功能更高的优先级（较低的优先级编号），那么只要用户任务准备就绪，系统功能就不会被执行。 用户任务优先级的范围在高优先级端被限制。 这样做是为了使用户程序不会干扰运动插补。 运动插值指的是引起动作完成所需的更新。
下表列出了一些其他系统功能的优先级。

表 15–1. 系统功能优先级表

优先	系统功能	延迟函数的影响
-8	Maximum priority	新动作延迟。
-1	Motion Planner	新动作延迟。
4	TP Jog	从示教器点动延迟。
54	Error Logger	更新系统错误日志已延迟。
73	KCL	执行KCL命令延迟。

表15-1. 系统功能优先级表（续）

优先	系统功能	延迟函数的影响
82	CRT manager	CRT软键的处理延迟。
88	TP manager	一般示教器活动延迟。
143	Lowest priority	不延迟上述任何一项。

15.4.2 时间分割

如果两个或多个具有相同优先级的任务准备运行，则它们将通过时间分片或交替使用系统来共享系统资源。
时间片允许执行相同优先级的其他任务，但不能执行较低优先级的任务。
任务的默认时间片是256毫秒。 其他值可以使用%TIMESLICE指令或内置的SET_TSK_ATTR来设置。

15.5 开始任务

有几种方法可以开始一项任务。

- KCL RUN命令。 请参阅附录C“KCL命令字母描述”。
- 操作面板启动键。 请参阅适用于特定应用的FANUC Robotics安装和操作手册。
- 用户操作面板启动信号。 请参阅适用于特定应用的FANUC Robotics安装和操作手册。
- 用户操作面板PNS信号。 有关更多信息，请参见第15.5.1节“从用户操作面板（UOP）PNS信号中运行程序”。
- 示教器移位 - FWD键。 有关详细信息，请参阅适用于特定应用的FANUC Robotics安装和操作手册，“测试程序和运行生产”章节。
- 示教器程序执行RUN指令。 有关更多信息，请参阅第15.5.2节“子任务”。
- KAREL程序执行内置的RUN_TASK。 有关更多信息，请参阅第15.5.2节“子任务”。

在任何情况下，如果需要不可用的运动控制，则该任务不会开始运行。

15.5.1 从用户操作面板（UOP）PNS信号中运行程序

一个程序被执行：

- 如果UOP PNS信号的二进制值非零并且UOP PROGSTART信号有效
- 如果当前有一个名为“PNSnnnn”的程序，其中nnnn是PNS信号的十进制值加上$ SHELLCFG.$ jobbase的当前值。

程序不执行：

- 如果PNS信号的二进制值为零。

只要没有运动组重叠，就可以用这种方式启动多个程序。

如果从PNS确定的任务名称处于暂停状态，则PROGSTART信号被解释为CONTINUE信号。

如果 $ SHELLCFG.$ contonly 为TRUE，则这是PNS / PROGSTART信号的唯一功能。

如果 $ SHELLCFG.$ useabort 为TRUE，则可以使用PNS信号来中止正在运行的任务。 要中止的任务的名称与PROGSTART信号使用的名称相同。 在这种情况下，中止由UOP CSTOPI信号触发

15.5.2 子任务

正在运行的任务可以创建新的任务。 这个新任务被称为子任务。 请求创建子任务的任务称为父任务。 在示教器程序中，通过执行RUN指令创建新任务。 在KAREL程序中，可以使用内置的RUN_TASK创建新任务。

父母和子女任务可能不需要相同的运动组。 但是，在RUN_TASK的情况下，可以释放对子任务使用的运动组的控制。

一旦创建了子任务，它将独立于其父任务运行，但有以下例外情况：

- 如果父任务继续并且其子任务已暂停，则子任务也会继续。
- 如果父任务处于STEP模式，则子任务也将进入STEP模式。

如果您希望子任务完全独立于父级，则KAREL程序可以使用 KCL 或 KCL_NOWAIT 内置函数发起另一个任务，以发出KCL> RUN命令。

15.6 任务控制和监控

有三种环境可以从中控制和监视任务：

1. 示教器程序（TPP） - 第15.6.1节
2. KAREL 程序 - 第15.6.2节
3. KCL 命令 - 第15.6.3节

15.6.1 来自 TPP 程序

TPP指令 RESUME_PROG 可用于继续执行 PAUSED 任务。

15.6.2 来自 KAREL 程序

有很多内置控件用于控制和监视其他任务。 请参阅附录A中对这些内置插件的描述。

- RUN_TASK 执行任务。
- CONT_TASK 恢复执行 PAUSED 任务。
- PAUSE_TASK 暂停任务。
- ABORT_TASK 中止任务。
- CONTINUE 条件处理程序操作会导致执行任务。
- ABORT 条件处理程序操作会导致任务中止。
- PAUSE 条件处理程序操作会导致任务暂停。
- GET_TSK_INFO 确定指定的任务是正在运行，暂停还是中止。 还确定正在执行什么程序和行号，以及该任务等待什么（如果有的话）。

15.6.3 来自 KCL

以下KCL命令可用于控制和监视任务的状态。 有关更多信息，请参阅附录C “KCL命令字母描述”。

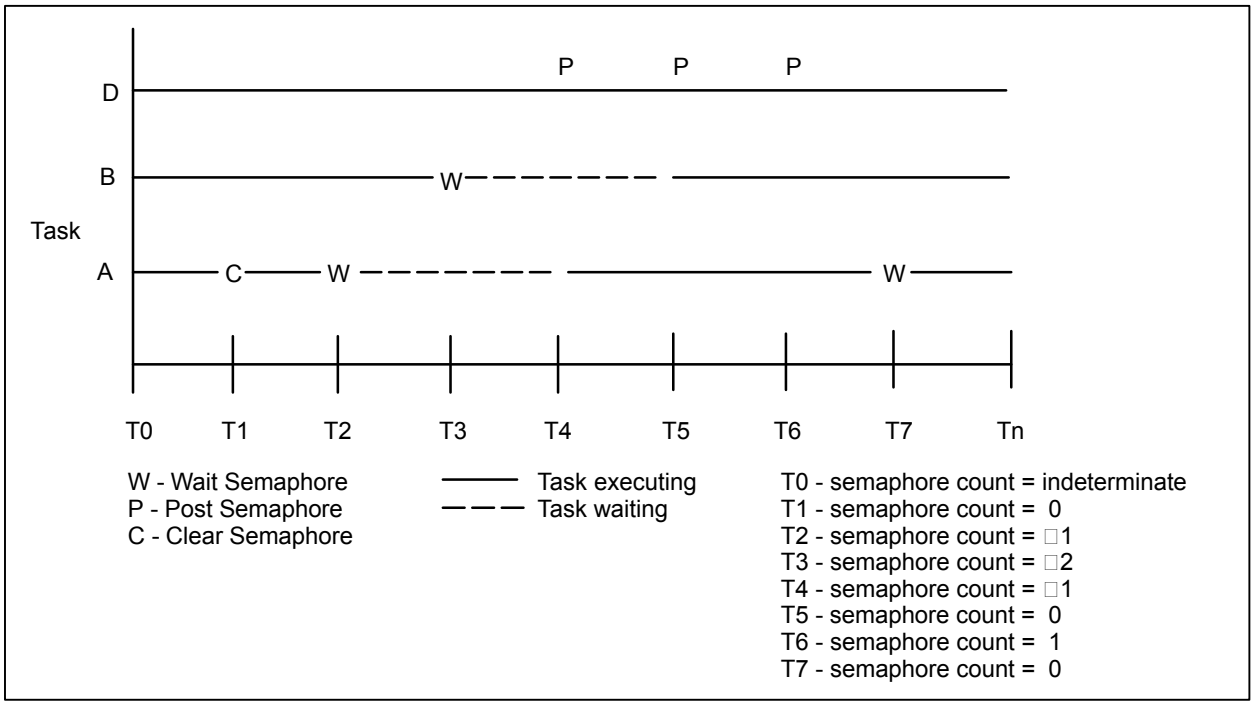
- RUN <task_name> 启动或继续任务。
- CONT <task_name> 继续一项任务。
- PAUSE <task_name> 暂停任务。
- ABORT <task_name> 中止任务。
- SHOW TASK <task_name> 显示任务的状态。
- SHOW TASKS 显示所有任务的状态。

15.7 使用信号量和任务同步

良好的设计决定了单独的任务能够独立运作。 但是，他们也应该能够互动。
KAREL控制器支持计数信号量。 信号量允许执行以下操作：

- 清除信号量（KAREL：CLEAR_SEMA 内置）：将信号计数设置为零。
  所有信号在冷启动时被清除。在使用之前清除信号量是一种很好的做法。在几个任务开始共享信号量之前，这些任务中只有一个应该清除信号量。
- 发布到信号量（KAREL：POST_SEMA 内置）：为信号计数添加一个。
  如果信号计数为零或更大，则在发布后信号时，信号计数将加1。等待信号量的下一个任务将减少信号计数并继续执行。参考图15-1。
  如果信号计数为负，则在发出信号后，信号计数将加1。等待信号最长的任务将继续执行。参考图15-1。
- 读取信号量（KAREL：SEMA_COUNT 内置）：返回当前的信号计数。
- 等待信号量（KAREL：PEND_SEMA 内置，SIGNAL SEMAPHORE 动作）：
  如果在发出等待信号时信号计数大于零，信号计数将递减，任务将继续执行。参考图15-1。
- 如果信号计数小于或等于零（负值），等待信号量将减少信号计数，任务将等待由信号量释放。任务按先进先出的原则发布。例如，如果任务A在信号量1上等待，则任务B在信号量1上等待。当任务D发布信号量1时，只有任务A被释放。参考图15-1。

图15-1. 使用信号量进行任务同步



示例：信号量可用于实现充当请求服务器的任务。 在以下示例中，主任务等待服务器完成其操作。 信号量[4]用于控制对 rqst_param 或R [5]的访问。 信号量[5]用于向服务器任务发出服务请求的信号; 信号量[6]被服务器用来表示操作完成。

主要任务包含以下KAREL：

主任务

```
--KAREL
CLEAR_SEMA(4)
CLEAR_SEMA(5)
CLEAR_SEMA(6)
RUN TASK('server',0,TRUE,TRUE,1,STATUS)
PEND_SEMA(4,max_time,time_out)
rqst_param=10
POST_SEMA(5)
PEND_SEMA(6,max_time,time_out)
```

服务器任务将包含以下KAREL代码：

服务任务

```
--KAREL
POST_SEMA (4)
WHILE TRUE DO
  PEND_SEMA(5,max_time,time_out)
  IF rqst_param=10 THEN
     do_something
  ENDIF
  POST_SEMA(4)
  POST_SEMA(6)
ENDWHILE
```

示例：信号量和任务同步程序示例中的程序示例 - 主要通过信号量和任务同步程序示例 - TASK B显示信号量和任务如何一起用于同步。 MAIN_TASK.KL用于初始化信号量（MOTION_CTRL），然后运行TASK_A.KL和TASK_B.KL。 然后MAIN_TASK.KL在完成之前等待TASK_A和TASK_B中止。 TASK_A等待直到您按F1然后将机器人移动到HOME位置。 TASK_B等待直到您按F2然后沿着路径移动机器人。

信号量和任务同步程序示例 - 主任务

```
PROGRAM main_task
%nolockgroup
VAR
  motion_ctrl: INTEGER
  tsk_a_done : BOOLEAN
  tsk_b_done : BOOLEAN
  tmr        : INTEGER
  status     : INTEGER
-----------------------------------------------
--                                           --
--  INIT_LOCK:                               --
--                                           --
--                                           --
--                                           --
--                                           --
--                                           --
--                                           --
-----------------------------------------------
ROUTINE init_lock
BEGIN
 CLEAR_SEMA (motion_ctrl)  -- 在使用之前确保信号量为零。
 POST_SEMA  (motion_ctrl)  -- 使 motion_ctrl 立即可用
END init_lock
-----------------------------------------------
```

在使用信号量之前，初始化信号量以确保其计数为零。然后发布这个信号量，它将允许第一个pend到信号量继续执行。

```
--                                                    --
--  IS_TSK_DONE : 找出指定的任务是否正在运行。          --
--                如果任务中止，则返回TRUE，否则        --
--                  返回FALSE。                       --
--
------------------------------------------------
ROUTINE is_tsk_done (task_name:STRING): BOOLEAN
VAR
 status  : INTEGER    -- GET_TSK_INFO 的操作状态
 task_no : INTEGER    -- 接收 task_name 的当前任务编号
 attr_out: INTEGER    -- 接收 TSK_STATUS 输出
 dummy   : STRING[2]  -- 不接收任何信息
BEGIN
  GET_TSK_INFO (task_name, task_no, TSK_STATUS, attr_out, dummy, status)
  IF (attr_out = PG_ABORTED) THEN
    RETURN (TRUE)  -- 如果任务中止，则返回 TRUE
  ENDIF
  RETURN(FALSE)    -- 否则任务不会中止并返回
   FALSE
END is_tsk_done
BEGIN
  motion_ctrl = 1  -- 信号量允许运动控制
  init_lock   -- 确保只执行一次
  FORCE_SPMENU ( tp_panel, spi_tpuser, 1)  -- 强制示教器
                                           -- 用户屏幕可见
   RUN_TASK('task_a', 1, FALSE, FALSE, 1, status)  -- 运行 task_a
   RUN_TASK('task_b', 1, FALSE, FALSE, 1, status)  -- 运行 task_b
   REPEAT
     tsk_a_done = is_tsk_done ('task_a')
     tsk_b_done = is_tsk_done ('task_b')
     delay (100)
   UNTIL (tsk_a_done and tsk_b_done)   -- 重复，直到task_a
    END main_task    -- 和task_b中止
```

信号量和任务同步程序示例 - 任务A

```
PROGRAM task_a
%nolockgroup
VAR
  motion_ctrl FROM main_task: INTEGER
  home_pos                  : XYZWPR
  status                    : INTEGER
------------------------------------------------
--                                                --
--  RUN_HOME    : 锁定机器人运动控制。 这项        --
--                                                --
```

```
--                任务是移动机器人，必须有控制权。   --
--                                                  --
--                                                  --
------------------------------------------------
ROUTINE run_home
VAR
  time_out: BOOLEAN
BEGIN
   PEND_SEMA(motion_ctrl,-1,time_out)-- 从其他任务中锁定motion_ctrl
                                    --让其他任务远离移动机器人 LOCK_GROUP
   (1, status)
   SET_POS_REG(1, home_pos, status)
   do_move   -- 调用TP程序移动到原位
   UNLOCK_GROUP (1, status)
   POST_SEMA(motion_ctrl)    -- 解锁motion_ctrl
                             -- 允许其他任务移动机器人
END  run_home
BEGIN
   set_cursor (tpfunc, 1, 4, status)
   write tpfunc ('HOME',CR)
   wait for TPIN[129]+     -- 等待F1被按下
   run_home
END task_a
```

信号量和任务同步程序示例 - 任务B

```
PROGRAM task_b
%nolockgroup
VAR
  motion_ctrl FROM main_task : INTEGER
  work_path                  : PATH
  move_pos                   : XYZWPR
  status                     : INTEGER
------------------------------------------------
--                                              --
--  do_work    :                                --
--                将机器人锁定在其他任务之外并继  --
--                续工作。此任务正在进行运动，必  --
--                须锁定运动控制，以便其他任务不  --
--                会尝试同时进行运动。            --
--                                              --
------------------------------------------------
ROUTINE do_work
VAR
  time_out: BOOLEAN
```

```
BEGIN
   PEND_SEMA (motion_ctrl,-1,time_out) -- 锁定来自其他的motion_ctrl
                                       -- 任务保持其他任务
     --  moving robot
   LOCK_GROUP (1, status)
   FOR i=1 to PATH_LEN(work_path) DO
    move_pos = work_path[i]
    SET_POS_REG(1, move_pos, status)
    do_move    -- 调用TP程序移动到路径节点
   ENDFOR
   UNLOCK_GROUP (1, status)
   POST_SEMA(motion_ctrl)     -- 解锁motion_ctrl允许
                                          -- 移动机器人的其他任务
END  do_work
BEGIN
   set_cursor(tpfunc, 1, 10, status)
   write tpfunc('WORK',CR)
   wait for TPIN[131]+      -- 等到按下F2
   do_work
END task_b
```

15.8 使用队列进行任务通信

仅在KAREL中支持队列。 队列是整数的先进/先出列表。 它们用于按顺序将信息传递给另一个任务。 队列由QUEUE_TYPE类型的用户变量和ARRAY OF INTEGER组成。 队列中的最大条目数由阵列的大小决定。

队列支持以下操作：

- INIT_QUEUE初始化队列并将其设置为空。
- APPEND_QUEUE将一个整数添加到队列中的条目列表中。
- GET_QUEUE：从队列中读取最旧的（顶部）条目并将其删除。

这些以及与队列相关的其他内置函数（DELETE_QUEUE，INSERT_QUEUE，COPY_QUEUE）在附录A中描述。

QUEUE_TYPE数据类型具有一个用户可访问元素n_entries。 这是已添加到队列但未读出的条目数。 与队列一起使用的整数数组由队列内置函数使用，不应由KAREL程序引用。

示例：以下示例说明了一个功能更强大的请求服务器，其中多个任务正在发布请求，请求者不等待请求完成。

请求程序将包含以下代码：

请求程序

```
--声明
 VAR
  rqst_queue FROM server: QUEUE_TYPE
  rqst_data FROM server: ARRAY[100] OF INTEGER
  status:  INTEGER
  seq_no:  INTEGER
  -- 发送到队列 --
APPEND_QUEUE (req_code, rqst_queue, rqst_data, seq_no, status)
```

服务程序任务将包含以下代码：

服务程序

```
PROGRAM server
VAR
  rqst_queue: QUEUE_TYPE
  rqst_data : ARRAY[100] OF INTEGER
  status    : INTEGER
  seq_no    : INTEGER
  rqst_code : INTEGER
BEGIN
 INIT_QUEUE(rqst_queue)   --初始化
  WHILE TRUE DO             --服务循环
      WAIT FOR rqst_code.n_entries > 0
      GET_QUEUE (rqst_queue, rqst_data, rqst_code, seq_no, status)
      SELECT rqst_code OF
 CASE (1): do_something
      ENDSELECT
  ENDWHILE
END server
```

附录 A

KAREL 语言字母描述

目录

附录 A KAREL 语言字母描述 A–1
A.1 比释 A–9
A.2 - A - KAREL 赤谷擦释 A–18
A.2.1 ABORT Action A–18
A.2.2 ABORT Condition A–18
A.2.3 ABORT Statement A–19
A.2.4 ABORT_TASK Built-In Procedure A–19
A.2.5 ABS Built-In Function A–20
A.2.6 ACOS Built-In Function A–20
A.2.7 ACT_SCREEN Built-In Procedure A–21
A.2.8 ACT_TBL Built-In Procedure A–22
A.2.9 ADD_BYNAMEPC Built-In Procedure A–24
A.2.10 ADD_DICT Built-In Procedure A–25
A.2.11 ADD_INTPC Built-In Procedure A–26
A.2.12 ADD_REALPC Built-In Procedure A–27
A.2.13 ADD_STRINGPC Built-In Procedure A–29
A.2.14 %ALPHABETIZE Translator Directive A–30
A.2.15 APPEND_NODE Built-In Procedure A–30
A.2.16 APPEND_QUEUE Built-In Procedure A–31
A.2.17 APPROACH Built-In Function A–32
A.2.18 ARRAY Data Type A–33
A.2.19 ARRAY_LEN Built-In Function A–34
A.2.20 ASIN Built-In Function A–34
A.2.21 Assignment Action A–35
A.2.22 Assignment Statement A–36
A.2.23 ATAN2 Built-In Function A–38
A.2.24 ATTACH Statement A–39
A.2.25 ATT_WINDOW_D Built-In Procedure A–39

A.2.26 ATT_WINDOW_S Built-In Procedure A–40
A.2.27 AVL_POS_NUM Built-In Procedure A–41
A.3 - B - KAREL LANGUAGE DESCRIPTION A–42
A.3.1 BOOLEAN Data Type A–42
A.3.2 BYNAME Built-In Function A–43
A.3.3 BYTE Data Type A–44
A.3.4 BYTES_AHEAD Built-In Procedure A–45
A.3.5 BYTES_LEFT Built-In Function A–46
A.4 - C - KAREL LANGUAGE DESCRIPTION A–47
A.4.1 CALL_PROG Built-In Procedure A–47
A.4.2 CALL_PROGLIN Built-In Procedure A–48
A.4.3 CANCEL Action A–49
A.4.4 CANCEL Statement A–49
A.4.5 CANCEL FILE Statement A–50
A.4.6 CHECK_DICT Built-In Procedure A–51
A.4.7 CHECK_EPOS Built-In Procedure A–52
A.4.8 CHECK_NAME Built-In Procedure A–53
A.4.9 CHR Built-In Function A–53
A.4.10 CLEAR Built-In Procedure A–54
A.4.11 CLEAR_SEMA Built-In Procedure A–55
A.4.12 CLOSE FILE Statement A–55
A.4.13 CLOSE HAND Statement A–56
A.4.14 CLOSE_TPE Built-In Procedure A–56
A.4.15 CLR_IO_STAT Built-In Procedure A–57
A.4.16 CLR_PORT_SIM Built-In Procedure A–57
A.4.17 CLR_POS_REG Built-In Procedure A–58
A.4.18 %CMOSVARS Translator Directive A–59
A.4.19 %CMOS2SHADOW Translator Directive A–59
A.4.20 CNC_DYN_DISB Built-In Procedure A–59
A.4.21 CNC_DYN_DISE Built-In Procedure A–60
A.4.22 CNC_DYN_DISI Built-In Procedure A–61
A.4.23 CNC_DYN_DISP Built-In Procedure A–62
A.4.24 CNC_DYN_DISR Built-In Procedure A–62
A.4.25 CNC_DYN_DISS Built-In Procedure A–63
A.4.26 CNCL_STP_MTN Built-In Procedure A–64
A.4.27 CNV_CNF_STRG Built-In Procedure A–64
A.4.28 CNV_CONF_STR Built-In Procedure A–65
A.4.29 CNV_INT_STR Built-In Procedure A–66
A.4.30 CNV_JPOS_REL Built-In Procedure A–67
A.4.31 CNV_REAL_STR Built-In Procedure A–67
A.4.32 CNV_REL_JPOS Built-In Procedure A–68
A.4.33 CNV_STR_CONF Built-In Procedure A–69
A.4.34 CNV_STR_INT Built-In Procedure A–70
A.4.35 CNV_STR_REAL Built-In Procedure A–71
A.4.36 CNV_STR_TIME Built-In Procedure A–71
A.4.37 CNV_TIME_STR Built-In Procedure A–72
A.4.38 %COMMENT Translator Directive A–73
A.4.39 COMPARE_FILE Built-in Procedure A–73
A.4.40 CONDITION...ENDCONDITION Statement A–75
A.4.41 CONFIG Data Type A–77
A.4.42 CONNECT TIMER Statement A–78
A.4.43 CONTINUE Action A–79
A.4.44 CONTINUE Condition A–79
A.4.45 CONT_TASK Built-In Procedure A–80
A.4.46 COPY_FILE Built-In Procedure A–81

A.4.47 COPY_PATH Built-In Procedure A–82
A.4.48 COPY_QUEUE Built-In Procedure A–83
A.4.49 COPY_TPE Built-In Procedure A–85
A.4.50 COS Built-In Function A–86
A.4.51 CR Input/Output Item A–86
A.4.52 CREATE_TPE Built-In Procedure A–87
A.4.53 CREATE_VAR Built-In Procedure A–88
A.4.54 %CRTDEVICE A–90
A.4.55 CURJPOS Built-In Function A–91
A.4.56 CURPOS Built-In Function A–92
A.4.57 CURR_PROG Built-In Function A–93

A.5 - D - KAREL LANGUAGE DESCRIPTION A–93
A.5.1 DAQ_CHECKP Built-In Procedure A–93
A.5.2 DAQ_REGPIPE Built-In Procedure A–94
A.5.3 DAQ_START Built-In Procedure A–96
A.5.4 DAQ_STOP Built-In Procedure A–98
A.5.5 DAQ_UNREG Built-In Procedure A–99
A.5.6 DAQ_WRITE Built-In Procedure A–100
A.5.7 %DEFGROUP Translator Directive A–102
A.5.8 DEF_SCREEN Built-In Procedure A–102
A.5.9 DEF_WINDOW Built-In Procedure A–103
A.5.10 %DELAY Translator Directive A–104
A.5.11 DELAY Statement A–105
A.5.12 DELETE_FILE Built-In Procedure A–106
A.5.13 DELETE_NODE Built-In Procedure A–107
A.5.14 DELETE_QUEUE Built-In Procedure A–107
A.5.15 DEL_INST_TPE Built-In Procedure A–108
A.5.16 DET_WINDOW Built-In Procedure A–109
A.5.17 DISABLE CONDITION Action A–109
A.5.18 DISABLE CONDITION Statement A–110
A.5.19 DISCONNECT TIMER Statement A–111
A.5.20 DISCTRL_ALPH Built_In Procedure A–112
A.5.21 DISCTRL_FORM Built_In Procedure A–114
A.5.22 DISCTRL_LIST Built-In Procedure A–116
A.5.23 DISCTRL_PLMN Built-In Procedure A–117
A.5.24 DISCTRL_SBMN Built-In Procedure A–119
A.5.25 DISCTRL_TBL Built-In Procedure A–122
A.5.26 DISMOUNT_DEV Built-In Procedure A–125
A.5.27 DISP_DAT_T Data Type A–125
A.5.28 DOSFILE_INF Built-In Procedure A–127

A.6 - E - KAREL LANGUAGE DESCRIPTION A–128
A.6.1 ENABLE CONDITION Action A–128
A.6.2 ENABLE CONDITION Statement A–129
A.6.3 %ENVIRONMENT Translator Directive A–129
A.6.4 ERR_DATA Built-In Procedure A–131
A.6.5 ERROR Condition A–132
A.6.6 EVAL Clause A–133
A.6.7 EVENT Condition A–134
A.6.8 EXP Built-In Function A–134

A.7 - F - KAREL LANGUAGE DESCRIPTION A–135
A.7.1 FILE Data Type A–135
A.7.2 FILE_LIST Built-In Procedure A–135
A.7.3 FOR...ENDFOR Statement A–137
A.7.4 FORCE_SPMENU Built-In Procedure A–138

A.7.5 FORMAT_DEV Built-In Procedure A–141
A.7.6 FRAME Built-In Function A–142
A.7.7 FROM Clause A–144
A.8 - G - KAREL LANGUAGE DESCRIPTION A–145
A.8.1 GET_ATTR_PRG Built-In Procedure A–145
A.8.2 GET_FILE_POS Built-In Function A–147
A.8.3 GET_JPOS_REG Built-In Function A–148
A.8.4 GET_JPOS_TPE Built-In Function A–148
A.8.5 GET_PORT_ASG Built-in Procedure A–149
A.8.6 GET_PORT_ATR Built-In Function A–151
A.8.7 GET_PORT_CMT Built-In Procedure A–153
A.8.8 GET_PORT_MOD Built-In Procedure A–154
A.8.9 GET_PORT_SIM Built-In Procedure A–155
A.8.10 GET_PORT_VAL Built-In Procedure A–156
A.8.11 GET_POS_FRM Built-In Procedure A–157
A.8.12 GET_POS_REG Built-In Function A–158
A.8.13 GET_POS_TPE Built-In Function A–158
A.8.14 GET_POS_TYP Built-In Procedure A–159
A.8.15 GET_PREG_CMT Built-In-Procedure A–160
A.8.16 GET_QUEUE Built-In Procedure A–161
A.8.17 GET_REG Built-In Procedure A–162
A.8.18 GET_REG_CMT A–163
A.8.19 GET_SREG_CMT Builtin Procedure A–163
A.8.20 GET_STR_REG Built-In Procedure A–164
A.8.21 GET_TIME Built-In Procedure A–164
A.8.22 GET_TPE_CMT Built-in Procedure A–165
A.8.23 GET_TPE_PRM Built-in Procedure A–166
A.8.24 GET_TSK_INFO Built-In Procedure A–168
A.8.25 GET_USEC_SUB Built-In Procedure A–170
A.8.26 GET_USEC_TIM Built-In Function A–171
A.8.27 GET_VAR Built-In Procedure A–171
A.8.28 GO TO Statement A–175
A.9 - H - KAREL LANGUAGE DESCRIPTION A–176
A.9.1 HOLD Action A–176
A.9.2 HOLD Statement A–177
A.10 - I - KAREL LANGUAGE DESCRIPTION A–178
A.10.1 IF ... ENDIF Statement A–178
A.10.2 IN Clause A–179
A.10.3 %INCLUDE Translator Directive A–180
A.10.4 INDEX Built-In Function A–181
A.10.5 INI_DYN_DISB Built-In Procedure A–181
A.10.6 INI_DYN_DISE Built-In Procedure A–183
A.10.7 INI_DYN_DISI Built-In Procedure A–184
A.10.8 INI_DYN_DISP Built-In Procedure A–186
A.10.9 INI_DYN_DISR Built-In Procedure A–187
A.10.10 INI_DYN_DISS Built-In Procedure A–188
A.10.11 INIT_QUEUE Built-In Procedure A–189
A.10.12 INIT_TBL Built-In Procedure A–190
A.10.13 IN_RANGE Built-In Function A–201
A.10.14 INSERT_NODE Built-In Procedure A–202
A.10.15 INSERT_QUEUE Built-In Procedure A–203
A.10.16 INTEGER Data Type A–204
A.10.17 INV Built-In Function A–205
A.10.18 IO_MOD_TYPE Built-In Procedure A–206

A.10.19 IO_STATUS Built-In Function A–207
A.11 - J - KAREL LANGUAGE DESCRIPTION A–208
A.11.1 J_IN_RANGE Built-In Function A–208
A.11.2 JOINTPOS Data Type A–209
A.11.3 JOINT2POS Built-In Function A–209
A.12 - K - KAREL LANGUAGE DESCRIPTION A–211
A.12.1 KCL Built-In Procedure A–211
A.12.2 KCL_NO_WAIT Built-In Procedure A–212
A.12.3 KCL_STATUS Built-In Procedure A–213
A.13 - L - KAREL LANGUAGE DESCRIPTION A–213
A.13.1 LN Built-In Function A–213
A.13.2 LOAD Built-In Procedure A–214
A.13.3 LOAD_STATUS Built-In Procedure A–215
A.13.4 LOCK_GROUP Built-In Procedure A–216
A.13.5 %LOCKGROUP Translator Directive A–217
A.14 - M - KAREL LANGUAGE DESCRIPTION A–218
A.14.1 MIRROR Built-In Function A–218
A.14.2 MODIFY_QUEUE Built-In Procedure A–219
A.14.3 MOTION_CTL Built-In Function A–221
A.14.4 MOUNT_DEV Built-In Procedure A–221
A.14.5 MOVE_FILE Built-In Procedure A–222
A.14.6 MSG_CONNECT Built-In Procedure A–223
A.14.7 MSG_DISCO Built-In Procedure A–225
A.14.8 MSG_PING A–225
A.15 - N - KAREL LANGUAGE DESCRIPTION A–226
A.15.1 NOABORT Action A–226
A.15.2 %NOABORT Translator Directive A–227
A.15.3 %NOBUSYLAMP Translator Directive A–227
A.15.4 NODE_SIZE Built-In Function A–227
A.15.5 %NOLOCKGROUP Translator Directive A–229
A.15.6 NOMESSAGE Action A–230
A.15.7 NOPAUSE Action A–230
A.15.8 %NOPAUSE Translator Directive A–231
A.15.9 %NOPAUSESHFT Translator Directive A–231
A.16 - O - KAREL LANGUAGE DESCRIPTION A–232
A.16.1 OPEN FILE Statement A–232
A.16.2 OPEN HAND Statement A–233
A.16.3 OPEN_TPE Built-In Procedure A–233
A.16.4 ORD Built-In Function A–234
A.16.5 ORIENT Built-In Function A–235
A.17 - P - KAREL LANGUAGE DESCRIPTION A–236
A.17.1 PATH Data Type A–236
A.17.2 PATH_LEN Built-In Function A–238
A.17.3 PAUSE Action A–238
A.17.4 PAUSE Condition A–239
A.17.5 PAUSE Statement A–240
A.17.6 PAUSE_TASK Built-In Procedure A–241
A.17.7 PEND_SEMA Built-In Procedure A–242
A.17.8 PIPE_CONFIG Built-In Procedure A–242
A.17.9 POP_KEY_RD Built-In Procedure A–243
A.17.10 Port_Id Action A–244
A.17.11 Port_Id Condition A–245
A.17.12 POS Built-In Function A–246

A.17.13 POS2JOINT Built-In Function A–246
A.17.14 POS_REG_TYPE Built-In Procedure A–248
A.17.15 POSITION Data Type A–249
A.17.16 POST_ERR Built-In Procedure A–250
A.17.17 POST_ERR_L Built-In Procedure A–251
A.17.18 POST_SEMA Built-In Procedure A–253
A.17.19 PRINT_FILE Built-In Procedure A–253
A.17.20 %PRIORITY Translator Directive A–254
A.17.21 PROG_BACKUP Built-In Procedure A–255
A.17.22 PROG_CLEAR Built-In Procedure A–258
A.17.23 PROG_RESTORE Built-In Procedure A–260
A.17.24 PROG_LIST Built-In Procedure A–262
A.17.25 PROGRAM Statement A–263
A.17.26 PULSE Action A–264
A.17.27 PULSE Statement A–265
A.17.28 PURGE CONDITION Statement A–266
A.17.29 PURGE_DEV Built-In Procedure A–267
A.17.30 PUSH_KEY_RD Built-In Procedure A–268

A.18 - Q - KAREL LANGUAGE DESCRIPTION A–269
A.18.1 QUEUE_TYPE Data Type A–269

A.19 - R - KAREL LANGUAGE DESCRIPTION A–269
A.19.1 READ Statement A–269
A.19.2 READ_DICT Built-In Procedure A–271
A.19.3 READ_DICT_V Built-In-Procedure A–272
A.19.4 READ_KB Built-In Procedure A–273
A.19.5 REAL Data Type A–277
A.19.6 Relational Condition A–279
A.19.7 RELAX HAND Statement A–280
A.19.8 RELEASE Statement A–281
A.19.9 REMOVE_DICT Built-In Procedure A–281
A.19.10 RENAME_FILE Built-In Procedure A–282
A.19.11 RENAME_VAR Built-In Procedure A–283
A.19.12 RENAME_VARS Built-In Procedure A–283
A.19.13 REPEAT ... UNTIL Statement A–284
A.19.14 RESET Built-In Procedure A–285
A.19.15 RESUME Action A–286
A.19.16 RESUME Statement A–286
A.19.17 RETURN Statement A–287
A.19.18 ROUND Built-In Function A–288
A.19.19 ROUTINE Statement A–289
A.19.20 RUN_TASK Built-In Procedure A–290

A.20 - S - KAREL LANGUAGE DESCRIPTION A–291
A.20.1 SAVE Built-In Procedure A–291
A.20.2 SAVE_DRAM Built-In Procedure A–292
A.20.3 SELECT ... ENDSELECT Statement A–293
A.20.4 SELECT_TPE Built-In Procedure A–294
A.20.5 SEMA_COUNT Built-In Function A–294
A.20.6 SEMAPHORE Condition A–295
A.20.7 SEND_DATAPC Built-In Procedure A–295
A.20.8 SEND_EVENTPC Built-In Procedure A–296
A.20.9 SET_ATTR_PRG Built-In Procedure A–297
A.20.10 SET_CURSOR Built-In Procedure A–299
A.20.11 SET_EPOS_REG Built-In Procedure A–300
A.20.12 SET_EPOS_TPE Built-In Procedure A–301

A.20.13 SET_FILE_ATR Built-In Procedure A–302
A.20.14 SET_FILE_POS Built-In Procedure A–303
A.20.15 SET_INT_REG Built-In Procedure A–304
A.20.16 SET_JPOS_REG Built-In Procedure A–304
A.20.17 SET_JPOS_TPE Built-In Procedure A–305
A.20.18 SET_LANG Built-In Procedure A–306
A.20.19 SET_PERCH Built-In Procedure A–307
A.20.20 SET_PORT_ASG Built-In Procedure A–308
A.20.21 SET_PORT_ATR Built-In Function A–309
A.20.22 SET_PORT_CMT Built-In Procedure A–311
A.20.23 SET_PORT_MOD Built-In Procedure A–312
A.20.24 SET_PORT_SIM Built-In Procedure A–313
A.20.25 SET_PORT_VAL Built-In Procedure A–314
A.20.26 SET_POS_REG Built-In Procedure A–315
A.20.27 SET_POS_TPE Built-In Procedure A–316
A.20.28 SET_PREG_CMT Built-In-Procedure A–317
A.20.29 SET_REAL_REG Built-In Procedure A–317
A.20.30 SET_REG_CMT Built-In-Procedure A–317
A.20.31 SET_SREG_CMT Built-in Procedure A–318
A.20.32 SET_STR_REG Built-in Procedure A–319
A.20.33 SET_TIME Built-In Procedure A–319
A.20.34 SET_TPE_CMT Built-In Procedure A–321
A.20.35 SET_TRNS_TPE Built-In Procedure A–321
A.20.36 SET_TSK_ATTR Built-In Procedure A–322
A.20.37 SET_TSK_NAME Built-In Procedure A–323
A.20.38 SET_VAR Built-In Procedure A–324
A.20.39 %SHADOWVARS Translator Directive A–327
A.20.40 SHORT Data Type A–327
A.20.41 SIGNAL EVENT Action A–328
A.20.42 SIGNAL EVENT Statement A–328
A.20.43 SIGNAL SEMAPHORE Action A–329
A.20.44 SIN Built-In Function A–329
A.20.45 SQRT Built-In Function A–330
A.20.46 %STACKSIZE Translator Directive A–330
A.20.47 STD_PTH_NODE Data Type A–331
A.20.48 STOP Action A–331
A.20.49 STOP Statement A–332
A.20.50 STRING Data Type A–333
A.20.51 STR_LEN Built-In Function A–334
A.20.52 STRUCTURE Data Type A–334
A.20.53 SUB_STR Built-In Function A–335

A.21 - T - KAREL LANGUAGE DESCRIPTION A–336
A.21.1 TAN Built-In Function A–336
A.21.2 %TIMESLICE Translator Directive A–337
A.21.3 %TPMOTION Translator Directive A–337
A.21.4 TRANSLATE Built-In Procedure A–337
A.21.5 TRUNC Built-In Function A–339

A.22 - U - KAREL LANGUAGE DESCRIPTION A–340
A.22.1 UNHOLD Action A–340
A.22.2 UNHOLD Statement A–340
A.22.3 UNINIT Built-In Function A–341
A.22.4 %UNINITVARS Translator Directive A–341
A.22.5 UNLOCK_GROUP Built-In Procedure A–342
A.22.6 UNPAUSE Action A–343
A.22.7 UNPOS Built-In Procedure A–344

A.22.8 USING ... ENDUSING Statement A–344
A.23 - V - KAREL LANGUAGE DESCRIPTION A–345
A.23.1 V_CAM_CALIB *i*RVision Built-In Procedure A–345
A.23.2 V_GET_OFFSET *i*RVision Built-In Procedure A–347
A.23.3 V_GET_PASSFL *i*RVision Built-In Procedure A–348
A.23.4 V_GET_QUEUE *i*RVision Built-in Procedure A–350
A.23.5 V_INIT_QUEUE *i*RVision Built-in Procedure A–350
A.23.6 V_RALC_QUEUE *i*RVision Built-in Procedure A–351
A.23.7 V_RUN_FIND *i*RVision Built-In Procedure A–351
A.23.8 V_SET_REF *i*RVision Built-In Procedure A–353
A.23.9 V_START_VTRK *i*RVision Built-in Procedure A–354
A.23.10 V_STOP_VTRK *i*RVision Built-in Procedure A–354
A.23.11 VAR_INFO Built-In Procedure A–355
A.23.12 VAR_LIST Built-In Procedure A–357
A.23.13 VECTOR Data Type A–360
A.23.14 VOL_SPACE Built-In Procedure A–361
A.23.15 VREG_FND_POS *i*RVision Built-in Procedure A–362
A.23.16 VREG_OFFSET *i*RVision Built-in Procedure A–363
A.24 - W - KAREL LANGUAGE DESCRIPTION A–364
A.24.1 WAIT FOR Statement A–364
A.24.2 WHEN Clause A–364
A.24.3 WHILE...ENDWHILE Statement A–365
A.24.4 WITH Clause A–365
A.24.5 WRITE Statement A–366
A.24.6 WRITE_DICT Built-In Procedure A–367
A.24.7 WRITE_DICT_V Built-In Procedure A–368
A.25 - X - KAREL LANGUAGE DESCRIPTION A–369
A.25.1 XML_ADDTAG Built-In Procedure A–369
A.25.2 XML_GETDATA Built-In Procedure A–370
A.25.3 XML_REMTAG Built-In Procedure A–371
A.25.4 XML_SCAN Built-In Procedure A–371
A.25.5 XML_SETVAR Built-In Procedure A–373
A.25.6 XYZWPR Data Type A–374
A.25.7 XYZWPREXT Data Type A–375
A.26 - Y - KAREL LANGUAGE DESCRIPTION A–375
A.27 - Z - KAREL LANGUAGE DESCRIPTION A–375

A.1 概述

本附录按字母顺序描述了每个标准的KAREL语言元素，包括：

- 数据类型
- 可执行语句和子句
- 条件处理程序条件和操作
- 内置例程
- 译者指令

每个元素的典型用法的简要示例包括在每个描述中。

注意如果在程序执行期间遇到任何未初始化的变量作为内置例程的参数，程序将暂停并显示错误。 使用KCL> ABORT命令初始化变量，KCL> SET VARIABLE命令或中止程序。

约定

本节按字母顺序描述KAREL语言的每个标准元素。 每个描述包括以下信息：

- **目的：**表示元素在语言中的特定用途
- **语法：**描述访问KAREL中元素所需的正确语法。 表A-1描述了使用的语法表示法。

表A-1. 语法表示法

句法	含义	例子	结果
< >	包含的词是可选的	AAA <BBB>	AAA AAA BBB
{ }	包含的词是可选的， 可以重复的	AAA {BBB}	AAA AAA BBB AAA BBB BBB AAA BBB BBB BBB
|	分隔选择	AAA | BBB	AAA BBB
< | >	如果只能使用一个或不使用，则分开备选方案	AAA <BBB | CCC>	AAA AAA BBB AAA CCC
||	必须使用一种替代方案	AAA || BBB | CCC ||	AAA BBB AAA CCC

表 A–1. 语法表示法（续）

句法	含义	例子	结果
{ | }	可以使用任何替代方案的组合	AAA {BBB | CCC}	AAA AAA BBB AAA CCC AAA BBB CCC AAA CCC BBB AAA BBB CCC BBB BBB
< < | > >	允许嵌套符号。 首先查看最里面的符号以查看它描述的内容，然后查看下一个最内层以查看它描述的内容，依此类推。	AAA <BBB <CCC | DDD>	AAA AAA BBB AAA BBB CCC AAA BBB DDD

如果内置函数是一个函数，则使用以下表示法来标识函数返回的值的数据类型：

Function Return Type: data_typ

功能和过程的输入和输出参数数据类型标识为：

[in] param_name: data_type

[out] param_name: data_type

其中：

[in] 指定传递给例程的参数的数据类型

[out] 指定从例程传回程序的参数的数据类型

%ENVIRONMENT Group指定内置函数和过程的%ENVIRONMENT组，由离线转换程序使用。 有效值为：BYNAM, CTDEF, ERRS, FDEV, FLBT, IOSETUP, KCL, MEMO, MIR, MOTN, MULTI, PATHOP, PBQMGR, REGOPE, STRNG, SYSDEF, TIM, TPE, TRANS, UIF, VECTR. SYSTEM组由离线翻译器自动使用。

- **详细信息：**列出适用于语言元素的特定规则。 斜体字体用于表示用户在元素的语法内输入的关键字。
- **另请参阅：**将读者引用到文档中可以找到更多信息的位置。

- **示例**：显示元素的简要示例和说明。 表A-2至表A-8按元素类型列出了本附录中描述的KAREL语言元素。 表A-9按字母顺序列出了这些元素。

表 A–2. 动作

```
ABORT Action
Assignment Action
CANCEL Action
CONTINUE Action
DISABLE CONDITION Action
ENABLE CONDITION Action
HOLD Action
NOABORT Action
NOMESSAGE Action
NOPAUSE Action
PAUSE Action
Port_Id Action
PULSE Action
RESUME Action
SIGNAL EVENT Action
SIGNAL SEMAPHORE Action
STOP Action
UNHOLD Action
UNPAUSE Action
```

表 A–3. 子句

```
EVAL Clause
FROM Clause
IN Clause
WHEN Clause
WITH Clause
```

表 A–4. 条件

```
ABORT Condition
CONTINUE Condition
ERROR Condition
EVENT Condition
PAUSE Condition
Port_Id Condition
Relational Condition
SEMAPHORE Condition
```

表 A–5. 数据类型

```
ARRAY Data Type
BOOLEAN Data Type
BYTE Data Type
CONFIG Data Type
DISP_DAT_T Data Type
FILE Data Type
INTEGER Data Type
JOINTPOS Data Type
PATH Data Type
POSITION Data Type
QUEUE_TYPE Data Type
REAL Data Type
SHORT Data Type
STD_PTH_NODE Data Type
STRING Data Type
STRUCTURE Data Type
VECTOR Data Type
XYZWPR Data Type
XYZWPREXT Data Type
```

表 A–6. 指令

```
%ALPHABETIZE
%CMOSVARS
%CMOS2SHADOW
%COMMENT
%CRTDEVICE
%DEFGROUP
%DELAY
%ENVIRONMENT
%INCLUDE
%LOCKGROUP
%NOABORT
%NOBUSYLAMP
%NOLOCKGROUP
%NOPAUSE
%NOPAUSESHFT
%PRIORITY
%SHADOWVARS
%STACKSIZE
%TIMESLICE
%TPMOTION
%UNINITVARS
```

表 A–7. KAREL 内置常规摘要（续）

类别	标识		
Byname	CALL_PROG CALL_PROGLIN	CURR_PROG FILE_LIST	PROG_LIST VAR_INFO VAR_LIST
Data Acquisition	DAQ_CHECKP DAQ_REGPIPE	DAQ_START DAQ_STOP	DAQ_UNREG DAQ_WRITE
Error Code Handling	ERR_DATA	POST_ERR	POST_ERR_L
File and Device Operation	CHECK_NAME COMPARE_FILE COPY_FILE DELETE_FILE DISMOUNT_DEV DOSFILE_INF FORMAT_DEV	MOUNT_DEV MOVE_FILE PRINT_FILE PURGE_DEV RENAME_FILE	XML_ADDTAG XML_GETDATA XML_REMTAG XML_SCAN XML_SETVAR

表 A–7. KAREL 内置常规摘要（续）

类别	标识		
Serial I/O, File Usage	BYTES_AHEAD BYTES_LEFT CLR_IO_STAT GET_FILE_POS GET_PORT_ATR	IO_STATUS MSG_CONNECT MSG_DISCO MSG_PING PIPE_CONFIG	SET_FILE_ATR SET_FILE_POS SET_PORT_ATR VOL_SPACE
Process I/O Setup	CLR_PORT_SIM GET_PORT_ASG GET_PORT_CMT GET_PORT_MOD	GET_PORT_SIM GET_PORT_VAL IO_MOD_TYPE SET_PORT_ASG	SET_PORT_CMT SET_PORT_MOD SET_PORT_SIM SET_PORT_VAL
KCL Operation	KCL	KCL_NO_WAIT	KCL_STATUS
Memory Operation	CLEAR CREATE_VAR LOAD LOAD_STATUS	PROG_BACKUP PROG_CLEAR PROG_RESTORE RENAME_VAR	RENAME_VARS SAVE SAVE_DRAM
Mirror	MIRROR		
Motion and Program Control	CNCL_STP_MTN	MOTION_CTL	RESET
Multi-programming	ABORT_TASK CLEAR_SEMA CONT_TASK GET_TSK_INFO LOCK_GROUP	PAUSE_TASK PEND_SEMA POST_SEMA RUN_TASK	SEMA_COUNT SET_TSK_ATTR SET_TSK_NAME UNLOCK_GROUP
Path Operation	APPEND_NODE COPY_PATH	DELETE_NODE INSERT_NODE	NODE_SIZE PATH_LEN
Personal Computer Communications	ADD_BYNAMEPC ADD_INTPC	ADD_REALPC ADD_STRINGPC	SEND_DATAPC SEND_EVENTPC
Position	CHECK_EPOS CNV_JPOS_REL CNV_REL_JPOS CURPOS CURJPOS	FRAME IN_RANGE J_IN_RANGE JOINT2POS	POS POS2JOINT SET_PERCH UNPOS
Queue Manager	APPEND_QUEUE COPY_QUEUE DELETE_QUEUE	GET_QUEUE INIT_QUEUE	INSERT_QUEUE MODIFY_QUEUE

表 A–7. KAREL 内置常规摘要（续）

类别	标识		
Register Operation	CLR_POS_REG GET_JPOS_REG GET_POS_REG GET_PREG_CMT GET_REG GET_REG_CMT	GET_SREG_CMT GET_STR_REG POS_REG_TYPE SET_EPOS_REG SET_INT_REG SET_JPOS_REG	SET_POS_REG SET_PREG_CMT SET_REAL_REG SET_REG_CMT SET_SREG_CMT SET_STR_REG
String Operation	CNV_CNF_STRG CNV_CONF_STR CNV_INT_STR CNV_REAL_STR	CNV_STR_CONF CNV_STR_INT CNV_STR_REAL	STR_LEN SUB_STR
System	ABS ACOS ARRAY_LEN ASIN ATAN2 BYNAME CHR	COS EXP GET_VAR INDEX INV LN ORD	ROUND SET_VAR SIN SQRT TAN TRUNC UNINIT
Time-of-Day Operation	CNV_STR_TIME CNV_TIME_STR	GET_TIME GET_USEC_SUB	GET_USEC_TIM SET_TIME
TPE Program	AVL_POS_NUM CLOSE_TPE COPY_TPE CREATE_TPE DEL_INST_TPE GET_ATTR_PRG GET_JPOS_TPE	GET_POS_FRM GET_POS_TPE GET_POS_TYP GET_TPE_CMT GET_TPE_PRM OPEN_TPE SELECT_TPE	SET_ATTR_PRG SET_EPOS_TPE SET_JPOS_TPE SET_POS_TPE SET_TPE_CMT SET_TRNS_TPE
Translate	TRANSLATE		
User Interface	ACT_SCREEN ADD_DICT ATT_WINDOW_D ATT_WINDOW_S CHECK_DICT CNC_DYN_DISB CNC_DYN_DISE CNC_DYN_DISI CNC_DYN_DISP CNC_DYN_DISR CNC_DYN_DISS DEF_SCREEN DEF_WINDOW	DET_WINDOW DISCTRL_ALPH DISCTRL_FORM DISCTRL_LIST DISCTRL_PLMN DISCTRL_SBMN DISCTRL_TBL FORCE_SPMENU INI_DYN_DISB INI_DYN_DISE INI_DYN_DISI INI_DYN_DISP INI_DYN_DISR	INI_DYN_DISS INIT_TBL POP_KEY_RD PUSH_KEY_RD READ_DICT READ_DICT_V READ_KB REMOVE_DICT SET_CURSOR SET_LANG WRITE_DICT WRITE_DICT_V

表 A–7. KAREL 内置常规摘要（续）

类别	标识		
Vector	APPROACH	ORIENT	
Vision Operation	V_CAM_CALIB V_GET_OFFSET V_GET_PASSFL V_GET_QUEUE	V_INIT_QUEUE V_RALC_QUEUE V_RUN_FIND V_SET_REF	V_START_VTRK V_STOP_VTRK VREG_FND_POS VREG_OFFSET

表 A–8. 项目

CR Input/Output Item

表 A–9. 声明

```
ABORT Statement
Assignment Statement
ATTACH Statement
CANCEL Statement
CANCEL FILE Statement
CLOSE FILE Statement
CLOSE HAND Statement
CONDITION..ENDCONDITION Statement
CONNECT TIMER Statement
DELAY Statement
DISABLE CONDITION Statement
DISCONNECT TIMER Statement
ENABLE CONDITION Statement
FOR..ENDFOR Statement
GO TO Statement
HOLD Statement
IF..ENDIF Statement
OPEN FILE Statement
OPEN HAND Statement
PAUSE Statement
PROGRAM Statement
PULSE Statement
PURGE CONDITION Statement
READ Statement
RELAX HAND Statement
RELEASE Statement
REPEAT...UNTIL Statement
RESUME Statement
RETURN Statement
ROUTINE Statement
SELECT..ENDSELECT Statement
SIGNAL EVENT Statement
STOP Statement
UNHOLD Statement
USING..ENDUSING Statement
WAIT FOR Statement
WHILE..ENDWHILE Statement
WRITE Statement
```

A.2 - A - KAREL 语言字母描述

A.2.1 ABORT 功能

用途：中止执行正在运行或已暂停的任务

语法： ABORT <PROGRAM[n]>

细节:

- 如果任务执行正在运行或暂停，则ABORT操作将中止任务执行。
- ABORT操作后可以跟随PROGRAM [n]，其中n是要中止的任务编号。 使用GET_TASK_INFO获取任务编号。
- 如果未指定PROGRAM [n]，则当前任务执行被中止。

另请参见：GET_TSK_INFO 内置功能

第6章 条件处理器

示例：有关详细的程序示例，请参见第B.6节 “路径变量和条件处理程序（PTH_MOVE.KL）” 。

A.2.2 ABORT 条件

用途：监视任务执行的中止

语法： ABORT <PROGRAM[n]>

- 当任务被中止时，ABORT条件满足。 条件处理程序指定的操作将被执行。
- 如果未指定PROGRAM [n]，则使用当前任务编号。
- 如果任务执行被中止，那么作为例程调用的操作将不会执行。
- 子句PROGRAM [n]后跟ABORT条件，其中n是要监控的任务号。 使用GET_TSK_INFO获取任务编号。

另请参见：CONDITION ... ENDCONDITION语句，GET_TSK_INFO内置功能，第6章 条件处理器，附录E,"语法图"，了解其他语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.6节 “路径变量和条件处理程序” （PTH_MOVE.KL）

第B.10节“使用动态显示内置程序”（DYN_DISP.KL）

A.2.3 ABORT 声明

用途：终止任务执行并取消任何动作进展（或未决）

语法： ABORT <PROGRAM[n]>

- ABORT 后，该程序无法恢复。 它必须重新启动。
- 该语句后可以跟随PROGRAM [n]，其中n是要中止的任务编号。

另请参见： “语法图”，了解其他语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.2.4 ABORT_TASK 内置程序

用途：中止指定的运行或暂停的任务

语法： ABORT_TASK(task_name, force_sw, cancel_mtn_sw, status)

Input/Output 参数：

[in] task_name :STRING

[in] force_sw :BOOLEAN

[in] cancel_mtn_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :MULTI

细节:

- *task_name* 是要中止的任务的名称。 如果任务名称为“* ALL *”，则除了具有“忽略中止请求”属性集的任务外，所有正在执行或暂停的任务都将中止。
- *force_sw*，如果为true，则指定即使任务设置了“忽略中止请求”也中止任务。 如果task_name是'* ALL *'，则忽略force_sw。
- *cancel_mtn_sw* 指定是否取消属于指定任务的所有组的动作。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：CONT_TASK，RUN_TASK，PAUSE_TASK内置程序，NO_ABORT操作，%NO_ABORT译者指令，第15章 多任务

示例：有关详细程序示例，请参见第B.10节"使用动态显示内置程序"（DYN_DISP.KL）。

A.2.5 ABS 内置函数

用途：返回参数x的绝对值，该参数可以是INTEGER或REAL表达式

语法： ABS(x)

函数返回类型 :INTEGER or REAL

Input/Output 参数：

[in] x :INTEGER or REAL expression

%ENVIRONMENT Group :SYSTEM

细节:

- 返回x的绝对值，其数据类型与x相同。

示例：有关详细的程序示例，请参见第B.7节"列出文件和程序以及操作字符串（LIST_EX.KL）"。

A.2.6 ACOS 内置函数

用途：以指定参数的度数返回反余弦（cos-1）

语法： ACOS(x)

函数返回类型：REAL

Input/Output 参数：

[in] x :REAL

%ENVIRONMENT Group :SYSTEM

细节:

- x必须介于-1.0和1.0之间; 否则程序将因错误而中止。
- 返回x的反余弦值。

示例：以下示例将ans_r设置为-1的反余弦并将此值写入屏幕。 以下示例的输出为180度。

ACOS 内置函数

```
routine take_acos
var
  ans_r:  real
begin
  ans_r = acos (-1)
  WRITE ('acos -1 ', ans_r, CR)
END take_acos
```

第二个示例导致程序中止，因为输入值小于-1且不在有效范围内。

ACOS 内置函数

```
routine take_acos
var
  ans_r:  real
begin
  ans_r = acos (-1.5) -- causes program to abort
  WRITE ('acos -1.5 ', ans_r, CR)
END take_acos
```

A.2.7 ACT_SCREEN 内置程序

用途 ：激活屏幕

语法：ACT_SCREEN

Input/Output 参数：

[in] screen_name :STRING

[out] old_screen_n :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节:

- 清除与屏幕关联的显示设备，并显示连接到屏幕的所有窗口。
- *screen_name* 必须是包含先前定义的屏幕名称的字符串，请参阅内置的DEF_SCREEN。
- 然后在 *old_screen_n* 中返回此替换的名称。
- 在激活新屏幕之前需要选择USER或USER2菜单，否则状态将设置为9093。

  - 要在激活屏幕之前强制选择示教器用户菜单，请使用

  FORCE_SPMENU（tp_panel，SPI_TPUSER，1）。

  - 要在激活屏幕之前强制选择CRT / KB用户菜单，请使用

  FORCE_SPMENU（crt_panel，SPI_TPUSER，1）。

- 如果退出并重新进入USER菜单，只要名为ACT_SCREEN的KAREL任务继续运行，您的屏幕就会重新激活。 当KAREL任务中止时，将重新激活系统的用户屏幕。 有关系统用户屏幕的详细信息，请参阅第7.10节。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见：DEF_SCREEN内置过程

示例：有关详细的程序示例，请参阅以下部分：第

B.12节"从字典文件显示列表"（DCLST_EX.KL）

A.2.8 ACT_TBL 内置程序

用途 ：在示教器上显示标签时对键进行操作

语法：ACT_TBL(action, def_item, table_data, term_char, attach_wind, status)

Input/Output 参数：

[in,out] action :INTEGER

[in,out] def_item :INTEGER

[in,out] table_data :XWORK_T

[out] term_char :INTEGER

[out] attach_wind :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节:

- 如果需要在每次击键时进行特殊处理，或者如果需要在不退出表格菜单的情况下进行功能键处理，则只应使用INIT_TBL和ACT_TBL内置例程而不是DISCTRL_TBL。
- 在使用此内置函数之前，必须调用内置的INIT_TBL。 如果INIT_TBL发生错误，请勿继续。
- *action* 必须是包含文件KLEVKEYS.KL中定义的常量之一。 ACT_TBL将按键操作并返回。以下键具有特殊含义：
  - ky_disp_updt（初始显示） - 显示表格标题，显示功能键标签。

    默认项目在第一行显示并突出显示。 显示剩余行。
    为所有值启动动态显示。 这应该是传递给ACT_TBL的第一个密钥。
  - ky_reissue（读取键） - ACT_TBL读取一个键，对其进行操作，并将其返回。
  - ky_cancel（取消表） - 取消所有动态显示。 如果term_char不等于ky_new_menu，这应该是传递给ACT_TBL的最后一个键。 如果选择了新菜单，ACT_TBL将取消动态显示，您不应使用ky_cancel。
- *def_item* 是包含要在输入表格时突出显示的项目的行。返回时，def_item是包含按下终止字符时当前突出显示的项目的行。
- *table_data* 用于显示和控制表。不要更改此数据;它在内部使用。
- *term_char* 接收一个代码，指示终止表的字符或其他条件。密钥终止条件的代码在包含文件KLEVKEYS.KL中定义。通常返回的键是预定义的常量，如下所示：
  - ky_undef - 没有按下终止字符
  - ky_select - 选择了一个可选项
  - key_new_menu - 选择了一个新菜单
  - ky_f1到ky_f10 - 选择了一个功能键
- 如果在操作为ky_disp_updt时需要将表管理器连接到显示设备，并且当操作为ky_cancel时将其与显示设备分离，则应将attach_wind设置为TRUE。如果已附加，则此参数可以设置为FALSE。通常这应该是真的。
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。

示例：请参阅内置INIT_TBL的示例，第A.10.12节。

A.2.9 ADD_BYNAMEPC 内置程序

用途：在给定数据缓冲区的情况下，将整数，实数或字符串值添加到KAREL字节中。

语法：ADD_BYNAMEPC(dat_buffer, dat_index, prog_name, var_name, status)

Input/Output 参数：

[in] dat_buffer :ARRAY OF BYTE

[in,out] dat_index :INTEGER

[in] prog_name :STRING

[in] var_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PC

细节:

- *dat_buffer* - anarrayofup到244个字节。
- *dat_index* - 放置字符串值的起始字节数。
- *prog_name* - 指定包含指定变量的程序的名称。
- *var_name* - 指静态程序变量。 这仅由整数，实数或字符串变量支持（不支持数组和结构）。
- *status* - 尝试操作的状态。 如果不是0，则发生错误并且数据为
- 没有放入缓冲区。

内置的ADD_BYNAMEPC以与KAREL内置插件ADD_INTPC，ADD_REALPC和ADD_STRINGPC相同的方式将整数，实数和字符串值添加到数据缓冲区。

另请参见：ADD_BYNAMEPC，ADD_INTPC，ADD_REALPC，ADD_STRINGPC

示例：有关内置ADD_BYNAMEPC的示例，请参阅以下内容。

ADD_BYNAMEPC 内置程序

```
PROGRAM TESTBYNM
%ENVIRONMENT PC
CONST
  er_abort = 2
VAR
```

```
  dat_buffer:  ARRAY[100] OF BYTE
  index:       INTEGER
  status:      INTEGER
BEGIN
  index = 1
  ADD_BYNAMEPC(dat_buffer,index, 'TESTDATA','INDEX',status)
  IF status<>0 THEN
    POST_ERR(status,'',0,er_abort)
  ENDIF
END testbynm
```

A.2.10 ADD_DICT 内置程序

用途：将指定的字典添加到指定的语言。

语法： ADD_DICT(file_name, dict_name, lang_name, add_option, status)

Input/Output 参数：

[in] file_name :STRING

[in] dict_name :STRING

[in] lang_name :STRING

[in] add_option :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *file_name* 指定要添加的字典文件的设备，路径和文件名。 假定文件类型为“.TX”（文本文件）。
- *dict_name* 指定在读取和写入字典元素时要使用的字典的名称。 仅使用4个字符。
- *lang_name* 指定字典将添加到哪种语言。 应使用以下预定义常量之一：

  dp_default

  dp_english

dp_japanese

dp_french

dp_german

dp_spanish

- 除非需要多种语言，否则应使用默认语言。
- *add_option* 应该如下：
- *dp_dram* 字典将被加载到DRAM内存并保留到下一个INIT START。
- *status* 说明了尝试操作的状态。 如果不等于0，则添加字典文件时发生错误。

另请参阅：READ_DICT，WRITE_DICT，REMOVE_DICT内置程序，第10章字典和表格

示例：有关详细的程序示例，请参阅以下部分：

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.12节“字典文件”（DCALPHEG.UTX）

A.2.11 ADD_INTPC 内置程序

用途：将一个INTEGER值（类型16 -10HEX）添加到KAREL byte 数据缓冲区中。

语法：ADD_INTPC(dat_buffer, dat_index, number, status)

Input/Output 参数：

[in] dat_buffer :ARRAY OF BYTE

[in,out] dat_index :INTEGER

[in] number :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PC

细节:

- *dat_buffer* - 一个最多244个字节的数组。
- *dat_index* - 放入整数值的起始字节数。

- *number* - 要放入缓冲区的整数值。
- *status* - 尝试操作的状态。 如果不为0，则发生错误，数据未放入缓冲区。

KAREL内置函数ADD_BYNAMEPC,ADD_INTPC,ADD_REALPC和ADD_STRINGPC可用于以下列方式格式化KAREL字节缓冲区：将INTEGER数据添加到缓冲区，如下所示（缓冲区字节以HEX格式显示）：

起始索引 = dat_index

2个字节 - 变量类型

4个字节 - 数字

2个字节的零(0) - 缓冲标记结束

以下是从 index = 1开始放入KAREL字节数组的 INTEGER 的示例：

0 10 0 0 0 5 0 0

其中：

0 1 0 = INTEGER 变量类型

0 0 0 5 = 整数 5

0 0 = 缓冲区中的数据结束

从内置返回时，index = 7。

另请参见： ADD_BYNAMEPC, ADD_INTPC, ADD_REALPC, ADD_STRINGPC

示例： 请参阅内置函数 SEND_DATAPC 中的 TESTDATA 示例。

A.2.12 ADD_REALPC 内置程序

用途: 将REAL值(类型17 - 11 HEX)添加到KAREL字节数据缓冲区中。

语法： ADD_REALPC(dat_buffer, dat_index, number, status)

Input/Output 参数：

[in] dat_buffer :ARRAY OF BYTE

[in,out] dat_index :INTEGER

[in] number :REAL

[out] status :INTEGER

%ENVIRONMENT Group :PC

细节：

- *dat_buffer* - 最多244个字节的数组。
- *dat_index* - 放置实际值的起始字节数。
- *number* - 放入缓冲区的实际值。
- *status* - 尝试操作的状态。 如果不为0，则发生错误，数据未放入缓冲区。

KAREL内置函数 ADD_BYNAMEPC, ADD_INTPC, ADD_REALPC,ADD_STRINGPC 可用于以下列方式格式化KAREL字节缓冲区：

REAL数据按如下方式添加到缓冲区（缓冲区字节以HEX格式显示）：

起始索引 = dat_index

2个字节 - 变量类型

4个字节 - 数字

2个字节的零（0） - 缓冲区标记的结束

以下是从 index = 1 开始放入KAREL字节数组的REAL示例：

0 11 43 AC CC CD 0 0

其中：

0 11 = REAL 变量类型

43 AC CC CD = 实数 345.600006

0 0 = 缓冲区中的数据结束

从内置返回时，index = 7。

另请参见：ADD_BYNAMEPC, ADD_INTPC, ADD_REALPC, ADD_STRINGPC

示例：请参阅内置函数SEND_DATAPC中的TESTDATA示例。

A.2.13 ADD_STRINGPC 内置程序

用途：将字符串值（类型209 - D1 HEX）添加到KAREL字节数据缓冲区中。

语法：ADD_STRINGPC(dat_buffer, dat_index, item, status)

Input/Output 参数：

[in] dat_buffer :ARRAY OF BYTE

[in,out] dat_index :INTEGER

[in] item :string

[out] status :INTEGER

%ENVIRONMENT Group :PC

细节：

- *dat_buffer* - 最多244个字节的数组。
- *dat_index* - 放置字符串值的起始字节数。
- *item* - 要放入缓冲区的字符串值。
- *status* - 尝试操作的状态。 如果不为0，则发生错误，数据未放入缓冲区。

KAREL 内置函数 ADD_BYNAMEPC, ADD_INTPC, ADD_REALPC,ADD_STRINGPC 可用于以下列方式格式化KAREL字节缓冲区：

STRING数据被添加到缓冲区，如下所示：

起始索引= dat_index

2个字节 - 变量类型

1个字节 - 文本字符串的长度

文字字节

2个字节的零（0） - 缓冲区标记的结束

以下是从 index = 1开始放入KAREL字节数组的STRING示例：

0 D1 7 4D 48 53 48 45 4C 4C 0 0 0

其中：

0 D1 = STRING 变量类型

7 = 字符串中有7个字符 'MHSHELL'

4D 48 53 48 45 4C 4C 0 = 'MHSHELL' 结束字符串 0

0 0 = 缓冲区中的数据结束

从内置返回时，index = 12。

另请参阅：ADD_BYNAMEPC, ADD_INTPC, ADD_REALPC, ADD_STRINGPC

示例：请参阅内置函数SEND_DATAPC中的TESTDATA示例。

A.2.14 %ALPHABETIZE 译者指令

用途：指定在加载p代码时按字母顺序创建静态变量。

语法：%ALPHABETIZE

细节：

- 静态变量可以在KAREL程序中以任何顺序声明，%ALPHABETIZE 将使它们在 DATA 菜单或 KCL> SHOW VARS 列表中按字母顺序显示。

  示例：有关详细的程序示例，请参阅以下部分：

第B.2节 "复制路径变量" （CPY_PTH.KL）

第B.5节 "使用寄存器内置函数" （REG_EX.KL）

第B.6节 "路径变量和条件处理程序" （PTH_MOVE.KL）

第B.10节 "使用动态显示内置" （DYN_DISP.KL）

第B.12节 "从字典文件显示列表" （DCLST_EX.KL）

A.2.15 APPEND_NODE 内置程序

用途：将未初始化的节点添加到PATH参数的末尾。

语法：APPEND_NODE(path_var, status)

Input/Output 参数：

[in] path_ var :PATH

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节:

- *path_var* 是路径变量，而变量已经过去了。
- 附加的PATH节点未初始化。 可以通过直接引用其NODEDATA结构来为节点分配值。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：DELETE_NODE，INSERT_NODE内置过程

示例：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

A.2.16 APPEND_QUEUE 内置程序

用途：如果队列未满，则将条目追加到队列。

语法：APPEND_QUEUE(value, queue, queue_data, sequence_no, status)

Input/Output 参数：

[in] value :INTEGER

[in,out] queue :QUEUE_TYPE

[in,out] queue_data :ARRAY OF INTEGER

[out] sequence_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBQMGR

细节:

- *value* 指定要附加到队列的值。
- *queue* 指定队列的队列变量。
- *queue_data* 指定用于保存队列中数据的数组。 此数组的长度确定队列中的最大条目数。

- 返回 *sequence_no*，其中包含刚刚附加的条目的序列号。
- 如果条目可以附加到队列，则返回状态为零。 否则返回61001， “队列已满。”

另请参见：DELETE_QUEUE，INSERT_QUEUE内置过程。 有关更多信息和示例，请参见第15.8节“使用队列进行任务通信”。

A.2.17 APPROACH 内置函数

用途：返回表示 POSITION 参数的z轴的单位 VECTOR。

语法：APPROACH(posn)

函数返回类型: VECTOR

Input/Output 参数 :

[in] posn :POSITION

%ENVIRONMENT Group :VECTR

细节:

- 返回由参数posn的逼近向量（正z轴）组成的VECTOR。

示例：此程序允许您创建一个距离z轴另一个位置500 mm的位置。

APPROACH 函数

```
PROGRAM p_approach
VAR
  start_pos  : POSITION
  app_vector : VECTOR
BEGIN
  app_vector = APPROACH (start_pos)  --sets app_vector equal to the
                                     --z-axis of start_pos
  start_pos.location = start_pos.location + app_vector *500
                                  --creates start_pos + 500 mm
                                  --in z direction
  END p_approach
```

注意对于旧版本的KAREL，已经留有方法。 您现在应该直接访问POSITION的向量（即posn。方法）。

A.2.18 ARRAY 数据类型

用途：将变量，函数返回类型或例程参数定义为 ARRAY 数据类型。

语法： ARRAY<[size{,size}]> OF data_type

其中：

size：INTEGER 文字或常量

data_type：除 **PATH** 以外的任何类型

细节：

- *size* 表示ARRAY变量中的元素数。
- *size* 必须在1到32767范围内，并且必须在普通的ARRAY变量声明中指定。控制器中的可用内存量可能会限制ARRAY的最大大小。
- 各个元素由ARRAY名称和下标大小引用。例如，table [1]引用ARRAY表中的第一个元素。
- 整个ARRAY只能在赋值语句中使用或作为例程调用中的参数使用。在赋值语句中，两个ARRAY变量必须具有相同的大小和 *data_type*。如果大小不同，程序将成功转换，但在执行期间将被中止，错误12304，“数组长度不匹配”。
- 声明ARRAY例程参数时未指定大小;任何大小的ARRAY都可以传递给ARRAY参数。
- 声明函数的ARRAY返回类型时不使用size。但是，返回的ARRAY必须与函数调用中指定的ARRAY具有相同的大小。
- 每个元素都是由 *data_type* 指定的相同类型。
- 有效的ARRAY运算符对应于ARRAY中各个元素的有效运算符。
- 可以仅以与ARRAY的数据类型对应的格式读取或写入阵列的各个元素。
- 可以定义多维的阵列。有关更多信息，请参阅第2章。
- 可以定义可变大小的数组。有关更多信息，请参阅第2章。

另请参阅：ARRAY_LEN内置函数，第5章 ROUTINES，有关在例程调用中将ARRAY变量作为参数传递的信息第7章 文件输入/输出操作

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.8节“沿着六边形路径生成和移动”（GEN_HEX.KL）

第B.9节“使用文件和设备内置函数”（FILE_EX.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.2.19 ARRAY_LEN 内置函数

用途：返回指定数组参数中包含的元素数

语法：ARRAY_LEN(ary_var)

函数返回类型：INTEGER

Input/Output 参数：

[in] ary_var :ARRAY

%ENVIRONMENT Group :SYSTEM

- 返回的值是为 ary_var 声明的元素数，而不是在 ary_var 中初始化的元素数。

示例：有关详细的程序示例，请参见第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）。

A.2.20 ASIN 内置函数

用途：以指定参数的度数返回arcsine（sin-1）

语法：ASIN(x)

函数返回类型：REAL

Input/Output 参数：

[in] x :REAL

%ENVIRONMENT Group :SYSTEM

Details:

- 返回x的反正弦值。

- x 必须介于 -1 和 1 之间，否则程序将因错误而中止。

示例：以下示例将 ans_r 设置为 -1 的 arcsine，并将此值写入屏幕。 以下示例的输出为-90度。

ASIN 内置函数

```
ROUTINE take_asin
VAR
  ans_r:  REAL
BEGIN
  ans_r = ASIN (-1)
  WRITE ('asin -1 ', ans_r, CR)
END take_asin
```

第二个示例导致程序中止，因为输入值小于-1且不在有效范围内。

ASIN 内置函数

```
ROUTINE take_asin
VAR
  ans_r:  REAL
BEGIN
  ans_r = ASIN (-1.5) -- causes program to abort
  WRITE ('asin -1.5 ', ans_r, CR)
END take_asin
```

A.2.21 赋值功能

用途：将变量的值设置为计算表达式的结果

语法：variable {[subscript{,subscript}]| . field} = expn

其中 :

variable : 任何 KAREL 变量

subscript : INTEGER 表达式

expn : 有效的 KAREL 表达式

field : 结构化变量中的任何字段

细节：

- *variable* 可以是任何用户定义的变量，具有写访问权限的系统变量或具有写访问权限的输出端口数组。
- *subscript* 用于访问数组的元素。
- *field* 用于访问结构中的字段。
- *expn* 必须与变量或变量元素的类型相同。
- 一个例外是可以将INTEGER表达式赋给REAL。 任何位置类型都可以相互分配。
- 只有具有写访问权限的系统变量（在表11-3中列为RW，“系统变量摘要”）才能在赋值语句的左侧使用。 可以在右侧使用具有只读（RO）或读写（RW）访问权限的系统变量。
- 输入端口数组不能在赋值语句的左侧使用。

另请参阅：第3章 使用运算符，有关表达式及其评估的详细信息，请参阅第6章 条件处理程序，以获取有关使用分配操作的更多信息。

示例：以下示例使用分配操作关闭 DOUT [1] 并在 EVENT [1] 打开时将 port_var 设置为DOUT [2]。

赋值功能

```
CONDITION[1]:
  WHEN EVENT[1] DO
    DOUT[1] = OFF
    port_var = DOUT[2]
ENDCONDITION
```

A.2.22 赋值声明

用途：将变量的值设置为计算表达式的结果

语法：variable {[subscript{,subscript}]| . field} = expn

其中：

variable：任何 KAREL 变量

subscript：INTEGER 表达式

expn：有效的 KAREL 表达式

field：结构化变量中的任何字段

细节：

- *variable* 可以是任何用户定义的变量，具有写访问权限的系统变量或具有写访问权限的输出端口数组。
- *subscript* 用于访问数组的元素。
- *field* 用于访问结构中的字段。
- *expn* 必须与 *variable* 的变量或元素具有相同的类型。
- 一个例外是可以将INTEGER表达式赋给REAL。任何位置类型都可以相互分配。INTEGER，SHORT和BYTE可以相互分配。
- 如果 *variable* 的类型为ARRAY，并且未提供下标，则表达式必须是相同类型和大小的ARRAY。在翻译期间将检测到类型不匹配。执行期间将检测到大小不匹配，并导致程序因错误12304“阵列长度不匹配”而中止。
- 如果 *variable* 是用户定义的结构，并且未提供任何字段，则表达式必须是相同类型的结构。
- 只有具有写访问权限的系统变量（在表11-3中列为RW，''系统变量摘要''）可以在赋值语句的左侧使用。可以在右侧使用具有只读（RO）或读写（RW）访问权限的系统变量。
- 如果只读系统变量作为参数传递给例程，则它们按值传递，因此任何尝试通过例程中的参数修改它们（带有赋值语句）都没有效果。
- 输入端口数组不能在赋值语句的左侧使用。

另请参阅：第3章 使用运算符，有关表达式及其评估的详细信息，第2章 语言元素。 有关更详细的程序示例，请参阅附录B“KAREL示例程序”。

示例：以下示例将INTEGER文字分配给INTEGER变量，然后按字面值和值递增该变量。

赋值声明

```
int_var = 5
  int_var = 5 + int_var
```

示例：下一个示例将系统变量$SPEED乘以REAL值。然后，它用于将ARRAY变量array_1，元素loop_count分配给系统变量$ SPEED的新值。

赋值声明

```
$SPEED = $SPEED * .25
  array_1[loop_count] = $SPEED
```

示例：最后一个示例将ARRAY array_1的所有元素分配给ARRAY array_2的所有元素，并将结构struc_var_1的所有字段分配给struc_var_2的那些元素。

赋值声明

```
array_2 = array_1
  struc_var_2 = struc_var_1
```

A.2.23 ATAN2 内置函数

用途：返回以正度x轴逆时针测量的实际角度，从正x轴到连接原点的线和将x和y坐标指定为x和y参数的点

语法： ATAN2(x1, y1)

函数返回类型：REAL

Input/Output 参数 ：

[in] x1 :REAL

[in] y1 :REAL

%ENVIRONMENT Group :SYSTEM

细节：

- *x1* 和 *y1* 指定点的 *x* 和 *y* 坐标。
- 如果 *x1* 和 *y1* 都为零，则解释器将中止程序。

示例：以下示例分别对*x*，*y*和*z*使用值100,200和300来计算方向分量方向。 然后将位置p1定义为具有方向作为其定向分量的位置。

ATAN2 内置函数

```
PROGRAM p_atan2
VAR
  p1 : POSITION
  x, y, z, direction : REAL
BEGIN
  x = 100  --  use appropriate values
  y = 200  --    for x,y,z on
  z = 300  --    your robot
  direction = ATAN2(x, y)
  p1 = POS(x, y, z, 0, 0, direction, 'n') --r orientation component
```

```
                                                --POS等于角度
END p_atan2                                     --由ATAN2返回(100,200)
```

A.2.24 ATTACH 声明

用途：为机器人手臂以及辅助和扩展轴提供KAREL程序的运动控制

语法： ATTACH

细节：

- 与RELEASE语句一起使用。 如果当前未从程序控制中释放运动控制，则ATTACH语句无效。
- 如果仍然启用示教器，则会延迟执行KAREL程序，直到禁用示教器。 任务状态将显示“附加完成”。
- 停止的动作只能在执行ATTACH语句后恢复。

另请参阅： RELEASE语句，附录E，"语法图"，以获取其他语法信息。

示例： 有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.2.25 ATT_WINDOW_D 内置程序

用途：将窗口附加到显示设备上的屏幕

语法： ATT_WINDOW_D(window_name, disp_dev_nam, row, col, screen_name, status)

Input/Output 参数：

[in] window_name :STRING

[in] disp_dev_nam :STRING

[in] row :INTEGER

[in] col :INTEGER

[out] screen_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 使指定窗口中的数据显示或附加到指定显示设备上当前活动的屏幕。
- *window_name* 必须是先前定义的窗口。
- *disp_dev_nam* 必须是已定义的显示设备之一:
- 'CRT' CRT设备
- 'TP' 示教器装置
- *row* 和 *col* 表示屏幕中的位置。 第1行表示第一行; 第1列表示最左侧的列。 整个窗口必须在定位的屏幕中可见。 例如，如果屏幕是24行乘80列（由其关联的显示设备定义）并且窗口是2行乘80列，则 *row* 必须在1-23范围内; *col* 必须是1。
- 活动屏幕的名称在 *screen_name* 中返回。 这可以用于稍后分离窗口。
- 如果窗口已连接到屏幕，则会出错。
- *status* 解释了尝试操作的状态。 如果不等于0，则发生错误。

A.2.26 ATT_WINDOW_S 内置程序

用途：将窗口附加到屏幕

语法：ATT_WINDOW_S(window_name, screen_name, row, col, status)

Input/Output 参数 :

[in] window_name :STRING

[in] screen_name :STRING

[in] row :INTEGER

[in] col :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 使指定窗口中的数据显示或附加到指定行和列的指定屏幕。
- *window_name* 和 *screen_name* 必须先前定义窗口和屏幕名称。
- *row* 和 *col* 表示屏幕中的位置。 第1行表示第一行; 第1列表示最左侧的列。 整个窗口必须在屏幕上可见。 例如，如果屏幕是24行乘80列（由其关联的显示设备定义），并且窗口是2行乘80列，则 *row* 必须在1-23范围内; *col* 必须是1。
- 如果屏幕当前处于活动状态，则数据将立即显示在设备上。 否则，显示的数据没有变化。
- 如果窗口已连接到屏幕，则会出错。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：第7.10节 “用户界面提示”，内置DET_WINDOW

示例：有关详细的程序示例，请参见第B.12节 “从字典文件显示列表” （DCLST_EX.KL）。

A.2.27 AVL_POS_NUM 内置程序

用途：返回示教程序中的第一个可用位置编号

语法：AVL_POS_NUM(open_id, pos_num, status)

Input/Output 参数：

[in] open_id :INTEGER

[out] pos_num : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :TPE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前必须打开一个程序。
- *pos_num* 设置为第一个可用的位置编号。
- *status* 解释了尝试操作的状态。 如果不等于0，则发生错误。

示例：有关详细的程序示例，请参见第B.14节 “对复制的示教器程序应用偏移” （CPY_TP.KL）。

A.3 - B - KAREL 语言描述

A.3.1 BOOLEAN 数据类型

用途：将变量，函数返回类型或例程参数定义为 BOOLEAN 数据类型

语法：BOOLEAN

细节：

- BOOLEAN 数据类型表示 BOOLEAN 预定义常量 TRUE，FALSE，ON 和 OFF。

  表A-10列出了用于表示布尔预定义常量的有效和无效 BOOLEAN 值的一些示例。

表 A–10. 有效和无效的BOOLEAN值

有效	无效	原因
TRUE	T	必须使用完整的单词
ON	1	无法使用INTEGER值

- TRUE和FALSE通常表示逻辑标志，ON和OFF通常表示信号状态。 TRUE和ON是等价的，FALSE和OFF也是如此。
- 有效的BOOLEAN运算符
  - AND，OR和NOT
  - 关系运算符(>,> =,=,<>,<和<=)
- 以下具有BOOLEAN值：
  - BOOLEAN常量，无论是预定义的还是用户定义的（例如，ON是预定义的常量）
  - 结构中的BOOLEAN变量和BOOLEAN字段
  - BOOLEAN元素的阵列
  - BOOLEAN函数返回的值，无论是用户定义的还是内置的(例如，IN_RANGE(pos_var))
  - 由使用关系运算符或BOOLEAN运算符的表达式生成的值(例如，x> 5.0)
  - 数字端口的值(例如，DIN [2])
- 只能将BOOLEAN表达式赋值给BOOLEAN变量，从BOOLEAN函数例程返回，或作为参数传递给BOOLEAN参数。

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.3节“将数据保存到默认设备”（SAVE_VR.KL）

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.9节“使用文件和设备内置函数”（FILE_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.3.2 BYNAME 内置函数

用途：允许KAREL程序将名称包含在STRING中的变量作为参数传递给KAREL例程。 这意味着程序员不必在程序创建和转换期间确定变量名称。

语法：BYNAME (prog_name, var_name, entry)

Input/Output 参数：

[in] prog_name :STRING

[in] var_name :STRING

[in,out] entry :INTEGER

%ENVIRONMENT Group :system

细节：

- 此内置函数仅可用于将参数传递给KAREL例程。
- *entry* 返回 *var_name* 所在的变量数据表中的条目号。 此变量不需要初始化，也不应修改。
- *prog_name* 指定包含指定变量的程序的名称。 如果 *prog_name* 等于''（双引号），则例程默认为正在执行的任务名称。

- *var_name* 必须引用静态的程序变量。
- 如果 *var_name* 不包含有效的变量名，或者该变量不是预期的类型作为例程参数，则程序将中止。
- 无法使用BYNAME传递系统变量。
- 无法使用BYNAME传递PATH数据类型。 但是，可以使用作为PATH的用户定义类型。

示例：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

A.3.3 BYTE 数据类型

用途：将变量定义为BYTE数据类型

语法：BYTE

细节：

- BYTE 的范围为（0≤n≥255）。 没有对字节进行未初始化的检查。
- 仅允许在阵列内或结构内使用 BYTE。
- 可以将BYTE分配给SHORT和INTEGER，并将SHORT和INTEGER分配给BYTE。 执行期间将检测到BYTE范围之外的指定值，并导致程序中止。

示例：以下示例定义BYTE数组和包含BYTE的结构。

BYTE 数据类型

```
PROGRAM byte_ex
%NOLOCKGROUP
  TYPE
    mystruct = STRUCTURE
      param1: BYTE
      param2: BYTE
      param3: SHORT
    ENDSTRUCTURE
  VAR
    array_byte: ARRAY[10] OF BYTE
    myvar: mystruct
  BEGIN
    array_byte[1] = 254
    myvar.param1 = array_byte[1]
END byte_ex
```

A.3.4 BYTES_AHEAD 内置程序

用途：返回当前在KAREL文件的预读缓冲区中的输入数据的字节数。 允许KAREL程序立即检查是否已从串行端口接收数据，并且可由程序读取。 套接字消息传递和管道也支持BYTES_AHEAD。

语法：BYTES_AHEAD(file_id, n_bytes, status)

Input/Output 参数：

[in] file_id :FILE

[out] n_bytes :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :FLBT

细节：

- *file_id* 指定打开的文件。
- 必须在ATR_READAHD属性设置为大于零的情况下打开*file_id*。
- *n_byte* 是read_ahead缓冲区中的字节数。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 将为非串行设备（如文件）返回非零状态。

另请参见：第7.3.1节 “文件属性”

示例：以下示例将从仍待读取的任何字节中清除端口2（FLPY :)。

BYTES_AHEAD 内置程序

```
ROUTINE purge_port
VAR
     s1         :  STRING[1]
     n_try      :  INTEGER
     n_bytes    :  INTEGER
     stat       :  INTEGER
BEGIN
  stat=SET_PORT_ATR (port_2, ATR_READAHD, 1) -- sets FLPY: to have a read
                                        -- 提前128字节的缓冲区
  OPEN FILE fi('RO', 'rdahd.tst')
  REPEAT
     BYTES_AHEAD (fi, n_bytes, stat)
                                --获取准备读取的字节数
```

```
    if (n_bytes = 0) then     --如果没有字节，则设置stat
       stat = 282
    endif
    if (n_bytes >= 1_) then   --有要读取的字节
       read fi(s1::1)         --一次读取一个字节
       stat=io_status (fi)    --获取读操作的状态
    endif
  UNTIL stat <> 0             --继续，直到不再有字节为止
END purge_port
BEGIN
-- main program text here
END bytes_ahd
```

A.3.5 BYTES_LEFT 内置函数

用途：返回当前输入数据记录中剩余的字节数

语法：BYTES_LEFT(file_id)

Function Return Type :INTEGER

Input/Output 参数：

[in] file_id :FILE

%ENVIRONMENT Group :FLBT

细节：

- *file_id* 指定打开的文件。
- 如果未执行读取或写入操作或最后一个操作是READ *file_id*（CR），则返回零。
- 如果 *file_id* 与打开的文件或打开到相应CRT / KB，示教器和视觉窗口的预定义“文件”之一不对应，则程序将中止。
- 注意：按下ENTER键时会创建一个进给字符（LF），并由BYTES_LEFT计算。
- 仅当从键盘（示教器或CRT / KB）输入数据时，此功能才会返回非零值，而不是从文件或端口输入。

警告

此功能专门用于从窗口读取以确定是否输入了更多数据。 不要将此功能与任何其他文件设备一起使用。 否则，您可能会伤害人员或损坏设备。

另请参见：第7.10.1节“示教器上的用户菜单”，第7.10.2节“UserMenuonthe CRT / KB”

示例：以下示例读取第一个数字qd_field，然后使用BYTES_LEFT确定用户是否输入了任何其他数字。 如果是，则读取这些数字。

BYTES_LEFT 内置函数

```
PROGRAM p_bytesleft
%NOLOCKGROUP
%ENVIRONMENT flbt
CONST
  default_1 = 0
  default_2 = -1
VAR rqd_field, opt_field_1, opt_field_2: INTEGER
BEGIN
  WRITE('Enter integer field(s): ')
  READ(rqd_field)
  IF BYTES_LEFT(TPDISPLAY) > 0 THEN
    READ(opt_field_1)
  ELSE
    opt_field_1 = default_1
  ENDIF
IF BYTES_LEFT(TPDISPLAY) > 0 THEN
    READ(opt_field_2)
  ELSE
    opt_field_2 = default_2
  ENDIF
END p_bytesleft
```

A.4 - C - KAREL 语言描述

A.4.1 CALL_PROG 内置程序

用途：允许KAREL程序调用外部KAREL或示教器程序。 这意味着程序员不必在运行时确定要调用的程序。

语法：CALL_PROG(prog_name, prog_index)

Input/Output 参数 :

[in] prog_name :STRING

[in,out] prog_index :INTEGER

%ENVIRONMENT Group :PBCORE

细节 :

- *prog_name* 是当前调用任务中要执行的程序的名称。
- *prog_index* 返回 *prog_name* 所在的程序表中的条目号。 此变量不需要初始化，也不应修改。
- CALL_PROG 不能用于运行内部或外部例程。

另请参见 : CURR_PROG 和 CALL_PROGLIN 内置函数

示例 : 有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

A.4.2 CALL_PROGLIN 内置程序

用途 : 允许KAREL程序从指定的行开始调用外部KAREL或示教器程序。 这意味着程序员在创建和翻译时不需要知道将调用哪个程序。 程序员可以在运行时决定这一点。

语法 : CALL_PROGLIN(prog_name, prog_line, prog_index, pause_entry)

Input/Output 参数 :

[in] prog_name :STRING

[in] prog_line :INTEGER

[in,out] prog_index :INTEGER

[in] pause_entry :BOOLEAN

%ENVIRONMENT Group :BYNAM

细节 :

- *prog_name* 是当前调用任务中要执行的程序的名称。
- *prog_line* 指定开始执行示教程序的行。 0或1用于程序的开头。

- KAREL程序 总是在程序开始时执行。
- *prog_index* 返回 *prog_name* 所在的程序表中的条目号。 此变量不需要初始化，也不应修改。
- *pause_entry* 指定在进入程序时是否暂停程序执行。
- CALL_PROGLIN 不能用于运行内部或外部例程。

另请参见：CURR_PROG 和 CALL_PROG 内置函数

示例：有关详细的程序示例，请参见第B.5节“使用寄存器内置函数”（REG_EX.KL）。

A.4.3 CANCEL 动作

用途：终止正在进行的任何动作

语法：CANCEL <GROUP[n{,n}]>

细节：

- 取消当前正在进行或正在等待（但未停止）一个或多个组的动议。
- CANCEL 不会取消已经停止的动作。 要取消已停止的动作，请使用 CNCL_STP_MTN 内置例程。
- 如果group子句不存在，则将取消任务所控制的所有组（在定义条件时）。 特别是，如果包含条件处理程序定义的程序包含 %NOLOCKGROUP 指令，则 CANCEL 操作不会取消任何组中的运动。
- 如果某个动作被取消并且是与其他组一起进行 SIMULTANEOUS 或 COORDINATED 运动的一部分，则会取消所有组的动作。
- 机器人和辅助或延长轴平稳减速停止。 动议的其余部分被取消。
- 取消的动议视为已完成，无法恢复。
- 全局条件处理程序中的CANCEL操作取消任何挂起的动作。

A.4.4 CANCEL 声明

用途：终止正在进行的任何动作。

语法：CANCEL <GROUP[n{,n}]>

细节：

- 取消当前正在进行或正在等待（但未停止）一个或多个组的动议。
- CANCEL不会取消已经停止的动作。要取消已停止的动作，请使用CNCL_STP_MTN内置例程。
- 如果group子句不存在，则将取消该任务所控制的所有组。
- 特别是，如果使用CANCEL语句的程序包含 %NOLOCKGROUP 指令，则CANCEL语句不会取消任何组中的运动。
- 如果取消的动作是与其他组同时或协调动作的一部分，则取消所有组的动作。
- 机器人和辅助轴平稳减速停止。动议的其余部分被取消。
- 取消的动议视为已完成，无法恢复。
- 取消不会影响停止的动作。停止的动作可以恢复。
- 如果中断例程执行CANCEL语句并且被中断的语句是动作语句，则当被中断的程序恢复时，通常会使用动作语句后面的语句继续执行。
- 如果在条件处理程序调用的例程中使用CANCEL，则CANCEL可能无法按预期工作。在调用例程之前，运动可能已经放在停止的运动队列上。直接在条件处理程序中使用CANCEL操作以确保取消运动。
- 对于其他任务，无法取消运动。

另请参阅：附录E，''语法图'，以获取其他语法信息

A.4.5 CANCEL FILE 声明

用途: 取消正在进行的READ或WRITE语句。

语法： CANCEL FILE [file_var]

其他 :

file_var : 一个 FILE 变量

细节 :

- 用于取消指定文件的输入或输出
- 内置函数IO_STATUS可用于确定CANCEL FILE操作是否成功，或者是否无法确定失败原因。

另请参阅：IO_STATUS内置函数，第7章 文件输入/输出操作，附录E，''语法图'，以获取其他语法信息

示例：以下示例读取整数，但如果按下F1键则取消读取。

CANCEL FILE 声明

```
PROGRAM can_file_ex
%ENVIRONMENT FLBT
%ENVIRONMENT UIF
%NOLOCKGROUP
  VAR
    int_var: INTEGER
  ROUTINE cancel_read
    BEGIN
      CANCEL FILE TPDISPLAY
    END cancel_read
  BEGIN
    CONDITION[1]:
      WHEN TPIN[ky_f1]+ DO
        cancel_read
      ENABLE CONDITION[1]
    ENDCONDITION

    ENABLE CONDITION[1]
    REPEAT
      -- Read an integer, but cancel if F1 pressed
      CLR_IO_STAT(TPDISPLAY)
      WRITE(CR, 'Enter an integer: ')
      READ(int_var)
    UNTIL FALSE
end can_file_ex
```

A.4.6 CHECK_DICT 内置程序

用途：检查指定的字典以查找指定的元素

语法：CHECK_DICT(dict_name, element_no, status)

Input/Output 参数：

[in] dict_name :STRING

[in] element_no :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *dict_name* 是要检查的字典的名称。
- *element_no* 是字典中的元素编号。
- *status* 说明了尝试操作的状态。 如果不等于0，则找不到该元素。

另请参见：ADD_DICT，READ_DICT，WRITE_DICT，REMOVE_DICT内置过程。 有关DISCTRL_LIST内置过程和第10章 字典和表单，请参阅程序示例

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.4.7 CHECK_EPOS 内置程序

用途：检查指定位置是否有效，并且移动到此位置时不会产生运动错误

语法：CHECK_EPOS (eposn, uframe, utool, status <, group_no>)

Input/Output 参数：

[in] eposn :XYZWPREXT

[in] uframe :POSITION

[in] utool :POSITION

[out] status :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *eposn* 是要检查的 XYZWPREXT 位置。
- *uframe* 指定与*eposn*一起使用的*uframe*位置。
- *utool* 指定与*eposn*一起使用的*utool*位置。
- 状态说明了检查的状态。 如果位置可达，则状态将为0。
- *group_no* 是可选的，但如果指定则是eposn的组号。 如果未指定，则使用程序的默认组。

另请参见：GET_POS_FRM

示例：请参阅第B.8节（GEN_HEX.KL）

A.4.8 CHECK_NAME 内置程序

用途：检查指定的文件或程序名称是否存在非法字符。

语法：CHECK_NAME (name_spec, status)

Input/Output 参数：

[in] name_spec :STRING

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *name_spec* 指定要检查非法字符的字符串。 该字符串可以是文件名或程序名。 它不应包括文件或程序的扩展名。 此内置函数不处理特殊系统名称，如* SYSTEM *。

A.4.9 CHR 内置函数

用途：返回与数字代码对应的字符

语法：CHR (code)

Function Return Type :STRING

Input/Output 参数：

[in] code :INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- *code* 表示ASCII，Graphic或Multinational字符集的字符的数字代码。
- 返回分配了代码值的单个字符串。

另请参阅：附录D，“ASCII字符代码”

示例：有关详细的程序示例，请参阅以下部分：

第B.4节“标准例程”（ROUT_EX.KL）

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.4.10 CLEAR 内置程序

用途: 从内存中清除指定的程序和/或变量

语法：CLEAR(file_spec, status)

Input/Output 参数：

[in] file_spec :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *file_spec* 指定要清除的程序名称和数据类型。 以下类型有效：

  no ext：KAREL或Teach Pendant程序和变量.TP：Teach Pendant program.PC：KAREL program.VR：KAREL变量
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

示例：以下示例清除KAREL程序，清除程序的变量，并清除示教程序。

CLEAR 内置程序

```
-- Clear KAREL program
  CLEAR('test1.pc', status)
  -- Clear KAREL variables
  CLEAR('testvars.vr', status)
```

```
-- Clear Teach Pendant program
CLEAR('prg1.tp', status)
```

A.4.11 CLEAR_SEMA 内置程序

用途：通过将计数设置为零来清除指示的信号量

语法：CLEAR_SEMA(semaphore_no)

Input/Output 参数：

[in] semaphore_no :INTEGER

%ENVIRONMENT Group :MULTI

细节：

- *semaphore_no* 指示的信号量被清除。
- *semaphore_no* 必须在1到控制器上定义的信号量数量的范围内。
- 所有信号量都在COLD启动时清除。 在使用信号量之前清除信号量是一种好习惯。 在几个任务开始共享一个信号量之前，这些任务中只有一个应该清除信号量。

另请参阅：POST_SEMA，PEND_SEMA内置过程，SEMA_COUNT内置函数，第14章“多任务处理”中的示例

A.4.12 CLOSE FILE 声明

用途：断开FILE变量与数据文件或通信端口之间的关联

语法：CLOSE FILE file_var

其中：

file_var：一个 FILE 变量

细节：

- *file_var* 必须是OPEN FILE语句中使用的静态变量。
- 与 *file_var* 关联的任何缓冲数据都将写入文件或端口。
- 内置函数IO_STATUS将始终返回零。

另请参阅：IO_STATUS内置函数，第7章 文件输入/输出操作，附录E，''语法图'，以获取其他语法信息

示例：有关详细的程序示例，请参见第B.12节 “从字典文件显示列表”（DCLST_EX.KL）。

A.4.13 CLOSE HAND 声明

用途：使指定的 HAND 关闭

语法：CLOSE HAND hand_num

其中：

hand_num : INTEGER 表达式

细节：

- 声明的实际效果取决于I / O系统中HAND信号的设置方式。
- *hand_num* 的有效值范围是1-2。 否则，程序将中止并显示错误。
- 如果*hand_num*的值在范围内但@hand未连接，则该语句无效。
- 如果*hand_num*的值在范围内但是尚未分配由该值表示的HAND信号，则程序将中止并显示错误。

另请参阅：第14章 输入/输出系统，有关HAND信号的更多信息，请参见附录E，''语法图'，以获取其他语法信息

示例：以下示例将机器人移动到第一个位置并关闭*hand_num*指定的指针。

CLOSE HAND 声明

```
SET_POS_REG(1, p1, status)
move_pr - 调用TP程序进行移动
CLOSE HAND hand_num
```

A.4.14 CLOSE_TPE 内置程序

用途：关闭指定的示教程序

语法：CLOSE_TPE(open_id, status)

Input/Output 参数：

[in] open_id :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 表示要关闭的示教器程序。 所有打开的挂件程序必须先关闭才能执行。 任何未关闭的程序将保持打开状态，直到打开它的KAREL程序中止或运行完成。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：OPEN_TPE内置过程

示例：有关详细的程序示例，请参见第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）。

A.4.15 CLR_IO_STAT 内置程序

用途：清除文件参数上最后一次操作的结果

语法：CLR_IO_STAT(file_id)

Input/Output 参数：

[in] file_id :FILE

%ENVIRONMENT Group :PBCORE

细节：

- 导致 IO_STATUS 返回的 *file_id* 上的最后一个操作结果被清除为零。

另请参阅：I/O-STATUS内置函数

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.4.16 CLR_PORT_SIM 内置程序

用途：将指定的端口设置为未模拟

语法：CLR_PORT_SIM(port_type, port_no, status)

Input/Output 参数 :

[in] port_type :INTEGER

[in] port_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :iosetup

细节 :

- *port_type* 指定要取消模拟的端口类型的代码。 代码在FR中定义：KLIOTYPS.KL。
- *port_no* 指定要取消模拟的端口号。
- 如果参数有效并且清除了指定端口的模拟，则返回 *status* 为零。

另请参见：GET_PORT_SIM，SET_PORT_SIM内置过程

A.4.17 CLR_POS_REG 内置程序

用途：删除指定位置寄存器中指定组的所有数据

语法：CLR_POS_REG(register_no, group_no, status)

Input/Output 参数 :

[in] register_no :INTEGER

[in] group_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :REGOPE

细节 :

- *register_no* 指定应清除其数据的寄存器号。
- 如果 *group_no* 为零，则清除所有组的数据。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：SET_POS_REG内置过程，GET_POS_REG内置函数

示例：以下示例清除前100个位置寄存器。

CLR_POS_REG 内置程序

```
FOR register_no = 1 to 100 DO
    CLR_POS_REG(register_no, 0, status)
  ENDFOR
```

A.4.18 %CMOSVARS 译者指令

用途：指定KAREL变量的默认存储是永久内存。

语法：%CMOSVARS

细节：

- 如果在程序中指定了 %CMOSVARS，则默认情况下将在永久内存中创建所有静态变量。
- 如果未指定 %CMOSVARS，则默认情况下将在临时内存中创建所有静态变量。
- 如果程序指定 %CMOSVARS，但并非所有静态变量都需要在永久存储器中创建，则IN DRAM子句可用于所选变量。

另见：第A.10.2节 “DRAM条款”

示例：有关详细的程序示例，请参阅以下部分：第B.10节 “使用动态显示内置”（DCLST_EX.KL）

第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）

A.4.19 %CMOS2SHADOW 译者指令

用途：指示翻译器将所有CMOS变量放入影子存储器中。

语法：%CMOS2SHADOW

A.4.20 CNC_DYN_DISB 内置程序

用途：根据指定窗口中 BOOLEAN 变量的值取消动态显示。

语法：CNC_DYN_DISB (b_var, window_name, status)

Input/Output 参数：

[in] b_var :BOOLEAN

[in] window_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *b_var* 是布尔变量，其动态显示将被取消。
- *window_name* 必须是先前定义的窗口名称。 有关预定义的窗口名称，请参见第7.10.1节和第7.10.2节。
- 如果此窗口中此变量有多个活动显示，则取消所有显示。
- 如果没有动态显示激活指定此变量和窗口，则*status*返回错误。 如果不等于0，则发生错误。

另请参阅：INI_DYN_DISB内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.4.21 CNC_DYN_DISE 内置程序

用途：根据指定窗口中 INTEGER 变量的值取消动态显示。

语法：CNC_DYN_DISe (e_var, window_name, status)

Input/Output 参数：

[in] e_var :INTEGER

[in] window_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *e_var* 是要取消动态显示的整数变量。
- 有关上面列出的其他参数的说明，请参阅CNC_DYN_DISB内置过程。

另请参阅：INI_DYN_DISE内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.4.22 CNC_DYN_DISI 内置程序

用途：取消在指定窗口中动态显示 INTEGER 变量。

语法：CNC_DYN_DISI(int_var, window_name, status)

Input/Output 参数：

[in] int_var :INTEGER

[in] window_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *int_var* 是要取消动态显示的整数变量。
- 有关上面列出的其他参数的说明，请参阅CNC_DYN_DISB内置过程。

另请参见：INI_DYN_DISI内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.4.23 CNC_DYN_DISP 内置程序

用途：根据指定窗口中端口的值取消动态显示。

语法：CNC_DYN_DISP(port_type, port_no, window_name, status)

Input/Output 参数：

[in] port_type :INTEGER

[in] port_no :INTEGER

[in] window_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *port_type* 和 *port_no* 是整数值，指定要取消动态显示的端口。
- 有关上面列出的其他参数的说明，请参阅CNC_DYN_DISB内置过程。

另请参阅：INI_DYN_DISP内置过程以获取有关 *port_type* 代码的信息。

示例：有关详细的程序示例，请参阅以下部分：

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.4.24 CNC_DYN_DISR 内置程序

用途：取消在指定窗口中动态显示 REAL 数字变量。

语法：CNC_DYN_DISR(real_var, window_name, status)

Input/Output 参数：

[in] real_var :REAL

[in] window_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *real_var* 是REAL变量，其动态显示将被取消。
- 有关上面列出的其他参数的说明，请参阅CNC_DYN_DISB内置过程。

另请参见：INI_DYN_DISR内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.4.25 CNC_DYN_DISS 内置程序

用途：取消在指定窗口中动态显示STRING变量。

语法：CNC_DYN_DISS(str_var, window_name, status)

Input/Output 参数：

[in] str_var :STRING

[in] window_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *str_var* 是STRING变量，其动态显示将被取消。
- 有关上面列出的其他参数的说明，请参阅CNC_DYN_DISB内置过程。

另请参见：INI_DYN_DISS内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.4.26 CNCL_STP_MTN 内置程序

用途：取消所有停止的动作

语法：CNCL_STP_MTN

%ENVIRONMENT Group :motn

- 对于程序控制的所有组，所有停止的动作都将被取消。
- 动议陈述后的陈述将被执行。
- 如果当前没有动作停止，CNCL_STP_MTN将无效。
- 对于其他任务，无法取消运动。

示例：以下示例将取消紧急停止发生后程序控制的所有组的所有停止运动。

CNCL_STP_MTN 内置程序

```
ROUTINE e_stop_hndlr
  BEGIN
  CNCL_STP_MTN
END e_stop_hndlr
CONDITION[100]:
  WHEN ERROR[estop] DO
    UNPAUSE
    ENABLE CONDITION[100]
    e_stop_hndlr
END CONDITION
ENABLE CONDITION[100]
```

A.4.27 CNV_CNF_STRG 内置程序

用途：使用可选的group_no将指定的CONFIG转换为STRING

语法：CNV_CNF_STRG(source, target <,group_no>)

Input/Output 参数：

[in] source :CONFIG

[out] target :STRING

[in] group_no :INTEGER

%ENVIRONMENT Group :STRNG

细节：

- *target* 接收source指定的配置的 STRING 形式。
- *target* 必须足够长，以接受连接到控制器的机器人手臂的有效配置字符串。 否则，程序将因错误而中止。
- 使用25的长度通常是足够的，因为任何机器人的最长配置字符串是25个字符长。
- 转换期间使用 *group_no* 的配置掩码。 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了*group_no*，则它必须在1到指定的组总数的范围内
- 控制器。

另请参阅：CNV_STR_CONF内置过程

示例：以下示例将配置从posn位置转换为STRING，并使用组2中的配置掩码将其置于 *config_string* 中。然后，该字符串将显示在屏幕上。

CNV_CNF_STRG 内置程序

```
CNV_CNF_STRG(posn.pos_config, config_string, 2)
 WRITE('Configuration of posn in group 2: ', config_string, cr)
```

A.4.28 CNV_CONF_STR 内置程序

用途：将指定的 CONFIG 转换为 STRING

语法：CNV_CONF_STR(source, target)

Input/Output 参数：

[in] source :CONFIG

[out] target :STRING

%ENVIRONMENT Group :STRNG

细节：

- *target* 接收 source 指定的配置的 STRING 形式。

- *target* 必须足够长，以接受连接到控制器的机器人手臂的有效配置字符串。 否则，程序将因错误而中止。

  使用25的长度通常是足够的，因为任何机器人的最长配置字符串是25个字符长。

另请参阅：CNV_STR_CONF内置过程

示例：以下示例将配置从posn位置转换为STRING并将其放入config_string中。 然后该字符串显示在屏幕上。

CNV_CONF_STR 内置程序

```
CNV_CONF_STR(posn.pos_config, config_string)
 WRITE('Configuration of posn: ', config_string, cr)
```

A.4.29 CNV_INT_STR 内置程序

用途：将指定的 INTEGER 格式化为 STRING

语法：CNV_INT_STR(source, length, base, target)

Input/Output 参数：

[in] source :INTEGER expression

[in] length :INTEGER expression

[in] base :INTEGER expression

[out] target :STRING expression

%ENVIRONMENT Group :PBCORE

细节：

- *source* 是要格式化为STRING的INTEGER。
- *length* 指定目标的最小长度。 如果需要包含源的内容和至少一个前导空白，则目标的实际长度可以更大。
- *base* 表示要表示号码的号码系统。 *base*必须在2-16或0（零）范围内，表示基数10。
- 如果*length*或*base*的值无效，则返回未初始化的目标。
- 如果目标未被声明为足够长以包含源和至少一个前导空白，则返回一个空白，其余的声明长度用“*”填充。

另请参阅：CNV_STR_INT内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.7节 “列出文件和程序以及操作字符串”（LIST_EX.KL）

A.4.30 CNV_JPOS_REL 内置程序

用途：允许KAREL程序将各个关节角度检查为REAL值

语法：CNV_JPOS_REL(jointpos, real_array, status)

Input/Output 参数：

[in] joint_pos :JOINTPOS

[out] real_array :ARRAY [num_joints] OF REAL

[out] status :INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- *joint_pos* 是KAREL关节位置数据类型之一：JOINTPOS，或JOINTPOS1到JOINTPOS9。
- *num_joints* 可以小于系统中的关节数。 如果实际的关节数未知，则可以使用值9。 第一个关节将存储在第一个 *real_array* 元素中，等等。将忽略多余的数组元素。
- *real_array* 元素的度量以度为单位。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：CNV_REL_JPOS内置过程

示例：有关详细的程序示例，请参见第B.14节 “对复制的示教器程序应用偏移”（CPY_TP.KL）。

A.4.31 CNV_REAL_STR 内置程序

用途 ：将指定的REAL值格式化为STRING

语法：CNV_REAL_STR(source, length, num_digits, target)

Input/Output 参数：

[in] source :REAL expression

[in] length :INTEGER expression

[in] num_digits :INTEGER expression

[out] target :STRING

%ENVIRONMENT Group :STRNG

细节：

- *source* 是要格式化的REAL值。
- *length* 指定目标的最小长度。 如果需要包含源的内容和至少一个前导空白，则目标的实际长度可以更大。
- *num_digits* 指定小数点右侧显示的位数。 如果
- *num_digits* 是负数，源将以科学计数法格式化（其中ABS（*num_digits*）表示小数点右侧的位数。）如果*num_digits*为0，则小数点被抑制。
- 如果*length*或*num_digits*无效，则返回未初始化的目标。
- 如果声明的目标长度不足以包含带有一个前导空白的源，则返回带有一个前导空白的目标，其余的声明长度填充“*”（星号）。

另请参阅：CNV_STR_REAL 内置过程

示例：以下示例将*cur_volts*中的REAL数转换为STRING并将其放入*volt_string*。 cur_volts的最小长度指定为七个字符，小数点后面有两个字符。 然后，*volt_string*的内容显示在屏幕上。

CNV_REAL_STR 内置程序

```
cur_volts = AIN[2]
CNV_REAL_STR(cur_volts, 7, 2, volt_string)
WRITE('Voltage=',volt_string,CR)
```

A.4.32 CNV_REL_JPOS 内置程序

用途：允许KAREL程序操纵关节位置的各个角度

细节：CNV_REL_JPOS(real_array, joint_pos, status)

Input/Output 参数：

[in] real_array :ARRAY [num_joints] OF REAL

[out] joint_pos :JOINTPOS

[out] status :INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- *real_array* 必须具有声明的大小，等于或大于系统中的关节数。 如果实际的关节数未知，则可以将值9用于 *num_joints*。 第一个数组元素将存储在第一个关节中，依此类推。 多余的数组元素将被忽略。 如果数组不够大，程序将以无效的参数错误中止。
- 如果对应于关节角度的任何 *real_array* 元素未初始化，则程序将暂停一个未初始化的变量错误。
- *real_array* 元素的度量是度。
- *joint_pos* 是KAREL关节位置类型之一：JOINTPOS，或JOINTPOS1到JOINTPOS9。
- *joint_pos* 接收 *real_array* 的联合位置形式。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

示例：有关详细的程序示例，请参阅以下部分：

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.8节“沿着六边形路径生成和移动”（GEN_HEX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.4.33 CNV_STR_CONF 内置程序

用途：将指定的配置字符串转换为CONFIG数据类型

语法：CNV_STR_CONF(source, target, status)

Input/Output 参数：

[in] source :STRING

[out] target :CONFIG

[out] status :INTEGER

%ENVIRONMENT Group :STRNG

细节：

- *target* 接收 *source* 指定的配置字符串的 CONFIG 格式。
- *source* 必须是连接到控制器的机器人手臂的有效配置字符串。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：CNV_CONF_STR内置过程

示例：以下示例将位置*xyz_pos*的配置设置为config_string指定的配置，然后将TCP移动到该位置。

CNV_STR_CONF 内置程序

```
CNV_STR_CONF(config_string, xyz_pos.config_data, status)
SET_POS_REG(1, xyz_pos, status) -- Put  xyz_pos in PR[1]
move_to_pr -- Call TP program to move to PR[1]
```

A.4.34 CNV_STR_INT 内置程序

用途 ：将指定的STRING转换为INTEGER

语法：CNV_STR_INT(source, target)

Input/Output 参数：

[in] source :STRING

[out] target :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *source* 转换为 INTEGER 并存储在 *target* 中。
- 如果 *source* 不包含 INTEGER 的有效表示，则将 *target* 设置为未初始化。

另请参阅：CNV_INT_STR内置过程

示例：有关详细的程序示例，请参阅以下部分：第B.2节 “复制路径变量” （CPY_PTH.KL）

第B.12节 “从字典文件显示列表” （DCLST_EX.KL）

A.4.35 CNV_STR_REAL 内置程序

用途：将指定的STRING转换为REAL

语法：CNV_STR_REAL(source, target)

Input/Output 参数：

[in] source :STRING

[out] target :REAL

%ENVIRONMENT Group :PBCORE

细节：

- 将 *source* 转换为REAL数字并将结果存储在 *target* 中。
- 如果 *source* 不是REAL数的有效十进制表示，则 *target* 将被设置为未初始化。 *source* 可能包含nn.nnEsnn形式的科学记数法，其中s是+或 - 符号。

另请参阅：CNV_REAL_STR内置过程

示例：以下示例将STRING str转换为REAL并将其置于速率中。

CNV_STR_REAL 内置程序

```
REPEAT
 WRITE('Enter rate:')
 READ(str)
 CNV_STR_REAL(str, rate)
UNTIL NOT UNINIT(rate)
```

A.4.36 CNV_STR_TIME 内置程序

用途：将时间的字符串表示形式转换为时间的整数表示形式。

语法：CNV_STR_TIME(source, target)

Input/Output 参数：

[in] source :STRING

[out] target :INTEGER

%ENVIRONMENT Group :TIM

细节：

- 字符串参数 *source* 的大小是STRING [20]。
- 必须使用''DD-MMM-YYY HH：MM：SS''格式输入来源。 秒说明符''SS，''是可选的。 如果未指定秒，则使用值零（0）。 如果 *source* 无效，则 *target* 将设置为0。
- *target* 可与SET_TIME内置过程一起使用，以重置系统上的时间。 如果 *target* 为0，则不会更改系统上的时间。

另请参见：SET_TIME内置过程

示例：以下示例使用CNV_STR_TIME过程将用户以“DD-MMM-YYY HH：MM：SS”格式输入的STRING变量str_time转换为时间int_time的INTEGER表示形式。 然后使用SET_TIME将KAREL系统内的时间设置为int_time指定的时间。

CNV_STR_TIME 内置程序

```
WRITE('Enter the new time : ')
READ(str_time)
CNV_STR_TIME(str_time,int_time)
SET_TIME(int_time)
```

A.4.37 CNV_TIME_STR 内置程序

用途 ：将 INTEGER 时间表示转换为 STRING

语法：CNV_TIME_STR(source, target)

Input/Output 参数：

[in] source :INTEGER

[out] target :STRING

%ENVIRONMENT Group :TIM

细节：

- GET_TIME 内置过程用于确定 INTEGER 时间表示。 CNV_TIME_STR 用于将源转换为目标，它将以“DD-MMM-YYY HH：MM：”格式显示。

另请参阅：GET_TIME内置过程

示例：有关详细的程序示例，请参见第B.9节“使用文件和设备内置函数”（FILE_EX.KL）。

A.4.38 %COMMENT 译者指令

用途：指定最多16个字符的注释

语法：%COMMENT = ’sssssssssssssss’

其中 sssssssssssssss = space

细节：

- 注释最长可达16个字符。
- 加载时间内，注释将作为程序属性存储，并可显示在示教器或CRT / KB上。
- 在 PROGRAM语句 之后，但在任何 CONST，TYPE 或 VAR 部分之前，必须使用 %COMMENT。

另请参阅：SET_ATTR_PRG和GET_ATTR_PRG内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.3节“将数据保存到默认设备”（SAVE_VR.KL）

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.9节“使用文件和设备内置函数”（FILE_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.4.39 COMPARE_FILE 内置程序

用途：将一个文件的内容与另一个文件进行比较

语法：COMPARE_FILE(filea, fileb, result_file, ascii_flag, diff_count, status)

Input/Output 参数：

[in] filea :STRING

[in] fileb :STRING

[out] result_file :FILE

[in] ascii_flag:BOOLEAN

[out] diff_count :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *filea* 指定要比较的文件的设备，名称和类型。 如果未指定设备，则使用默认设备。 您必须同时指定名称和类型。
- *fileb* 指定要比较的文件的设备，名称和类型。 如果未指定设备，则使用默认设备。 您必须同时指定名称和类型。
- *result_file* 这必须是一个打开的FILE变量，用于写入。 比较结果将写入此文件。
- *ascii_flag* 指定应根据ASCII（文本）内容比较文件。
- *diff_count* 有多少ASCII（文本）行不同。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见：COPY_FILE，RENAME_FILE，DELETE_FILE内置过程
以下示例说明了自从将参考副本保存到MC：\ ref后，如何使用此内置程序来检测是否有任何TP程序已更改。 要尝试将md：*。ls复制到mc：\ ref（或您选择的其他文件存储区域），然后运行此程序。

图A-1. COMPARE_FILE 内置过程的示例

```
------------------------------------------------------------
PROGRAM compfiles
------------------------------------------------------------
VAR
status: integer
prog_name: string[128]
logfile: file
file_ary: array [1638] of string[MAX_PROG_NAM]
```

```
tempstore in CMOS: string[40]
index: integer
findex: integer
n_progs: integer
diff_count: integer


BEGIN
  prog_name = '*'
  index = 0
  n_progs = 1
  diff_count=1
  IF uninit(tempstore) THEN
    tempstore = 'MC:\ref'
  ENDIF

  OPEN FILE logfile ('RW', 'CMPFILES.LOG')
  PROG_LIST(prog_name, 2, 0, 1, file_ary, n_progs, index)
  FOR index = 1 TO n_progs DO

  -- COMPARE_FILE( filename1: string;   (IN)  用于比较的第一个文件名
  --               filename2: string;   (IN)  用于比较的第二个文件名
  --               out_file: file;      (OUT) 打开KAREL文件以进行输出
  --               ascii_flag: boolean; (IN)  如果为TRUE, 则比较为ASCII
  --               count: integer;      (OUT) ASCII-差异数二进制第一个不同的字节
  --
  --               status: integer)     (OUT) 结果如无法打开文件

    COMPARE_FILE('MD:\' + file_ary[index]+'.LS',
                 tempstore + '\' + file_ary[index]+'.LS', logfile, TRUE,
                 diff_count, status)
    write logfile ('Compared ' + file_ary[index], ' ', status, ' ',
                   diff_count, ' ', index, CR)
  ENDFOR
  CLOSE FILE logfile
END compfiles
```

A.4.40 CONDITION...ENDCONDITION 声明

用途：定义全局条件处理程序

语法：CONDITION[cond_hand_no]: [with_list]

WHEN cond_list DO action_list

{WHEN cond_list DO action_list}

ENDCONDITION

细节：

- *cond_hand_no* 指定与条件处理程序关联的数字，且必须在1-1000的范围内。如果程序超出此范围，程序将中止并显示错误。
- 如果已存在具有指定编号的条件处理程序，则旧的条件处理程序将替换为新的条件处理程序。
- 可选的[with_list]可用于指定条件处理程序限定符。有关更多信息，请参阅WITH子句。
- 必须同时满足单个WHEN子句中列出的所有条件才能触发条件处理程序。
- 必须由AND运算符或OR运算符分隔多个条件。不允许混合使用AND和OR。
- 当同时满足WHEN子句的相应条件时，将采取DO之后列出的动作。
- 多个动作由逗号分隔或在新行上分隔。
- 在CONDITION语句中不允许调用函数例程。
- 条件处理程序最初被禁用，并在触发时再次禁用。使用ENABLE语句或操作（指定条件处理程序编号）来启用它。
- 使用DISABLE语句或操作来停用条件处理程序。
- 条件处理程序仍然定义，随后可以通过ENABLE语句或操作重新激活。
- PURGE语句可用于删除条件处理程序的定义。
- 条件处理程序仅对定义它们的任务是已知的。两个不同的任务可以使用相同的 *cond_hand_no*，即使它们指定了不同的条件。

另请参阅：第6章 条件处理程序，附录E，''语法图'，'了解其他语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.4.41 CONFIG 数据类型

用途：将变量或结构字段定义为CONFIG数据类型

语法：CONFIG

细节：

- CONFIG 将变量或结构字段定义为由定义机器人配置的字段组成的高度紧凑的结构。
- CONFIG 包含以下预定义字段：
  - CFG_TURN_NO1：INTEGER
  - CFG_TURN_NO2：INTEGER
  - CFG_TURN_NO3：INTEGER
  - CFG_FLIP：BOOLEAN
  - CFG_LEFT：BOOLEAN
  - CFG_UP：BOOLEAN
  - CFG_FRONT：BOOLEAN
- 变量和结构字段可以声明为CONFIG。
- 可以使用通常的结构字段表示法访问和设置CONFIG数据类型的子字段。
- 声明为CONFIG的变量和字段可以
  - 彼此分配。
  - 作为参数传递。
  - 写入和读取未格式化的文件。
- CONFIG变量或结构字段的每个子字段都可以作为参数传递给例程，但始终按值传递。
- CONFIG字段是每个POSITION和XYZWPR变量和字段的一部分。
- 尝试将值分配给对于字段来说太大的CONFIG子字段会导致中止错误。

示例：以下示例显示如何使用通常的structure.field表示法访问和设置CONFIG结构的子字段。

CONFIG 数据类型

```
VAR
 config_var1,  config_var2:  CONFIG
 pos_var:  POSITION
 seam_path:  PATH
```

```
 i:  INTEGER
BEGIN
 config_var1 = pos_var.config_data
 config_var1 = config_var2
 config_var1.cfg_turn_no1 = 0
 IF pos_var.config_data.cfg_flip THEN...
 FOR i = 1 TO PATH_LEN(seam_path) DO
 seam_path[i].node_pos.config_data = config_var1
 ENDFOR
```

A.4.42 CONNECT TIMER 声明

用途：导致INTEGER变量开始更新为毫秒时钟

语法：CONNECT TIMER TO clock_var

其中：

clock_var：静态的，用户定义的INTEGER变量

细节：

- 只要程序正在运行或暂停，*clock_var* 目前会增加系统变量 $ SCR.$ COND_TIME 的值每 $ SCR.$ COND_TIME 毫秒
- 并继续，直到程序断开计时器，结束或中止。例如，如果
- $ SCR.$ COND_TIM E = 32 然后 *clock_var* 将每32毫秒递增32。
- 您应该在使用CONNECT TIMER语句之前初始化 *clock_var* 以确保正确的起始值。
- 如果变量未初始化，它将保持一段短时间（最多32毫秒），然后将设置为一个非常大的负值（-2.0E31 + 32毫秒）并从该值递增。
- 程序可以在连接时将 *clock_var* 重置为任何值。
- 初始化为零的 *clock_var* 在大约23天后从大约20亿到大约减去20亿。
- 如果 *clock_var* 是例程中的系统变量或局部变量，则无法转换程序。
- 注意如果两个使用相同变量的CONNECT TIMER语句在两个不同的任务中执行，则计时器将提前两倍。例如，计时器将每$ SCR增加2 * $ SCR.$ COND_TIME.$ COND_TIME ms。但是，如果在同一任务中执行使用相同变量的两个或多个CONNECT TIMER语句，则不会发生这种情况。

另请参阅：附录E 了解其他语法信息，DISCONNECT TIMER语句

示例：有关详细的程序示例，请参阅以下部分：

第B.8节“沿六边形路径生成和移动”（GEN_HEX.KL）

第B.1节“设置用于监视的数字输出端口”（DOUT_EX.KL）

A.4.43 CONTINUE 动作

用途：继续执行暂停的任务

语法：CONTINUE <PROGRAM[n]>

细节：

- CONTINUE操作不会恢复停止的动作。
- 如果暂停程序执行，CONTINUE操作将继续执行程序。
- CONTINUE操作之后可以是PROGRAM [n]子句，其中n是要继续的任务编号。使用GET_TSK_INFO获取指定任务名称的任务编号。
- 执行CONTINUE时，任务可以处于中断例程中。但是，您应该了解以下情况，因为CONTINUE仅影响当前的中断级别，并且任务的中断级别可能会独立暂停或运行。
  - 如果中断例程和任务都暂停，CONTINUE将继续中断例程，但任务将保持暂停状态。
  - 如果中断例程正在运行并且任务暂停，则CONTINUE似乎无效，因为它将尝试继续运行中断例程。
  - 如果中断例程暂停且任务正在运行，则CONTINUE将继续执行中断例程。

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.4.44 CONTINUE 条件

用途：程序执行继续时满足的条件

语法：CONTINUE <PROGRAM[n]>

细节：

- CONTINUE 条件监视程序执行。
- 如果程序执行暂停，则从CRT/KB发出 CONTINUE 或从操作面板发出 CYCLE START 的 CONTINUE 操作将继续执行程序并满足 CONTINUE 条件。

- CONTINUE 条件后面可以跟PROGRAM [n]，其中n是要继续的任务编号。 使用 GET_TSK_INFO 获取指定任务名称的任务编号。

示例：在以下示例中，正在监视程序执行。 程序继续时，将打开数字输出。

CONTINUE 条件

```
CONDITION[1]:
  WHEN CONTINUE DO DOUT[1] = ON
ENDCONDITION
```

A.4.45 CONT_TASK 内置程序

用途：继续指定的任务

语法：CONT_TASK(task_name, status)

Input/Output 参数：

[in] task_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :MULTI

细节：

- *task_name* 是要继续的任务的名称。 如果任务未暂停，则状态将返回错误。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。
- 执行CONT_TASK时，任务可以处于中断例程中。 但是，您应该注意以下情况，因为CONT_TASK仅影响当前的中断级别，并且任务的中断级别可能会独立暂停或运行。
  - 如果中断例程和任务都暂停，CONT_TASK将继续中断例程，但任务将保持暂停状态。
  - 如果中断例程正在运行并且任务暂停，则CONT_TASK似乎无效，因为它将尝试继续运行中断例程。
  - 如果中断例程暂停且任务正在运行，CONT_TASK将继续中断例程。

另请参阅：RUN_TASK，ABORT_TASK，PAUSE_TASK内置程序，第15章 多任务

示例：以下示例提示用户输入任务名称并继续执行任务。 有关更多示例，请参阅第15章 多任务。

CONT_TASK 内置程序

```
PROGRAM cont_task_ex
   %ENVIRONMENT MULTI
     VAR
       task_str: STRING[12]
       status: INTEGER
     BEGIN
       WRITE('Enter task name to continue:')
       READ(task_str)
       CONT_TASK(task_str, status)
   END cont_task_ex
```

A.4.46 COPY_FILE 内置程序

用途：使用 overwrite 选项将一个文件的内容复制到另一个文件

语法：COPY_FILE(from_file, to_file, overwrite_sw, nowait_sw, status)

Input/Output 参数：

[in] from_file :STRING

[in] to_file :STRING

[in] overwrite_sw :BOOLEAN

[in] nowait_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *from_file* 指定要从中复制的文件的设备，名称和类型。 可以使用通配符（*）字符指定 *from_file*。 如果未指定设备，则使用默认设备。 您必须同时指定名称和类型。 但是，这些可以是通配符（*）字符。
- *to_file* 指定要复制的文件的设备，名称和类型。 可以使用通配符（*）字符指定 *to_file*。 如果未指定设备，则使用默认设备。

- *overwrite_sw* 指定文件是否应该被覆盖（如果存在）。
- 如果 *nowait_sw* 为TRUE，则在执行命令时继续执行程序。 如果为FALSE，程序将停止，包括条件处理程序，直到操作完成。 如果程序中有时间紧急的条件处理程序，则将它们放在另一个作为单独任务执行的程序中。
- 如果程序在复制期间中止，则复制将在中止之前完成。
- 如果要复制的设备在复制期间变满，则会返回错误。 注意 *nowait_sw* 在此版本中不可用，应设置为FALSE。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：RENAME_FILE，DELETE_FILE内置过程

示例：有关详细的程序示例，请参见第B.9节 “使用文件和设备内置函数”（FILE_EX.KL）。

A.4.47 COPY_PATH 内置程序

用途：将完整路径，路径的一部分或反向节点顺序（包括关联数据）的路径复制到另一个相同类型的路径变量。

语法：COPY_PATH (source_path, start_node, end_node, dest_path, status)

Input/Output 参数：

[in] source_path :PATH

[in] start_node :INTEGER

[in] end_node :INTEGER

[in] dest_path :PATH

[out] status :INTEGER

%ENVIRONMENT Group :pathop

细节：

- *source_path* 指定要从中复制的源路径。 此路径可以是标准路径或用户定义的路径。
- *start_node* 指定要复制的第一个节点的编号。 值为0将复制完整路径，包括标头信息。 *start_node* 编号必须介于0和源路径中的最高节点编号之间。 否则，将返回错误状态。

- *end_node* 指定要复制的最后一个节点的编号。 值为0将复制完整路径，包括标头信息。 *end_node* 编号必须介于0和源路径的最高节点编号之间。 否则，将返回错误状态。
- *dest_path* 指定要复制到的目标路径。 此路径可以是标准路径或用户定义的路径。 但是，*dest_path* 类型必须与 *source_path* 类型相同。 如果它们不相同，将返回错误状态。
- 如果参数有效且COPY_PATH操作成功，则返回状态0。 非零状态表示COPY_PATH操作不成功。

  注意要将完整路径从一个路径变量复制到另一个相同的路径变量，请将 *start_node* 和 *end_node* 参数设置为0（零）。

部分路径复制到目标路径的示例。

执行COPY_PATH（P1,2,5，P2）命令将节点2通过路径P1的节点5（包括）复制到路径P2的节点1到4，前提是P1的路径长度大于或等于5。 目标路径P2将成为4节点路径。 原始目标路径被完全覆盖。

源路径以与目标路径相反的顺序复制的示例。

执行COPY_PATH（P1,5,2，P2）命令将节点5通过路径P1的节点2（包括）复制到路径P2的节点1到4，前提是P1的路径长度大于或等于5。 目标路径P2将成为4节点路径。 原始目标路径被完全覆盖。

具体地，上述命令将P1的节点5复制到P2的节点1，将P1的节点4复制到P2的节点2，依此类推。

示例：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

A.4.48 COPY_QUEUE 内置程序

用途：将队列中的一个或多个连续条目复制到整数数组中。 这些条目不会被删除，而是被复制，从最旧的条目开始并继续到最新条目，或者直到输出数组或整数已满。 参数指定要跳过的列表头部（最旧的条目）的条目数。

语法：COPY_QUEUE(queue, queue_data, sequence_no, n_skip, out_data, n_got, status)

Input/Output 参数：

[in] queue_t :QUEUE_TYPE

[in] queue_data :ARRAY OF INTEGER

[in] n_skip :INTEGER

[in] sequence_no :integer

[out] out_data :ARRAY OF INTEGER

[out] n_got :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBQMGR

细节：

- *queue_t* 指定要从中读取值的队列的队列变量。
- *queue_data* 指定要从中读取值的队列的数组变量。
- *sequence_no* 指定要复制的最旧条目的序列号。 如果 *sequence_no* 为零，则复制的起始点由 *n_skip* 参数确定。
- *n_skip* 指定要跳过的最旧条目数。 值为零表示返回最旧的条目。
- *out_data* 是要复制值的整数数组; 数组的大小是返回的最大值。
- 返回 *n_got*，返回的条目数。 这将是以下之一：
  - Zero 如果队列中有 *n_skip* 或更少的条目。
  - （queue_to n_entries_skip）如果这小于ARRAY_LEN（out_data）
  - ARRAY_LEN（out_data）如果它小于或等于queue.n_entries - n_skip
- *status* 返回零

另请参阅：APPEND_QUEUE，DELETE_QUEUE，INSERT_QUEUE内置过程，第15.8节 “使用队列进行任务通信”

示例：以下示例获取作业队列的一个“页面”并调用例程*disp_queue*来显示它。 如果页面没有条目，则例程返回FALSE; 否则例程返回TRUE。

COPY_QUEUE 内置程序

```
PROGRAM copy_queue_x
%environment PBQMGR
VAR
  job_queue FROM global_vars: QUEUE_TYPE
  job_data FROM global_vars: ARRAY[100] OF INTEGER
ROUTINE disp_queue(data: ARRAY OF INTEGER;
                   n_disp: INTEGER) FROM disp_prog
ROUTINE disp_page(data_array: ARRAY OF INTEGER;
```

```
                    page_no: INTEGER): BOOLEAN
VAR
  status: INTEGER
  n_got: INTEGER
BEGIN
  COPY_QUEUE(job_queue, job_data,
             (page_no - 1) * ARRAY_LEN(data_array), 0,
             data_array, n_got, status)
  IF (n_got = 0) THEN
    RETURN (FALSE)
  ELSE
    disp_queue(data_array, n_got)
    RETURN (TRUE)
  ENDIF
END disp_page
BEGIN
END copy_queue_x
```

A.4.49 COPY_TPE 内置程序

用途：将一个示教器程序复制到另一个示教器程序。

语法：COPY_TPE(from_prog, to_prog, overwrite_sw, status)

Input/Output 参数：

[in] from_prog :STRING

[in] to_prog :STRING

[in] overwrite_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :TPE

细节：

- *from_prog* 指定要复制的教导程序名称，不带.tp扩展名。
- *to_prog* 指定新的示教程序名称，没有.tp扩展名，将 *from_prog* 复制到。
- *overwrite_sw*，如果设置为TRUE，将自动覆盖 *to_prog*（如果它已经存在且当前未被选中）。 如果设置为FALSE，则 *to_prog* 如果已存在则不会被覆盖。
- *status* 说明了尝试操作的状态。 如果不等于0，则不会发生复制。

另请参见：CREATE_TPE内置过程

示例：有关详细的程序示例，请参见第B.14节 “对复制的示教器程序应用偏移” （CPY_TP.KL）。

A.4.50 COS 内置函数

用途：返回REAL角度参数的REAL余弦值，以度为单位指定

语法：COS(angle) Function Return Type :REAL

Input/Output 参数：

[in] angle :REAL expression

%ENVIRONMENT Group :SYSTEM

细节：

- *angle* 是指定在 ±18000 度范围内的角度。 否则，程序将因错误而中止。

示例：有关详细的程序示例，请参见第B.8节 “沿六边形路径生成和移动” （GEN_HEX.KL）。

A.4.51 CR 输入/输出项目

用途：可以在READ或WRITE语句中用作数据项以指定回车符

语法：CR

细节：

- 当CR用作READ语句中的数据项时，它指定忽略当前输入行中的任何剩余数据。将从下一个输入行的开头读取下一个数据项。
- 当CR用作WRITE语句中的数据项时，它指定对同一文件的后续输出将显示在新行上。

另请参阅：附录E，''语法图'，以获取其他语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.3节“将数据保存到默认设备”（SAVE_VR.KL）

第B.4节“标准例程”（ROUT_EX.KL）

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.8节“沿着六边形路径生成和移动”（GEN_HEX.KL）

第B.9节“使用文件和设备内置函数”（FILE_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.4.52 CREATE_TPE 内置程序

用途：创建指定名称的示教程序

语法：CREATE_TPE(prog_name, prog_type, status)

Input/Output 参数：

[in] prog_name :STRING

[in] prog_type :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :TPE

细节：

- *prog_name* 指定要创建的程序的名称。
- *prog_type* 指定要创建的程序的类型。 以下常量对程序类型有效：PT_MNE_UNDEF：未定义子类型的TPE程序PT_MNE_JOB：TPE作业PT_MNE_PROC：TPE进程 PT_MNE_MACRO：TPE宏

- *status* 说明了尝试操作的状态。 如果它不等于0，则发生错误。 一些可能的错误如下：

  7015 存在指定的程序

  9030 程序名称为NULL

  9031 从程序名称顶部删除num

  9032 从程序名称中删除空格

  9036 内存不够

  9038 程序名称中的字符无效

- 创建程序以引用系统上的所有运动组。 创建程序时没有任何注释或任何其他程序属性。 创建程序后，SET_ATTR_PRG可用于指定程序属性。

另请参见：SET_ATTR_PRG 内置过程

A.4.53 CREATE_VAR 内置程序

用途：创建指定的KAREL变量

语法：CREATE_VAR(var_prog_nam, var_nam, typ_prog_nam, type_nam, group_num, inner_dim, mid_dim, outer_dim, status, <mem_pool>)

Input/Output 参数：

[in] var_prog_nam :STRING

[in] var_nam :STRING

[in] typ_prog_nam :STRING

[in] type_nam :STRING

[in] group_num :INTEGER

[in] inner_dim :INTEGER

[in] mid_dim :INTEGER

[in] outer_dim :INTEGER

[out] status :INTEGER

[in] mem_pool :INTEGER

%ENVIRONMENT Group :MEMO

细节：

- *var_prog_nam* 指定应在其中创建变量的程序名称。如果 *var_prog_nam* 为''，则使用默认值，即当前正在执行的程序的名称。
- *var_nam* 指定将创建的变量名称。
- 如果要将变量创建为用户定义的类型，则必须已在系统中创建用户定义的类型。*typ_prog_nam* 指定用户定义类型的程序名称。 如果 *typ_prog_nam* 为''，则使用默认值，即当前正在执行的程序的名称。
- *type_nam* 指定要创建的变量的类型名称。 以下类型名称有效：

  'ARRAY OF BYTE'

  'ARRAY OF SHORT'

  'BOOLEAN'

  'CAM_SETUP'

  'CONFIG'

  'FILE'

  'INTEGER'

  'JOINTPOS'

  'JOINTPOS1'

  'JOINTPOS2'

  'JOINTPOS3'

  'JOINTPOS4'

  'JOINTPOS5'

  'JOINTPOS6'

  'JOINTPOS7'

  'JOINTPOS8'

'JOINTPOS9'

'MODEL'

'POSITION'

'REAL'

'STRING[n]', 其中n是字符串长度; 如果未指定，则默认值为12。

'VECTOR'

'VIS_PROCESS'

'XYZWPR'

'XYZWPREXT'

任何其他类型名称都被视为用户定义的类型。

- *group_num* 指定要用于位置数据类型的组编号。
- *inner_dim* 指定最内层数组的尺寸。例如，对于ARRAY [10,20,30] OF INTEGER，*inner_dim* = 30。如果变量不是数组，则 *inner_dim* 应设置为0。
- *mid_dim* 指定中间数组的尺寸。例如，对于ARRAY [10,20,30] OF INTEGER，*mid_dim* = 20。如果变量不是2-D数组，则 *mid_dim* 应设置为0。
- *outer_dim* 指定最外层数组的尺寸。例如，对于ARRAY [10,20,30] OF INTEGER，*outer_dim* = 10。如果变量不是3-D数组，则 *outer_dim* 应设置为0。
- *status* 说明了尝试操作的状态。如果不等于0，则发生错误。
- *mem_pool* 是一个可选参数，它指定从中创建变量的内存池。如果未指定，则在DRAM中创建变量，该变量是临时存储器。必须在每次上电时重新创建DRAM变量，并且值始终重置为未初始化。
- 如果 *mem_pool* = -1，则该变量在CMOS RAM中创建，该RAM是永久存储器。

另请参阅：CLEAR，RENAME_VAR内置过程

示例：有关详细的程序示例，请参见第B.2节"复制路径变量"（CPY_PTH.KL）。

A.4.54 %CRTDEVICE

用途：指定CRT / KB设备是默认设备

语法：%CRTDEVICE

细节：

- 指定 INPUT/OUTPUT 窗口将是READ和WRITE语句中的缺省值，而不是 TPDISPLAY 窗口。

示例：有关如何使用此设备的详细程序示例，请参见第B.2节 “复制路径变量”（CPY_PTH.KL）。

A.4.55 CURJPOS 内置函数

用途：返回指定轴组的刀具中心点（TCP）的当前关节位置，即使其中一个轴处于超程中

语法：CURJPOS(axs_lim_mask, ovr_trv_mask <,group_no>)

函数返回类型：JOINTPOS

Input/Output 参数：

[out] axs_lim_mask :INTEGER

[out] ovr_trv_mask :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。
- *axs_lim_mask* 指定哪些轴超出轴限制。
- *ovr_trv_mask* 指定哪些轴处于超行程中。
- **注意**此版本中不提供 *axis_limit_mask* 和 *ovr_trv_mask*，可以设置为0。

另请参见：CURPOS内置函数，第8章 位置数据

示例：以下示例获取机器人的当前关节位置。

CURJPOS 内置函数

```
PROGRAM getpos
VAR
 jnt: JOINTPOS
```

```
BEGIN
 jnt=CURJPOS(0,0)
END getpos
```

A.4.56 CURPOS 内置函数

用途：返回指定轴组的刀具中心点（TCP）的当前笛卡尔位置，即使其中一个轴处于超程中

语法：CURPOS(axis_limit_mask, ovr_trv_mask <,group_no>)

函数返回类型：XYZWPREXT

Input/Output 参数：

[out] axis_limit_mask :INTEGER

[out] ovr_trv_mask :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。
- 该组必须是运动学的。
- 返回工具中心点（TCP）相对于指定组的系统变量$ UFRAME的当前值的当前位置。
- *axis_limit_mask* 指定哪些轴超出轴限制。
- *ovr_trv_mask* 指定哪些轴处于超行程中。
- 注意此版本中不提供 *axis_limit_mask* 和 *ovr_trv_mask*，如果已设置，将忽略它。

另请参见：第8章 位置数据

示例：有关详细的程序示例，请参见第B.5节“使用寄存器内置函数”。

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.4.57 CURR_PROG 内置功能

用途：返回当前正在执行的程序的名称。

语法：CURR_PROG

函数返回类型：STRING[12]

%ENVIRONMENT Group :BYNAM

细节：

- 分配给 CURR_PROG 的变量必须使用字符串变量长度≥12声明

示例：有关详细的程序示例，请参见第B.2节 “复制路径变量”（CPY_PTH.KL）。

A.5 - D - KAREL 语言描述

A.5.1 DAQ_CHECKP 内置程序

用途：检查管道的状态以及可从管道读取的字节数。

语法：DAQ_CHECKP(pipe_num, pipe_stat, bytes_avail)

Input/Output 参数：

[in] pipe_num :INTEGER

[out] pipe_stat :INTEGER

[out] bytes_avail :INTEGER

细节：

- *pipe_num* 是要检查的管道（1 - 5）的编号。
- *pipe_stat* 是返回的管道的状态。 状态是以下标志的组合：
  - DAQ_PIPREG 是管道注册时（值= 1）。
  - DAQ_ACTIVE 是管道处于活动状态，即已启动（值= 2）。
  - DAQ_CREATD 是创建管道的时间（值= 4）。
  - DAQ_SNAPSH 是管道处于快照模式（值= 8）。
  - DAQ_1STRD 是第一次读取管道时（值= 16）。

- DAQ_OVFLOW 是管道溢出的时候（值= 32）。
- DAQ_FLUSH 是刷新管道的时间（值= 64）。

• *bytes_avail* 是可从管道读取的字节数。

DAQ_CHECKP 内置程序

pipe_stat 返回的参数可以与AND进行AND运算

以上标志常量，用于确定管道是否已注册，是否处于活动状态等等。 例如，在写入之前必须检查管道是否处于活动状态。

DAQ_CHECKP 内置程序

在调用DAQ_CHECKP时，永远不会为写入管道的任务设置DAQ_OVFLOW标志。 此标志仅适用于从管道读取的任务。

另请参见：DAQ_WRITE内置。

示例：请参阅内置函数DAQ_WRITE中的DAQWRITE示例。 注意此内置仅在加载DAQ或数据监视器选项时可用。

A.5.2 DAQ_REGPIPE 内置程序

用途：注册管道以在KAREL中使用。

语法：DAQ_REGPIPE(pipe_num, mem_type, pipe_size, prog_name, var_name, pipe_name, stream_size, and status)

Input/Output 参数：

[in] pipe_num :INTEGER

[in] mem_type :INTEGER

[in] pipe_size :INTEGER

[in] prog_name :STRING

[in] var_name :STRING

[in] pipe_name :STRING

[in] stream_size :INTEGER

[out] status :INTEGER

细节：

- *pipe_num* 是要注册的管道（1-5）的编号。
- *mem_type* 允许您分配用于管道的内存。可以使用以下常量：
  - DAQ_DRAM 允许您分配DRAM内存。
  - DAQ_CMOS 允许您分配CMOS内存。
- *pipe_size* 是管道的大小，表示为它可以容纳的数据记录的数量。数据记录大小本身由 *var_name* 的数据类型确定。
- *prog_name* 是包含要用于写入管道的变量的程序的名称。如果作为空字符串传递，则使用当前程序的名称。
- *var_name* 是变量的名称，用于定义用于写入管道的数据类型。注册后，您可以将此数据类型的任何变量写入管道。
- *pipe_name* 是管道文件的名称。例如，如果管道名称作为'foo.dat'传递，则可以使用文件字符串'PIP：FOO.DAT'访问管道。即使管道仅用于发送到PC，也需要具有扩展名的唯一文件名。
- *stream_size* 是自动流式传输到输出文件的记录数（如果管道作为流式管道启动）。对指定变量的单次写入构成管道中的单个记录。如果流大小设置为零，则管道不会自动将记录流式传输到文件设备;所有数据都将保存在管道中，直到读取管道。使用 *stream_size* 在管道用于向PC发送数据时帮助优化网络负载。如果它为零或一，监视任务将在管道中看到每个数据记录后立即发送。如果数字是两个或更多，监视器将等待管道中有那么多数据记录，然后再将它们全部发送到PC。以这种方式，可以最小化发送网络分组的开销。数据不会在管道中停留的时间长于FRCPipe.StartMonitor方法提供的FlushTime参数指定的时间。
- *status* 是尝试操作的状态。如果不为0，则发生错误并且未注册管道。

另请参见：DAQ_UNREG内置。

DAQ_REGPIPE 内置程序

管道必须先注册才能启动并写入数据。 注册操作告诉系统如何在使用时配置管道。 注册后，管道配置为接受每条记录写入一定数量的数据，由指定变量的大小决定。

要更改管道的配置，必须首先使用DAQ_UNREG取消注册管道，然后重新注册。

示例：以下示例注册KAREL管道1以在程序中写入变量。

DAQ_REGPIPE 内置程序

```
PROGRAM DAQREG
%ENVIRONMENT DAQ
CONST
  er_abort = 2
VAR
  status:  INTEGER
  datavar: INTEGER
BEGIN
  -- 将管道1 DRAM注册为kldaq.dat
  -- 它可以在管道溢出之前保存100份datavar变量

  DAQ_REGPIPE(1, DAQ_DRAM, 100, '', 'datavar', &
              'kldaq.dat', 0, status)
  IF status<>0 THEN
    POST_ERR(status,' ',0,er_abort)
  ENDIF
END DAQREG
```

注意 此内置仅在加载DAQ或数据监视器选项时可用。

A.5.3 DAQ_START 内置程序

用途：激活KAREL管道进行写入。

语法：DAQ_START(pipe_num, pipe_mode, stream_dev, status)

Input/Output 参数：

[in] pipe_num :INTEGER

[in] pipe_mode :INTEGER

[in] stream_dev :STRING

[out] status :INTEGER

细节：

- *pipe_num* 是要启动的管道（1 - 5）的编号。 管道必须先前已注册
- *pipe_mode* 是用于管道的输出模式。 使用以下常量：
  - DAQ_SNAPSHT 是快照模式（每次读取管道都会产生所有管道的内容）。
  - DAQ_STREAM 是流模式（每次从同一个管道文件读取将导致自上次读取以来写入的数据）。
- *stream_dev* 是将自动流式传输记录的设备。 如果在注册期间将流大小设置为0，则忽略此参数。
- *status* 是尝试操作的状态。 如果不为0，则发生错误，管道未注销。

另请参阅：内置DAQ_REGPIPE和内置DAQ_STOP，

DAQ_START 内置程序

此内置调用可以来自与写入任务相同的任务/程序，也可以来自单独的激活/取消激活任务。写入任务可以处于休眠状态，直到管道启动，此时它开始写入数据。

DAQ_START 内置程序

PC应用程序发出FRCPipe时会自动启动管道。 StartMonitor方法。 在这种情况下，KAREL应用程序无需调用DAQ_START来激活管道。

DAQ_START 内置程序

使用引用计数方案跟踪管道的启动和停止。 也就是说，两个DAQ_START和FRCPipe.StartMonitor调用的任意组合都需要任何梳理

示例：以下示例以流模式启动KAREL管道1。

DAQ_START 内置程序

```
PROGRAM PIPONOFF
%ENVIRONMENT DAQ
CONST
  er_abort = 2
VAR
```

```
  status:  INTEGER
  tpinput: STRING[1]
BEGIN
  -- 提示打开管道
  WRITE('Press 1 to start pipe')
  READ (tpinput)
  IF tpinput = '1' THEN
    -- start pipe 1
    DAQ_START(1, DAQ_STREAM, 'RD:', status)
    IF status<>0 THEN
      POST_ERR(status,' ',0,er_abort)
    ELSE
      -- prompt to turn off pipe
      WRITE('Press any key to stop pipe')
      READ (tpinput)
      -- stop pipe 1
      DAQ_STOP(1, FALSE, status)
      IF status<>0 THEN
        POST_ERR(status,' ',0,er_abort)
      ENDIF
    ENDIF
  ENDIF
END PIPONOFF
```

注意 此内置仅在加载DAQ或数据监视器选项时可用。

A.5.4 DAQ_STOP 内置程序

用途：停止写入KAREL管道。

语法：DAQ_STOP(pipe_num, force_off, status)

Input/Output 参数：

[in] pipe_num :INTEGER

[in] force_off :BOOLEAN

[out] status :INTEGER

细节：

- *pipe_num* 是要停止的管道（1 - 5）的编号。

- 如果TRUE强制关闭管道，则发生*f orce_off*，即使另一个应用程序在管道上发出了启动请求。 如果设置为FALSE，如果已使用停止请求计算了所有启动请求，则管道将关闭，否则它将保持打开状态。
- *status* 是尝试操作的状态。 如果不为0，则发生错误并且管道未停止。

另请参见：DAQ_START内置。

DAQ_STOP 内置程序

每个管道上的启动/停止机制对引用计数起作用。 管道在第一个启动请求时启动，并计算每个后续启动请求。 如果收到管道的停止请求，则计数减少。

DAQ_STOP 内置程序

如果没有强制关闭管道，并且计数不为零，则管道保持打开状态。 通过将`force_off`标志设置为`TRUE`，无论计数如何，管道都将关闭。 计数重置。

DAQ_STOP 内置程序

`PC`应用程序发出的`FRCPipe.StopMonitor`方法相当于对`DAQ_STOP`的调用。

示例：请参阅内置函数DAQ_START中的PIPONOFF示例。

注意 此内置仅在加载DAQ或数据监视器选项时可用。

A.5.5 DAQ_UNREG 内置程序

用途：取消注册以前注册的KAREL管道，以便它可以用于其他数据。

语法：DAQ_UNREG(pipe_num, status)

Input/Output 参数：

[in] pipe_num :INTEGER

[out] status :INTEGER

细节：

- *pipe_num* 是要取消注册的管道（1 - 5）的编号。

- *status* 是尝试操作的状态。 如果不为0，则发生错误，管道未注销。

另请参见：DAQ_REGPIPE内置。

DAQ_UNREG 内置程序

取消注册管道允许为不同的数据大小，管道大小，管道名称等重新配置管道。 在使用DAQ_REGPIPE重新注册之前，必须取消注册管道。

示例：以下示例取消注册KAREL管道1。

DAQ_UNREG 内置程序

```
PROGRAM DAQUNREG
%ENVIRONMENT DAQ
CONST
  er_abort = 2
VAR
  status:  INTEGER
BEGIN
  -- unregister pipe 1
  DAQ_UNREG(1, status)
  IF status<>0 THEN
    POST_ERR(status,' ',0,er_abort)
  ENDIF
END DAQUNREG
```

注意 此内置仅在加载DAQ或数据监视器选项时可用。

A.5.6 DAQ_WRITE 内置程序

用途：将数据写入KAREL管道。

语法：DAQ_WRITE(pipe_num, prog_name, var_name, status)

Input/Output 参数：

[in] pipe_num :INTEGER

[in] prog_name :STRING

[in] var_name :STRING

[out] status :INTEGER

细节：

- *pipe_num* 是写入数据的管道（1 - 5）的编号。
- *prog_name* 是包含要写入的变量的程序的名称。 如果作为空字符串传递，则使用当前程序的名称。
- *var_name* 是要写入的变量的名称。
- *status* 是尝试操作的状态。 如果不为0，则发生错误并且未写入数据。

另请参见：DAQ_REGPIPE和DAQ_CHECKP。

DAQ_WRITE 内置程序

您不必使用相同的变量将数据写入用于注册管道的管道。 唯一的要求是写入的变量的数据类型与用于注册管道的变量的类型相匹配。

DAQ_WRITE 内置程序

如果PC应用程序正在监视管道，则每次调用DAQ_WRITE都将导致FRCPipe_Receive事件。

示例：以下示例注册KAREL管道2，并在管道处于活动状态时写入管道2。

DAQ_WRITE 内置程序

```
PROGRAM DAQWRITE
%ENVIRONMENT DAQ
%ENVIRONMENT SYSDEF
CONST
  er_abort = 2
TYPE
  daq_data_t = STRUCTURE
    count: INTEGER
    dataval: INTEGER
  ENDSTRUCTURE
VAR
  status:  INTEGER
  pipestat: INTEGER
  numbytes: INTEGER
  datavar: daq_data_t
BEGIN
  -- register 10KB pipe 2 in DRAM as kldaq.dat
  DAQ_REGPIPE(2, DAQ_DRAM, 100, '', 'datavar', &
```

```
            'kldaq.dat', 1, status)
  IF status<>0 THEN
    POST_ERR(status,' ',0,er_abort)
  ENDIF
  -- use DAQ_CHECKP to monitor status of pipe
  DAQ_CHECKP(2, pipestat, numbytes)
  datavar.count = 0
  WHILE (pipestat AND DAQ_PIPREG) > 0 DO  -- do while registered
    -- update data variable
    datavar.count = datavar.count + 1
    datavar.dataval = $FAST_CLOCK
    -- check if pipe is active
    IF (pipestat AND DAQ_ACTIVE) > 0 THEN
      -- write to pipe
      DAQ_WRITE(2, '', datavar, status)
      IF status<>0 THEN
        POST_ERR(status,' ',0,er_abort)
      ENDIF
    ENDIF
    -- put in delay to reduce loading
    DELAY(200)
    DAQ_CHECKP(2, pipestat, numbytes)
  ENDWHILE
END DAQWRITE
```

注意 此内置仅在加载DAQ或数据监视器选项时可用。

A.5.7 %DEFGROUP 译者指令

用途：指定转换程序要使用的默认运动组

语法：%DEFGROUP = n

细节：

- *n* 是运动组的编号。
- 范围是 1 到控制器上的组数。
- 如果未指定 %DEFGROUP，则使用组1。

A.5.8 DEF_SCREEN 内置程序

用途：定义一个屏幕

语法：DEF_SCREEN(screen_name, disp_dev_name, status)

Input/Output 参数：

[in] screen_name :STRING

[in] disp_dev_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- 定义与指定显示设备关联的屏幕，可以将窗口连接到该屏幕并激活（显示）。
- *screen_name* 必须是唯一的有效名称（字符串），长度为一到四个字符。
- *disp_dev_name* 必须是已定义的显示设备之一，否则返回错误。 以下是预定义的显示设备：
- 'TP'示教吊坠设备'CRT'CRT / KB设备
- *status* 说明了尝试操作的状态。 （如果不等于0，则发生错误。）

另请参阅：ACT_SCREEN内置过程

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.5.9 DEF_WINDOW 内置程序

用途：定义一个窗口

语法：DEF_WINDOW(window_name, n_rows, n_cols, options, status)

Input/Output 参数：

[in] window_name :STRING

[in] n_rows :INTEGER

[in] n_cols :INTEGER

[in] options :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 定义一个窗口，该窗口可以随后附加到屏幕，打开文件，写入或回显输入，并在其中动态显示信息。
- *window_name* 必须是有效的名称字符串，长度为1到4个字符，并且不得复制具有相同名称的窗口。
- *n_rows* 和 *n_cols* 以标准大小的字符指定窗口的大小。任何包含双宽或双宽双高字符的行将只包含这么多字符的一半。第一行和第一列从1开始。
- 选项必须是以下之一：
  0：没有选择
  wd_com_cursr：公共光标
  wd_scrolled：垂直滚动
  wd_com_cursr + wd_scrolled：公共光标+垂直滚动
- 如果指定了公共光标，则无论使用何种文件变量，写入离开光标的位置都是下一次写入的位置。此外，为与此窗口关联的任何文件变量设置的任何显示属性都将应用于与该窗口关联的所有文件变量。如果未指定，则为窗口打开的每个文件变量单独维护光标位置和显示属性（字符大小属性除外，它们始终应用于窗口的当前行）。 common-cursor属性对于可以由多个任务写入的窗口以及这些写入端到端显示的位置非常有用。一个例子可能是日志显示。
- 如果指定了垂直滚动并且接收到换行符，换行符，索引向下字符并且光标位于窗口的最后一行，则除了顶行之外的所有行都会向上移动并清除底行。如果写入了索引向上字符，则除了底线之外的所有行都向下移动并且顶行被清除。如果未指定，则清除底部或顶部线，但窗口的其余部分不受影响。
- *status* 说明了尝试操作的状态。如果不等于0，则发生错误。

另请参见：ATT _WINDOW_D，ATT_WINDOW_S内置过程

A.5.10 %DELAY 译者指令

用途：设置程序执行每250毫秒延迟的时间。 默认情况下，每个程序每250ms延迟8ms。 这允许CPU执行其他功能，例如为CRT/KB和Teach Pendant用户界面提供服务。 %DELAY 提供了一种从默认值更改的方法，并允许更多CPU用于系统任务，例如用户界面。

语法：%DELAY = n

细节：

- *n* 是延迟时间（以毫秒为单位）。
- 如果未指定 DELAY，则默认延迟时间为8 ms
- 如果 *n* 设置为0，程序将尝试使用100%的可用CPU时间。 这可能导致示教器和CRT / KB因其优先级较低而无法运行。 如果程序将等待运动或I / O，则延迟为0是可以接受的。
- 在显示一个程序时，禁止执行其他程序。 中断例程（从条件处理程序调用的例程）也将被延迟。
- 非常大的延迟值将严重阻碍所有程序的运行。
- 要延迟一个程序而不是另一个程序，请使用DELAY语句而不是%DELAY。

A.5.11 DELAY 声明

用途：使程序的执行暂停指定的毫秒数

语法：DELAY time_in_ms

其中：

time_in_ms： 一个 INTEGER（整数） 表达式

细节：

- 如果在延迟时间运动有效，则运动继续。
- *time_in_ms* 是以毫秒为单位的时间。 实际延迟将从零到$ SCR.$ cond_time 毫秒小于舍入时间。
- 时间规格为零不起作用。
- 如果程序在延迟过程中暂停，则延迟将继续定时。
- 如果暂停程序中的延迟时间在程序仍处于暂停状态时到期，则程序在恢复时不再延迟，将继续执行延迟后的语句。 否则，恢复后，程序将在继续执行之前完成延迟时间。
- 程序暂停时，中止程序或从CRT/KB发出RUN，可终止任何正在进行的延迟。
- 当一个程序正在等待延迟到期时，KCL> SHOW TASK命令将显示DELAY的保留。
- 大于一天或小于零的时间值会导致程序中止并出现错误。

另请参阅：附录E，“语法图”，以获取其他语法信息

示例：请参阅以下各节以获取详细的程序示例：

第B.11节“操作动态显示变量的值”（CHG_DATA.KL）

第B.1节“设置数字输出端口进行监测”（DOUT_EX.KL）

A.5.12 DELETE_FILE 内置程序

用途：删除指定的文件

语法：DELETE_FILE(file_spec, nowait_sw, status)

Input/Output 参数：

[in] file_spec :STRING

[in] nowait_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *file_spec* 指定要删除的文件的设备，名称和类型。 可以使用通配符（*）字符指定 *file_spec*。 如果未指定设备名称，则使用默认设备。
- 如果 *nowait_sw* 为TRUE，则在执行命令时继续执行程序。 如果为FALSE，程序将停止，包括条件处理程序，直到操作完成。 如果程序中有时间紧急的条件处理程序，则将它们放在另一个作为单独任务执行的程序中。
  注意 *nowait_sw* 在此版本中不可用，应设置为FALSE。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：COPY_FILE，RENAME_FILE内置过程

示例：有关详细的程序示例，请参见第B.3节“将数据保存到默认设备”（SAVE_VRS.KL）。

A.5.13 DELETE_NODE 内置程序

用途：从PATH中删除路径节点

语法：DELETE_NODE(path_var, node_num, status)

Input/Output 参数：

[in] path_var :PATH

[in] node_num :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PATHOP

细节：

- *node_num* 指定要从 *path_var* 指定的PATH中删除的节点。
- 经过删除节点的所有节点都将重新编号。
- *node_num* 必须在1到PATH_LEN（*path_var*）的范围内。 如果超出此范围，则返回状态并显示错误。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：APPEND_NODE，INSERT_NODE内置过程

示例：有关详细的程序示例，请参见第B.2节 “复制路径变量”（CPY_PTH.KL）。

A.5.14 DELETE_QUEUE 内置程序

用途：从队列中删除条目

语法：DELETE_QUEUE(sequence_no, queue, queue_data, status)

Input/Output 参数：

[in] sequence_no :INTEGER

[in,out] queue_t :QUEUE_TYPE

[in,out] queue_data :ARRAY OF INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBQMGR

细节：

- 使用 COPY_QUEUE 获取序列号列表。
- *sequence_no* 指定要删除的条目的序列号。 使用 COPY_QUEUE 获取序列号列表。
- *queue_t* 指定队列的队列变量。
- *queue_data* 指定用于保存队列中数据的数组。 此数组的长度确定队列中的最大条目数。
- 如果指定的序列号不在队列中，则 *status* 返回61003，''错误序列号，'。

另请参阅：APPEND_QUEUE，COPY_QUEUE，INSERT_QUEUE内置过程，第15.8节“使用队列进行任务通信”

A.5.15 DEL_INST_TPE 内置程序

用途：删除指定示教程序中的指定指令

语法：DEL_INST_TPE(open_id, lin_num, status)

Input/Output 参数：

[in] open_id :INTEGER

[in] lin_num :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :TPE

细节：

- *open_id* 指定打开的示教程序。 在调用内置 DEL_INST_TPE 之前，必须使用内置的 OPEN_TPE 打开具有读/写访问权限的程序。
- *lin_num* 指定要删除的指令的行号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：CREATE_TPE，CLOSE_TPE，COPY_TPE，OPEN_TPE，SELECT_TPE内置过程

A.5.16 DET_WINDOW 内置程序

用途：从屏幕上分离窗口

语法：DET_WINDOW(window_name, screen_name, status)

Input/Output 参数：

[in] window_name :STRING

[in] screen_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 从指定的屏幕中删除指定的窗口。
- *window_name* 和 *screen_name* 必须有效且已定义。
- 重新显示此窗口隐藏的其他窗口的区域。 清除此窗口占用的任何区域，而不是任何其他窗口。
- 如果窗口未连接到屏幕，则会发生错误。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：DEF_WINDOW，ATT_WINDOW_S，ATT_WINDOW_D内置过程

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.5.17 DISABLE CONDITION 动作

用途：在条件处理程序中用于禁用指定的条件处理程序

语法：DISABLE CONDITION [cond_hand_no]

其中：

cond_hand_no : INTEGER 表达式

细节：

- 如果未定义条件处理程序，则 DISABLE CONDITION 无效。

- 如果已定义条件处理程序但当前未启用，则 DISABLE CONDITION 无效。
- 禁用条件处理程序时，不会测试其条件。 因此，如果再次激活，则必须在激活后满足条件。
- 使用 ENABLE CONDITION 语句或操作重新激活已禁用的条件处理程序。
- *cond_hand_no* 必须在1-1000的范围内。 否则，程序将因错误而中止。

另请参阅：第6章“条件处理程序”，以获取有关在条件处理程序中使用DISABLE CONDITION的更多信息

示例：以下示例在触发条件编号1时禁用条件处理程序编号2。

DISABLE CONDITION 动作

```
CONDITION[1]:
  WHEN EVENT[1] DO
  DISABLE CONDITION[2]
ENDCONDITION
```

A.5.18 DISABLE CONDITION 声明

用途：禁用指定的条件处理程序

语法：DISABLE CONDITION [cond_hand_no]

其中：

cond_hand_no : INTEGER 表达式

细节：

- 如果未定义条件处理程序，则 DISABLE CONDITION 无效。
- 如果已定义条件处理程序但当前未启用，则 DISABLE CONDITION 无效。
- 禁用条件处理程序时，不会测试其条件。 因此，如果再次激活，则必须在激活后满足条件。
- 使用 ENABLE CONDITION 语句或操作重新激活已禁用的条件处理程序。
- *cond_hand_no* 必须在1-1000的范围内。 否则，程序将因错误而中止。

另请参阅：第6章 条件处理程序，有关在条件处理程序中使用DISABLE CONDITION的更多信息，请参阅附录E，''语法图'，以获取其他语法信息

示例：以下示例允许操作员选择是否查看计数。

DISABLE CONDITION 声明

```
PROGRAM p_disable
VAR
  count    : INTEGER
  answer   : STRING[1]
ROUTINE showcount
BEGIN
  WRITE ('count = ',count::10,CR)
END showcount
BEGIN
  CONDITION[1]:
    WHEN EVENT[1] DO              -- 条件[1]显示计数
       showcount
    ENABLE CONDITION[1]
  ENDCONDITION
  ENABLE CONDITION[1]
  count = 0
  WRITE ('do you want to see count?')
  READ (answer,CR)
  IF answer = 'n'
    THEN DISABLE CONDITION[1]    -- 禁用状态[1]
  ENDIF                          -- 计数不会显示
  FOR count = 1 TO 13 DO
    SIGNAL EVENT[1]
  ENDFOR
END p_disable
```

A.5.19 DISCONNECT TIMER 声明

用途：停止更新先前作为计时器连接的时钟变量

语法：DISCONNECT TIMER timer_var

其中：

timer_var：静态的，用户定义的 INTEGER 变量

细节：

- 如果 *timer_var* 当前未作为计时器连接，则 DISCONNECT TIMER 语句无效。
- 如果 *timer_var* 是系统或局部变量，则不会翻译该程序。

另请参阅：附录E，''语法图'，有关其他语法信息，请参阅CONNECT TIMER语句

示例：以下示例将TCP移至PR [1]中的初始位置，将INTEGER变量timevar设置为0并连接计时器。 移动到PR [2]中的目标位置后，计时器断开。

DISCONNECT TIMER 声明

```
move_to_pr1 - 调用TP程序转移到PR [1]

timevar = 0
CONNECT TIMER TO timevar

move_to_pr2 - 调用TP程序移动到PR[2]
DISCONNECT TIMER timevar
```

A.5.20 DISCTRL_ALPH 内置程序

用途：显示和控制指定窗口中的字母数字字符串条目。

语法：DISCTRL_ALPH(window_name, row, col, str, dict_name, dict_ele, term_char, status)

Input/Output 参数：

[in] window_name :STRING

[in] row :INTEGER

[in] col :INTEGER

[in,out] str :STRING

[in] dict_name :STRING

[in] dict_ele :INTEGER

[out] term_char :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *window_name* 标识当前显示 *str* 的窗口。有关可用于 *window_name* 的窗口列表，另请参见第7.10.1节或第7.10.2节。
- *row* 指定显示 *str* 的行号。
- *col* 指定显示 *str* 的列号。
- *str* 指定要修改的KAREL字符串，该字符串当前显示在位置行和列的 *window_name* 上。
- *dict_name* 指定包含可以输入的单词的字典。 *dict_name* 也可以设置为以下预定义值之一。
  'PROG'：程序名称输入
  'COMM'：评论条目
- *dict_ele* 指定单词的字典元素编号。 *dict_ele* 最多可包含5行，最后一行不接受“& new_line”。请参阅下面的示例。
- 如果使用 *dict_name* 的预定义值，则忽略 *dict_ele*。
- *term_char* 接收指示终止菜单的字符的代码。密钥终止条件的代码在包含文件FR：KLEVKEYS.KL中定义。以下预定义常量是通常返回的键：
  ky_enter
  ky_prev
  ky_new_menu
- DISCTRL_ALPH 将显示和控制示教器设备的字符串输入。要显示和控制CRT / KB设备的字符串条目，必须创建INTEGER变量 *device_stat*，并将其设置为 *crt_panel*。要将控制设置为示教器设备，请将 *device_stat* 设置为 *tp_panel*。请参阅下面的示例。
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。

注意 如果USER或USER2菜单是所选菜单，DISCTRL_ALPH将仅显示和控制字符串条目。因此，在调用DISCTRL_ALPH强制USER菜单之前，请使用FORCE_SPMENU（device_stat，SPI_TPUSER，1）。

另请参阅：ACT_SCREEN，DISCTRL_LIST内置过程

示例：有关详细的程序示例，请参见第B.13节“使用DISCTRL_ALPHA内置”（DCLAP_EX.KL）。

A.5.21 DISCTRL_FORM 内置程序

用途：在示教器或CRT/KB屏幕上显示和控制窗体

语法：DISCTRL_FORM(dict_name, ele_number, value_array, inact_array, change_array, term_mask, def_item, term_char, status)

Input/Output 参数：

[in] dict_name : STRING

[in] ele_number : INTEGER

[in] value_array : ARRAY OF STRING

[in] inactive_array : ARRAY OF BOOLEAN

[out] change_array : ARRAY OF BOOLEAN

[in] term_mask : INTEGER

[in,out] def_item : INTEGER

[out] term_char : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :PBcore

细节：

- *dict_name* 是包含表单的字典的四字符名称。
- *ele_number* 是表单的元素编号。
- *value_array* 是一个变量名称数组，对应于每个编辑或仅显示表单中的数据项。 每个变量名称都可以指定为'[prog_name] var_name'。
  - [prog_name]是包含指定变量的程序的名称。 如果未指定[prog_name]，则使用正在执行的当前程序。 '[* SYSTEM *]'应该用于系统变量。
  - *var_name* 必须引用静态的全局程序变量。
  - *var_name* 可以包含节点号，字段名和/或下标。
  - *var_name* 还可以指定带索引的端口变量。 例如，'DIN [1]'。
- *inactive_array* 是一个布尔数组，对应于表单中的每个项目。
  - 每个布尔值默认为FALSE，表示它处于活动状态。
  - 您可以将任何布尔值设置为TRUE，这将使该项无效且不可选。

- 数组大小可以大于或小于表单中的项目数。
- 如果未使用 *inactive_array*，则可以使用数组大小1。 该阵列不需要初始化。

- *change_array* 是一个布尔数组，对应于每个编辑或仅显示表单中的数据项。
  - 如果设置了相应的值，则布尔值将设置为TRUE，否则设置为FALSE。您不需要初始化数组。
  - 数组大小可以大于或小于表单中数据项的数量。
  - 如果未使用 *change_array*，则可以使用数组大小 1。
- *term_mask* 是一个按位掩码，表示将终止表单的条件。这应该是包含文件klevkmsk.kl中定义的常量的OR。

  kc_func_key - 功能键

  kc_enter_key - 输入和返回键

  kc_prev_key - PREV键

  如果选择了可选项或新菜单，则无论 *term_mask* 如何，表单都将始终终止。
- 对于版本6.20和6.21，*def_item* 会在输入表单时收到要突出显示的项目。 *def_item* 返回按下终止字符时当前突出显示的项目。
- 对于版本6.22及更高版本，*def_item* 会在输入表单时接收您要突出显示的项目。在显示表单时，*def_item* 会不断更新，并包含当前突出显示的项目编号
- *term_char* 接收一个代码，指示终止表单的字符或其他条件。密钥终止条件的代码在包含文件klevkeys.kl中定义。通常返回的键是预定义的常量，如下所示：

  ky_undef - 没有按下终止字符

  ky_select - 选择了一个可选项

  ky_new_menu - 选择了一个新菜单

  ky_f1 - 选择了功能键1

  ky_f2 - 选择了功能键2

  ky_f3 - 选择了功能键3

  ky_f4 - 选择了功能键4

ky_f5 - 选择了功能键5

ky_f6 - 选择了功能键6

ky_f7 - 选择了功能键7

ky_f8 - 选择了功能键8

ky_f9 - 选择了功能键9

ky_f10 - 选择了功能键10

- DISCTRL_FORM 将在示教器设备上显示该表格。 要在 CRT/KB 设备上显示表单，您必须创建一个 INTEGER 变量 *device_stat*，并将其设置为 *crt_panel*。要将控制设置为示教器设备，请将 *device_stat* 设置为 *tp_panel*。
- *status* 说明了尝试操作的状态。 如果 *status* 返回0以外的值，则表示发生了错误。

注意 如果USER2菜单是所选菜单，DISCTRL_FORM 将仅显示表单。 因此，在调用 DISCTRL_FORM 强制USER2菜单之前，请使用 FORCE_SPMENU（device_stat，SPI_TPUSER2,1）。

另请参见：第10章字典和表格，了解更多详细信息和示例。

A.5.22 DISCTRL_LIST 内置程序

用途：在指定窗口的列表中显示和控制光标移动和选择

语法：DISCTRL_LIST(file_var, display_data, list_data, action, status)

Input/Output 参数：

[in] file_ var :FILE

[in,out] display_data :DISP_DAT_T

[in] list_data :ARRAY OF STRING

[in] action :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 必须打开 *file_var* 到要显示列表数据的窗口。

- *display_data* 用于显示列表。 有关详细信息，请参阅 DISP_DAT_T 数据类型。
- *list_data* 包含要显示的数据列表。
- 操作必须是以下之一：

  dc_disp：定位游标，如 *display_data* 中所定义

  dc_up：将光标向上移动一行

  dc_dn：将光标向下移动一行

  dc_lf：将光标向左移动一个字段

  dc_rt：将光标向右移动一个字段
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 使用 DISCTRL_FORM 是在窗口中显示和控制信息的首选方法。

另请参阅：DISCTRL_FORM内置程序，第7.10.1节“示教器上的用户菜单”，第7.10.2节“CRT / KB上的用户菜单”，第10章 字典和表单

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

> 警告
> 不检查输入参数的有效性。 您必须确保输入参数有效; 否则，内置可能无法正常工作。

A.5.23 DISCTRL_PLMN 内置程序

用途：在上拉菜单中创建和控制光标移动和选择

语法：DISCTRL_PLMN(dict_name, element_no, ftn_key_no, def_item, term_char, status)

Input/Output 参数：

[in] dict_name :STRING

[in] element_no :INTEGER

[in] ftn_key_num :INTEGER

[in,out] def_item :INTEGER

[out] term_char :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group : UIF

细节：

- 字典中的菜单数据包含一个枚举值列表，这些值在教导设备的上拉菜单中显示和选择。最多应使用9个值。每个值都是一个最多12个字符的字符串。
  从 *element_no* 开始的一系列连续字典元素定义了值。每个值必须放在一个单独的元素中，并且不能以&new_line结尾。字符按顺序分配数值1..9。最后一个字典元素必须是“”。
- *dict_name* 指定包含菜单数据的字典的名称。
- *element_no* 是字典中第一个菜单项的元素编号。
- *ftn_key_num* 是应显示上拉菜单的功能键。
- *def_item* 是输入菜单时应突出显示的项目。 1指定第一个项目。返回时，*def_item* 是按下终止字符时当前突出显示的项目。
- *term_char* 接收指示终止菜单的字符的代码。密钥终止条件的代码在包含文件FROM：KLEVKEYS.KL中定义。通常返回的键是预定义的常量，如下所示：
  ky_enter - 选择了一个菜单项
  ky_prev - 未选择菜单项
  ky_new_menu - 选择了一个新菜单
  ky_f1
  ky_f2
  ky_f3
  ky_f4
  ky_f5
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。

示例：在此示例中，字典文件TPEXAMEG.TX在控制器上作为“EXAM”加载。TPPLMN.KL调用DISCTRL_PLMN来显示和处理功能键3上方的上拉菜单。

DISCTRL_PLMN 内置程序

```
-----------------------------------------------
TPEXAMEG.TX
-----------------------------------------------
$subwin_menu
"Option 1"
$
"Option 2"
$
"Option 3"
$
"Option 4"
$
"Option 5"
$
"......"
-----------------------------------------------
TPPLMN.KL
-----------------------------------------------
PROGRAM tpplmn
%ENVIRONMENT uif
VAR
  def_item: INTEGER
  term_char: INTEGER
  status: INTEGER
BEGIN
  def_item = 1
  DISCTRL_PLMN('EXAM', 0, 3, def_item, term_char, status)
  IF term_char = ky_enter THEN
    WRITE (CR, def_item, ' was selected')
  ENDIF
END tpplmn
```

A.5.24 DISCTRL_SBMN 内置程序

用途：在子窗口菜单中创建和控制光标移动和选择

语法：DISCTRL_SBMN(dict_name, element_no, def_item, term_char, status)

Input/Output 参数：

[in] dict_name :STRING

[in] element_no :INTEGER

[in,out] def_item :INTEGER

[out] term_char :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group : UIF

细节：

- 字典中的菜单数据由显示的枚举值列表组成
  并从示教器设备上的'subm'子窗口中选择。最多可以有5个子窗口页面，最多35个值。每个值都是一个最多16个字符的字符串。如果使用4个或更少的枚举值，则每个字符串最多可包含40个字符。
  从 *element_no* 开始的一系列连续字典元素定义了值。每个值必须放在一个单独的元素中，并且不能以＆new_line结尾。字符按顺序分配数值1..35。最后一个字典元素必须是“”。
- *dict_name* 指定包含菜单数据的字典的名称。
- *element_no* 是字典中第一个菜单项的元素编号。
- *def_item* 是输入菜单时应突出显示的项目。 1指定第一个项目。返回时，*def_item* 是按下终止字符时当前突出显示的项目。
- *term_char* 接收指示终止菜单的字符的代码。密钥终止条件的代码在包含文件FROM：KLEVKEYS.KL中定义。通常返回的键是预定义的常量，如下所示：
  ky_enter - 选择了一个菜单项
  ky_prev - 未选择菜单项
  ky_new_menu - 选择了一个新菜单
  ky_f1
  ky_f2
  ky_f3
  ky_f4
  ky_f5
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。

示例：在此示例中，字典文件TPEXAMEG.TX在控制器上作为“EXAM”加载。
TPSBMN.KL调用DISCTRL_SBMN来显示和处理子窗口菜单。

DISCTRL_SBMN 内置程序

```
-----------------------------------------------
TPEXAMEG.TX
-----------------------------------------------
$subwin_menu
"Red"
$
"Blue"
$
"Green"
$
"Yellow"
$
"Brown"
$
"Pink"
$
"Mauve"
$
"Black"
$
"Lime"
$
"Lemon"
$
"Beige"
$
"Blue"
$
"Green"
$
"Yellow"
$
"Brown"
$
"\a"
-----------------------------------------------
TPSBMN.KL
-----------------------------------------------
PROGRAM tpsbmn
%ENVIRONMENT uif
VAR
  def_item: INTEGER
  term_char: INTEGER
  status: INTEGER
BEGIN
  def_item = 1
```

```
  DISCTRL_SBMN('EXAM', 0, def_item, term_char, status)
  IF term_char = ky_enter THEN
    WRITE (CR, def_item, ' was selected')
  ENDIF
END tpsbmn
```

A.5.25 DISCTRL_TBL 内置程序

用途：显示和控制示教器上的表单

语法：DISCTRL_TBL(dict_name, ele_number, num_rows, num_columns, col_data, inact_array, change_array, def_item, term_char, term_mask, value_array, attach_wind, status)

Input/Output 参数：

[in] dict_name :STRING

[in] ele_number :INTEGER

[in] num_rows :INTEGER

[in] num_columns :INTEGER

[in] col_data :ARRAY OF COL_DESC_T

[in] inact_array :ARRAY OF BOOLEAN

[out] change_array :ARRAY OF BOOLEAN

[in,out] def_item :INTEGER

[out] term_char :INTEGER

[in] term_mask :INTEGER

[in] value_array :ARRAY OF STRING

[in] attach_wind :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group : UIF

细节：

- DISCTRL_TBL类似于INIT_TBL和ACT_TBL内置例程，如果每次击键都不需要进行特殊处理，则应使用它。
- *dict_name* 是包含表头的字典的四字符名称。
- *ele_number* 是表头的元素编号。
- *num_rows* 是表中的行数。
- *num_columns* 是表中的列数。
- *col_data* 是列描述符结构的数组，表中的每列都有一个。 有关完整说明，请参阅本附录中的INIT_TBL内置例程。
- *inact_array* 是一个布尔数组，对应于表中的每一列。

  - 您可以将每个布尔值设置为TRUE，这将使该列处于非活动状态。 这意味着您无法将光标移动到此列。

  - 数组大小可以小于或大于表中的项目数。

  - 如果未使用 *change_array*，则可以使用数组大小1，并且不需要初始化该数组。
- *change_array* 是BOOLEAN的二维数组，对应于表中的格式化数据项。

  - 如果设置了相应的值，则布尔值将设置为TRUE，否则设置为FALSE。您不需要初始化数组。

  - 数组大小可以小于或大于表中数据项的数量。

  - 如果未使用 *change_array*，则可以使用数组大小 1。
- *def_item* 是包含要在输入表格时突出显示的项目的行。返回时，*def_item* 是包含按下终止字符时当前突出显示的项目的行。
- *term_char* 接收一个代码，指示终止表的字符或其他条件。密钥终止条件的代码在包含文件FROM：KLEVKEYS.KL中定义。通常返回的键是预定义的常量，如下所示：

  ky_undef - 没有按下终止字符

  ky_select - 选中的可选项

  ky_new_menu - 选择了一个新菜单

  ky_f1 - 选择了功能键1

  ky_f2 - 选择了功能键2

  ky_f3 - 选择了功能键3

  ky_f4 - 选择了功能键4

ky_f5 - 选择了功能键5

ky_f6 - 选择了功能键6

ky_f7 - 选择了功能键7

ky_f8 - 选择了功能键8

ky_f9 - 选择了功能键9

ky_f10 - 选择了功能键10

- *term_mask* 是一个按位掩码，表示将终止请求的条件。这应该是包含文件FROM：KLEVKMSK.KL中定义的常量的OR。

  kc_display - 可显示的键

  kc_func_key - 功能键

  kc_keypad - 键盘和编辑键

  kc_enter_key - 输入和返回键

  kc_delete - 删除和退格键

  kc_lr_arw - 左右箭头键

  kc_ud_arw - 向上和向下箭头键

  kc_other - 其他键（如Prev）
- *value_array* 是一个变量名数组，对应于表中的每一列数据项。每个变量名称都可以指定为“[prog_name] var_name”。

  - [prog_name]指定包含指定变量的程序的名称。如果未指定[prog_name]，则使用当前正在执行的程序。

  - *var_name* 必须引用静态的全局程序变量。

  - *var_name* 可以包含节点号，字段名和/或下标。
- 如果需要将表管理器窗口连接到显示设备，则 *attach_wind* 应设置为1。如果已附加，则此参数可以设置为0。
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。

示例：有关设置字典文本和初始化参数的示例，请参阅INIT_TBL内置例程。

A.5.26 DISMOUNT_DEV 内置程序

用途：卸载指定的设备。

语法：DISMOUNT_DEV (device, status)

Input/Output 参数：

[in] device :STRING

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *device* 指定要卸载的设备。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：MOUNT_DEV，FORMAT_DEV内置过程

示例：有关详细的程序示例，请参见第B.9节“使用文件和设备内置函数”（FILE_EX.KL）。

A.5.27 DISP_DAT_T 数据类型

用途：定义在DISCTRL_LIST内置中使用的数据类型

语法：

disp_dat_t = STRUCTURE

win_start : ARRAY [4] OF SHORT

win_end : ARRAY [4] OF SHORT

curr_win : SHORT

cursor_row : SHORT

lins_per_pg : SHORT

curs_st_col : ARRAY [10] OF SHORT

curs_en_col : ARRAY [10] OF SHORT

curr_field : SHORT

last_field : SHORT

curr_it_num : SHORT

sob_it_num : SHORT

eob_it_num : SHORT

last_it_num : SHORT

menu_id : SHORT

ENDSTRUCTURE

细节：

- *disp_dat_t* 可用于在四个不同窗口中显示列表。该列表最多可包含10个字段。左右箭头在字段之间移动。向上和向下箭头在一个字段内移动。
- *win_start* 是每个窗口的起始行。
- *win_end* 是每个窗口的结束行。
- *curr_win* 定义要显示的窗口。计数从零开始（0将显示第一个窗口）。
- *cursor_row* 是当前光标行。
- *lins_per_pg* 是用于向上和向下分页的每页行数。
- *curs_st_col* 是每个字段的游标起始列。示教器的范围是0-39。
- *curs_en_col* 是每个字段的光标结束列。示教器的范围是0-39。
- *curr_field* 是光标所在的当前字段。计数从零开始（0将光标设置为第一个字段）。
- *last_field* 是列表中的最后一个字段。
- *curr_it_num* 是光标所在的项目编号。
- *sob_it_num* 是数组中第一个项目的项目编号。
- *eob_it_num* 是数组中最后一项的项目编号。
- *last_it_num* 是列表中最后一项的项目编号。
- *menu_id* 是当前菜单标识符。未实现。可能没有初始化。

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLIST_EX.KL）。

A.5.28 DOSFILE_INF 内置程序

用途：在value参数中以字符串形式返回设备的信息。

语法：DOSFILE_INF(device, item, value_str, status)

Input/Output 参数：

[in] device :STRING

[in] item :INTEGER

[out] value_str :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- DOSFILE_INF 仅适用于R-30iB控制器
- 设备可以是 UD1：
- *item* 是要返回的数据的常量：
  - DF_MANUFNAME（制造商名称）
  - DF_PRODNAME（产品名称）
  - DF_SERIALNO（序列号）
  - DF_PRODID（由USB机构分配的产品ID号）
  - DF_VENDID（由USB授权机构分配的供应商ID号）
- *value_str* 是返回数据的字符串。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

示例：以下示例获取有关UD1设备的信息。

DOSFILE_INF 内置程序

```
program dfinfo
%NOLOCKGROUP
var
value: string[60]
status: integer
begin
value = ''
DOSFILE_INF('UD1:', DF_MANUFNAME, value, status)
```

```
write (cr, '1: ', status, ', ', value, cr)
value = ''
DOSFILE_INF('UD1:', DF_PRODNAME, value, status)
write (cr, '2: ', status, ', ', value, cr)
value = ''
DOSFILE_INF('UD1:', DF_SERIALNO, value, status)
write (cr, '3: ', status, ', ', value, cr)
delay (2000)
value = ''
DOSFILE_INF('UD1:', DF_PRODID, value, status)
write (cr, '4: ', status, ', ', value, cr)
value = ''
DOSFILE_INF('UD1:', DF_VENDID, value, status)
write (cr, '5: ', status, ', ', value, cr)
end dfinfo
```

A.6 - E - KAREL 语言描述

A.6.1 ENABLE CONDITION 功能

用途：启用指定的条件处理程序

语法： ENABLE CONDITION [cond_hand_no]

其中 :

cond_hand_no : INTEGER 表达式

细节 :

- 启用条件时没有效果
  - 条件处理程序未定义
  - 条件处理程序已定义，但已启用
- *cond_hand_no* 必须在1-1000的范围内。 否则，程序将中止并出错。
- 当启用条件处理程序时，每次扫描条件处理程序时都会测试其条件。 如果满足条件，则执行相应的操作并禁用条件处理程序。 发出ENABLE CONDITION语句或操作来重新激活它。
- 使用DISABLE CONDITION语句或操作来取消激活已启用的条件处理程序。
- 条件处理程序只对定义它们的任务是已知的。 一项任务不能启用其他任务条件。

另请参阅：禁用条件操作，第6章 条件处理器

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.6.2 ENABLE CONDITION 声明

用途：启用指定的条件处理程序

语法：ENABLE CONDITION [cond_hand_no]

其中：

cond_hand_no : INTEGER 表达式

细节：

- 启用条件时没有效果
  - 条件处理程序未定义
  - 条件处理程序已定义，但已启用
- *cond_hand_no* 必须在1-1000的范围内。 否则，程序将中止并出错。
- 当启用条件处理程序时，每次扫描条件处理程序时都会测试其条件。 如果满足条件，则执行相应的操作并禁用条件处理程序。 发出 ENABLE CONDITION 语句或操作来重新激活它。
- 使用 DISABLE CONDITION 语句或操作来取消激活已启用的条件处理程序。
- 条件处理程序只对定义它们的任务是已知的。 一项任务不能启用其他任务条件。

另请参阅：DISABLE CONDITION语句，第6章 条件处理程序，附录E，''语法图'，'了解其他语法信息

示例：有关详细的程序示例，请参阅以下部分。

第B.6节 “路径变量和条件处理程序”（PTH_MOV.KL）

第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）

A.6.3 %ENVIRONMENT 译者指令

用途：加载环境文件。

语法：%ENVIRONMENT path_name

- 脱机转换器用于指定应加载二进制文件 path_name.ev。 环境文件包含预定义常量，端口，类型，系统变量和内置插件的定义。
- 所有.EV文件在安装控制器软件时加载。 因此，控制器的转换器将忽略 %ENVIRONMENT 语句，因为它已经加载了.EV文件。
- *path_name* 可以是下列之一：
  - BYNAM
  - CTDEF (允许程序访问 CRT / KB 系统变量)
  - ERRS
  - FDEV
  - FLBT
  - IOSETUP
  - KCLOP
  - MEMO
  - MIR
  - MOTN
  - MULTI
  - PATHOP
  - PBCORE
  - PBQMGR
  - REGOPE
  - STRNG
  - SYSDEF (允许程序访问大多数系统变量)
  - SYSTEM
  - TIM
  - TPE
  - TRANS
  - UIF
  - VECTR
- 如果您的KAREL程序中没有指定 %ENVIRONMENT 语句，则离线翻译器将加载 TRMNEG.TX 中指定的所有.EV文件。 翻译器必须能够在当前目录或其中一个PATH目录中找到这些文件。

- 如果至少指定了一个 %ENVIRONMENT 语句，那么离线翻译器将只加载您在KAREL程序中指定的文件。 指定您自己的 %ENVIRONMENT 语句将减少翻译所需的内存量，并且速度会更快，尤其是如果您不需要系统变量，因为 SYSDEF.EV 是最大的文件。
- SYSTEM.EV和PBCORE.EV由翻译器自动加载，不应在您的KAREL程序中指定。 如果无法找到SYSTEM.EV，脱机转换器将打印消息“Continuing without system defined symbols”。 不要忽视这个消息。 确保加载了SYSTEM.EV文件。

示例：请参阅以下各节以获取详细的程序示例：

第B.3节“将数据保存到默认设备”（SAVE_VR.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.14节“在复制的示教器上应用胶印”（CPY_TP.KL）

A.6.4 ERR_DATA 内置程序

用途：从错误历史中读取请求的错误并返回错误

语法：ERR_DATA(seq_num, error_code, error_string, cause_code, cause_string, time_int, severity, prog_nam)

Input/Output 参数：

[in,out] seq_num :INTEGER

[out] error_code :INTEGER

[out] error_string :STRING

[out] cause_code :INTEGER

[out] cause_string :STRING

[out] time_int :INTEGER

[out] severity :INTEGER

[out] prog_nam :STRING

%ENVIRONMENT Group :ERRS

- *seq_num* 是请求的前一个错误的序列号。 如果需要历史记录中最老的错误，*seq_num* 应该设置为0。 如果最近的错误是需要的，*seq_num* 应该被设置为MAXINT。
- *seq_num* 被设置为返回错误的序列号。
  - 如果 *seq_num* 的初始值大于日志中最新错误的序列号，则 *seq_num* 将返回为零，并且不会返回其他数据。
  - 如果 *seq_num* 的初始值小于日志中最旧错误的序列号，则返回最旧的错误。
- *error_code* 返回错误代码，*error_string* 返回错误消息。 *error_string* 的长度必须至少为40个字符，否则程序将中止并出现错误。
- 如果 *cause_code* 存在并且 *cause_string* 返回消息，则 *cause_code* 返回原因代码。*cause_string* 的长度必须至少为40个字符，否则程序将中止并出现错误。
- *error_code* 和 *cause_code* 采用以下格式：
  ffccc（十进制）
  其中ff代表错误的设施代码。
  ccc表示指定设施内的错误代码。
  有关错误设施代码，请参阅第6章“条件处理程序”。
- *time_int* 返回发布 *error_code* 的时间。时间采用编码格式，应使用 CNV_TIME_STR 内置来获取日期和时间字符串。
- 严重性返回以下 *error_codes* 之一：0:WARNING 1:PAUSE 2:ABORT
- 如果执行程序时发生错误，*prog_nam* 将指定发生错误的程序的名称。
- 如果错误由 POST_ERR 发布，或者错误与特定程序无关（例如，急停），则 *prog_nam* 返回为' "" '。
- 在 POST_ERR 之后立即调用 ERR_DATA 可能不会返回自错误实际出现在错误日志中之前 POST_ERR 返回后刚发布的错误。

另请参见：POST_ERR内置过程

A.6.5 ERROR 条件

用途：将错误指定为条件

语法：ERROR[n]

其中：

n : INTEGER 表达式或星号 (*)

细节 :

- 如果 *n* 是 INTEGER，则表示错误代码编号。 发生指定错误时，条件满足。
- 如果 *n* 是星号 (*) ，则表示通配符。 发生任何错误时满足条件。
- 该条件是一个事件条件，这意味着只有在检测到错误时执行的扫描才满足该条件。 在随后的扫描中不会记住该错误。

另请参阅 : 第6章 条件处理程序，了解有关使用条件的更多信息。 适用于特定应用的FANUC机器人设置和操作手册，列出所有错误代码

例如 : 详细程序示例请参考第B.1节 “设置数字输出端口进行监视” (DOUT_EX.KL) 。

A.6.6 EVAL 子句

用途 : 允许在条件处理程序定义中计算表达式

语法 : EVAL(expression)

其中 :

expression : 一个有效的KAREL表达式

细节 :

- 在定义条件处理程序时计算 *expression* ，而不是在扫描期间动态计算。
- *expression* 可以是任何不包含函数调用的有效表达式。

另请参阅 : 第6章 条件处理程序，有关使用条件的更多信息

示例 : 以下示例导致延迟，直到AIN [force]大于计算的表达式 (10 * f_scale) 。

EVAL 子句

```
WRITE ('Enter force scale: ')
READ (f_scale)
REPEAT
  DELAY(1000)
UNTIL AIN[force] > EVAL(10 * f_scale)
```

A.6.7 EVENT 条件

用途：指定执行SIGNAL EVENT语句或具有该事件编号的操作时满足条件的事件的编号

语法：EVENT[event_no]

其中：

event_no：是一个 INTEGER 表达式

细节：

- 事件可以用作用户定义的事件代码，在发出信号时变为 TRUE。
- SIGNAL EVENT 语句或操作用于表示发生了事件。
- *event_no* 必须在-32768到32767的范围内。

另请参阅：SIGNAL EVENT 动作，以及CONDITION或SIGNAL EVENT 声明

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.6.8 EXP 内置函数

用途：返回等于e（大约2.71828）的REAL值，该值增加到REAL参数指定的功率

语法：EXP(x)

函数返回类型：REAL

Input/Output 参数：

[in] x :REAL

%ENVIRONMENT Group :SYSTEM

细节：

- EXP 返回 e（自然对数的基数）上升到幂 x。
- x 必须小于80.否则，程序将暂停并显示错误。

示例：以下示例使用EXP内置来计算表达式的指数（-6.44 + timevar /（timevar + 20））。

EXP 内置函数

```
WRITE (CR, 'Enter time needed for move:')
  READ (timevar)
  distance = timevar *
             EXP(-6.44 + timevar/(timevar + 20))
  WRITE (CR, CR, 'Distance for move:', distance::10::3)
```

A.7 - F - KAREL 语言描述

A.7.1 FILE 数据类型

用途：将变量定义为FILE数据类型

语法： file

细节：

- FILE 允许您将静态变量声明为文件。
- 必须在 OPEN FILE,READ,WRITE,CANCEL FILE和CLOSE FILE 语句中使用FILE变量。
- 您可以将FILE变量作为参数传递给例程。
- 几个内置例程需要一个FILE变量作为参数，例如 BYTES_LEFT,CLR_IO_STAT,GET_FILE_POS,IO_STATUS,SET_FILE_POS。
- FILE变量具有以下限制：
  - FILE变量必须是静态变量。
  - 永远不会保存FILE变量。
  - FILE变量不能是函数返回值。
  - 结构中不允许使用FILE类型，但允许在数组中使用。
  - 不允许使用此可变数据类型，包括相互分配。

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

A.7.2 FILE_LIST 内置程序

用途：生成指定设备上具有指定名称和类型的文件列表。

语法：FILE_LIST(file_spec, n_skip, format, ary_nam, n_files, status)

Input/Output 参数：

[in] file_spec :STRING

[in] n_skip :INTEGER

[in] format :INTEGER

[out] ary_nam :ARRAY of STRING

[out] n_files :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :BYNAM

细节：

- *file_spec* 指定要查找的文件列表的设备，名称和类型。可以使用通配符（*）字符指定 *file_spec*。
- 当第一次使用FILE_LIST时，如果存在的文件多于声明的ary_nam .Setn_skip长度为0，则使用 *n_skip*。如果 *ary_nam* 完全填充了变量名，请将该数组复制到另一个ARRAY STRING并再次执行FILE_LIST，其中 *n_skip* 等于 *n_files*。第二次调用FILE_LIST将跳过第一次传递中找到的文件，只找到剩余的文件。
- *format* 指定文件名和文件类型的格式。以下值对格式有效：
  仅1个 *file_name*，没有空白2仅 *file_type*，没有空白3 file_name.file_type，没有空白4 filename.ext大小日期时间总长度为40个字符。
  - 文件名以字符1开头。
  - *file_type* (扩展名)以字符10开头。
  - 大小以字符21开始。
  - 日期从26号字符开始。
  - 时间从字符36开始。

  仅当设备支持时间戳时才返回日期和时间; 否则只存储filename.ext大小。
- *ary_nam* 是存储文件名的 STRING 的 ARRAY。 如果 *ary_nam* 的字符串长度不足以存储格式化信息，则将返回错误。
- *n_files* 是存储在 *ary_name* 中的文件数。

- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：VAR_LIST，PROG_LIST内置过程

示例：有关详细的程序示例，请参见第B.7节 “列出文件和程序以及操作字符串” （LIST_EX.KL）。

A.7.3 FOR...ENDFOR 声明

用途：基于 INTEGER 计数器的循环构造

语法：FOR count = initial || TO | DOWNTO || final

DO{stmnt} ENDFOR

其中：

[in]count : INTEGER 变量

[in]initial : INTEGER 表达式

[in]final : INTEGER 表达式

[in]stmnt : 可执行的 KAREL 语句

细节：

- 最初，*count* 设置为 *initial* 和 *final* 的值。对于每次迭代，将计数与 *final* 进行比较。
- 如果使用 TO，则每次循环迭代计数递增。
- 如果使用 DOWNTO，则每次循环迭代都会递减计数。
- 如果 *count* 大于使用 TO 的 *final*，则永远不会执行 *stmnt*。
- 如果 *count* 小于 *final* 使用 DOWNTO，则 *stmnt* 永远不会在第一次迭代时执行。
- 如果比较在第一次迭代时没有失败，则FOR循环将执行等于ABS（最终 - 初始l）+ 1的次数。
- 如果 *final* = *initial*，则循环执行一次。
- 在进入循环之前评估初始值。因此，在循环执行期间更改初始值和最终值不会影响执行的迭代次数。
- 退出循环时 *count* 的值未初始化。
- 永远不要在 FOR 循环中发出 GO TO语句。如果 GO TO 语句导致程序退出FOR循环，则程序可能会因 “运行时堆栈溢出” 错误而中止。
- 切勿在 FOR 循环中包含 GO TO 标签。通过 GO TO 语句输入FOR循环通常会导致程序在遇到 ENDFOR 语句时因 “运行时堆栈下溢” 错误而中止。
- 如果 *count* 是系统变量或ARRAY元素，则不会翻译程序。

- 切勿在 FOR 循环中包含 GO TO 标签。 通过 GO TO 语句输入 FOR 循环通常会导致程序在遇到 ENDFOR 语句时因“运行时堆栈下溢”错误而中止。
- 如果 *count* 是系统变量或 ARRAY 元素，则不会翻译程序。

另请参阅：附录E，''语法图'，以获取其他语法信息。

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.7.4 FORCE_SPMENU 内置程序

用途：强制显示指定的菜单

语法：FORCE_SPMENU(device_code, spmenu_id, screen_no)

Input/Output 参数：

[in] device_code :INTEGER

[in] spmenu_id :INTEGER

[in] screen_no :INTEGER

%ENVIRONMENT Group :pbcore

细节：

- *device_code* 指定设备，并且应该是以下预定义常量之一：

  tp_panel 示教器设备

  crt_panel CRT设备

- *spmenu_id* 和 *screen_no* 指定要强制的菜单。 以SPI_开头的预定义常量定义 *spmenu_id*，以SCR_开头的预定义常量定义 *screen_no*。 如果未列出SCR_，请使用1。

  SPI_TPHINTS — 实用程序提示

  SPI_TPPRGADJ — UTILITIES Prog Adjust

  SPI_TPMIRROR — 实用镜像

  SPI_TPSHIFT — 实用程序转换

  SPI_TPTSTRUN — 测试周期

  SPI_TPMANUAL, SCR_MACMAN — 手动宏

  SPI_TPOTREL — 手动OT版本

  SPI_TPALARM, SCR_ALM_ALL — 报警报警日志

  SPI_TPALARM, SCR_ALM_MOT — ALARM运动日志

  SPI_TPALARM, SCR_ALM_SYS — ALARM系统日志

  SPI_TPALARM, SCR_ALM_APPL — ALARM Appl Log

  SPI_TPDIGIO — I/O数字

  SPI_TPANAIO — I/O模拟

  SPI_TPGRPIO — I/O组

  SPI_TPROBIO — I/O机器人

  SPI_TPUOPIO — I/O UOP

  SPI_TPSOPIO — I/O SOP

  SPI_TPPLCIO — I/O PLC

  SPI_TPSETGEN — 设置常规

  SPI_TPFRAM — SETUP坐标系

  SPI_TPPORT — SETUP端口初始化

  SPI_TPMMACRO, SCR_MACSETUP — 设置宏

  SPI_TPREFPOS — 设置参考位置

SPI_TPPWORD — 设置密码

SPI_TPHCCOMM — 通信设置

SPI_TPSYRSR — 设置RSR/PNS

SPI_TPFILS — 文件

SPI_TPSTATUS, SCR_AXIS — 状态轴

SPI_TPMEMORY — 状态存储器

SPI_TPVERSN — 状态版本ID

SPI_TPPRGSTS — 状态程序

SPI_TPSFTY — 状态安全信号

SPI_TPUSER — USER

SPI_TPSELECT — 选择

SPI_TPTCH — 编辑

SPI_TPREGIS, SCR_NUMREG — 数据寄存器

SPI_SFMPREG, SCR_POSREG — 数据位置寄存器

SPI_TPSYSV, SCR_NUMVAR — DATA KAREL Vars

SPI_TPSYSV, SCR_POSVAR — DATA KAREL Posns

SPI_TPPOSN — POSITION

SPI_TPSYSV, SCR_CLOCK — 系统时钟

SPI_TPSYSV, SCR_SYSVAR — 系统变量

SPI_TPMASCAL — SYSTEM Master / Cal

SPI_TPBRKCTR — SYSTEM Brake Cntrl

SPI_TPAXLM — 系统轴限制

SPI_CRTKCL, SCR_KCL — KCL>（仅限crt_panel）

SPI_CRTKCL, SCR_CRT — KAREL EDITOR（仅限

crt_panel）

SPI_TPUSER2 — 表单/表格管理器菜单

另请参阅：ACT_SCREEN内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.4节“标准例程”（ROUT_EX.KL）

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.12节“字典文件”（DCLISTEG.UTX）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.7.5 FORMAT_DEV 内置程序

用途：删除任何现有信息，并在指定设备上记录目录和其他内部信息。

语法：FORMAT_DEV(device, volume_name, nowait_sw, status)

Input/Output 参数：

[in] device :STRING

[in] volume_name :STRING

[in] nowait_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *device* 指定要初始化的设备。
- *volume_name* 充当特定存储介质单元的标签。 *volume_name* 最多可以包含11个字符，如果指定了更多，则将截断为11个字符。
- •如果 *nowait_sw* 为TRUE，则在执行命令时继续执行程序。 如果为FALSE，程序将停止，包括条件处理程序，直到操作完成。 如果程序中有时间紧急的条件处理程序，则将它们放在另一个作为单独任务执行的程序中。
- 注意 *nowait_sw* 在此版本中不可用，应设置为FALSE。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：MOUNT_DEV，DISMOUNT_DEV内置过程

示例：有关详细的程序示例，请参见第B.9节“使用文件和设备内置函数”（FILE_EX.KL）。

A.7.6 FRAME 内置函数

用途：返回一个POSITION数据类型的帧，表示由三个（或四个）POSITION参数指定的坐标系的转换。

语法：FRAME(pos1, pos2, pos3 <,pos4>)

函数返回类型：Position

Input/Output 参数：

[in]pos1 :POSITION

[in]pos2 :POSITION

[in]pos3 :POSITION

[in]pos4 :POSITION

%ENVIRONMENT Group :SYSTEM

细节：

- 返回值的计算方法如下：

  — 除非提供 *pos4* 参数，否则假定 *pos1* 为原点。 见图A-2。

  — 如果提供了*pos4*，原点移动到 *pos4*，新坐标框在空间中保持与第一个坐标系相同的方向。见图A-2。

  — x轴平行于从 *pos1* 到 *pos2* 的直线。

  — xy平面定义为包含 *pos1*，*pos2* 和 *pos3* 的平面，*pos3* 位于平面的正半部分。

  — y轴垂直于x轴和xy平面。

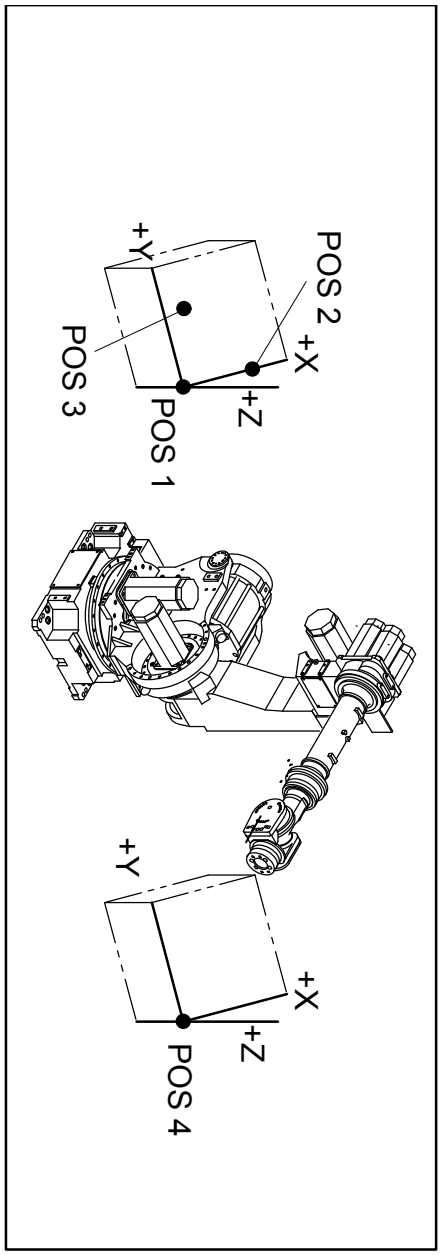
  — z轴通过*pos1*并垂直于xy平面。 正方向由右手规则决定。

  — 结果的配置设置为*pos1*或*pos4*的配置（如果提供）。

- *pos1* 和 *pos2* 参数必须至少相距10毫米，pos3必须距离连接pos1和pos2的线至少10毫米。

  如果不满足任一条件，程序将暂停并显示错误。

图A-2. FRAME 内置函数



示例：以下示例允许操作员将坐标系设置为托盘，以便码垛程序能够沿托盘坐标系中的x，y，z方向移动TCP。

FRAME 内置函数

```
WRITE('Teach corner_1, corner_2, corner_3',CR)
  RELEASE --允许操作员打开示教器并示教位置
```

```
ATTACH  --将运动控制返回到程序
$UFRAME = FRAME (corner_1, corner_2, corner_3)
```

A.7.7 FROM 条件

用途：表示程序外部的变量或例程，允许在程序之间共享数据和/或例程

语法：FROM prog_name

其中：

prog_name：any KAREL program identifier

细节：

- FROM子句可以是类型，变量或例程声明的一部分。
- 类型，变量或例程属于 *prog_name* 指定的程序。
- 在FROM子句中，*prog_name* 可以是任何程序的名称，包括声明类型，变量或例程的程序。
- 如果在例程声明中使用FROM子句并在程序执行期间调用，则声明的主体必须出现在指定的程序中，并且必须加载该程序。
- 在例程的声明部分中声明变量时，不能使用FROM子句。

示例：有关详细的程序示例，请参阅以下部分：

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.8 - G - KAREL 语言描述

A.8.1 GET_ATTR_PRG 内置程序

用途: 从指定的示教器或KAREL程序获取属性数据

语法 : GET_ATTR_PRG(program_name, attr_number, int_value, string_value, status)

Input/Output 参数 :

[in] program_name :STRING

[in] attr_number :INTEGER

[out] int_value :INTEGER

[out] string_value :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节 :

- *program_name* 指定从中获取属性的程序。
- *attr_number* 是要返回值的属性。 以下属性是有效的:

  AT_PROG_TYPE : 程序类型

  AT_PROG_NAME : 程序名称 (String[12])

  AT_OWNER : Owner (String[8])

  AT_COMMENT : 注释 (String[16])

  AT_PROG_SIZE : 程序的大小

  AT_ALLC_SIZE : 分配内存的大小

  AT_NUM_LINE : 行数

  AT_CRE_TIME : 创建（加载）时间

  AT_MDFY_TIME : 修改时间

  AT_SRC_NAME : 源文件（或原始文件）名称 (String[128])

AT_SRC_VRSN：源文件版本A.

AT_DEF_GROUP：默认运动组掩码（用于任务属性）。 见表A-16。

AT_PROTECT：保护代码; 1：保护关闭; 2：AT_STORAGE保护：存储类型;

TPSTOR_CMOS;TPSTOR_SHADOW;TPSTOR_FILE;TPSTOR_SHOD

AT_STK_SIZE：堆栈大小（用于任务属性）

AT_TASK_PRI：任务优先级（用于任务属性）

AT_DURATION：时间片持续时间（对于任务属性）

AT_BUSY_OFF：忙灯关闭（用于任务属性）

AT_IGNR_ABRT：忽略中止请求（对于任务属性）

AT_IGNR_PAUS：忽略暂停请求（对于任务属性）

AT_CONTROL：控制代码（用于任务属性）

- 为 AT_PROG_TYPE 返回的程序类型将是以下常量之一：

  PT_KRLPRG：Karel 程序

  PT_MNE_UNDEF：未定义子类型的示教器程序

  PT_MNE_JOB：示教器 job

  PT_MNE_PROC：示教器 process

  PT_MNE_MACRO：示教器 宏

- 如果属性数据是一个数字，它将返回 *int_value*，并且 *string_value* 不会被修改。
- 如果属性数据是一个字符串，它将返回到 *string_value* 中，并 *int_value* 不被修改。
- *status* 解释了尝试操作的状态。 如果它不等于 0，则发生错误。 可能发生的一些错误是：

  7073 *program_name* 中指定的程序不存在。

  17027 *string_value* 不足以包含属性字符串。 该值已被截断以适合。

  17033 *attr_number* 具有非法值。

另请参见：SET_ATTR_PRG, GET_TSK_INFO, SET_TSK_ATTR 内置程序

A.8.2 GET_FILE_POS 内置函数

用途：返回指定文件中的当前文件位置（下一个读取或写入操作发生的位置）

语法： GET_FILE_POS(file_id)

函数返回类型：INTEGER

Input/Output 参数：

[in] file_id :FILE

%ENVIRONMENT Group :FLBT

细节：

- GET_FILE_POS 返回要读取或写入文件的下一个字节之前的字节数。
- 行结束符在返回的值中计数。
- 与 *file_id* 关联的文件必须是打开的。 否则，程序会中止并出现错误。
- 如果与 *file_id* 关联的文件以只读方式打开，则它不能作为压缩文件存在于 FROM 或 RAM 磁盘上。

> **警告**
> **只有在 RAM磁盘 设备上打开的文件才支持 GET_FILE_POS。 不要在其他设备上使用 GET_FILE_POS; 否则，您可能会伤害人员并损坏设备。**

示例：以下示例打开 filepos.dt 数据文件，将位置存储在文件中的 my_path 中，并构建一个目录以访问它们。

GET_FILE_POS 内置函数

```
OPEN FILE file_id ('RW','filepos.dt')
  FOR i = 1 TO PATH_LEN(my_path) DO
    temp_pos = my_path[i].node_pos
    pos_dir[i] = GET_FILE_POS(file_id)
    WRITE file_id (temp_pos)
  ENDFOR
```

A.8.3 GET_JPOS_REG 内置函数

用途：从指定的寄存器获取 JOINTPOS 值

语法：GET_JPOS_REG(register_no, status <,group_no>)

函数返回类型：REGOPE

Input/Output 参数：

[in] register_no :INTEGER

[out] status :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的位置寄存器。
- 如果省略 *group_no*，则假定该程序的默认组。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组的总数的范围内。
- GET_JPOS_REG 以 JOINTPOS 格式返回位置。 使用 POS_REG_TYPE 来确定位置表示。
- *status* 解释了尝试操作的状态。 如果不等于 0，则发生错误。

另请参阅：GET_POS_REG，SET_JPOS_REG，SET_POS_REG 内置过程

示例：有关详细的程序示例，请参见第B.5节 “使用寄存器内置程序”（REG_EX.KL）

A.8.4 GET_JPOS_TPE 内置函数

用途：从指定示教程序中的指定位置获取 JOINTPOS 值。

语法：GET_JPOS_TPE(open_id, position_no, status <, group_no>)

函数返回类型：JOINTPOS

Input/Output 参数：

[in] open_id :INTEGER

[in] position_no :INTEGER

[out] status :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定示教程序。 在调用此内置函数之前必须打开一个程序。
- *position_no* 指定要获取的程序中的位置。
- 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。
- 没有对位置表示进行转换。 位置数据必须是JOINTPOS格式。 如果存储的位置不在JOINTPOS中，则返回错误状态。 使用GET_POS_TYP获取位置表示。
- 如果程序中的指定位置未初始化，则返回的JOINTPOS值未初始化，状态设置为17038，“未初始化的TPE位置”。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：SET_JPOS_TPE，GET_POS_TPE，SET_POS_TPE内置函数

示例：有关详细的程序示例，请参见第B.14节“对复制的示教器程序应用偏移”（CPY_TPE.KL）。

A.8.5 GET_PORT_ASG 内置程序

用途：允许KAREL程序确定分配了指定逻辑端口的物理端口。

语法：GET_PORT_ASG(log_port_type, log_port_no, rack_no, slot_no, phy_port_type, phy_port_no, n_ports, status)

Input/Output 参数：

[in] log_port_type :INTEGER

[in] log_port_no :INTEGER

[out] rack_no :INTEGER

[out] slot_no :INTEGER

[out] phy_port_type :INTEGER

[out] phy_port_no :INTEGER

[out] n_ports :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *log_port_type* 指定正在访问其分配的端口类型的代码。代码在FR中定义：KLIOTYPS.KL。
- *log_port_no* 指定正在访问其分配的端口号。
- 随机架包含端口模块返回 *rack_no*。对于过程I / O板，存储器映像和虚拟端口，这是零;对于Allen-Bradley 端口，这是16。
- 返回 *phy_port_type*，并指定分配的端口类型。通常这与 *log_port_type* 相同。例外情况是 *log_port_type* 是组类型（io_gpin或io_gpout）还是将端口分配给内存映像或虚拟端口。
- 返回 *phy_port_no*，其中包含分配给的端口号。如果 *log_port_type* 是一个组，则这是该组的最低有效位的端口号。
- 返回 *n_ports*，其中包含分配给逻辑端口的物理端口数。除非 *log_port_type* 是组类型，否则在所有情况下都是1。在这种情况下，*n_ports* 表示组中的位数。
- 如果参数有效并且指定了指定的端口，则 *status* 返回零。否则，将返回错误代码。

示例：以下示例向调用方返回模块机架和插槽号，*port_number* 以及分配给指定组输入端口的位数。 返回一个布尔值，指示端口是否分配给DIN端口。 如果未分配端口，则返回非零状态。

GET_PORT_ASG 内置程序

```
PROGRAM getasgprog
  %ENVIRONMENT IOSETUP
  %INCLUDE FR:\kliotyps
     ROUTINE get_gin_asg(gin_port_no: INTEGER;
                         rack_no: INTEGER;
                         slot_no: INTEGER;
                         frst_port_no: INTEGER;
                         n_ports: INTEGER;
                         asgd_to_din: BOOLEAN): INTEGER
       VAR
```

```
        phy_port_typ: INTEGER
        status: INTEGER

      BEGIN
      GET_PORT_ASG(io_gpin, gin_port_no, rack_no, slot_no,
                   phy_port_typ, frst_port_no, n_ports, status)
      IF status <> 0 THEN
        RETURN (status)
      ENDIF
      asgd_to_din = (phy_port_typ = io_din)
      END get_gin_asg
BEGIN
END getasgprog
```

A.8.6 GET_PORT_ATR 内置函数

用途：从指定端口获取属性

语法：GET_PORT_ATR(port_id, atr_type, atr_value)

函数返回类型：INTEGER

Input/Output 参数：

[in] port_id :INTEGER

[in] atr_type :INTEGER

[out] atr_value :INTEGER

%ENVIRONMENT Group :FLBT

细节：

- *port_id* 指定要查询的端口。 使用以下预定义常量之

  一：port_1

  port_2

  port_3

  port_4

  port_5

- *atr_type* 指定要返回其当前设置的属性。 使用以下预定义常量之一：

  atr_readahd：预读缓冲区

  atr_baud：波特率

  atr_parity：奇偶校验

  atr_sbits：停止位

  atr_dbits：数据长度

  atr_xonoff：Xon / Xoff

  atr_eol：行尾

  atr_modem：调制解调器线

  atr_value 接收指定属性的当前值。

- GET_PORT_ATR 将此操作的状态返回给端口。

另请参见：SET_PORT_ATR内置函数，第7章 文件输入/输出操作

示例：以下示例将端口设置为所需的配置（如果尚未设置为指定的配置）。

GET_PORT_ATR 内置函数

```
PROGRAM port_atr
%ENVIRONMENT FLBT
VAR
 stat:    INTEGER
 atr_value: INTEGER
BEGIN
-- 将预读缓冲区设置为所需的值，如果尚未正确的话
 stat=GET_PORT_ATR(port_2,atr_readahd,atr_value)
 IF(atr_value <> 2) THEN
  stat=SET_PORT_ATR(port_2,atr_readahd,2) --设置为256个字节
ENDIF
-- 如果尚未设置，则将波特率设置为9600
 stat=GET_PORT_ATR(port_2,atr_baud,atr_value)
 IF(atr_value <> BAUD_9600) THEN
  stat=SET_PORT_ATR(port_2,atr_baud,baud_9600)
ENDIF
-- 如果尚未设置，则将奇偶校验设置为偶数
 stat=GET_PORT_ATR(port_2,atr_parity,atr_value)
 IF(atr_value <> PARITY_EVEN) THEN
  stat=SET_PORT_ATR(port_2,atr_parity,PARITY_EVEN)
```

```
 ENDIF
-- 如果尚未设置，则将停止位设置为1
 stat=GET_PORT_ATR(port_2,atr_sbits,atr_value)
 IF(atr_value <> SBITS_1) THEN
  stat=SET_PORT_ATR(port_2,atr_sbits,SBITS_1)
 ENDIF
-- 如果尚未设置，则将数据位设置为5
 stat=GET_PORT_ATR(port_2,atr_dbits,atr_value)
 IF(atr_value <> DBITS_5) THEN
  stat=SET_PORT_ATR(port_2,atr_dbits,DBITS_5)
 ENDIF
-- 将 xonoff 设置为未使用，如果尚未设置
 stat=GET_PORT_ATR(port_2,atr_xonoff,atr_value)
 IF(atr_value <> xf_not_used) THEN
  stat=SET_PORT_ATR(port_2,atr_xonoff,xf_not_used)
ENDIF
-- 设置行结束标记，如果尚未设置
 stat=GET_PORT_ATR(port_2,atr_eol,atr_value)
 IF(atr_value <> 65) THEN
  stat=SET_PORT_ATR(port_2,atr_eol,65)
 ENDIF
END port_atr
```

A.8.7 GET_PORT_CMT 内置程序

用途：允许KAREL程序确定为指定逻辑端口设置的注释

语法：GET_PORT_CMT(port_type, port_no, comment_str, status)

Input/Output 参数：

[in] port_type :INTEGER

[in] port_no :INTEGER

[out] comment_str :STRING

[out] status :INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定要返回其注释的端口类型的代码。 代码在FR中定义：KLIOTYPS.KL。

- *port_no* 指定正在设置其注释的端口号。
- 将返回 *comment_str* 以及指定端口的注释。 这应该声明为长度至少为16个字符的 STRING。
- 如果参数有效并且为指定端口返回注释，则 *status* 返回零。

另请参见：GET_PORT_VAL,GET_PORT_MOD,SET_PORT_CMT,SET_PORT_VAL,SET_PORT_MOD 内置过程。

A.8.8 GET_PORT_MOD 内置程序

用途：允许KAREL程序确定为指定的逻辑端口设置的特殊端口模式

语法：GET_PORT_MOD(port_type, port_no, mode_mask, status)

Input/Output 参数：

[in] port_type :INTEGER

[in] port_no :INTEGER

[out] mode_mask :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定要返回其模式的端口类型的代码。 代码在FR中定义：KLIOTYPS.KL。
- *port_no* 指定正在设置其模式的端口号。
- 返回 *mode_mask*，并带有一个掩码，指定打开哪些模式。 定义了以下模式：

  1：反向模式

  港口意识逆转; 如果端口设置为TRUE，则物理输出设置为FALSE。 如果端口设置为FALSE，则物理输出设置为TRUE。 如果物理输入为TRUE，则在读取端口时，将返回FALSE。 如果物理输入为FALSE，则在读取端口时，将返回TRUE。

  2：互补模式

逻辑端口分配给两个物理端口，其值互补。 在这种情况下，*port_no* 必须是奇数。 如果端口n设置为TRUE，则端口n设置为TRUE，端口n + 1设置为FALSE。 如果端口n设置为FALSE，则端口n设置为FALSE，端口n + 1设置为TRUE。 这仅对输出端口有效。

- 如果参数有效并且为指定端口返回指定的模式，则 *status* 返回零。

示例：以下示例获取指定端口的模式。

GET_PORT_MOD_内置程序

```
PROGRAM getmodprog
%ENVIRONMENT IOSETUP
%INCLUDE FR:\kliotyps
 ROUTINE get_mode( port_type:  INTEGER;
       port_no:   INTEGER;
       reverse:   BOOLEAN;
       complementary: BOOLEAN): INTEGER
  VAR
   mode:  INTEGER
   status: INTEGER
  BEGIN
   GET_PORT_MOD(port_type, port_no, mode, status)
   IF (status <>0) THEN
    RETURN (status)
   ENDIF
   IF (mode AND 1) <> 0 THEN
    reverse = TRUE
   ELSE
    reverse = FALSE
   ENDIF
   IF (mode AND 2) <> 0 THEN
    complementary = TRUE
   ELSE
    complementary = FALSE
   ENDIF
   RETURN (status)
  END get_mode
 BEGIN
 END getmodprog
```

A.8.9 GET_PORT_SIM 内置程序

用途：获取端口模拟状态

语法：GET_PORT_SIM(port_type, port_no, simulated, status)

Input/Output 参数：

[in] port_type :INTEGER

[in] port_no :INTEGER

[out] simulated :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定要获取的端口类型的代码。 代码在FRS中定义：KLIOTYPS.KL。
- *port_no* 指定返回模拟状态的端口号。
- 如果正在模拟端口，则模拟返回TRUE，否则返回FALSE。
- 如果端口有效，则 *status* 返回零。

另请参见：GET_PORT_MOD，SET_PORT_SIM，SET_PORT_MOD内置过程。

A.8.10 GET_PORT_VAL 内置程序

用途：允许KAREL程序确定指定逻辑端口的当前值

语法：GET_PORT_VAL(port_type, port_no, value, status)

Input/Output 参数：

[in] port_type :INTEGER

[in] port_no :INTEGER

[out] value :STRING

[out] status :INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定要返回其注释的端口类型的代码。 代码在FR中定义：KLIOTYPS.KL。

- *port_no* 指定正在设置其注释的端口号。
- 返回值与指定端口的当前值（状态）。 对于BOOLEAN端口类型（即DIN），这将是0 = OFF，或1 = ON。
- 如果参数有效并且为指定端口返回注释，则返回零状态。

另请参见：GET_PORT_CMT,GET_PORT_MOD,SET_PORT_CMT,SET_PORT_VAL,SET_PORT_MOD 内置过程。

A.8.11 GET_POS_FRM 内置程序

用途：获取指定示教程序中指定位置的 uframe 编号和 utool 编号。

语法：GET_POS_FRM(open_id, position_no, gnum, ufram_no, utool_no, status)

Input/Output 参数：

[in] open_id :INTEGER

[in] position_no :INTEGER

[in] gnum :INTEGER

[out] ufram_no :INTEGER

[out] utool_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :pbcore

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前必须打开一个程序。
- *position_no* 指定示教程序中的位置。
- *gnum* 指定位置的组号。
- 返回 *ufram_no*，其坐标系号为 *position_no*。
- 返回 *utool_no*，其工具号为 *position_no*。
- 如果指定的位置 *position_no* 未初始化，则状态设置为17038，“未初始化的TPE位置”。
- *status* 表示尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_POS_TYP，CHECK_EPOS。

A.8.12 GET_POS_REG 内置函数

用途: 从指定的寄存器获取XYZWPR值

语法： GET_POS_REG(register_no, status <,group_no>) 函

数返回类型：XYZWPREXT

Input/Output 参数：

[in] register_no :INTEGER

[out] status :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的位置寄存器。
- 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。
- GET_POS_REG 以 XYZWPREXT 格式返回位置。 使用 POS_REG_TYPE 确定位置表示。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见： GET_JPOS_REG，SET_JPOS_REG，SET_POS_REG，GET_REG内置过程。

示例： 有关详细的程序示例，请参见第B.5节“使用寄存器内置函数”（REG_EX.KL）。

A.8.13 GET_POS_TPE 内置函数

用途： 从指定示教程序中的指定位置获取 XYZWPREXT 值

语法： GET_POS_TPE(open_id, position_no, status <, group_no>)

数返回类型：XYZWPREXT

Input/Output 参数：

[in] open_id : INTEGER

[in] position_no : INTEGER

[out] status : INTEGER

[in] group_no : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前必须打开一个程序。
- *position_no* 指定要获取的程序中的位置。
- 未对位置表示进行转换。 位置数据必须是 XYZWPR 或 XYZWPREXT，否则返回错误状态。 使用 GET_POS_TYP 获取位置表示。
- 如果程序中的指定位置未初始化，则返回的 XYZWPR 值未初始化，状态设置为 17038，“未初始化的TPE位置”。
- 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_JPOS_TPE,SET_JPOS_TPE,SET_POS_TPE,GET_POS_TYP内置过程。

示例：请参见第B.14节 “将偏移应用于复制的示教器程序”（CPY_TP.KL），详细的程序示例。

A.8.14 GET_POS_TYP 内置程序

用途：获取指定示教程序中指定位置的位置表示

语法：GET_POS_TYP(open_id, position_no, group_no, posn_typ, num_axs, status)

Input/Output 参数：

[in] open_id :INTEGER

[in] position_no :INTEGER

[in] group_no :INTEGER

[out] posn_typ :INTEGER

[out] num_axs :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前必须打开一个程序。
- *position_no* 指定程序中的位置。
- *group_no* 指定组号。
- 位置类型由 *posn_typ* 返回。 *posn_typ* 定义如下：2:XYZWPR 6:XYZWPREXT 9:JOINTPOS
- 如果它处于关节位置，则表示中的轴编号由 *num_axs* 返回。
- 如果程序中的指定位置未初始化，则状态设置为17038，“未初始化的TPE位置”。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

示例：有关详细的程序示例，请参见第B.14节“对复制的示教器程序应用偏移”（CPY_T.KL）。

A.8.15 GET_PREG_CMT 内置程序

用途：根据给定的寄存器号检索KAREL位置寄存器的注释信息。

语法：GET_PREG_CMT (register_no, comment_string, status)

Input/Output 参数：

[in] register_no: INTEGER

[out] comment_string: STRING

[out] status: INTEGER

%ENVIORNMENT group: REGOPE

细节：

- *register_no* 指定从哪个位置寄存器中检索注释。 给定位置寄存器的注释在 *comment_string* 中返回。

A.8.16 GET_QUEUE 内置程序

用途：从队列中检索指定的最旧条目

语法：GET_QUEUE(queue, queue_data, value, status, sequence_no)

Input/Output 参数：

[in,out] queue_t :QUEUE_TYPE

[in,out] queue_data :ARRAY OF INTEGER

[out] value :INTEGER

[out] sequence_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBQMGR

细节：

- *queue_t* 指定要从中获取值的队列的队列变量。
- *queue_data* 指定包含队列数据的数组变量。
- 返回值，从队列中获取最旧的条目。
- 返回 *sequence_no*，其中包含返回条目的序列号。
- 如果从队列中成功获取条目，则返回状态为零。 否则，返回值61002，''Queue为空，''。

另请参阅：MODIFY_QUEUE 内置过程，第15.8节 “使用队列进行任务通信

示例：在以下示例中，例程 *get_nxt_err* 返回错误队列中最旧的条目，如果队列为空，则返回零。

GET_QUEUE 内置程序

```
PROGRAM get_queue_x
  %environment PBQMGR
  VAR
    error_queue FROM global_vars: QUEUE_TYPE
    error_data FROM global_vars: ARRAY[100] OF INTEGER
```

```
ROUTINE get_nxt_err: INTEGER
VAR
  status: INTEGER
  value: INTEGER
  sequence_no: INTEGER

BEGIN
GET_QUEUE(error_queue, error_data, value, sequence_no, status)
IF (status = 0) THEN
  RETURN (value)
ELSE
  RETURN (0)
ENDIF
END get_nxt_err
BEGIN
END get_queue_x
```

A.8.17 GET_REG 内置程序

用途：从指定的寄存器获取 INTEGER 或 REAL 值

语法： GET_REG(register_no, real_flag, int_value, real_value, status)

Input/Output 参数：

[in] register_no :INTEGER

[out] real_flag :BOOLEAN

[out] int_value :INTEGER

[out] real_value :REAL

[out] status :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的寄存器。
- 如果指定的寄存器具有实际值，则 *real_flag* 被设置为 TRUE 并且对寄存器内容赋予 *real_value*。 否则，将 *real_flag* 设置为 FALSE，并将 *int_value* 设置为寄存器的内容。
- *status* 解释了尝试操作的状态。 如果不等于 0，则发生错误。

例如：有关详细的程序示例，请参阅第B.5节“使用寄存器内置程序”（REG_EX.KL）。

A.8.18 GET_REG_CMT

用途：根据给定的寄存器号检索KAREL寄存器的注释信息。

语法：GET_REG_CMT (register_no, comment_string, status)

Input/Output 参数：

[in] register_no: INTEGER

[out] comment_string: STRING

[out] status: INTEGER

%ENVIRONMENT group: REGOPE

细节：

- *register_no* 指定从哪个寄存器检索注释。 给定寄存器的注释在 *comment_string* 中返回。

A.8.19 GET_SREG_CMT 内置程序

用途：从指定的字符串寄存器中获取注释。

语法：GET_SREG_CMT(register_no, comment, status)

Input/Output 参数：

[in] register_no :INTEGER

[out] comment :STRING[254]

[out] status :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的字符串寄存器。
- 注释包含指定字符串寄存器的注释。

- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_STR_REG，SET_STR_REG，SET_SREG_CMT内置过程。

A.8.20 GET_STR_REG 内置程序

用途：从指定的字符串寄存器中获取值。

语法： GET_STR_REG(register_no, value, status)

Input/Output 参数：

[in] register_no :INTEGER

[out] value :STRING[254]

[out] status :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的字符串寄存器。
- *value* 包含指定字符串寄存器的值。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_SREG_CMT，SET_STR_REG，SET_SREG_CMT内置过程。

A.8.21 GET_TIME 内置程序

用途：从 KAREL系统 中检索当前时间（以整数表示）

语法： GET_TIME(i)

Input/Output 参数：

[out] i :INTEGER

%ENVIRONMENT Group :TIM

细节：

- 我拥有存储在 KAREL系统中的当前时间的 INTEGER 表示。 该值以32位 INTEGER 格式表示如下：

表_A–11. __INTEGER 表示当前时间

31–25	24–21	20–16
year	month	day
15–11	10–5	4–0
hour	minute	second

- 各个字段的内容如下:
  - — DATE:

    Bits 31-25 — 自1980年起

    Bits 24-21 — 月（1-12）

    Bits 20-16 — 每月的一天
  - — TIME:

    Bits 15-11 — 小时数 (0-23)

    Bits 10-5 — 分钟数 (0-59)

    Bits 4-0 — 2秒增量 (0-29)
- 可以比较 INTEGER 值以确定一次是否比另一次更新。
- 使用 CNV_TIME_STR 内置程序将 INTEGER 转换为''DD-MMM-YYY HH：MM：SS''STRING 格式。

另请参见：CNV_TIME_STR 内置过程

例如：有关详细的程序示例，请参见第B.9节 “使用文件和设备内置程序”（FILE_EX.KL）。

A.8.22 GET_TPE_CMT 内置程序

用途：该内置功能使 KAREL程序 能够读取与示教器程序中指定位置相关的注释。

语法：GET_TPE_CMT(open_id, pos_no, comment, status)

Input/Output 参数：

[in] open_id :INTEGER

[in] pos_no :INTEGER

[out] comment :STRING

[out] status :INTEGER

%ENVIRONMENT Group :TPE

细节：

- *open_id* 指定从先前调用 OPEN_TPE 返回的 *open_id*。
- *pos_no* 指定要从中获得评论的TPP程序中的位置编号。
- 注释与指定的位置相关联，如果位置没有注释，则返回零长度字符串。 如果字符串变量对于注释太短，则返回错误并且字符串不会更改。
- 如果操作成功，状态指示为零，否则将显示错误代码。

另请参阅：SET_TPE_CMT 和 OPEN_TPE 以获取更多内置过程。

A.8.23 GET_TPE_PRM 内置程序

用途：当参数在 TPE CALL 或 MACRO 指令中传递时，获取参数的值。

语法：GET_TPE_PRM(param_no, data_type, int_value, real_value, str_value, status)

Input/Output 参数：

[in] param_no :INTEGER

[out] data_type :INTEGER

[out] int_value :INTEGER

[out] real_value :REAL

[out] str_value :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *param_no* 表示参数的编号。 最多可以有十个参数。
- *data_type* 表示参数的数据类型，如下所示：

— 1：INTEGER

— 2：REAL

— 3：STRING

- 如果 *data_type* 是 INTEGER，则 *int_value* 是参数的值。
- 如果 *data_type* 为 REAL，则 *real_value* 是参数的值。
- 如果 *data_type* 是 STRING，*str_value* 是参数的值。
- 状态解释了尝试操作的状态。 如果不等于 0，则发生错误。
- 如果 *param_no* 指定的参数不存在，则返回状态17042，即错误消息： “ROUT-042 WARN TPE 参数不存在” 。 如果返回此错误，请确认主TPE程序中的 CALL或MACRO 命令中的 *param_no* 和参数。

另请参阅：特定应用FANUC机器人设置和操作手册，了解有关在示教器 CALL或MACRO 指令中使用参数的信息。

示例：以下示例显示了使用内置 GET_TPE_PRM 的 KAREL程序 检索的具有CALL参数的宏（发送事件）的实现。

GET_TPE_PRM 内置程序

```
Macro table entry for the Send Event macro:
109 [Send Event             ] [SENDEVNT]--[  0]
Teach pendant program, TEST1.TP, which uses the Send Event
macro:
1:  ! Send Event 7
   2:  ! Wait for PC answer
   3:  ! Answer in REG 5
   4:  Send Event(7,1,5)
   5:  IF R[5]<9999,JMP LBL[10]
   6:  ! Error in macro
   7:  !
   8:  LBL[10]
Teach pendant program SENDEVNT.TP, which implements the
Send Event macro by calling the GESNDEVT KAREL
program and passing the CALL parameters from Send Event:
   1:  !Send Event Macro
   2:  CALL GESNDEVT(AR[1],AR[2],AR[3])
Snippet of the KAREL program GESNDEVT.KL, which gets the
parameter information using the GET_TPE_PRM
built-in:
 PROGRAM GESNDEVT
  . . .
 BEGIN
  --  Send Event(event_no [,wait_sw [,status_reg]] )
  -- get parameter 1 (mandatory parameter)
```

```
Get_tpe_prm(1, data_type, event_no,real_value,string_value,status)
IF status<>0 THEN  -- 17042 "ROUT-042 TPE parameters do not exist"
  POST_ERR(status, '', 0, er_abort)
ELSE
  IF data_type <> PARM_INTEGER THEN  -- make sure parm is an integer
    POST_ERR(er_pceventer, '1', 0, er_abort)
  ELSE
    IF (event_no < MIN_EVENT) OR (event_no > MAX_EVENT) THEN
      POST_ERR(er_illevent, '', 0, er_abort)
   ENDIF
  ENDIF
ENDIF

-- get second parameter (optional)
Get_tpe_prm(2, data_type, wait_sw,real_value,string_value,status)
IF status<>0 THEN
  IF status = ER17042 THEN   -- "ROUT-142 Parameter doesn't exist"
     wait_sw = 0  -- DEFAULT no wait
  ELSE
    POST_ERR(status, '', 0, er_warn) -- other error
  ENDIF
. . .
```

A.8.24 GET_TSK_INFO 内置程序

用途：获取指定任务属性的值

语法：GET_TSK_INFO(task_name, task_no, attribute, value_int, value_str, status)

Input/Output 参数：

[in,out] task_name :STRING

[in,out] task_no :INTEGER

[in] attribute :INTEGER

[out] value_int :INTEGER

[out] value_str :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *task_name* 是感兴趣的任务的名称。 只有在 *task_no* 未初始化或设置为 0 时，才将 *task_name* 用作输入，否则，将 *task_name* 视为输出参数。
- *task_no* 是感兴趣的任务编号。 如果 *task_no* 未初始化或设置为0，则将其作为输出参数返回。
- 属性是要返回其值的任务属性。 除非另有说明，否则它将在 *value_int* 中返回。 以下属性是有效的：

  TSK_HOLDCOND：任务保留条件

  TSK_LINENUM：当前执行的行号

  TSK_LOCKGRP：锁定组

  TSK_MCTL：运动控制组

  TSK_NOABORT：忽略中止请求

  TSK_NOBUSY：忙灯关闭

  TSK_NOPAUSE：忽略暂停请求

  TSK_NUMCLDS：子任务的数量

  TSK_PARENT：父任务编号

  TSK_PAUSESFT：暂停轮班

  TSK_PRIORITY：任务优先级

  TSK_PROGNAME：value_str中返回的当前程序名称

  TSK_PROGTYPE：程序类型 - 请参阅下面的说明

  TSK_ROUTNAME：value_str中返回的当前例程名称

  TSK_STACK：堆栈大小

  TSK_STATUS：任务状态 - 请参阅下面的说明

  TSK_STEP：单步任务

  TSK_TIMESLIC：以ms为单位的时间片持续时间

  TSK_TPMOTION：TP动作启用

  TSK_TRACE：跟踪启用

TSK_TRACELEN：跟踪数组的长度

- TSK_STATUS 是任务状态：返回值为：

  PG_RUNACCEPT：已接受运行请求

  PG_ABORTING：Abort已被接受

  PG_RUNNING：任务正在运行

  PG_PAUSED：任务暂停

  PG_ABORTED：任务中止

- TSK_PROGTYPE 是程序类型。 返回值为：

  PG_NOT_EXEC：程序尚未执行

  PG_MNEMONIC：教导挂件程序正在执行或正在执行

  PG_AR_KAREL：KAREL程序正在执行或正在执行

- *status* 说明尝试操作的状态。 如果它不等于0，则发生错误。

另请参见：第15章 多任务

示例：请参阅第15章“多任务”中的示例

A.8.25 GET_USEC_SUB 内置程序

用途：返回INTEGER值，指示经过的时间（以微秒为单位）（1 / 1,000,000）。

语法：us_delta = GET_USEC_SUB(us2, us1)

函数返回类型：INTEGER

Input/Output 参数：

[in] us2: INTEGER

[in] us1: INTEGER

%ENVIRONMENT Group: TIM

细节：

- *us2* 是 GET_USEC_TIM 第二次返回。
- *us1* 是第一次从 GET_USEC_TIM 返回。

- 返回值是经过时间*us2* 的 INTEGER 表示 - *us1*（以微秒为单位）。
- 这是为了测量快速操作。 结果将在2分钟后换行，并且将不再有效。

示例：以下示例测量递增数字的时间量（以微秒为单位）。

图 A–3. GET_USEC_SUB 内置函数

```
i = 0
us1 = GET_USEC_TIM
i = i + 1
us_delta = GET_USEC_SUB(GET_USEC_TIM, us1)
WRITE ('Time to increment a number: ', us_delta, ' us', CR)
```

A.8.26 GET_USEC_TIM 内置函数

用途：返回一个INTEGER值，指示KAREL系统中当前时间(以微秒为单位(1 / 1,000,000))。

语法：us = GET_USEC_TIM

函数返回类型：INTEGER

Input/Output 参数：

None

%ENVIRONMENT Group: TIM

细节：

- 返回值是存储在KAREL系统中的当前时间（以微秒为单位）的INTEGER表示。
- 此函数与 GET_USEC_SUB 内置函数一起使用，以确定操作的已用时间。

A.8.27 GET_VAR 内置程序

用途：允许KAREL程序检索指定变量的值

语法：GET_VAR(entry, prog_name, var_name, value, status)

Input/Output 参数 :

[in,out] entry :INTEGER

[in] prog_name :STRING

[in] var_name :STRING

[out] value :Any valid KAREL data type except PATH

[out] status :INTEGER

%ENVIRONMENT Group :SYSTEM

细节 :

- 条目返回 *var_name* 所在设备目录中 *var_name* 的变量数据表中的条目号。这个变量不应该被修改。
- *prog_name* 指定包含指定变量的程序的名称。如果 *prog_name* 为空，它将默认为正在执行的当前任务名称。将 *prog_name* 设置为'* SYSTEM *'以获取系统变量。 *prog_name* 也可以访问环形机器人上的系统变量。
- *var_name* 必须引用一个静态的程序变量。
- *var_name* 可以包含节点号，字段名称和/或下标。
- 如果 *var_name* 和 *value* 均为ARRAY，则复制的元素数量将等于两个数组中较小的数组的大小。
- 如果 *var_name* 和*value* 都是STRING，则复制的字符数将等于两个字符串中较小者的大小。
- 如果 *var_name* 和*value* 都是相同类型的STRUCTURE，则 *value* 将是 *var_name* 的精确副本。
- 值是 *var_name* 的值。
- 状态解释了尝试操作的状态。如果不等于0，则发生错误。
- 如果 *var_name* 的值未初始化，则值将被设置为未初始化，并且状态将被设置为12311。
- 所有机器人的指定名称可在系统变量$ PH_MEMBERS []中找到。这还包括有关机器人状态的信息。环索引是此系统变量的数组索引。 KAREL用户可以通过引用此系统变量中的名称和其他信息而不是显式名称来编写通用程序。

另请参阅：SET_VAR内置过程，Internet选项手册，以获取有关在环中访问机器人变量的信息。

警告

使用GET_VAR修改系统变量可能会导致意外结果。

示例1：要在环中访问MHROB03机器人上的$ TP_DEFPROG，请参阅在MHROB03上访问$ TP_DEFPROG。

在 MHROB03上访问$ TP_DEFPROG

GET_VAR (entry , '\\ MHROB03 \ * system *' , '$ TP_DEFPROG' , strvar , status)

示例2：GET_VAR内置过程显示两个程序，*util_prog* 和 *task*。 程序 *util_prog* 使用FOR循环来增加INTEGER变量 *num_of_parts* 的值。

util_prog 还将值分配给ARRAY *part_array*。 程序任务使用两个GET_VAR语句来检索 *num_of_parts* 和 part_array [3]的值。 *num_of_parts* 的值分配给INTEGER变量 *count* , part_array [3]分配给STRING变量 *part_name*。 最后一个GET_VAR语句将count的值放入另一个INTEGER变量newcount中。

GET_VAR 内置程序

```
PROGRAM util_prog
    VAR
      j, num_of_parts : INTEGER
      part_array      : ARRAY[5] OF STRING[10]
    BEGIN
      num_of_parts = 0

      FOR j = 1 to 20 DO
        num_of_parts = num_of_parts + 1
      ENDFOR
      part_array[1] = 10
      part_array[2] = 20
      part_array[3] = 30
      part_array[4] = 40
      part_array[5] = 50
    END util_prog
  PROGRAM task
    VAR
      entry, status    : INTEGER
      count, new_count : INTEGER
      part_name        : STRING[20]
    BEGIN
      GET_VAR(entry, 'util_prog', 'part_array[3]', part_name, status)
      WRITE('Part Name is Now....>', part_name, cr)

      GET_VAR(entry, 'util_prog', 'num_of_parts', count, status)
```

```
      WRITE('COUNT Now Equals....>', count, cr)

      GET_VAR(entry, 'task', 'count', new_count, status)

    END task
```

在GET_VAR SET_VAR内置过程中，从环中所有机器人的环中的所有机器人上设置数组[ipgetset] set_data [x，y]。 在此数组中，x是源机器人索引，y是目标机器人索引：

GET_VAR SET_VAR 内置程序

```
FOR idx = 1 TO $PH_ROSIP.$NUM_MEMBERS DO
  IF idx = $PH_ROSIP.$MY_INDEX THEN
    -- This will work but it this robot so is inefficient
  ELSE
    SELECT $PH_MEMBERS[idx].$STATE OF
    CASE (0) : -- Offline
      sstate = ' Offline'
    CASE (1) : -- Online
      sstate = ' Online '
    CASE (2) : -- Synchronized
      sstate = ' Synch  '
      CNV_INT_STR(idx, 1, 10, sidx)
      prog_name = '\\' + $PH_MEMBERS[idx].$NAME + '\ipgetset'
      var_name = 'set_data['+ smy_index + ',' + sidx + ']'
      GET_VAR(entry, prog_name, var_name, set_data[$PH_ROSIP.$MY_INDEX,
             idx], status[idx])
      IF status[idx] = 0 THEN
        IF uninit(set_data[$PH_ROSIP.$MY_INDEX, idx]) THEN
          set_data[$PH_ROSIP.$MY_INDEX, idx] = 0
        ELSE
          set_data[$PH_ROSIP.$MY_INDEX, idx] = set_data[$PH_ROSIP.$MY_INDEX,
             idx] + 1
        ENDIF
        SET_VAR(entry, prog_name, var_name, set_data[$PH_ROSIP.$MY_INDEX, idx],
             status[idx])
      ENDIF
    ENDSELECT
  ENDIF
ENDFOR
```

示例3 GET_VAR和SET_VAR也可用于设置寄存器值。

这适用于程序名为* posreg *和* numreg *的本地机器人。 对于本地机器人，它具有与GET_POS_REG，GET_REG和SET_REG，SET_POS_REG内置函数类似的功能。 内置插件仅适用于本地机器人。 您可以使用机器人名称作为程序名称的一部分，通过GET_VAR和SET_VAR访问环中的机器人。

对于数字寄存器上的GET_VAR，KAREL变量的类型必须与寄存器的类型或错误匹配，返回“不兼容的值”。 在下面的示例中，如果数字寄存器10是实数值，则将返回错误，并且将在错误情况下设置实际值。

如果位置寄存器被锁定并且您尝试设置它，则返回锁定错误位置寄存器。 请参阅使用GET_VAR和SET_VAR设置寄存器值。

使用 GET_VAR 和 SET_VAR 设置寄存器值

```
program GETREG
%nolockgroup
VAR
entry: integer
status: integer
int_data: integer
real_data: real
posext_data: xyzwprext


BEGIN
  GET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[10]', int_data, status)
  IF status <> 0 THEN
    GET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[10]', real_data, status)
  ENDIF
  GET_VAR(entry, '\\mhrob01\*posreg*', '$POSREG[1, 10]', posext_data, status)
  SET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[20]', int_data, status)
  SET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[21]', real_data, status)
  SET_VAR(entry, '\\mhrob01\*posreg*', '$POSREG[1, 20]', posext_data, status)
end GETREG
```

A.8.28 GO TO 声明

目的：将控制转移到指定的语句

语法：|| GO TO | GOTO || stmnt_label

其中：

stmnt_label：有效的 KAREL 标识符

细节：

- 必须在与 GO TO 语句相同的例程或程序体中定义 *stmnt_label*。
- 标签标识符后跟双冒号（::）。 可执行语句可能会也可能不会在同一行上。
- GO TO 应仅用于特殊情况，例如WHILE，REPEAT和FOR循环等常规控制结构难以实现或难以实现。

另请参阅：第2.1.5节"标签",有关标签规则的更多信息,附录E,"语法图",以获取其他语法信息

示例：以下示例根据DIN [1]的状态将TCP从一个位置移动到另一个位置。

GO TO 声明

```
BEGIN
  IF NOT DIN[1] THEN
    move_to_p1 -- Call TP program to do move
  ELSE
    GO TO end_it
  ENDIF
  IF NOT DIN[1] THEN
    move_to_p2 -- Call TP program to do move
  ELSE
    GO TO end_it
  ENDIF
  END_IT::
```

A.9 - H - KAREL 语言描述

A.9.1 HOLD 动作

用途：使当前动作保持并阻止后续动作的开始

语法：HOLD <GROUP[n,{,n}]>

细节：

- 任何正在进行的动作都会进行。 机器人和辅助轴或扩展轴减速停止。
- 执行 HOLD 后的尝试动作也会保持。 HOLD 不能被发出议案的条件处理程序覆盖。
- HOLD 使用 UNHOLD 声明或操作释放。

- 如果 *group* 子句不存在，则任务所控制的所有组（在定义条件时）将被取消。
- 如果持有的动作是与其他组同时或协调动作的一部分，则保留所有组的动作。
- 不能为不同的任务保留动作。

另请参阅：第8章 位置数据，有关启动和停止运动的更多信息

示例：以下示例显示了当DIN [1]打开时保持运动的条件处理程序。

HOLD 动作

```
CONDITION[1]:
WHEN DIN[1] = ON DO
  HOLD
ENDCONDITION
```

A.9.2 HOLD 声明

用途：使当前动作保持不变，并阻止后续动作的启动

语法：HOLD <GROUP[n{,n}]>

细节：

- 任何正在进行的动作被保持。机器人和辅助轴或扩展轴减速到停止。
- 还执行 HOLD 后的尝试动作。 发出动作的条件处理程序不能覆盖 HOLD。
- 使用 UNHOLD 声明或操作释放 HOLD。
- 如果在保持动作时执行RELEASE语句，则取消所有保持的动作。
- 如果 *group* 子句不存在，则将取消该任务所控制的所有组。
- 如果停止，恢复，取消或保持的动作是与其他组同时或协调动作的一部分，则所有组的动作将被停止，恢复，取消或保持。
- 不能为不同的任务保留动作。

另请参阅：第8章 位置数据，有关启动和停止运动的更多信息，请参阅附录E，''语法图'，以获取其他语法信息

A.10 - I - KAREL 语言描述

A.10.1 IF ... ENDIF 声明

用途 ：如果BOOLEAN表达式为TRUE，则执行一系列语句; 如果条件为FALSE，则可以执行备用序列。

语法： IF bool_exp THEN

{ true_stmnt } < ELSE

{ false_stmnt } >ENDIF

其中：

bool_exp：BOOLEAN

true_stmnt：可执行的KAREL语句

false_stmnt：可执行的KAREL语句

细节：

- 如果 *bool_exp* 的计算结果为TRUE，则执行 *true_stmnt* 中包含的语句。 然后继续执行ENDIF之后的第一个语句。
- 如果 *bool_exp* 的计算结果为FALSE且未指定ELSE子句，则执行将直接跳转到ENDIF之后的第一个语句。
- 如果 *bool_exp* 的计算结果为FALSE且指定了ELSE子句，则执行 *false_stmnt* 中包含的语句。 然后继续执行ENDIF之后的第一个语句。
- IF语句可以嵌套在 *true_stmnt* 或 *false_stmnt* 中。

另请参阅：附录E，''语法图'，以获取其他语法信息示例：有关详细的程序示例，请参阅以下部分：

第B.3节 “将数据保存到默认设备” （ SAVE_VR.KL ）

第B.5节 “使用寄存器内置函数” （ REG_EX.KL ）

第B.6节 “路径变量和条件处理程序” （ PTH_MOVE.KL ）

第B.7节 “列出文件和程序以及操作字符串” （ LIST_EX.KL ）

第B.9节 “使用文件和设备内置函数” （ FILE_EX.KL ）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.10.2 IN 条件

用途：指定将在何处创建变量

语法：IN (CMOS | DRAM | SHADOW)

细节：

- IN 子句可以是变量声明的一部分。 它应该在FROM子句之前指定。
- IN CMOS 指定将在永久存储器中创建变量。
- IN DRAM 指定将在临时存储器中创建变量。
- IN SHADOW 指定对变量所做的任何更改都将保留在CMOS中。 写入此类变量的速度较慢，但读取速度要快得多。 这是一种很好的存储器类型，可用于当前在CMOS中的配置参数。
- IN UNINIT_DRAM 指定启动时DRAM变量为 UNINITIALIZED。
- 如果未指定IN子句，则在临时存储器中创建所有变量; 除非指定 %CMOSVARS 或 %SHADOW 指令，否则所有变量都将在永久内存中创建。
- 在例程的声明部分中声明变量时，不能使用IN子句。

另请参见：第1.4.1节，%CMOSVARS转换器指令，%SHADOW转换器指令，%SHADOWCMOS转换器指令

示例：有关详细的程序示例，请参阅以下部分：

在DRAM中，第B.2节“复制路径变量”（CPY_PTH.KL）

在CMOS中，第B.10节“使用动态显示内置”（DYN_DISP.KL）或第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

A.10.3 %INCLUDE 译者指令

用途：在翻译时在程序中插入其他文件。

语法：%INCLUDE file_spec

细节：

- *file_spec* 是要包含的文件的名称。它具有以下细节：
  - 指定的文件名称不能超过12个字符。
  - 文件类型默认为.KL，因此它不会出现在指令中。
- %INCLUDE 指令必须单独出现在一行上。
- 指定的文件通常包含声明，例如 CONST或VAR 声明。但是，它们可以包含程序的任何部分，包括可执行语句甚至其他语句
  %INCLUDE 指令。
- 包含的文件本身可以包含其他文件，最大深度为三个嵌套包含文件。包含文件的总数没有限制。
- 在翻译文件时，如果 KAREL语言 翻译器遇到 %INCLUDE 指令，它将开始翻译包含的文件，就像它是原始文件的一部分一样。当包含整个文件时，翻译器将恢复原始文件。
- 附录A中的一些示例引用了以下包含文件：
  %INCLUDE FR：\ klevkmsk
  %INCLUDE FR：\ klevkeys
  %INCLUDE FR：\ klevccdf
  %INCLUDE FR：\ kliotyps
  这些文件包含可在您的KAREL程序中使用的常量。如果您在控制器上进行翻译，则可以直接从FROM磁盘中包含它们。
  包含文件也可在OLPC磁盘上使用，并作为安装过程的一部分复制到硬盘中。
- 示例：请参阅以下各节以获取详细的程序示例：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.10.4 INDEX 内置函数

用途：返回另一个指定STRING参数中指定STRING参数第一次出现的第一个字符的索引。

如果未找到参数，则返回0值。

语法：INDEX(main, find)

函数返回类型：INTEGER

Input/Output 参数：

[in] main :STRING

[in] find :STRING

%ENVIRONMENT Group :SYSTEM

细节：

- 返回值是 *main* 中与第一次出现的 *find* 的第一个字符对应的索引位置，如果在 *main* 中没有出现 *find*，则返回0。

示例：以下示例使用INDEX内置函数在 part_desc 中查找字符串“Old”的第一个匹配项。

INDEX 内置函数

```
class_key = 'Old'
   part_desc = 'Refurbished Old Part'
   IF INDEX(part_desc, class_key) > 0 THEN
      in_class = TRUE
   ENDIF
```

A.10.5 INI_DYN_DISB 内置程序

用途：启动BOOLEAN变量的动态显示。 此过程显示STRING ARRAY的元素，具体取决于BOOLEAN变量的当前值。

语法：INI_DYN_DISB (b_var, window_name, field_width, attr_mask, char_size, row, col, interval, strings, status)

Input/Output 参数：

[in] b_var :BOOLEAN

[in] window_name :STRING

[in] field_width :INTEGER

[in] attr_mask :INTEGER

[in] char_size :INTEGER

[in] row :INTEGER

[in] col :INTEGER

[in] interval :INTEGER

[in] strings :ARRAY OF STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- 动态显示基于 *b_var* 的值启动.Ifb_var为FALSE，显示字符串[1];如果 *b_var* 为TRUE，则显示字符串[2]。如果 *b_var* 未初始化，则显示*的字符串。 *b_var* 和字符串都必须是静态（非本地）变量。
- *window_name* 必须是先前定义的窗口名称。有关预定义的窗口名称，请参见第7.10.1节和第7.10.2节。
- 如果 *field_width* 不为零，则如果 strings [n]的元素短于此指定宽度，则显示将使用空白进行扩展。取消动态显示时，该区域被清除。
- *attr_mask* 是一个按位掩码，用于指示字符显示属性。这应该是以下常量之一：

  0：正常

  1：粗体（仅在CRT上支持）

  2：闪烁（仅在CRT上支持）

  4：下划线

  8：Reversevideo
- 要具有多个显示属性，请使用OR运算符将常量属性值组合在一起。例如，要将文本显示为粗体并带下划线，请使用1或4。

- *char_size* 指定数据是以正常，双倍宽度还是双倍高度，双倍宽度大小显示。 这应该是以下常量之一：

  0：正常

  1：双倍宽（仅在CRT上支持）

  2：双高，双宽
- *row* 和 *col* 指定要在其中显示数据的窗口中的位置。
- *interval* 表示显示更新之间的最小时间间隔（以毫秒为单位）。 这必须大于零。 实际时间可能更长，因为格式化显示的任务以低优先级运行。
- strings [n]包含将要显示的文本。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见：CNC_DYN_DISB内置程序

示例：有关详细的程序示例，请参见第B.10节“使用动态显示内置”（DYN_DISP.KL）。

A.10.6 INI_DYN_DISE 内置程序

用途：启动INTEGER变量的动态显示。 此过程显示STRING ARRAY的元素，具体取决于INTEGER变量的当前值。

语法：INI_DYN_DISE (e_var, window_name, field_width, attr_mask, char_size, row, col, interval, strings, status)

Input/Output 参数：

[in] e_var :INTEGER

[in] window_name :STRING

[in] field_width :INTEGER

[in] attr_mask :INTEGER

[in] char_size :INTEGER

[in] row :INTEGER

[in] col :INTEGER

[in] interval :INTEGER

[in] strings :ARRAY OF STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- 动态显示基于 *e_var* 的值启动.Ife_var的值为n，显示字符串[n + 1]; 如果e_var具有负值，或者大于或等于字符串数组长度的值，则会显示一个'？'字符串。 *e_var* 和 *string* 都必须是静态（非本地）变量。
- 有关上面列出的其他参数的说明，请参阅 INI_DYN_DISB 内置过程。

另请参见：CNC_DYN_DISE 内置程序

示例：有关详细的程序示例，请参见第B.10节“使用动态显示内置”（DYN_DISP.KL）。

A.10.7 INI_DYN_DISI 内置程序

用途：在指定窗口中启动INTEGER变量的动态显示。

语法：INI_DYN_DISI(i_var, window_name, field_width, attr_mask, char_size, row, col, interval, buffer_size, format, status)

Input/Output 参数：

[in] i_var :INTEGER

[in] window_name :STRING

[in] field_width :INTEGER

[in] attr_mask :INTEGER

[in] char_size :INTEGER

[in] row :INTEGER

[in] col :INTEGER

[in] interval :INTEGER

[in] buffer_size :INTEGER

[in] format :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- • *i_var* 是要启动其动态显示的整数。
  • 如果 *field_width* 不为零，则如果 *i_var* 小于此指定宽度，则显示将使用空白进行扩展。 取消动态显示时，该区域被清除。
  • 未实现 *buffer_size*。
  • *format* 用于打印变量。 这可以作为用单引号括起来的文字传递。 格式字符串以%开头，以转换字符结尾。 在%和转换字符之间可以按顺序：
  - 标志（以任何顺序），修改规范：
  - : 指定此字段的左侧调整。
  + : 指定始终使用符号打印数字。
  0指定用前导零填充数字字段宽度。

— 一个指定最小字段宽度的数字。 转换后的参数将打印在至少这个范围内的字段中。 如果需要，它将在左侧（或右侧，如果需要调整左侧）填充以填充场宽。

— 一个句点，它将字段宽度与精度分开。

— 一个数字，精度，指定要从字符串打印的最大字符数，或浮点值小数点后的位数，或整数的最小位数。

说明符格式必须包含表A-12中的一个转换字符。

表A-12. 转换字符

字符	参数类型; 打出为
d	INTEGER; 十进制数
o	INTEGER; 无符号八进制表示法（没有前导零）。
x,X	INTEGER; 无符号十六进制表示法（没有前导0x或0X），使用abcdef或ABCDEF表示10，...，15。
u	INTEGER; 无符号十进制表示法。
s	STRING; 从字符串打出字符直到字符串结尾或精度给出的字符数。

表A-12. 转换字符（续）

字符	参数类型; 打出为
f	REAL; 形式为[-] mmm.dddddd的十进制表示法，其中d的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
e,E	REAL; 形式的十进制表示法[-] m.dddddde + -xx或[-] m.ddddddE + -xx，其中d的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
g,G	REAL; 如果指数小于-4或大于或等于精度，则使用%e或%E; 否则使用%f。 不打出尾随零和尾随小数指针。
%	没有参数被转换; 打出%。

- 有关上面列出的其他参数的说明，请参阅 INI_DYN_DISB 内置过程。

另请参见：CNC_DYN_DISI，DEF_WINDOW内置过程

示例：有关详细的程序示例，请参见第B.10节“使用动态显示内置”（DYN_DISP.KL）。

A.10.8 INI_DYN_DISP 内置程序

用途：根据端口类型和端口号，启动指定窗口中端口值的动态显示。

语法：INI_DYN_DISP (port_type, port_no, window_name, field_width, attr_mask, char_size, row, col, interval, strings, status)

Input/Output 参数：

[in] port_type :INTEGER

[in] port_no :INTEGER

[in] window_name :STRING

[in] field_width :INTEGER

[in] attr_mask :INTEGER

[in] char_size :INTEGER

[in] row :INTEGER

[in] col :INTEGER

[in] interval :INTEGER

[in] strings :ARRAY OF STRING

[out] status :INTEGER

细节：

- *port_type* 指定要显示的端口类型。 代码在FROM：KLIOTYPS.KL中定义。
  - 如果 *port_type* 是BOOLEAN端口（例如DIN），如果为FALSE，则显示字符串[1]; 如果变量为TRUE，则显示字符串[2]。
  - 如果 *port_type* 是INTEGER端口（例如，GIN），如果端口的值是n，则将显示字符串[n + 1]。 如果端口的值大于或等于字符串数组的长度，则显示一个'？'字符串。
- *port_no* 指定要显示的端口号。
- 有关上面列出的其他参数的说明，请参阅INI_DYN_DISB内置过程。

另请参见：CNC_DYN_DISP内置程序
示例：有关详细的程序示例，请参见第B.10节“使用动态显示内置”（DYN_DISP.KL）。

A.10.9 INI_DYN_DISR 内置程序

用途：在指定窗口中启动 REAL 变量的动态显示。

语法：INI_DYN_DISR(r_var, window_name, field_width, attr_mask, char_size, row, col, interval, buffer_size, format, status)

Input/Output 参数：

[in] r_var :REAL

[in] window_name :STRING

[in] field_width :INTEGER

[in] attr_ mask :INTEGER

[in] char_size :INTEGER

[in] row :INTEGER

[in] col :INTEGER

[in] interval :INTEGER

[in] buffer_size :INTEGER

[in] format :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *r_var* 是REAL变量，其动态显示将被启动。
- 如果 *field_width* 不为零，则如果 *r_var* 小于此指定宽度，则显示将使用空白进行扩展。取消动态显示时，该区域被清除。
- 有关上面列出的其他参数的说明，请参阅INI_DYN_DISI内置过程。

另请参见：CNC_DYN_DISR内置程序
示例：有关详细的程序示例，请参见第B.10节“使用动态显示内置”（DYN_DISP.KL）。

A.10.10 INI_DYN_DISS 内置程序

用途：在指定窗口中启动 STRING 变量的动态显示。

语法：INI_DYN_DISS(s_var, window_name, field_width, attr_mask, char_size, row, col, interval, buffer_size, format, status)

Input/Output 参数：

[in] s_var :STRING

[in] window_name :STRING

[in] field_width :INTEGER

[in] attr_ mask :INTEGER

[in] char_size :INTEGER

[in] row :INTEGER

[in] col :INTEGER

[in] interval :INTEGER

[in] buffer_size :INTEGER

[in] format :STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *s_var* 是STRING变量，其动态显示将被启动。
- 如果 *field_width* 不为零，则如果 *s_var* 小于此指定宽度，则显示将使用空白进行扩展。取消动态显示时，该区域被清除。
- 有关上面列出的其他参数的说明，请参阅 INI_DYN_DISI 内置过程。

另请参见：CNC_DYN_DISS，INI_DYN_DISI内置程序

示例：有关详细的程序示例，请参见第B.10节 “使用动态显示内置”（DYN_DISP.KL）。

A.10.11 INIT_QUEUE 内置程序

用途：设置队列变量条目以使队列中没有条目

语法：INIT_QUEUE(queue)

Input/Output 参数：

[out] queue_t :QUEUE_TYPE

%ENVIRONMENT Group： PBQMGR

细节：

- *queue_t* 是要初始化的队列

另请参阅：GET_QUEUE，MODIFY_QUEUE内置过程，QUEUE_TYPE数据类型，第15.8节 “使用队列进行任务通信”

示例：以下示例初始化一个名为job_queue的队列。

INIT_QUEUE 内置程序

```
PROGRAM init_queue_x
%environment PBQMGR
VAR
  job_queue FROM globals: QUEUE_TYPE
BEGIN
  INIT_QUEUE(job_queue)
END init_queue_x
```

A.10.12 INIT_TBL 内置程序

用途：初始化示教器上的表单

语法：INIT_TBL(dict_name, ele_number, num_rows, num_columns, col_data, inact_array, change_array, value_array, vptr_array, table_data, status)

Input/Output 参数：

[in] dict_name :STRING

[in] ele_number :INTEGER

[in] num_rows :INTEGER

[in] num_columns :INTEGER

[in] col_data :ARRAY OF COL_DESC_T

[in] inact_array :ARRAY OF BOOLEAN

[in] change_array :ARRAY OF ARRAY OF BOOLEAN

[in] value_array :ARRAY OF STRING

[out] vptr_array :ARRAY OF ARRAY OF INTEGER

[in,out] table_data :XWORK_T

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- 只应使用 INIT_TBL 和 ACT_TBL 内置例程而不是 DISCTRL_TBL

如果需要对每次击键进行特殊处理，或者需要在不退出表格菜单的情况下进行功能键处理。

- 在使用内置的 ACT_TBL之前必须调用 INIT_TBL。如果使用内置的 DISCTRL_TBL，则不需要调用。
- *dict_name* 是包含表头的字典的四字符名称。
- *ele_number* 是表头的元素编号。
- *num_rows* 是表中的行数。
- *num_columns* 是表中的列数。
- *col_data* 是列描述符结构的数组，表中的每列都有一个。 它包含以下字段：
  - *item_type*：此列中值的数据类型。 定义了以下数据类型常量：

    TPX_INT_ITEM - Integer 类型

    TPX_REL_ITEM - Real 类型

    TPX_FKY_ITEM - 功能键枚举类型

    TPX_SUB_ITEM - 子窗口枚举类型

    TPX_KST_ITEM - KAREL String 类型

    TPX_KSL_ITEM - KAREL String 标签类型（可以选择，不能编辑）

    TPX_KBL_ITEM - KAREL Boolean类型

    TPX_BYT_ITEM - Byte 类型

    TPX_SHT_ITEM - Short 类型

    TPX_PBL_ITEM - 端口布尔类型

    TPX_PIN_ITEM - 端口整数类型

  - *start_col*：此数据列的显示字段的起始字符列（1..40）。
  - *field_width*：此数据列的显示字段的宽度。
  - *num_ele*：用于显示某些数据类型值的Dictionary元素。 格式

    这些数据类型的字典元素如下：

    - TPX_FKY_ITEM：枚举值放在功能键标签上。 最多可以有2页功能键标签，最多10个标签。 每个标签都是一个最多8个字符的字符串。 但是，标签后面跟着另一个标签的最后一个字符应该留空或两个标签一起运行。

- 单个字典元素定义所有标签值。 必须使用&new_line将每个值放在单独的行上。 这些值按顺序分配给功能键F1..F5，F6..F10和数值1..10。 未标记的功能键应留空。 如果第二个功能键页面上有任何标签，F6..F10，键5和10的标签必须在第8列中包含字符''>'。如果键F6..F10上没有标签，则行 不必为最后一个非空白标签后的任何键标签指定。

例：

```
$ example_fkey_label_c
   "" &new_line
   "F2" &new_line
   "F3" &new_line
   "F4" &new_line
   "F5 >" &new_line
   "F6" &new_line
   "F7" &new_line
   "" &new_line
   "" &new_line
   " >"
```

– -- TPX_SUB_ITEM：从显示设备上的子窗口中选择枚举值。 最多可以有5个子窗口页面，最多35个值。 每个值都是一个最多16个字符的字符串。

– - 以enum_dict开头的一系列连续字典元素定义值。 每个值必须放在一个单独的元素中，并且不能以&new_line结尾。 字符按顺序分配数值1..35。 最后一个字典元素必须是“\ a”。

– **例：**

```
$ example_sub_win_enum_c
   "Red"
   $
   "Blue"
   $
   "Green"
   $
   "Yellow"
   $
   "\a"
```

- TPX_KBL_ITEM，TPX_PBL_ITEM：''true''和''false''值按顺序放在功能键标签F4和F5上。 每个标签都是一个最多8个字符的字符串。 但是，“true”标签的最后一个字符应留空，否则两个标签将一起运行。

- 标签值的单个字典元素。 必须使用&new_line将每个值放在单独的行上，首先使用''false''值。

例：

```
$ example_boolean_c
   "OFF" &new_line
   "ON"
```

- *enum_dict*：用于显示数据类型TPX_FKY_ITEM，TPX_SUB_ITEM的字典名称，TPX_KBL_ITEM或TPX_PBL_ITEM

- *format_spec*：格式字符串用于打印出数据值。 格式字符串包含格式说明符。 格式字符串还可以包含格式说明符之前或之后的任何所需字符。 格式说明符本身以%开头，以转换字符结尾。 在%和转换字符之间可能按顺序：

  - -标志（以任何顺序），修改规范：

    -：指定此字段的左侧调整。

    +：指定始终使用符号打印数字。

    space：如果第一个字符不是符号，则会以空格为前缀。

    0：指定用前导零填充数字字段宽度。

  – 一个指定最小字段宽度的数字。 转换后的参数将打印在至少这个范围内的字段中。 如果需要，它将在左侧（或右侧，如果需要调整左侧）填充以填充场宽。
  – 一个句点，它将字段宽度与精度分开。
  – 一个数字，精度，指定要从字符串打印的最大字符数，或浮点值小数点后的位数，或整数的最小位数。

- 说明符格式必须包含下表中的一个转换字符：

表 A-13. 转换字符

字符	参数类型；打出为
d	INTEGER; 十进制数。
o	INTEGER; 无符号八进制表示法（没有前导零）。
x,X	INTEGER; 无符号十六进制表示法（没有前导0x或0X），使用abcdef或ABCDEF表示10，...，15。
u	INTEGER; 无符号十进制表示法。

表 A-13. 转换字符（续）

字符	参数类型；打出为
s	STRING; 从字符串打出字符直到字符串结尾或精度给出的字符数。
f	REAL; 形式为[-] mmm.dddddd的十进制表示法，其中d的数量由精度给出。 默认精度为6; 精度为0会抑制小数点。
e,E	REAL; 形式的十进制表示法[-] m.dddddde + -xx或[-] m.ddddddE + -xx，其中d的数量由精度给出。 默认精度为6; 精度为0
g,G	REAL; 如果指数小于-4或大于或等于精度，则使用%e或%E; 否则使用%f。 不打出尾随零和尾随小数指针。
%	没有参数被转换; 打出%。

竹侍 > “%d”或“%-10s”

可以与 *col_data* 区域的 *item_type* 字段中指定的数据类型一起使用的格式说明符跟随：

TPX_INT_ITEM %d, %o, %x, %X, %u

TPX_REL_ITEM %f, %e, %E, %g, %G

TPX_FKY_ITEM %s

TPX_SUB_ITEM %s

TPX_KST_ITEM %s

TPX_KSL_ITEM %s

TPX_KBL_ITEM %s

TPX_BYT_ITEM %d, %o, %x, %X, %u, %c

TPX_SHT_ITEM %d, %o, %x, %X, %u

TPX_PBL_ITEM %s

TPX_PIN_ITEM %d, %o, %x, %X, %u

— max integer：数据类型为的最大值 TPX_INT_ITEM, TPX_BYT_ITEM, 或 TPX_SHT_ITEM.

— *min_integer*：数据类型为TPX_INT_ITEM，TPX_BYT_ITEM或最小值TPX_SHT_ITEM。

— *max_real*：实数的最大值。

— *min_real*：实数的最小值。

— *clear_flag*：如果数据类型为TPX_KST_ITEM，则1导致在进入之前清除该字段字符和0导致它不被清除。

— *lower_case*：如果数据类型为TPX_KST_ITEM，则1允许字符以大写或小写形式输入到字符串，0将它们限制为大写。

- *inact_array* 是一个布尔数组，对应于表中的每一列。
  — 您可以将每个布尔值设置为TRUE，这将使该列处于非活动状态。 这意味着该列无法被光标化。
  — 数组大小可以小于或大于表中的项目数。
  — 如果未使用 *inact_array*，则可以使用数组大小1，并且不需要初始化该数组。
- *change_array* 是一个二维的布尔数组，对应于表中的格式化数据项。
  — 如果设置了相应的值，则布尔值将设置为TRUE，否则设置为FALSE。您不需要初始化数组。
  — 数组大小可以小于或大于表中数据项的数量。
  — 如果未使用 *change_array*，则可以使用数组大小 1。
- *value_array* 是一个变量名数组，对应于表中的数据列。每个变量名称都可以指定为“[prog_name] var_name”。
  — [prog_name]指定包含指定变量的程序的名称。如果未指定[prog_name]，则使用当前正在执行的程序。
  — *var_name* 必须引用静态的全局程序变量。
  — *var_name* 可以包含节点号，字段名和/或下标。
  — 每个命名变量必须是长度为 *num_rows* 的KAREL数组。其数据类型和值应与col_data中相应列的 *item_type* 字段的值一致，如下所示：

– TPX_INT_ITEM：包含所需值的 INTRER 的 ARRAY OF INTEGER。
– TPX_REL_ITEM：包含所需值的 ARRAY OF REAL。
– TPX_FKY_ITEM：INTRER 的 ARRAY，其值指的是 *col_data* 中 *enum_ele* 字段中指定的字典元素中的项。 最多可以有2个功能键页，或10个可能的功能键枚举值。
– TPX_SUB_ITEM：INTRER的ARRAY，其值指的是 *col_data* 中 *enum_ele* 字段中指定的字典元素中的项。 最多可以有28个子窗口枚举值。

- TPX_KST_ITEM：包含所需值的 STRING ARRAY。
- TPX_KST_ITEM：包含所需值的 STRING ARRAY。
- TPX_KSL_ITEM：包含所需值的 STRING 的 ARRAY。用户无法编辑这些值。如果选择了一个，ACT_TBL 将返回。
- TPX_KBL_ITEM：包含所需值的 BOARAN 阵列。由 *col_data* 中的 *enum_ele* 字段指定的字典元素应该具有两个元素，其中 false 项为 first，true 项为 second。
  TPX_BYT_ITEM：包含所需值的字节阵列。 “-” TPX_SHT_ITEM：包含所需值的SHORT的ARRAY OF SHORT。 “-” TPX_PBL_ITEM：包含端口名称的STRING ARRAY，例如''DIN [5]''。 “-” TPX_PIN_ITEM：包含端口名称的STRING ARRAY，例如''GOUT [3]''。
- TPX_BYT_ITEM：包含所需值的字节阵列。
- TPX_SHT_ITEM：包含所需值的SHORT的ARRAY OF SHORT。
- TPX_PBL_ITEM：包含端口名称的STRING ARRAY，例如''DIN [5]''。
- TPX_PIN_ITEM：包含端口名称的STRING ARRAY，例如''GOUT [3]''。

- *vptr_array* 是一个整数数组，对应于 *value_array* 中的每个变量名。 不要更改此数据; 它在内部使用。
- *table_data* 用于显示和控制表。 不要更改此数据; 它在内部使用。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

竹待 > 在此示例中，TPXTABEG.TX 在控制器上作为“XTAB”加载。 TPEXTBL 调用 INIT_TBL 来初始化一个包含五列和四行的表。 它在循环中调用 ACT_TBL 来读取和处理按下的每个键。

INIT_TBL 内置程序

```
-----------------------------------------------
  TPXTABEG.TX
-----------------------------------------------
  $title
  &reverse "DATA Test Schedule" &standard &new_line
  "E1: " &new_line
  "      W(mm) TEST  C(%%) G(123456) COLOR"
  ^1
  ?2
  $function_keys
  "f1"       &new_line
  "f2"       &new_line
  "f3"       &new_line
  "f4"       &new_line
  "  HELP >" &new_line
  "f6"       &new_line
```

```
"f7"       &new_line
"f8"       &new_line
"f9"       &new_line
"f10    >"
$help_text "Help text goes here...
"$enum1
"" &new_line
"" &new_line
"TRUE" &new_line
"FALSE" &new_line
""
$enum2
"Red"
$
"Blue"
$
"Green"
$
"Yellow"
$
"Brown"
$
"Pink"
$
"Mauve"
$
"Black"
$
"......"
-------------------------------------------
TPEXTBL.KL
-------------------------------------------
PROGRAM tpextbl
%ENVIRONMENT uif
%INCLUDE FROM:\klevccdf
%INCLUDE FROM:\klevkeysVAR
  dict_name: STRING[6]
  ele_number: INTEGER
  num_rows: INTEGER
  num_columns: INTEGER
  col_data: ARRAY[5] OF COL_DESC_T
  inact_array: ARRAY[5] OF BOOLEAN
  change_array: ARRAY[4,5] OF BOOLEAN
  value_array: ARRAY[5] OF STRING[26]
  vptr_array: ARRAY[4,5] OF INTEGER
  table_data: XWORK_T
  status: INTEGER
```

```
  action: INTEGER
  def_item: INTEGER
  term_char: INTEGER
  attach_sw: BOOLEAN
  save_action:  INTEGER
  done: BOOLEAN
  value1: ARRAY[4] OF INTEGER
  value2: ARRAY[4] OF INTEGER
  value3: ARRAY[4] OF REAL
  value4: ARRAY[4] OF STRING[10]
  value5: ARRAY[4] OF INTEGER
BEGIN
  def_item = 1
  value_array[1] = 'value1'
  value_array[2] = 'value2'
  value_array[3] = 'value3'
  value_array[4] = 'value4'
  value_array[5] = 'value5'
  value1[1] = 21
  value1[2] = 16
  value1[3] = 1
  value1[4] = 4
  value2[1] = 3
  value2[2] = 2
  value2[3] = 3
  value2[4] = 2
  value3[1] = -13
  value3[2] = 4.1
  value3[3] = 23.9
  value3[4] = -41
  value4[1] = 'XXX---'
  value4[2] = '--X-X-'
  value4[3] = 'XXX-XX'
  value4[4] = '-X-X--'
  value5[1] = 1
  value5[2] = 1
  value5[3] = 2
  value5[4] = 3
  inact_array[1] = FALSE
  inact_array[2] = FALSE
  inact_array[3] = FALSE
  inact_array[4] = FALSE
  inact_array[5] = FALSE
  col_data[1].item_type = TPX_INT_ITEM
  col_data[1].start_col = 6
  col_data[1].field_width = 4
  col_data[1].format_spec = '%3d'
```

```
col_data[1].max_integer = 99
col_data[1].min_integer = -99
col_data[2].item_type = TPX_FKY_ITEM
col_data[2].start_col = 12
col_data[2].field_width = 5
col_data[2].format_spec = '%s'
col_data[2].enum_ele = 3  -- enum1 element number
col_data[2].enum_dict = 'XTAB'
col_data[3].item_type = TPX_REL_ITEM
col_data[3].start_col = 18
col_data[3].field_width = 5
col_data[3].format_spec = '%3.1f'
col_data[4].item_type = TPX_KST_ITEM
col_data[4].start_col = 26
col_data[4].field_width = 6
col_data[4].format_spec = '%s'
col_data[5].item_type = TPX_SUB_ITEM
col_data[5].start_col = 34
col_data[5].field_width = 6
col_data[5].format_spec = '%s'
col_data[5].enum_ele = 4  -- enum2 element number
col_data[5].enum_dict = 'XTAB'
dict_name = 'XTAB'
ele_number = 0  -- title element number
num_rows = 4
num_columns = 5
def_item = 1
attach_sw = TRUE
INIT_TBL(dict_name, ele_number, num_rows, num_columns, col_data,
         inact_array, change_array, value_array, vptr_array,
         table_data, status)
IF status <> 0 THEN
  WRITE('INIT_TBL status = ', status, CR);
  ELSE
    def_item = 1
    -- 表的初始显示
    ACT_TBL(ky_disp_updt, def_item, table_data, term_char,
            attach_sw, status)
    IF status <> 0 THEN
      WRITE(CR, 'ACT_TBL status = ', status)
    ENDIF
  ENDIF
  IF status = 0 THEN
    -- 循环直到选择终止键。
    done = FALSE
    action = ky_reissue -- read new key
    WHILE NOT done DO
```

```
            -- Read new key, act on it, and return it
            ACT_TBL(action, def_item, table_data, term_char,
                    attach_sw, status)save_action = action
            action = ky_reissue -- read new key
            -- Debug only
            WRITE TPERROR (CHR(cc_home) + CHR(cc_clear_win))
            -- Process termination keys.
            SELECT (term_char) OF
              CASE (ky_select, ky_new_menu):
                done = TRUE;
              CASE (ky_f1):
                -- Perform F1
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F1 pressed')
              CASE (ky_f2):
                -- Perform F2
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F2 pressed')
              CASE (ky_f3):
                -- Perform F3
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F3 pressed')
              CASE (ky_f4):
                -- Perform F4
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F4 pressed')
              CASE (ky_f5):
                -- Perform F5 Help
                action = ky_help
              CASE (ky_f6):
                -- Perform F6
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F6 pressed')
              CASE (ky_f7):
                -- Perform F7
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F7 pressed')
              CASE (ky_f8):
                -- Perform F8
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F8 pressed')
CASE (ky_f9):
                -- Perform F9
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F9 pressed')
              CASE (ky_f10):
                -- Perform F10
```

```
            SET_CURSOR(TPERROR, 1, 1, status)
            WRITE TPERROR ('F10 pressed')
          CASE (ky_undef):
            -- Process special keys.
            SELECT (save_action) OF
              CASE (ky_f1_s):
                -- Perform Shift F1
                SET_CURSOR(TPERROR, 1, 1, status)
                WRITE TPERROR ('F1 shifted pressed')
              ELSE:
            ENDSELECT
          ELSE:
            action = term_char  -- act on this key
          ENDSELECT
        ENDWHILE
          IF term_char <> ky_new_menu THEN
          -- Cancel the dynamic display
          ACT_TBL(ky_cancel, def_item, table_data, term_char,
                  attach_sw, status)
        ENDIF
      ENDIF
    END tpextbl
```

A.10.13 IN_RANGE 内置函数

用途：返回一个BOOLEAN值，指示一组轴是否可以访问指定的位置参数

语法：IN_RANGE(posn)

功能返回类型: BOOLEAN

Input/Output 参数 :

[in] posn : XYZWPREXT

%ENVIRONMENT Group :SYSTEM

细节 :

- 如果 *posn* 位于轴组的工作范围内，则返回值为 TRUE; 否则，返回FALSE。
- 当前的$ UFRAME和$ UTOOL应用于 *posn*。

另请参阅：CHECK_EPROS内置程序

示例：以下示例在将TCP移动到工作范围之前检查新位置是否在工作范围内。

IN_RANGE 内置函数

```
IF IN_RANGE(part_slot) THEN
  SET_POS_REG(1, part_slot, status)
move_to_pr —— 调用TP程序转移到PR [1]
  ELSE WRITE('I can't get there!',CR)
ENDIF
```

A.10.14 INSERT_NODE 内置程序

用途：在指定的路径节点号之前的指定PATH参数中插入未初始化的节点

语法：INSERT_NODE(path_var, node_num, status)

Input/Output 参数：

[in] path_var :PATH

[in] node_num :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PATHOP

细节：

- *node_num* 指定要在其之前插入新的未初始化节点的路径节点的索引号。
- 可以通过直接引用其 NODEDATA 结构来为新节点分配值。
- 插入节点后面的所有节点都重新编号。
- 有效的 *node_num* 值在 *node_num* => 1和 *node_num* <= PATH_LEN（path_va r）的范围内。
- 如果 *node_num* 不是有效的节点号，则返回状态并显示错误。
- 如果程序没有足够的RAM用于 INSERT_NODE 请求，则会发生错误。
- 如果程序暂停，则不会重试 INSERT_NODE 请求。

另请参阅：DELETE_NODE，APPEND_NODE内置过程

示例：在以下示例中，PATH_LEN内置用于将变量长度设置为等于 *path_var* 中的节点数。INSERT_NODE在 *path_var* 中的最后一个节点之前插入一个新的路径节点。

INSERT_NODE 内置程序

```
length = PATH_LEN(path_var)
INSERT_NODE(path_var, length, status)
```

A.10.15 INSERT_QUEUE 内置程序

用途：如果队列未满，则将条目插入队列

语法：INSERT_QUEUE(value, sequence_no, queue, queue_data, status)

Input/Output 参数：

[in] value :INTEGER

[in] sequence_no :INTEGER

[in,out] queue_t :QUEUE_TYPE

[in,out] queue_data :ARRAY OF INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBQMGR

细节：

- *value* 指定要插入队列的值 *queue_t*。
- *sequence_no* 指定要在其中插入新条目的条目的序列号。
- *queue_t* 指定队列的队列变量。
- *queue_data* 指定用于保存队列中数据的数组。 此数组的长度确定队列中的最大条目数。
- 状态以61002返回，“队列已满”，如果队列中的条目没有空间，则为61003，“错误序列号”，如果指定的序列号不在队列中。

另请参阅：MODIFY_QUEUE,APPEND_QUEUE,DELETE_QUEUE内置过程，第15.8节“使用队列进行任务通信”

示例：在以下示例中，例程 *ins_in_queue* 将条目值添加到指定条目*sequence_no*）之后的队列（queue_t和queue_data）; 如果成功则返回TRUE; 否则返回FALSE。

INSERT_QUEUE 内置程序

```
PROGRAM ins_queue_x
%environment PBQMGR
ROUTINE ins_in_queue(value: INTEGER;
                     sequence_no: INTEGER;
                     queue_t: QUEUE_TYPE;
                     queue_data: ARRAY OF INTEGER): BOOLEAN
VAR
  status: INTEGER
BEGIN
INSERT_QUEUE(value, sequence_no, queue_t, queue_data, status)
return (status = 0)
END ins_in_queue
BEGIN
END ins_queue_x
```

A.10.16 INTEGER 数据类型

用途：将变量，函数返回类型或例程参数定义为 INTEGER 数据类型

语法：INTEGER

细节：

- INTEGER 变量或参数可以采用 -2147483648 到 +2147483646 范围内的整数值。
- INTEGER 文字由一系列数字组成，可选地前面带有加号或减号。 它们不能包含小数点，逗号，空格，美元符号（$）或其他标点字符。 （见表A-14）

表 A-14. 有效和无效的INTEGER文字

有效	无效	原因
1	1.5	小数点不允许（必须是整数）
–2500450	–2,500,450	逗号不允许
+65	+6 5	空格不允许

- 如果将 INTEGER 参数传递给期望 REAL 参数的例程，则将其视为 REAL 并按值传递。

- 只能将INTEGER表达式分配给INTEGER变量，从INTEGER函数例程返回，或作为参数传递给INTEGER参数。
- 有效的INTEGER运算符是：
  - 算术运算符(+, -, *, /, DIV, MOD)
  - 关系运算符(>, >=, =, <>, <, <=)
  - 按位运算 (AND, OR, NOT)

另请参阅：第5章 ROUTINES，有关通过值的更多信息，请参阅第7章 文件输入/输出操作，有关格式说明符的更多信息
示例：有关详细的程序示例，请参阅附录B.

A.10.17 INV 内置函数

用途：用于坐标框架转换，使用相对位置运算符（:)来确定坐标系中POSITION的坐标值，该坐标值与记录POSITION的坐标系不同

语法：INV(pos)

函数返回类型：POSITION

Input/Output 参数：

[in] pos :POSITION

%ENVIRONMENT Group :SYSTEM

细节：

- 返回值是 *pos* 参数的倒数。
- 返回的POSITION的配置将是 *pos* 参数的配置。

示例：以下示例使用 INV 内置函数来确定 part_pos 的 POSITION，并参考以 rack_pos 为原点的坐标系。 part_pos 和 rack_pos 最初都是在用户框架中教授和记录的。 然后指示机器人移动到该位置。

INV 内置函数

```
PROGRAM p_inv
VAR
  rack_pos, part_pos, p1 : POSITION
BEGIN
  p1 = INV(rack_pos):part_pos
  SET_POS_REG(1, p1, status)
  move_to_pr1 -- 调用TP程序转移到PR [1]
END p_inv
```

A.10.18 IO_MOD_TYPE 内置程序

用途：允许KAREL程序确定指定机架/插槽中的模块类型

语法：IO_MOD_TYPE(rack_no, slot_no, mod_type, status)

Input/Output 参数：

[in] rack_no :INTEGER

[in] slot_no :INTEGER

[out] mod_type :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *rack_no* 是包含端口模块的机架。 对于过程I / O板，这是零; 对于Allen-Bradley和Genius端口，这是16。
- *slot_no* 是包含端口模块的插槽。 对于过程I / O板，这是SLC-2链中电路板的位置。
- *mod_type* 是模块类型。

  6 A16B-2202-470

  7 A16B-2202-472

  8 A16B-2202-480
- 如果参数有效且状态为具有指定机架/插槽号的模块或板，则返回零状态，如下所示：

示例：以下示例向调用方返回指定机架和插槽号中的模块。

IO_MOD_TYPE 内置程序

```
PROGRAM iomodtype
%ENVIRONMENT IOSETUP
   ROUTINE get_mod_type(rack_no: INTEGER;
                        slot_no: INTEGER;
                        mod_type: INTEGER): INTEGER
```

```
    VAR
      status: INTEGER
    BEGIN
      IO_MOD_TYPE(rack_no, slot_no, mod_type, status)
      RETURN (status)
    END get_mod_type
BEGIN
END iomodtype
```

A.10.19 IO_STATUS 内置函数

用途：返回 INTEGER 值，指示文件参数上最后一次操作失败的成功或类型

语法：IO_STATUS(file_id)

函数返回类型：INTEGER

Input/Output 参数：

[in] file_id :FILE

%ENVIRONMENT Group :PBCORE

细节：

- IO_STATUS 可以在 OPEN FILE,READ,WRITE,CANCEL FILE 或 CLOSE FILE语句之后使用。 根据操作结果，如果成功或在特定于应用程序的FANUC机器人设置和操作手册中列出的错误之一，它将返回0。 一些常见错误如表A-15所示。

表 A-15. IO_STATUS 错误

0	对指定文件的上一次操作成功
2021	RAM 磁盘设备的文件结束
10006	软盘设备的文件结束
12311	未初始化的变量
12324	非法的开放模式字符串
12325	非法文件字符串
12326	文件 var 已被使用
12327	打开文件失败

表 A-15. IO_STATUS 错误 (续)

12328	文件未打开
12329	无法写入变量
12330	写文件失败
12331	无法读取变量
12332	读取数据太短
12333	用于读取的非法ASCII字符串
12334	读取文件失败
12335	无法打开 pre_defined 文件
12336	无法关闭 pre_defined 文件
12338	关闭文件失败
12347	读取I / O值失败
12348	写入I / O值失败
12358	读取请求超时
12359	读取请求是嵌套的
12367	格式错误的底数

- 使用READ *file_id* (cr) 清除任何IO_STATUS错误。
- 如果 *file_id* 与打开的文件或打开到相应CRT/KB，示教器和视觉窗口的预定义“文件”之一不对应，则程序将中止并显示错误。

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLIST_EX.KL）。

A.11 - J - KAREL 语言描述

A.11.1 J_IN_RANGE 内置函数

用途: 返回一个BOOLEAN值，指示一组轴是否可以到达指定的关节位置参数

语法： J_IN_RANGE(posn)

功能返回类型: BOOLEAN

Input/Output 参数 :

[in] posn :JOINTPOS

%ENVIRONMENT Group :SYSTEM

细节：

- 如果 *posn* 在工作范围内，则返回值为TRUE; 否则，返回FALSE。

另请参阅：IN_RANGE内置函数，CHECK_EPROS内置过程

A.11.2 JOINTPOS 数据类型

用途：将变量，函数返回类型或例程参数定义为JOINTPOS数据类型。

语法：JOINTPOS<n> <IN GROUP[m]>

细节：

- JOINTPOS 包含组中每个轴位置的 REAL 表示，以度或毫米（mm）表示。
- *n* 指定轴的数量，默认值为9。 以字节为单位的大小为4 + 4 * n。
- JOINTPOS 可以跟随 IN GROUP [m]，其中m表示使用数据的运动组。 默认值为% DEFGROUP 指令指定的组或 1。
- CNV_REL_JPOS 和 CNV_JPOS_REL 内置函数可用于访问实际值。
- 可以将 JOINTPOS 分配给其他位置类型。 请注意，某些运动组（例如单轴定位器）没有 XYZWPR 表示。 如果您尝试将 JOINTPOS 分配给此类组的 XYZWPR 或 POSITION 类型，将导致运行时错误。

示例：有关详细的程序示例，请参阅以下部分：

第B.5节“使用寄存器内置函数”（REG_EX.KL）
第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）
第B.8节“沿着六边形路径生成和移动”（GEN_HEX.KL）
第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.11.3 JOINT2POS 内置函数

用途：此例程用于通过调用正向运动学例程将关节角度（in_jnt）转换为笛卡尔位置（out_pos）。

语法： JOINT2POS (in_jnt - Joint angles can be converted to Cartesian, uframe, utool, config_ref, out_pos, wjnt_cfg, ext_ang, and status).

Input/Output 参数：

[in] in_jnt :Jointpos

[in] uframe :POSITION

[in] utool :POSITION

[in] config_ref :INTEGER

[out] out_pos :POSITION

[out] wjnt_cfg :CONFIG

[out] ext_ang :ARRAY OF REAL

[out] status :INTEGER

%ENVIRONMENT Group :MOTN

细节：

- 输入 *in_jnt* 定义为要转换为笛卡尔位置的关节角度。
- 输入 *uframe* 是笛卡尔位置的用户坐标系。
- 输入 *utool* 定义为相应的工具坐标系。
- 输入 *config_ref* 是一个表示所需解决方案类型的整数。 下面列出的值是有效的。 此外，可以使用括号中的预定义常量，并可以根据需要添加值。 一个示例包括：*config_ref* = HALF_SOLN + CONFIG_TCP。
  — 0：（FULL_SOLN）= 默认值
  — 1：（HALF_SOLN）= 腕关节（xyz456）。 该值不计算/使用wpr。
  — 2：（CONFIG_TCP）= 腕关节配置（上/下）基于固定的手腕。
  — 4：（APPROX_SOLN）= 近似解。 此值减少了某些机器人的计算时间。
  — 8：（NO_TURNS）= 忽略手腕转弯数。 使用最近的路径 4,5 和 6（使用 *ref_jnt*）。
  — 16：（NO_M_TURNS）= 忽略长轴（仅限J1）转数。 使用最近的路径。
- 输出 *out_pos* 是对应于输入关节角度的笛卡尔位置。
- 输出 *wjnt_cfg* 是腕关节配置。 当 *config_ref* 对应于 HALF_SOLN 时，将输出该值。

- 输出 *ext_ang* 包含扩展轴的关节角度值（如果存在）。
- 输出 *status* 解释了尝试操作的状态。 如果它不等于0，则表示发生了错误。

A.12 - K - KAREL 语言描述

A.12.1 KCL 内置程序

用途：将 STRING 参数指定的 KCL 命令发送到 KCL 以供执行。

语法： KCL (command, status)

Input/Output 参数：

[in] command :STRING

[out] status :INTEGER

%ENVIRONMENT Group :kclop

细节：

- 命令必须包含有效的KCL命令。
- 命令不能超过126个字符。
- 程序执行等待，直到完成KCL命令的执行或检测到错误。
- 所有KCL命令都像在命令级别输入一样执行，但破坏性命令除外，例如CLEAR ALL，不需要确认。
- *status* 表示命令是否已成功执行。
- 如果正在执行KCL命令文件且 $ STOP_ON_ERR 为FALSE，则KCL内置将继续运行完成。 如果没有发生错误，将返回检测到的第一个错误或0。

另请参阅：KCL_NO_WAIT，KCL_STATUS内置过程

示例：以下示例将显示程序并等待完成。 状态将是此操作的结果。

KCL 内置程序

```
PROGRAM kcl_test
VAR
 command :STRING[20]
 status :INTEGER
BEGIN
```

```
 command = 'SHOW PROGRAMS'
 KCL (command, status)
END kcl_test
```

示例：有关另一个示例，请参阅示例程序仅显示数据项。

A.12.2 KCL_NO_WAIT 内置程序

用途：将STRING参数指定的KCL命令发送到KCL执行，但在继续执行程序之前不等待命令完成。

语法：KCL_NO_WAIT (command, status)

Input/Output 参数：

[in] command :STRING

[out] status :INTEGER

%ENVIRONMENT Group :kclop

细节：

- 命令必须包含有效的KCL命令。
- *status* 表示KCL是否接受了该命令。
- 程序执行等待，直到KCL接受命令或检测到错误。

另请参阅：KCL，KCL_STATUS内置程序

示例：以下示例将加载程序，但不会等到程序在返回之前加载。 状态将指示命令是否被接受。

KCL_NO_WAIT 内置程序

```
PROGRAM kcl_test
VAR
 command :STRING[20]
 status :INTEGER
BEGIN
 command = 'Load prog test_1'
 KCL_NO_WAIT (command, status)
 delay 5000
 status = KCL_STATUS
END kcl_test
```

A.12.3 KCL_STATUS 内置程序

用途：从KCL或KCL_NO_WAIT内置过程返回上次执行的命令的状态。

语法： KCL_STATUS

函数返回类型 : INTEGER

%ENVIRONMENT Group :kclop

细节：

- 从 KCL 或 KCL_NO_WAIT 内置函数返回上次执行的命令的状态。
- 程序执行等待，直到 KCL 可以返回状态。

另请参阅：KCL_NO_WAIT，KCL内置程序

A.13 - L - KAREL 语言描述

A.13.1 LN 内置函数

用途：返回指定 REAL 参数的自然对数

语法： LN(x)

函数返回类型 : REAL

Input/Output 参数 :

[in] x : REAL

%ENVIRONMENT Group :SYSTEM

细节：

- 返回值是 *x* 的自然对数。
- *x* 必须大于零。 否则，程序将因错误而中止。

示例：以下示例返回输入变量 a 的自然对数，并将其分配给变量 b。

LN 内置函数

```
WRITE(CR, CR, 'enter a number =')
  READ(a,CR)
```

```
b = LN(a)
```

A.13.2 LOAD 内置程序

用途：加载指定的文件

语法：LOAD (file_spec, option_sw, status)

Input/Output 参数：

[in] file_spec :STRING

[in] option_sw :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *file_spec* 指定要加载的文件的设备，名称和类型。 以下类型有效：

  .TP 示教器程序

  .PC KAREL 程序

  .VR KAREL 变量

  .SV KAREL 系统变量

  .IO I/O 配置数据

  没有 ext KAREL 程序和变量

- *option_sw* 指定加载期间要执行的选项类型。

  以下值对.TP文件有效：

  — 1 如果程序已存在，则覆盖程序。 如果 *option_sw* 不为1且程序存在，则将返回错误。

  以下值对.SV文件有效：

  — 1 转换系统变量。

- 对于所有其他类型，将忽略 *option_sw*。

- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

以下注释适用于 R-30iB 控制器：

注意必须在机器人控制器上安装 KAREL选项 才能加载 KAREL程序。

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

A.13.3 LOAD_STATUS 内置程序

用途：确定指定的KAREL程序及其变量是否已加载到内存中

语法：LOAD_STATUS(prog_name, loaded, initialized)

Input/Output 参数：

[in] prog_name :STRING

[out] loaded :BOOLEAN

[out] initialized :BOOLEAN

%ENVIRONMENT Group :PBCORE

细节：

- *prog_name* 必须是程序，不能成为 Routine。
- 如果 *prog_name* 当前已加载到内存中，则 *loaded* 返回值TRUE。 如果未加载 *prog_name*，则返回 FALSE。
- 如果 *prog_name* 中的任何变量已初始化，则 *initialized* 返回值TRUE。 如果 *prog_name* 中的所有变量都未初始化，则返回FALSE。
- 如果加载或初始化为FALSE，请使用LOAD内置过程加载 *prog_name* 及其变量。

示例：有关详细的程序示例，请参阅以下部分：

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

A.13.4 LOCK_GROUP 内置程序

用途：锁定指定的一组轴的运动控制

语法：LOCK_GROUP(group_mask, status)

Input/Output 参数：

[in] group_mask : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :MULTI

细节：

- *group_mask* 指定为正在运行的任务锁定的轴组。 组编号必须在 1 到控制器上定义的组的总数的范围内。
- *group_mask* 通过设置所需组的位来指定。

Table A–16. Group_mask 设置

GROUP	十进制	BIT
Group 1	1	1
Group 2	2	2
Group 3	4	3

要指定多个组，请选择表A-16中显示的与所需组对应的十进制值。 然后使用OR运算符将它们连接在一起。 例如，要指定组1，3，请输入“1或4”。

- 对指定的运动组获得运动控制。
- 如果一个或多个组无法锁定，则会返回错误，并且任何可用的组都将被锁定。
- 如果组未被其他任务锁定，则移动组会自动锁定组。
- 如果某个任务尝试移动已被另一个任务锁定的组，它将被暂停。
- 状态解释了尝试操作的状态。 如果不等于 0，则发生错误。

示例：以下示例解锁组1，2和3，然后锁定组3.
有关更多示例，请参阅第15章 多任务。

LOCK_GROUP 内置程序

```
%ENVIRONMENT MOTN
%ENVIRONMENT MULTI
  VAR
    status: INTEGER
  BEGIN
    REPEAT
      -- Unlock groups 1, 2, and 3
      UNLOCK_GROUP(1 OR 2 OR 4, status)
      IF status = 17040 THEN
        CNCL_STP_MTN -- or RESUME
      ENDIF
      DELAY 500
    UNTIL status = 0
    -- Lock only group 3
    LOCK_GROUP(4, status)
  END lock_grp_ex
```

A.13.5 %LOCKGROUP 译者指令

用途：指定从该程序调用该程序或程序时要锁定的运动组。

语法：%LOCKGROUP = n, n ,...

细节：

- *n* 是要锁定的运动组的编号。
- *n* 的范围是1到控制器上的组数。
- 当调用程序或例程时，任务将尝试为所有指定的组获取运动控制，前提是它们尚未锁定它们。 如果无法获得运动控制，任务将暂停。
- 如果未指定 %LOCKGROUP，则所有组都将被锁定。
- 如果不应锁定组，则可以指定 %NOLOCKGROUP 指令。

另请参见：%NOLOCKGROUP 指令,LOCK_GROUP,UNLOCK_GROUP 内置过程

A.14 - M - KAREL 语言描述

A.14.1 MIRROR 内置函数

用途：确定指定位置变量的镜像。

语法：MIRROR (old_pos, mirror_frame, orientation_flag) Function

返回类型：XYZWPREXT

Input/Output 参数：

[in] old_pos :POSITION

[in] mirror_frame :POSITION

[in] orient_flag :BOOLEAN

%ENVIRONMENT Group :MIR

细节：

- 必须相对于同一用户帧定义 *old_pos* 和 *mirror_frame*。
- *old_pos* 指定要生成其镜像的值。
- *mirror_frame* 指定要在其 *xz_plane* 上生成图像的值。
- 如果 *orient_flag* 为TRUE，则 *old_pos* 的 *orientation* 和 *location* 组件都将被镜像。如果为FALSE，则仅镜像该位置，并且新镜像位置的方向与 *old_pos* 相同。
- 返回的镜像位置不保证是可到达的位置，因为镜像位置可以在机器人的工作范围之外。

另请参阅：适用于特定应用的FANUC机器人设置和操作手册，“高级功能”一章

示例：以下示例获取机器人的当前位置，创建镜像框架，并生成围绕y轴镜像的镜像位置。

MIRROR 内置函数

```
PROGRAM mir_exam
VAR
 cur_pos:   XYZWPREXT
 org_pos:   POSITION
 mir_pos:   XYZWPREXT
```

```
 mir_posa:  POSITION
 pos_frame:  XYZWPREXT
 frame:   POSITION
 orient_flag: BOOLEAN
BEGIN
 cur_pos = curpos(0,0) -- Get the current position of the robot
 cur_pos.x = 1000.00  -- Create a new position which is at(1000,0,300,w,p,r)
 cur_pos.y = 0.0
 cur_pos.z = 300.00
 SET_EPOS_REG(1, cur_pos, status)
 move_to_pr1 -— Call TP program to move to PR[1]
             -- The robot is now at a known position:
             --(1000,0,300,w,p,r) where (w,p,r) have not
             --changed from the original position.
 pos_frame = curpos(0,0) -- Create a frame used to mirror about.
 pos_frame.w = 0   -- By setting (w,p,r) to 0, the x-z plane of
 pos_frame.p = 0   -- pos_frame will be parallel to the world's x-z
 pos_frame.r = 0    -- plane. pos_frame now set to (1000,0,300,0,0,0)
 frame = pos_frame   -- Convert the mirror frame to a POSITION type.
 cur_pos.y = 200   -- Move 200mm in the y direction.
 SET_EPOS_REG(1, cur_pos, status)
 move_to_pr1    -- Current position is (1000,200,300,w,p,r)
 org_pos = cur_pos   -- Convert org_pos to a POSITION type.
 orient_flag = FALSE  -- Send Mirror current position: (1000, 200, 300,
                      -- w,p,r), and mirror frame: (1000,0,300,0,0,0).
                      -- Mirrors about the y axis without mirroring the
                      -- orientation (w,p,r).
 mir_pos = mirror(org_pos, frame, orient_flag)
         -- mir_pos is the mirrored position: (1000, -200,
         -- 300, w, p, r).
         -- The orientation is the same as org_pos.
 orient_flag = TRUE  -- The mirrored position includes mirroring of
                     -- the tool orientation.
 mir_posa = mirror(org_pos,frame,orient_flag)
         -- mir_posa is the mirrored position where Normal
         -- Orient, & Approach vectors have been mirrored.
end mir_exam
```

A.14.2 MODIFY_QUEUE 内置程序

用途：替换队列条目的值。

语法：MODIFY_QUEUE(value, sequence_no, queue_t, queue_data, status)

Input/Output 参数：

[in] value :INTEGER

[in] sequence_no :INTEGER

[in,out] queue_t :QUEUE_TYPE

[in,out] queue_data :ARRAY OF INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBQMGR

细节：

- *value* 指定要插入队列的值。
- *sequence_no* 指定要修改其值的条目的序列号
- *queue_t* 指定队列的队列变量
- *queue_data* 指定用于保存队列中数据的数组。 此数组的长度确定队列中的最大条目数。
- 如果指定的序列号不在队列中，则返回 *status* 61003，''错误序列号'。

另请参阅：COPY_QUEUE,GET_QUEUE,DELETE_QUEUE内置过程 第15.8节 “使用队列进行任务通信”

示例：在以下示例中，例程 *update_queue* 替换指定条目的值（*sequence_no*）；队列（队列和 *queue_data* 具有新值（值））。

MODIFY_QUEUE 内置程序

```
PROGRAM mod_queue_x
%ENVIRONMENT PBQMGR
ROUTINE update_queue(value: INTEGER;
                     sequence_no: INTEGER;
                     queue_t: QUEUE_TYPE;
                     queue_data: ARRAY OF INTEGER)
VAR
  status: INTEGER
BEGIN
MODIFY_QUEUE(value, sequence_no, queue_t, queue_data, status)
return
END update_queue
BEGIN
END mod_queue_x
```

A.14.3 MOTION_CTL 内置函数

用途：确定KAREL程序是否具有指定轴组的运动控制

语法： MOTION_CTL<(group_mask)>

函数返回类型：BOOLEAN

Input/Output 参数：

[in] group_mask :INTEGER

%ENVIRONMENT Group :MOTN

细节：

- 如果省略 *group_mask*，则假定程序的默认组掩码。
- 默认的 *group_mask* 由 %LOCKGROUP 和 %NOLOCKGROUP 指令决定。
- 通过设置所需组的位来指定 *group_mask*。

表 A-17. Group_mask 设置

GROUP	DECIMAL	BIT
Group 1	1	1
Group 2	2	2
Group 3	4	3

要指定多个组，请选择表A-17中所示的十进制值，这些值对应于所需的组。 然后使用OR运算符将它们连接在一起。 例如，要指定组1,3，请输入“1 OR 4”。

- 如果KAREL程序具有指定轴组的运动控制，则返回TRUE。

A.14.4 MOUNT_DEV 内置程序

用途：安装指定的设备

语法： MOUNT_DEV (device, status)

Input/Output 参数：

[in] device : STRING

[out] status :INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *device* 指定要安装的设备。
- *status* 说明尝试操作的状态。 如果它不等于0，则发生错误。

另请参见：DISMOUNT_DEV和FORMAT_DEV

示例：有关详细的程序示例，请参见第B.9节“使用文件和设备内置函数”（FILE_EX.KL）。

A.14.5 MOVE_FILE 内置程序

用途：将指定文件从一个内存文件设备移动到另一个内存文件设备

语法：MOVE_FILE (file_spec, status)

Input/Output 参数：

[in] file_spec : string

[out] status : integer

%ENVIRONMENT Group :FDEV

细节：

- *file_spec* 指定要移动的文件的设备，名称和类型。 该文件应存在于FROM或RAM磁盘上。
- 如果 *file_spec* 是FROM磁盘上的文件，则该文件将移动到RAM磁盘，反之亦然。
- 通配符（*）可用于替换整个文件名，文件名的第一部分，文件名的最后部分，或文件名的第一部分和最后部分。 文件类型也可以以相同的方式使用通配符。 如果 *file_spec* 指定多个文件，则它们都将移动到另一个磁盘。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

示例：在以下示例中，所有.KL文件都从RAM磁盘移动到FROM磁盘。

MOVE_FILE 内置程序

```
PROGRAM move_files
```

```
%NOLOCKGROUP
%ENVIRONMENT FDEV
VAR
  status: INTEGER
BEGIN
  MOVE_FILE('RD:\*.KL', status)
  IF status <> 0 THEN
    POST_ERR(status, '', 0, 0)
  ENDIF
END move_files
```

A.14.6 MSG_CONNECT 内置程序

用途：将客户端或服务器端口连接到另一台计算机以在Socket 通讯中使用。

语法： MSG_CONNECT (tag, status)

Input/Output 参数：

[in] tag :STRING

[out] status :INTEGER

%ENVIRONMENT Group :FLBT

细节：

- *tag* 是客户端端口（C1：-C8）或服务器端口（S1：S8）的名称。
- *status* 解释了尝试操作的状态。 如果它不等于0，则发生错误。

另请参阅：FANUC Robotics Internet选项设置和操作手册中的Socket 通讯。

示例：以下示例连接到S8：并读取消息。 消息显示在示教器屏幕上。

MSG_CONNECT 内置程序

```
PROGRAM tcpserv8
VAR
  file_var :  FILE
  tmp_int  :  INTEGER
  tmp_int1 :  INTEGER
  tmp_str  :  string [128]
  tmp_str1 :  string [128]
  status   :  integer
```

```
  entry    :  integer
BEGIN
   SET_FILE_ATR (file_var, ATR_IA)
  -- Set up S8 server tag
  DISMOUNT_DEV('S8:',status)
  MOUNT_DEV('S8:',status)
  write  (' Mount Status = ',status,cr)
  status = 0
  IF status = 0 THEN
    -- Connect the tag
    write ('Connecting ..',cr)
    MSG_CONNECT ('S8:',status)
    write  ('Connect Status = ',status,cr)
    IF status < > 0 THEN
      MSG_DISCO('S8:',status)
      write  (' Connecting..',cr)
      MSG_CONNECT('S8:',status)
      write  (' Connect Status = ',status,cr)
      ENDIF
    IF status = 0 THEN
      -- OPEN S8:
      write ('Opening',cr)
      OPEN FILE file_var ('rw','S8:')
      status = io_status(file_var)
      FOR tmp_int  1 TO 1000 DO
        write  ('Reading',cr)
        BYTES_AHEAD(file_var, entry, status)
        -- Read 10 bytes
        READ file_var (tmp_str::10)
        status = i/o_status(file_var)
       --Write 10 bytes
       write (tmp_str::10,cr)
       status = io_status(file_var)
       ENDFOR
     CLOSE FILE file_var
     write  ('Disconnecting..',cr)
     MSG_DISCO('S8:',status)
     write  ('Done.',cr)
   ENDIF
 ENDIF
END tcpserv8
```

A.14.7 MSG_DISCO 内置程序

用途：断开客户端或服务器端口与另一台计算机的连接。

语法：MSG_DISCO (tag, status)

Input/Output 参数：

[in] tag :STRING

[out] status :INTEGER

%ENVIRONMENT Group :FLBT

细节：

- *tag* 是客户端端口（C1：-C8）或服务器端口（S1：S8）的名称。
- *status* 说明了尝试操作的状态。 如果它不等于0，则发生错误。

另请参阅：FANUC Robotics Internet 选项设置和操作手册中的Socket 通讯。

示例：有关更多示例，请参阅MSG_CONNECT内置过程。

A.14.8 MSG_PING

语法：MSG_PING (host name, status)

Input/Output 参数：

[in] host name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :FLBT

细节：

- 主机名是要执行检查的主机的名称。 主机的条目必须存在于主机条目表（或在机器人上加载和配置的DNS选项）中。
- *status* 说明了尝试操作的状态。 如果它不等于0，则发生错误。

另请参阅：FANUC Robotics Internet选项设置和操作手册中的Ping。

示例：以下示例对主机名“fido”执行PING检查。 它将结果写在示教器上。

MSG_PING 内置程序

```
PROGRAM pingtest
VAR
 Tmp_int  : INTEGER
 Status   : integer
BEGIN
  WRITE('pinging..',cr)
  MSG_PING('fido',status)
  WRITE('ping Status = ',status,cr)
END pingtest
```

A.15 - N - KAREL 语言描述

A.15.1 NOABORT 功能

用途：防止发生外部错误时程序执行中止

细节：

- NOABORT 操作通常对应于一个 ERROR [n]。
- 如果程序本身被中止（即，执行ABORT语句，运行时错误），则 NOABORT 操作将被忽略，并且程序执行将被中止。

示例：以下示例使用全局条件处理程序来测试错误编号11038“脉冲不匹配”。 如果发生此错误，NOABORT 操作将阻止程序执行中止。

NOABORT 功能

```
PROGRAM noabort_ex
%NOLOCKGROUP
BEGIN
  --Pulse Mismatch condition handler
  CONDITION[801]:
    WHEN ERROR[11038] DO
    NOABORT
  ENDCONDITION
  ENABLE CONDITION[801]
END noabort_ex
```

A.15.2 %NOABORT 译者指令

用途：指定用于中止的掩码

语法：%NOABORT = ERROR + COMMAND

细节：

- ERROR 和 COMMAND 定义如下：

```
ERROR : ignore abort error severity
   COMMAND : ignore abort command
```

- 任何组合的 ERROR 和 COMMAND 可指定。
- 如果程序本身中止（例如，执行ABORT语句，运行时错误），则 %NOABORT 指令将被忽略，并且程序执行将被中止。
- 该指令仅对使用 %NOLOCKGROUP 的程序有效。 如果程序有运动控制，则 %NOABORT 指令将被忽略，程序执行将被中止。

A.15.3 %NOBUSYLAMP 译者指令

用途：指定在执行期间忙灯将关闭。

语法：%NOBUSYLAMP

细节：

- 繁忙的指示灯可以在执行任务期间通过内置的 SET_TSK_ATTR 进行设置。

A.15.4 NODE_SIZE 内置函数

用途：返回PATH节点的大小（以字节为单位）

语法：NODE_SIZE(path_var)

函数返回类型：INTEGER

Input/Output 参数：

[in] path_var： PATH

%ENVIRONMENT Group :PATHOP

细节：

- 返回的值是单个PATH节点的大小，包括位置数据类型大小和任何关联数据。
- 返回的值可用于计算文件中随机访问文件的位置。

例子：下面的例子程序读取一个路径，同时用前面的移动重叠读取。 例程read_header读取路径头并准备读取节点。 例程 read_node 读取一个路径节点。

NODE_SIZE 内置函数

```
PROGRAM read_and_mov
VAR my_path: PATH
    xyz_pos: XYZWPR
    path_base: INTEGER
    node_size: INTEGER
    max_node_no: INTEGER
    i: INTEGER
    file_var:  FILE
--
ROUTINE read_header
BEGIN
  READ file_var(my_path[0])
  IF IO_STATUS(file_var) <> 0 THEN
    WRITE('HEADER READ ERROR:',IO_STATUS(file_var),cr)
    ABORT
  ENDIF
  max_node_no = PATH_LEN(my_path)
  node_size = NODE_SIZE(my_path)
  path_base = GET_FILE_POS(file_var)
END read_header
--
ROUTINE read_node(node_no: INTEGER)
VAR status: INTEGER
BEGIN
  SET_FILE_POS(file_var, path_base+(node_no-1)*node_size, status)
  READ file_var(my_path[node_no])
END read_node
--
BEGIN
  SET_FILE_ATR(file_var, atr_uf)
  OPEN FILE F1('RO','PATHFILE.DT')
  read_header
  FOR i = 1 TO max_node_no DO
    read_node(i)
    xyz_pos = my_path[i]
    SET_POS_REG(1 xyz_pos, status)
    move_to_pr1 —— Call TP program to move to node
  ENDFOR
```

```
  CLOSE FILE file_var
END read_and_mov
```

A.15.5 %NOLOCKGROUP 译者指令

用途：指定调用该程序时不需要锁定运动组，或者在此程序中定义的例程。.

语法：%NOLOCKGROUP

细节：

- 当调用程序或例程时，任务不会尝试获取运动控制。
- 如果未指定 %NOLOCKGROUP，则在调用程序或例程时将锁定所有组，并且该任务将尝试获取运动控制。如果无法获得运动控制，任务将暂停。
- 任务将在执行程序或例程时保持运动控制。当它退出程序或例程时，该任务会自动解锁所有运动组。
- 如果任务包含正在执行或已停止的动作，则会保留任务执行直到动作完成。停止的动作必须恢复并完成或取消。
- 如果运动控制程序使用 %NOLOCKGROUP 指令调用程序或程序中定义的例程，程序将保持运动控制，即使不需要。
- UNLOCK_GROUP 内置例程可用于释放控制权。
- 如果在具有 %NOLOCKGROUP 指令的程序中遇到运动语句，则该任务将尝试为所有需要的组（如果它尚未拥有它）获取运动控制。如果无法获得运动控制，任务将暂停。

另请参见：%LOCKGROUP 转换器指令，LOCK_GROUP，UNLOCK_GROUP内置过程

示例：请参阅以下各节以获取详细的程序示例：

第B.3节“将数据保存到默认设备”（SAVE_VR.KL）

第B.5节“使用注册内置程序”（REG_EX.KL）

第B.10节“使用动态显示内置插件”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

第B.12节“从字典文件中显示列表”（DCLST_EX.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

A.15.6 NOMESSAGE 功能

用途：禁止显示和记录错误消息

语法：NOMESSAGE

细节：

- 显示和记录错误消息仅在相应条件中指定的错误编号被抑制。
- 使用通配符（*）来抑制所有消息。
- 即使使用 NOMESSAGE，中止错误消息仍会显示并记录。

例如：详细程序示例请参考第B.1节“设置数字输出端口进行监视”（DOUT_EX.KL）。

A.15.7 NOPAUSE 功能

用途：在程序暂停时恢复程序执行，或阻止程序执行暂停

语法：NOPAUSE

细节：

- NOPAUSE操作通常对应于ERROR [n]或PAUSE条件。
- 程序将被恢复，即使它在错误发生之前暂停。
- 如果程序本身暂停，NOPAUSE操作将被忽略，程序执行将暂停。

示例：以下示例使用全局条件处理程序来测试错误号12311.如果发生此错误，NOPAUSE操作将阻止程序执行被暂停，NOMESSAGE操作将抑制通常显示错误号12311的错误消息。这将 允许例程 uninit_error 不中断地执行。

NOPAUSE 功能

```
ROUTINE uninit_error
  BEGIN
    WRITE ('Uninitialized operand',CR)
    WRITE ('Use KCL> SET VAR to initialize operand',CR)
```

```
    WRITE ('Press Resume at Test/Run screen to ',cr)
    WRITE ('continue program',cr)
    PAUSE  --pauses program (undoes NOPAUSE action)
  END uninit_error
CONDITION[1]:
  WHEN ERROR[12311] DO
    NOPAUSE, NOMESSAGE, uninit_error
ENDCONDITION
```

A.15.8 %NOPAUSE 译者指令

用途：指定用于暂停的掩码

语法：%NOPAUSE = ERROR + COMMAND + TPENABLE

细节：

- 掩码的位如下所示：

ERROR ： 忽略暂停错误严重性

COMMAND ： 忽略暂停命令

TPENABLE ： 在TP启用时忽略暂停的请求

- 可以指定 ERROR，COMMAND 和 TPENABLE 的任意组合。
- 如果程序本身暂停，则 %NOPAUSE 指令将被忽略，程序执行将暂停。
- 该指令仅对使用 %NOLOCKGROUP 的程序有效。 如果程序有运动控制，则 %NOPAUSE 指令将被忽略，程序执行将暂停。

A.15.9 %NOPAUSESHFT 译者指令

用途：指定如果移位键被释放，任务不会暂停。

语法：%NOPAUSESHFT

细节：

- 该属性可以在任务执行期间由 SET_TSK_ATTR 内置例程设置。

A.16 - O - KAREL 语言描述

A.16.1 OPEN FILE 声明

用途：将数据文件或通信端口与文件变量关联

语法：OPEN FILE file_var (usage_string, file_string)

其中：

file_var：FILE

usage_string：a STRING

file_string：a STRING

细节：

- *file_var* 必须是另一个 OPEN FILE 语句尚未使用的静态变量。
- *usage_string* 由以下内容组成：

  'RO'：只读

  'RW'：读写

  'AP'：追加

  'UD'：更新
- *file_string* 标识数据文件名和类型，窗口或键盘或通信端口。
- SET_FILE_ATR 内置例程可用于设置文件的属性。
- 当程序中止或正常退出时，任何打开的文件都将关闭。 程序暂停时文件不会关闭。
- 使用 IO_STATUS 内置函数验证打开文件操作是否成功。

另请参阅：IO_STATUS 内置函数，SET_FILE_ATR 内置过程，第7章 文件输入/输出操作，第9章 文件系统，附录E，''语法图''了解更多语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

A.16.2 OPEN HAND 声明

用途：打开机器人的 HAND

语法：OPEN HAND hand_num

其中 :

hand_num : INTEGER 表达式

细节 :

- 声明的实际效果取决于HAND信号的设置方式。 请参阅第13章 输入/输出系统。
- *hand_num* 必须是1-2范围内的值。 否则，程序将中止并显示错误。
- 如果 *hand_num* 的值在范围内但未连接手，则该语句无效。
- 如果 *hand_num* 的值在范围内，但尚未分配由该值表示的HAND信号，则程序将中止并显示错误。

另请参阅：附录D，''语法图''以获取更多语法信息

示例：以下示例将TCP移动到位置寄存器PR [2]并打开由INTEGER变量 *hand_num* 指定的机械手。

OPEN HAND 声明

```
move_to_pr2  ——  Call TP program to move to PR[2]
OPEN HAND hand_num
```

A.16.3 OPEN_TPE 内置程序

用途：打开指定的示教程序

语法：OPEN_TPE(prog_name, open_mode, reject_mode, open_id, status)

Input/Output 参数 :

[in] prog_name :STRING

[in] open_mode :INTEGER

[in] reject_mode :INTEGER

[out] open_id :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *prog_name* 指定要打开的示教程序的名称。 *prog_name* 必须是全大写字母。
- 必须先使用 CLOSE_TPE 关闭 *prog_name*，然后才能执行 *prog_name*。
- *open_mode* 确定程序的访问代码。 访问代码定义如下：

  0：无

  TPE_RDACC：读取访问权限

  TPE_RWACC：读/写访问
- *reject_mode* 确定程序的拒绝代码。 带有拒绝代码的程序无法被其他程序打开。 拒绝代码定义如下：

  TPE_NOREJ：无

  TPE_RDREJ：读取拒绝

  TPE_WRTREJ：写入拒绝

  TPE_RWREJ：读/写拒绝

  TPE_ALLREJ：全部拒绝
- *open_id* 表示已打开程序的ID号。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。
- 当KAREL程序中止或正常退出时，所有打开的示教器程序将自动关闭。

另请参见：CREATE_TPE 内置过程，COPY_TPE 内置过程，AVL_POS_NUM 内置过程

示例：请参见第B.14节“将偏移应用于复制的示教器程序”（CPY_TP.KL），详细的程序示例。

A.16.4 ORD 内置函数

用途：返回与index参数引用的STRING参数中的字符对应的数字ASCII代码

语法： ORD(str, str_index)

函数返回类型：INTEGER

Input/Output 参数：

[in] str :STRING

[in] str_index :INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- 返回值表示指定字符的 ASCII 数字代码。
- *str_index* 指定参数 *str* 中字符的索引位置。 值为1表示第一个字符。
- 如果 *str_index* 小于1或大于当前 *str* 的长度，则程序暂停并显示错误。

另请参阅：附录D，''字符代码''

示例：有关详细的程序示例，请参见第B.12节 “从字典文件显示列表”（DCLST_EX.KL）。

A.16.5 ORIENT 内置函数

用途：返回表示指定POSITION参数的y轴（方向矢量）的单位VECTOR

语法： ORIENT(posn)

函数返回类型：VECTOR

Input/Output 参数：

[in] posn： POSITION

%ENVIRONMENT Group :VECTR

细节：

- 您可以直接访问 POSITION 的 Orient Vector，而不是使用此内置功能。
- 返回值是 *posn* 的方向向量。
- 定向矢量是工具坐标系中的正 *y* 方向。

A.17 - P - KAREL 语言描述

A.17.1 PATH 数据类型

用途：将变量或例程参数定义为PATH数据类型

语法： PATH

细节：

- PATH 是一个变长的元素列表，称为路径节点，从1到 PATH 中的节点数。
- 未定义与 PATH 变量一起使用的有效运算符。
- PATH 变量被索引（或下标），就好像它是ARRAY变量一样。例如，*tool_track* [1]指的是名为 *tool_track* 的 PATH 的第一个节点。
- 未初始化的 PATH 长度为零。
- PATH 变量不能在例程中声明为本地，也不能从函数返回。
- 只能将 PATH 表达式分配给PATH变量或作为参数传递给PATH参数。
- PATH 变量可以指定构成每个路径节点的数据的数据结构。
- PATH 变量可以指定构成路径头的数据结构。这可用于指定用于记录路径的UFRAME和/或 UTOOL。它还可以指定一个轴组，其当前位置定义了一个桌面坐标系，相对于该桌面坐标系记录了机器人数据。
- 可以使用PATH一词后面的任何一个，两个或两个子句声明PATH：
  — NODEDATA = node_struct_name，指定构成路径节点的数据结构。-PATHHEADER = header_struct_name，指定构成路径头的结构。

如果两个字段都存在，则它们可以按任何顺序出现，并以逗号和可选的新行分隔。

- 如果未指定 NODEDATA，则默认为附录A中描述的 STD_PTH_NODE 结构。
- 如果未指定 PATHHEADER，则没有（用户可访问的）路径标头。
- 可以使用语法引用 PATHHEADER 结构的元素 path_var_name.header_field_name。
- 可以使用语法引用 NODEDATA 结构的元素 path_var_name[node_no].node_field_name。

- 可以使用 path_var1 = path_var2 语句将路径头结构从一个路径复制到另一个路径。
- 可以使用 path_var [2] = path_var [1]语句将路径节点结构从一个节点复制到另一个节点。
- 可以将路径传递给远程传输，以便与 THATHHEADER 和 NODEDATA 类型匹配。 作为参数传递给内置例程的路径可以是任何类型。
- 只要例程参数与 NODEDATA 结构的类型相同，路径节点就可以作为参数传递给例程。
- 可以使用没有位置类型元素的 NODEDATA 结构声明路径。 这可以是维护列表结构的有用方法。

另请参阅：APPEND_NODE,DELETE_NODE,INSERT_NODE内置过程,PATH_LEN,NODE_SIZE内置函数

示例：以下示例显示了PATH变量的不同声明。

PATH 数据类型

```
TYPE
      node_struct = STRUCTURE
        node_posn: XYZWPR IN GROUP[1]
        aux_posn: JOINTPOS IN GROUP[2]
        weld_time: INTEGER
        weld_current: INTEGER
      ENDSTRUCTURE
    hdr_struct = STRUCTURE
        uframe1: POSITION
        utool: POSITION
        speed: REAL
      ENDSTRUCTURE
    VAR
      path_1a: PATH PATHHEADER = hdr_struct, NODEDATA = node_struct
      path_1b: PATH NODEDATA = node_struct, PATHHEADER = hdr_struct
      path_2:  PATH NODEDATA = node_struct -- no header
      path_3:  PATH -- NODEDATA is STD_PTH_NODE
      path_4:  PATH PATHHEADER = hdr_struct -- NODEDATA is STD_PTH_NODE
```

以下示例显示如何引用 NODEDATA 和 PATHHEADER 结构的元素。

PATH 数据类型

```
-- Using declarations for path_1a:
-- Using NODEDATA fields:
 path_1a[1].node_posn = CURPOS(0, 0)
```

```
 cnt_dn_time = path_1a[node_no].weld_time
-- Using PATHHEADER fields:
 path_1a.utool = tool_1
```

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

A.17.2 PATH_LEN 内置函数

用途：返回PATH参数中的节点数

语法：PATH_LEN(path_nam)

函数返回类型：INTEGER

Input/Output 参数：

[in] path_nam：PATH

%ENVIRONMENT Group :PBCORE

细节：

- 返回值对应于 PATH 变量参数中的节点数。
- 使用未初始化的 PATH 调用 PATH_LEN 将返回零值。

另请参见：COPY_PATH 内置

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.17.3 PAUSE 动作

用途：暂停执行正在运行的任务

语法：PAUSE <PROGRAM[n]>

细节：

- PAUSE 操作按以下方式暂停任务执行：
  — 任何已启动的动议将持续至完成。
  — 文件保持打开状态。
  — 所有连接的计时器继续递增。
  — 执行中的所有PULSE语句继续执行。
  — 继续感测条件处理程序中指定的条件。
  — 完成除例行调用操作之外的任何操作。 恢复程序时执行例行调用操作。
- PAUSE 操作之后可以是 PROGRAM [n] 子句，其中n是要暂停的任务编号。
- 使用 GET_TSK_INFO 查找任务编号。

另请参见：UNPAUSE 动作

A.17.4 PAUSE 条件

用途：监视程序执行的暂停

语法：PAUSE < PROGRAM [n] >

细节：

- 程序暂停时（例如，错误，PAUSE 语句或 PAUSE 操作）暂停 PAUSE 条件。
- 如果对应于 PAUSE 条件的操作之一是例程调用，则必须指定 NOPAUSE 操作以允许执行例程。
  此外，被调用的例程需要包含 PAUSE 语句，以便系统可以完全处理原始暂停的原因。
- PAUSE 条件之后可以是 PROGRAM [n]子句，其中n是要暂停的任务编号。
- 使用 GET_TSK_INFO 查找任务编号。

示例：以下示例在全局条件处理程序中扫描PAUSE条件。 如果满足此条件，DOUT [1]将打开。 CONTINUE操作继续执行程序; ENABLE重新启用条件处理程序。

PAUSE 条件

```
CONDITION[1]:
  WHEN PAUSE DO
    DOUT[1] = TRUE
    CONTINUE
    ENABLE CONDITION[1]
ENDCONDITION
```

A.17.5 PAUSE 声明

用途：暂停执行 KAREL 程序

语法：PAUSE < PROGRAM [n] >

细节：

- PAUSE 语句以下列方式暂停程序执行：
  — 任何已启动的动议将持续至完成。
  — 文件保持打开状态。
  — 所有连接的计时器继续递增。
  — 执行中的所有PULSE语句继续执行。
  — 继续感测条件处理程序中指定的条件。
  — 完成除例行呼叫操作之外的任何操作。 恢复程序时执行例行调用操作。
- PAUSE 语句之后可以是 PROGRAM [n]子句，其中 *n* 是要暂停的任务编号。
- 使用 GET_TSK_INFO 查找任务编号。

另请参阅：附录E，''语法图'，'了解更多语法信息

示例：如果DIN [1]为TRUE，则以下示例使用PAUSE语句暂停KAREL程序。 消息 “''程序已暂停。 按RESUME功能键继续''将显示在CRT / KB屏幕上。

PAUSE 声明

```
PROGRAM p_pause
BEGIN
   IF DIN[1] THEN
      WRITE ('Program is Paused. ')
      WRITE ('Press RESUME function key to continue', CR)
      PAUSE
```

```
   ENDIF
END p_pause
```

A.17.6 PAUSE_TASK 内置程序

用途：暂停指定的执行任务

语法：PAUSE_TASK(task_name, force_sw, stop_mtn_sw, status)

Input/Output 参数：

[in] task_name :STRING

[in] force_sw :BOOLEAN

[in] stop_mtn_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group :MULTI

细节：

- *task_name* 是要暂停的任务的名称。 如果任务名称为"* ALL *"，则除了具有“忽略暂停请求”属性集的任务外，所有正在执行的任务都将暂停。
- *force_sw* 指定即使任务具有“忽略暂停请求”属性集，是否应暂停任务。 如果 *task_name* 为'* ALL *'，则忽略此参数。
- *stop_mtn_sw* 指定是否停止属于指定任务的所有运动组。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：RUN_TASK,CONT_TASK,ABORT_TASK 内置程序，第15章 多任务

示例：以下示例暂停用户指定的任务并停止任何动作。 有关更多示例，请参阅第15章 多任务。

PAUSE_TASK 内置程序

```
PROGRAM pause_ex
%ENVIRONMENT MULTI
VAR
  task_str: STRING[12]
  status  : INTEGER
BEGIN
  WRITE('Enter task name to pause:')
```

```
  READ(task_str)
  PAUSE_TASK(task_str, TRUE, TRUE, status)
END pause_ex
```

A.17.7 PEND_SEMA 内置程序

用途：暂停执行任务，直到信号量的值大于零或max_time到期为止

语法：PEND_SEMA(semaphore_no, max_time, time_out)

Input/Output 参数：

[in] semaphore_no :INTEGER

[in] max_time :INTEGER

[out] time_out :BOOLEAN

%ENVIRONMENT Group :MULTI

细节：

- PEND_SEMA 减少信号量的值。
- *semaphore_no* 指定要使用的信号量号。
- *semaphore*_no 必须在1到控制器上定义的信号量数量的范围内。
- *max_time* 指定到期时间，以毫秒为单位。 如果需要，*max_time* 值为-1表示永远等待。
- 在延续时，如果 *max_time* 到期而没有信号量变为非零，则将 *time_out* 设置为TRUE，否则将其设置为 FALSE。

另请参阅：POST_SEMA，CLEAR_SEMA内置过程，SEMA_COUNT内置函数，第15章多任务

示例：请参阅第15章“多任务”中的示例

A.17.8 PIPE_CONFIG 内置程序

用途：配置管道以供特殊使用。

语法：pipe_config(pipe_name, cmos_flag, n_sectors, record_size, form_dict, form_ele, status)

Input/Output 参数 :

[in] pipe_name :STRING

[in] cmos_flag :BOOLEAN

[in] n_sectors :INTEGER

[in] record_size :INTEGER

[in] form_dict :STRING

[in] form_ele :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :FLBT

细节 :

- *pipe_name* 是管道文件的名称。 如果该文件不存在，则将使用此操作创建该文件。
- CMOS_flag 如果设置为TRUE，则将管道数据置于CMOS中。 默认情况下，管道数据位于DRAM中。
- *n_sectors* 要分配给管道的1024字节扇区数。 默认值为8。
- *record_size* 管道中二进制记录的大小。 如果设置为0，则管道将被视为ASCII。 如果管道是二进制的并且将作为格式化数据打印，则必须将其设置为记录长度。
- *form_dict* 是包含格式信息的字典的名称。
- *form_ele* 是 *form_dict* 中包含格式信息的元素编号。
- *status* 说明了尝试操作的状态。 如果它不等于0，则发生错误。

另请参见 : 有关详细信息，请参见第9.3.4节中的“PIP：Device”。

A.17.9 POP_KEY_RD 内置程序

用途 : 从键盘设备恢复键输入

语法 : POP_KEY_RD(key_dev_name, pop_index, status)

Input/Output 参数 :

[in] key_dev_name :STRING

[in] pop_index :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 恢复指定键盘设备上的所有暂停读取请求。
- 如果暂停时没有活动的读取请求，则此操作将不会恢复任何输入。 这不是错误。
- *key_dev_name* 必须是已定义的键盘设备之一：

  'TPKB'：教导吊坠键盘设备

  'CRKB'：CRT键盘设备
- pop_id 从 PUSH_KEY_RD 返回，应该用于重新激活读取请求。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：PUSH_KEY_RD，READ_KB 内置过程

示例：请参阅READ_KB内置例程的示例。

A.17.10 Port_Id 动作

用途：将port数组元素的值设置为表达式的结果

语法： port_id[n] = expression

其中：

port_id：输出端口数组

n :an INTEGER

expression：变量，常量或EVAL子句

细节：

- *expression* 的值分配给n引用的端口数组元素。
- 端口数组必须是可由KAREL程序写入的输出端口数组。 请参阅第2章“语言元素”。
- *expression* 可以是用户定义的静态变量，可以由KAREL程序读取的系统变量，任何常量或EVAL子句。

- 如果 *expression* 是一个变量，则使用的值是采取操作时变量的当前值，而不是在定义条件处理程序时。
- 如果 *expression* 是EVAL子句，则在定义条件处理程序时对其进行求值，并在执行操作时分配该值。
- 表达式必须与 *port_id* 的类型相同。
- 您不能直接将端口数组元素分配给端口数组元素。
- 如果表达式是在启用条件处理程序时未初始化的变量，则程序将因错误而中止。

另请参阅：第6章 条件处理程序，第7章 文件输入/输出操作，关系条件，附录A，''KAREL语言按字母顺序描述''

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.17.11 Port_Id 条件

用途：监控数字端口信号

语法：<NOT> port_id[n] < + | - >

其中：

port_id：一个端口数组

n： 一个 INTEGER

细节：

- *n* 指定要监视的端口阵列信号。
- *port_id* 必须是具有读访问权限的预定义 BOOLEAN 端口阵列标识符之一。 请参阅第2章 “语言元素”。
- 对于事件条件，仅使用+或 - 替代。
- 对于状态条件，仅使用NOT替代方案。

另请参阅：第6章条件处理程序，第7章文件输入/输出操作，关系条件，附录A，''KAREL语言按字母顺序描述''

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.17.12 POS 内置函数

用途：返回由指定的位置参数 (x,y,z)，方向参数 (w,p,r) 和配置参数(c)组成的 XYZWPR

语法： POS(x, y, z, w, p, r, c)

函数返回类型： XYZWPR

Input/Output 参数：

[in] x, y, z, w, p, and r :REAL

[in] c :CONFIG

%ENVIRONMENT Group :SYStem

细节：

- *c* 必须是连接到控制器的机器人的有效配置。 CNV_STR_CONF 可用于将字符串转换为 CONFIG 变量。
- *x*，*y* 和 *z* 是位置的笛卡尔值（以毫米为单位）。 每个参数必须在 ±10000000 mm（±10 km）范围内。 否则，程序暂停并显示错误。
- *w*，*p*，和 *r* 是方向的 yaw, pitch, 和 roll 值（以度为单位）。 每个参数必须在±180度的范围内。 否则，程序暂停并显示错误。

另请参见：第8章 位置数据

示例：以下示例使用POS 内置函数以数字方式指定POSITION next_pos。

POS 内置函数

```
CNV_STR_CONF('n', config_var, status)
next_pos = POS(100,-400.25,0.5,10,-45,30,config_var)
```

A.17.13 POS2JOINT 内置函数

用途：该例程用于通过调用逆运动学例程将笛卡尔位置(in_pos)转换为关节角度(out_jnt)。

语法： POS2JOINT (ref_jnt, in_pos, uframe, utool, config_ref, wjnt_cfg, ext_ang, out_jnt, and status).

Input/Output 参数：

[in] ref_jnt :JOINTPOS

[in] in_pos :POSITION

[in] uframe :POSITION

[in] utool :POSITION

[in] config_ref :INTEGER

[in] wjnt_cfg :CONFIG

[in] ext_ang :ARRAY OF REAL

[out] out_jnt :JOINTPOS

[out] status :INTEGER

%ENVIRONMENT Group :MOTN

细节：

- 输入 *ref_jnt* 是参考关节角度，表示机器人在移动到当前位置之前的位置。
- 输入 *in_pos* 是要转换为关节角度的机器人笛卡尔位置。
- 输入 *uframe* 是笛卡尔位置的用户坐标系。
- 输入 *utool* 是相应的工具坐标系。
- 输入 *config_ref* 是一个表示所需解决方案类型的整数。下面列出的值是有效的。此外，可以使用括号中的预定义常量，并可以根据需要添加值。一个示例包括：*config_ref* = HALF_SOLN + CONFIG_TCP。
  — 0：（FULL_SOLN）=默认值
  — 1：（HALF_SOLN）=腕关节（XYZ456）。这不计算/使用WPR。
  — 2：（CONFIG_TCP）=腕关节配置（上/下）基于固定的手腕。
  — 4：（APPROX_SOLN）=近似解。减少一些机器人的计算时间.- 8：
  （NO_TURNS）=忽略手腕转弯数。使用最近的路径4,5和6（使用*ref_jnt*）。
  — 16：（NO_M_TURNS）=忽略长轴（仅限J1）转数。使用最近的路径。
- 输入 *wjnt_cfg* 是腕关节配置。当 *config_ref* 对应于 HALF_SOLN 时，必须输入该值。
- 输入 *ext_ang* 包含扩展轴的关节角度值（如果存在）。
- 输出 *out_jnt* 是对应于笛卡尔位置的关节角度

- 输出 *status* 解释了尝试操作的状态。 如果它不等于0，则表示发生了错误。

A.17.14 POS_REG_TYPE 内置程序

用途：返回指定位置寄存器的位置表示

语法：POS_REG_TYPE (register_no, group_no, posn_type, num_axes, status)

Input/Output 参数：

[in] register：INTEGER

[in] group_no：INTEGER

[out] posn_type：INTEGER

[out] num_axes：INTEGER

[out] status：INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定位置寄存器。
- 如果省略 *group_no*，则假定程序的默认组。
- 如果指定了*group_no*，则它必须在1到控制器上定义的组总数的范围内。
- *posn_type* 返回位置类型。 *posn_type* 定义如下：
  - — 1：POSITION
  - — 2：XYZWPR
  - — 6：XYZWPREXT
  - — 9：JOINTPOS
- 如果位置类型是 JOINTPOS，*num_axes* 将返回表示中的轴数。 如果位置类型是 XYZWPREXT，则 *num_axes* 仅返回扩展轴的数量。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_POS_REG,GET_JPOS_REG,SET_POS_REG,SET_JPOS_REG 内置程序

示例：以下示例确定寄存器中的位置类型，并使用适当的内置来获取数据。

POS_REG_TYPE 内置程序

```
PROGRAM get_reg_data
%NOLOCKGROUP
%ENVIRONMENT REGOPE
VAR
  entry: INTEGER
  group_no: INTEGER
  jpos: JOINTPOS
  maxpregnum: integer
  num_axes: INTEGER
  posn_type: INTEGER
  register_no: INTEGER
  status: INTEGER
  xyz: XYZWPR
  xyzext: XYZWPREXTBEGIN
  group_no = 1
  GET_VAR(entry, '*POSREG*' ,'$MAXPREGNUM', maxpregnum, status)
  -- 每个寄存器的循环
  FOR register_no = 1 to 10 DO
    -- 获取位置寄存器类型
    POS_REG_TYPE(register_no, group_no, posn_type, num_axes, status)
    -- 获取位置寄存器
    WRITE('PR[', register_no, '] of type ', posn_type, CR)
    SELECT posn_type OF
      CASE (2):
        xyz = GET_POS_REG(register_no, status)
      CASE (6):
        xyzext = GET_POS_REG(register_no, status)
      CASE (9):
        jpos = GET_JPOS_REG(register_no, status)
      ELSE:
    ENDSELECT
  ENDFOR
END get_reg_data
```

A.17.15 POSITION 数据类型

用途：将变量，函数返回类型或例程参数定义为 POSITION 数据类型

语法：POSITION <IN GROUP[n]>

细节：

- POSITION 由定义法线，方向，逼近和位置向量的矩阵以及指定配置字符串的组件组成，总共56个字节。
- 配置字符串表示联合放置和多个转弯，用于描述机器人在特定位置时的配置。
- 始终针对特定坐标系引用 POSITION。
- POSITION 数据类型可用于表示引用框架，在这种情况下，将忽略配置组件。
- 可以使用相对位置运算符（:)完成坐标框架转换。
- 可以将 POSITION 指定给其他位置类型。
- 有效的 POSITION 运算符是
  — 相对位置（:)运算符
  — 大致相等（ > = < ）运算符
- POSITION 后面可以有 IN GROUP [n]，其中n表示使用数据的运动组。默认值为 %DEFGROUP 指令指定的组，或1。
- 可以访问或设置 POSITION 变量的组件，就好像它们的定义如下：

POSITION 数据类型

```
POSITION = STRUCTURE
     NORMAL:  VECTOR          -- read-only
     ORIENT:  VECTOR          -- read-only
     APPROACH:  VECTOR        -- read-only
     LOCATION:  VECTOR        -- read-write
     CONFIG_DATA:  CONFIG     -- read-write
   ENDSTRUCTURE
```

另请参见：POS，UNPOS 内置函数

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.17.16 POST_ERR 内置程序

用途：将错误代码和原因代码发布到错误报告系统，以显示并保留错误的历史记录

语法：POST_ERR(error_code, parameter, cause_code, severity)

Input/Output 参数：

[in] error_code :INTEGER

[in] parameter :STRING

[in] cause_code :INTEGER

[in] severity :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *error_code* 是要发布的错误。
- 如果在字典文本中指定了 *%s*，参数将包含在 *error_code* 的消息中。 如果没有必要，则输入空字符串。
- *cause_code* 是错误的原因。 如果没有适用的原因，可以使用0。
- *error_code* 和 *cause_code* 采用以下格式：

  ff ccc (十进制)

  这里

  ff 代表错误的设施代码。
- ccc 表示指定设施内的错误代码。
- *severity* 定义如下：

  0：WARNING, 任务执行没有改变

  1：PAUSE, 执行所有任务并停止所有动作

  2：ABORT, 所有任务并取消

另请参阅：ERR_DATA 内置过程，相应的应用程序特定的FANUC机器人设置和操作手册，“错误代码”

示例：有关详细的程序示例，请参见第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）。

A.17.17 POST_ERR_L 内置程序

用途：将具有本地严重性的错误代码发布到错误报告系统，以显示和保留错误的历史记录

语法：POST_ERR_L(error_code, parameter, cause_code, severity)

Input/Output Parameters:

[in] error_code :INTEGER

[in] parameter :STRING

[in] cause_code :INTEGER

[in] severity :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *error_code* 是要发布的错误。
- 如果在字典文本中指定了 *%s*，则参数将包含在 *error_code* 的消息中。 如果没有必要，则输入空字符串。
- *cause_code* 是错误的原因。 如果没有适用的原因，可以使用0。
- *error_code* 和 *cause_code* 采用以下格式：

  `ffccc（十进制）`

  其中

  ff 表示错误的设施代码。

  ccc 表示指定工具内的错误代码。

- *severity* 定义如下：

  ERSEV_NONE：没有严重性。

  ERSEV_WARN：警告，任务执行没有变化。

  ERSEV_PAUSE：PAUSE全局，暂停所有任务并在当前运动段后停止所有运动

  ERSEV_PAUSEL：PAUSE本地，暂停本地任务并在当前运动段之后停止本地任务的所有运动。

  ERSEV_STOP：STOP全局，暂停所有任务并停止所有动作。

  ERSEV_STOPL：STOP Local，暂停本地任务并停止本地任务的所有动作。

  ERSEV_SERVO：SERVO Global，关闭所有伺服电源并暂停所有任务。

ERSEV_SERVOL：SERVO Local，关闭本地任务运动组的伺服电源并暂停本地任务。

ERSEV_ABORT：ABORT Global，中止所有任务并取消所有动作。

ERSEV_ABORTL：ABORT本地，中止本地任务并取消本地任务的所有动作。

ERSEV_SYSTEM：SYSTEM全局，存在系统问题并阻止任何进一步的操作。

另请参阅：POST_ERR内置

A.17.18 POST_SEMA 内置程序

用途：在指示的信号量的值中添加一个

语法：POST_SEMA(semaphore_no)

Input/Output 参数：

[in] semaphore_ no：INTEGER

%ENVIRONMENT Group：MULTI

细节：

- *semaphore_no* 指示的信号量增加 1。
- *semaphore_no* 必须在 1 到控制器上定义的信号量数量的范围内。

另请参阅：PEND_SEMA,CLEAR_SEMA内置程序,SEMA_COUNT内置函数,第15章多任务

示例：请参阅第15章“多任务”中的示例

A.17.19 PRINT_FILE 内置程序

用途：将ASCII文件的内容输出到默认设备

语法：PRINT_FILE(file_spec, nowait_sw, status)

Input/Output 参数：

[in] file_spec :STRING

[in] nowait_sw :BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group : FDEV

细节：

- *file_spec* 指定要输出的文件的设备，名称和类型。
- 如果 *nowait_sw* 为 TRUE，则在执行命令时继续执行程序。 如果为 FALSE，程序将停止，包括条件处理程序，直到操作完成。 如果程序中有时间紧急的条件处理程序，请将它们放在另一个作为单独任务执行的程序中。

  注意 *nowait_sw* 在此版本中不可用，应设置为 FALSE。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

A.17.20 %PRIORITY 译者指令

用途：指定任务优先级

语法：%PRIORITY = n

细节：

- *n* 是优先级，定义如下：

  1 到 89 : 比动作低，比用户界面高

  90 到 99 : 低于用户界面
- 值越低，任务优先级越高。
- 默认优先级为50.有关如何将指定的优先级转换为系统优先级的更多信息，请参见第15.4节“任务调度”。
- 可以在任务执行期间通过 SET_TSK_ATTR 内置例程设置优先级。

示例：通常，错误处理任务会出现错误，并且在发生错误时，它会尽快处理错误恢复。 在这种情况下，错误处理任务可能需要比其他任务具有更高的优先级，因此“*n*”应小于50。

%PRIORITY 译者指令

```
%PRIORITY = 49
```

A.17.21 PROG_BACKUP 内置程序

用途：将指定程序和所有被调用程序从执行存储器保存到存储设备。 如果被叫程序调用其他程序，它们将以递归方式保存。 您可以指定保存任何关联的程序变量。

语法：PROG_BACKUP (file_spec, prog_type, max_size, write_prot, status)

Input/Output 参数：

[in] file_spec :STRING

[in] prog_type :INTEGER

[in] max_size: INTEGER

[in] write_prot: BOOLEAN

[out] status :INTEGER

%ENVIRONMENT Group: CORE

细节：

- *file_spec* 指定要保存的设备和程序。 如果指定了文件类型，则会将其忽略。
- *prog_type* 指定要保存的程序类型。 有效类型是：
  PBR_VRTYPE：VR - 包含变量的程序
  PBR_MNTYPE：JB，PR，MR，TP
  PBR_JBTYPE：JB - 仅限工作计划
  PBR_PRTYPE：PR - 仅处理程序
  PBR_MRTYPE：MR - 仅限宏程序
  PBR_PCTYPE：VR - 保存VR文件而不是PC文件
  PBR_ALLTYPE：所有程序VR，JB，PR，MR，TP
  PBR_NVRTYPE：除VR外的所有程序
  PBR_NMRTYPE：JB，PR，TP（除宏外的所有TP）
- *max_size* 指定备份程序所需的磁盘空间的最大大小（以KB为单位）。 如果存储设备上没有足够的内存，则不会备份任何程序，状态将等于2002，“FILE-002设备已满”。

- 如果备份程序所需的磁盘空间超过 *max_size*，则备份将继续。如果没有足够的空间来保存所有程序，备份可能仍会失败。返回状态将等于2002，“FILE-002设备已满”。在这种情况下，将存在部分备份。要防止出现这种情况，请确保 *max_size* 足够大以防止出现此错误。
- *write_prot*，如果为true，则指定应保存写保护程序。如果为false，则指定不应保存写入受保护程序。
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。
- 系统将保持循环并处理尽可能多的程序，即使出现错误也是如此。例如，如果八个中缺少一个程序，则保存剩余的七个程序。在这种情况下，错误“程序不存在”将返回给处于状态的用户。在这种情况下，错误发布在每个程序的错误中的程序名称。原因代码是从保存例程返回的任何内容。
- 如果在存储设备上指定了子目录，则将创建该子目录（如果该目录尚不存在）。所有程序都将保存到子目录中。
- 如果文件已存在但未发生更改，则不会覆盖该文件。
- 如果文件已存在但程序已更改，则将覆盖该文件并且不会返回任何错误。
- 带有变量的同名KAREL或示教器程序必须作为被调用程序存在于内存中，否则系统将无法保存VR。
- PROG_BACKUP,PROG_CLEAR和PROG_RESTORE 内置函数考虑除宏之外的所有程序引用。这包括以下语句中引用的所有程序：CALL，RUN,ERROR_PROG,RESUME_PROG和MONITOR。

示例：以下示例将具有相应扩展名的ANS00003保存到FR：device上的GMX_211子目录。无论程序类型如何，它都将保存ANS00003递归调用的所有程序。 它不会保存KAREL变量。如果FR：设备上的可用空间少于200k，则会失败。

```
VAR
    status: INTEGER
BEGIN
    PROG_BACKUP('FR:\GMX_211\ANS00003', PBR_NVRTYPE, 200, TRUE, status)
```

示例：以下示例将具有相应文件扩展名的ANS00003保存到FR：device上的GMX_211子目录。 它将保存由ANS00003递归调用的JB，PR，MR或TP程序。 它不会保存写保护程序。如果FR：设备上的可用空间少于100k，则会失败。

```
VAR
    status: INTEGER
BEGIN
    PROG_BACKUP('FR:\GMX_211\ANS00003', PBR_MNTYPE, 100, FALSE, status)
```

示例：以下示例将MAIN保存到MC：具有相应文件扩展名的设备。 它将保存MAIN递归调用的所有程序和变量。 如果MC：设备上的可用空间少于300k，则会失败。

```
VAR
    status: INTEGER
BEGIN
    PROG_BACKUP('MC:\MAIN', PBR_ALLTYPE, 300, TRUE, status)
```

A.17.22 PROG_CLEAR 内置程序

用途：从执行存储器中清除指定的程序和所有被调用的程序。 如果被叫程序调用其他程序，它们将被递归清除。 您可以指定也清除任何关联的程序变量。 从其他程序引用的变量不会被清除。

语法： PROG_CLEAR (prog_name, prog_type, status)

Input/Output Parameters:

[in] prog_name :STRING

[in] prog_type :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group: CORE

细节：

- *prog_name* 指定要清除的根程序。
- *prog_type* 指定要清除的程序类型。有效类型是：
  PBR_VRTYPE：VR - 包含变量的程序
  PBR_MNTYPE：JB, PR, MR, TP
  PBR_JBTYPE：JB - 仅限job程序
  PBR_PRTYPE：PR - 仅处理程序
  PBR_MRTYPE：MR - 仅限宏程序
  PBR_PCTYPE：VR - 保存VR文件而不是PC文件
  PBR_ALLTYPE：所有程序 VR, JB, PR, MR, TP
  PBR_NVRTYPE：除VR外的所有程序
  PBR_NMRTYPE：JB, PR, TP（除宏外的所有TP）
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。
- 系统将保持循环并清除尽可能多的程序。如果缺少其中一个被调用的程序，则这不是错误。但是，如果该缺失的程序调用其他程序，那么将找不到其他程序，并且不会被清除。对于未清除的每个程序，错误将与错误中的程序名一起发布。原因代码是从清除例程返回的任何内容。

- 清除任何VR数据都可能被其他程序引用。 任何变量清除操作都将忽略此错误。
- 如果标识为清除的程序是所选程序，则不会清除它。 在这种情况下返回错误"程序正在使用"。作为对策，使用必须使用内置的SELECT_TPE（ ）来选择不在clear set中的程序。
- PROG_BACKUP,PROG_CLEAR 和 PROG_RESTORE 内置函数考虑除宏之外的所有程序引用。 这包括以下语句中引用的所有程序：CALL,RUN,ERROR_PROG,RESUME_PROG和MONITOR。

示例：以下示例从内存中清除ANS00003.TP。 它将清除ANS00003递归调用的所有程序，无论程序类型如何，并从内存中清除它们。 它不会清除写保护程序。 它不会清除任何KAREL变量。

```
VAR
    status: INTEGER
BEGIN
    PROG_CLEAR('ANS00003.TP', PBR_NVRTYPE, status)
```

示例：以下示例从内存中清除ANS00003.TP程序。 它将仅清除由ANS00003递归调用的JB，PR，MR和TP程序。 它不会清除写保护程序。

```
VAR
    status: INTEGER
BEGIN
        PROG_CLEAR('ANS00003.TP', PBR_MNTYPE, status)
```

A.17.23 PROG_RESTORE 内置程序

用途：将指定程序和所有被调用程序恢复（加载）到执行内存中。 如果被调用的程序调用其他程序，它们将被递归加载。 如果VR文件存在，也将加载任何关联的程序变量。

语法： PROG_RESTORE (file_spec, status)

Input/Output 参数：

[in] file_spec :STRING

[out] status :INTEGER

%ENVIRONMENT Group: CORE

细节：

- *file_spec* 指定要还原的存储设备和程序。如果指定了文件类型，则会将其忽略。
- *status* 说明尝试操作的状态。如果不等于0，则发生错误。
- 系统将保持循环并处理尽可能多的程序，即使出现错误也是如此。例如，如果八个中缺少一个程序，则加载剩余的七个程序。在这种情况下，错误“文件不存在”将返回给处于状态的用户。在这种情况下，错误发布在每个程序的错误中的程序名称。原因代码是从加载例程返回的任何内容。
- 如果指定了子目录，则从该子目录加载被调用的程序。该子目录中的任何额外文件都不会自动加载。
- 如果程序已存在，则不会还原，也不会返回错误。
- 同名的 KAREL 或TP程序必须已作为被调用程序存在于内存中，否则系统将不会加载VR。
- 即使变量已存在，也将恢复VR类型。也就是说，系统将使用VR文件中保存的值覆盖任何现有变量值。
- 无法恢复ASCII程序（.LS）。
- 如果没有足够的可用内存，则会返回错误并且还原不完整。
- PROG_BACKUP,PROG_CLEAR和PROG_RESTORE内置函数考虑除宏之外的所有程序引用。这包括以下语句中引用的所有程序：CALL,RUN,ERROR_PROG,RESUME_PROG和MONITOR。

示例：以下示例从FR：device上的GMX_211子目录恢复ANS00003.TP。 无论程序类型如何，它都将恢复ANS00003递归调用的所有程序。 如果VR文件位于还原目录中，它将还原VR文件。

```
VAR
    status: INTEGER
BEGIN
    PROG_RESTORE('FR:\GMX_211\ANS00003.TP', status)
```

示例：以下示例从FR：device上的GMX_211子目录恢复ANS00003.TP。 它将仅恢复由ANS00003递归调用的TP程序。

```
VAR
    status: INTEGER
BEGIN
    PROG_RESTORE('FR:\GMX_211\ANS00003', status)
```

示例：以下示例通过查找其文件类型从MC：device恢复MAIN。 它将恢复MAIN递归调用的所有程序和变量。

```
  VAR
    status: INTEGER

BEGIN
    PROG_RESTORE('MC:\MAIN', status)
```

A.17.24 PROG_LIST 内置程序

用途：返回程序名称列表。

语法：prog_list(prog_name, prog_type, n_skip, format, ary_name, n_progs <,f_index>)

Input/Output 参数：

[in] prog_name :STRING

[in] prog_type :INTEGER

[in] n_skip :INTEGER

[in] format :INTEGER

[out] ary_name :ARRAY of string

[out] n_progs :INTEGER

[out] status :INTEGER

[in,out] f_index :INTEGER

%ENVIRONMENT Group :BYNAM

细节：

- *prog_name* 指定要在 *ary_name* 中返回的程序的名称。 *prog_name* 可以使用通配符（*）字符，以指示应在 *ary_name* 中返回与 *prog_type* 匹配的所有程序。
- *prog_type* 指定要检索的程序类型。 有效类型是：

  1：VR - 仅包含变量的程序

  2：JB, PR, MR, TP3 :JB - 仅限job程序

  4：PR - 仅处理程序

  5：MR - 仅限宏程序

  6：仅限PC - KAREL程序

  7：所有程序 VR, JB, PR, MR, TP, PC

  8：除VR外的所有程序

- *n_skip* 此参数仅在程序名称使用*而程序类型使用7时使用。当存在比声明的 *ary_name* 长度更多的程序时，否则使用 *f_index* 参数进行多次调用。首次使用 PROG_LIST 时将 *n_skip* 设置为0。如果 *ary_name* 完全填充了程序名，请将该数组复制到另一个 STRINGS ARRAY 并再次执行 PROG_LIST，其中 *n_skip* 等于 *n_skip* + *n_progs*。然后调用 PROG_LIST 将跳过之前传递中找到的程序，并仅查找剩余的程序。
- *format* 指定程序名称的格式。以下值对格式有效：
  1：程序名称，没有空格
  2：'程序名称程序类型'
  总长度= 15个字符
- *prog_name* = 12个字符后跟空格
- *prog_type* = 2个字符
- *ary_name* 是 STRING 的 ARRAY，用于存储程序名称。
- *n_progs* 是存储在 *ary_name* 中的变量数。
- 如果成功，状态将返回零。
- *f_index* 是快速索引的可选参数。如果将 *prog_name* 指定为复杂通配符（除直*之外的任何其他内容），则应使用此参数。对 PROG_LIST 的第一次调用将 *f_index* 和 *n_skip* 都设置为零。然后将在内部使用 *f_index* 快速查找下一个 *prog_name*。一旦特定 *prog_name* 的列表开始，请勿更改 *f_index*。

另请参见：VAR_LIST内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.17.25 PROGRAM 声明

用途：标识KAREL源程序中的程序名称。

语法：PROGRAM prog_name

其中：

prog_name：有效的KAREL标识符

细节：

- 它必须是程序中的第一个语句（除了注释）。
- 用于命名程序的标识符不能在程序中用于任何其他目的，例如标识变量或常量。
- *prog_name* 还必须出现在END语句中，该语句标记程序的可执行部分的结尾。
- 程序名称可用于从程序中调用程序作为过程例程，其方式与例程名称用于调用过程例程的方式相同。

示例：有关如何使用PROGRAM语句的更多详细示例，请参阅附录B“KAREL示例程序”。

A.17.26 PULSE 动作

用途：将数字输出端口脉冲指定的毫秒数

语法：PULSE DOUT[port_no] FOR time_in_ms

其中：

port_no：INTEGER 变量或文字

time_in_ms：一个 INTEGER

细节：

- *port_no* 必须是有效的数字输出端口号。
- *time_in_ms* 指定脉冲的持续时间（以毫秒为单位）。
- 如果 *time_in_ms* 持续时间为零，则不会发生脉冲。 否则，周期向上舍入到下一个8毫秒的倍数。
- 脉冲始终在脉冲开始时打开端口，并在脉冲结束时关闭端口。
- 如果端口''常开，''负脉冲可以通过将端口设置为反极性或通过执行以下顺序来完成：

  DOUT [n] = FALSE

  DELAY x

  DOUT [n] = TRUE
- POWSE 操作中不允许NOWAIT。
- 如果在脉冲进行过程中暂停程序，脉冲将在正确的时间结束。

- 如果程序在脉冲进行过程中被中止，则端口将保持中止发生时的状态。
- 如果 *time_in_ms* 为负数或大于 86,400,000（24小时），程序将中止并显示错误。

另请参阅：第6章 条件处理程序，第7章 文件输入/输出操作

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.17.27 PULSE 声明

用途：将数字输出端口脉冲指定的毫秒数。

语法：PULSE DOUT[port_no] FOR time_in_ms < NOWAIT >

其中：

port_no：INTEGER 变量或文字

time_in_ms：一个 INTEGER

细节：

- *port_no* 必须是有效的数字输出端口号。
- *time_in_ms* 指定脉冲的持续时间（以毫秒为单位）。
- 如果 *time_in_ms* 持续时间为零，则不会发生脉冲。 否则，周期向上舍入到下一个8毫秒的倍数。

  脉冲的实际持续时间比舍入值小0到8毫秒。

  例如，如果指定了100，则将其向上舍入为104（8的下一个倍数）毫秒。 实际持续时间为96到104毫秒。
- 脉冲始终在脉冲开始时打开端口，并在脉冲结束时关闭端口。
- 如果端口 “常开”，则可以通过将端口设置为反向极性或执行以下顺序来实现负脉冲:

```
DOUT[n] = FALSE
DELAY x
DOUT[n] = TRUE
```

- 如果在 PULSE 语句中指定了 NOWAIT，则下一个KAREL语句将与脉冲同时执行。
- 如果未在 PULSE 语句中指定 NOWAIT，则在脉冲完成之前不会执行下一个KAREL语句。

另请参阅：附录E，''语法图''以获取更多语法信息

示例：在以下示例中，数字输出是脉冲式的，然后是第二个数字输出的脉冲。 因为指定了NOWAIT，所以DOUT [start_air]将在DOUT [5]完成之前执行。

PULSE 声明

```
PULSE DOUT[5] FOR (seconds * 1000) NOWAIT
PULSE DOUT[start_air] FOR 50 NOWAIT
```

A.17.28 PURGE CONDITION 声明

用途：从系统中删除条件处理程序的定义

语法：PURGE CONDITION[cond_hand_no]

其中：

cond_hand_no：INTEGER 表达式

细节：

- 如果没有使用指定的数字定义条件处理程序，则该语句无效。
- PURGE CONDITION 语句仅用于清除全局条件处理程序。
- PURGE CONDITION 语句将清除启用的条件。
- 如果先前已定义具有指定编号的条件处理程序，则必须先将其清除，然后再将其替换为新的条件处理程序。

另请参阅：ENABLE CONDITION语句第6章条件处理程序，附录E，''语法图''了解更多语法信息

示例：在以下示例中，如果BOOLEAN变量ignore_cond为TRUE，则将使用PURGE语句清除全局条件处理程序CONDITION [1]; 否则启用CONDITION [1]。

PURGE CONDITION 声明

```
IF ignore_cond THEN
  PURGE CONDITION[1]
```

```
ELSE
  ENABLE CONDITION[1]
ENDIF
```

A.17.29 PURGE_DEV 内置程序

用途：通过释放任何不再需要的已用块来清除指定的内存文件设备

语法：PURGE_DEV (device, status)

Input/Output 参数：

[in] device：STRING

[out] status：INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *device* 指定要清除的内存文件设备。对于FROM磁盘，设备应设置为“FR：”，对于RAM磁盘，设置应设置为“RD：”，对于两个磁盘，设置应设置为“MF：”。
- 仅当设备没有足够的内存来执行操作时，才需要执行清除操作。如果FROM磁盘已满，'FR：'设备将返回85001。如果RAM磁盘已满，'RD：'设备将返回85020。
- 清除操作将擦除以前使用但不再需要的文件块。这些被称为垃圾块。如果删除或覆盖文件，FROM磁盘可能包含许多垃圾块。 RAM磁盘通常不包含垃圾块，但在文件复制期间断电时可能会发生这种情况。
- VOL_SPACE内置可用于确定FROM磁盘上的垃圾块数。硬件限制可能会减少实际释放的块数。
- 必须安装设备，并且设备上不能打开任何文件，否则将返回错误。
- *status* 说明尝试操作的状态。如果不等于0则发生错误。如果没有发生错误，则返回85023，但没有清除任何块。

示例：返回到第B.9节“使用文件和设备内置命令”（FILE_EX.KL），以获取更详细的程序示例。

A.17.30 PUSH_KEY_RD 内置程序

用途：暂停键盘设备的键输入

语法：PUSH_KEY_RD(key_dev_name, key_mask, pop_index, status)

Input/Output 参数：

[in] key_dev_name :STRING

[in] key_mask :INTEGER

[out] pop_index :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 挂起指定键盘设备上的所有读取请求，该请求使用任何指定的键类（作为 *accept_mask* 或 *term_mask*）。
- 如果没有活动的读取请求，则将一组空输入记录为已暂停。 这不是错误。
- *key_dev_name* 必须是已定义的键盘设备之一：

  'TPKB'：教导吊坠键盘设备

  'CRKB'：CRT键盘设备
- *key_mask* 是一个按位掩码，表示将被挂起的字符类。 这应该是包含文件 klevkmsk.kl 中定义的常量的OR。

  kc_display：可显示的键

  kc_func_key：功能键

  kc_keypad：键盘和编辑键

  kc_enter_key：输入和返回键

  kc_delete：删除和退格键

  kc_lr_arw：左右箭头键

  kc_ud_arw：向上和向下箭头键

  kc_other：其他键（如Prev）

- 返回 *pop_id*，应在调用 POP_KEY_RD 时使用，以重新激活读取请求。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：POP_KEY_RD内置过程

示例：有关示例，请参阅READ_KB内置过程。

A.18 - Q - KAREL 语言描述

A.18.1 QUEUE_TYPE 数据类型

用途：定义在QUEUE内置例程中使用的数据类型

语法： queue_type = STRUCTURE

n_entries : INTEGER

sequence_no : INTEGER

head : INTEGER

tail : INTEGER

ENDSTRUCTURE

细节：

- *queue_type* 用于初始化和维护QUEUE内置例程的队列数据。 不要更改此数据; 它在内部使用。

另请参阅：APPEND_QUEUE, DELETE_QUEUE, INSERT_QUEUE, COPY_QUEUE, GET_QUEUE, INIT_QUEUE, MODIFY_QUEUE 内置过程

A.19 - R - KAREL 语言描述

A.19.1 READ 声明

用途：从串行I / O设备或文件读取数据

语法： READ < file_var > (data_item {,data_item})

其中：

file_var : 一个 FILE 变量

data_item : 一个变量标识符及其可选的格式说明符或保留字CR

细节：

- 如果在 READ语句 中未指定 *file_var*，则使用默认的 TPDISPLAY。 %CRTDEVICE 指令会将默认值更改为 INPUT。
- 如果指定了 *file_var*，它必须是输入设备之一（INPUT,CRTPROMPT,TPDISPLAY,TPPROMPT）或在 OPEN FILE 语句中设置的变量。
- 如果使用 UF选项 设置了 file_var 属性，则会以二进制形式将数据传输到指定的变量中。否则，数据以ASCII文本形式传输。
- *data_item* 可以是具有RW访问或用户定义变量的系统变量。
- 当执行READ语句时，数据读取从下一个非空白输入字符开始，到所有输入类型（字符串除外）的下一个空白，行尾或文件结束之前的最后一个字符结束。
- 如果 *data_item* 是 ARRAY类型，则必须提供下标。
- 如果 *data_item* 的类型为 PATH，则可以指定读取整个路径，读取特定节点（[n]）或读取一定范围的节点（[n .. m]）。
- 可选的格式说明符可用于控制为每个 *data_item* 读取的数据量。格式说明符的作用取决于正在读取的项目的数据类型以及数据是以文本（ASCII）还是二进制（未格式化）形式。
- 可以用作数据项的保留字CR指定当前输入行中的任何剩余数据将被忽略。下一个数据项将从下一个输入行的开始读取。
- 如果在输入过程中发生文件读取和任何错误，则将读取的变量以及数据列表中最多达CR的所有后续变量设置为未初始化。
- 如果 *file_var* 是一个窗口设备，并且在输入过程中出现任何错误，则会显示一条错误消息，指示错误的数据项，并提示您输入无效数据项的替换项并重新输入所有后续项。
- 使用 IO_STATUS（*file_var*）来确定读取操作是否成功。
- 注意阅读CR不应该用于无格式模式。

另请参见：第7章 文件输入/输出操作有关READ格式说明符的更多信息，IO_STATUS 内置函数附录E “语法图”，以获取更多语法信息

示例：请参阅以下各节以获取详细的程序示例：

第B.10节 “使用动态显示内置插件”（DYN_DISP.KL）

第B.12节 “从字典文件中显示列表”（DCLST_EX.KL）

第B.13节 “使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.19.2 READ_DICT Built-In Procedure

用途：从字典中读取信息

语法：READ_DICT(dict_name, element_no, ksta, first_line, last_line, status)

Input/Output 参数：

[in] dict_name：STRING

[in] element_no：INTEGER

[out] ksta：ARRAY OF STRING

[in] first_line：INTEGER

[out] last_line：INTEGER

[out] status：INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *dict_name* 指定要从中读取的字典的名称。
- *element_no* 指定要读取的元素编号。 此元素编号在字典文件中用$指定。
- *ksta* 是一个 KAREL STRING ARRAY，用于存储从字典文本文件中读取的信息。
- 如果 *ksta* 太小而无法存储所有数据，则数据将被截断，状态将设置为33008，“字典元素已截断”。
- *first_line* 表示 *ksta* 的数组元素，在该元素处开始存储信息。
- *last_line* 返回一个值，指示 *ksta* 数组中使用的最后一个元素。
- *status* 说明尝试操作的状态。 如果不等于0，则从字典文件中读取元素时发生错误。
- &new_line是唯一可以使用READ_DICT从字典文本文件中读取的保留属性代码。READ_DICT内置忽略所有其他保留属性代码。

另请参见：ADD_DICT，WRITE_DICT，REMOVE_DICT内置过程。 有关DISCTRL_LIST内置过程的信息，请参阅程序示例。 第10章 字典和表单

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.19.3 READ_DICT_V 内置程序

用途：从具有格式化变量的字典中读取信息

语法：READ_DICT_V(dict_name, element_no, value_array, ksta, status)

Input/Output 参数：

[in] dict_name：STRING

[in] element_no：INTEGER

[in] value_array：ARRAY OF STRING

[out] ksta：ARRAY OF STRING

[out] status：INTEGER

%ENVIRONMENT Group :UIF

细节：

- *dict_name* 指定要从中读取的字典的名称。
- *element_no* 指定要读取的元素编号。这个数字在字典文件中用$指定。
- *value_array* 是一个变量名数组，对应于字典文本中的每个格式化数据项。每个变量名称都可以指定为“[prog_name] var_name”。

  — [prog_name]指定包含指定变量的程序的名称。如果未指定，则使用当前正在执行的程序。

  — *var_name* 必须引用静态变量。

  — *var_name* 可以包含节点号，字段名和/或下标。
- *ksta* 是一个KAREL STRING ARRAY，用于存储从字典文本文件中读取的信息。
- 如果 *ksta* 太小而无法存储所有数据，则数据将被截断，状态将设置为33008，“字典元素已截断”。
- *status* 说明尝试操作的状态。如果不等于0，则从字典文件中读取元素时发生错误。
- &new_line是唯一可以使用 READ_DICT_V 从字典文本文件中读取的保留属性代码。READ_DICT_V 内置忽略所有其他保留属性代码。

另请参阅：WRITE_DICT_V内置程序，第10章 字典和表单

示例：在以下示例中，TPTASKEG.TX包含将显示系统变量的字典文本信息。 此信息是字典中的第一个元素。 元素编号从0开始.util_prog使用READ_DICT_V读取文本并将其显示在示教器上。

READ_DICT_V 内置程序

```
-------------------------------------------
TPTASKEG.TX
-------------------------------------------
$ "Maximum number of tasks = %d"
-------------------------------------------
UTILITY PROGRAM:
-------------------------------------------
PROGRAM util_prog
  %ENVIRONMENT uif
  VAR
    ksta: ARRAY[1] OF STRING[40]
    status: INTEGER
    value_array: ARRAY[1] OF STRING[30]
  BEGIN
    value_array[1] = '[*system*].$scr.$maxnumtask'
    ADD_DICT('TPTASKEG', 'TASK', dp_default, dp_open, status)
    READ_DICT_V('TASK', 0, value_array, ksta, status)
    WRITE(ksta[i], cr)
END util_prog
```

A.19.4 READ_KB 内置程序

用途：从键盘设备读取并等待完成

语法：READ_KB(file_var, buffer, buffer_size, accept_mask, term_mask, time_out, init_data, n_chars_got, term_char, status)

Input/Output 参数：

[in] file_var：FILE

[out] buffer：STRING

[in] buffer_size：INTEGER

[in] accept_mask：INTEGER

[in] time_out：INTEGER

[in] term_mask : INTEGER

[in] init_data : STRING

[out] n_chars_got : INTEGER

[out] term _char : INTEGER

[out] stat us : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 使指定类别字符的数据存储在用户提供的缓冲区中，直到满足终止条件或缓冲区已满。读取终止时返回调用方。
- 如果对CRT / KB使用READ_KB，则会返回“原始”CRT字符。 要获得示教器等效键代码，您必须执行以下功能：

  ```
  tp_key = $CRT_KEY_TBL[crt_key + 1]
  ```

  此映射允许您在CRT / KB和示教器设备之间使用通用软件。
- READ_KB 和其他一些实用程序在KAREL程序中使用名为 *device_stat* 的变量来建立KAREL程序和用户界面显示之间的关联。例如，如果您有一个调用READ_KB的任务[MAINUIF]，则用于建立关联的变量是[MAINUIF] *device_stat*。如果未设置 *device_stat*，则只能在单屏模式或左窗格中读取字符。
- *device_stat* 必须设置为运行应用程序的 pane ID。对于标准单模/单色挂件，*device_stat* = 1。要在右侧窗格中进行交互，请设置 *device_stat* = 2。要在右下方窗格中进行交互，请设置 *device_stat* = 3。外部 Internet Explorer 连接使用窗格4-9。对于CRT / KB，设置 *device_stat* = 255。
- 在打开与READ_KB相关联的键盘文件之前，必须将[MAINUIF] *device_stat* 设置为正确的窗格ID。 iPendant的窗格ID可以是1,2或3。
- 在窗格ID 1中运行的应用程序存储在$ TP_CURSCRN中.Pane ID 2存储在$ UI_CURSCRN [1]中，或者通常为$ UI_CURSCRN [device_stat-1]。 CRT应用程序使用$ CT_CURSCRN。
- *file_var* 必须对键盘设备开放。如果 *file_var* 也与窗口关联，则字符将回显到窗口。
- 字符存储在 *buffer* 中，最大为 *buffer_size* 或字符串的大小，以较小者为准。
- *accept_mask* 是一个按位掩码，表示将被接受为输入的字符类。这应该是包含文件klevkmsk.kl中定义的常量的OR。

kc_display：可显示的键

kc_func_key：功能键

kc_keypad：键盘和编辑键

kc_enter_key：输入和返回键

kc_delete：删除和退格键

kc_lr_arw：左右箭头键

kc_ud_arw：向上和向下箭头键

kc_other：其他键（如Prev）

- *accept_mask* 为零是合理的; 这意味着不接受任何字符作为输入。 等待将作为 *term_char* 返回的单个键时使用此方法。 在这种情况下，*buffer_size* 将为零。
- 如果 *accept_mask* 包含可显示的字符，则以下字符（如果接受）具有以下含义：

  — 删除字符 - 如果光标不在字段的第一个位置，则删除光标左侧的字符。

  — 左箭头和右箭头 - 假设光标尚未处于第一个或最后一个位置，将光标从其当前位置向左或向右放置一个字符。

  — 向上和向下箭头 - 将先前输入的输入读取到同一文件中。
- *term_mask* 是一个按位掩码，表示将终止请求的条件。 这应该是包含文件 klevkmsk.kl 中定义的常量的 OR。

  kc_display：可显示的键

  kc_func_key：功能键

  kc_keypad：键盘和编辑键

  kc_enter_key：输入和返回键

  kc_delete：删除和退格键

  kc_lr_arw：左右箭头键

  kc_ud_arw：向上和向下箭头键

  kc_other：其他键（如Prev）
- *time_out* 指定自动取消输入操作的时间（以毫秒为单位）。 值 -1 表示没有超时。

- *init_data_p* 指向一个字符串，该字符串显示为输入字段的初始值。 这不能超过 *buffer_size*。
- *n_chars_got* 设置为读取终止时输入缓冲区中的字符数。
- *term_char* 接收一个代码，指示终止表单的字符或其他条件。 密钥终止条件的代码在包含文件klevkeys.kl中定义。 通常返回的键是预定义的常量，如下所示：

  ky_up_arw

  ky_dn_arw

  ky_rt_arw

  ky_lf_arw

  ky_enter

  ky_prev

  ky_f1

  ky_f2

  ky_f3

  ky_f4

  ky_f5

  ky_next
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

示例：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

以下示例暂停任何示教器读取，使用READ_KB读取单个键，然后恢复任何挂起的读取。

READ_KB 内置程序

```
PROGRAM readkb
%NOLOCKGROUP
%ENVIRONMENT flbt
%ENVIRONMENT uif
%INCLUDE FR:eklevkmsk
VAR
  file_var: FILE
  key: INTEGER
```

```
  n_chars_got: INTEGER
  pop_index: INTEGER
  status: INTEGER
  str: STRING[1]

BEGIN
  -- 暂停任何未完成的TP键盘读取
  PUSH_KEY_RD('TPKB', 255, pop_index, status)
  IF (status = 0) THEN
    WRITE (CR, 'pop_index is ', pop_index)
  ELSE
    WRITE (CR, 'PUSH_KEY_RD status is ', status)
  ENDIF
  -- 使用PASALL和FIELD属性以及NOECHO打开TP Keyboard的文件
  SET_FILE_ATR(file_var, ATR_PASSALL)
  SET_FILE_ATR(file_var, ATR_FIELD)
  OPEN FILE file_var ('RW', 'KB:TPKB')
  -- 从TP键盘读取一个键
  READ_KB(file_var, str, 1, 0, kc_display+kc_func_key+kc_keypad+
          kc_enter_key+kc_lr_arw+kc_ud_arw+kc_other, 0, '',
  n_chars_got, key, status)
  IF (status = 0) THEN
    WRITE (CR, 'key is ', key, ', n_chars_got = ', n_chars_got)
  ELSE
    WRITE (CR, 'READ_KB status is ', status)
  ENDIF
  CLOSE FILE file_var
  -- 恢复任何未完成的TP键盘读取
  POP_KEY_RD('TPKB', pop_index, status)
  IF (status <> 0) THEN

    WRITE (CR, 'POP_KEY_RD status is ', status)
  ENDIF
END readkb
```

A.19.5 REAL 数据类型

用途：将变量，函数返回类型或 Routine 参数定义为 REAL 数据类型，其数值包含小数点和小数部分，或以科学计数法表示的数字

语法：REAL

细节：

- REAL变量和表达式的值可以在-3.4028236E+38到-30范围内 -1.175494E-38,0.0，和+1.175494E-38至+3.4028236E+38，具有大约七位小数的重要性。 否则，程序将因“实际溢出”错误而中止。
- 定义REAL常量或文字时，小数点是必需的（使用科学记数法时除外）。 定义REAL变量时，小数点不是必需的，只要它被声明为REAL即可。
- 科学记数法允许并受以下规则管辖：
  — 小数点向左移动，因此 INTEGER 部分中只剩下一位数字。
  — 小数部分后跟字母 E（大写或小写）和 ±INTEGER。

  此部分指定REAL数的大小。 例如，123.5表示为1.235E2。

  — 可以省略小数部分和小数点。 例如，100.0可以表示为1.000E2，即1.E2或1E2。
- 幅度在-1.175494E-38和+ 1.175494E-38之间的所有REAL变量都被视为0.0。
- 只能将REAL或INTEGER表达式赋值给REAL变量，从REAL函数例程返回，或作为参数传递给REAL参数。
- 如果在任何这些实例中使用INTEGER表达式，则将其视为REAL值。 如果INTEGER变量用作REAL参数的参数，则它始终按值传递，而不是通过引用传递。
- 有效的REAL运算符（参见表A-18）：
  — 算术运算符（+，+，*，/）
  — 关系运算符（>，> =，=，<>，<，<=）

表 A–18. 有效和无效的REAL运算符

有效	无效	原因
1.5	15	小数点是必需的（15是INTEGER而不是REAL）
1.	.	必须包括INTEGER或小数部分
+2500.450	+2,500.450	逗号不允许
1.25E-4	1.25E -4	空格不允许

示例：有关详细的程序示例，请参阅以下部分：

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.8节“沿着六边形路径生成和移动”（GEN_HEX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.19.6 Relational 条件

用途：用于测试两个操作数之间的关系

语法：variable <[subscript]> rel_op expression

其中：

variable：静态INTEGER或REAL变量或BOOLEAN端口数组元素

subscript：INTEGER表达式（仅用于端口数组）

rel_op：关系运算符

expression：静态变量，常量或EVAL子句

细节：

- 关系条件是状态条件，意味着在每次扫描期间都会测试关系。
- 可以使用以下关系运算符：

  =：相等

  <>：不相等

  <：不到

  = <：小于或等于

  >：大于

  > =：大于或等于

- 两个操作数必须具有相同的数据类型，并且只能是 INTEGER，REAL 或 BOOLEAN 类型。 INTEGER 值可用于需要 REAL 值的位置，并将被视为 REAL 值。
- *variable* 可以是任何端口数组信号，用户定义的静态变量或可由 KAREL 程序读取的系统变量。
- *expression* 可以是用户定义的静态变量，可以由 KAREL 程序读取的系统变量，任何常量或 EVAL 子句。
- 必须在启用条件处理程序之前初始化关系条件中使用的变量。

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口” （ DOUT_EX.KL ）。

A.19.7 RELAX HAND 声明

用途：关闭由一个信号控制的工具的打开信号，或关闭由一对信号控制的工具的打开和关闭信号。

语法：RELAX HAND hand_num

其中 :

hand_num：INTEGER表达式

细节 :

- 声明的实际效果取决于如何设置 HAND 信号。 请参阅第13章"输入/输出系统"。
- *hand_num* 必须是1-2范围内的值。 否则，程序将中止并显示错误。
- 如果 *hand_num* 的值在范围内但未连接手，则该语句无效。
- 如果 *hand_num* 的值在范围内，但尚未分配由该值表示的 HAND 信号，则程序将中止并显示错误。

另请参阅：第13章"输入/输出系统","附录D","语法图",了解更多语法信息

示例：在以下示例中，使用 RELAX HAND 语句放松由夹具指定的机器人手。 然后机器人在关闭手之前移动到 POSITION pstart。

RELAX HAND 声明

```
PROGRAM p_release
%NOPAUSE=TPENABLE
%ENVIRONMENT uif
BEGIN
 RELAX HAND gripper
 SET_POS_REG(1, pstart, status) —— 在PR[1]中放置位置
 move_to_pr1 —— 调用TP程序转移到PR [1]
 CLOSE HAND gripper
END p_release
```

A.19.8 RELEASE 声明

用途：从KAREL程序中释放机器人手臂和辅助或扩展轴的所有运动控制，以便在KAREL程序运行时由示教器控制它们

语法： RELEASE

细节：

- 在执行RELEASE语句之前停止的运动只能在执行下一个ATTACH语句后恢复。
- 如果在处于释放状态时从程序启动运动，程序将中止，并显示以下错误：''MCTRL因为已释放而被拒绝。''
- 如果在运动正在进行或处于HOLD状态时执行RELEASE，程序将中止，并显示以下错误：“分离请求失败”。
- 所有KAREL任务的所有运动控制都将被释放。

另请参阅：附录E,''语法图','了解更多语法信息

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.19.9 REMOVE_DICT 内置程序

用途：从指定的语言或所有现有语言中删除指定的字典

语法： REMOVE_DICT(dict_name, lang_name, status)

Input/Output 参数：

[in] dict_name：STRING

[in] lang_name：STRING

[out] status：INTEGER

%ENVIRONMENT Group :UIF

细节：

- *dict_name* 指定要删除的字典的名称。
- *lang_name* 指定应从中删除字典的语言。 应使用以下预定义常量之一：

dp_default

dp_english

dp_japanese

dp_french

dp_german

dp_spanish

如果 *lang_name* 为''，则将从其存在的所有语言中删除它。

- *status* 说明了尝试操作的状态。 如果不等于0，则删除字典文件时发生错误。

另请参阅：ADD_DICT内置过程，第10章 字典和表单

示例：有关详细的程序示例，请参见第B.12节 “从字典文件显示列表”（DCLST_EX.KL）。

A.19.10 RENAME_FILE 内置程序

用途：重命名指定的文件名

语法：RENAME_FILE(old_file, new_file, nowait_sw, status)

Input/Output 参数：

[in] old_file：STRING

[in] new_file：STRING

[in] nowait_sw：BOOLEAN

[out] status：INTEGER

%ENVIRONMENT Group :FDEV

细节：

- *old_file* 指定要重命名的文件的设备，名称和类型。
- *new_file* 指定要重命名为的文件的名称和类型。
- 如果 *nowait_sw* 为 TRUE，则在执行命令时继续执行程序。 如果为 FALSE，程序将停止，包括条件处理程序，直到操作完成。

如果程序中有时间紧急的条件处理程序，请将它们放在另一个作为单独任务执行的程序中。

注意 *nowait_sw* 在此版本中不可用，应设置为FALSE。

- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：COPY_FILE，DELETE_FILE 内置过程

A.19.11 RENAME_VAR 内置程序

用途：将指定程序中的指定变量重命名为新变量名称

语法：RENAME_VAR(prog_nam, old_nam, new_nam, status)

Input/Output 参数：

[in] prog_nam：STRING

[in] old_nam：STRING

[in] new_nam：STRING

[out] status：INTEGER

%ENVIRONMENT Group :MEMO

细节：

- *prog_nam* 是包含要重命名的变量的程序的名称。
- *old_nam* 是变量的当前名称。
- *new_nam* 是变量的新名称。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参见：CREATE_VAR，SET_VAR内置过程

A.19.12 RENAME_VARS 内置程序

用途：将指定程序中的所有变量重命名为新程序名称

语法：RENAME_VARS(old_nam, new_nam, status)

Input/Output 参数：

[in] old_nam : STRING

[in] new_nam : STRING

[out] status : INTEGER

%ENVIRONMENT Group :MEMO

细节：

- *old_nam* 是程序的当前名称。
- *new_nam* 是程序的新名称。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：CREATE_VAR，RENAME_VARS 内置过程

A.19.13 REPEAT ... UNTIL 声明

用途：重复语句直到 BOOLEAN 表达式求值为 TRUE

语法：REPEAT

{ statement }

UNTIL boolean_exp

其中：

statement：有效的 KAREL 可执行语句

boolean_exp：一个 BOOLEAN 表达式

细节：

- 在执行 REPEAT 循环体中的语句之后评估 *boolean_exp*，以确定是否应该再次执行语句。
- 继续执行 *statement* 并计算 *boolean_exp*，直到它等于T RUE。
- *statement* 将始终至少执行一次。



警告

确保您的 REPEAT 语句包含一个由某些条件修改的 BOOLEAN 标志，以及一个终止循环的 UNTIL 语句。 如果没有，你的程序可以无限循环。

另请参阅：附录E，''语法图'，'了解更多语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.9节“使用文件和设备内置函数”（FILE_EX.KL）

第B.11节“操纵动态显示变量的值”（CHG_DATA.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.19.14 RESET 内置程序

用途：重置控制器

语法：RESET(successful)

Input/Output 参数：

[out] successful：BOOLEAN

%ENVIRONMENT Group :MOTN

细节：

- 即使存在阻止重置控制器的条件， *success* 也将为TRUE。
- 要确定复位操作是否成功，请延迟1秒并检查 OPOUT [3]（故障LED）。 如果为FALSE，则重置操作成功。
- 重置失败或完成后，RESET内置后的语句不会执行。 CRT或示教器上的状态显示将在复位期间显示PAUSED。
- 当复位正在进行时，控制器会显示当前状态，但在此期间，不会触发PAUSE条件处理程序。

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.19.15 RESUME 动作

用途：重新启动任务发出的最后一个停止动作

语法：RESUME <GROUP[n{,n}]>

细节：

- 动作集是在发出 STOP 语句或动作时发出但尚未终止的一组动作。
- 如果没有停止的动作集，则 RESUME 不会产生任何动作。
- 如果已停止多个动作集，则 RESUME 将重新启动最近停止的未恢复动作集。 随后的 RESUME 将以后进先出顺序启动其他 RESUME。
- 包含动态动作的动作按照最初发布的顺序恢复。
- 如果在发出 RESUME 动作时正在进行动作，则在当前动作完成后发生任何恢复的动作。
- 如果 group 子句不存在，则将恢复任务所控制的所有组（定义条件时）。
- 如果停止，恢复，取消或保持的动作是与其他组同时或协调动作的一部分，则停止，恢复，取消或保持所有组的动作。
- 无法为其他任务恢复运动。

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.19.16 RESUME 声明

用途：重新启动任务发出的最后一个停止动作

语法：RESUME <GROUP[n{,n}]>

- 动作集是在发出STOP语句或动作时发出但尚未终止的一组动作。
- 如果没有停止的动作集，则RESUME不会产生任何动作。
- 如果已停止多个动作集，则RESUME将重新启动最近停止的未恢复动作集。 随后的RESUME将以后进先出顺序启动其他RESUME。
- 停止动作组中的那些动作按原先发出的顺序恢复。

- 如果在发出RESUME语句时正在进行动作，则在当前动作完成后发生任何恢复的动作(s)。
- 如果 group 子句不存在，则将恢复任务所控制的所有组。
- 如果停止，恢复，取消或保持的动作是与其他组同时或协调动作的一部分，则停止，恢复，取消或保持所有组的动作。
- 无法为其他任务恢复运动。

另请参阅：附录E，''语法图''以获取更多语法信息

示例：在以下示例中，如果DIN [1]为ON，则停止运动。 按下F1后恢复。

RESUME 声明

```
CONDITION[1]:
  WHEN DIN[1] = ON DO
    STOP
  ENDCONDITION
move_to_pr1 ── Call TP program to move to PR[1]
IF DIN[1] THEN
  WRITE(' Motion stopped')
  WRITE(CR, 'Motion and the program will resume')
  WRITE(CR, '    when F1 of teach pendant is pressed')
  WAIT FOR TPIN[129]
  RESUME
ENDIF
```

A.19.17 RETURN 声明

用途：将控制从例程/程序返回到调用例程/程序，可选择返回 issult

语法： RETURN < (value) >

细节：

- 从函数返回时需要 *value*，但从过程返回时不允许使用 *value*。 *value* 的数据类型必须与函数声明中使用的类型相同。
- 如果主程序执行RETURN语句，则执行终止且无法恢复。 所有正在进行的动议将正常完成。
- 如果未指定RETURN，则END语句用作返回。

- 如果函数例程返回END语句而不是RETURN语句，程序将中止12321错误，'结束函数的结束STMT'。'

另请参阅：附录E，''语法图'，'了解更多语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.7节“列出文件和程序以及操作字符串”（LIST_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

A.19.18 ROUND 内置函数

用途：返回最接近指定REAL参数的INTEGER值

语法：ROUND(x)

Function Return Type :INTEGER

Input/Output 参数：

[in] x :REAL

%ENVIRONMENT Group :SYStem

细节：

- 返回值是最接近 REAL 值 *x* 的 INTEGER 值，如以下规则所示：
  — 如果 *x* >= 0，则令 n 为正 INTEGER，使得 n <= x <= n + 1
  — 如果 *x* >= n + 0.5，则返回 n + 1; 否则，返回n。
  — 如果 *x* <= 0，则令 n 为负 INTEGER，使得 n> = x> = n - 1
  — 如果 *x* <= n - 0.5，则返回 n - 1; 否则，返回 n。
- *x* 必须在 -2147483648 到 +2147483646 的范围内。 否则，程序将因错误而中止。

另请参见：TRUNC内置函数

示例：有关详细的程序示例，请参见第B.7节“列出文件和程序以及操作字符串”（LIST_.EX.KL）。

A.19.19 ROUTINE 声明

用途：指定一个例程名称，包含参数和类型，以及函数例程的返回值数据类型

语法： ROUTINE name < param_list > <: return_type >

其中：

name：一个有效的KAREL标识符

param_list：如下面所描述的

return_type：任何可以由函数返回的数据类型，即除FILE，PATH和视觉类型之外的任何类型

细节：

- *name* 指定例程名称。
- *param_list* 的格式是（name_group {; name_group}）

  — *name_group* 的格式为 param_name：param_type

  — *param_name* 是一个参数，可以在例程体内用作数据类型 *param_type* 的变量。

  — 如果 aparam_type 或 return_type 是一个 ARRAY，大小被排除。 如果 *param_type* 是 STRING，则排除字符串长度。
- 当例程正文遵循 ROUTINE 语句时，*param_list* 中的名称用于将传入的参数与例程正文中的参数的引用关联起来。
- 例程来自另一个程序时，参数列表中的名称不重要，但必须存在才能指定参数的数量和数据类型。
- 如果 ROUTINE 语句包含 *return_type*，则该例程是一个函数例程并返回一个值。 否则，这是一个程序例程。
- ROUTINE 语句后面必须有一个例程主体或一个FROM子句。

示例：请参阅以下各节以获取详细的程序示例：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.4节“标准例程”（ROUT_EX.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.7节“列出文件和程序以及操纵字符串”（LIST_EX.KL）

第B.8节“生成并沿六角形路径移动 “（GEN_HEX.KL）

第B.9节“使用文件和设备内置程序”（FILE_EX.KL）

第B.10节“使用动态显示内置插件”（DYN_DISP.KL）

第B.12节“从字典文件中显示列表”（DCLST_EX.KL）

第B.14节“在复制的示教器上应用胶印”（CPY_TP.KL）

第B.1节“设置数字输出端口进行监测”（DOUT_EX.KL）

A.19.20 RUN_TASK 内置程序

用途：将指定程序作为子任务运行

语法：RUN_TASK (task_name, line_number, pause_on_sft, tp_motion, lock_mask, status)

Input/Output 参数：

[in] task_name：STRING

[in] line_number：INTEGER

[in] pause_on_sft：BOOLEAN

[in] tp_motion：BOOLEAN

[in] lock_mask：INTEGER

[out] status：INTEGER

%ENVIRONMENT Group :MULTI

细节：

- *task_name* 是要运行的任务的名称。 这会创建一个子任务。 执行此内置任务称为父任务。
- 如果任务已经存在并暂停，它将继续。 新任务不会创建。
- *line_number* 指定执行开始的行。 使用0从程序的开始处开始。 这仅适用于示教器程序。
- 如果 *pause_on_sft* 为 TRUE，则释放示教器切换键时任务暂停。
- 如果 *tp_motion* 为 TRUE，任务可以在示教器启用时执行动作。 如果 *tp_motion* 为 TRUE，则TP必须启用。
- 如果 *tp_motion* 为TRUE且示教器已启用，则在 *lock_mask* 中指定的动作组的控制将从父任务转移到子任务。 组编号必须在1到控制器上定义的组的总数的范围内。 位1指定组1，位2指定组2，依此类推。

表 A–19. Group_mask 设置

GROUP	DECIMAL	BIT
Group 1	1	1
Group 2	2	2
Group 3	4	3

要指定多个组，请选择表A-19中所示的十进制值，这些值对应于所需的组。 然后使用OR运算符将它们连接在一起。 例如，要指定组1和3，请输入“1 OR 4”。

- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：CONT_TASK，PAUSE_TASK，ABORT_TASK内置程序，第15章 多任务
示例：有关详细的程序示例，请参见第B.10节 “使用动态显示内置”（DYN_DISP.KL）。

A.20 - S - KAREL 语言描述

A.20.1 SAVE 内置程序

用途：将程序或变量保存到指定的文件中

语法：SAVE (prog_nam, file_spec, status)

Input/Output 参数：

[in] prog_nam :STRING

[in] file_spec :STRING

[out] status :INTEGER

%ENVIRONMENT Group :MEMO

细节：

- *prog_nam* 指定程序名称。 如果程序名称为 "*"，则保存指定类型的所有程序或变量。必须将 *prog_name* 设置为 "* SYSTEM *" 才能保存所有系统变量。

- *file_spec* 指定要保存到的文件的设备，名称和类型。 该类型还意味着是否正在保存程序或变量。

以下类型有效：

.TP：示教器程序

.VR：KAREL变量

.SV：KAREL系统变量

.IO：I / O配置数据

- 如果指定设备上已存在 *file_spec*，则会返回错误，但不会发生保存。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：CLEAR，LOAD 内置程序

示例：有关详细的程序示例，请参见第B.3节 “将数据保存到默认设备”（SAVE_VRS.KL）。

A.20.2 SAVE_DRAM 内置程序

用途：将RAM变量内容保存到 FlashROM。

语法：SAVE_DRAM (prog_nam, status)

Input/Output 参数：

[in] prog_nam: STRING

[out] status: INTEGER

%ENVIRONMENT Group: MEMO

细节：

- *prog_nam* 指定程序名称。 此操作会将 DRAM 中任何变量的当前值保存到指定程序的 FlashROM 中。 上电时，这些保存的值将自动加载到 DRAM 中。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

A.20.3 SELECT ... ENDSELECT 声明

用途：允许执行一系列语句序列中的一个，具体取决于INTEGER表达式的值。

语法：SELECT case_val OF

CASE(value{,value}):

{statement}

{ CASE(value{, value}):

{statement} }

<ELSE:

{ statement }>

ENDSELECT

其中：

case_val：INTEGER 表达式

value：INTEGER 常量或文字

statement：有效的KAREL可执行语句

细节：

- *case_val* 与每个子句中 CASE 后面的每个值进行比较。 如果它等于其中任何一个，则执行 CASE 和下一个子句之间的语句。
- SELECT 语句中最多可以使用1000个CASE子句。
- 如果在多个CASE中列出了相同的INTEGER值，则只会执行第一个匹配CASE后面的语句序列。
- 如果使用ELSE子句且表达式 *case_val* 与 CASE 子句中的任何值都不匹配，则执行关键字 ELSE 和 ENDSELECT 之间的语句。
- 如果没有使用 ELSE 子句且表达式 *case_val* 与 CASE 子句中的任何值都不匹配，则程序将以“'CASE中的不匹配'”错误中止。

另请参阅：附录E，''语法图'，'了解更多语法信息

示例：有关详细的程序示例，请参阅以下部分：

第B.3节“将数据保存到默认设备”（SAVE_VR.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.14节“将偏移应用于复制的示教器程序”（CPY_TP.KL）

A.20.4 SELECT_TPE 内置程序

用途：选择指定名称的程序

语法：SELECT_TPE(prog_name, status) Input/

Output 参数：

[in] prog_name :STRING

[out] status： :INTEGER

%ENVIRONMENT Group :TPE

细节：

- *prog_name* 指定要作为示教器默认选择的程序的名称。 这是示教器“正在使用”的程序。 如果按下 CYCLE START 按钮或按下示教器 FWD键，也会执行该程序。
- *status* 说明了尝试操作的状态。 如果它不等于0，则表示发生了错误。

另请参阅：OPEN_TPE 内置过程

示例：有关详细的程序示例，请参见第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）。

A.20.5 SEMA_COUNT 内置功能

用途：返回指定信号量的当前值

语法：SEMA_COUNT (semaphore_no)

函数返回类型：INTEGER

Input/Output 参数：

[in] semaphore_no： INTEGER

%ENVIRONMENT Group :MULTI

细节：

- 返回 *semaphore_no* 指示的信号量的值。
- 每个 POST_SEMA 调用和 SIGNAL SEMAPHORE 动作指定相同的 *semaphore_no*，该值递增。 每次 PEND_SEMA 调用都会减少它。
- 如果 SEMA_COUNT 大于零，则 PEND_SEMA 调用将立即“通过”。 如果它是-n（减去n），则此信号量上有n个待处理的任务。

另请参阅：POST_SEMA，PEND_SEMA，CLEAR_SEMA内置程序，第15章 多任务

示例：请参阅第15章“多任务”中的示例

A.20.6 SEMAPHORE 条件

用途：监视指定信号量的值

语法：SEMAPHORE[semaphore_no]

细节：

- *semaphore_no* 指定要使用的信号量号。
- *semaphore_no* 必须在 1 到控制器上定义的信号量数量的范围内。
- 当指示的信号量的值大于零时，满足条件（TRUE）。

A.20.7 SEND_DATAPC 内置程序

用途：将事件消息和其他数据发送到 PC。

语法：SEND_DATAPC(event_no, dat_buffer, status)

Input/Output 参数：

[in] event_no :INTEGER

[in] dat_buffer :ARRAY OF BYTE

[out] status :INTEGER

%ENVIRONMENT Group :PC

细节：

- *event_no* - GEMM事件编号。 有效值为0到255。
- *dat_buffer* - 最多244个字节的数组。 KAREL内置函数 ADD_BYNAMEPC,ADD_INTPC,ADD_REALPC和ADD_STRINGPC可用于格式化KAREL字节缓冲区。 实际的数据缓冲区格式取决于PC的需求。 控制器上没有错误检查dat_buffer格式。
- *status* - 尝试操作的状态。 如果不为0，则发生错误，并且事件请求未发送到PC。

另请参见：ADD_BYNAMEPC, ADD_INTPC, ADD_REALPC, ADD_STRINGPC

示例：以下示例使用数据缓冲区将事件12发送到PC。

SEND_DATAPC 内置程序

```
PROGRAM TESTDATA
%ENVIRONMENT PC
CONST
  er_abort = 2
VAR
  dat_buffer:   ARRAY[100] OF BYTE
  index:        INTEGER
  status:       INTEGER
BEGIN
  index = 1
  ADD_INTPC(dat_buffer,index,55,status)
  ADD_REALPC(dat_buffer,index,123.5,status)
  ADD_STRINGPC(dat_buffer,index,'YES',status)

  -- 将事件12和数据缓冲区发送到PC
  SEND_DATAPC(12,dat_buffer,status)
  IF status<>0 THEN
    POST_ERR(status,'',0,er_abort)
  ENDIF
END testdata
```

A.20.8 SEND_EVENTPC 内置程序

用途：向 PC 发送事件消息。

语法：SEND_EVENTPC(event_no, status)

Input/Output 参数：

[in] event_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PC

细节：

- *event_no* - GEMM事件编号。 有效值为0到255。
- *status* - 尝试操作的状态。 如果不为0，则发生错误，并且事件请求未发送到PC。

示例：以下示例将事件12发送到PC。

SEND_EVENTPC 内置程序

```
PROGRAM TESTEVT
%ENVIRONMENT PC
CONST
  er_abort = 2
VAR
  status:  INTEGER
BEGIN
  -- 将事件12发送到PC
  SEND_EVENTPC(12,status)     -- 调用内建在这里
  IF status<>0 THEN
    POST_ERR(status,'',0,er_abort)
  ENDIF
END testevt
```

A.20.9 SET_ATTR_PRG 内置程序

用途：设置指定示教器或KAREL程序的属性数据

语法：SET_ATTR_PRG(program_name, attr_number, int_value, string_value, status)

Input/Output 参数：

[in] program_name : STRING

[in] attr_number : INTEGER

[in] int_value : INTEGER

[in] string_value : STRING

[out] status : INTEGER

%ENVIRONMENT Group :TPE

细节 :

- *program_name* 指定要为其设置属性数据的程序。
- *attr_number* 是要设置其值的属性。以下属性有效：

  AT_PROG_TYPE：（ # ）程序类型

  AT_PROG_NAME：程序名称（String [12]）

  AT_OWNER：所有者（String [8]）

  AT_COMMENT：注释（String [16]）

  AT_PROG_SIZE：（ # ）程序大小

  AT_ALLC_SIZE：（ # ）已分配内存的大小

  AT_NUM_LINE：（ # ）行数

  AT_CRE_TIME：（ # ）创建（加载）时间

  AT_MDFY_TIME：（ # ）修改时间

  AT_SRC_NAME：源文件（或原始文件）名称（String [128]）

  AT_SRC_VRSN：源文件版本

  AT_DEF_GROUP：默认运动组（用于任务属性）

  AT_PROTECT：保护代码; 1：保护关闭; 2：保护开启

  AT_STORAGE：存储类型; TPSTOR_CMOS; TPSTOR_SHADOW; TPSTOR_FILE; TPSTOR_SHOD

  AT_STK_SIZE：堆栈大小（用于任务属性）

  AT_TASK_PRI：任务优先级（用于任务属性）

  AT_DURATION：时间片持续时间（对于任务属性）

  AT_BUSY_OFF：忙灯关闭（用于任务属性）

  AT_IGNR_ABRT：忽略中止请求（对于任务属性）

  AT_IGNR_PAUS：忽略暂停请求（对于任务属性）

AT_CONTROL：控制代码（用于任务属性）

(#) --- 无法设置。

- 如果属性数据是数字，则将其设置为 *int_value*，并忽略 *string_value*。
- 如果属性数据是字符串，则将其设置为 *string_value*，并忽略 *int_value*。
- *status* 说明尝试操作的状态。 如果它不等于0，则表示发生了错误。 可能发生的一些错误是：

  7073 *program_name* 中指定的程序不存在

  7093 程序的属性在运行时无法设置

  17033 *attr_number* 具有非法值或无法设置

A.20.10 SET_CURSOR 内置程序

用途：在窗口中设置光标位置

语法：SET_CURSOR(file_var, row, col, status)

Input/Output 参数：

[in] file_var : FILE

[in] row : INTEGER

[in] col : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- 设置对窗口打开的指定文件的当前光标，以便后续写入将在指定位置开始。 *file_var* 必须对窗口打开。
- *row* 值1表示窗口的顶行。 *col* 值为1表示窗口的最左列。 *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：DEF_WINDOW 内置过程

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

A.20.11 SET_EPOS_REG 内置程序

用途：在指定的寄存器中存储 XYZWPREXT 值

语法：SET_EPOS_REG(register_no, posn, status <, group_no>)

Input/Output 参数：

[in] register_no：INTEGER

[in] posn：XYZWPREXT

[out] status：INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定存储值的位置寄存器。
- 位置数据以 XYZWPREXT 表示形式设置。
- *status* 说明尝试操作的状态。 如果它不等于 0，则发生错误。
- 如果省略 *group_no*，则假定程序的默认组。 其他组的数据不会更改。
- 如果指定了 *group_no*，则它必须在 1 到控制器上定义的组总数的范围内。

另请参见：SET_POS_REG,SET_JPOS_REG内置过程,GET_POS_REG,GET_JPOS_REG内置函数

示例：以下示例设置指定寄存器的扩展位置。

SET_EPOS_REG 内置程序

```
PROGRAM spe
%environment REGOPE
VAR
  cur_pos: XYZWPREXT
```

```
  posget: XYZWPREXT
  status: INTEGER
  v_mask, g_mask: INTEGER
  reg_no: INTEGER
BEGIN
  reg_no = 1
  cur_pos = CURPOS(v_mask,g_mask)
  SET_EPOS_REG(reg_no,cur_pos,status)
  posget = GET_POS_REG(reg_no,status)
END spe
```

A.20.12 SET_EPOS_TPE 内置程序

用途：在指定示教程序的指定位置存储 XYZWPREXT 值

语法：SET_EPOS_TPE (open_id, position_no, posn, status <,group_no>)

Input/Output 参数：

[in] open_id : INTEGER

[in] position_no : INTEGER

[in] posn : XYZWPREXT

[out] status : INTEGER

[in] group_no : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前必须打开一个程序。
- *position_no* 指定程序中存储值的位置。
- 必须已存在使用 *position_no* 的动作指令，否则示教器程序将不使用该位置。
- 位置数据以 XYZWPREXT 表示设置，无转换。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 如果省略 *group_no*，则假定程序的默认组。 其他组的数据
- 没有改变。

- 如果指定了 group_no，则它必须在 1 到控制器上定义的组总数的范围内。

A.20.13 SET_FILE_ATR 内置程序

用途：在打开文件之前设置文件的属性

语法：SET_FILE_ATR(file_id, atr_type <,atr_value>)

Input/Output 参数：

[in] file_id: FILE

[in] atr_type：INTEGER 表达式

[in] atr_value：INTEGER 表达式

%ENVIRONMENT Group :PBCORE

细节：

- *file_id* 是将在OPEN FILE,WRITE,READ 和 / 或 CLOSE FILE 语句中使用的文件变量。
- *atr_type* 指定要设置的属性类型。 应使用表7-2中指定的预定义常量。
- 根据设置的属性类型，*atr_value* 是可选的。

XML 相关

用途：在打开属性文件之前将其设置为XML

语法：SET_FILE_ATR(xml_file, ATR_XML)

Input/Output 参数：

[in] xml_file: FILE

[in] ATR_XML: INTEGER 表达式

细节：

- *xml_file* 是将在OPEN FILE，WRITE，READ和/或CLOSE FILE语句中使用的文件变量。
- ATR_XML 指定要设置的属性类型。 应使用表7-2中指定的预定义常量。

- 该文件必须作为 RO 文件打开。 在打开文件之前，您不能执行任何其他 XMI 操作。

另请参见：SET_PORT_ATR 内置函数，第7.3.1节 “设置文件属性” 和第9.5节 “格式化XML输入”

示例：请参见第9.5节 “格式化XML输入”

A.20.14 SET_FILE_POS 内置程序

用途：将要在指定文件中发生的下一个READ或WRITE操作的文件位置设置为新指定文件位置的值

语法：SET_FILE_POS(file_id, new_file_pos, status)

Input/Output 参数：

[in] file_id：FILE

[in] new_file_pos：INTEGER expression

[out] status：INTEGER variable

%ENVIRONMENT Group :FLBT

细节：

- 必须在FROM或RAM磁盘上打开和解压缩与 *file_id* 关联的文件。 否则，程序将中止并显示错误。
- *new_file_pos* 必须在-1到 file.at_eof 中的字节数的范围内：指定要在文件末尾设置文件位置。
  at_sof：指定在文件开头设置文件位置。
  — 任何其他值都会导致文件从文件开头设置指定的字节数。
- 如果 *new_file_pos* 介于-1和文件中的字节数之间，则 *status* 设置为0，表示文件位置已成功设置。如果不等于0，则发生错误。

另请参阅：第7章 文件输入/输出操作

示例：以下示例打开filepos.dt数据文件，从目录设置文件位置，从文件中读取位置，并将位置存储在PATH，*my_path* 中。

SET_FILE_POS 内置程序

```
OPEN FILE file_id ('RW','filepos.dt')
```

```
FOR i = 1 TO PATH_LEN(my_path) DO
  SET_FILE_POS(file_id, pos_dir[i], status)
  IF status = 0 THEN
    READ file_id (temp_pos)
    my_path[i].node_pos = temp_pos
  ENDIF
ENDFOR
```

A.20.15 SET_INT_REG 内置程序

用途：在指定的寄存器中存储一个整数值

语法：SET_INT_REG(register_no, int_value, status)

Input/Output 参数：

[in] register_no：INTEGER

[in] int_value：INTEGER

[out] status：INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定将存储 *int_value* 的寄存器。
- *status* 解释了尝试操作的状态。 如果不等于 0，则发生错误。

另请参见：GET_INT_REG，GET_REAL_REG，SET_REAL_REG内置过程

例如：有关详细的程序示例，请参阅 第B.5节“使用寄存器内置程序”（REG_EX.KL）。

A.20.16 SET_JPOS_REG 内置程序

用途：在指定的寄存器中存储 JOINTPOS 值

语法：SET_JPOS_REG(register_no, jpos, status<, group_no>)

Input/Output 参数：

[in] register_no：INTEGER

[in] jpos : JOINTPOS

[out] status :INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定存储位置 *jpos* 的位置寄存器。
- 位置数据在 JOINTPOS 表示中设置，没有转换。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 如果省略 *group_no*，则假定程序的默认组。 其他组的数据不会更改。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。

另请参见：GET_JPOS_REG, GET_POS_REG, SET_POS_REG, POS_REG_TYPE内置程序

示例：有关详细的程序示例，请参见第B.5节“使用寄存器内置函数”（REG_EX.KL）。

A.20.17 SET_JPOS_TPE 内置程序

用途：在指定示教程序的指定位置存储 JOINTPOS 值

语法：SET_JPOS_TPE(open_id, position_no, posn, status<, group_no>)

Input/Output 参数：

[in] open_id : INTEGER

[in] position_no : INTEGER

[in] posn : JOINTPOS

[out] status : INTEGER

[in] group_no :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前，必须使用OPEN_TPE内置函数打开程序，并具有读/写访问权限。
- *position_no* 指定程序中存储值的位置。
- 位置数据在 JOINTPOS 表示中设置，没有转换。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 如果省略 *group_no*，则假定程序的默认组。 其他组的数据不会更改。
- 如果指定了 *group_no*，则它必须在 1 到控制器上定义的组总数的范围内。

另请参见：GET_JPOS_TPE,GET_POS_TPE,SET_POS_TPE,GET_POS_TYP内置过程

示例：请参见第B.14节 “将偏移应用于复制的示教器程序”（CPY.TP.KL），详细的程序示例。

A.20.18 SET_LANG 内置程序

用途：更改当前语言

语法：SET_LANG(lang_name, status)

Input/Output 参数：

[in] lang_name :STRING

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *lang_name* 指定应该从哪个语言读取/写入字典。 使用以下预定义常量之一：

  dp_default

  dp_english

  dp_japanese

  dp_french

  dp_german

dp_spanish

- 只读系统变量 $LANGUAGE 指示当前使用的语言。
- *status* 说明了尝试操作的状态。 如果不等于0，则设置语言时出错。
- 如果没有将字典加载到指定的语言，将返回错误33003，“找不到语言的dict”。 KCL命令“SHOW LANGS”可用于查看系统中创建的语言。

另请参见：第10章 字典和表单

示例：有关详细的程序示例，请参见第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）。

A.20.19 SET_PERCH 内置程序

用途：设置一组轴的接触前位置和精确度

语法：SET_PERCH(jpos, tolerance, indx)

Input/Output 参数：

[in] jpos：JOINTPOS

[in] tolerance：ARRAY[6] of REAL

[in] indx：INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- *jpos* 的值转换为弧度并存储在系统变量$ REFPOS1 [indx].$ perch_pos 中。
- *tolerance* 数组转换为度数并存储在系统变量$ REFPOS1 [indx].$ perchtol 中。 如果 *tolerance* 数组未初始化，则会生成错误。
- *indx* 指定要在 $ REFPOS1 数组中设置的元素编号。
- 从指定位置 *jpos* 隐含轴组。 如果JOINTPOS不在组1中，则使用系统变量$ REFPOSn，其中 *n* 对应于 *jpos* 的组编号，*indx* 必须设置为1。

另请参阅：适用于特定应用的FANUC机器人设置和操作手册，以设置参考位置

示例：在以下示例中，$ REFPOS1 [2].$ perchpos 和 $ REFPOS1 [2].$ perchtol根据 perch_pos 和 tolerance [i] 设置。

SET_PERCH 内置程序

```
VAR
  perch_pos: JOINTPOS IN GROUP[1]
BEGIN
  FOR i = 1 to 6 DO
    tolerance[i] = 0.01
  ENDFOR
  SET_PERCH (perch_pos, tolerance, 2)
END
```

A.20.20 SET_PORT_ASG 内置程序

用途：允许KAREL程序将一个或多个逻辑端口分配给指定的物理端口

语法：SET_PORT_ASG(log_port_type, log_port_no, rack_no, slot_no, phy_port_type, phy_port_no, n_ports, status)

Input/Output 参数：

[in] log_port_type：INTEGER

[in] log_port_no：INTEGER

[in] rack_no：INTEGER

[in] slot_no：INTEGER

[in] phy_port_type：INTEGER

[in] phy_port_no：INTEGER

[in] n_ports：INTEGER

[out] status：INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *log_port_type* 指定要分配的端口类型的代码。 代码以KLIOTYPS.KL定义。

- *log_port_no* 指定要分配的端口号。
- *rack_no* 是包含端口模块的机架。对于过程I / O板，存储器映像和虚拟端口，这是零;对于 Allen-Bradley 和 Genius 端口，这是16。
- *slot_no* 是包含端口模块的插槽。对于过程I / O板，这是SLC-2链中的序列。对于内存映像和虚拟端口，这是零;对于 Allen-Bradley 和 Genius 端口，这是1。
- *phy_port_type* 是要分配给的端口类型。通常这与*log_port_type*相同。例外情况是 *log_port_type* 是组类型（*io_gpin*或*io_gpout*）还是将端口分配给内存映像或虚拟端口.
- *phy_port_no* 是要分配给的端口号。如果log_port_type是一个组，则这是该组的最低有效位的端口号。
- *n_ports* 是要分配给逻辑端口的物理端口数。如果 *log_port_type* 是组类型，则 *n_ports* 表示组中的位数。设置数字I / O时，n_ports是您正在配置的点数。在大多数情况下，这将是8，但可能是1到8。
- 如果参数有效，则返回零状态。否则，将返回错误代码。如果指定的端口不存在或者不允许将 *log_port_type* 分配给 *phy_port_type*，则分配无效。

例如，无法将GIN分配给DOUT。 *log_port_type* 和 *phy_port_type* 都不是系统端口类型（例如，SOPIN）。

注意分配在下次启动之前不会生效。

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.20.21 SET_PORT_ATR 内置功能

用途：设置端口的属性

语法：SET_PORT_ATR(port_id, atr_type, atr_value)

函数返回类型：INTEGER

Input/Output 参数：

[in] port_id： INTEGER

[in] atr_type： INTEGER

[in] atr_value： INTEGER

%ENVIRONMENT Group :FLBT

细节：

- *port_id* 是预定义常量之一，如下所示：

  PORT_1

  PORT_2

  port_3

  port_4

- *atr_type* 指定要设置的属性类型。 应使用以下预定义常量之一：

  atr_readahd：预读缓冲区

  atr_baud：波特率

  atr_parity：奇偶校验

  atr_sbits：停止位

  atr_dbits：数据长度

  atr_xonoff：XON / XOFF

  atr_eol：行尾

  atr_modem：调制解调器线

- *atr_value* 指定属性类型的值。 请参见下一页的表A-20，其中包含可接受的具有相应值的预定义属性类型。

表 A–20. 属性值

ATR_TYPE	ATR_VALUE
atr_readahd	任何整数，表示128字节的倍数（例如：atr_value = 1表示缓冲区长度为128字节。
atr_baud	baud_9600 baud_4800 baud_2400 baud_1200
atr_parity	parity_none parity_even parity_odd

表 A–20. 属性值（续）

ATR_TYPE	ATR_VALUE
atr_sbits	sbits_1 sbits_15 sbits_2
atr_dbits	dbits_5 dbits_6 dbits_7 dbits_8
atr_xonoff	xf_not_used xf_used
atr_eol	ASCII码值，请参阅附录D“字符代码”
atr_modem	md_not_used md_use_dsr md_nouse_dsr md_use_dtr md_nouse_dtr md_use_rts md_nouse_rts

- 返回的整数是此操作到端口的状态。

另请参阅：SET_FILE_ATR内置过程，第7.3.1节“设置文件和端口属性”，以获取更多信息

示例：请参阅GET_PORT_ATR 内置函数的示例。

A.20.22 SET_PORT_CMT 内置程序

用途：允许KAREL程序为指定的逻辑端口设置示教器上显示的注释

语法：SET_PORT_CMT(port_type, port_no, comment_str, status)

Input/Output 参数：

[in] port_type：INTEGER

[in] port_no：INTEGER

[in] comment_str：STRING

[out] status : INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定正在设置其模式的端口类型的代码。 代码以 KLIOTYPS.KL 定义。
- *port_no* 指定正在设置其模式的端口号。
- *comment_str* 是一个字符串，其值是指定端口的注释。 这不得超过16个字符。
- 如果参数有效并且可以为指定的端口设置指定的模式，则返回零状态。

另请参见：ET_PORT_VALUE, SET_PORT_MOD, GET_PORT_CMT, GET_PORT_VALUE, GET_PORT_MOD内置过程

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.20.23 SET_PORT_MOD 内置程序

用途：允许KAREL程序为指定的逻辑端口设置（或重置）特殊端口模式

语法：SET_PORT_MOD(port_type, port_no, mode_mask, status)

Input/Output 参数：

[in] port_type : INTEGER

[in] port_no : INTEGER

[in] mode_mask : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定正在设置其模式的端口类型的代码。 代码以 KLIOTYPS.KL 定义。
- *port_no* 指定正在设置其模式的端口号。
- *mode_mask* 是一个掩码，指定打开哪些模式。 定义了以下模式：

1：反向模式 - 端口感被反转; 如果端口设置为TRUE，则物理输出设置为FALSE。 如果端口设置为FALSE，则物理输出设置为TRUE。 如果在读取端口时物理输入为TRUE，则返回FALSE。 如果在读取端口时物理输入为FALSE，则返回TRUE.ports。

2：互补模式 - 逻辑端口分配给两个物理端口，其值互补。 在这种情况下，port_no必须是奇数。 如果端口n设置为TRUE，则端口n设置为TRUE，端口n + 1设置为FALSE。 如果端口n设置为FALSE，则端口n设置为FALSE，端口n + 1设置为TRUE。 这仅对输出有效注意模式设置在下次启动之前不会生效。

- 如果参数有效并且可以为指定的端口设置指定的模式，则返回零状态。

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.20.24 SET_PORT_SIM 内置程序

用途：设置模拟的端口

语法：SET_PORT_SIM(port_type, port_no, value, status)

Input/Output 参数：

[in] port_type : INTEGER

[in] port_no : INTEGER

[in] value : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定要模拟的端口类型的代码。 代码以 KLIOTYPS.KL 定义。
- *port_no* 指定要模拟的端口号。
- *value* 指定要设置的初始值。
- 如果模拟端口，则 *status* 返回零。

另请参见：SET_PORT_ASG，GET_PORT_ASG，GET_PORT_SIM内置过程

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口” （ DOUT_EX.KL ）。

A.20.25 SET_PORT_VAL 内置程序

用途：允许KAREL程序为指定的逻辑端口设置指定的输出（或模拟输入）

语法：SET_PORT_VAL(port_type, port_no, value, status)

Input/Output 参数：

[in] port_type : INTEGER

[in] port_no : INTEGER

[in] value : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :IOSETUP

细节：

- *port_type* 指定正在设置其模式的端口类型的代码。 代码以 KLIOTYPS.KL 定义。
- *port_no* 指定正在设置其模式的端口号。
- *value* 表示要分配给指定端口的值。 如果 *port_type* 是BOOLEAN（即DOUT），则应为 0 = OFF 或 1 = ON。 如果模拟端口，则此字段可用于设置输入端口。
- 如果参数有效并且可以为指定的端口设置指定的模式，则 *status* 返回零。

另请参见：SET_PORT_VALUE, SET_PORT_MOD, GET_PORT_CMT, GET_PORT_VALUE, GET_PORT_MOD 内置过程

示例：以下示例设置指定端口的值。

SET_PORT_VAL 内置程序

```
PROGRAM setvalprog
%ENVIRONMENT IOSETUP
%INCLUDE FR:\kliotyps
 ROUTINE set_value(port_type: INTEGER;
       port_no:  INTEGER;
```

```
      g_value:  BOOLEAN): INTEGER
 VAR
  value: INTEGER
  status: INTEGER
 BEGIN
  IF g_value THEN
   value = 1
  ELSE
   value = 0;
  ENDIF
  SET_PORT_VAL (port_type, port_no, value, status)
  RETURN (status)
 END set_value
BEGIN
END setvalprog
```

A.20.26 SET_POS_REG 内置程序

用途：在指定的位置寄存器中存储 XYZWPR 值

语法：SET_POS_REG(register_no, posn, status<, group_no>)

Input/Output 参数：

[in] register_no：INTEGER

[in] posn：XYZWPR

[out] status：INTEGER

[in] group_no：INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定存储值的位置寄存器。
- 位置数据在 XYZWPR 表示中设置，没有转换。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 如果省略 *group_no*，则假定程序的默认组。 其他组的数据不会更改。
- 如果指定了 *group_no*，则它必须在 1 到控制器上定义的组总数的范围内。

示例：有关详细的程序示例，请参见第B.5节 “使用寄存器内置函数”（REG_EX.KL）。

A.20.27 SET_POS_TPE 内置程序

用途：在指定示教程序的指定位置存储 XYZWPR 值

语法：SET_POS_TPE(open_id, position_no, posn, status<, group_no>)

Input/Output 参数：

[in] open_id : INTEGER

[in] position_no : INTEGER

[in] posn : XYZWPR

[out] status : INTEGER

[in] group_no : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前，必须使用 OPEN_TPE 内置函数打开程序，并具有读/写访问权限。
- *position_no* 指定程序中存储值的位置。
- 必须已存在使用 *position_no* 的动作指令，否则示教器程序将不使用该位置。
- 位置数据在 XYZWPR 表示中设置，没有转换。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 如果省略 *group_no*，则假定程序的默认组。 其他组的数据不会更改。
- 如果指定了 *group_no*，则它必须在1到控制器上定义的组总数的范围内。

示例：有关详细的程序示例，请参见第B.14节 “对复制的示教器程序应用偏移”（CPY_TP.KL）。

A.20.28 SET_PREG_CMT 内置程序

用途：根据给定的寄存器编号和给定的注释设置 KAREL 位置寄存器的注释信息。

语法：SET_PREG_CMT (register_no, comment_string, status)

Input/Output 参数：

[in] register_no: INTEGER

[in] comment_string: STRING

[out] status: INTEGER

%ENVIRONMENT group: REGOPE

A.20.29 SET_REAL_REG 内置程序

用途：在指定的寄存器中存储 REAL 值

语法：SET_REAL_REG(register_no, real_value, status)

Input/Output 参数：

[in] register_no : INTEGER

[in] real_value : REAL

[out] status : INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定将存储 *real_value* 的寄存器。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：SET_INT_REG，GET_REAL_REG，GET_INT_REG内置过程

A.20.30 SET_REG_CMT 内置程序

用途：根据给定的寄存器编号和注释设置KAREL寄存器的注释信息。

语法：SET_REG_CMT (register_no, comment_string, status

Input/Output 参数：

[in] register_no: INTEGER

[in] comment_string: STRING

[out] status: INTEGER

%ENVIRONMENT group REGOPE

细节：

- *register_no* 指定从哪个寄存器中检索注释。 *comment_string* 表示用于设置给定寄存器的注释的数据。 如果 *comment_string* 超过16个字符，则内置函数将截断该字符串。

A.20.31 SET_SREG_CMT 内置程序

用途：设置指定字符串寄存器的注释。

语法：SET_SREG_CMT(register_no, comment, status)

Input/Output 参数：

[in] register_no :INTEGER

[in] comment :STRING[254]

[out] status :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的字符串寄存器。
- *comment* 包含要设置为指定字符串寄存器的注释。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_STR_REG，GET_SREG_CMT，SET_STR_REG内置过程。

A.20.32 SET_STR_REG 内置程序

用途：设置指定字符串寄存器的值。

语法：SET_STR_REG(register_no, value, status)

Input/Output 参数：

[in] register_no :INTEGER

[in] value :STRING[254]

[out] status :INTEGER

%ENVIRONMENT Group :REGOPE

细节：

- *register_no* 指定要获取的字符串寄存器。
- *value* 包含要设置为指定字符串寄存器的值。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参见：GET_STR_REG，GET_SREG_CMT，SET_SREG_CMT内置过程。

A.20.33 SET_TIME 内置程序

用途：设置KAREL系统中的当前时间

语法：SET_TIME(i)

Input/Output 参数：

[in] i : INTEGER

%ENVIRONMENT Group :TIM

细节：

- *i* 在 KAREL 系统中持有 INTEGER 时间表示。 该值以32位 INTEGER 格式表示，如下所示：

表 A–21. 32位INTEGER时间格式

31–25	24–21	20–16
year	month	day
15–11	10–5	4–0
hour	minute	second

- 各个领域的内容如下：
  - 日期：
    第15-9位 - 自1980年以来的年份
  - 第8-5位 - 月（1-12）
  - 第4-0位 - 月的一天

— TIME:

第31-25位 - 小时数（0-23）

第24-21位 - 分钟数（0-59）

第20-16位 - 2秒增量（0-29）

- 可以使用GET_TIME和CNV_STR_TIME内置过程确定此值。
- 如果i为0，则不会更改系统上的时间。
- 可以比较INTEGER值以确定一次是否比另一次更新。

另请参阅：CNV_STR_TIME，GET_TIME内置过程

示例：以下示例使用 CNV_STR_TIME 内置过程将用户以“DD-MMM-YYY HH：MM：SS”格式输入的 STRING 变量 *str_time* 转换为时间 *int_time* 的 INTEGER 表示形式。 然后使用 SET_TIME 将 KAREL 系统内的时间设置为 *int_time* 指定的时间。

SET_TIME 内置程序

```
WRITE('Enter the new time : ')
READ(str_time)
CNV_STR_TIME(str_time,int_time)
SET_TIME(int_time)
```

A.20.34 SET_TPE_CMT 内置程序

用途：为KAREL程序提供设置与示教程序中指定位置关联的注释的功能。

语法：SET_TPE_CMT(open_id, pos_no, comment, status)

Input/Output 参数：

[in] open_id :INTEGER

[in] pos_no :INTEGER

[in] comment :STRING

[out] status :INTEGER

%ENVIRONMENT Group :TPE

细节：

- *open_id* 指定从先前调用 OPEN_TPE 返回的 *open_id*。 必须在 OPEN_TPE 调用中使用 TPE_RWACC 的开放模式。
- *pos_id* 指定示教程序中用于获取注释的位置编号。 必须记录指定的位置。
- *comment* 是与指定位置关联的注释。 零长度字符串可用于确保位置没有注释。 如果字符串超过16个字符，则会被截断并使用，并返回警告错误。
- 如果操作成功，则 *status* 指示零，否则将显示错误代码。

另请参阅：GET_TPE_CMT 和 OPEN_TPE 以获取更多内置过程。

A.20.35 SET_TRNS_TPE 内置程序

用途：在指定示教程序的指定位置存储POSITION值

语法：SET_TRNS_TPE(open_id, position_no, posn, status)

Input/Output 参数：

[in] open_id : INTEGER

[in] position_no : INTEGER

[in] posn : POSITION

[out] status : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *open_id* 指定打开的示教程序。 在调用此内置函数之前必须打开一个程序。
- *position_no* 指定程序中用于存储 *posn* 指定的值的位置。 其他组的数据不会更改。
- 位置数据在 POSITION 表示中设置，没有转换。
- *posn* 是 *position_no* 的组号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

A.20.36 SET_TSK_ATTR 内置程序

用途：设置指定的运行任务属性的值

语法：SET_TSK_ATTR(task_name, attribute, value, status)

Input/Output 参数：

[in] task_name : STRING

[in] attribute : INTEGER

[in] value : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *task_name* 是指定的运行任务的名称。 空白的 *task_name* 将指示调用任务。
- *attribute* 是要设置其值的任务属性。 以下属性有效：

TSK_PRIORITY：优先级，请参阅%PRIORITY以获取值信息

TSK_TIMESLIC：时间片持续时间，有关值信息，请参阅 %TIMESLICE

TSK_NOBUSY：忙灯关闭，请参阅 %NOBUSYLAMP

TSK_NOABORT：忽略中止请求

Pg_np_error：错误没有中止

Pg_np_cmd：命令没有中止

TSK_NOPAUSE：忽略暂停请求

pg_np_error：错误没有暂停

pg_np_end：启用TP时命令没有暂停

pg_np_tpenb：没有暂停

TSK_TRACE：跟踪启用

TSK_TRACELEN：跟踪时存储的最大行数

TSK_TPMOTION：TP动作启用，请参阅 %TPMOTION 以获取值信息

TSK_PAUSESFT：暂停时转移，%NOPAUSESHFT 反转

- *value* 取决于所设置的任务属性。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：GET_TSK_INFO 内置过程

示例：请参阅第15章“多任务”中的示例

A.20.37 SET_TSK_NAME 内置程序

用途：设置指定任务的名称

语法：SET_TSK_NAME(old_name, new_name, status)

Input/Output 参数：

[in] old_name： STRING

[in] new_name : STRING

[out] status : INTEGER

%ENVIRONMENT Group :MULTI

细节：

- *task_name* 是感兴趣的任务的名称。 空白的 *task_name* 将指示调用任务。
- *new_name* 将成为新的任务名称。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：GET_ATTR_PRG 内置过程

示例：请参阅第15章“多任务”中的示例

A.20.38 SET_VAR 内置程序

用途：允许KAREL程序设置指定变量的值

语法：SET_VAR(entry, prog_name, var_name, value, status)

Input/Output 参数：

[in,out] entry : INTEGER

[in] prog_name : STRING

[in] var_name : STRING

[in] value : 除PATH之外的任何有效KAREL数据类型

[out] status : INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- *entry* 返回 *var_name* 所在的设备目录中 *var_name* 的变量数据表中的条目号。 不应修改此变量。
- *prog_name* 指定包含指定变量的程序的名称。 如果 *prog_name* 为''，则默认为当前正在执行的任务名称。 *prog_name* 还可以访问环中机器人的系统变量。

- 使用'* SYSTEM *'的 *prog_name* 设置系统变量。
- *var_name* 必须引用静态变量。
- *var_name* 可以包含节点号，字段名和/或下标。
- 如果 *var_name* 和 *value* 都是 ARRAY，则复制的元素数量将等于两个数组中较小者的大小。
- 如果 *var_name* 和 *value* 都是 STRING，则复制的字符数将等于两个字符串中较小者的大小。
- 如果 *var_name* 和 *value* 都是相同类型的 STRUCTURE，则*value* 将是 *var_name* 的精确副本。
- *var_name* 将设置为*value*。
- 如果 *value* 未初始化，则*var_name* 的值将设置为 *uninitialized*，*status* 将设置为 12311.*value* 必须是调用程序中的静态变量。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

> **警告**
>
> 使用SET_VAR修改系统变量可能会导致意外结果。

- 可以在系统变量$ PH_MEMBERS []中找到所有机器人的指定名称。 这还包括有关机器人状态的信息。 环索引是此系统变量的数组索引。 KAREL用户可以通过引用此系统变量中的名称和其他信息而不是显式名称来编写通用程序。

另请参阅：CREATE_VAR，RENAME_VAR内置过程，Internet选项手册，以获取有关在环中访问机器人变量的信息。
示例1：要在环中访问MHROB03机器人上的$ TP_DEFPROG，请参阅访问MHROB03上的$ TP_DEFPROG。

在MHROB03上访问$ TP_DEFPROG

```
SET_VAR(entry, '\\MHROB03\*system*', '$TP_DEFPROG', strvar, status)
```

Example 2: In GET_VAR SET_VAR Example , an array [ipgetset]set_data[x,y] is set on all robots in the ring from all robots in the ring. In this array x is the source robot index and y is the destination robot index:

GET_VAR SET_VAR 例子

```
FOR idx = 1 TO $PH_ROSIP.$NUM_MEMBERS DO
 IF idx = $PH_ROSIP.$MY_INDEX THEN
    -- 这可行，但这个机器人效率低下
    ELSE
  SELECT $PH_MEMBERS[idx].$STATE OF
   CASE (0) : -- Offline
    sstate = ' Offline'
   CASE (1) : -- Online
    sstate = ' Online '
   CASE (2) : -- Synchronized
    sstate = ' Synch  '
     CNV_INT_STR(idx, 1, 10, sidx)
     prog_name = '\\' + $PH_MEMBERS[idx].$NAME + '\ipgetset'
     var_name = 'set_data['+ smy_index + ',' + sidx + ']'
     GET_VAR(entry, prog_name, var_name, set_data[$PH_ROSIP.$MY_INDEX,
          idx], status[idx])
     IF status[idx] = 0 THEN
      IF uninit(set_data[$PH_ROSIP.$MY_INDEX, idx]) THEN
       set_data[$PH_ROSIP.$MY_INDEX, idx] = 0
      ELSE
        set_data[$PH_ROSIP.$MY_INDEX, idx] = set_data[$PH_ROSIP.$MY_INDEX,
          idx] + 1
      ENDIF
       SET_VAR(entry, prog_name, var_name, set_data[$PH_ROSIP.$MY_INDEX, idx],
          status[idx])
     ENDIF
    ENDSELECT
  ENDIF
ENDFOR
```

示例3：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

示例4 GET_VAR 和 SET_VAR 也可用于设置寄存器值。

这适用于程序名为* posreg *和* numreg *的本地机器人。 对于本地机器人，它具有与 GET_POS_REG，GET_REG和SET_REG，SET_POS_REG 内置函数类似的功能。 内置插件仅适用于本地机器人。 您可以使用机器人名称作为程序名称的一部分，通过 GET_VAR 和 SET_VAR 访问环中的机器人。

对于数字寄存器上 SET_VAR 的情况，寄存器类型将设置为KAREL变量的类型。 在执行该代码之后的下面的示例中，数字寄存器20将是整数，并且数字寄存器21将是实数。

如果位置寄存器被锁定并且您尝试设置它，则返回锁定的错误位置寄存器。 请参阅使用 GET_VAR 和 SET_VAR 设置寄存器值。

使用 GET_VAR 和 SET_VAR 设置寄存器值

```
program GETREG
%nolockgroup
VAR
entry: integer
status: integer
int_data: integer
real_data: real
posext_data: xyzwprext


BEGIN
  GET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[10]', int_data, status)
  IF status <> 0 THEN
    GET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[10]', real_data, status)
  ENDIF
  GET_VAR(entry, '\\mhrob01\*posreg*', '$POSREG[1, 10]', posext_data, status)
  SET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[20]', int_data, status)
  SET_VAR(entry, '\\mhrob01\*numreg*', '$NUMREG[21]', real_data, status)
  SET_VAR(entry, '\\mhrob01\*posreg*', '$POSREG[1, 20]', posext_data, status)
end GETREG
```

A.20.39 %SHADOWVARS 译者指令

用途：该指令指定默认情况下所有变量都在 SHADOW 中创建。

语法：%SHADOWVARS

A.20.40 SHORT 数据类型

用途：将变量定义为 SHORT 数据类型

语法：SHORT

细节：

- SHORT，定义为2个字节，范围为（-32768 <= n >= 32766）。 分配给（32767）的SHORT变量被视为未初始化。
- 只允许在数组内或结构内使用SHORT。

- SHORT 可以分配给 BYTE 和 INTEGER，BYTE 和 INTEGER 可以分配给 SHORT。 在执行期间检测到 SHORT 范围之外的指定值，并导致程序中止。

A.20.41 SIGNAL EVENT 动作

用途：发出可能满足条件处理程序或释放等待程序的事件的信号

语法：SIGNAL EVENT[event_no]

其中：

event_no：INTEGER 表达式

细节：

- 您可以使用 SIGNAL EVENT 操作指示已发生用户定义的事件。
- 发出信号并且不记住时发生 *event_no*。 因此，如果 WHEN子 句将事件作为其唯一条件，则将发生关联的操作。
- 如果指定了在发出事件信号时未满足的其他条件，则即使其他条件在另一时间满足，也不会采取操作。
- *event_no* 必须在 -32768 到 32767 的范围内。否则，程序将中止并显示错误。

另请参阅：EVENT 条件

示例：有关详细的程序示例，请参见第B.1节“设置监视数字输出端口”（DOUT_EX.KL）。

A.20.42 SIGNAL EVENT 声明

用途：发出可能满足条件处理程序或释放等待程序的事件的信号

语法：SIGNAL EVENT [event_no]

其中：

event_no：an INTEGER

细节：

- 您可以使用 SIGNAL EVENT 语句指示已发生用户定义的事件。
- 发出信号并且不记住时发生 *event_no*。 因此，如果 WHEN 子句将事件作为其唯一条件，则将发生关联的操作。

- 如果指定了在发出事件信号时未满足的其他条件，则即使其他条件在另一时间满足，也不会采取操作。
- *event_no* 必须在 -32768 到 32767 的范围内。否则，程序将中止并显示错误。

另请参阅：附录E，''语法图''了解更多语法信息，EVENT条件

示例：请参阅DISABLE CONDITION语句示例程序。

A.20.43 SIGNAL SEMAPHORE 动作

用途：将一个添加到指示的信号量的值

语法：SIGNAL SEMAPHORE[semaphore_no]

其中：

semaphore_no：INTEGER 表达式

细节：

- *semaphore_no* 指示的信号量增加 1。
- *semaphore_no* 必须在 1 到控制器上定义的信号量数量的范围内。

另请参见：第15.8节 “任务通信” 以获取更多信息和示例。

A.20.44 SIN 内置函数

用途：返回REAL值，该值是指定角度参数的正弦值

语法：SIN(angle)

函数返回类型：REAL

Input/Output 参数：

[in] angle：REAL

%ENVIRONMENT Group :SYSTEM

细节：

- *angle* 指定以度为单位的角度。

- *angle* 必须在 ±18000 度范围内。 否则，程序将因错误而中止。

示例：有关详细的程序示例，请参见第B.8节 “沿六边形路径生成和移动”（GEN_HEX.KL）。

A.20.45 SQRT 内置函数

用途：返回REAL值，该值是指定REAL参数的正平方根

语法： SQRT(x)

函数返回类型：REAL

Input/Output 参数：

[in] x： REAL

%ENVIRONMENT Group :SYSTEM

细节：

- *x* 不得为负数。 否则，程序将因错误而中止。

示例：以下示例计算表达式的平方根（a*a+b*b），并指示这是三角形的斜边。

SQRT 内置函数

```
c = SQRT(a*a+b*b)
WRITE ('The hypotenuse of the triangle is ',c::6::2)
```

A.20.46 %STACKSIZE 译者指令

用途：用长字指定堆栈大小。

语法： %STACKSIZE = n

细节:

- *n* 是堆栈大小。
- *n* 的默认值为300（1200字节）。

另请参见：第5.1.6节“堆栈使用”，了解有关计算堆栈大小的信息

A.20.47 STD_PTH_NODE 数据类型

用途：定义在 PATH 中使用的数据类型。

语法：STD_PTH_NODE = STRUCTURE

node_pos : POSITION in GROUP[1]

group_data : GROUP_ASSOC in GROUP[1] (不再使用)

common_data : COMMON_ASSOC (不再使用)

ENDSTRUCTURE

细节：

- 如果 PATH 声明中省略了 NODEDATA 子句，则 STD_PTH_NODE 将是默认值。
- PATH 中的每个节点都属于这种类型。

A.20.48 STOP 动作

用途：停止正在进行的任何动作，使其处于可恢复状态

语法：STOP <GROUP[n{,n}]>

细节：

- 任何正在进行的动作都会减速停止。 未完成的动作以及任何未决的动作被组合在一个运动和平面运动中。
- 单个 STOP 动作可能会叠加多个动作。
- 如果 KAREL程序正在等待正在进行的议案的完成，它将继续等待。
- 堆栈运动集可以从堆栈中删除，并使用 RESUME 语句或操作重新启动，或者通过从操作员界面（CRT / KB）发出 RESUME 来重新启动。
- 如果 *group* 子句不存在，则将停止任务所控制的所有组（定义条件时）。
- 如果停止，恢复，取消或保持的动作是与其他组同时或协调动作的一部分，则所有组的动作将被停止，恢复，取消或保持。

- 不能为不同的任务停止运动。

另请参阅：RESUME 声明

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.20.49 STOP 声明

用途：停止正在进行的任何动作，使其处于可恢复状态

语法：STOP <GROUP[n{,n}]>

细节：

- 任何正在进行的动作都会减速停止。 未完成的动作以及任何未决的动作被组合在一个运动和平面运动中。
- 一个STOP语句可能会堆叠多个动作。
- 如果KAREL程序正在等待正在进行的议案的完成，它将继续等待。
- 堆栈运动集可从堆栈中移除，并使用RESUME语句或操作重新启动，或从CRT / KB发出RESUME。
- 如果组条件不存在，则任务所控制的所有组都将停止。
- 如果停止，恢复，取消或保持的动作是与其他组的SIMULTANEOUS或COORDINATED动作的一部分，则所有组的动作将被停止，恢复，取消或保持。
- 运动不能停止执行其他任务。

  另请参阅：RESUME 动作，RESUME 声明，附录E，''语法图，'了解更多语法信息

  示例：如果数字输入为ON，则以下示例停止运动。

STOP 声明

```
IF DIN[2] THEN
  STOP
ENDIF
```

A.20.50 STRING 数据类型

用途：将变量或例程参数定义为STRING数据类型

语法： STRING[length]

其中：

length：INTEGER 常量或文字

细节：

- *length*，字符串的物理长度，表示为 STRING 变量分配空间的最大字符数。
- *length* 必须在1到254范围内，并且必须在 STRING 变量声明中指定。
- 声明 STRING 例程参数时不使用长度值; 任何长度的 STRING 都可以传递给 STRING 参数。
- 尝试将 STRING 分配给长度超过变量物理长度的 STRING 变量会导致 STRING 值在右侧截断到 STRING 变量的物理长度。
- 只能将 STRING 表达式分配给 STRING 变量或作为参数传递给 STRING 参数。
- 函数不能返回 STRING 值。
- 有效的 STRING 运算符是：

  — 关系运算符（ >, > =, =, < >, <, 和< = ）

  — 连接运算符（ ）
- STRING 文字由一系列用单引号（撇号）括起来的ASCII字符组成。 示例在下表中给出。

表 A–22. STRING 文字示例

有效	无效	原因
'123456'	123456	没有引号123456是INTEGER文字

示例：有关详细的程序示例，请参阅以下部分：

第B.2节 "复制路径变量"（CPY_PTH.KL）

第B.3节 "将数据保存到默认设备"（SAVE_VR.KL）

第B.7节 "列出文件和程序以及操作字符串"（LIST_EX.KL）

第B.9节“使用文件和设备内置函数”（FILE_EX.KL）

第B.10节“使用动态显示内置”（DYN_DISP.KL）

第B.12节“从字典文件显示列表”（DCLST_EX.KL）

第B.13节“使用DISCTRL_ALPHA内置”（DCALP_EX.KL）

第B.14节“对复制的示教器程序应用偏移”（CPY_TP.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.20.51 STR_LEN 内置函数

用途：返回指定 STRING 参数的当前长度

语法：STR_LEN(str)

函数返回类型：INTEGER

Input/Output 参数：

[in] str： STRING

%ENVIRONMENT Group :SYSTEM

细节：

- 返回值是当前存储在 *str* 参数中的 STRING 的长度，而不是其声明中指定的 STRING 的最大长度。

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.20.52 STRUCTURE 数据类型

用途：将数据类型定义为用户定义的结构

语法：new_type_name = STRUCTURE

field_name_1: type_name_1

field_name_2: type_name_2

...

ENDSTRUCTURE

细节：

- 用户定义的结构是由组件字段列表组成的数据类型，每个组件字段可以是标准数据类型，也可以是另一个先前定义的用户数据类型。
- 当加载包含用户定义类型变量的程序时，将根据先前创建的定义检查这些类型的定义。如果不存在，则创建它。
- 以下数据类型不允许作为数据结构的一部分：
  — STRUCTURE 定义（允许声明结构的类型）
  — PATH 类型
  — 文件类型
  — 视觉类型
  — 可变长度数组
  — 数据结构本身或直接或间接包含它的任何类型
- 变量不能定义为结构，而是定义为先前定义为结构的数据类型

另请参见：第2.4.2节“用户定义的数据结构”

示例：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

A.20.53 SUB_STR 内置功能

用途：返回指定 STRING 参数的一部分的副本

语法：SUB_STR(src, strt, len)

函数返回类型：STRING

Input/Output 参数：

[in] src：STRING

[in] strt：INTEGER

[in] len：INTEGER

%ENVIRONMENT Group :SYSTEM

细节：

- 函数返回 *src* 的子字符串。
- 子字符串的长度是由 *len* 指定的字符数，它从 *strt* 指定的索引位置开始。
- *strt* 必须是积极的。 否则，程序将中止并显示错误。 如果 *strt* 大于 *src* 的长度，则返回空字符串。
- *len* 必须是积极的。 否则，程序将中止并显示错误。 如果 *len* 大于返回 STRING 的声明长度，则返回的 STRING 将被截断以适合。

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量” (CPY_PTH.KL)

第B.7节“列出文件和程序以及操作字符串” (LIST_EX.KL)

第B.9节“使用文件和设备内置函数” (FILE_EX.KL)

A.21 - T - KAREL 语言描述

A.21.1 TAN 内置函数

用途：返回REAL值，该值是指定REAL参数的正切值

语法： TAN(angle)

函数返回类型：REAL

Input/Output 参数：

[in] angle： REAL

%ENVIRONMENT Group :SYSTEM

细节：

- 角度值必须在 ±18000 度范围内。 否则，程序将因错误而中止。

示例：以下示例使用TAN内置函数指定可变角度的正切值。 切线应等于SIN（角度）除以COS（角度）。

TAN 内置函数

```
WRITE ('enter an angle:')
READ (angle,CR)

ratio = SIN(angle)/COS(angle)
```

```
IF ratio = TAN(angle) THEN
  WRITE ('ratio is correct',CR)
ENDIF
```

A.21.2 %TIMESLICE 译者指令

用途：支持具有相同优先级的任务的循环类型时间切片

语法：%TIMESLICE = n

细节：

- *n* 指定一个切片的任务执行时间（以毫秒为单位）。 默认值为256毫秒。
- 时间片值必须大于0。
- 如果有其他具有相同优先级的任务可以运行，则此值是连续执行任务的最长持续时间。
- 仅当同时执行多个具有相同优先级的KAREL任务时，此功能才有效。
- 可以在任务执行期间通过 SET_TSK_ATTR 内置例程设置时间片持续时间。

A.21.3 %TPMOTION 译者指令

用途：指定示教器打开时启用任务动作

语法：%TPMOTION

细节：

- 可以在任务执行期间通过 SET_TSK_ATTR 内置例程设置此属性。

A.21.4 TRANSLATE 内置程序

用途：将KAREL源文件（.KL文件类型）转换为可以加载到内存中并执行的p代码（.PC文件类型）。

语法：TRANSLATE(file_spec, listing_sw, status)

Input/Output 参数：

[in] file_spec：STRING

[in] listing_sw : BOOLEAN

[out] status : INTEGER

%ENVIRONMENT Group :trans

细节：

- *file_spec* 指定要转换的文件的设备，名称和类型。如果未指定设备，则将使用默认设备。该类型（如果指定）将被忽略，并使用.KL代替。
- *p* 代码文件将在默认设备上创建。应将默认设备设置为RAM磁盘。
- *listing_sw* 指定是否应创建.LS文件。 .LS文件包含源代码行的列表以及可能发生的任何错误。 .LS文件将在默认设备上创建。
- 当 TRANSLATE 内置执行时，KAREL程序将等待。如果KAREL程序暂停，转换将继续直到完成。如果KAREL程序中止，转换也将被中止，并且 .PC 文件将不会被创建。
- 如果KAREL程序必须在转换过程中继续执行，请改为使用 KCL_NO_WAIT 内置。
- *status* 说明尝试操作的状态。如果返回数字0，则转换成功。如果不是，则发生错误。

  一些状态代码如下所示：

  0 转换成功了

  未安装 268 Translator 选项

  35084 文件无法打开或创建.KL文件无法找到或默认设备不是RAM磁盘

  - 1 转换不成功（详情请参阅.LS文件）

示例：以下示例程序将创建，翻译，加载并运行另一个名为 hello 的程序。

TRANSLATE 内置程序

```
OPEN FILE util_file ('RW', 'hello.kl')
  WRITE util_file ('PROGRAM hello', CR)
  WRITE util_file ('%NOLOCKGROUP', CR)
  WRITE util_file ('BEGIN', CR)
  WRITE util_file (' WRITE (''hello world'', CR)', CR)
  WRITE util_file ('END hello', CR)
  CLOSE FILE util_file
  TRANSLATE('hello', TRUE, status)
  IF status = 0 THEN
```

```
    LOAD('hello.pc', 0, status)
  ENDIF
  IF status = 0 THEN
    CALL_PROG('hello', prog_index)
  ENDIF
```

A.21.5 TRUNC 内置功能

用途：通过删除 REAL 参数的小数部分，将指定的 REAL 参数转换为 INTEGER 值

语法： TRUNC(x)

函数返回类型：INTEGER

Input/Output 参数：

[in] x： REAL

%ENVIRONMENT Group :SYStem

细节：

- 返回的值是删除任何小数部分后的 *x* 值。 例如，如果 *x* = 2.3，则删除 .3 并返回值2。
- *x* 必须在 -2147483648 到 +2147483583 的范围内。 否则，程序将中止并显示错误。
- ROUND 和 TRUNC 都可用于将 REAL 表达式转换为 INTEGER 表达式。

另请参见： ROUND内置函数

示例： 以下示例使用TRUNC 内置确定实际的英里 INTEGER 值除以小时数来获得mph。

TRUNC 内置功能

```
WRITE ('enter miles driven, hours used: ')
  READ (miles,hours,CR)
  mph = TRUNC(miles/hours)
  WRITE ('miles per hour=',mph::5)
```

A.22 - U - KAREL 语言描述

A.22.1 UNHOLD 动作

用途：释放 HOLD 动作

语法：UNHOLD <GROUP [n{,n}]>

细节：

- 执行最后一次HOLD时正在进行的任何动作都将恢复。
- 如果动作不被保持，则该动作无效。
- 如果执行 RELEASE 语句，则取消保持的动作。
- 如果组子句不存在，则将恢复所有组或任务具有控制权（当条件定义时）。
- 不能为不同的任务停止运动。

示例：以下示例显示了一个全局条件处理程序，它发出UNHOLD操作以恢复机器人运动时按下 TPIN [1]。

UNHOLD 动作

```
CONDITION[1]:
  WHEN TPIN[1]+ DO
    UNHOLD
ENDCONDITION
```

A.22.2 UNHOLD 声明

用途：释放 HOLD 动作

语法：UNHOLD <GROUP [n{,n}]>

细节：

- 执行最后一次HOLD时正在进行的任何动作都将恢复。
- 如果动作不被保持，该声明无效。
- 如果执行RELEASE语句，则取消保持的动作。
- 如果组子句不存在，则将恢复所有组或任务具有控制权（当条件定义时）。

- 不能为不同的任务停止运动。

另请参阅：附录E，''语法图'，'了解更多语法信息

示例：以下示例启动到PR [1]的移动并保持运动。 如果DIN [1]为ON，则UNHOLD允许程序恢复运动。

UNHOLD 声明

```
move_to_pr1 —— Call TP program to move to PR[1]
HOLD
IF DIN[1] THEN
  UNHOLD
ENDIF
```

A.22.3 UNINIT 内置函数

用途：返回一个BOOLEAN值，指示指定参数是否未初始化

语法：UNINIT(variable)

函数返回类型: BOOLEAN

Input/Output 参数：

[in] variable : 任何 KAREL 变量

%ENVIRONMENT Group :SYSTEM

细节：

- 如果变量未初始化，则返回TRUE值。 否则，返回FALSE。
- 变量可以是除未定义的ARRAY，PATH或结构之外的任何数据类型。

示例：有关详细的程序示例，请参见第B.12节“从字典文件显示列表”（DCLST_EX.KL）。

A.22.4 %UNINITVARS 译者指令

用途：该指令指定默认情况下所有变量都是未初始化的。

语法：%UNINITVARS

A.22.5 UNLOCK_GROUP 内置程序

用途：解锁指定轴组的运动控制

语法：UNLOCK_GROUP(group_mask, status)

Input/Output 参数：

[in] group_mask :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :MULTI

细节：

- group_mask 指定要为正在运行的任务解锁的 group 轴组。 group 编号必须在1到控制器上定义的组总数的范围内。

表 A–23. Group_mask 设置

GROUP	DECIMAL	BIT
Group 1	1	1
Group 2	2	2
Group 3	4	3

要指定多个组，请选择表A-23中所示的十进制值，这些值对应于所需的组。 然后使用OR运算符将它们连接在一起。 例如，要指定组1,3，请输入“1 OR 4”。

- 解锁组表示任务已完成与运动组完成。
- 当任务完成执行（或中止）时，程序锁定的所有运动组将自动解锁。
- 如果在调用 UNLOCK_GROUP 内置时运动正在执行或挂起, 则状态设置为 17039, "执行运动存在"。
- 如果在调用 UNLOCK_GROUP 内置时停止运动, 则状态设置为 17040, "存在停止的运动".
- 如果在具有 %NOLOCKGROUP 指令的程序中遇到动作语句，则任务将尝试获取所有必需组的动作控制（如果尚未拥有）。 如果无法获得运动控制，任务将暂停。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

另请参阅：LOCK_GROUP 内置过程，第15章 多任务，以获取更多信息和示例。

示例：以下示例解锁组1,2和3，然后锁定组3。

UNLOCK_GROUP 内置程序

```
PROGRAM lock_grp_ex
%ENVIRONMENT MOTN
%ENVIRONMENT MULTI
  VAR
    status: INTEGER
  BEGIN
    REPEAT
      -- 解锁组1, 2, 3
      UNLOCK_GROUP(1 OR 2 OR 4, status)
      IF status = 17040 THEN
        CNCL_STP_MTN        -- or RESUME
      ENDIF
      DELAY 500
    UNTIL status = 0
    -- 仅锁定组3
    LOCK_GROUP(4, status)
END lock_grp_ex
```

A.22.6 UNPAUSE 动作

用途：恢复程序执行时间，使其足以执行 ROUTINE 操作

语法：UNPAUSE

细节：

- 如果将 ROUTINE 作为操作调用，但程序执行暂停，则仅在 ROUTINE 的持续时间内恢复执行，然后再次暂停。
- 如果调用了多个 ROUTINE，则在再次暂停执行之前将执行所有 ROUTINE。
- UNPAUSE 引起的恢复和暂停不满足任何 RESUME 和 PAUSE 条件。

另请参阅：RESUME，PAUSE 操作

示例：有关详细的程序示例，请参见第B.1节 “设置监视数字输出端口”（DOUT_EX.KL）。

A.22.7 UNPOS 内置程序

用途：将指定的 REAL 变量设置为指定 XYZWPR 变量的位置 (x,y,z) 和方向 (w,p,r) 组件，并将指定的 CONFIG 变量设置为 XYZWPR 的配置组件

语法： UNPOS(posn, x, y, z, w, p, r, c)

Input/Output 参数 :

[in] posn :XYZWPR

[out] x, y, z :REAL

[out] w, p, r :REAL

[out] c :CONFIG

%ENVIRONMENT Group :SYSTEM

细节 :

- *x, y, z, w, p* ,和 *r* 参数设置为 *x* , *y* 和 *z* 位置坐标以及 *posn* 的 yaw, pitch, 和 roll 方向角。
- *c* 返回 *posn* 的配置。

示例：以下示例使用 UNPOS 内置将 *x* 添加 100 到 *x* location 参数。

UNPOS 内置程序

```
UNPOS (CURPOS,x,y,z,w,p,r,config)
next_pos = POS (x+100,y,z,w,p,r,config)
SET_POS_REG(1, next_pos, status) -- Put next_pos in PR[1]
move_to_pr1 -- Call TP program to move to PR[1]
```

A.22.8 USING ... ENDUSING 声明

用途：定义一系列可执行语句，其中可以访问 STRUCTURE 类型变量的字段，而无需重复变量的名称。

语法： USING struct_var{,struct_var} DO

{statement} ENDUSING

其中 :

struct_var：STRUCTURE 类型的变量

statement：可执行的KAREL语句

细节：

- 在可执行语句中，如果同一名称同时是字段名称和变量名称，则使用字段名称。
- 如果相同的字段名称出现在多个变量中，则使用 USING 语句中最右侧的变量。
- 当转换器看到任何字段时，它会从右到左搜索 USING 语句中列出的结构类型变量。

示例：有关详细的程序示例，请参见第B.2节“复制路径变量”（CPY_PTH.KL）。

A.23 - V - KAREL 语言描述

A.23.1 V_CAM_CALIB *i*RVision 内置程序

用途：查找单个平面或多个平面校准的校准网格。

语法：V_CAM_CALIB (cal_name, func_code, status)

[in] cal_name : STRING

[in] func_code : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group: CVIS

细节：

- *cal_name* 是在设置模式下创建的校准设置文件的名称。
- *func_code* 是要查找的平面编号。

  (1) 个用于第一平面或单平面校准和两个

  (2) 用于多平面校准中的第二个平面。 见表A-24。

表 A–24. 功能代码值

校准类型	功能代码
网格图案校准	指定校准平面的索引：1或2。
3DL校准	指定校准平面的索引：1或2。
视觉跟踪校准	不支持
简单的2D校准	不支持

- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 校准文件必须已存在并已设置。 此内置将重新校准已校准的文件。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

例子:

```
------------------------------------------------------------------------
PROGRAM cal
------------------------------------------------------------------------
%NOLOCKGROUP
%ENVIRONMENT CVIS
%ALPHABETIZE
%COMMENT = 'IRVision CAL Built-in'
%NOPAUSE = ERROR + COMMAND + TPENABLE


VAR
  STATUS      : INTEGER
----------------------------------------

BEGIN
 -- 找一个校准平面
  V_CAM_CALIB('CAL1',1,STATUS)
    -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_CAM_CALIB failed - ERROR ', STATUS, CR)
    ABORT
  ENDIF

END cal
```

A.23.2 V_GET_OFFSET *i*RVision 内置程序

用途：从视觉过程获取视觉偏移并将其存储在指定的视觉寄存器中。

语法：V_GET_OFFSET(vp_name, register_no, status)

[in] vp_name: STRING

[in] register_no: INTEGER

[out] status: INTEGER

% ENVIRONMENT GROUP: CVIS

细节：

- *vp_name* 是在设置模式下创建的视觉过程的名称 *register_no* 是放置偏移和视觉数据的寄存器编号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 在 V_RUN_FIND 内置过程之后使用此命令。 如果在执行 V_GET_OFFSET 时尚未完成图像处理，则等待图像处理完成。 V_GET_OFFSET 将工件的视觉偏移存储在视觉寄存器中。 当视觉过程找到多个工件时，应重复调用 V_GET_OFFSET。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

示例程序：

```
------------------------------------------------------------------------
PROGRAM vision
------------------------------------------------------------------------
%NOLOCKGROUP
%ENVIRONMENT cvis
%ALPHABETIZE
%COMMENT = 'IRVision Built-in'
%NOPAUSE = ERROR + COMMAND + TPENABLE


VAR
  STATUS      : INTEGER
  visprocess  : STRING[8]
  int_value   : INTEGER
  real_value  : REAL
```

```
--
----------------------------------------

BEGIN


  -- 从TP程序或MACRO传递的视觉过程的名称

  -- TP示例CALL VISION ('VP1')
  GET_TPE_PRM(1,3,int_value,real_value,visprocess,STATUS)

  -- V_RUN_FIND,拍摄图像并运行视觉过程
  V_RUN_FIND(visprocess, 0, STATUS)

  -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_RUN_FIND FAILED with ERROR CODE ', STATUS, CR)
  ABORT
  ENDIF

  -- 从run_find命令获取第一个偏移量
  -- 将偏移量放入VR [1]
  -- 多次调用V_GET_OFFSET以从多个部分获得偏移

  V_GET_OFFSET(visprocess, 1, STATUS)

  -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_GET_OFFSET FAILED with ERROR CODE ', STATUS, CR)
  ABORT
  ENDIF
  -- 从VR [1]获取所有偏移值，以便将它们放入PR中
  VREG_OFFSET(1,1,status)
END vision
```

A.23.3 V_GET_PASSFL *i*RVision 内置程序

用途：获取防错视觉过程的状态。 然后它将结果存储在指定的数字寄存器中。

语法：V_GET_PASSFL (vp_name, register_no, status)

[in] vp_name : STRING

[in] register_no : INTEGER

[out] status : INTEGER

%ENVIRONMENT GROUP: CVIS

细节 :

- *vp_name* 是在设置模式下创建的防错视觉过程的名称
- *register_no* 是命令将使用错误校对操作状态设置的寄存器编号.-零 - 错误校对操作失败
  — 一次 - 错误校对操作成功
  — 两个 - 错误校对操作未确定，因为父工具失败
- *status* 说明了尝试操作的状态。 如果 *status* 不等于0，则发生错误。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

例子 :

```
-----------------------------------------------------------------------
PROGRAM eproof
-----------------------------------------------------------------------
%NOLOCKGROUP
%ENVIRONMENT CVIS
%ALPHABETIZE
%COMMENT = 'IRVision EP Built-in'
%NOPAUSE = ERROR + COMMAND + TPENABLE


VAR
  STATUS       : INTEGER
----------------------------------------

BEGIN
    -- 运行防错视觉流程'EP1'
  V_RUN_FIND('EP1',0,STATUS)
    -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_RUN_FIND FAILED with ERROR CODE ', STATUS, CR)
    ABORT
  ENDIF

   -- 将防错结果放入R [1]
  V_GET_PASSFL('EP1',1,STATUS)
```

```
    -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_GET_PASSFL FAILED with ERROR CODE ', STATUS, CR)
    ABORT
  ENDIF

END eproof
```

A.23.4 V_GET_QUEUE *i*RVision 内置程序

用途：从指定队列中获取一部分，并将信息存储在视觉寄存器中。

语法：V_GET_QUEUE (queue_num, vr_num, timeout, status)

[in] queue_num: INTEGER

[in] vr_num: INTEGER

[in] timeout: INTEGER

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *queue_num* 是从中获取部件的队列号。
- *vr_num* 是放置偏移的视觉寄存器（VR）的编号。
- 超时是等待超时发生的时间。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 当找到拾取的目标时，此命令将现有编码器的值设置为跟踪触发器。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

A.23.5 V_INIT_QUEUE *i*RVision 内置程序

用途：初始化指定的队列。

语法：V_INIT_QUEUE (queue_num, status)

[in] queue_num: INTEGER

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *queue_num* 是要初始化的队列号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 执行命令时保留在队列中的所有工件信息都被清除。
- 当使用多个机器人在一个传送带上拾取工件时，必须为所有机器人单独执行此命令。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

A.23.6 V_RALC_QUEUE *i*RVision 内置程序

用途：将从队列接收的部件放置或重新分配回队列以供下游机器人使用。

语法：V_RALC_QUEUE (queue_num, vr_num, status)

[in] queue_num: INTEGER

[in] vr_num: INTEGER

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *queue_num* 是重新分配部件的队列号。
- *vr_num* 是要重新分配的偏移量的视觉寄存器（VR）的编号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 此内置需要 iRVision KAREL接口软件选件（J870）。

A.23.7 V_RUN_FIND *i*RVision 内置程序

用途：启动 iRVision 流程。 当指定的视觉处理具有多个摄像机视图时，将对指定的摄像机视图执行位置。

语法：

V_RUN_FIND(vp_name,camera_view,status)

[in] vp_name: STRING

[in] camera_view: INTEGER

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *vp_name* 是在设置模式下创建的视觉过程的名称
- *camera_view* 是摄像机视图的编号。 用于多相机视觉处理。 值-1将运行所有视图。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

示例程序：

```
------------------------------------------------------------------------
PROGRAM vision
------------------------------------------------------------------------
%NOLOCKGROUP
%ENVIRONMENT cvis
%ALPHABETIZE
%COMMENT = 'IRVision Built-in'
%NOPAUSE = ERROR + COMMAND + TPENABLE


VAR
  STATUS      : INTEGER
  visprocess  : STRING[8]
  int_value   : INTEGER
  real_value  : REAL



--
----------------------------------------

BEGIN

  -- 从TP程序或MACRO TP示例CALL VISION ('VP1') 传递的视觉过程的名称
```

```
  GET_TPE_PRM(1,3,int_value,real_value,visprocess,STATUS)

  -- V_RUN_FIND, 拍摄图像并运行视觉过程
  V_RUN_FIND(visprocess, 0, STATUS)

  -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_RUN_FIND FAILED with ERROR CODE ', STATUS, CR)
    ABORT
  ENDIF

  -- V_GET_OFFSET, 从run_find命令获取第一个偏移量
  -- 将偏移量放入VR [1]
  -- 多次调用V_GET_OFFSET以从多个部分获得偏移

  V_GET_OFFSET(visprocess, 1, STATUS)

  -- 如果状态为零则成功
  IF STATUS <> 0 THEN
    WRITE ('V_GET_OFFSET FAILED with ERROR CODE ', STATUS, CR)
  ABORT
  ENDIF
  -- 从VR [1]获取所有偏移值，以便将它们放入PR中
  VREG_OFFSET(1,1,status)
END vision
```

A.23.8 V_SET_REF *i*RVision 内置程序

用途：运行V_RUN_FIND后，在指定的视觉过程中设置参考位置。

语法：

V_SET_REF(vp_name, status)

[in] vp_name: STRING

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *vp_name* 是在设置模式下创建的视觉过程的名称。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。

- 如果在为视觉处理执行 SET_REFERENCE 时，设置PC上的视觉处理仍保持打开状态，则无法将参考位置写入视觉处理过程，从而导致警报。 关闭设置窗口，然后重新执行命令。
- 当视觉过程找到多个工件时，将具有最高分数的工件的位置设置为参考位置。
- 建议在摄像机视图中只放置一个工件，以便将错误的位置设置为参考位置。
- 此内置功能需要 iRVision KAREL接口软件选件（J870）。

A.23.9 V_START_VTRK *i*RVision 内置程序

用途：启动指定的视觉跟踪视觉过程。

语法：V_START_VTRK (vp_name, status)

[in] vp_name: STRING

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *vp_name* 是视觉跟踪视觉过程的名称。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。
- 执行此命令后，iRVision 将监视指定的条件（例如传送带移动距离）。 每次满足指定条件时，iRVision 都会执行指定的视觉处理。 当多个机器人与一个传送带一起使用时，此命令仅在 iRVision 所在的机器人中执行。
- 此内置功能需要 iRVision KAREL接口软件选件（J870）。

A.23.10 V_STOP_VTRK *i*RVision 内置程序

用途：停止指定的视觉跟踪视觉过程。

语法：V_STOP_VTRK (vp_name, status)

[in] vp_name: STRING

[out] status: INTEGER

%ENVIRONMENT GROUP: CVIS

细节：

- *vp_name* 是视觉跟踪视觉过程的名称。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。
- 当多个机器人与一个传送带一起使用时，此命令仅在 iRVision 所在的机器人中执行。
- 此内置功能需要 iRVision KAREL 接口软件选件（J870）。

A.23.11 VAR_INFO 内置程序

用途：允许KAREL程序确定有关内部或外部程序变量的数据类型和数字信息

语法：VAR_INFO(prog_name, var_name, uninit, type_nam, type_value, dims, slen, status)

Input/Output 参数：

[in] prog_name: STRING

[in] var_name: STRING

[out] uninit_b :BOOLEAN

[out] type_nam :STRING

[out] dims:ARRAY[3] OF INTEGER

[out] type_value :INTEGER

[out] status :INTEGER

[out] slen :INTEGER

%ENVIRONMENT Group :BYNAM

细节：

- *prog_name* 指定包含指定变量的程序的名称。 如果 *prog_name* 为空，则默认为当前正在执行的程序。
- *var_name* 必须引用静态程序变量。
- *var_name* 可以包含节点号，字段名和/或下标。
- 如果 *var_name* 指定的变量未初始化，则 *uninit_b* 将返回值TRUE，如果 *var_name* 指定的变量已初始化，则返回FALSE。

- *type_nam* 返回指定 *var_name* 类型名称的 STRING
- *type_value* 返回与 *var_name* 的数据类型对应的 INTEGER。 下表列出了有效的数据类型及其代表性的 INTEGER 值。

表 A–25. 有效数据类型

数据类型	值
POSITION	1
XYZWPR	2
XYZWPREXT	6
INTEGER	16
REAL	17
BOOLEAN	18
VECTOR	19
VIS_PROCESS	21
MODEL	22
SHORT	23
BYTE	24
JOINTPOS1	25
CONFIG	28
FILE	29
PATH	31
CAM_SETUP	32
JOINTPOS2	41
JOINTPOS3	57
JOINTPOS4	73
JOINTPOS5	89
JOINTPOS6	105
JOINTPOS7	121
JOINTPOS8	137
JOINTPOS9	153
JOINTPOS	153
STRING	209
用户定义的类型	210

- *dims* 返回数组的维度（如果有）。 *dims* 数组的大小应为 3。
  — 如果不是数组，则 *dims* [1] = 0
  — 如果不是二维数组，则 *dims* [2] = 0
  — 如果不是三维数组，则 *dims* [3] = 0
- *slen* 返回 *var_name* 指定的变量的声明长度（如果它是STRING变量）。
- *status* 说明尝试操作的状态。 如果不等于0，则发生错误。

示例：以下示例从程序任务中检索有关位于 util_prog 中的变量计数器的信息。

VAR_INFO 内置程序

```
PROGRAM util_prog
  VAR
    counter, i : INTEGER
  BEGIN
    counter = 0
    FOR i = 1 TO 10 DO
      counter = counter + 1
    ENDFOR
  END util_prog
PROGRAM task
  VAR
    uninit_b           : BOOLEAN
    type_name          : STRING[12]
    type_code          : INTEGER
    slen, status       : INTEGER
    alen               : ARRAY[3] OF INTEGER
  BEGIN
    VAR_INFO('util_prog', 'counter', uninit_b, type_name, type_code,
             alen, slen, status)
    WRITE('counter : ', CR)
    WRITE('UNINIT : ', uninit_b, '    TYPE : ', type_name, cr)
  END task
```

A.23.12 VAR_LIST 内置程序

用途：使用指定的名称和数据类型在指定的KAREL程序中查找变量

语法：VAR_LIST(prog_name, var_name, var_type, n_skip, format, ary_nam, n_vars, status)

Input/Output 参数：

[in] prog_name : STRING

[in] var_name : STRING

[in] var_type : INTEGER

[in] n_skip : INTEGER

[in] format : INTEGER

[out] ary_nam : ARRAY of STRING

[out] n_vars : INTEGER

[out] status : INTEGER

%ENVIRONMENT Group :BYNAM

细节 :

- *prog_name* 指定包含指定变量的程序的名称。 可以使用通配符（*）指定 *prog_name*，该字符指定所有已加载的程序。
- *var_name* 是要查找的变量的名称。 可以使用通配符（*）指定 *var_name*，该字符指定可以找到 *prog_name* 的所有变量。
- *var_type* 表示要找到的变量的数据类型。 以下是有效数据类型的列表：

表 A–26. 有效数据类型

数据类型	值
所有变量类型	0
POSITION	1
XYZWPR	2
INTEGER	16
REAL	17
BOOLEAN	18
VECTOR	19
VIS_PROCESS	21
MODEL	22
SHORT	23
BYTE	24

表 A–26. 有效数据类型（续）

数据类型	值
JOINTPOS1	25
CONFIG	28
FILE	29
PATH	31
CAM_SETUP	32
JOINTPOS2	41
JOINTPOS3	57
JOINTPOS4	73
JOINTPOS5	89
JOINTPOS6	105
JOINTPOS7	121
JOINTPOS8	137
JOINTPOS9	153
JOINTPOS	153
STRING	209
用户定义的类型	210

- 当第一次使用 VAR_LIST 时，如果存在的变量多于声明的 ary_nam .Setn_skip 长度为 0，则使用 *n_skip*。 如果 *ary_nam* 完全填充了变量名，请将该数组复制到另一个 STRING 的 ARRAY 并再次执行 VAR_LIST，其中 *n_skip* 等于 *n_vars*。 对 VAR_LIST 的调用将跳过第一遍中找到的变量，并仅查找剩余的变量。
- *format* 指定程序名称和变量名称的格式。 以下值对格式有效：

```
1=prog_name only, no blanks
2 =var_name only, no blanks
3 =[prog_name]var_name, no blanks
4 ='prog_name var_name   ',
总长度= 27个字符, prog_name
以字符1开头, var_name以字符16开头。
```

- *ary_nam* 是 STRING 的 ARRAY，用于存储变量名称。 如果 *ary_nam* 中声明的STRING 长度不足以存储格式化数据，则返回状态并显示错误。

- *n_vars* 是存储在 *ary_name* 中的变量数。
- 如果成功，*status* 将返回零。

另请参见：FILE_LIST，PROG_LIST内置过程

示例：有关详细的程序示例，请参见第B.2节 “复制路径变量”（CPY_PTH.KL）。

A.23.13 VECTOR 数据类型

用途：将变量，函数返回类型或 Routine 参数定义为 VECTOR 数据类型

语法： VECTOR

细节：

- VECTOR 由三个REAL值组成，表示三维笛卡尔坐标中的位置或方向。
- 只能将 VECTOR 表达式分配给 VECTOR 变量，从 VECTOR 函数例程返回，或作为参数传递给 VECTOR 参数。
- 有效的VECTOR运算符是：
  — 加法（+）和减法（ - ）数学运算符
  — Equal（=）和不等于（<>）的关系运算符
  — 交叉产品（ # ）和内部产品（@）运算符。
  — 乘法（*）和除法（÷）运算符
  — 相对位置（:)运算符
- 可以访问或设置 VECTOR 变量的组件字段，就好像它们的定义如下：

VECTOR 数据类型

```
VECTOR = STRUCTURE
 X:  REAL
 Y:  REAL
 Z:  REAL
ENDSTRUCTURE
Note:  All fields are read-write
```

示例：以下示例将 VECTOR 显示为变量声明，作为例程中的参数，以及函数例程返回类型。

VECTOR 数据类型

```
VAR
  direction, offset : VECTOR
ROUTINE calc_offset(offset_vec:VECTOR):VECTOR FROM util_prog
```

A.23.14 VOL_SPACE 内置程序

用途：返回指定设备的总字节数，可用字节数和卷名

语法：VOL_SPACE(device, total, free, volume)

Input/Output 参数：

[in] device :STRING

[out] total :INTEGER

[out] free :INTEGER

[out] volume :STRING

%ENVIRONMENT Group :FLBT

细节：

- 设备可以是：

  RD：RAM磁盘返回所有三个参数，但卷名称为''，因为它不受支持。 必须安装 RAM 磁盘才能进行查询。

  FR：FROM 磁盘返回所有三个参数，但卷名称为''，因为它不受支持。 必须安装 FROM 磁盘才能进行查询。

  FR：Flash ROM 的大小。 这仅设置总参数。

  DRAM：DRAM 的大小。 这仅设置总参数。

  CMOS：CMOS ROM 的大小。 这仅设置总参数。

  TPP：存储示教程序的系统内存区域。

  PERM：永久 CMOS RAM 存储器的区域，存储系统变量和选定的 KAREL 变量。

TEMP：用于加载的KAREL程序，KAREL变量，程序执行信息和系统操作的临时DRAM内存区域。

SYSTEM：存储系统软件和选项的临时DRAM内存区域。 此内存将保存到FROM并在启动时恢复。

注意所有设备名称必须以冒号（:)结尾。

- *total* 是内存的原始大小，以字节为单位。
- *free* 是可用内存量，以字节为单位。
- *volume* 是使用的存储设备的名称。

另请参见：第1.4.1节 “状态显示和指示器” 中的状态存储器，适用于特定应用的FANUC机器人设置和操作手册。

示例：以下示例获取有关不同设备的信息。

VOL_SPACE 内置程序

```
PROGRAM space
%NOLOCKGROUP
%ENVIRONMENT FLBT
VAR
  total:  INTEGER
  free:   INTEGER
  volume: STRING [30]
BEGIN
  VOL_SPACE('rd:', total, free, volume)
  VOL_SPACE('fr:', total, free, volume)
  VOL_SPACE('frp:', total, free, volume)
  VOL_SPACE('fr:', total, free, volume)
  VOL_SPACE('dram:', total, free, volume)
  VOL_SPACE('cmos:', total, free, volume)
  VOL_SPACE('tpp:', total, free, volume)
  VOL_SPACE('perm:', total, free, volume)
  VOL_SPACE('temp:', total, free, volume)
END space
```

A.23.15 VREG_FND_POS *i*RVision 内置程序

用途：使用指定视觉寄存器中找到的位置数据填充指定的位置寄存器。

语法：VREG_FND_POS (visreg_no, camera_view, posreg_no, status)

[in] visreg_no : INTEGER

[in] camera_view : INTEGER

[in] posreg_no : INTEGER

[out] status : INTEGER

%ENVIRONMENT GROUP: CVIS

细节 :

- *visreg_no* 是包含偏移数据的视觉寄存器（VR）编号。
- *camera_view* 是多视图视觉过程的指定摄像机视图。
- *posreg_no* 是要用偏移数据填充的位置寄存器（PR）编号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

A.23.16 VREG_OFFSET *i*RVision 内置程序

用途：使用指定视觉寄存器中的偏移数据填充指定的位置寄存器。

语法：VREG_OFFSET(visreg_no, posreg_no, status)

[in] visreg_no : INTEGER

[in] posreg_no : INTEGER

[out] status : INTEGER

%ENVIRONMENT GROUP: CVIS

细节 :

- *visreg_no* 是包含偏移数据的视觉寄存器（VR）编号
- *posreg_no* 是要用偏移数据填充的位置寄存器（PR）编号。
- *status* 说明了尝试操作的状态。 如果不等于0，则发生错误。
- 此内置需要 iRVision KAREL 接口软件选件（J870）。

A.24 - W - KAREL 语言描述

A.24.1 WAIT FOR 语句

用途：延迟程序执行的继续，直到满足一些条件

语法： WAIT FOR cond_list

其中：

cond_list: 一个或多个条件

细节：

- 必须同时满足单个 WAIT FOR 语句中的所有条件才能继续执行。

另请参见：第6章 条件处理器，附录E“语法图”，了解更多语法信息

示例：请参阅以下各节以获取详细的程序示例：

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.1节“设置数字输出端口进行监测”（DOUT_EX.KL）

A.24.2 WHEN 子句

用途：用于在全局条件处理程序中指定条件/操作对

语法： WHEN cond_list DO action_list

其中：

cond_list：一个或多个条件

action_list：一个或多个条件

细节：

- 对于要触发的条件处理程序，必须同时满足单个 WHEN 子句的 *cond_list* 中的所有条件。
- *action_list* 表示当同时满足 WHEN 子句的相应条件时要采取的动作列表。

- 在 CONDITION 语句中不允许调用函数例程，因此不能在 WHEN 子句中使用。
- CONDITION 语句可以包含多个 WHEN 子句。

另请参见：第6章 条件处理器，附录E“语法图”，了解更多语法信息

示例：请参阅以下各节以获取详细的程序示例：

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.10节“使用动态显示内置插件”（DYN_DISP.KL）

第B.1节“设置数字输出端口进行监测”（DOUT_EX.KL）

A.24.3 WHILE...ENDWHILE 语句

用途：只要 BOOLEAN 表达式保持为 TRUE，即可执行动作

语法：WHILE boolean_exp DO{ statement }ENDWHILE

其中：

boolean_exp：一个 BOOLEAN 表达式

声明：一个有效的 KAREL 可执行语句

细节：

- *boolean_exp* 在每次迭代之前进行评估。
- 只要 *boolean_exp* 为 TRUE，就会执行循环中的语句。
- 如果 *boolean_exp* 为 FALSE，则控制权转移到 ENDWHILE 之后的语句，并且循环体中的语句不执行。

另请参阅：附录E，“语法图”，获取更多语法信息

例如：详细程序示例请参考第B.1节“设置数字输出端口进行监视”（DOUT_EX.KL）。

A.24.4 WITH 子句

用途：用于条件处理程序来指定条件处理程序限定符

语法： WITH param_spec {, param_spec}

其中：

param_spec is of the form : with_sys_var = value

with_sys_var：可在 WITH 子句中使用的系统变量之一

value：与系统变量类型对应的类型的表达式

细节：

- 指定的实际系统变量不会更改。
- $ PRIORITY 和 $ SCAN_TIME 是条件处理程序限定符，只有在 WITH 子句是条件处理程序语句的一部分时才可以在 WITH 子句中使用。

A.24.5 WRITE 语句

用途：将数据写入串行设备或文件

语法： WRITE <file_var> (data_item { ,data_item }) where:

file_var : 一个 FILE 变量

data_item : 一个表达式及其可选的格式说明符或保留字CR

细节：

- 如果在 WRITE 语句中未指定 *file_var*，则使用默认的 TPDISPLAY。 %CRTDEVICE 指令会将默认值更改为 OUTPUT。
- 如果指定了 *file_var*，它必须是其中一个输出设备或与其中一个相等的变量。
- 如果使用UF选项设置 file_var 属性，则会以二进制形式将数据传输到指定的文件或设备。否则，数据以 ASCII文本形式传输。
- *data_item* 可以是任何有效的 KAREL 表达式。
- 如果 *data_item* 是 ARRAY类型，则必须提供下标。
- 如果 *data_item* 是 PATH类型，则可以指定读取整个路径，读取特定节点[n]或读取[n .. m]节点范围。
- 可选的格式说明符可用于控制为每个 *data_item* 写入的数据量。

- 可以用作数据项的保留字CR指定要写入 *file_var* 的下一个数据项将从下一行开始。
- 使用 IO_STATUS Built-In 来确定写入操作是否成功。

另请参阅：PATH 数据类型，有关编写 PATH 变量的更多信息，请参阅第7章文件输入/输出操作，以获取有关格式说明符和 *file_vars* 的更多信息。 附录E， “语法图” ，了解更多的语法信息

示例：有关更详细的程序示例，请参阅附录B “KAREL示例程序” 。

A.24.6 WRITE_DICT 内置程序

用途：从字典中写入信息

语法：WRITE_DICT(file_var, dict_name, element_no, status)

Input/Output 参数：

[in] file_var :FILE

[in] dict_name :STRING

[in] element_no :INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *file_var* 必须打开到字典文本出现的窗口。
- *dict_name* 指定要从中写入的字典的名称。
- *element_no* 指定要写入的元素编号。 这个数字在字典文件中用''$''指定。
- *status* 解释了尝试操作的状态。 如果不等于0，则从字典文件中写入元素时发生错误。

另请参见：READ_DICT，REMOVE_DICT内置过程，第10章字典和表格

示例：有关详细的程序示例，请参见第B.12节 “从字典文件显示列表” （DCLST_EX.KL）。

A.24.7 WRITE_DICT_V 内置程序

用途：使用格式化变量从字典中写入信息

语法：WRITE_DICT_V(file_var, dict_name, element_no, value_array, status)

Input/Output 参数：

[in] file_var :FILE

[in] dict_name :STRING

[in] element_no :INTEGER

[in] value_array :ARRAY OF STRING

[out] status :INTEGER

%ENVIRONMENT Group :UIF

细节：

- *必须将 file_var* 打开到要显示字典文本的窗口。
- *dict_name* 指定要写入的字典的名称。
- *element_no* 指定要写入的元素编号。 这个数字在字典文件中用$指定。
- *value_array* 是一个变量名数组，对应于字典文本中的每个格式化数据项。 每个变量名称可以指定为“[prog_name] var_name”。
  - [prog_name]指定包含指定变量的程序的名称。 如果未指定，则使用当前正在执行的程序。
  - *var_name* 必须引用静态的全局程序变量。
  - *var_name* 可以包含节点号，字段名和/或下标。
- *status* 说明尝试操作的状态。 如果不等于0，则从字典文件中写入元素时发生错误。

另请参阅：READ_DICT_V内置过程，第10章字典和形式

示例：在以下示例中，TPTASKEG.TX包含将显示系统变量的字典文本信息。 此信息是字典中的第一个元素，元素编号为startat0。 util_prog使用WRITE_DICT_V显示示教器上的文本。

WRITE_DICT_V 内置程序

```
-----------------------------------------------
TPTASKEG.TX
```

```
-------------------------------------------
$ "Maximum number of tasks = %d"
-------------------------------------------
UTILITY PROGRAM:
-------------------------------------------
PROGRAM util_prog
  %ENVIRONMENT uif
  VAR
    status: INTEGER
    value_array: ARRAY[1] OF STRING[30]
  BEGIN
    value_array[1] = '[*system*].$scr.$maxnumtask'
    ADD_DICT('TPTASKEG', 'TASK', dp_default, dp_open, status)
    WRITE_DICT_V(TPDISPLAY, 'TASK', 0, value_array, status)
END util_prog
```

A.25 - X - KAREL 语言描述

A.25.1 XML_ADDTAG 内置程序

用途：将标记“tag_name”与“xml_file”关联。

语法：XML_ADDTAG(xml_file, tag_name, numchar, caseflag, tag_ident, status)

Input/Output 参数：

[in] xml_file :FILE

[in] tag_name:STRING

[in] numchar:INTEGER

[in] caseflag:boolean

[in] tag_ident:INTEGER

[out] status :INTEGER

%ENVIRONMENT Group :PBCORE

细节：

- *xml_file* 指定设置了AR_XML属性的打开KAREL文件
- *tag_name* 您想要了解的标记的名称

- *numchar* 指定查找标记时要使用的字符数
- *caseflag* 如果为TRUE，则指定*tag_name*是否区分大小写
- *tag_ident* 用户与标记关联的应用程序标识符有一些用户无法使用的系统标记标识符。这用于允许您打开他的标签和用户返回代码。当扫描遇到注册的标签时，它将返回*tag_ident*。在注册标签之前必须打开文件。 *tag_ident* 应该是应用程序中的唯一编号。这允许应用程序根据返回标识符进行选择。注意：系统会为错误和扫描限制状态返回保留一些标识符。这允许应用程序在select语句中轻松包含这些常量。
- *status* 说明了尝试操作的状态。如果不等于0，则从字典文件中写入元素时发生错误。如果用户尚未打开文件并设置XML属性，则返回状态将为错误。

示例：请参阅第9.5节。

A.25.2 XML_GETDATA 内置程序

用途：返回与导致返回的标记关联的属性名称和值。

语法：xml_getdata(xml_file, numattr, attrnames, attrvalues, text, textdone)

Input/Output 参数：

[in] xml_file :FILE

[out] numattr: INTEGER

[out] attrnames: ARRAY OF STRING

[out] attrvalues: ARRAY OF STRING

[out] textdata: STRING

[out] textdone: BOOLEAN

[out] status: INTEGER

细节：

- *xml_file* 设置了AR_XML属性的打开KAREL文件
- *numattr* 表示属性的数量
- *attrnames* 表示属性名称
- *attrvalues* 表示属性值

- *textdata* 表示标记后面的文本
- *textdone* 如果为FALSE，则要读取更多文本
- *status* 表示操作的结果

A.25.3 XML_REMTAG 内置程序

用途：从列表中删除标记名称。

语法：xml_remtag(xml_file, tag_name, tag_ident, status)

Input/Output 参数：

[in] xml_file :FILE

[in] tag_name: STRING

[OUT] status: INTEGER

细节：

- *xml_file* 打开设置了AR_XML属性的KAREL文件
- *tag_name* 表示要删除的标记的名称
- *status* 表示操作的结果

A.25.4 XML_SCAN 内置程序

用途：扫描先前打开的XML文件

语法：XML_SCAN(xml_file, tag_name, func_code, status)

Input/Output 参数：

[in] xml_file :FILE

[out] tag_name:STRING

[out] tag_ident:INTEGER

[out] func_code::INTEGER

[out] status :INTEGER

细节 :

- *xml_file* 打开设置了AR_XML属性的KAREL文件
- *tag_name* 找到的标记系统的名称
- *tag_ident* 标记在addtag调用中关联的用户
- *func_code* 功能代码，启动等
- 状态操作结果
- 找到注册的标签时
  - Func_code = XML_START 或 XML_END 或 XML_STEND
  - Tag_ident = 注册时与标记关联的值
- 文本缓冲区已满时
  - Func_code = XML_TXCONT
  - Tag_ident =注册时与标记关联的值
- 扫描50行后
  - Status = XML_SCANLIM，只是在遇到时调用内置函数
  - Tag_ident = XML_SCANLIM
  - 这不是错误，并表明还会有更多的错误
- 如果遇到解析错误
  - *status* = 有些错误
  - Tag_ident = XML_ERROR
- 在文件末尾，Status = SUCCESS
- 有效的解析错误是：
  - XML_TAG_SIZE 标记中的字符太多
  - XML_ATTR_SIZE 属性中的字符太多
  - XML_NOSLASH / 字符的使用无效
  - XML_INVTAG 标记中的字符无效
  - XML_UNMATCHATTR 属性没有值
  - XML_UNMATCHTAG 结束标记，没有匹配的开始
  - XML_INVATTR 属性中的字符无效
  - XML_NOFILE 找不到文件
  - XML_TAGNEST 标记嵌套级别太深
  - 注释中的XML_COMMENT错误

- 如果标签不包含起始和结束信息，系统将为标签的开头和标签的末尾提供单独的返回。当为END标签制作时，属性数据无效。
- *tag_ident* 将是用户为注册标签返回的*tag_ident*。 如果系统由于其他原因返回，则*tag_ident*可能包含系统标记数据。

A.25.5 XML_SETVAR 内置程序

用途：根据与导致返回的标记关联的属性设置变量 [prog_name] var_name。

语法：xml_setvar(xml_file, prog_name, var_name, status)

Input/Output 参数：

[in] xml_file :FILE

[in] prog_name:STRING

[in] var_name:STRING

[out] status:INTEGER

细节：

- 在这种情况下，属性的文本将与KAREL变量的文本字段名称匹配。 所以这个类型的变量：

```
xmlstrct_t = STRUCTURE
  first: integer
  second: real
  third: BOOLEAN
  fourth: string[20]
ENDSTRUCTURE
```

可以通过XML设置：

<xmlstrct_t first="123456" second="7.8910" third="1" fourth="A string"\>

- XML标记名称不需要与TYPE名称匹配。 字段名称和属性名称的关联基于对 xml_setvar 的调用中的[program]变量。

A.25.6 XYZWPR 数据类型

用途：将变量，函数返回类型或例程参数定义为XYZWPR数据类型

语法：XYZWPR <IN GROUP [n]>

细节：

- XYZWPR 由三个指定笛卡尔位置（x，y，z）的REAL组件，三个指定方向（w，p，r）的REAL组件和指定CONFIG数据类型的组件组成，总共32个字节。
- 配置字符串表示联合放置和多个转弯，用于描述机器人处于特定位置时的配置。
- 始终相对于特定坐标系引用位置。
- 可以访问或设置XYZWPR变量的组件，就好像它们的定义如下：

XYZWPR 数据类型

```
XYZWPR = STRUCTURE
 X: REAL
 Y: REAL
 Z: REAL
 W: REAL
 P: REAL
 R: REAL
 CONFIG_DATA:  CONFIG
ENDSTRUCTURE
```

注意：所有字段都是读写访问权限。

示例：有关详细的程序示例，请参阅以下部分：

第B.2节“复制路径变量”（CPY_PTH.KL）

第B.5节“使用寄存器内置函数”（REG_EX.KL）

第B.6节“路径变量和条件处理程序”（PTH_MOVE.KL）

第B.8节“沿着六边形路径生成和移动”（GEN_HEX.KL）

第B.1节“设置监视数字输出端口”（DOUT_EX.KL）

A.25.7 XYZWPREXT 数据类型

用途：将变量，函数返回类型或例程参数定义为XYZWPREXT

语法：XYZWPREXT <IN GROUP [n]>

细节：

- XYZWPREXT 由三个指定笛卡尔位置（x，y，z）的REAL组件，三个指定方向（w，p，r）的REAL组件和指定配置字符串的组件组成。 它还包括三个扩展轴，总共44个字节。
- 配置字符串表示联合放置和多个转弯，用于描述机器人处于特定位置时的配置。
- 始终相对于特定坐标系引用位置。
- 可以访问或设置XYZWPREXT变量的组件，就像它们的定义如下：

XYZWPREXT 数据类型

```
XYZWPRext = STRUCTURE
 X: REAL
 Y: REAL
 Z: REAL
 W: REAL
 P: REAL
 R: REAL
 CONFIG_DATA:  CONFIG
 EXT1: REAL
 EXT2: REAL
 EXT3: REAL
ENDSTRUCTURE
--注意：所有字段都是读写访问权限。
```

A.26 - Y - KAREL 赤谷擦释

没有以“Y”开头的KAREL描述。

A.27 - Z - KAREL 赤谷擦释

没有以“Z”开头的KAREL描述。

附录 B

KAREL 示例程序

目录

附录 B	KAREL 示例程序	B–1
B.1	SETTING UP DIGITAL OUTPUT PORTS FOR PROCESS MONITORING	B–6
B.2	COPYING PATH VARIABLES	B–18
B.3	SAVING DATA TO THE DEFAULT DEVICE	B–28
B.4	STANDARD ROUTINES	B–31
B.5	USING REGISTER BUILT-INS	B–33
B.6	PATH VARIABLES AND CONDITION HANDLERS PROGRAM	B–38
B.7	LISTING FILES AND PROGRAMS AND MANIPULATING STRINGS	B–44
B.8	GENERATING AND MOVING ALONG A HEXAGON PATH	B–49
B.9	USING THE FILE AND DEVICE BUILT-INS	B–54
B.10	USING DYNAMIC DISPLAY BUILT-INS	B–58
B.11	MANIPULATING VALUES OF DYNAMICALLY DISPLAYED VARIABLES	B–68
B.12	DISPLAYING A LIST FROM A DICTIONARY FILE	B–70
B.12.1	Dictionary Files	B–80
B.13	USING THE DISCTRL_ALPHA BUILT-IN	B–81
B.13.1	Dictionary Files	B–85
B.14	APPLYING OFFSETS TO A COPIED TEACH PENDANT PROGRAM	B–85

This appendix contains some KAREL program examples. These programs are meant to show you how to use the KAREL built-ins and commands described in Appendix A , "KAREL Language Alphabetical Description."

This section includes examples of how to use the KAREL built-ins and commands in a program. Refer to Appendix A , for more detailed information on each of the KAREL built-ins and commands.

Table B–1 lists the programs in this section, their main function, the built-ins used in each program, and the section to refer to for the program listing.

Conventions

Each program in this appendix is divided into five sections.

Section 0 - Lists each element of the KAREL language that is used in the example program.

Section 1 - Contains the program and environment declarations.

Section 2 - Contains the constant, variable, and type declarations.

Section 3 - Contains the routine declarations.

Section 4 - Contains the main body of the program.

Table B–1. KAREL Example Programs

Program Name	Program Function	Built-ins Used	Section to Refer
CPY_PTH.KL	Copies path variables.	APPEND_NODE BY_NAME CALL_PROG CNV_INT_STR COPY_PATH CREATE_VAR CURPOS DELETE_NODE LOAD PATH_LEN PROG_LIST READ_KB SET_CURSOR SET_FILE_ATR SET_VAR SUB_STR VAR_LIST	Section B.2
SAVE_VRS.KL	Saves data to the default device.	DELETE_FILE SAVE	Section B.3

Table B–1. KAREL Example Programs (Cont'd)

Program Name	Program Function	Built-ins Used	Section to Refer
ROUT_EX.KL	Contains standard routines that are used throughout the program examples.	CHR FORCE_SPMENU	Section B.4
REG_EX.KL	Uses Register built-ins.	CALL_PROGLIN CHR CURPOS GET_JPOS_REG GET_POS_REG GET_REG POS_REG_TYP SET_INT_REG SET_JPOS_REG SET_POS_REG FORCE_SPMENU	Section B.5
PTH_MOVE.KL	Teaches and moves along a path. Also uses condition handlers.	CHR CNV_REL_JPOS PATH_LEN SET_CURSOR	Section B.6
LIST_EX.KL	Lists files and programs, and manipulate strings.	ABS ARRAY_LEN CNV_INT_STR FILE_LIST LOAD LOAD_STATUS PROG_LIST ROUND SUB_STR	Section B.7
GEN_HEX.KL	Generates a hexagon, and moves along the path.	CNV_REL_JPOS COS CURPOS SIN	Section B.8
FILE_EX.KL	Uses the File and Device built-ins.	CNV_TIME_STR COPY_FILE DISMOUNT_DEV FORMAT_DEV GET_TIME MOUNT_DEV SUB_STR	Section B.9

Table B–1. KAREL Example Programs (Cont'd)

Program Name	Program Function	Built-ins Used	Section to Refer
DYN_DISP.KL	Uses Dynamic Display built-ins.	ABORT_TASK CNC_DYN_DISB CNC_DYN_DISE CNC_DYN_DISP CNC_DYN_DISS CNC_DYN_DISI CNC_DYN_DISR INI_DYN_DISB INI_DYN_DISE INI_DYN_DISP INI_DYN_DISS INI_DYN_DISI INI_DYN_DISR LOAD LOAD_STATUS RUN_TASK	Section B.10
CHG_DATA.KL	Processes and changes values of dynamically displayed variables.		Section B.11
DCLST_EX.KL	Displays a list from a dictionary file.	ADD_DICT ACT_SCREEN ATT_WINDOW_S CHECK_DICT CLR_IO_STAT CNV_STR_INT DEF_SCREEN DISCTRL_LIST FORCE_SPENU IO_STATUS ORD READ_DICT REMOVE_DICT SET_FILE_ATR SET_WINDOW STR_LEN UNINIT WRITE_DICT	Section B.12.1
DCLISTEG.UTX	Dictionary file.	N/A	Section B.12

Table B–1. KAREL Example Programs (Cont'd)

Program Name	Program Function	Built-ins Used	Section to Refer
DCALP_EX.KL	Uses the DISCTRL_ALPHA Built-in.	ADD_DICT CHR DISCTRL_ALPH FORCE_SPEMU POST_ERR SET_CURSOR SET_LANG	Section B.13
DCALPHEG.UTX	Dictionary file.	N/A	Section B.13.1
CPY_TP.KL	Applies offsets to copied teach pendant pro grams.	AVL_POS_NUM CHR CLOSE_TPE CNV_JPOS_REL CNV_REL_JPOS COPY_TPE GET_JPOS_TYP GET_POS_TPE GET_POS_TYP OPEN TPE PROG_LIST SELECT_TPE SET_JPOS_TPE SET_POS_TPE	Section B.14

Table B–1. KAREL Example Programs (Cont'd)

Program Name	Program Function	Built-ins Used	Section to Refer
DOUT_EX.KL	Sets up digital output ports for process monitoring. The DOUTs are used to monitor the status of the external equipment and to show the current status of the process. The equipment status DOUTs are simulated, but in practice they are looked up to the actual external equipment as a feedback response. The robot is moved along a path until the external equipment needs servicing, which is triggered by the DOUT values.	CHRPATH_LEN CURPOS DELAY FORCE_SPMENU RESET SET_PORT_ASG SET_PORT_CMT SET_PORT_MOD SET_PORT_SIM	Section B.1

B.1 设置数字输出端口进行过程监控

该程序设置数字输出端口进行过程监控。 DOUT 将监视外部设备状态并显示过程的当前状态。 设备状态 DOUT 是模拟的，但实际上是作为反馈响应连接到实际的外部设备。 机器人沿路径移动，直到外部设备需要维修，这是由 DOUT 值触发的。

设置数字输出端口进行过程监控 - 概述

```
------------------------------------------------------------------------------
----    DOUT_EX.Kl
------------------------------------------------------------------------------
------------------------------------------------------------------------------
----    Section 0:  Detail about DOUT_EX.kl
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:
```

```
----    Action:
----            CONTINUE                        Sec 4-A
----            ENABLE CONDITION                Sec 3-B; 4-C
----            NOMESSAGE                       Sec 4-A
----            RESUME                          Sec 4-C
----            ROUTINE CALL                    Sec 4-A,C
----            SIGNAL EVENT                    Sec 4-C
----            STOP                            Sec 4-C
----            UNPAUSE                         Sec 4-A
----    Clauses:
----            FROM                            Sec 3-A
----            WHEN                            Sec 4-A,C
----    Conditions:
----            ERROR[xxx]                      Sec 4-A
----            EVENT                           Sec 4-C
----            RELATIONAL condition            Sec 4-A
----    Data types:
----            BOOLEAN                         Sec 2
----            INTEGER                         Sec 2
----            PATH                            Sec 2
----            XYZWPREXT                       Sec 2
----            STRING                          Sec 2
----            XYZWPR                          Sec 2
```

设置数字输出端口用于监控示教器程序 - 概述续

```
----    Directives:
----            ALPHABETIZE                     Sec 1
----            COMMENT                         Sec 1
----            CMOSVARS                        Sec 1
----            INCLUDE                         Sec 1
----    Built-in Functions & Procedures:
----            CHR                             Sec 3-E; 4-D
----            CURPOS                          Sec 3-E
----            DELAY                           Sec 3-B,E
----            FORCE_SPMENU                    Sec 3-E; 4-D
----            PATH_LEN                        Sec 4-B; 4-D
----            RESET                           Sec 3-B
----            SET_PORT_ASG                    Sec 3-D
----            SET_PORT_CMT                    Sec 3-D
----            SET_PORT_MOD                    Sec 3-C
----            SET_PORT_SIM                    Sec 4-D
----            SET_POS_REG                     Sec 3-E, 4-D
----            SET_EPOS_REG                    Sec 3-E
----    Statements:
----            ABORT                           Sec 3-D; 4-B,D
```

```
----            ATTACH                              Sec 4-B
----            CONNECT TIMER                       Sec 4-A
----            CONDITION...ENDCONDITON             Sec 4-A,C
----            ENABLE CONDITION                    Sec 3-B,E; 4-A,C
----            FOR...ENDFOR                        Sec 3-D
----            IF...THEN...ENDIF                   Sec 3-B,C,D; 4-B,C,D
----            RELEASE                             Sec 4-B
----            ROUTINE                             Sec 3-A,B,C,D,E,F
----            WAIT FOR                            Sec 3-E
----            WHILE...ENDWHILE                    Sec 4-B
----            WRITE                               Sec 3-B,D,E; 4-B,D
----    Reserve Words:
----            BEGIN                               Sec 3-B,C,D,E; 4
----            CONST                               Sec 2
----            CR                                  Sec 3-B,D,E; 4-B,D
----            END                                 Sec 3-B,C,D,E, 4-D
----            NOT                                 Sec 3-B; 4-C
----            PROGRAM                             Sec 1
----            VAR                                 Sec 2
----    Predefined FILE names:
----            TPFUNC                              Sec 4-D
```

为过程监控设置数字输出端口 - 声明部分

```
-------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
-------------------------------------------------------------------------------
PROGRAM DOUT_EX                 -- Define the program name
%ALPHABETIZE                    -- CReate the variables in alphabetical order
%NOPAUSE  = TPENABLE            -- Do not pause the program if TP is ENABLED.
                                -- during execution.
%COMMENT  = 'PORT/CH DOUT_EX'
%CMOSVARS                       -- Make sure variables are stored in CMOS
%INCLUDE KLIOTYPS
-------------------------------------------------------------------------------
----    Section 2:  Constant and Variable Declarations
-------------------------------------------------------------------------------
CONST
-- Condition Handler Numbers
   CONT_CH    = 2                           -- Continue execution condition
   EQIP_FAIL  = 3                           -- Equipment Failure Condition
   RESTART    = 6                           -- Restart condition Handler
   SERV_DONE  = 4                           -- Servicing Done condition
   UNINIT_CH  = 10                          -- Monitor for uninit error
   WARMED_UP  = 5                           -- Event to notify eqip is ready
-- Process DOUT numbers ( 1 thru 6 are complementary DOUT )
```

```
--                      ( 3 and 4 are simulated DOUT )
   EQIP_READY = 1                          -- Equipment Ready
   EQIP_NOT_RD= 2                          -- Equipment Not Ready
   EQIP_ERROR = 3                          -- Equipment Failed during process
   EQIP_FIXED = 4                          -- Equipment Fixed after failure
   EQIP_ON    = 5                          -- Turn Eqip-1 ON DOUT
   EQIP_OFF   = 6                          -- Turn Eqip-1 OFF DOUT
   NODE_PULSE = 7                          -- Node Pulsing DOUT
   FINISH     = 8                          -- Path Finishing signal DOUT
-- Process Constants
   SUCCESS    = 0                          -- Successful Operation Status
   UNASIGNED  = 13007                      -- Unassigned Port Deletion Error
VAR
   cont_timer,
   last_node, node_ind,
   status              :INTEGER            -- Status from builtin calls
   prg_abrt            :BOOLEAN            -- Set when the program is aborted
   pth1                :PATH               -- Process Path
   stop_pos            :XYZWPREXT          -- Process Stop Position
   perch_pos           :XYZWPR             -- Perch Position
   tmp_xyz             :XYZWPR             -- XYZWPR variable for temporary use
   indx                :INTEGER            -- Used a FOR loop counter
   ports_ready         :BOOLEAN            -- Check if ports assigned
   cmt_str             :STRING[10]         -- Comment String
```

设置数字输出端口进行过程监控 - 声明例程

```
-----------------------------------------------------------------------------
----    Section 3:  Routine Declaration
-----------------------------------------------------------------------------
-----------------------------------------------------------------------------
----    Section 3-A:  TP_CLS  Declaration
----                  This routine is from ROUT_EX.kl and will
----                  clear the TP USER menu screen and force
----                  it to be visible.
-----------------------------------------------------------------------------
ROUTINE tp_cls FROM rout_ex             -- ROUT_EX must also be loaded.
-----------------------------------------------------------------------------
----    Section 3-B:  port_init  Declaration
----                  This routine assigns a value to ports_ready, which
----                  allows the ports to be initialized.  It resets the
----                  controller so that program execution may be continued
----                  automatically though the CONT_CH condition handler.
-----------------------------------------------------------------------------
ROUTINE init_port
VAR
```

```
        reset_ok: BOOLEAN
BEGIN
  ports_ready = FALSE              -- Set false so ports will be initialized
  DELAY 500;
  RESET(reset_ok)                  -- Reset the controller
  IF (NOT reset_ok) THEN
     WRITE('Reset Failed', CR)
   ENDIF
  cont_timer = 0                   -- Set a timer to continue the process
  ENABLE CONDITION[CONT_CH]        -- Enabled the CONT_CH which continues
                                   -- program execution
END init_port
```

设置数字输出端口进行过程监控 - 声明例程

```
--------------------------------------------------------------------------------
----    Section 3-C:  SET_MODE Declaration
----                  Sets up the mode of IO's.  Depending on the passed
----                  parameter the IO ports will be set to REVERSE
----                  and/or COMPLEMENTARY mode.  When the ports are set
----                  to REVERSE mode, the TRUE condition is represented by
----                  a FALSE signal.  When COMPLEMENTARY mode is selected
----                  for a port (odd number port), the  port n and n+1 are
----                  complementary signal of each other.
--------------------------------------------------------------------------------
ROUTINE set_mode(port_type:     INTEGER;
                 port_no:       INTEGER;
                 reverse:       BOOLEAN;
                 complmnt:      BOOLEAN)
VAR
    mode:           INTEGER
BEGIN  -- set_mode
 IF reverse THEN
    mode = 1                             -- Set the reverse mode
 ELSE
    mode = 0
 ENDIF
 IF complmnt THEN
    mode = mode OR 2                     -- Set complementary mode
 ENDIF
 SET_PORT_MOD(port_type, port_no, mode, status)
END set_mode
```

设置数字输出端口进行过程监控 - 声明例程

```
--------------------------------------------------------------------------
----    Section 3-D:  SETUP_PORTS Declaration
----          This section assumes that you do not have an AB or GENIUS I/O
----          or any other external I/O board.  Therefore, any previous port
----          assignments are no longer needed for this application, and
----          can be deleted.
--------------------------------------------------------------------------
ROUTINE setup_ports
VAR
    port_n  : INTEGER
BEGIN
-- Delete DIGITAL OUTPUT PORTS 1 thru 48
  FOR port_n = 0 to 5 DO
    -- Indexing of 0 to 5 may not be obvious, But look into the DIGITAL
    -- OUT Configuration screen in TP, you will see the 8 DIGITAL OUTPUT
    -- ports are grouped together in configuration.
    SET_PORT_ASG(IO_DOUT, port_n*8+1, 0, 0, 0, 0, 0, status)
    IF (status <> SUCCESS) AND (status <> UNASIGNED) THEN
      -- Verify that deletion by SET_PORT_ASG was successful
      WRITE ('SET_PORT_ASG built-in  for DOUT (deletion) failed',CR)
      WRITE ('Status = ',status,CR)
    ENDIF
  ENDFOR
    --  Assign the DIGITAL PORTS 1 THRU 48 as memory images.
  FOR port_n = 0 TO 5 DO
    SET_PORT_ASG(IO_DOUT, port_n*8+1, 0, 0, io_mem_boo, port_n*8+1, 8, status)
    IF (status <> 0 ) THEN     -- Verify that SET_PORT_ASG was successful
      WRITE ('SET_PORT_ASG built-in  for DOUT (assignment) failed',CR)
      WRITE ('Status = ',status,CR)
    ENDIF
  ENDFOR
  -- Suppose equipment-1 is turned ON by the DOUT[1] = TRUE signal and
  -- turned OFF by the DOUT[2] = TRUE signal. To avoid both signals being
  -- TRUE or FALSE at the same time, set DOUT[1] to be a complement.
  -- Once the DOUT[1] is set in complementary mode, the DOUT[1] and
  -- DOUT[2] will always show the opposite signal of each other.
  -- Thus avoiding the confusion of turning the equipment OFF and ON
  -- at the same time.
  -- Set port-1, port-3 and port-5 to COMPLEMENTARY mode.
  FOR port_n = 1 to 6 DO
    SET_MODE(io_dout, port_n, TRUE, TRUE)
    IF (status <> SUCCESS) THEN
      WRITE ('SET_PORT_MODE Failed on port ',1,CR)
      WRITE ('With Status = ',status,CR)
    ENDIF
  ENDFOR
```

设置数字输出端口进行过程监控 - 声明例程

```
-- Set appropriate comments for the ports.
  SET_PORT_CMT(IO_DOUT, EQIP_READY, 'Equip-READY ',status)
  IF (status <> 0 ) THEN              -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, EQIP_NOT_RD, 'E - NOT READY',status)
  IF (status <> 0 ) THEN              -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, EQIP_ERROR, 'Equip- ERROR',status)
  IF (status <> 0 ) THEN             -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, EQIP_FIXED, 'Equip- FIXED',status)
  IF (status <> 0 ) THEN              -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, EQIP_ON, 'Equip- ON',status)
  IF (status <> 0 ) THEN             -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, EQIP_OFF, 'Equip- OFF',status)
  IF (status <> 0 ) THEN             -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, NODE_PULSE, 'Pulse @ node',status)
  IF (status <> 0 ) THEN             -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
  SET_PORT_CMT(IO_DOUT, FINISH, 'Finish PATH',status)
  IF (status <> 0 ) THEN             -- Verify SET_PORT_CMT was successful
    WRITE ('SET_PORT_CMT built-in failed',CR)
    WRITE ('Status = ',status,CR)
  ENDIF
```

设置数字输出端口进行过程监控 - 声明例程

```
  TP_CLS   -- clear the teach pendant USER screen

  WRITE ('PORT SETUP IS COMPLETE',CR)
  WRITE ('AT THIS POINT YOU NEED TO COLD START',CR)
  WRITE ('Configuration changes of PORTs will not',CR)
  WRITE ('take effect until after a COLD START.',CR,CR)
  WRITE ('Once the controller is ready after',CR)
  WRITE ('COLD START, re-load this program',CR)
  WRITE ('rerun.',CR)

  ports_ready = TRUE      -- Set the ports_ready variable so re-execution of
                          -- this routine, setup_ports, is not performed.
  -- Aborting program to allow for the cold start.
  ABORT
END setup_ports
-----------------------------------------------------------------------------
----    Section 3-E:  SERVICE_RTN interrupt routine Declaration
----                  This routine waits until the equipment has been
----                  serviced and then moves the robot back to where
----                  it was before servicing.  It then sets the DOUT
----                  to notify that the equipment is ready.
-----------------------------------------------------------------------------
ROUTINE service_rtn
BEGIN
  TP_CLS
-- store the current position, where the process is stopped due to failure
-- so after resuming the process can be started from this point.
  stop_pos = CURPOS(0,0)
-- move the robot to the perch position so the equipment
-- can be worked on safely.
  SET_POS_REG(1, perch_pos, status) —— Put perch_pos in PR[1]
  move_to_pr -- Call TP program to move to PR[1]
  WRITE (chr(139),' PLEASE READ ',chr(143),CR)     --Display in reverse video
  WRITE ('Equipment - 1 failed during',CR)
  WRITE ('processing. Motions have been stopped.',CR)
  WRITE ('Please Fix the equipment then',CR)
  WRITE ('SET DOUT[',EQIP_FIXED,'] = TRUE ',CR)
        --Display the following message in reverse video
  WRITE (chr(139), 'IMPORTANT: Once the DOUT is set, current',CR)
  WRITE ('STOPPED motion will be RESUMED',chr(143),CR)
  WAIT FOR DOUT[EQIP_FIXED]  -- wait until equipment has been fixed
```

设置数字输出端口进行过程监控 - 声明例程

```
  -- Move to the point where the process was stopped
  SET_EPOS_REG(1, stop_pos, status)
```

```
  move_to_pr -- Call TP program to move to PR[1]
  -- Enable the SERVICE-DONE condition handler to resume the process.
  ENABLE CONDITION[SERV_DONE]
  -- Wait a sufficient time to allow equipment to warm up and get ready for
  -- processing after the fix is completed.
  WRITE ('Continuing the process.....',CR)
  DELAY 2000

  --Signal that the equipment is now ready.
  DOUT[EQIP_READY] = TRUE

  -- Force the teach pendant back to the IO screen
  FORCE_SPMENU(tp_panel, SPI_TPDIGIO, 1)
END service_rtn
-----------------------------------------------------------------------------
----    Section 3-F:  Routines frst_nod, mid_nods and end_nod are TP
----    routines for doing moves with Time Before clauses
-----------------------------------------------------------------------------
ROUTINE frst_nod FROM frst_nod      -- frst_nod must also be loaded.
--   1:L PR[1] 100mm/sec CNT100 TB   0.00sec,DO[1:NODE_PULSE]=PULSE,1.0sec  ;
--   2:L PR[1] 100mm/sec CNT100 TB   0.00sec,DO[2:EQUIP_ON]=PULSE,2.0sec    ;
ROUTINE mid_nods FROM mid_nods      -- mid_nods must also be loaded.
--   1:L P[1] 100mm/sec CNT100 TB     0.00sec,DO[1:NODE_PULSE]=PULSE,1.0sec  ;
ROUTINE end_nod FROM end_nod        -- end_nod must also be loaded.
--   1:L P[1] 100mm/sec FINE TB       .20sec,DO[3:FINISH]=ON    ;
--   2:L P[1] 100mm/sec FINE TB       0.00sec,DO[1:NODE_PULSE]=PULSE,1.0sec  ;
```

设置数字输出端口进行过程监控 - 主程序

```
-----------------------------------------------------------------------------
----    Section 4:  Main Program
-----------------------------------------------------------------------------
BEGIN   -- DOUT_EX
-----------------------------------------------------------------------------
----    Section 4-A: Global Condition Handler Declaration
-----------------------------------------------------------------------------
CONDITION[UNINIT_CH]:
 WHEN ERROR[12311] DO                  -- Trap UNINITIALIZATION error
   NOMESSAGE                           -- Supress the error message
   UNPAUSE                             -- UNPAUSE
   init_port                           -- Allow ports to be initialized.
ENDCONDITION
ENABLE CONDITION[UNINIT_CH]
CONNECT TIMER to cont_timer
CONDITION [CONT_CH]:
   WHEN cont_timer > 1000 DO
```

```
    CONTINUE
ENDCONDITION
```

设置数字输出端口进行过程监控 - 主程序

```
-------------------------------------------------------------------------------
----    Section 4-B: Verify PATH variable, pth1, has been taught.
-------------------------------------------------------------------------------
tp_cls                          -- Routine Call; Clears the TP USER menu and
                                -- forces the TP USER menu to be visible.
-- Check the number of nodes in the path
IF PATH_LEN(pth1) = 0 THEN                   -- Path is empty (no nodes)
   WRITE ('You need to teach the path.',CR) -- Display instructions
   WRITE ('before executing this program.',CR)
   WRITE ('Teach the PATH variable pth1', CR, 'and restart the program',CR)
   WRITE ('PROGRAM ABORTED',CR)
   ABORT                            -- ABORT the task. do not continue
                                    -- There are no nodes to move to
ENDIF
-- Set Perch Position
-- This position is used in the service_rtn routine
IF UNINIT(perch_pos) THEN

   WRITE ('PERCH POSITION is not recorded.',cr)
   WRITE ('RELEASing Motion Control to TP.',cr)
   WRITE ('Please Move robot to desired Perch Pos',cr)
   -- Wait until the DEADMAN switch is HELD and
   -- TP is TURNED ON to move robot from TP.
   WHILE ((TPIN[248] = ON) AND (TPIN[247] = ON)) DO
     WRITE TPPROMPT(CHR(128),CHR(137),'Hold Down the DEAD-MAN switch')
     DELAY 500
   ENDWHILE
   -- Release motion control from the KAREL program to the
   -- TP control.  Robot can be moved to desired Perch
   -- position with out disturbing the flow of this KAREL task.
   RELEASE
   WHILE (TPIN[249] = OFF ) DO
     WRITE TPPROMPT(CHR(128),CHR(137),'Turn the TP ON')
     DELAY 1000
   ENDWHILE
   WRITE ('ROBOT is ready to move from TP',cr)
   WRITE ('After moving ROBOT to PERCH position ',cr)
   WRITE ('Turn OFF the TP then RELEASE DEADMAN ',cr)
   WHILE (TPIN[249] = ON ) DO
     WRITE TPPROMPT(CHR(128),CHR(137),'Turn OFF TP, after MOVE is done
     DELAY 10000
```

```
   ENDWHILE
```

设置数字输出端口进行过程监控 - 主程序

```
-- KAREL program execution will not continue passed ATTACH
   -- statement until the TP is turned OFF.
   -- Wait until the TP is TURNED OFF after move from TP is completed.
   WHILE (TPIN[249] = ON ) DO
     DELAY 2000
   ENDWHILE
  -- At this point the robot is positioned to the desired
   -- Perch position. Get the motion
   -- control back from TP and record the perch position.
   ATTACH
   perch_pos = CURPOS(0,0,1)
ENDIF
-------------------------------------------------------------------------------
----Section 4-C: Set up Ports and Declare Process dependant condition
handler
-------------------------------------------------------------------------------
-- Port assignments need to be assigned only once and take effect
-- after the controller is COLD STARTED.
-- The ports_ready variable is used to determine if the ports have
-- already been assigned by this program.
-- Therefore only the first execution of this program will assign the ports
IF  NOT(ports_ready) THEN
   setup_ports
ENDIF
-- Define a condition handler to trap equipment failure.
-- If equipment fails during the process, then the DOUT[EQIP_ERROR] is
-- set to TRUE. Which will stop the motion and require the equipment to be
-- fixed before motion can be resumed.
CONDITION[EQIP_FAIL]:
  WHEN DOUT[EQIP_ERROR]  DO
    STOP
    DOUT[EQIP_FIXED] = FALSE
    DOUT[EQIP_READY] = FALSE
    ENABLE CONDITION[RESTART]
    service_rtn
ENDCONDITION
ENABLE CONDITION[EQIP_FAIL]
-- Define a condition handler to monitor the servicing process.
-- Once Servicing/Fixing of equipment is complete, wait for the equipment
-- to be in READY mode.  When the equipment is READY, signal an event
-- which will restart the process where it left off.  The SERV_DONE
-- condition handler is ENABLED from the SERVICE_RTN interrupt routine.
```

```
CONDITION[SERV_DONE]:
   WHEN DOUT[EQIP_READY] DO
    SIGNAL EVENT[WARMED_UP]
    DOUT[EQIP_ERROR] = FALSE
ENDCONDITION
```

设置数字输出端口进行过程监控 - 主程序

```
-- Define a condition handler to monitor when the warm up is complete, then
-- resume the stopped motion and continue the process.  Also re-enable
-- the EQIP_FAIL condition handler to continue monitoring for equipment
-- failure.
CONDITION[RESTART]:
  WHEN EVENT[WARMED_UP] DO
    RESUME
    ENABLE CONDITION[EQIP_FAIL]
ENDCONDITION
------------------------------------------------------------------------------
----    Section 4-D: Do process manipulation
------------------------------------------------------------------------------
-- Using the PATH_LEN built-in find out the last node of the path
last_node = PATH_LEN(pth1)
-- Setting EQIP_ERROR/EQIP_FIXED number ports to be simulated.
-- This setup does not require cold start, can change the port to be
-- simulated on the fly.
SET_PORT_SIM(io_dout, NODE_PULSE, 1, status)
IF (status <> SUCCESS) THEN
  WRITE ('SET_PORT_SIM Failed on port ',indx,CR)
  WRITE ('With Status = ',status,CR)
ENDIF
SET_PORT_SIM(io_dout, FINISH, 1, status)
IF (status <> SUCCESS) THEN
  WRITE ('SET_PORT_SIM Failed on port ',indx,CR)
  WRITE ('With Status = ',status,CR)
ENDIF
```

设置数字输出端口进行过程监控 - 主程序

```
WRITE (' NOW YOU WILL SEE THE DOUT[',NODE_PULSE,'] PULSE',CR)
WRITE (' as the robot moves through every node.',CR,CR)
WRITE (' To simulate EQUIPMENT failure, change ',CR)
WRITE (' DOUT[',EQIP_ERROR,'] = TRUE. ',CR)
WRITE (' Press ''ENTER'' to Continue',CR)
READ(CR)
-- Change the TP display to the DI/O Screen
```

```
FORCE_SPMENU(tp_panel, SPI_TPDIGIO, 1)
-- Moving along path when equipment is ready.
-- Need to turn on equipment-1 for 1/2 second when robot position
-- is at 1st node.  Pulse the DOUT[NODE_PULSE] for every node
-- Turn on the DOUT[FINISH] about 200 ms before the last node.
IF DOUT[EQIP_READY] THEN
  tmp_xyz = pth1[1] -- Convert path node to XYZWPR
  SET_POS_REG(1, tmp_xyz, status) -- Put position in PR[1]
  frst_nod -- Call TP program to do move
  FOR node_ind = 2 TO (last_node - 1) DO
    tmp_xyz = pth1[node_ind]
    SET_POS_REG(1, tmp_xyz, status)
    mid_nods
  ENDFOR
  tmp_xyz = pth1[last_node]
  SET_POS_REG(1, tmp_xyz, status)
  end_nod
  ELSE
  FORCE_SPMENU(TP_PANEL,SPI_TPUSER,1)
  WRITE (' Equipment is not READY',CR)
  WRITE (' Set equipment to READY MODE',CR)
  WRITE (' before executing this program.',CR)
  WRITE (' SET DOUT[',EQIP_READY,'] = TRUE ',CR)
  ABORT
ENDIF
WRITE TPFUNC    (CHR(128),CHR(137))  -- Home Cursor and Clear to End-of-line
                                     -- This will remove the ABORT displayed
                                     -- above F1.
END DOUT_EX
```

B.2 COPYING PATH VARIABLES

This example shows the different ways of copying and appending PATH variables. The PATH Data Type can be copied from one to another only with hard coded path variable names. However, user defined paths can be copied from one to another. The path variable names can be determined during execution of the program.

Copy Path Variables Program - Overview

```
------------------------------------------------------------------------------
----    Detail about CPY_PTH.Kl
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:    In Section:
----    Action:
----    Clauses:
```

```
----            FROM                        Sec 3-A
----            IN DRAM                     Sec 2
----            WHEN                        Sec 4-A
----    Conditions:
----    Data types:
----            ARRAY OF STRING             Sec 2
----            BOOLEAN                     Sec 2; 3-C
----            FILE                        Sec 2
----            INTEGER                     Sec 2; 3-B,C
----            PATH                        Sec 2
----            STRING                      Sec 2; 3-B
----            STRUCTURE...ENDSTRUCTURE    Sec 2
----            USER DEFINED PATH           Sec 2
----            XYZWPR                      Sec 2
----    Directives:
----            ALPHABETIZE                 Sec 1
----            COMMENT                     Sec 1
----            CMOSVARS                    Sec 1
----            CRTDEVICE                   Sec 1
----            INCLUDE                     Sec 2
```

Copy Path Variables Program - Overview and Declaration Section

```
----    Built-in Functions & Procedures:
----            APPEND_NODE                 Sec 4-D
----            BYNAME                      Sec 4-E
----            CALL_PROG                   Sec 4-B
----            COPY_PATH                   Sec 3-C; 4-D
----            CNV_INT_STR                 Sec 4-E
----            CREATE_VAR                  Sec 4-E
----            CURPOS                      Sec 4-B
----            DELETE_NODE                 Sec 4-C
----            LOAD                        Sec 4-B
----            PATH_LEN                    Sec 4-C,E
----            PROG_LIST                   Sec 4-B
----            READ_KB                     Sec 3-B
----            SET_CURSOR                  Sec 4-E
----            SET_FILE_ATR                Sec 4-A
----            SET_POS_REG                 Sec 4-D
----            SET_VAR                     Sec 4-B
----            SUB_STR                     Sec 4-E
----            VAR_LIST                    Sec 4-E
----    Statements:
----            ABORT                       Sec 4-C,E
----            CLOSE FILE                  Sec 4-E
----            FOR .... ENDFOR             Sec 3-C; 4-C,D,E
```

```
----            IF...THEN...ENDIF           Sec 3-B,C; 4-B,C,D,E
----            OPEN FILE                   Sec 4-A
----            REPEAT...UNTIL              Sec 3-B; 4-E
----            ROUTINE                     Sec 3
----            WRITE                       Sec 3-B,C; 4-A,B,C,E
----            USING...ENDUSING            Sec 4-D
----    Reserve Word:
----            BEGIN                       Sec 3-B,C; 4
----            END                         Sec 3-B,C; 4-E
----            PROGRAM                     Sec 1
----            TYPE                        Sec 2
----            VAR                         Sec 2
----     Predefined File Names:
----           CRTFUNC                      Sec 3-B
----           CRTPROMPT                    Sec 3-B,C
-------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
-------------------------------------------------------------------------------
PROGRAM CPY_PTH
%ALPHABETIZE
%COMMENT = 'COPY PATH'         -- Display information by default to
CRT/KB
%CRTDEVICE
%CMOSVARS                     -- Use CMOS RAM to store all static variables,
                              -- except those specified with IN DRAM
```

Copy Path Variables Program - Declaration Section

```
-------------------------------------------------------------------------------
----    Section 2:  Constant, Variable and Type Declarations
-------------------------------------------------------------------------------
CONST
   SUCCESS       = 0 -- The value returned from all built-ins when successful
TYPE
    node_struc    = STRUCTURE      -- Create a user defined node structure
      posn_dat      :XYZWPR
    ENDSTRUCTURE
    user_path     = PATH nodedata = node_struc  --Create a user defined path
VAR
    pth1,
    pth2,
    pth3
    pth4            :PATH         -- These are system defined PATHs
    upth1,
    upth2,
    upth3,
```

```
    upth4           :user_path    -- These are user defined PATHs

    p1_len,
    p2_len,
    status, node_ind,
    total_node      :INTEGER
    F1_press,
    F2_press        :BOOLEAN
    src_num,
    des_num         :INTEGER
    dummy_str,
    src_var,
    des_var         :STRING[20]
    cur_name        :STRING[12]
    entry           :INTEGER
    var_type        :INTEGER
    mem_loc         :INTEGER
```

Copy Path Variables Program - Storing Variables in Memory

```
-- Store the following variables in DRAM, which is temporary memory
    indx      IN DRAM  :INTEGER
    prog_name IN DRAM  :STRING[10]
    prog_type IN DRAM  :INTEGER
    n_match   IN DRAM  :INTEGER
    n_skip    IN DRAM  :INTEGER
    format    IN DRAM  :INTEGER
    ary_nam   IN DRAM  :ARRAY[5] OF STRING[20]
    prog_indx IN DRAM  :INTEGER
    do_copy   IN DRAM  :BOOLEAN
    crt_kb    IN DRAM  :FILE
%INCLUDE KLEVKMSK     -- system supplied file: definition of KC_FUNC_KEY
%INCLUDE KLEVKEYS     -- system supplied file: definition of KY_F1 & KY_F2
```

Copy Path Variables Program - Monitor User Response

```
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----    Section 3-A:  CRT_CLS Declaration
--------------------------------------------------------------------------------
ROUTINE CRT_CLS FROM rout_ex -- include this routine from the file rout_ex.kl
--------------------------------------------------------------------------------
----  Section 3-B:  YES_NO Declaration
----                LABEL the F1 key as YES and F2 key as NO, ask for user
```

```
----                  confirmation.  These two keys are monitored by global
----                  condition handler,so User response can be trapped.
--------------------------------------------------------------------------------
ROUTINE YES_NO
VAR
 key_press : INTEGER
 str : STRING[1]
 n_chars: INTEGER
 l_status: INTEGER
 BEGIN  ---- YES_NO
  WRITE CRTFUNC   (CHR(128),CHR(137))  --- Clear  Window, Home Cursor
  -- Display YES above F1 & NO above F2 & clear rest of Function window
  WRITE CRTFUNC   (’   YES     NO ’,chr(129))
  F1_press = FALSE
  F2_press = FALSE
  REPEAT -- until user presses either the F1 or F2 key
    -- Read just the function keys of the CRT/KB.
    -- The read will be satisfied only when a function key is pressed.
    READ_KB (crt_kb, str , 0, 0, kc_func_key, -1,
             ’’, n_chars, key_press, l_status)
    -- key_press must be converted from a "raw" CRT character to the teach
    -- pendant equivalent character.g
    key_press = $CRT_KEY_TBL[key_press+1]
    IF (key_press = ky_f1) THEN -- The user pressed F1
      F1_press = true
    ENDIF
    IF (key_press = ky_f2) THEN -- The user pressed F2
      F2_press = true
    ENDIF
  UNTIL ((f1_press = TRUE) OR (F2_press = TRUE))
  WRITE CRTFUNC    (CHR(128),CHR(137))  --- Clear Window, Home Cursor
  WRITE CRTPROMPT  (CHR(128),CHR(137))  --- Clear Window, Home Cursor
END YES_NO
```

Copy Path Variables Program - Copying Path Variables

```
--------------------------------------------------------------------------------
---- Section 3-C: PTH_CPY Declaration
----       Copy one user defined path variable to another user defined path
----       variable. The first parameter is the source path.  The second
----       parameter is the destination path. The path parameters can only
----       be passed using BYNAME and the paths must be user defined
--------------------------------------------------------------------------------
ROUTINE PTH_CPY(src_path: USER_PATH; des_path: USER_PATH)
VAR
    node_indx  :INTEGER
```

```
    do_it       :BOOLEAN
    1_stat      :INTEGER
BEGIN --- pth_cpy
CRT_CLS     -- Clear the CRT/KB USER Menu screen
do_it = true
    WRITE ('Perform copy?',CR)
    yes_no
    do_it = F1_press -- F1_press will be true only if the user selected
YES
    IF (do_it) THEN
     --  Copy the entire path of src_path to des_path
       COPY_PATH (src_path, 0,0, des_path, l_stat)
        IF (l_stat <> 0) THEN
          WRITE ('Error in COPY_PATH', l_stat, CR)
        ELSE
          WRITE ('Path Copy function Completed ',cr)
        ENDIF
    ELSE
      WRITE ('Path Copy function canceled by choice',cr,cr)
    ENDIF
END PTH_CPY
-----------------------------------------------------------------------------
----   Section 3-D:  Routine move_to_pr is a TP
----   routine for doing moves
-----------------------------------------------------------------------------
ROUTINE move_to_pr FROM move_to_pr      -- move_to_pr must also be loaded.
--   1:J PR[1] 100% FINE    ;
```

Copy Path Variables Program - Opens CRT/KB & Sends Data to Default Device

```
-----------------------------------------------------------------------------
----    Section 4:  Main Program
-----------------------------------------------------------------------------
BEGIN --- CPY_PATH
-----------------------------------------------------------------------------
----    Section 4-A: Open CRT KB for reading YES/NO inputs from user
-----------------------------------------------------------------------------
CRT_CLS --  will force the CRT USER menu to be visible & clear the screen
 SET_FILE_ATR(crt_kb, ATR_FIELD) -- Needed so the read is satisfied with one
                                 -- character.
 OPEN FILE crt_kb ('RO', 'KB:crkb') -- Open a file to the CRT/KB
                                    -- Used within the YES_NO routine.
-----------------------------------------------------------------------------
----    Section 4-B: Check if SAVE_VRS.PC is loaded. If loaded then execute
-----------------------------------------------------------------------------
---- First check if the "SAVE_VRS" program is loaded or not.
```

```
prog_name = 'SAVE_VRS' -- Only interested in SAVE_VRS program
prog_type = 6          -- Interested only in PC type files
n_skip = 0             -- First time do not skip any files
format = 1             -- Return just the filename
do_copy = TRUE
WRITE ('Checking Program List',cr)
PROG_LIST(prog_name, prog_type, n_skip, format, ary_nam, n_match, status)
  IF (status <>SUCCESS ) THEN
    IF (status = 7073 ) THEN  ---- Program does not exist error
        --- Program SAVE_VRS is not loaded on the controller.
      WRITE ( 'LOADING ',prog_name, CR)
      LOAD (prog_name+'.PC', 0, status)
      IF (status <> SUCCESS) THEN
        WRITE ('Error loading ', prog_name,cr)
        WRITE CRTPROMPT('Copy paths WITHOUT saving program variables?',CR)
        YES_NO
        do_copy = F1_press -- F1_press is true only if user selected
YES
                           -- Copy without saving variables.
      ENDIF
    ELSE
     -- The program listing failed.
      WRITE ('PROG_LIST built-in failed',cr,' with Status = ',status,cr)
      WRITE CRTPROMPT('Copy paths WITHOUT saving program variables?',CR)
      YES_NO
      do_copy = F1_press -- F1_press is true only if user selected
YES
    ENDIF              -- Copy without saving variables.
  ENDIF
```

Copy Path Variables Program - Checks Path Initialization

```
IF (status = SUCCESS)  THEN
  -- This is one way to set variables within another program without
  -- using the FROM clause in the variable section.
  -- It is very useful if you want to have run-time independent code,
  -- where the program or variable name you are setting is not
  -- known until run-time.
  cur_name = CURR_PROG
   SET_VAR (entry, prog_name, 'del_vr', TRUE, status)
   SET_VAR (entry, prog_name, 'prog_name',cur_name , status)
   SET_VAR (entry, prog_name, 'sav_type', 1, status)
   SET_VAR (entry, prog_name, 'dev', 'FLPY:', status)
   WRITE ('Saving program variables before copy', CR)
   CALL_PROG(prog_name,prog_indx) -- call SAVE_VRS
ENDIF
```

```
--------------------------------------------------------------------------
----    Section 4-C: Check for initialization of PATHs pth1 and pth2.
--------------------------------------------------------------------------
IF (NOT do_copy) THEN
  WRITE ('Program exiting, unable to save variables,',cr)
  WRITE ('before copying path''s content',cr)
  -- NOTICE:
  --  Two single quotes will display as one single quote
  --  so this write statement will appear as :
  --  "before copying path's content"
  ABORT
ENDIF
WRITE ('Checking Variable initialization',cr)
-- Check if the pth variables are initialized.
p1_len = PATH_LEN(pth1)  ; p2_len = PATH_LEN(pth2)
IF ( (p1_len = 0) OR (p2_len = 0) )THEN
  WRITE  ('PTH1 or PTH2 is empty path',cr)
  WRITE  ('Please make sure both paths are taught then restart',cr)
  ABORT  -- Cannot copy uninitialized variables.
ENDIF
-- Check if the pth3 variable is initialized.
IF  (PATH_LEN(pth3) <> 0) THEN
  WRITE ('Deleting nodes from pth3',cr) -- Delete the old path of pth3
  FOR indx = PATH_LEN(pth3) DOWNTO 1 DO
    -- its easy to delete nodes from the end instead of deleting node from
    -- the front end. Since after every deletion the nodes are renumbered.
    DELETE_NODE(pth3, indx, status) -- Delete last node of pth3
    IF status <> SUCCESS THEN
       WRITE ('While Deleting ',indx, ' node',cr)
       WRITE ('DELETE_NODE unsuccessful: Status = ',status,cr)
    ENDIF
  ENDFOR
ENDIF
```

Copy Path Variables Program - Path Initialization

```
--------------------------------------------------------------------------
----    Section 4-D: Add pth1 and pth2 together to create pth3.
----                 Move along pth1 and pth2.
----                 Move backwards through pth3.
--------------------------------------------------------------------------
total_node = p1_len + p2_len     -- Total number of nodes needed for pth3
-- Copy the node data from pth1 to pth3
WRITE ('copying pth1 to pth3',cr)
COPY_PATH (pth1, 0,0, pth3, status)
     IF (status <> 0) THEN
```

```
      WRITE ('ERROR in COPY_PATH', status, CR)
    ENDIF
-- Create the required number of nodes for pth3.
-- We know that pth3 now has PATH_LEN(pth3) nodes.
WRITE ('Appending nodes to pth3',cr)
FOR indx = p1_len+1 TO total_node DO -- Append the correct number of nodes.
    APPEND_NODE(pth3, status)
    IF (status <> 0) THEN
      WRITE ('While Appending ',indx, ' node',cr)
      WRITE ('APPEND_NODE unsuccessful: Status = ',status,cr)
    ENDIF
ENDFOR
-- Append the node data of pth2 to pth3.
WRITE ('Appending pth2 to pth3',cr)
FOR indx = p1_len+1 TO total_node DO
  USING pth2[indx - p1_len] DO
    pth3[indx].node_pos    = node_pos
  ENDUSING
ENDFOR
-- Move along the path pth1 and pth2
WRITE ('Moving Along Path pth1',cr)
FOR node_ind = 1 TO p1_len DO
  tmp_xyz = pth1[node_ind]
  SET_POS_REG(1, tmp_xyz, status)
  move_to_pr -- Call TP program to move to PR[1]
ENDFOR
WRITE ('Moving Along Path pth2',cr)
FOR node_ind = 1 TO p2_len DO
  tmp_xyz = pth2[node_ind]
  SET_POS_REG(1, tmp_xyz, status)
  move_to_pr -- Call TP program to move to PR[1]
ENDFOR
--Copy pth3 in reverse order to pth4
COPY_PATH (pth3, PATH_LEN(pth3), 1, pth4, status)
IF (status <> 0) THEN
  WRITE ('ERROR in COPY_PATH', status, CR)
ENDIF--- Move along pth4 which is a reverse order of pth3.
WRITE ('Moving Along Path pth4',cr)
FOR node_ind = 1 TO PATH_LEN(pth4)  DO
  tmp_xyz = pth4[node_ind]
  SET_POS_REG(1, tmp_xyz, status)
  move_to_pr -- Call TP program to move to PR[1]
ENDFOR
```

Copy Path Variables Program - Copy User Defined Paths

```
--------------------------------------------------------------------------------
----    Section 4-E: Copy User Defined Paths.
----                 Copy one user defined path to another user defined path,
----                 where the user specifies which paths to be copied.
--------------------------------------------------------------------------------
CRT_CLS
SET_CURSOR(OUTPUT,2,10, status)     -- Position cursor nicely on CRT
IF (status <> 0 ) THEN
   WRITE ('SET_CURSOR built-in failed with status = ',status,cr)
ENDIF
-- write message in reverse video and then set back to normal video
WRITE (chr(139),' COPY PATH FUNCTION',chr(143),CR,cr)
WRITE ('Currently you have the following ',cr)
WRITE ('User Defined Paths',cr,cr)
n_skip = 0
var_type = 31                      -- Get listing of only PATH type variables
REPEAT
VAR_LIST ('CPY_PTH', '*',var_type, n_skip, 2, ary_nam, n_match, status)
  FOR indx = 1 TO n_match DO
   IF (SUB_STR (ary_nam[indx], 1, 4) = 'UPTH') THEN -- Verify it's one of
                                                    -- the user defined paths
    WRITE (ary_nam[indx], CR)
   ENDIF
  ENDFOR
  n_skip = n_skip + n_match
UNTIL (n_match < ARRAY_LEN(ary_nam))
Write ('Enter the source path number:')
  READ(src_num);
Write ('Enter the destination path number:')
  READ(des_num);
CNV_INT_STR(src_num,2,0,dummy_str)       -- Convert source number to string
src_var = 'UPTH'+ SUB_STR(dummy_str,2,1) -- SUB_STR will remove the leading
                                         -- blank from dummy_str before
                                         -- concatenating to create the
var_type = 0                             -- source variable name
VAR_LIST ('CPY_PTH', src_var, var_type, 0, 2, ary_nam, n_match, status)
IF (status <> SUCCESS) THEN
  WRITE ('Var_list unsuccessful for src_var: status ', status, cr)
ENDIF
```

Copy Path Variables Program - Copy User Defined Paths Continued

```
-- If the variable does not exist create it.
IF (n_match = 0) THEN
  CREATE_VAR ('', src_var, '', 'USER_PATH', 1, 0, 0, 0, status, mem_loc)
  IF (status <> SUCCESS) THEN
```

```
    WRITE ('Error creating ', src_var, ':', status, cr)
  ENDIF
ENDIF
--Create the destination variable name
CNV_INT_STR(des_num,2,0,dummy_str)      -- Convert des_num to a string
des_var = 'UPTH'+ SUB_STR(dummy_str,2,1)-- The SUB_STR will remove the leading
                                        -- blank from dummy_str before
                                        -- concatenating to create the
                                        -- source variable name
-- Verify that the des_var variable exists.
VAR_LIST ('CPY_PTH', des_var, var_type, 0, 2, ary_nam, n_match, status)
IF (status <> SUCCESS) THEN
  WRITE ('Var_list unsuccessful for des_vr: status', status, cr)
ENDIF
-- If the variable does not exist create it.
IF (n_match = 0) THEN
  CREATE_VAR ('', des_var, '', 'USER_PATH', 1, 0, 0, 0, status, mem_loc)
  IF (status <> SUCCESS) THEN
    WRITE ('Error creating ', des_var, ':', status, cr)
  ENDIF
ENDIF
-- Copy the specified source path to the specified destination path
pth_cpy(BYNAME('', src_var, indx), BYNAME('', des_var, indx) )
-- Close file before quitting
CLOSE FILE crt_kb
WRITE ('CPY_PTH example completed',cr)
END CPY_PTH
```

B.3 SAVING DATA TO THE DEFAULT DEVICE

This program will save variables or teach pendant programs to the default device. If the user specified to overwrite the file then the file will be deleted before performing the save.

Note This program is called by the CPY_PTH.KL program. Refer to Section B.2 , for information on CPY_PTH.KL.

Saving Data Program - Overview

```
------------------------------------------------------------------------------
----    SAVE_VRS.KL
------------------------------------------------------------------------------
----    Section 0:  Detail about SAVE_VRS.KL
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:    In Section:
----    Actions:
```

```
----    Clauses:
----    Conditions:
----    Data types:
----            BOOLEAN                     Sec 2
----            INTEGER                     Sec 2
----            STRING                      Sec 2
----    Directives:
----            COMMENT                     Sec 1
----            ENVIRONMENT                 Sec 1
----            NOLOCKGROUP                 Sec 1
----    Built-in Functions & Procedures:
----            DELETE_FILE                 Sec 4-B
----            SAVE                        Sec 4-B
----    Statements:
----            IF, THEN, ENDIF             Sec 4-B
----           SELECT, CASE, ENDSELECT      Sec 4-A
----            WRITE                       Sec 4-B
----    Reserve Words:
----            BEGIN                       Sec 4
----            CONST                       Sec 2
----            CR                          Sec 4-B
----            END                         Sec 4-B
----            PROGRAM                     Sec 1
----            VAR                         Sec 2
```

Saving Data Program - Declarations Section

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM SAVE_VRS
%NOLOCKGROUP
%COMMENT = 'Save .vr, .tp, .sv'
%ENVIRONMENT MEMO
%ENVIRONMENT FDEV
--------------------------------------------------------------------------------
----    Section 2:  Constant, Variable and Type Declarations
--------------------------------------------------------------------------------
CONST
  DO_VR  = 1       -- Save variable file(s)
  DO_TP  = 2       -- Save TP program(s)
  DO_SYS = 3       -- Save system variables
  SUCCESS = 0      -- The value expected from all built-in calls.
VAR
  sav_type  : INTEGER     -- Specifies the type of save to perform
  prog_name : STRING[12]  -- The program name to save
```

```
  status    : INTEGER    -- The status returned from the built-in calls
  file_spec : STRING[30]  -- The created file specification for SAVE
  dev       : STRING[5]   -- The device to save to specify whether to
  del_vr    : BOOLEAN   -- delete file_spec before performing the SAVE.
-------------------------------------------------------------------------
----    Section 3:  Routine Declaration
-------------------------------------------------------------------------
```

Saving Data Program - Create File Spec

```
-------------------------------------------------------------------------
----    Section 4:  Main Program
-------------------------------------------------------------------------
BEGIN -- SAVE_VRS
-------------------------------------------------------------------------
----    Section 4-A: Create the file_spec, which contains the device, file
----                 name and type to be saved.
-------------------------------------------------------------------------
  SELECT (sav_type) OF
    CASE (DO_VR):
     -- If prog_name is '*' then all PC variables will be saved with the
     -- correct program name, irregardless of the file name part of
     -- file_spec.
     file_spec = dev+prog_name+'.VR'   -- Create the variable file name
    CASE (DO_TP):
     -- If prog_name is '*' then all TP programs will be saved with the
     -- correct TP program name, irregardless of the prog_name part of
     -- file_spec.
     file_spec = dev+prog_name+'.TP'   -- Create the TP program name
    CASE (DO_SYS):
     prog_name = '*SYSTEM*'
     file_spec = dev+'ALLSYS.SV'       -- All system variables will be
                                       -- saved into this one file.
  ENDSELECT
```

Saving Data Program - Delete/Overwrite

```
-------------------------------------------------------------------------
----    Section 4-B: Decide whether to delete the file before saving
----                 and then perform the SAVE.
-------------------------------------------------------------------------
   -- If the user specified to delete the file before saving, then
   -- delete the file and verify that the delete was successful.
   -- It is possible that the delete will return a status of:
   --  10003 : "file does not exist", for the FLPY: device
```

```
    --    OR
    --  85014 : "file not found", for all RD: and FR: devices
    -- We will disregard these errors since we do not care if the
    -- file did not previously exist.
    IF (del_vr = TRUE)  THEN
      DELETE_FILE (file_spec, FALSE, status) -- Delete the file.
      IF (status <> SUCCESS) AND (status <> 10003) AND
         (status <> 85014) THEN
        WRITE ('Error ', status,' in attempt to delete ',cr, file_spec,cr)
      ENDIF
    ENDIF
    -- If prog_name is specified as an '*' for either .tp or .vr files then
    -- the SAVE builtin will save the appropriate files/programs with the
    -- correct names.
    SAVE (prog_name, file_spec, status) -- Save the variable/program
    IF (status <> SUCCESS) THEN       -- Verify SAVE was successful
      WRITE ('error saving ', file_spec, 'variables', status, cr)
    ENDIF
END  SAVE_VRS
```

B.4 STANDARD ROUTINES

This program is made up of several routines which are used through out the examples. The following is a list of the routines within this file:

- CRT_CLS Clears the CRT/KB USER Menu screen
- TP_CLS Clears the teach pendant USER Menu screen

Standard Routines - Overview

```
    ------------------------------------------------------------------------
    ----    ROUT_EX.KL
    ------------------------------------------------------------------------
    ----    Section 0:  Detail about ROUT_EX.kl
    ------------------------------------------------------------------------
    ---- Elements of KAREL Language Covered:    In Section:
    ----    Actions:
    ----    Clauses:
    ----    Conditions:
    ----    Data types:
    ----    Built-in Functions & Procedures:
    ----            CHR                        Sec 3-A,B
    ----            FORCE_SPMENU               Sec 3-A,B
    ----    Statements:
    ----            ROUTINE                    Sec 3-A,B
```

```
   ----            WRITE                       Sec 3-A,B
   ----
   ----    Reserve Words:
   ----            BEGIN                        Sec 3-A,B; 4
   ----            CR                           Sec 3-B
   ----            END                          Sec 3-A,B; 4
   ----            PROGRAM                      Sec 1
   ----    Predefined File Names:
   ----            CRTERROR                    Sec 3-A
   ----            CRTFUNC                     Sec 3-A
   ----            CRTPROMPT                   Sec 3-A
   ----            CRTSTATUS                   Sec 3-A
   ----            OUTPUT                      Sec 3-A
   ----            TPERROR                     Sec 3-B
   ----            TPFUNC                      Sec 3-B
   ----            TPSTATUS                    Sec 3-B
   ----            TPPROMPT                    Sec 3-B
```

Standard Routines - Declaration Section

```
------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
------------------------------------------------------------------------
PROGRAM ROUT_EX
%NOLOCKGROUP   ---- Don't lock any motion groups
%COMMENT = 'MISC_ROUTINES'
------------------------------------------------------------------------
----    Section 2:  Constant and Variable Declarations
------------------------------------------------------------------------
------------------------------------------------------------------------
----    Section 3:  Routine Declarations
------------------------------------------------------------------------
------------------------------------------------------------------------
----    Section 3-A:  CRT_CLS Declaration
----                  Clear the predefined windows:
----                  CRTPROMPT, CRTSTATUS, CRTFUNC, CRTERROR, OUTPUT
----                  Force Display of the CRT/KB USER SCREEN.
------------------------------------------------------------------------
ROUTINE CRT_CLS
BEGIN  ---- CRT_CLS
--See Chapter 7.9.2 for more information on the PREDEFINED window names
  WRITE CRTERROR   (CHR(128),CHR(137))  -- Clear Window, Home Cursor
  WRITE CRTSTATUS  (CHR(128),CHR(137))  -- Clear Window, Home Cursor
  WRITE CRTPROMPT  (CHR(128),CHR(137))  -- Clear Window, Home Cursor
```

```
  WRITE CRTFUNC    (CHR(128),CHR(137))  -- Clear Window, Home Cursor
  WRITE OUTPUT     (CHR(128),CHR(137))  -- Clear Window, Home Cursor
  FORCE_SPMENU(CRT_PANEL,SPI_TPUSER,1) -- Force the CRT USER Menu
                                       -- to be visible last.  This will
                                       -- avoid the screen from flashing
                                       -- since the screen will be clean
                                       -- when you see it.
END CRT_CLS
```

Standard Routines - Clears Screen and Displays Menu

```
--------------------------------------------------------------------------
----    Section 3-B:   TP_CLS Declaration
                       Clear the predefined windows:
----                   TPERROR, TPSTATUS, TPPROMPT, TPFUNC TPDISPLAY
----                   Force Display of the TP USER Menu SCREEN.
--------------------------------------------------------------------------
ROUTINE TP_CLS
BEGIN
  WRITE (CHR(128),CHR(137)) -- By default this will clear TPDISPLAY
  WRITE TPERROR (CR,'                                          ',CR)
  WRITE TPSTATUS(CR,'                                          ',CR)
  WRITE TPPROMPT(CR,'                                          ',CR)
  WRITE TPFUNC  (CR,'                                          ',CR)
  FORCE_SPMENU(TP_PANEL,SPI_TPUSER,1) -- Force the USER menu screen
     -- to be visible last.
     -- This will avoid the screen from
     -- flashing since the screen will
     -- be clean when you see it.
END TP_CLS
--------------------------------------------------------------------------
----    Section 4:  Main Program
--------------------------------------------------------------------------
BEGIN -- ROUT_EX
END  ROUT_EX
```

B.5 USING REGISTER BUILT-INS

This program demonstrates the use of the REGISTER builtins. REG_EX.KL retrieves the current position and stores it in PR[1]. Then it executes the program PROG_VAL.TP. PROG_VAL will modify the value within the Position Register PR[1].

After PROG_VAL is completed, REG_EX.KL retrieves the PR[1] position. The position is then manipulated and restored in PR[2], and an INTEGER number is stored in R[1]. A different teach

pendant program, PROG_1.TP, is executed which loops through some positions and stores a value to R[2]. The number of loops depends on the value of the R[1] (which was initially set by the KAREL program.)

After PROG_1.TP has completed, the KAREL program gets the value from R[2] and verifies it was the expected value.

The PROG_VAL.TP teach pendant program should look similar to the following.

```
PROG_VAL                                JOINT 10%
1:  !POSITION REG VALUE ;
2:J P[1:ABOVE JOINT] 100% FINE ;
3:  PR[1,2]=600 ;
4:L PR[1] 100.0 Inch/mm FINE ;
5:J P[1:ABOVE JOINT]100% FINE ;
```

The PROG_VAL.TP teach pendant program does the following:

- Moves to position 1 in joint mode.
- Changes the 'y' location of the position in Position Register 1, PR[1] (which was set by the KAREL program).
- Moves to the new PR[1] position.
- Finally moves back to position 1.

The PROG_1.TP teach pendant program should look similar to the following.

```
PROG_1                                  JOINT 10%
1:  LBL[1:START] ;
2:  IF R[1]=0, JMP LBL[2] ;
3:J P[1] 100% FINE ;
4:J P[2] 100% FINE ;
5:  R[1]=R[1]□1 ;
6:  JMP LBL[1] ;
7:  LBL[2:DONE] ;
8:  R[2]=1 ;
```

The PROG_1.TP teach pendant program does the following:

- Checks the value of the R[1].
- If the value of R[1] is not 0, then moves to J P[1] and J P[2] and decrements the value of R[1]. PROG_1.TP continues in this loop until the Register R[1] is zero.

- After the looping is complete, PROG_1.TP stores value 1 in R[2], which will be checked by the KAREL program.

Using Register Built-ins Program - Overview

```
------------------------------------------------------------------------------
----    REG_EX.Kl
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:         In Section:
----    Actions:
----    Clauses:
----    Conditions:
----    Data types:
----                BOOLEAN                       Sec 2
----                JOINTPOS                      Sec 2
----                REAL                          Sec 2
----                XYZWPR                        Sec 2
----    Directives:
----                ALPHABETIZE                   Sec 1
----                COMMENT                       Sec 1
----                NOLOCKGROUP                   Sec 1
----    Built-in Functions & Procedures:
----                CALL_PROGLIN                   Sec 4-A, 4-C
----                CHR                            Sec 4
----                CURPOS                         Sec 4-A
----                FORCE_SPMENU                   Sec 4
----                GET_POS_REG                    Sec 4-B
----                GET_JPOS_REG                   Sec 4-B
----                GET_REG                        Sec 4-C
----                POS_REG_TYP                    Sec 4-B
----                SET_JPOS_REG                   Sec 4-B
----                SET_INT_REG                    Sec 4-B
----                SET_POS_REG                    Sec 4-A
----    Statements:
----                WRITE                          Sec 4, 4-A,B,C
----                IF..THEN..ELSE..ENDIF          Sec 4-A,B,C
----                SELECT...CASE...ENDSELECT      Sec 4-B
----    Reserve Words:
----                BEGIN                          Sec 4
----                CONST                          Sec 2
----                CR                             Sec 4-A,B,C
----                END                            Sec 4-C
----                PROGRAM                        Sec 4
----                VAR                            Sec 2
```

Using Register Built-ins Program - Declaration Section

```
--------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------
PROGRAM reg_ex
%nolockgroup
%comment = 'Reg-Ops'
%alphabetize
--------------------------------------------------------------------------
----    Section 2:  Variable Declaration
--------------------------------------------------------------------------
CONST
    cc_success     = 0         -- Success status
    cc_xyzwpr      = 2     -- Position Register has an XYZWPR
    cc_jntpos      = 9      -- Position Register has a JOINTPOS
VAR
    xyz                 :XYZWPR
    jpos                :JOINTPOS
    r_val               :REAL
    prg_indx,
    i_val,
    pos_type,
    num_axes,
    status              :INTEGER
    r_flg               :BOOLEAN
--------------------------------------------------------------------------
----    Section 3: Routine Declaration
--------------------------------------------------------------------------
--------------------------------------------------------------------------
----    Section 4: Main program
--------------------------------------------------------------------------
BEGIN -- REG_EX
 write(chr(137),chr(128));           -- Clear the TP USER menu screen
 FORCE_SPMENU(TP_PANEL,SPI_TPUSER,1) -- Force the TP USER menu to be
                                     -- visible
```

Using Register Built-ins - Storing and Manipulating Positions

```
--------------------------------------------------------------------------
---- Section 4-A: Store current position in PR[1] and  execute PROG_VAL.TP
--------------------------------------------------------------------------
 WRITE('Getting Current Position',cr)
 xyz = CURPOS(0,0)                    -- Get the current position
 WRITE('Storing Current position to PR[1]',cr)
 SET_POS_REG(1,xyz, status)           -- Store the position in PR[1]
```

```
 IF (status = cc_success) THEN        -- verify SET_POS_REG is successful
   WRITE('Executing "PROG_VAL.TP"',cr)
   CALL_PROGLIN('PROG_VAL',2,prg_indx, FALSE)
                                        --Execute 'PROG_VAL.TP' starting
                                       -- at line 2.  Do not pause on
                                       -- entry of PROG_VAL.
-------------------------------------------------------------------------
---- Section 4-B: Get new position from PR[1]. Manipulate and store in PR[2]
-------------------------------------------------------------------------
  WRITE('Getting Position back from PR[1]',cr)
    -- Decide what type of position is stored in Position Register 1, PR[1]
   POS_REG_TYPE(1, 1, pos_type, num_axes, status)
   IF (status = cc_success) THEN
   -- Get the position back from PR[1], using the correct builtin.
   -- This position was modified in PROG_VAL.TP
     SELECT pos_type OF
       CASE (cc_xyzwpr):
         xyz= GET_POS_REG(1, status)
       CASE (cc_jntpos):
         jpos = GET_JPOS_REG(1, status)
         xyz = jpos
       ELSE:
         write ('The position register set to invalid type', pos_type,CR)
         status = -1                    -- set status so do not continue.
     ENDSELECT
     IF (status = cc_success) THEN     -- Verify GET_POS_REG/GET_JPOS_REG is
                                       -- successful
       xyz.x = xyz.x+10                -- Manipulate the position.
       xyz.z = xyz.z-10
       jpos  = xyz                     -- Convert to a JOINTPOS
       WRITE('Setting New Position to PR[2]',cr)
       SET_JPOS_REG(2,jpos,status)     -- Set the JOINTPOS into PR[2]
       IF (status = cc_success) THEN   -- Verify SET_JPOS_REG is successful
          WRITE('Setting Integer Value to R[1]',cr)
          SET_INT_REG(1, 10, status)   -- Set the value 10 into R[1]
```

Using Register Built-ins - Executing Program and Checking Register

```
-------------------------------------------------------------------------
----    Section 4-C: Execute PROG_1.TP and check the R[2]
-------------------------------------------------------------------------
        IF (status=cc_success) THEN --Verify SET_INT_REG is successful
            WRITE('Executing "PROG_1.TP"',cr)
            CALL_PROGLIN('PROG_1',1, prg_indx, FALSE)
   --Execute PROG_1.TPstarting on first line.
   --Do not pause on entry of PROG_1.
```

```
            WRITE('Getting Value from R[2]',cr)
            GET_REG(2,r_flg, i_val, r_val, status) --Get R[2] value
            IF (status = cc_success) THEN      --Verify GET_REG success
              IF (r_flg) THEN                   --REAL value in register
              WRITE('Got REAL value from R[2]',cr)
                IF (r_val <> 1.0) THEN                  --Verify value set
                 WRITE ('PROG_1 failed to set R[2]',cr)-- by PROG_1_TP
                  WRITE ('PROG_1 failed to set R[2]',cr)
                ENDIF
              ELSE                  --Register contained an INTEGER
                WRITE('Got INTEGER value from R[2]',cr)
                IF (i_val <> 1) THEN     --Verify value set by
  WRITE ('PROG_1 failed to set R[2]',cr) --PROG_1.TP
                ENDIF
             ENDIF
           ELSE                      --GET_REG was NOT successful
             WRITE('GET_REG Failed',cr,' Status = ',status,cr)
           ENDIF
         ELSE                      --SET_INT_REG was NOT successful
           WRITE('SET_INT_REG Failed, Status = ',status,cr)
         ENDIF
       ELSE                        --SET_JPOS_REG was NOT successful
         WRITE('SET_JPOS_REG Failed, Status = ',status,cr)
       ENDIF
     ELSE                          -- GET_POS_REG was NOT Successful
       WRITE('GET_POS_REG Failed, Status = ',status,cr)
     ENDIF
   ELSE
     WRITE ('POS_REG_TYPE Failed, Status =', status, cr)
   ENDIF
 ELSE                              -- SET_POS_REG was NOT successful
   WRITE('SET_POS_REG Failed, Status = ',status,cr)
 ENDIF
 IF (status = cc_success) THEN; WRITE ('Program Completed
Successfully',cr)
 ELSE ;                     WRITE ('Program Aborted due to error',cr)
 ENDIF
END reg_ex
```

B.6 PATH VARIABLES AND CONDITION HANDLERS PROGRAM

This program checks to determine if PATH variables are taught or not. If the paths are taught, the robot moves to a joint position and then loops along a path 5 times.

This example also sets up two global condition handlers.

- The first condition handler detects if the user has pushed a teach pendant key, and if so aborts the program.
- The second condition handler sets a variable when the program is aborted.

Path Variables and Condition Handlers Program - Overview

```
------------------------------------------------------------------------
----    PTH_MOVE.Kl
------------------------------------------------------------------------
------------------------------------------------------------------------
----    Section 0:  Detail about PTH_MOVE.kl
------------------------------------------------------------------------
---- Elements of KAREL Language Covered:       In Section:
----    Actions:
----            ABORT                                Sec 4-A
----    Clauses:
----            WHEN                                 Sec 4-A
----            FROM                                 Sec 3-A
----    Conditions:
----            ABORT                                Sec 4-A
----    Data types:
----            ARRAY OF REAL                        Sec 2
----            BOOLEAN                              Sec 2
----            INTEGER                              Sec 2
----            JOINTPOS6                            Sec 2
----            PATH                                 Sec 2
----            XYZWPR                               Sec 2
----    Directives:
----            ALPHABETIZE                          Sec 1
----            COMMENT                              Sec 1
----            ENVIRONMENT                          Sec 1
----    Built-in Functions & Procedures:
----            PATH_LEN                             Sec 4-C
----            CHR                                  Sec 3-B; 4-B,D
----            CNV_REL_JPOS                         Sec 4-D
----            SET_CURSOR                           Sec 4-B
----            SET_POS_REG                          4-D
```

Path Variables and Condition Handlers Program - Overview Continued

```
----    Statements:
----            Abort                              Sec 4-C
----            CONDITION...ENDCONDITION           Sec 4-A
----            FOR...ENDFOR                       Sec 4-D
```

```
----            ROUTINE                                 Sec 3-A, B, C
----            WAIT FOR                                Sec 3-B
----            WRITE                                   Sec 3-B; 4-B,C,D
----    Reserved Words:
----            BEGIN                                   Sec 3-A,B, 4
----            CONST                                   Sec 2
----            END                                     Sec 3-A,B: 4-D
----            VAR                                     Sec 2
----            PROGRAM                                 Sec 1
----    Predefined File Names:
----            TPFUNC                                  Sec 3-B; 4-D
----            TPDISPLAY                               Sec 4-B
```

Path Variables and Condition Handlers Program - Declaration Section

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM PTH_MOVE         -- Define the program name
%ALPHABETIZE             -- Create the variables in alphabetical order
%COMMENT     = 'PATH MOVES'
%ENVIRONMENT PATHOP      -- Necessary for PATH_LEN
%ENVIRONMENT UIF         -- Necessary for SET_CURSOR
--------------------------------------------------------------------------------
----    Section 2:  Constant and Variable Declarations
--------------------------------------------------------------------------------
CONST
    CH_ABORT   = 1                          -- Number associated with the
                                            -- abort Condition handler
    CH_F1      = 2                          -- Number associated with the
                                            -- F1 key Condition handler
VAR
    status              :INTEGER            -- Status from built-in calls
    node_ind            :INTEGER            -- Index used when moving along path
    loop_pth            :INTEGER            -- Used in a FOR loop counter
    prg_abrt            :BOOLEAN            -- Set when program is aborted
    pth1                :PATH
    strt_jnt            :JOINTPOS6          -- Starting position of a move
    via_pos             :XYZWPR             -- Via point for a circular move
    des_pos             :XYZWPR             -- Destination point
    tmp_xyz             :XYZWPR             -- Temporary XYZWPR
    real_ary            :ARRAY[6] OF REAL   -- This is used for creating
                                            -- a joint position with 6 axes
    index               :INTEGER            -- FOR loop counter
```

Path Variables and Condition Handlers Program - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
----    Section 3-A:  TP_CLS Declaration
----     ROUTINE TP_CLS FROM ROUT_EX        -- ROUT_EX must also be loaded.
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----    Section 3-B:  YES_NO Declaration
---                  Display choices on the function line of the TP.
----                  Asks for user response.
---                  F1 key is monitored by the Global condition handler
----                 [CH_F1] and the F2 is monitored here.
----                 If F1 is pressed the program will abort.
---                  But, if the F2 is pressed the program will continue.
--------------------------------------------------------------------------------
ROUTINE YES_NO
BEGIN
  WRITE TPFUNC (CHR(137))                    -- Home Cursor in Function window
  WRITE TPFUNC ('  ABORT  CONT')             -- Display Function key options
  WAIT FOR TPIN[131]                         -- Wait for user to respond to
                                             -- continue.  If the user presses
                                             -- F1 (abort) condition handler
                                             -- CH_ABORT will abort program.
  WRITE TPFUNC (CHR(137))                    -- Home Cursor in Function window
  WRITE TPFUNC ('  ABORT',chr(129))          -- Redisplay just Abort option and
                                             -- clear rest of Function window
END YES_NO
--------------------------------------------------------------------------------
----   Section 3-C:  Routines move_to_pr and move_circ are TP
----   routines for doing moves
--------------------------------------------------------------------------------
ROUTINE move_to_pr FROM move_to_pr      -- move_to_pr must also be loaded.
--   1:J PR[1] 100% FINE    ;
ROUTINE move_circ FROM move_circ        -- move_circ must also be loaded.
--   1:C PR[1]
--    :  PR[2] 100mm/sec FINE    ;
```

Path Variables and Condition Handlers Program - Declare Condition Handlers

```
--------------------------------------------------------------------------------
----    Section 4:  Main Program
--------------------------------------------------------------------------------
BEGIN   -- PTH_MOVE
--------------------------------------------------------------------------------
```

```
----    Section 4-A: Global Condition Handler Declaration
--------------------------------------------------------------------------------
CONDITION[CH_ABORT]:
  WHEN ABORT DO       -- When the program is aborting set prg_abrt flag.
                      -- This will be triggered if this program aborts itself
                      --  or if an external mechanism aborts this program.
    prg_abrt = TRUE   -- You may then have another task which detects
                      -- prg_abrt being set, and does shutdown operations
                      -- (ie: set DOUT/GOUT's, send signals to a PLC)
ENDCONDITION
CONDITION[CH_F1]:
  WHEN TPIN[129] DO   -- Monitor TP 'F1' Key. If 'F1' key is pressed,
    ABORT             -- abort the program.
ENDCONDITION
prg_abrt = false               -- Initialize variable which is set only if
                               -- the program is aborted and CH_ABORT is
                               -- enabled.
ENABLE CONDITION[CH_ABORT]     -- Start scanning abort condition as defined.
ENABLE CONDITION[CH_F1]        -- Start scanning F1 key condition as defined.
--------------------------------------------------------------------------------
----    Section 4-B: Display banner message and wait for users response
--------------------------------------------------------------------------------
TP_CLS                                -- Routine Call; Clears the TP USER
                                      -- menu, and forces the TP USER menu
                                      -- to be visible.
SET_CURSOR(TPDISPLAY,2,13, status)   -- Set cursor position in TP USER menu
IF (status <> 0 ) THEN               -- Verify that SET_CURSOR was successful
   WRITE ('SET_CURSOR built-in failed with status = ',status,cr)
   YES_NO                            -- Ask whether to quit, due to error.
ENDIF
--- Write heading in REVERSE video, then turn reverse video off
WRITE (chr(139),' PLEASE READ ',chr(143),CR)
WRITE (cr,' *** F1 Key is labelled as ABORT key *** ')
WRITE (cr,' Any time the F1 key is pressed the program')
WRITE (cr,' will abort. However, the F2 key is active ')
WRITE (cr,' only when the function key is labeled.',cr,cr)
YES_NO  -- Wait for user response
```

Path Variables and Condition Handlers Program - Teach and Move Along Path

```
--------------------------------------------------------------------------------
----    Section 4-C: Verify PATH variable, pth1, has been taught
--------------------------------------------------------------------------------
-- Check the number of nodes in the path
IF PATH_LEN(pth1) = 0 THEN                    -- Path is empty (has no nodes)
   WRITE ('You need to teach the path.',cr)  -- Display instructions to user
```

```
   WRITE ('before executing this program.',cr)
   WRITE ('Teach the PATH variable pth1', CR, ' and restart the program',cr)
   ABORT                                    -- Simply ABORT the task
                                            -- do not continue since there
ENDIF                                       -- are no nodes to move to
--------------------------------------------------------------------------
----    Section 4-D: Creating a joint position and moving along paths
--------------------------------------------------------------------------
FOR indx = 1 to 6 DO                        -- Set all joint angles to zero
    real_ary[indx] = 0.0
ENDFOR
real_ary[5] = 90.0                          -- Make sure that the position
                                            -- is not at a singularity point.
CNV_REL_JPOS(real_ary, strt_jnt, status)  -- Convert real_ary values into
                                            -- a joint position, strt_jnt
IF (status <> 0 ) THEN                      -- Converting joint position
                                            -- was NOT successful
   WRITE ('CNV_REL_JPOS built-in failed with status = ',status,cr)
   YES_NO                                   -- Ask user if want to continue.
ELSE                                        -- Converting joint position was
                                            -- successful.
   -- The start position, strt_jnt, has been created and is located at
   -- axes 1-4 = 0.0, axes 5 = 90.0, axes 6 = 0.0.
   via_pos = strt_jnt                       -- Copy the strt_jnt to via_pos
   via_pos.x = via_pos.x +200               -- Add offset to the x location
   via_pos.y = via_pos.y +200               -- Add offset to the y location
   -- The via position, via_pos, has been created to be the same position
   -- as strt_jnt except it has been offset in the x and y locations by
   -- 200 mm.
   des_pos = strt_jnt                       -- Copy the strt_jnt to des_pos
   des_pos.x = des_pos.x + 400              -- Add offset to the x location
  -- The destination position, des_pos, has been created to be the same
  -- position as strt_jnt except it has been offset in the x location by
  -- 400 mm.
```

Path Variables and Condition Handlers Program - Move Along Path

```
   tmp_xyz =  strt_jnt              -- Convert start position to XYZWPR
   SET_POS_REG(1, tmp_xyz, status) -- Put start position in PR[1]
   move_to_pr                       -- Call TP program to move to PR[1]
   tmp_xyz =  des_pos               -- Convert destination position to XYZWPR
   SET_POS_REG(1, tmp_xyz, status) -- Put destination position in PR[1]
   tmp_xyz =  via_pos               -- Convert via position to XYZWPR
   SET_POS_REG(2, tmp_xyz, status) -- Put via position in PR[2]
   WRITE (cr,'Circular move to Destination Position',cr)
   move_circ                         -- Call TP program to
```

```
                                        -- Move robot to destination
                                        -- position using circular motion
                                        -- via the via_pos
ENDIF
--- Execute the same path for 5 times.
FOR loop_pth = 1 TO 5 DO
   WRITE ('Moving along pth1 ',loop_pth::2, ' times',cr)
      -- Display the loop iteration
      -- NOTICE: that "loop_pth::2" will cause 2 blanks to be
      -- displayed after "pth1 '" and before loop_pth.
   FOR node_ind = 1 TO PATH_LEN(pth1) DO
     tmp_xyz =  pth1[node_ind]
     SET_POS_REG(1, tmp_xyz, status)
     move_to_pr
   ENDFOR
ENDFOR
WRITE TPFUNC    (CHR(128),CHR(137))  -- Home Cursor and Clear to
                                     -- End-of-line.  This will remove
                                     -- the ABORT displayed above F1.
WRITE ('pth_move Successfully Completed',cr)
END PTH_MOVE
```

B.7 LISTING FILES AND PROGRAMS AND MANIPULATING STRINGS

This program displays the list of files on the FLPY: device, and lists the programs loaded on the controller. It also shows basic STRING manipulating capabilities, using semi-colons(;) as a statement separator, and nesting IF statements.

Listing Files and Programs and Manipulating Strings - Overview

```
------------------------------------------------------------------------------
----    LIST_EX.Kl
------------------------------------------------------------------------------
----    Section 0:  Detail about LIST_EX.kl
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:    In Section:
----    Actions:
----    Clauses:
----            FROM                       Sec 3-B
----
----    Conditions:
----    Data types:
----            ARRAY OF STRING            Sec 2
----            BOOLEAN                    Sec 2, 3
```

```
----            INTEGER                         Sec 2, 3
----            STRING                          Sec 2
----    Directives:
----            %COMMENT                        Sec 1
----            %NOLOCKGROUP                    Sec 1
----    Built-in Functions & Procedures:
----            ABS                             Sec 4-A
----            ARRAY_LEN                       Sec 4-C&D
----            CNV_INT_STR                     Sec 4-A
----            FILE_LIST                       Sec 4-C
----            LOAD                            Sec 4-B
----            LOAD_STATUS                     Sec 4-B
----            PROG_LIST                       Sec 4-D
----            ROUND                           Sec 4-A
----            SUB_STR                         Sec 4-A
```

Listing Files and Programs and Manipulating Strings - Overview Continued

```
----    Statements:
----           FOR .... ENDFOR                  Sec 3-B
----           IF...THEN...ENDIF                Sec 4-A,B,C,D
----           ROUTINE                          Sec 3-A,B,C
----           REPEAT...UNTIL                   Sec 4-C,D
----           RETURN                           Sec 3-A
----           WRITE                            Sec 3-B; 4-A,B
----    Reserve Words:
----            BEGIN                           Sec 3-A,B; 4
----            CONST                           Sec 2
----            CR                              Sec 3-B; 4-A,B
----            END                             Sec 3-A,B, 4-B
----            PROGRAM                         Sec 1
----            VAR                             Sec 2
----
----    Operators:
----            MOD                             Sec 3-A
----            /                               Sec 3-A
----            *                               Sec 3-A
----    Devices Used:
----            FLPY:                           Sec 4-C
----    Basic Concepts:
----         Semi-colon(;) as statement separator
----         Nested IF..THEN..ELSE..IF..THEN..ELSE..ENDIF..ENDIF structure
----         Concatenation of STRINGS using '+'
----
```

Listing Files and Programs and Manipulating Strings - Declarations Section

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM LIST_EX
%NOLOCKGROUP   ---- Don't lock any motion groups
%COMMENT = 'FILE_LIST'
--------------------------------------------------------------------------------
----    Section 2:  Constant and Variable Declarations
--------------------------------------------------------------------------------
CONST
      INCREMENT   = 13849
      MODULUS     = 65536
      MULTIPLIER  = 25173
VAR
      pr_cases      :STRING[6]  -- psuedo random number converted to string
      prg_nm        :STRING[50] -- Concatenated program name
      loaded        :BOOLEAN    -- Used to see if program is loaded
      initi         :BOOLEAN    -- Used to see if variables initialized
      indx1    :INTEGER   -- FOR loop index
      cases,                    -- Random number returned
      max_number,               -- Maximum random number
      seed     :INTEGER   -- Seed for generating a random number
      file_spec     :STRING[20] -- File specification for FILE_LIST
      n_files       :INTEGER    -- Number of files returned from FILE_LIST
      n_skip        :INTEGER    -- Number to skip for FILE_LIST & PROG_LIST
      format        :INTEGER    -- Format of returned names
                                -- For FILE_LIST & PROG_LIST
      ary_nam       :ARRAY[9] OF STRING[20] -- Returned names
                                            -- from FILE_LIST & PROG_LIST
      prog_name     :STRING[10] -- Program names to list from PROG_LIST
      prog_type     :INTEGER    -- Program types to list form PROG_LIST
      n_progs       :INTEGER    -- Number of programs returned from PROG_LIST
      status    :INTEGER   -- Status of built-in procedure call
```

Listing Files and Programs and Manipulating Strings - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----    Section 3-A:  RANDOM Declaration
---                  Creates a pseudo-random number and returns the number.
--------------------------------------------------------------------------------
ROUTINE random(seed : INTEGER) : REAL
```

```
BEGIN
  seed = (seed * MULTIPLIER + INCREMENT) MOD MODULUS
  RETURN(seed/65535.0)
END random
--------------------------------------------------------------------------------
----    Section 3-B:  DISPL_LIST  Declaration
----                  Display maxnum elements of ary_nam.
--------------------------------------------------------------------------------
ROUTINE displ_list(maxnum :INTEGER)
BEGIN
  FOR indx1 = 1 TO maxnum DO ;  WRITE (ary_nam[indx1],cr);   ENDFOR
  -- Notice the use of the semi-colon, which allows multiple statements
  -- on a line.
END displ_list
--------------------------------------------------------------------------------
----    Section 3-C:  TP_CLS  Declaration
----                This routine is from ROUT_EX.KL and will
----                clear the TP USER menu screen and force it to be visible.
--------------------------------------------------------------------------------
ROUTINE tp_cls FROM rout_ex
```

Listing Files and Programs and Manipulating Strings - Main Program

```
--------------------------------------------------------------------------------
----    Section 4:  Main Program
--------------------------------------------------------------------------------
BEGIN -- LIST_EX
tp_cls                -- Use routine from the rout_ex.kl file
--------------------------------------------------------------------------------
----  Section 4-A: Generate a pseudo random number, convert INTEGER to STRING
--------------------------------------------------------------------------------
max_number = 255 ;                          -- So the random number is 0..255
seed = 259 ;
  WRITE ('Manupulating String',cr)
  cases = ROUND(ABS((random(seed)*max_number)))-- Call random then take the
                                              -- absolute value of the number
                                              -- returned and round off the
                                              -- number.
  CNV_INT_STR(cases, 1, 0, pr_cases)          -- Convert cases to its
                                              -- ascii representation
  pr_cases =  SUB_STR(pr_cases, 2,3)          -- get at most 3 characters,
                                              -- starting at the second
                                              -- character, since first
                                              -- character is a blank.
```

Listing Files and Programs and Manipulating Strings - Create and Load Program

```
--------------------------------------------------------------------------------
----    Section 4-B: Build a program name from the number and try to load it
--------------------------------------------------------------------------------
  --- Build a random program name to show the manipulation of
  --- STRINGS and INTEGERs.
  prg_nm = 'MYPROG' + pr_cases + '.PC'    -- Concatenate the STRINGs together
                                          -- which create a program name
  --- Verify that the program is not already loaded
  WRITE ('Checking load status of ',prg_nm,cr)
  LOAD_STATUS(prg_nm, loaded, initi)
  IF (NOT loaded) THEN                     -- The program is not loaded
     WRITE ('Loading ',prg_nm,cr)
     LOAD(prg_nm, 0 , status)              -- Load in the program
     IF (status =  0 ) THEN                -- Verify load is successful
        WRITE ('Loading ','MYPROG' + pr_cases + '.VR',cr)
        LOAD('MYPROG' + pr_cases + '.VR', 0, status) -- Load the .vr file
        IF (status <> 0 ) THEN             -- Loading variables failed
           WRITE ('Loading of ', 'MYPROG' + pr_cases + '.VR', ' failed',cr)
           WRITE ('Status = ',status);
        ENDIF
     ELSE                                  -- Load of program failed
        IF (status = 10003) THEN           -- File does not exist
           WRITE (prg_nm, ' file does not exist',cr)
        ELSE
           WRITE ('Loading of ',prg_nm, ' failed',cr,'Status = ',status);
        ENDIF
     ENDIF

 ELSE                                      -- The program is already loaded
     IF (NOT initi) THEN                   -- Variables not initialized
        WRITE ('Loading ','MYPROG' + pr_cases + '.VR',cr)
        LOAD('MYPROG' + pr_cases + '.VR', 0, status) -- Load in variables
        IF (status <> 0 ) THEN             -- Load of variables failed
           WRITE ('Loading of ', 'MYPROG' + pr_cases + '.VR', ' failed',cr)
           WRITE ('Status = ',status);
        ENDIF
     ENDIF
 ENDIF
```

Listing Files and Programs and Manipulating Strings - List Programs

```
--------------------------------------------------------------------------------
---- Section 4-C: Check the file listing of the drive FLPY: and display them
--------------------------------------------------------------------------------
```

```
  --- Display a directory listing of files on the Flpy:
  file_spec = 'FLPY:*.*'      -- All files in FLPY: drive
  n_skip = 0                  -- First time do not skip any files
  format = 3                  -- Return list in filename.filetype format
  WRITE ('Doing File list',cr)
  REPEAT                      -- UNTIL all files have been listed
    FILE_LIST(file_spec, n_skip, format, ary_nam, n_files, status)
    IF (status <>0 ) THEN     -- Error occurred
       WRITE ('FILE_LIST builtin failed with Status = ',status,cr)
    ELSE
       displ_list(n_files)    -- Write the names to the TP USER menu
       n_skip = n_skip + n_files  -- Skip the files we already got.
    ENDIF
  UNTIL (ARRAY_LEN(ary_nam) <> n_files) -- When n_files does not equal
                                        -- declared size of ary_name then
                                        --  all files have been listed.
-------------------------------------------------------------------------------
----    Section 4-D: Show the programs loaded in controller
-------------------------------------------------------------------------------
  --- Display the list of programs loaded on the controller
  prog_name = '*'             -- All program names should be listed
  prog_type = 6               -- Only PC type files should be listed
  n_skip = 0                  -- First time do not skip any file
  format = 2                  -- Return list in  filename.filetype format
  WRITE ('Doing Program list',cr)
  REPEAT                     -- UNTIL all programs have been listed
    PROG_LIST(prog_name, prog_type, n_skip, format, ary_nam, n_progs, status)
    -- The program names are stored in ary_nam
    -- n_progs is the number of program names stored in ary_nam
    IF (status <>0 ) THEN
       WRITE ('PROG_LIST builtin failed with Status = ',status,cr)
    ELSE
       displ_list(n_progs)              -- Display the current list
       n_skip = n_skip + n_progs        -- Skip the programs already listed
    ENDIF
  UNTIL (ARRAY_LEN(ary_nam) <> n_progs)  -- When n_files does not equal the
                                        -- declared size of ary_name then all
                                        --  programs have been listed.
END LIST_EX
```

B.8 GENERATING AND MOVING ALONG A HEXAGON PATH

This program generates a hexagonal path and moves along each side of the hexagon.

Generate and Move Along Hexagon Path - Overview

```
--------------------------------------------------------------------------------
----    GEN_HEX.KL
--------------------------------------------------------------------------------
----    Section 0:  Detail about GEN_HEX.KL
--------------------------------------------------------------------------------
---- Elements of KAREL Language Covered: In Section:
----    Action:
----    Clauses:
----    Conditions:
----    Data types:
----            ARRAY OF REAL               Sec 3-C
----            ARRAY OF XYZWPR             Sec 2
----            INTEGER                     Sec 2; 3-B,C
----            JOINTPOS6                   Sec 2
----            REAL                        Sec 3-B
----    Directives:
              %COMMENT                    Sec 1
----    Built-in Functions & Procedures:
----            CHECK_EPOS                  Sec 4-B
----            CNV_REL_JPOS                Sec 3-C
----            COS                         Sec 3-B
----            SIN                         Sec 3-B
----            SET_POS_REG                 4-B
----            SET_JPOS_REG                Sec 3-C
----    Statements:
----            CONNECT TIMER               Sec 4-A
----            FOR ... ENDFOR              Sec 3-B,C; 4-B
----            ROUTINE                     Sec 3-A,B,C
----            WRITE                       Sec 4-A,B
----    Reserve Word:
----            BEGIN                       Sec 3-B,C; 4
----            CONST                       Sec 2
----            CR                          Sec 4-A
----            END                         Sec 3-B,C; 4-B
----            PROGRAM                     Sec 1
----            VAR                         Sec 2
```

Generate and Move Along Hexagon Path - Declaration Section

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM gen_hex
%COMMENT = 'HEXAGON'
```

```
--------------------------------------------------------------------------------
----    Section 2:  Constant and Variable Declaration
--------------------------------------------------------------------------------
CONST
  L_HEX_SIDE = 300            -- Length of one side of the hexagon
  NUM_AXES   = 6              -- Number of robot axes
VAR
  p_cntr        : JOINTPOS6   -- Center of the hexagon
  p_xyzwpr      : ARRAY[NUM_AXES] OF XYZWPR
                  -- Six vertices of the hexagon
  tmp_xyz       :XYZWPR
  clock,
  t_start,
  t_end,
  t_total       : INTEGER
  status,
  p_indx        : INTEGER
```

Generate and Move Along Hexagon Path - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----   Section 3-A:  Routines move_to_pr and movl_to_pr are TP
----   routines for doing moves
--------------------------------------------------------------------------------
ROUTINE move_to_pr FROM move_to_pr      -- move_to_pr must also be loaded.
--   1:J PR[1] 100% FINE    ;
ROUTINE movl_to_pr FROM movl_to_pr      -- movl_to_pr must also be loaded.
--   1:L PR[1] 1000mm/sec FINE    ;
--------------------------------------------------------------------------------
----    Section 3-B:  R_HEX_CENTER Declaration
----                  Calculates the hexagon points based on distance
----                  between point 1 and 4 of the hexagon.
--------------------------------------------------------------------------------
ROUTINE r_calc_hex
VAR
  p1_to_pcntr   : REAL -- Distance from the center of the hex to point 1
  vertice       : INTEGER   -- the index used specify each vertice of hexagon
BEGIN
  p1_to_pcntr = (L_HEX_SIDE / 2) + (L_HEX_SIDE * COS(60))
  p_xyzwpr[1] = p_cntr      -- p_cntr was calculated in r_hex_center
  p_xyzwpr[1].y = p_xyzwpr[1].y - p1_to_pcntr --set the first vertice of hex
  FOR vertice = 2 TO NUM_AXES DO   -- start at 2 since 1 is already set
    p_xyzwpr[vertice] = p_xyzwpr[1]   -- Intialize all vertices
```

```
  ENDFOR
  -- Calculating individual components for each vertice of the hexagon
  p_xyzwpr[2].x = p_xyzwpr[1].x + (L_HEX_SIDE * SIN(60))
  p_xyzwpr[2].y = p_xyzwpr[1].y + (L_HEX_SIDE * COS(60))
  p_xyzwpr[3].x = p_xyzwpr[1].x + (L_HEX_SIDE * SIN(60))
  p_xyzwpr[3].y = p_xyzwpr[1].y + (L_HEX_SIDE + (L_HEX_SIDE * COS(60)))

  p_xyzwpr[4].y = p_xyzwpr[1].y + (L_HEX_SIDE + (2 * (L_HEX_SIDE * COS(60)))
  p_xyzwpr[5].x = p_xyzwpr[1].x - (L_HEX_SIDE * SIN(60))
  p_xyzwpr[5].y = p_xyzwpr[3].y
  p_xyzwpr[6].x = p_xyzwpr[1].x - (L_HEX_SIDE * SIN(60))
  p_xyzwpr[6].y = p_xyzwpr[2].y
END r_calc_hex
```

Generate and Move Along Hexagon Path - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3-C:  R_HEX_CENTER Declaration
----                  Positions the face plate perpendicular
----                  to the xy world coordinate plane.
--------------------------------------------------------------------------------
ROUTINE r_hex_center
VAR
  status, indx   : INTEGER
  p_cntr_arry    : ARRAY[NUM_AXES] OF REAL
BEGIN
-- Initalize the center position array to zero
  FOR indx = 1 TO NUM_AXES DO
     p_cntr_arry[indx] = 0
  ENDFOR
-- Set JOINT 3 and 5 to -45 and 45 degrees
  p_cntr_arry[3] = -45
  p_cntr_arry[5] =  45  -- Convert the REAL array to a joint position,
                        -- p_cntr
  CNV_REL_JPOS(p_cntr_arry, p_cntr, status)
  SET_JPOS_REG(1, p_cntr, status) -- Put p_cntr in PR[1]
  move_to_pr    -- Call TP program to move to PR[1]

END r_hex_center
```

Generate and Move Along Hexagon Path - Main Program

```
--------------------------------------------------------------------------------
----    Section 4:  Main Program
--------------------------------------------------------------------------------
```

```
BEGIN  --- GEN_HEX
--------------------------------------------------------------------------------
----    Section 4-A: Connect timer, set uframe, call routines
--------------------------------------------------------------------------------
  clock = 0                    -- Initialize clock value to zero
  CONNECT TIMER TO clock       -- Connect the timer
  WRITE ('Moving to the center of the HEXAGON',CR) -- update user of process
  r_hex_center                 -- position the face plate of robot.
  WRITE ('Calculating the sides of HEXAGON',CR)  -- update user
  r_calc_hex                   -- Calculate the hexagon points
--------------------------------------------------------------------------------
----    Section 4-B: Move on sides of hexagon
--------------------------------------------------------------------------------
  WRITE ('Moving along the sides of the Hexagon',CR) -- Update user
  t_start = clock              -- Record the time before motion begins

  FOR p_indx = 1 TO 6 DO
   -- Verify that the position is reachable
   CHECK_EPOS ((p_xyzwpr[p_indx]), $UFRAME, $UTOOL, status)
   IF (status <> 0) THEN
    WRITE ('Unable to move to p_xyzwpr[', p_indx,']',CR);
   ELSE
    SET_POS_REG(1, p_xyzwpr[p_index], status)
    movl_to_pr             -- Call TP program to move to PR[1]
                           -- Move to each vertex of hexagon
   ENDIF
  ENDFOR
  SET_POS_REG(1, p_xyzwpr[1], status)
  movl_to_pr             -- Call TP program to move to PR[1]
                         -- Move back to first vertex of hexagon

  tmp_xyz = p_cntr
  SET_POS_REG(1, tmp_xyz, status)
  movl_to_pr               -- Move TCP in a straight
                           -- line to the center position
  t_end = clock            -- Record ending time
  WRITE('Total motion time = ',t_end-t_start,CR) --Display the total time for
                                                 -- motion.
                                                 -- NOTE that the total was
                                                 -- computed in the WRITE
                                                 -- statement.
END GEN_HEX
```

B.9 USING THE FILE AND DEVICE BUILT-INS

This program demonstrates how to use the File and Device built-ins. This program FORMATS and MOUNTS the RAM disk. Then copies files from the FLPY: device to RD:. If the RAM disk gets full the RAM disk size is increased and reformatted. This program continues until either all the files are copied successfully, or the built-in operations fail.

File and Device Built-ins Program - Overview

```
--------------------------------------------------------------------------------
----    FILE_EX.Kl
--------------------------------------------------------------------------------
----    Section 0:  Detail about FILE_EX.kl
--------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:            In Section:
----    Action:
----    Clauses:
----            FROM                              Sec 3
----    Conditions:
----    Data types:
----              BOOLEAN                           Sec 2
----             INTEGER                            Sec 2
----              STRING                            Sec 2
----    Directives:
----              COMMENT                           Sec 1
----             NOLOCKGROUP                        Sec 1
----    Built-in Functions & Procedures:
----              CNV_TIME_STR                      Sec 4-A
----              COPY_FILE                         Sec 4-B
----              DISMOUNT_DEV                      Sec 4-B
----              FORMAT_DEV                        Sec 4-B
----              GET_TIME                          Sec 4-A
----              MOUNT_DEV                         Sec 4-B
----              PURGE_DEV                         Sec 4-B
----              SUB_STR                           Sec 4-A
----    Statements:
----              IF...THEN...ELSE...ENDIF          Sec 4-B
----              REPEAT...UNTIL                    Sec 4-A
----              ROUTINE                           Sec 3
----              SELECT...ENDSELECT                Sec 4-B
----              WRITE                             Sec 4-A,B
```

File and Device Built-ins Program - Overview Continued

```
----    Reserve Words:
----              BEGIN                            Sec 4
```

```
----              CONST                          Sec 2
----              CR                             Sec 4-A,B
----              END                            Sec 4-B
----              PROGRAM                        Sec 4
----              VAR                            Sec 2
----    Devices Used:
----              FLPY                           Sec 4-B
----              MF3                            Sec 4-B
----              RD                             Sec 4-B
----              FR                             Sec 4-B
```

File and Device Built-ins Program - Declaration Section, Declare Routines

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM FILE_EX
%nolockgroup
%comment = 'COPY FILES'
--------------------------------------------------------------------------------
----    Section 2:  Variable Declaration
--------------------------------------------------------------------------------
CONST
    SUCCESS     = 0          -- Success status from builtins
    FINISHED    = TRUE        -- Finished Copy
    TRY_AGAIN   = FALSE       -- Try to copy again
    RD_FULL     = 85020       -- RAM disk full
    NOT_MOUNT   = 85005       -- Device not mounted
    FR_FULL     = 85001       -- FROM disk is full
    MNT_RD      = 85004       -- RAM disk must be mounted
  --Refer to

FANUC Robotics Controller KAREL
Setup and Operations Manual for an Error Code listing
VAR
    time_int    : INTEGER
    time_str    : STRING[30]
    status      : INTEGER
    cpy_stat    : BOOLEAN
    to_dev      : STRING[5]
--------------------------------------------------------------------------------
----    Section 3: Routine Declaration
--------------------------------------------------------------------------------
ROUTINE tp_cls FROM ROUT_EX
--------------------------------------------------------------------------------
----    Section 4: Main program
```

```
--------------------------------------------------------------------------------
BEGIN  -- FILE_EX
  tp_cls     -- from rout_ex.kl
--------------------------------------------------------------------------------
----    Section 4-A: Get Time and FORMAT ramdisk with date as volume name
--------------------------------------------------------------------------------
 GET_TIME(time_int)                                -- Get the system time
 CNV_TIME_STR(time_int, time_str)                  -- Convert the INTEGER time
                                                   -- to readable format
 WRITE ('Today is ', SUB_STR(time_str, 2,8),CR)    -- Display the date part
 WRITE ('Time is ', SUB_STR(time_str, 11,5),CR)    -- Display the time part
```

File and Device Built-ins Program - Mount and Copy to RAM Disk

```
--------------------------------------------------------------------------------
----    Section 4-B: Mount RAMDISK and start copying from FLPY to
MF3:
--------------------------------------------------------------------------------
to_dev = 'MF3:'
REPEAT                            -- Until all files have been copied
  cpy_stat = FINISHED
  WRITE('COPYing......',cr)
  -- Copy the files from FLPY: to to_dev and overwrite the file if it
  -- already exists.
  COPY_FILE('FLPY:*.kl', to_dev, TRUE, FALSE, status)
  SELECT (status) OF
    CASE (RD_FULL):               -- RAM disk is full
                                  -- Dismount and re-size the RAM-DISK
      WRITE ('DISMOUNTing RD: ....',cr)
      DISMOUNT_DEV('RD:', status)
                                  -- Verify DISMOUNT was successful or that
                                  -- the device was not already mounted
      IF (status = SUCCESS) OR (status = NOT_MOUNT) THEN
           -- Increase the size of RD:
        WRITE('Increasing RD: size...',cr)
        $FILE_MAXSEC = ROUND($FILE_MAXSEC * 1.2)
                                 -- Increase the RAM disk size
                                 -- Format the RAM-DISK
        WRITE('FORMATTING RD:......',cr)
        FORMAT_DEV('RD:','' ,FALSE, status) -- Format the RAM disk
        IF (status <> SUCCESS) THEN
          WRITE ('FORMAT of RD: failed, status:', status,CR)
          WRITE ('Copy incomplete',cr)
        ELSE
          cpy_stat = TRY_AGAIN
        ENDIF
```

```
        WRITE('MOUNTing RD:......',cr)
        MOUNT_DEV ('RD:', status)
        IF (status <> SUCCESS) THEN
          WRITE ('MOUNTing of RD: failed, status:', status,CR)
          WRITE ('Copy incomplete',cr)
        ELSE
          cpy_stat = TRY_AGAIN
        ENDIF
      ELSE
        WRITE ('DISMOUNT of RD: failed, status:', status,cr)
        WRITE ('Copy incomplete',cr)
      ENDIF
```

File and Device Built-ins Program - Mount and Copy to RAM Disk Continued

```
CASE (FR_FULL):     -- FROM disk is full
      WRITE ('FROM disk is full',CR, 'PURGING FROM.....', CR)
      PURGE_DEV ('FR:', status)   -- Purge the FROM
      IF (status <> SUCCESS) THEN
        WRITE ('PURGE of FROM failed, status:', status, CR)
        WRITE ('Copy incomplete', CR)
      ELSE
        cpy_stat = TRY_AGAIN
      ENDIF
CASE (NOT_MOUNT, MNT_RD):    -- Device is not mounted
      WRITE ('MOUNTing ',to_dev,'.....',CR)
      MOUNT_DEV(to_dev, status)
      IF (status <> SUCCESS) THEN
        WRITE ('MOUNTing of ',to_dev,': failed, status:', status, CR)
        WRITE ('Copy incomplete', CR)
      ELSE
        cpy_stat = TRY_AGAIN
      ENDIF
CASE (SUCCESS):
      WRITE ('Copy completed successfully!',CR)
ELSE:
      WRITE ('Copy failed, status:', status,CR)
ENDSELECT
UNTIL (cpy_stat = FINISHED)
END file_ex
```

B.10 USING DYNAMIC DISPLAY BUILT-INS

This program demonstrates how to use the dynamic display built-ins. This program initiates the dynamic display of various data types. It then executes another task, CHG_DATA, which changes the values of these variables.

Before exiting this program the dynamic displays are cancelled and the other task is aborted. If DYN_DISP is aborted, it will set a variable which CHG_DATA detects. This ensures that CHG_DATA cannot continue executing once DYN_DISP is aborted.

Using Dynamic Display Built-ins - Overview

```
------------------------------------------------------------------------------
----    DYN_DISP.Kl
------------------------------------------------------------------------------
----    Section 0:  Detail about DYN_DISP.KL
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:         In Section:
----    Actions:
----    Clauses:
----                FROM                            Sec 3-C
----                IN CMOS                         Sec 2
----                WHEN                            Sec 4
----    Conditions:
----                ABORT                           Sec 4
----    Data types:
----                BOOLEAN                         Sec 2
----                INTEGER                         Sec 2
----                REAL                            Sec 2
----                STRING                          Sec 2
----    Directives:
----                ALPHABETIZE                     Sec 1
----                COMMENT                         Sec 1
----                NOLOCKGROUP                     Sec 1
```

Using Dynamic Display Built-ins - Overview Continued

```
----    Built-in Functions & Procedures:
----                ABORT_TASK                      Sec 4-C
----                CNC_DYN_DISI                    Sec 4-C
----                CNC_DYN_DISR                    Sec 4-C
----                CNC_DYN_DISB                    Sec 4-C
----                CNC_DYN_DISE                    Sec 4-C
----                CNC_DYN_DISP                    Sec 4-C
----                CNC_DYN_DISS                    Sec 4-C
----                INI_DYN_DISI                    Sec 3-A, 4-A
```

```
----                INI_DYN_DISR                    Sec 3-B, 4-A
----                INI_DYN_DISB                    Sec 3-C, 4-A
----                INI_DYN_DISE                    Sec 3-D, 4-A
----                INI_DYN_DISP                    Sec 3-E, 4-A
----                INI_DYN_DISS                    Sec 3-F, 4-A
----                LOAD_STATUS                     Sec 4-B
----                LOAD                            Sec 4-B
----                RUN_TASK                        Sec 4-B
----
----    Statements:
----                CONDITION...ENDCONDITION        Sec 4
----                IF...THEN...ENDIF               Sec 4-A,B,C
----                READ                            Sec 4-C
----                ROUTINE                         Sec 3-A,B,C
----                WRITE                           Sec 4-A,B,C
----
----    Reserve Words:
----                BEGIN                           Sec 3-A,B; 4
----                CR                              Sec 4-A
----                CONST                           Sec 2
----                END                             Sec 3-A,B; 4-C
----                PROGRAM                         Sec 4
----                VAR                             Sec 2
----    Predefined File Variables:
----                TPPROMPT                        Sec 4-B,C
----    Predefined Windows:
----                T_FU                            Sec 3-A,B
```

Using Dynamic Display Built-ins - Declaration Section

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM DYN_DISP
%nolockgroup
%comment = 'Dynamic Disp'
%alphabetize
%INCLUDE KLIOTYPS
--------------------------------------------------------------------------------
----    Section 2:  Variable Declaration
--------------------------------------------------------------------------------
CONST
  cc_success     = 0      -- Success status
  cc_clear_win   = 128    -- Clear window
  cc_clear_eol   = 129    -- Clear to end of line
  cc_clear_eow   = 130    -- Clear to end of window
```

```
  CH_ABORT       = 1      -- Condition Handler to detect when program aborts
VAR
  Int_wind          :STRING[10]
  Rel_wind          :STRING[10]
  Field_Width       :INTEGER
  Attr_Mask         :INTEGER
  Char_Size         :INTEGER
  Row               :INTEGER
  Col               :INTEGER
  Interval          :INTEGER
  Buffer_Size       :INTEGER
  Format            :STRING[7]
  bool_names        :ARRAY[2] OF STRING[10]
  enum_names        :ARRAY[4] OF STRING[10]
  pval_names        :ARRAY[2] OF STRING[10]
  bool1 IN CMOS     :BOOLEAN
  enum1 IN CMOS     :INTEGER
  port_type         :INTEGER
  port_no           :INTEGER
  Str1  IN CMOS     :STRING[10]
  Int1  IN CMOS     :INTEGER    -- Using IN CMOS will create the variables
  Real1 IN CMOS     :REAL       -- in CMOS RAM, which is permanent memory.
  status            :INTEGER
  loaded,
  initialized       :BOOLEAN
  dynd_abrt         :BOOLEAN  -- Set to true when program aborts.
```

Using Dynamic Display Built-ins - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3: Routine Declaration
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----    Section 3-A: SET_INT Declaration
----                  Set all the input parameters for the INI_DYN_DISI call.
--------------------------------------------------------------------------------
ROUTINE Set_Int
Begin
  -- Valid predefined windows are described in
  -- Chapter 7.9.1, "USER MENU on the Teach Pendant
  --    Error  Line     --> 'ERR'   1  line
  --    Status Line     --> 'T_ST'  3  lines
  --    Display Window  --> 'T_FU'  10 lines
  --    Prompt Line     --> 'T_PR'  1  line
  --    Function Key    --> 'T_FK'  1  line
  Int_Wind          = 'T_FU'            -- Use the predefined display window
```

```
  Field_Width       = 0                 -- Use the minimum width necessary
  Attr_Mask         = 1 OR 4            -- BOLD and UNDERLINED
  Char_Size         = 0                 -- Normal
  Row               = 1                 -- Specify the location within 'T_FU'
  Col               = 16                --    to dynamically displayed
  Interval          = 250                -- 250ms between updates
  Buffer_Size       = 10                -- Minimum value required.
  Format            = '%-8d'            -- 8 character minimum field width
--- With this specification the INTEGER will be displayed as follows:
---               --------
---              |xxxxxxxx|
---               --------
---      Where the integer value  will be left justified.
---      The x's will be the integer value unless the integer value is
---      less then 8 characters, then the right side will be blanks up to
---      a total 8 characters.  If the integer value is greater than the 8
---      characters the width is dynamically increased to display the whole
---      integer value.  The INTEGER value will also be bold and underlined.
End Set_Int
```

Using Dynamic Display Built-ins - Declare Routines Continued

```
--------------------------------------------------------------------------------
----    Section 3-B: SET_REAL Declaration
----                 Set all the input parameters for the INI_DYN_DISR call.
--------------------------------------------------------------------------------
ROUTINE Set_Real
Begin
  Rel_Wind          = 'T_FU'            -- Use the predefined display window
  Field_Width       = 10                -- Maximum width of display.
  Attr_Mask         = 2 OR 8            -- blinking and reverse video
  Char_Size         = 0                 -- Normal
  Row               = 2                 -- Specify the location within 'TFU'
  Col               = 16                --    to dynamically display
  Interval          = 200               -- 200ms between update
  Buffer_Size       = 10                -- Minimum value required.
  Format            = '%2.2f'
--- With the format and field_width specification the REAL will be
--- displayed as follows:
---               ----------
---              |xxxx.xx   |
---               ----------
---      Where the real value  will be left justified.
---      There will always be two digits after the decimal point.
---      A maximum width of 10 will be used.
---      If the real value is less then 10 characters the right side will be
```

```
---      padded with blanks up to 10 character width.
---      If the real value exceeds 10 characters, the display width will not
---      expand but will display a ">" as the last character, indicating the
---      entire value is not displayed.
---      The value will also be blinking and in reverse video.
End Set_Real
```

Using Dynamic Display Built-ins - Declare Routines Continued

```
--------------------------------------------------------------------------------
----    Section 3-C: SET_BOOL Declaration
----                  Set all the input parameters for the INI_DYN_DISB call
--------------------------------------------------------------------------------
ROUTINE Set_Bool
Begin
  -- Valid predefined windows are described in
  -- Chapter 7.9.1, "USER MENU on the Teach Pendant
  --    Error  Line     --> 'ERR'   1  line
  --    Status Line     --> 'T_ST'  3  lines
  --    Display Window  --> 'T_FU'  10 lines
  --    Prompt Line     --> 'T_PR'  1  line
  --    Function Key    --> 'T_FK'  1  line
  Int_Wind            = 'T_FU'         -- Use the predefined display window
  Field_Width       = 10               -- Display 10 chars
  Attr_Mask         = 2                -- Blinking
  Char_Size         = 0                -- Normal
  Row               = 3                -- Specify the location within 'T_FU'
  Col               = 16               --    to dynamically displayed
  Interval          = 250              -- 250ms between updates
  Buffer_Size       = 10               -- Minimum value required
  bool_names[1]     = 'YES'            -- string display in bool_var is FALSE
  bool_names[2]     = 'NO'             -- string display in bool_var is TRUE
--- With this specification the BOOLEAN will be displayed as follows:
---               --------
---              |xxxxxxxx|
---               --------
---      Where the boolean value will be left justified.
---      The x's will be one of the strings 'YES' or 'NO', depending on
---      the value of bool1.  The string will be blinking.
End Set_Bool
```

Using Dynamic Display Built-ins - Declare Routines Continued

```
--------------------------------------------------------------------------------
----    Section 3-D: SET_ENUM Declaration
```

```
----                  Set all the input parameters for the INI_DYN_DISE call.
--------------------------------------------------------------------------------
ROUTINE Set_Enum
Begin
  -- Valid predefined windows are described in
  -- Chapter 7.9.1, "USER MENU on the Teach Pendant
  --    Error  Line     --> 'ERR'   1  line
  --    Status Line     --> 'T_ST'  3  lines
  --    Display Window  --> 'T_FU'  10 lines
  --    Prompt Line     --> 'T_PR'  1  line
  --    Function Key    --> 'T_FK'  1  line
  Int_Wind          = 'T_FU'          -- Use thE predefined display window
  Attr_Mask         = 8                 -- REVERSED
  Field_Width       = 10                -- Display to characters
  Char_Size         = 0                 -- Normal
  Row               = 4                 -- Specify the location within 'T_FU'
  Col               = 16                --    to dynamically displayed
  Interval          = 250               -- 250ms between updates
  Buffer_Size       = 10                -- Minimum value required
  enum_names[1]     = 'Enum-0'          -- value displayed if enum_var = 0
  enum_names[2]     = 'Enum-1'          -- value displayed if enum_var = 1
  enum_names[3]     = 'Enum-2'          -- value displayed if enum_var = 2
  enum_names[4]     = 'Enum-3'          -- value displayed if enum_var = 3
--- With this specification enum_var will be displayed as follows:
---                --------
---               |xxxxxxxx|
---                --------
---      Where one of the strings enum_names will be displayed,
---      depending on the integer value enum1.  If enum1 is outside
---      the range 0-3, a string of 10 '?'s will be displayed.
---      The string will be displayed in reversed video.
End Set_Enum
```

Using Dynamic Display Built-ins - Declare Routines Continued

```
--------------------------------------------------------------------------------
----    Section 3-E: SET_PORT Declaration
----                  Set all the input parameters for the INI_DYN_DISP call.
--------------------------------------------------------------------------------
ROUTINE Set_Port
Begin
  -- Valid predefined windows are described in
  -- Chapter 7.9.1, "USER MENU on the Teach Pendant
  --    Error  Line     --> 'ERR'   1  line
  --    Status Line     --> 'T_ST'  3  lines
  --    Display Window  --> 'T_FU'  10 lines
```

```
  --    Prompt Line     --> 'T_PR'  1  line
  --    Function Key    --> 'T_FK'  1  line
  Int_Wind          = 'T_FU'         -- Use the predefined display window
  Field_Width       = 10             -- Display to characters
  Attr_Mask         = 1              -- BOLD
  Char_Size         = 0              -- Normal
  Row               = 5              -- Specify the location within 'T_FU'
  Col               = 16             --    to dynamically displayed
  Interval          = 250            -- 250ms between updates
  Buffer_Size       = 10             -- Minimum value required.
  pval_names[1]     = 'RELEASED'     -- text displayed if key is not pressed
  pval_names[2]     = 'PRESSED'      -- text displayed if key is pressed
  port_type         = io_tpin        -- port type = TP key
  port_no           = 175            -- user-key 3
--- With this specification PRESSED or RELEASED will be displayed as follows:
---               --------
---              |xxxxxxxx|
---               --------
---      Where the string will be left justified.
---      The x's will be either 'RELEASED' or 'PRESSED'.
---      The string will also be normal video.
---      (Bold is not supported on the teach pendant.)
End Set_Port
```

Using Dynamic Display Built-ins - Declare Routines Continued

```
--------------------------------------------------------------------------------
----    Section 3-F: SET_STR Declaration
----                 Set all the input parameters for the INI_DYN_DISS call.
--------------------------------------------------------------------------------
ROUTINE Set_Str
Begin
  -- Valid predefined windows are described in
  -- Chapter 7.9.1, "USER MENU on the Teach Pendant
  --    Error  Line     --> 'ERR'   1  line
  --    Status Line     --> 'T_ST'  3  lines
  --    Display Window  --> 'T_FU'  10 lines
  --    Prompt Line     --> 'T_PR'  1  line
  --    Function Key    --> 'T_FK'  1  line
  Int_Wind           = 'T_FU'          -- Use th predefined display window
  Field_Width       = 10               -- Use the minimum width neccessary
  Attr_Mask         = 1 OR 4           -- BOLD and UNDERLINED
  Char_Size         = 0                -- Normal
  Row               = 6                -- Specify the location within 'T_FU'
  Col               = 16               --    to dynamically displayed
  Interval          = 250              -- 250ms between updates
```

```
  Buffer_Size       = 10                -- Minimum value required.
  Format            = '%10s'            -- 10 character minimum field width
--- With this specification the STRING will be displayed as follows:
---               --------
---              |xxxxxxxx|
---               --------
---      Where the string value will be left justified.
---      The x's will be the string value.
---      The STRING will also be underlined.
End Set_Str
-------------------------------------------------------------------------------
----    Section 3-G: TP_CLS Declaration
----              Clear the TP USER menus screen and force it to be visible.
-------------------------------------------------------------------------------
ROUTINE tp_cls FROM rout_ex
```

Using Dynamic Display Built-ins - Initiate Dynamic Displays

```
-------------------------------------------------------------------------------
----    Section 4: Main program
-------------------------------------------------------------------------------
BEGIN --- DYN_DISP
dynd_abrt = FALSE
CONDITION[CH_ABORT]:
  WHEN ABORT DO      -- When the program is aborting set dynd_abrt flag.
                     -- This will be triggered if this program aborts itself
                     --  or if an external mechanism aborts this program.
    dynd_abrt = TRUE -- CHG_DATA will detect this and complete execution.
ENDCONDITION
ENABLE CONDITION [CH_ABORT]
-------------------------------------------------------------------------------
----    Section 4-A: Setup variables, initiate dynamic display
-------------------------------------------------------------------------------
  TP_CLS                              -- Clear the TP USER screen
                                      -- Force display of the TP USER screen
  STATUS = cc_success                 -- Initialize the status variable
  -- Initialize the dynamically displayed variables
  Int1       = 1
  Real1      = 1.0
  Bool1      = FALSE
  Enum1      = 0
  Strl       = ''
  -- Display messages to the TP USER screen
  WRITE ('Current INT1 =',CR)
  WRITE ('Current REAL1=',CR)
  WRITE ('Current BOOL1=',CR)
```

```
  WRITE ('Current ENUM1=',CR)
  WRITE ('Current PORT =',CR)
  WRITE ('Current STR1 =',CR)
  Set_Int                -- Set parameter values for INTEGER DYNAMIC DISPLAY
  INI_DYN_DISI(Int1,Int_Wind,Field_Width,Attr_Mask,Char_Size,
               Row,Col, Interval, Buffer_Size, Format ,Status)
  IF Status <> cc_success THEN -- Check the status
     WRITE(' INI_DYN_DISI failed, Status=',status,CR)
  ENDIF
 Set_Bool               -- Set parameter values for BOOLEAN DYNAMIC
DISPLAY
  INI_DYN_DISB(Bool1,Int_Wind,Field_Width,Attr_Mask,Char_Size,
               Row,Col, Interval, bool_names,Status)
  IF Status <> cc_success THEN -- Check the status
     WRITE(' INI_DYN_DISB failed, Status=',status,CR)
  ENDIF
```

Using Dynamic Display Built-ins - Initiate Dynamic Displays

```
Set_Enum                  -- Set parameter values for Enumerated Integer
                          --    DYNAMIC DISPLAY
  INI_DYN_DISE(Enum1,Int_Wind,Field_Width,Attr_Mask,Char_Size,
               Row,Col, Interval, enum_names,Status)
  IF Status <> cc_success THEN -- Check the status
     WRITE(' INI_DYN_DISE failed, Status=',status,CR)
  ENDIF
  Set_Port               -- Set parameter values for Port DYNAMIC DISPLAY
  INI_DYN_DISP(port_type, port_no ,Int_Wind,Field_Width,Attr_Mask,Char_Size,
               Row,Col, Interval, pval_names, Status)
  IF Status <> cc_success THEN -- Check the status
     WRITE(' INI_DYN_DISP failed, Status=',status,CR)
  ENDIF
  Set_Real                  -- Set parameter values for REAL DYNAMIC DISPLAY
  INI_DYN_DISR(Real1,Rel_Wind,Field_Width,Attr_Mask,Char_Size,
               Row,Col, Interval, Buffer_Size, Format ,Status)
  IF Status <> cc_success THEN -- Check the status
     WRITE(' INI_DYN_DISR failed, Status=',status,CR)
  ENDIF
 Set_Str                -- Set parameter values for STRING DYNAMIC
DISPLAY
  INI_DYN_DISS(Str1,Int_Wind,Field_Width,Attr_Mask,Char_Size,
               Row,Col, Interval, Buffer_Size, Format ,Status)
  IF Status <> cc_success THEN -- Check the status
     WRITE(' INI_DYN_DISS failed, Status=',status,CR)
  ENDIF
```

Using Dynamic Display Built-ins - Execute Subordinate Task

```
--------------------------------------------------------------------------------
----    Section 4-B: Check on subordinate program and execute it.
--------------------------------------------------------------------------------
  -- Check the status of the other program which will change the value
  -- of the variables.
  LOAD_STATUS('chg_data', loaded, initialized)
  IF (loaded = FALSE ) THEN
    WRITE TPPROMPT(CHR(cc_clear_win))            -- Clear the prompt line
    WRITE TPPROMPT('CHG_DATA is not loaded. Loading now...')
    LOAD('chg_data.pc',0,status)
    IF (status =  cc_success) THEN              -- Check the status
      RUN_TASK('CHG_DATA',1,false,false,1,status)
      IF (Status <> cc_success) THEN              -- Check the status
        WRITE ('Changing the value of the variables',CR)
        WRITE ('by another program failed',CR)
        WRITE ('BUT you can try changing the values',CR)
        WRITE ('from KCL',CR)
      ENDIF
    ELSE
      WRITE ('LOAD Failed, status = ',status,CR)
    ENDIF
  ELSE
    RUN_TASK('CHG_DATA',1,false,false,1,status)
    IF (Status <> cc_success) THEN                -- Check the status
      WRITE ('Changing the value of the variables',CR)
      WRITE ('by another program failed',CR)
      WRITE ('BUT you can try changing the values',CR)
      WRITE ('from KCL',CR)
    ENDIF
  ENDIF
```

Using Dynamic Display Built-ins - User Response Cancels Dynamic Displays

```
--------------------------------------------------------------------------------
----    Section 4-C: Wait for user response, and cancel dynamic displays
--------------------------------------------------------------------------------
  WRITE TPPROMPT(CHR(cc_clear_win))        -- Clear the prompt line
  WRITE TPPROMPT('Enter a number to cancel DYNAMIC display: ')
  READ (CR)                                -- Read only one character
                                           -- See Chapter 7.7.1,
                                           -- "Formatting INTEGER Data Items"
  ABORT_TASK('CHG_DATA',TRUE, TRUE,status)   -- Abort CHG_DATA
  IF (status <> cc_success) THEN             -- Check the status
     WRITE(' ABORT_TASK failed, Status=',status,CR)
```

```
  ENDIF
  CNC_DYN_DISI(Int1, Int_Wind,Status)       -- Cancel display of Intl
  IF Status <> 0 THEN                       -- Check the status
     WRITE(' CND_DYN_DISI failed, Status=',status,CR)
  ENDIF
  CNC_DYN_DISR(Real1,Rel_Wind,Status)       -- Cancel display of Real1
  IF Status <> 0 THEN                       -- Check the status
     WRITE(' CND_DYN_DISR failed, Status=',status,CR)
  ENDIF
  CNC_DYN_DISB(Bool1, Int_Wind,Status)      -- Cancel display of Bool1
  IF Status <> 0 THEN                       -- Check the status
     WRITE(' CND_DYN_DISB failed, Status=',status,CR)
  ENDIF
  CNC_DYN_DISE(Enum1, Int_Wind,Status)     -- Cancel display of Enum1
  IF Status <> 0 THEN                      -- Check the status
     WRITE(' CND_DYN_DISE failed, Status=',status,CR)
  ENDIF
  CNC_DYN_DISP(port_type, Port_no, Int_Wind,Status) -- Cancel display of Port
  IF Status <> 0 THEN                       -- Check the status
     WRITE(' CND_DYN_DISP failed, Status=',status,CR)
  ENDIF
  CNC_DYN_DISS(Str1, Int_Wind,Status)        -- Cancel display of String
  IF Status <> 0 THEN                        -- Check the status
     WRITE(' CND_DYN_DISS failed, Status=',status,CR)
  ENDIF
END DYN_DISP
```

B.11 MANIPULATING VALUES OF DYNAMICALLY DISPLAYED VARIABLES

The CHG_DATA.KL program is called by DYN_DISP, where the actual variables are displayed dynamically. This program does some processing and changes the value of those variables.

Manipulate Dynamically Displayed Variables - Overview

```
------------------------------------------------------------------------------
----    CHG_DATA.KL
------------------------------------------------------------------------------
------------------------------------------------------------------------------
----    Section 0:  Detail about CHG_DATA.KL
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:          In Section:
----    Actions:
----    Clauses:
----                FROM                            Sec 2
```

```
----    Conditions:
----    Data types:
----                INTEGER                         Sec 2
----                REAL                            Sec 2
----
----    Directives:
----    Built-in Functions & Procedures:
----    Statements:
----                DELAY                           Sec 4
----                FOR....ENDFOR                   Sec 4
----                REPEAT...UNTIL                  Sec 4
----
----    Reserve Words:
----                BEGIN                           Sec 4
----                END                             Sec 4
----                PROGRAM                         Sec 1
----                VAR                             Sec 2
------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
------------------------------------------------------------------------------
PROGRAM CHG_DATA
%nolockgroup
%comment = 'Dynamic Disp2'
```

Manipulate Dynamically Displayed Variables - Declaration Section

```
------------------------------------------------------------------------------
----    Section 2:  Variable Declaration
------------------------------------------------------------------------------
VAR
 -- IF the following variables did NOT have IN CMOS, the following errors
 -- would be posted when loading this program:
 --   VARS-012 Create var -INT1 failed  VARS-038 Cannot change CMOS/DRAM type
 --   VARS-012 Create var -REAL1 failed VARS-038 Cannot change CMOS/DRAM type
 -- This indicates that there is a discrepancy between DYN_DISP and CHG_DATA.
 -- One program has specified to create the variables in DRAM the
 -- other specified CMOS.
  Int1 IN CMOS FROM dyn_disp   :INTEGER    -- dynamically displayed variable
  Real1 IN CMOS FROM dyn_disp  :REAL       -- dynamically displayed variable
  Bool1 IN CMOS FROM dyn_disp  :BOOLEAN    -- dynamically displayed variable
  Enum1 IN CMOS FROM dyn_disp  :INTEGER    -- dynamically displayed variable
  Str1 IN CMOS FROM dyn_disp   :STRING[10] -- dynamically displayed variable
  indx                          :INTEGER
  dynd_abrt FROM dyn_disp       :BOOLEAN   -- Set in dyn_disp when dyn_disp
                                           -- is aborting
------------------------------------------------------------------------------
```

```
----    Section 3: Routine Declaration
--------------------------------------------------------------------------------
```

Manipulate Dynamically Displayed Variables - Main Program

```
--------------------------------------------------------------------------------
----    Section 4: Main program
--------------------------------------------------------------------------------
BEGIN  -- CHG_DATA

 -- This demonstrates that the variables are changed from this task, CHG_DATA.
 -- The dynamic display initiated in task DYN_DISP, will continue
 -- to correctly display the updated values of these variables.
 -- Do real application processing.
 -- Simulated here in a FOR loop.
REPEAT
  FOR indx  = -9999 to 9999 DO
    int1  = (indx DIV 2) * 7
    real1 = (indx DIV 3)* 3.13
    bool1 = ((indx AND 4) = 0)
    enum1 = (ABS(indx) DIV 5) MOD 5
    Str1 = SUB_STR('123456789A', 1, (ABS(indx) DIV 6) MOD 7 + 1)
    delay 200   -- Delay for 1/5 of a second as if processing is going on.
  ENDFOR
UNTIL (DYND_ABRT)   -- This task is aborted from DYN_DISP.  However, if
                    -- DYN_DISP aborts abnormally (ie from a KCL> ABORT), it
                    -- will set DYND_ABRT, which will allow CHG_DATA to
                    -- complete execution.
END CHG_DATA
```

B.12 DISPLAYING A LIST FROM A DICTIONARY FILE

This program controls the display of a list which is read in from the dictionary file DCLISTEG.UTX. For more information on DCLISTEG.UTX refer to Section B.12.1 . DCLST_EX.KL controls the placement of the cursor along with the action taken for each command.

Note The use of DISCTRL_FORM is the preferred method of displaying information. DISCTRL_FORM will automatically take care of all the key inputs and is much easier to use. For more information, refer to Chapter 10 *DICTIONARIES AND FORMS* .

Display List from Dictionary File - Overview

```
--------------------------------------------------------------------------------
----   DCLST_EX.KL
--------------------------------------------------------------------------------
```

```
----    Section 0:  Detail about DCLST_EX.KL
--------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:      In Section:
----    Actions:
----    Clauses:
----            FROM                              Sec 3-E
----    Conditions:
----    Data types:
----            ARRAY OF STRING                    Sec 2
----            BOOLEAN                            Sec 2
----            DISP_DAT_T                         Sec 2
----            FILE                               Sec 2
----            INTEGER                            Sec 2
----            STRING                             Sec 2
----    Directives:
----            ALPHABETIZE                        Sec 1
----            COMMENT                            Sec 1
----            INCLUDE                            Sec 1
----            NOLOCKGROUP                        Sec 1
----    Built-in Functions & Procedures:
----            ADD_DICT                           Sec 3-B
----            ACT_SCREEN                         Sec 4-I
----            ATT_WINDOW_S                       Sec 4-C
----            CHECK_DICT                         Sec 3-B
----            CLR_IO_STAT                        Sec 3-A,C
----            CNV_STR_INT                        Sec 4-E
----            DEF_SCREEN                         Sec 4-B
----            DET_WINDOW                         Sec 4-I
----            DISCTRL_LIST                       Sec 4-G,H
----            FORCE_SPMENU                       Sec 4-A,B,C
----            IO_STATUS                          Sec 3-A,
----            ORD                                Sec 4-H
----            READ_DICT                          Sec 4-E,F
```

Display List from Dictionary File - Overview Continued

```
----            REMOVE_DICT                        Sec 4-I
----            SET_FILE_ATR                       Sec 4-G
----            STR_LEN                            Sec 4-F
----            UNINIT                             Sec 3-C
----            WRITE_DICT                         Sec 4-D,H,I
----    Statements:
----            ABORT                              Sec 3-B
----            CLOSE FILE                         Sec 4-I
----            FOR...ENDFOR                       Sec 4-F
----            IF...THEN...ENDIF                  Sec 3-A,B,C,D; 4-F,H,I
```

```
----            OPEN FILE                           Sec 3-A; 4-H
----            READ                                Sec 4-A,B,H
----            REPEAT...UNTIL                      Sec 4-H
----            ROUTINE                             Sec 3-A,B,C,D,E
----            SELECT...ENDSELECT                  Sec 4-H
----            WRITE                               Sec 3-A,B,C,D;4-A,I
----    Reserve Words:
----            BEGIN                               Sec 3-A,B,C,D;4
----            CR                                  Sec 4-A,B,C
----            END                                 Sec 3-A,B,C,D; 4-I
----            PROGRAM                             Sec 1
----            VAR                                 Sec 2
----    Predefined File Names:
----            TPDISPLAY                           Sec 4-D,G,H,I
----            TPFUNC                              Sec 4-D,H
----            TPPROMPT                            Sec 4-D,H,I
----            TPSTATUS                            Sec 4-D,I
----    Devices Used:
----            RD2U                                Sec 3-B
----    Predefined Windows:
----            ERR                                 Sec 4-C
----            T_ST                                Sec 4-C
----            T_FU                                Sec 4-C
----            T_PR                                Sec 4-C
----            T_FR                                Sec 4-C
```

Display List from Dictionary File - Declaration Section

```
--------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM DCLST_EX
%COMMENT='DISCTRL_LIST '
%ALPHABETIZE
%NOLOCKGROUP
%INCLUDE DCLIST -- the include file from the dictionary DCLISTEG.UTX
--------------------------------------------------------------------------------
----    Section 2:   Variable  Declarations
--------------------------------------------------------------------------------
VAR
  exit_Cmnd      : INTEGER
  act_pending    : INTEGER             -- decide if any action is pending
  display_data   : DISP_DAT_T          -- information needed for DICTRL_LIST
  done           : BOOLEAN             -- decides when to complete execution
  Kb_file        : FILE                -- file opened to the TPKB
  i              : INTEGER             -- just a counter
```

```
  key            : INTEGER              -- which key was pressed
  last_line      : INTEGER              -- number of last line of information
  list_data      : ARRAY[20] OF STRING[40] -- exact string information
  num_options    : INTEGER              -- number of items in list
  old_screen     : STRING[4]            -- previously attached screen
  status         : INTEGER              -- status returned from built-in call
  str            : STRING[1]            -- string read in from teach pendant
  Err_file       : FILE                 -- err log file
  Opened         : BOOLEAN              -- err log file open or not
```

Display List from Dictionary File - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----    Section 3-A: Op_Err_File Declaration
----                  Open the error log file.
--------------------------------------------------------------------------------
Routine Op_Err_File
Begin
  Opened = false
  Write TPPROMPT(CR,'Creating Auto Error File ..............')
  OPEN FILE Err_File ('RW','RD2U:\D_LIST.ERR')  -- open for output
  IF (IO_STATUS(Err_File) <> 0 ) THEN
     CLR_IO_STAT(Err_File)
     Write TPPROMPT('*** USE USER WINDOW FOR ERROR OUTPUT ***',CR)
  ELSE
     Opened = TRUE
  ENDIF
End Op_Err_File
--------------------------------------------------------------------------------
----    Section 3-B: Chk_Add_Dct  Declaration
----                  Check whether a dictionary is loaded.
----                  If not loaded then load in the dictionary.
--------------------------------------------------------------------------------
Routine Chk_Add_Dct
Begin -- Chk_Add_Dct
  -- Make sure 'DLST' dictionary is added.
    CHECK_DICT('DLST',TPTSSP_TITLE,STATUS)
    IF STATUS <> 0 THEN
       Write TPPROMPT(CR,'Loading Required Dictionary.............')
       ADD_DICT('RD2U:\DCLISTEG','DLST',DP_DEFAULT,DP_DRAM,STATUS)
       IF status <> 0 THEN
           WRITE TPPROMPT('ADD_DICT failed, STATUS=',STATUS,CR)
           ABORT  -- Without the dictionary this program can not continue.
```

```
      ENDIF
    ELSE
      WRITE TPPROMPT ('Dictionary already loaded in system.    ')
    ENDIF
End Chk_Add_Dct
```

Display List from Dictionary File - Declare Error Routines

```
--------------------------------------------------------------------------------
----    Section 3-C:  Log_Errors Declaration
----                   Log detected errors to a file to be reviewed later.
--------------------------------------------------------------------------------
ROUTINE Log_Errors(Out: FILE; Err_Str:STRING;Err_No:INTEGER)
BEGIN
  IF NOT Opened THEN  -- If error log file not opened then write errors to
                      -- screen
    WRITE (Err_Str,Err_No,CR)
  ELSE
    IF NOT UNINIT(Out)  THEN
      CLR_IO_STAT(Out)
      WRITE Out(Err_Str,Err_No,CR,CR)
    ELSE
      WRITE (Err_Str,Err_No, CR)
    ENDIF
  ENDIF
END Log_Errors
--------------------------------------------------------------------------------
----    Section 3-D:  Chk_Stat Declaration
----                   Check the global variable, status.
----                   If not zero then log input parameter, err_str,
----                   to error file.
--------------------------------------------------------------------------------
ROUTINE Chk_Stat ( err_str: STRING)
BEGIN -- Chk_Stat
  IF( status <> 0) then
    Log_Errors(Err_File, err_str,Status)
  ENDIF
END Chk_Stat
--------------------------------------------------------------------------------
----    Section 3-E:  TP_CLS Declaration
--------------------------------------------------------------------------------
ROUTINE TP_CLS FROM ROUT_EX
```

Display List from Dictionary File - Setup and Define Screen

```
-------------------------------------------------------------------------------
----    Section 4:  Main Program
-------------------------------------------------------------------------------
BEGIN -- DCLST_EX
-------------------------------------------------------------------------------
----    Section 4-A:  Perform Setup operations
-------------------------------------------------------------------------------
  TP_CLS   -- Call routine to clear and force the TP USER menu to be visible
  Write ('   ***** Starting DISCTRL_LIST Example *****', CR, CR)
  Chk_Add_Dct     -- Call routine to check and add dictionary
  Op_Err_File     -- Call routine open error log file
-------------------------------------------------------------------------------
----    Section 4-B:  Define and Active a screen
-------------------------------------------------------------------------------
  DEF_SCREEN('LIST', 'TP', status)   -- Create/Define a screen called LIST
   Chk_Stat ('DEF_SCREEN LIST')      -- Verify DEF_SCREEN was successful
  ACT_SCREEN('LIST', old_screen, status) -- activate the LIST screen that
                                         -- that was just defined.
   Chk_Stat ('ACT_SCREEN LIST')          -- Verify ACT_SCREEN was successful
-------------------------------------------------------------------------------
----    Section 4-C:  Attach windows to the screen
-------------------------------------------------------------------------------
  -- Attach the required windows to the LIST screen.
  -- SEE:
  --  Chapter 7.9.1 "USER Menu on the Teach Pendant,
  --  for more details on predefined window names.
  ATT_WINDOW_S('ERR', 'LIST', 1, 1, status)  -- attach the error window
    Chk_Stat('Attaching ERR')
  ATT_WINDOW_S('T_ST', 'LIST', 2, 1, status) -- attach the status window
    Chk_Stat('T_ST not attached')
  ATT_WINDOW_S('T_FU', 'LIST', 5, 1, status) -- attach the full window
    Chk_Stat('T_FU not attached')
  ATT_WINDOW_S('T_PR', 'LIST', 15, 1, status)-- attach the prompt window
    Chk_Stat('T_PR not attached')
  ATT_WINDOW_S('T_FK', 'LIST', 16, 1, status)-- attach the function window
    Chk_Stat('T_FK not attached')
```

Display List from Dictionary File - Write Elements to the Screen

```
-------------------------------------------------------------------------------
----    Section 4-D:  Write dictionary elements to windows
-------------------------------------------------------------------------------
    -- Write dictionary element,TPTSSP_TITLE, from DLST dictionary.
    -- Which will clear the status window, and display intro message in
    -- reverse video.
    WRITE_DICT(TPSTATUS, 'DLST', TPTSSP_TITLE, status)
```

```
      Chk_Stat( 'TPTSSP_TITLE not written')
    -- Write dictionary element,TPTSSP_FK1, from DLST dictionary.
    -- Which will display "[TYPE]" to the function line window.
    WRITE_DICT(TPFUNC, 'DLST', TPTSSP_FK1, status)
      Chk_Stat( 'TPTSSP_FK1 not written')
    -- Write dictionary element, TPTSSP_CLRSC, from DLST dictionary.
    -- Which will clear the teach pendant display window.
    WRITE_DICT(TPDISPLAY, 'DLST', TPTSSP_CLRSC, status)
      Chk_Stat( 'TPTSSP_CLRSC not written')
    -- Write dictionary element, TPTSSP_INSTR, from DLST dictionary.
    -- Which will display instructions to the prompt line window.
    WRITE_DICT(TPPROMPT, 'DLST', TPTSSP_INSTR, status)
      Chk_Stat( 'TPTSSP_INSTR not written')
--------------------------------------------------------------------------
----    Section 4-E:   Determine the number of menu options
--------------------------------------------------------------------------
    -- Read the dictionary element, TPTSSP_NUM, from DLST dictionary,
    -- Into the first element of list_data.
    -- list_data[1] will be an ASCII representation of the number of
    --  menu options.  last_line will be returned with the number of
    --  lines/elements used in list_data.
    READ_DICT('DLST', TPTSSP_NUM, list_data, 1, last_line, status)
      Chk_Stat( 'TPTSSP_NUM not read')
    -- convert the string into the INTEGER, num_options
    CNV_STR_INT(list_data[1], num_options)
```

Display List from Dictionary File - Initialize Display Data

```
--------------------------------------------------------------------------
----    Section 4-F:   Initialize the data structure, display_data
----                   Which is used to display the list of menu options.
--------------------------------------------------------------------------
    -- Initialize the display data structure
    -- In this example we only deal with window 1.
    display_data.win_start[1] = 1  -- Starting row for window 1.
    display_data.win_end[1]  = 10  -- Ending row for window 1.
    display_data.curr_win = 0      -- The current window to display, where
                                   --   zero (0) specifies first window.
    display_data.cursor_row = 1    -- Current row the cursor is on.
    display_data.lins_per_pg = 10  -- The number of lines scrolled when the
                                   -- user pushes SHIFT Up or Down. Usually
                                   -- it is the same as the window size.
    display_data.curs_st_col[1] = 0 -- starting column for field 1
    display_data.curs_en_col[1] = 0 -- ending column for field 1, will be
                                    -- updated a little later
    display_data.curr_field = 0     -- Current field, where
```

```
                                    --   zero (0) specifies the first field
    display_data.last_field = 0     -- Last field in the list (only using one
                                    --   field in this example).
    display_data.curr_it_num = 1    -- Current item number the cursor is on.
    display_data.sob_it_num = 1     -- Starting item number.
    display_data.eob_it_num = num_options  -- Ending item number, which is
                                           --  the number of options read in.
    display_data.last_it_num  = num_options-- Last item number, also the
                                            --   number of options read in
 -- Make sure the window end is not beyond total number of elements in list.
    IF display_data.win_end[1] > display_data.last_it_num THEN
      display_data.win_end[1] = display_data.last_it_num --reset to last item
    ENDIF
    -- Read dictionary element, TPTSSP_MENU, from dictionary DLST.
    -- list_data will be populated with the menu list information
    -- list_data[1] will contain the first line of information from
    -- the TPTSSP_MENU and list_data[last_line] will contain the last
    -- line of information read from the dictionary.
    READ_DICT('DLST', TPTSSP_MENU, list_data, 1, last_line, status)
     Chk_Stat('Reading menu list failed')
-- Determine longest list element & reset cursor end column for first field.
    FOR i = 1 TO last_line DO
      IF (STR_LEN(list_data[i]) > display_data.curs_en_col[1]) THEN
        display_data.curs_en_col[1] = STR_LEN(list_data[i])
      ENDIF
    ENDFOR
```

Display List from Dictionary File - Control Cursor Movement

```
-------------------------------------------------------------------------------
----    Section 4-G:   Display the list.
-------------------------------------------------------------------------------
    -- Initial Display the menu list.
    DISCTRL_LIST(TPDISPLAY, display_data, list_data, DC_DISP, status)
      Chk_Stat('Error displaying list')
    -- Open a file to the TPDISPLAY window with PASSALL and FIELD attributes
    -- and NOECHO
    SET_FILE_ATR(kb_file, ATR_PASSALL)  -- Get row teach pendant input
    SET_FILE_ATR(kb_file, ATR_FIELD)    -- so that a single key will
                                        -- satisfy the reads.
    SET_FILE_ATR(kb_file, ATR_NOECHO)   -- don't echo the keys back to
                                        -- the screen
    OPEN FILE Kb_file ('RW', 'KB:TPKB') -- open a file to the Teach pendant
                                        -- keyboard (keys)
      status = IO_STATUS(Kb_file)
      Chk_Stat('Error opening TPKB')
```

```
    act_pending = 0
    done = FALSE
--------------------------------------------------------------------------------
----    Section 4-H:  Control cursor movement within the list
--------------------------------------------------------------------------------
    REPEAT      -- Wait for a key input
      READ Kb_file (str::1)
      key = ORD(str,1)
      key = key AND 255           -- Convert the key to correct value.
      SELECT key OF               -- Decide how to handle key inputs
       CASE (KY_UP_ARW) :         -- up arrow key pressed
        IF act_pending <> 0 THEN  -- If a menu item was selected...
                                  -- Clear confirmation prompt
          WRITE_DICT(TPPROMPT, 'DLST', TPTSSP_CLRSC, status)
             -- Clear confirmation function keys
          WRITE_DICT(TPFUNC, 'DLST', TPTSSP_CLRSC, status)
        ENDIF
        DISCTRL_LIST(TPDISPLAY, display_data, list_data, DC_UP, status)
          Chk_Stat ('Error displaying list')
       CASE (KY_DN_ARW) : -- down arrow key pressed
        IF act_pending <> 0 THEN -- If a menu item was selected...
          -- Clear confirmation prompt
          WRITE_DICT(TPPROMPT, 'DLST', TPTSSP_CLRSC, status)
          -- Clear confirmation function keys
          WRITE_DICT(TPFUNC, 'DLST', TPTSSP_CLRSC, status)
        ENDIF
        DISCTRL_LIST(TPDISPLAY, display_data, list_data,
DC_DN, status)
           Chk_Stat ('Error displaying list')
```

Display List from Dictionary File - Control Cursor Movement Continued

```
CASE (KY_ENTER) :
        -- Perform later
CASE (KY_F4) : -- "YES" function key pressed
       IF act_pending <> 0 THEN -- If a menu item was selected...
          -- Clear confirmation prompt
          WRITE_DICT(TPPROMPT, 'DLST', TPTSSP_CLRSC, status)
          -- Clear confirmation function keys
          WRITE_DICT(TPFUNC, 'DLST', TPTSSP_CLRSC, status)
          IF act_pending = num_options THEN
            -- Exit the routine
            done = TRUE
          ENDIF
          -- Clear action pending
          act_pending = 0
```

```
        ENDIF
CASE (KY_F5) : -- "NO" function key pressed
               -- Clear confirmation prompt
      WRITE_DICT(TPPROMPT, 'DLST', TPTSSP_INSTR, status)
               -- Clear confirmation function keys
      WRITE_DICT(TPFUNC, 'DLST', TPTSSP_CLRSC, status)
               -- Clear action pending
       act_pending = 0
ELSE :         -- User entered an actual item number.  Calculate which
               -- row the cursor should be on and redisplay the list.
       IF ((key > 48) AND (key <= (48 + num_options))) THEN
         -- Translate number to a row
         key = key - 48
         display_data.cursor_row = key
        DISCTRL_LIST(TPDISPLAY,display_data,list_data,DC_DISP,status)
          Chk_Stat ('Error displaying list')
         key = KY_ENTER
       ENDIF
ENDSELECT
IF key = KY_ENTER THEN -- User has specified an action
  -- Write confirmation prompt for selected item
  WRITE_DICT (TPPROMPT, 'DLST',
     (TPTSSP_CNF1 - 1 + display_data.cursor_row), status)
  -- Display confirmation function keys
  WRITE_DICT(TPFUNC, 'DLST', TPTSSP_FK2, status)
  -- Set action pending to selected item
  act_pending = display_data.cursor_row -- this is the item selected
ENDIF
UNTIL done   -- repeat until the user selects the exit option
```

Display List from Dictionary File - Cleanup and Exit Program

```
--------------------------------------------------------------------------------
----    Section 4-I:  Cleanup before exiting program
--------------------------------------------------------------------------------
  -- Clear the TP USER menu windows
  WRITE_DICT(TPDISPLAY, 'DLST', TPTSSP_CLRSC, status)
  WRITE_DICT(TPSTATUS, 'DLST', TPTSSP_CLRSC, status)
  -- Close the file connected to the TP keyboard.
  CLOSE FILE Kb_file
  -- Close the error log file if it is open.
  IF opened THEN
     close file Err_File
  ENDIF
  Write TPPROMPT (cr,'Example Finished ')
  REMOVE_DICT ( 'LIST', dp_default, status) -- remove the dictionary
```

```
    write ('remove dict', status,cr)
    Chk_Stat ('Removing dictionary')
  ACT_SCREEN(old_screen, old_screen, status) -- activate the previous screen
    Chk_stat ('Activating old screen')
  DET_WINDOW('ERR', 'LIST', status)  -- Detach all the windows that were
    Chk_stat ('Detaching ERR window')
  DET_WINDOW('T_ST', 'LIST', status) -- previously attached.
    Chk_stat ('Detaching T_ST window')
  DET_WINDOW('T_FU', 'LIST', status)
    Chk_stat ('Detaching T_FU window')
  DET_WINDOW('T_PR', 'LIST', status)
    Chk_stat ('Detaching T_PR window')
  DET_WINDOW('T_FK', 'LIST', status)
    Chk_stat ('Detaching T_FK window')
END DCLST_EX
```

B.12.1 Dictionary Files

This ASCII dictionary file is used to create a teach pendant screen which is used with DCLST_EX.KL. For more information on DCLST_EX.KL refer to Section B.7 .

Dictionary File

```
.kl dclist
*
$-,TPTSSP_TITLE &home &reverse "Karel DISCTRL_LIST Example
"
&standard
$-,TPTSSP_CLRSC &home &clear_2_eow
$-,TPTSSP_FK1 &home" [TYPE]                                    "
$-,TPTSSP_FK2 &home"                          YES      NO    "
$-,TPTSSP_INSTR &home "Press 'ENTER' or number key to select." &clear_2_eol
* Add menu options here, "Exit" must be last option
* TPTSSP_NUM specifies the number of menu options
$-,TPTSSP_NUM "14"
$-,TPTSSP_MENU
" 1  Test Cycle 1" &new_line
" 2  Test Cycle 2" &new_line
" 3  Test Cycle 3" &new_line
" 4  Test Cycle 4" &new_line
" 5  Test Cycle 5" &new_line
" 6  Test Cycle 6" &new_line
" 7  Test Cycle 7" &new_line
" 8  Test Cycle 8" &new_line
```

```
" 9  Test Cycle 9" &new_line
" 10  Test Cycle 10" &new_line
" 11  Test Cycle 11" &new_line
" 12  Test Cycle 12" &new_line
" 13  Test Cycle 13" &new_line
" 14  EXIT"
* Confirmations must be in order
$-,TPTSSP_CNF1 &home"Perform test cycle 1? [NO]" &clear_2_eol
$-,TPTSSP_CNF2 &home"Perform test cycle 2? [NO]" &clear_2_eol
$-,TPTSSP_CNF3 &home"Perform test cycle 3? [NO]" &clear_2_eol
$-,TPTSSP_CNF4 &home"Perform test cycle 4? [NO]" &clear_2_eol
$-,TPTSSP_CNF5 &home"Perform test cycle 5? [NO]" &clear_2_eol
$-,TPTSSP_CNF6 &home"Perform test cycle 6? [NO]" &clear_2_eol
$-,TPTSSP_CNF7 &home"Perform test cycle 7? [NO]" &clear_2_eol
$-,TPTSSP_CNF8 &home"Perform test cycle 8? [NO]" &clear_2_eol
$-,TPTSSP_CNF9 &home"Perform test cycle 9? [NO]" &clear_2_eol
$-,TPTSSP_CNF10 &home"Perform test cycle 10? [NO]" &clear_2_eol
$-,TPTSSP_CNF11 &home"Perform test cycle 11? [NO]" &clear_2_eol
$-,TPTSSP_CNF12 &home"Perform test cycle 12? [NO]" &clear_2_eol
$-,TPTSSP_CNF13 &home"Perform test cycle 13? [NO]" &clear_2_eol
$-,TPTSSP_CNF14 &home"Exit? [NO]" &clear_2_eol
```

B.13 USING THE DISCTRL_ALPHA BUILT-IN

This program shows three different ways to use the DISCTRL_ALPH built-in. The DISCTRL_ALPH built-in displays and controls alphanumeric string entry in a specified window. Refer to Appendix A , "KAREL Language Alphabetic Description" for more information.

Method 1 allows a program name to be entered using the default value for the dictionary name. See Section 4-A in Using the DISCTRL_ALPHA Built-in - Enter Data from Teach Pendant .

Method 2 allows a comment to be entered using the default value for the dictionary name. See Section 4-B in Using the DISCTRL_ALPHA Built-in - Enter Data from Teach Pendant .

Method 3 uses a user specified dictionary name and element to enter a program name. See Section 4-C in Using the DISCTRL_ALPHA Built-in - Enter Data from CRT/KB .

This program also posts all errors to the controller.

Using the DISCTRL_ALPHA Built-in - Overview

```
------------------------------------------------------------------------------
----    DCALP_EX.KL
------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:          In Section:
----    Actions:
```

```
----    Clauses:
----    Conditions:
----    Data types:
----            INTEGER                              Sec 2
----            STRING                               Sec 2
----    Directives:
----            COMMENT                             Sec 1
----            INCLUDE                             Sec 1
----            NOLOCKGROUP                         Sec 1
----    Built-in Functions & Procedures:
----            ADD_DICT                             Sec 4-C
----            CHR                                  Sec 4-A,B,C
----            DISCTRL_ALPH                         Sec 4-A,B,C
----            FORCE_SPMENU                         Sec 4-A,B,C
----            POST_ERR                             Sec 4-A,B,C
----            SET_CURSOR                           Sec 4-A,B,C
----            SET_LANG                             Sec 4-C
----    Statements:
----            READ                                 Sec 4-A,B
----            WRITE                                Sec 4-A,B,C
----            IF...THEN...ENDIF                    Sec 4-A,B,C
----    Reserve Words:
----            BEGIN                                Sec 4
----            CONST                                Sec 2
----            CR                                   Sec 4-A,B,C
----            END                                  Sec 4-C
----            PROGRAM                              Sec 1
----            VAR                                  Sec 2
----    Predefined File Names:
----            OUTPUT                               Sec 4-C
----            TPDISPLAY                            Sec 4-A,B
----     Predefined Windows:
----            T_FU                                 Sec 4-A,B
----            C_FU                                 Sec 4-C
```

Using the DISCTRL_ALPHA Built-in - Declaration Section

```
-------------------------------------------------------------------------------
----    Section 1:  Program and Environment Declaration
-------------------------------------------------------------------------------
PROGRAM DCALP_EX
%COMMENT   = 'Display Alpha'
%NOLOCKGROUP
%INCLUDE KLEVKEYS    -- Necessary for the KY_ENTER
%INCLUDE DCALPH      -- Necessary for the ALPH_PROG Element, see section 4-C
-------------------------------------------------------------------------------
```

```
----    Section 2:  Constant and Variable  Declarations
--------------------------------------------------------------------------------
CONST
  cc_home      = 137
  cc_clear_win = 128
  cc_warn      = 0     -- Value passed to POST_ERR to display warning only.
  cc_pause     = 1     -- value passed to POST_ERR to pause program.
VAR
  status       : INTEGER
  device_stat  : INTEGER
  term_char    : INTEGER
  window_name  : STRING[4]
  prog_name    : STRING[12]
  comment      : STRING[40]
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
```

Using the DISCTRL_ALPHA Built-in - Enter Data from Teach Pendant

```
--------------------------------------------------------------------------------
----    Section 4:  Main Program
--------------------------------------------------------------------------------
BEGIN   -- DCALP_EX
--------------------------------------------------------------------------------
---- Section 4-A: Enter a program name from the teach pendant USER menu
--------------------------------------------------------------------------------
  WRITE (CHR(cc_home), CHR(cc_clear_win))   -- Clear TP USER menu
  FORCE_SPMENU(tp_panel, SPI_TPUSER, 1)     -- Force TP USER menu to be
                                            -- visible
  SET_CURSOR(TPDISPLAY, 12, 1, status)      -- reposition cursor
  WRITE ('prog_name: ')
  prog_name = ''                            -- initialize program name
  DISCTRL_ALPH('t_fu', 12, 12, prog_name, 'PROG', 0, term_char, status)
  IF status <> 0 THEN
    POST_ERR(status, '', 0, cc_warn)
  ENDIF
  IF term_char = ky_enter THEN              -- User pressed the ENTER key
    WRITE (CR, 'prog_name was changed:', prog_name, CR)
  ELSE
    WRITE (CR, 'prog_name was not changed')
  ENDIF
  WRITE (CR, 'Press enter to continue')
  READ (CR)
--------------------------------------------------------------------------------
---- Section 4-B: Enter a comment from the teach pendant
```

```
-------------------------------------------------------------------------------
  WRITE (CHR(cc_home) + CHR(cc_clear_win))-- Clear TP USER menu
  SET_CURSOR(TPDISPLAY, 12, 1, status)   -- reposition cursor
  comment = ' '                          -- Initialize the comment
  WRITE ('comment: ')                    -- Display message
  DISCTRL_ALPH('t_fu', 12, 10, comment, 'COMM', 0, term_char, status)
  IF status <> 0 THEN                    -- Verify DISCTRL_ALPH was successful
    POST_ERR(status, '', 0, cc_warn)     -- Post the status as a warning
  ENDIF
  IF term_char = ky_enter THEN
    WRITE (CR, 'comment was changed:', comment, CR) -- Display new comment
  ELSE
    WRITE (CR, 'comment was not changed', CR)
  ENDIF
  WRITE (CR, 'Press enter to continue')
  READ (CR)
```

Using the DISCTRL_ALPHA Built-in - Enter Data from CRT/KB

```
-------------------------------------------------------------------------------
---- Section 4-C: This section will perform program name entry from the
----              CRT/KB.  The dictionary name and element values are
----              explicitly stated here, instead of using the available
----               default values.
-------------------------------------------------------------------------------
  -- Set the dictionary language to English
  -- This is useful if you want to use this same code for multiple
  -- languages.  Then any time you load in a dictionary you check
  -- to see what the current language, $language, is and load the
  -- correct dictionary.
  -- For instance you could have a DCALPHJP.TX file which
  -- would be the Japanese dictionary. If the current language, $language,
  -- was set to Japanese you would load this dictionary.
  SET_LANG ( dp_english, status)
  IF (status <> 0) THEN
    POST_ERR (status, '', 0,cc_warn)   -- Post the status as a warning
  ENDIF
  -- Load the dcalpheg.tx file, using ALPH as the dictionary name,
  --  to the English language, using DRAM as the memory storage device.
  ADD_DICT ('DCALPHEG', 'ALPH', dp_english,  dp_dram, status)
  IF (status <> 0 ) THEN
    POST_ERR (status, '', 0, cc_pause)     -- Post the status and pause the
                                           -- program, since the dictionary
  ENDIF                                    -- must be loaded to continue.
 device_stat = crt_panel                     -- Give control to the CRT/KB
  WRITE OUTPUT (CHR(cc_home) + CHR(cc_clear_win))--Clear CRT/KB USER menu
```

```
  FORCE_SPMENU (device_stat, SPI_TPUSER, 1) -- Force the CRT/KB USER menu
                                            --  to be visible
  SET_CURSOR(OUTPUT, 12, 1, status)         -- Reposition the cursor
  WRITE OUTPUT ('prog_name: ')
  prog_name = ' '                           -- Initialize program name
  DISCTRL_ALPH('c_fu',12,12,prog_name,'ALPH',alph_prog,term_char,status)
--DISCTRL_ALPHA uses the ALPH dictionary and ALPH_PROG dictionary element
  IF status <> 0 THEN                       -- Verify DISCTRL_ALPH was
                                            -- successful.
    POST_ERR(status, '', 0, cc_warn)        -- Post returned status to the
  ENDIF                                     -- error ('err') window.
  IF term_char = ky_enter THEN
    WRITE (CR, 'prog_name was changed:', prog_name, CR)
  ELSE
    WRITE (CR, 'prog_name was NOT changed.', CR)
  ENDIF
  device_stat = tp_panel                     -- Make sure to reset
 END DCALP_EX
```

B.13.1 Dictionary Files

This ASCII dictionary file is used to write text to the specified screen. DCALPH_EG.UTX is also used with DCAL_EX.KL. For more information on DCAL_EX.KL, refer to Section B.13 .

DCALPHEG.UTX Dictionary File

```
.KL DCALPH
$, alpha_prog
" PRG       MAIN     SUB       TEST     >"&new_line
" PROC      JOB      MACRO               >"&new_line
" TEST1     TEST2    TEST3     TEST4    >"
```

Note The greater than symbol (>) in DCALPHEG.UTX Dictionary File is a reminder to use the NEXT key to scroll through the multiple lines. Also notice that the &new_line appears only on the first two lines. This ensures that the lines will scroll correctly.

B.14 APPLYING OFFSETS TO A COPIED TEACH PENDANT PROGRAM

This program copies a teach pendant program and then applies an offset to the positions within the newly created program. This is useful when you create the teach pendant program offline and then

realized that all the teach pendant positions are off by some determined amount. However, you should be aware that the utility PROGRAM ADJUST is far more adequate for this job.

Applying Offsets to Copied Teach Pendant Program - Overview

```
--------------------------------------------------------------------------------
----    CPY_TP.KL
--------------------------------------------------------------------------------
---- Elements of KAREL Language Covered:         In Section:
----    Actions:
----
----    Clauses:
----            FROM                              Sec 3-F
----
----    Conditions:
----
----    Data Types:
----            ARRAY OF REAL                      Sec 2
----            ARRAY OF STRING                    Sec 2
----            BOOLEAN                            Sec 2
----            INTEGER                            Sec 2; 3-B,C,D,E
----            JOINTPOS                           Sec 2
----            REAL                               Sec 2
----            STRING                             Sec 2
----            XYZWPR                             Sec 2
----    Directives:
----            ENVIRONMENT                       Sec 1
----    Built-in Functions & Procedures:
----            AVL_POS_NUM                        Sec 3-E
----            CHR                                Sec 3-B, 4
----            CLOSE_TPE                          Sec 3-E
----            CNV_REL_JPOS                       Sec 3-E
----            CNV_JPOS_REL                       Sec 3-E
----            COPY_TPE                           Sec 3-E
----            GET_JPOS_TPE                       Sec 3-E
----            GET_POS_TYP                        Sec 3-E
----            GET_POS_TPE                        Sec 3-E
----            OPEN_TPE                           Sec 3-E
----            PROG_LIST                          Sec 3-B
----            SELECT_TPE                         Sec 3-E
----            SET_JPOS_TPE                       Sec 3-E
----            SET_POS_TPE                        Sec 3-E
```

Applying Offsets to Copied Teach Pendant Program - Overview Continued

```
----    Statements:
```

```
----            FOR...ENDFOR                          Sec 3-B,D,E
----            IF ...THEN...ELSE...ENDIF             Sec 3-A,B,C,E
----            READ                                  Sec 3-B,C,D
----            REPEAT...UNTIL                        Sec 3-B,C,D
----            RETURN                                Sec 3-E
----            ROUTINE                               Sec 3-A,B,C,D,E,F
----            SELECT...ENDSELECT                    Sec 3-E
----            WRITE                                 Sec 3-B,C,D,E,4
----
----    Reserve Words:
----            BEGIN                                 Sec 3-A,B,C,D,E;
4
----            CONST                                 Sec 2
----            CR                                    Sec 3-B,C,D,E
----            END                                   Sec 3-A,B,C,D,E;
4
----            PROGRAM                               Sec 1
----            VAR                                   Sec 2
```

Applying Offsets to Copied Teach Pendant Program - Declaration Section

```
--------------------------------------------------------------------------------
----   Section 1:  Program and Environment Declaration
--------------------------------------------------------------------------------
PROGRAM CPY_TP
%ENVIRONMENT TPE             -- necessary for all xxx_TPE built-ins
%ENVIRONMENT BYNAM           -- necessary for PROG_LIST built-in
--------------------------------------------------------------------------------
----    Section 2:  Constant and Variable Declaration
--------------------------------------------------------------------------------
CONST
  ER_WARN = 0                    -- warning constant for use in POST_ERR
  SUCCESS = 0                    -- success constant
  JNT_POS = 9                    -- constant for GET_POS_TYP
  XYZ_POS = 2                    -- constant for GET_POS_TYP
  MAX_AXS = 9                    -- Maximum number of axes JOINTPOS has
VAR
 from_prog: STRING[13]           -- TP program name to be copied FROM
 to_prog  : STRING[13]           -- TP program name to be copied TO
 over_sw  : BOOLEAN              -- Decides whether to overwrite an existing
                                 --  program when performing COPY_TPE
 status   : INTEGER              -- Holds error status from the builtin calls
 off_xyz  : XYZWPR               -- Offset amount for the XYZWPR positions
 jp_off   : ARRAY [9] of REAL    -- Offset amount for the JOINT positions
 new_xyz  : XYZWPR               -- XYZWPR which has offset applied
 org_xyz  : XYZWPR               -- Original XYZWPR from to_prog
```

```
 new_jpos : JOINTPOS             -- JOINTPOS which as the offset applied
 org_jpos : JOINTPOS             -- Original JOINTPOS from to_prog
 open_id  : INTEGER              -- Identifier for the opened to_prog
 jp_org   : ARRAY [9] of REAL    -- REAL representation of org_jpos
 jp_new   : ARRAY [9] of REAL    -- REAL representation of jp_new
```

Applying Offsets to Copied Teach Pendant Program - Declare Routines

```
--------------------------------------------------------------------------------
----    Section 3:  Routine Declaration
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
----    Section 3-A: CHK_STAT Declaration
----                 Tests whether the status was successful or not.
----                 If the status was not successful the status is posted
--------------------------------------------------------------------------------
ROUTINE chk_stat (rec_stat: integer)
begin
 IF (rec_stat <> SUCCESS) THEN   -- if rec_stat is not SUCCESS
     -- then post the error
  POST_ERR (rec_stat, '', 0, ER_WARN)   -- Post the error to the system.
 ENDIF
END chk_stat
--------------------------------------------------------------------------------
----    Section 3-B: GetFromPrg Declaration
----                 Generate a list of loaded TPE programs.
----                 Lets the user select one of these programs
----                 to be the program to be copied, ie FROM_prog
--------------------------------------------------------------------------------
ROUTINE GetFromPrg
VAR
 tp_type   : INTEGER         -- Types of program to list
 n_skip    : INTEGER         -- Index into the list of programs
 format    : INTEGER         -- What type of format to store programs in
 n_progs   : INTEGER         -- Number of programs returned in prg_name
 prg_name  : ARRAY [8] of STRING[20] --Program names returned from PROG_LIST
 status    : INTEGER         -- Status of PROG_LIST call
 f_index   : INTEGER         -- Fast index for generating the program listing.
 arr_size  : INTEGER         -- Array size of prg_name
 prg_select: INTEGER         -- Users selection for which program to copy
 indx      : INTEGER         -- FOR loop counter which displays prg_name
```

Applying Offsets to Copied Teach Pendant Program - Generate Program List for User

```
BEGIN
```

```
  f_index = 0    -- Initialize the f_index
  n_skip  = 0    -- Initialize the n_skip
  tp_type = 2    -- find any TPE program
  format  = 1    -- return just the program name in prg_name
  n_progs = 0    -- Initialize the n_progs
  arr_size = 8   -- Set equal to the declared array size of prg_name
  prg_select = 0 -- Initialize the program selector
  REPEAT
     WRITE (chr(128),chr(137))  -- Clear the TP USER screen
     -- Get a listing of all TP program which begin with "TEST"
    PROG_LIST('TEST*',tp_type,n_skip,format,prg_name,n_progs,status,f_index)
     chk_stat (status)    --Check status from PROG_LIST
     FOR indx = 1 to n_progs DO
      WRITE (indx,':',prg_name[indx], CR) -- Write the list of programs out
     ENDFOR
     IF (n_skip > 0) OR ( n_prog >0) THEN
       WRITE ('select program to be copied:',CR)
       WRITE ('PRESS -1 to get next page of programs:')
       REPEAT
         READ (prg_select)  -- get program selection
       UNTIL ((prg_select >= -1) AND (prg_select <= n_progs)
                AND (prg_select <> 0))
     ELSE
       WRITE ('no TP programs to COPY', CR)
       WRITE ('Aborting program, since need',CR)
       WRITE ('at least one TP program to copy.',CR)
       ABORT
     ENDIF
     -- Check if listing is complete and user has not made a selection.
     IF ((prg_select = -1) AND (n_progs < arr_size)) THEN
       f_index = 0               --reset f_index to re-generate list.
       n_progs = arr_size       --set so the REPEAT/UNTIL will continue
     ENDIF
     -- Check if user user has made a selection
     IF (prg_select <> -1) then
       from_prog = prg_name[prg_select]-- Set from_prog to name selected
       n_progs = 0                -- Set n_prog to stop looping.
     ENDIF
  UNTIL (n_progs < arr_size)
END GetFromPrg
```

Applying Offsets to Copied Teach Pendant Program - Overwrite or Delete Program

```
--------------------------------------------------------------------------------
----    Section 3-C: GetOvrSw Declaration
----                 Ask user whether to overwrite the copied
```

```
----                  program, TO_prog, if it exists.
--------------------------------------------------------------------------------
ROUTINE GetOvrSw
VAR
  yesno  : INTEGER
BEGIN
  WRITE (CR, 'If Program already exists do you want',CR)
  WRITE ('to overwrite the file Yes:1, No:0 ? ')
  REPEAT
    READ (yesno)
  UNTIL ((yesno = 0) OR( yesno = 1))
  IF yesno = 1 then   --Set over_sw so program is overwriten if it exists
     over_sw = TRUE
  ELSE                --Set over_sw so program is NOT overwriten if it exists
     over_sw = FALSE
  ENDIF
END GetOvrSw
```

Applying Offsets to Copied Teach Pendant Program - Input Offset Positions

```
--------------------------------------------------------------------------------
----    Section 3-D: GetOffset Declaration
----                 Have the user input the offset for both
----                 XYZWPR and JOINTPOS positions.
--------------------------------------------------------------------------------
ROUTINE GetOffset
VAR
 yesno  : INTEGER
 index   : INTEGER
BEGIN
     --Get the XYZWPR offset, off_xyz
 REPEAT
   WRITE ( 'Enter offset for XYZWPR positions',CR)
   WRITE (' X = ')
   READ  (off_xyz.x)
   WRITE (' Y = ')
   READ  (off_xyz.y)
   WRITE (' Z = ')
   READ  (off_xyz.z)
   WRITE (' W = ')
   READ  (off_xyz.w)
   WRITE (' P = ')
   READ  (off_xyz.p)
   WRITE (' R = ')
   READ  (off_xyz.r)
   --Display the offset values the user input
```

```
   WRITE (' Offset XYZWPR position is',CR, off_xyz,CR)
   WRITE ('Is this offset correct? Yes:1, No:0 ? ')
   READ  (yesno)
 UNTIL (yesno = 1)    -- enter offset amounts until the user
    -- is satisfied.
 --Get the JOINTPOS offset, jp_off
 REPEAT
   WRITE ( 'Enter offset for JOINT positions',CR)
   FOR indx = 1 TO 6 DO     -- loop for number of robot axes
      WRITE (' J',indx,' = ')
      READ  (jp_off[indx])
   ENDFOR   WRITE ('JOINT position offset is', CR)
   FOR indx = 1 TO 6 DO
      write ( jp_off[indx],CR)    -- Display the values the user input
   ENDFOR
   WRITE ('Is this offset correct? Yes:1, No:0 ? ')
   READ  (yesno)
 UNTIL (yesno = 1)    -- Enter offset amounts until the user
END GetOffset   -- is satisfied
```

Applying Offsets to Copied Teach Pendant Program - Apply Offsets to Positions

```
--------------------------------------------------------------------------------
----    Section 3-E: ModifyPrg Declaration
----                 Apply the offsets to each position within the TP program
--------------------------------------------------------------------------------
ROUTINE ModifyPrg
VAR
  pos_typ : INTEGER  --The type of position returned from GET_POS_TYP
  num_axs : INTEGER  -- The number of axes if position is a JOINTPOS type
  indx_pos: INTEGER  -- FOR loop counter, that increments through TP position
  group_no: INTEGER  -- The group number of the current position setting.
  num_pos : INTEGER  -- The next available position number within TP program
  indx_axs: INTEGER  -- FOR loop counter, increments through REAL array
BEGIN
  SELECT_TPE ('', status)  -- Make sure the to_prog is currently not selected
  to_prog = 'MDFY_TP'      -- Set the to_prog to desired name.
  ------ Copy the from_prog to to_prog -----
  COPY_TPE (from_prog, to_prog, over_sw, status)
  chk_stat(status)           -- check status of COPY_TPE
--- If the user specified not to overwrite the TPE program and
---  the status returned is 7015, "program already exist",
---  then quit the program.  This will mean not altering the already
---  existing to_prog.
  IF ((over_sw = FALSE) AND (status = 7015)) THEN
      WRITE ('ABORTING:: PROGAM ALREADY EXISTS!',CR)
```

```
      RETURN
  ENDIF
--- Open the to_prog with the Read/Write access
 OPEN_TPE (to_prog, TPE_RWACC, TPE_RDREJ, open_id, status)
 chk_stat(status)              -- check status of OPEN_TPE
 group_no = 1
--- apply offset to each position within to_prog
--- The current number of position that the TPE program has is num_pos -1
 FOR indx_pos = 1 to num_pos-1 DO
   --Get the DATA TYPE of each position within the to_prog
   --If it is a JOINTPOS also get the number of axes.
   GET_POS_TYP (open_id, indx_pos, group_no, pos_typ, num_axs, status)
   chk_stat (status)
   WRITE('get_pos_typ status', status,cr)
```

Applying Offsets to Copied Teach Pendant Program - Apply Offsets to Positions Cont.

```
--Decide if the position, indx_pos, is a JOINTPOS or a XYZWPR
   SELECT pos_typ OF
     CASE (JNT_POS):       -- The position is a JOINTPOS
        FOR indx_axs = 1 TO MAX_AXS DO    -- initialize with default values
          jp_org[indx_axs] = 0.0          -- This avoids problems with the
          jp_new[indx_axs] = 0.0          -- CNV_REL_JPOS
        ENDFOR
        -- get the JOINTPOS P[indx_pos] from to_prog  -----
        org_jpos = GET_JPOS_TPE (open_id, indx_pos, status)
        chk_stat( status)
        -- Convert the JOINTPOS to a REAL array, in order to perform offset
        CNV_JPOS_REL (org_jpos,  jp_org, status)
        chk_stat (status)
        -- Apply the offset to the REAL array
        FOR indx_axs = 1 to num_axs DO
          jp_new[indx_axs] = jp_org[indx_axs] + jp_off[indx_axs]
        ENDFOR
        -- Converted back to a JOINTPOS.
        -- The input array, jp_new, must not have any uninitialized values
        -- or the error 12311 - "Data uninitialized" will be posted.
        -- This is why we previously set all the values to zero.
        CNV_REL_JPOS (jp_new, new_jpos, status)
        chk_stat(status)
        -- Set the new offset position, new_jpos, into the indx_pos
        SET_JPOS_TPE (open_id, indx_pos, new_jpos, status)
        chk_stat(status)
        write ('indx_pos', indx_pos, 'new_jpos',cr, new_jpos,cr)
      CASE (XYZ_POS): -- The position is a XYZWPR
         -- Get the XYZWPR position P[indx_pos] from to_prog
```

```
        org_xyz = GET_POS_TPE (open_id , indx_pos, status)
        chk_stat( status)               -- Check status from GET_POS_TPE
```

Applying Offsets to Copied Teach Pendant Program - Clears TP User Menu

```
-- Apply offset to the XYZWPR
        new_xyz.x = org_xyz.x + off_xyz.x
        new_xyz.y = org_xyz.y + off_xyz.y
        new_xyz.z = org_xyz.z + off_xyz.z
        new_xyz.w = org_xyz.w + off_xyz.w
        new_xyz.p = org_xyz.p + off_xyz.p
        new_xyz.r = org_xyz.r + off_xyz.r
         --Set the new offset position, new_xyz, into the indx_pos
        SET_POS_TPE (open_id, indx_pos, new_xyz, status)
        chk_stat (status)             -- Check status from SET_POS_TPE
   ENDSELECT
 ENDFOR
 ---Close TP program before quitting program
 CLOSE_TPE (open_id, status)
 chk_stat (status)                   --Check status from CLOSE_TPE
END ModifyPrg
--------------------------------------------------------------------------------
----    Section 3-F:  TP_CLS Declaration
----                  Clears the TP USER Menu screen and forces it to
----                  become visible.  The actual code resides in
ROUT_EX.KL
--------------------------------------------------------------------------------
ROUTINE TP_CLS FROM rout_ex
--------------------------------------------------------------------------------
----    Section 4:  Main Program
--------------------------------------------------------------------------------
BEGIN  -- CPY_TP
 tp_cls                             -- Clear the TP USER Menu screen
 GetFromPrg                         -- Get the TPE program to copy FROM
 GetOvrSw                           -- Get the TPE program name to copy TO
 GetOffset                          -- Get the offset for modifying
                                    -- the teach pendant program
 ModifyPrg                         -- Modify the copied program by the offset
END CPY_TP
```

附录C.

KCL 命令字母描述

目录

附录C KCL命令字母描述 C–1
C.1 ABORT command C–6
C.2 APPEND FILE command C–6
C.3 APPEND NODE command C–6
C.4 CHDIR command C–7
C.5 CLEAR ALL command C–7
C.6 CLEAR BREAK CONDITION command C–8
C.7 CLEAR BREAK PROGRAM command C–8
C.8 CLEAR DICT command C–9
C.9 CLEAR PROGRAM command C–9
C.10 CLEAR VARS command C–9
C.11 COMPRESS DICT command C–10
C.12 COMPRESS FORM command C–10
C.13 CONTINUE command C–11
C.14 COPY FILE command C–11
C.15 CREATE VARIABLE command C–12
C.16 DELETE FILE command C–13
C.17 DELETE NODE command C–13
C.18 DELETE VARIABLE command C–14
C.19 DIRECTORY command C–14
C.20 DISABLE BREAK PROGRAM command C–15

C.21 DISABLE CONDITION command C–15
C.22 DISMOUNT command C–15
C.23 EDIT command C–16
C.24 ENABLE BREAK PROGRAM C–16
C.25 ENABLE CONDITION command C–16
C.26 FORMAT command C–17
C.27 HELP command C–17
C.28 HOLD command C–18
C.29 INSERT NODE command C–18
C.30 LOAD ALL command C–18
C.31 LOAD DICT command C–19
C.32 LOAD FORM command C–19
C.33 LOAD MASTER command C–20
C.34 LOAD PROGRAM command C–20
C.35 LOAD SERVO command C–21
C.36 LOAD SYSTEM command C–21
C.37 LOAD TP command C–22
C.38 LOAD VARS command C–23
C.39 LOGOUT command C–24
C.40 MKDIR command C–24
C.41 MOUNT command C–24
C.42 MOVE FILE command C–25
C.43 PAUSE command C–25
C.44 PURGE command C–26
C.45 PRINT command C–26
C.46 RECORD command C–27
C.47 RENAME FILE command C–27
C.48 RENAME VARIABLE command C–28
C.49 RENAME VARS command C–28
C.50 RESET command C–29
C.51 RMDIR command C–29
C.52 RUN command C–30
C.53 RUNCF command C–30
C.54 SAVE MASTER command C–31
C.55 SAVE SERVO command C–31
C.56 SAVE SYSTEM command C–31
C.57 SAVE TP command C–32
C.58 SAVE VARS command C–33
C.59 SET BREAK CONDITION command C–33
C.60 SET BREAK PROGRAM command C–34
C.61 SET CLOCK command C–35

C.62 SET DEFAULT command C–35
C.63 SET GROUP command C–35
C.64 SET LANGUAGE command C–36
C.65 SET LOCAL VARIABLE command C–36
C.66 SET PORT command C–37
C.67 SET TASK command C–37
C.68 SET TRACE command C–38
C.69 SET VARIABLE command C–38
C.70 SET VERIFY command C–39
C.71 SHOW BREAK command C–39
C.72 SHOW BUILTINS command C–40
C.73 SHOW CONDITION command C–40
C.74 SHOW CLOCK command C–40
C.75 SHOW CURPOS command C–41
C.76 SHOW DEFAULT command C–41
C.77 SHOW DEVICE command C–41
C.78 SHOW DICTS command C–41
C.79 SHOW GROUP command C–41
C.80 SHOW HISTORY command C–42
C.81 SHOW LANG command C–42
C.82 SHOW LANGS command C–42
C.83 SHOW LOCAL VARIABLE command C–42
C.84 SHOW LOCAL VARS command C–44
C.85 SHOW MEMORY command C–44
C.86 SHOW PROGRAM command C–44
C.87 SHOW PROGRAMS command C–45
C.88 SHOW SYSTEM command C–45
C.89 SHOW TASK command C–45
C.90 SHOW TASKS command C–46
C.91 SHOW TRACE command C–46
C.92 SHOW TYPES command C–46
C.93 SHOW VARIABLE command C–47
C.94 SHOW VARS command C–47
C.95 SHOW data_type command C–48
C.96 SIMULATE command C–48
C.97 SKIP command C–49
C.98 STEP OFF command C–50
C.99 STEP ON command C–50
C.100 TRANSLATE command C–50
C.101 TYPE command C–51
C.102 UNSIMULATE command C–51

C.103 WAIT command C–52

This section describes each KCL command in alphabetical order. Each description includes the purpose of the command, its syntax, and details of how to use it. Examples of each command are also provided.

The following notation is used to describe KCL command syntax:

- < > indicates optional arguments to a command
- | indicates a choice which must be made
- { } indicates an item can be repeated
- file_spec: <device_name:><\\host_name\><path_name\>file_name.file_type |

  <device_name:><\\host_name\>‘host_specific_name’
- path_name: <file_name\><dir\dir\. . .>
- file_name: maximum of 36 characters, no file type

device_name: is a two to five-character optional field, followed by a colon. The first character is a letter, the remaining characters must be alphanumeric. If this field is left blank, the default device from the system variable $DEVICE will be used.

host_name: is a one to eight character optional field. The host_name selects the network node that receives this command. It must be preceded by two backslashes and separated from the remaining fields by a backslash.

path_name : file_name\<path_name> - is a recursively defined optional field, each field consisting of a maximum of 36 characters. It is used to select the file subdirectory. The root or source directory is handled as a special case. It is designated by a file_name of zero length. For example, access to the subdirectory SYS linked off of the root would have path name ‘\SYS’. A fully qualified file_spec using this path_name would look like this, ‘C1:\HOST\SYS\FILE.KL’.

file_name: from one to 36 characters

file_type: from zero to three characters

KCL commands can be abbreviated allowing you to type in fewer letters as long as the abbreviated version remains unique among all keywords. For example, ‘‘ABORT’’ can be ‘‘AB’’ but ‘‘CONTINUE’’ must be ‘‘CONT’’ to distinguish it from ‘‘CONDITION.’’

path_names, file_names, and file_types that contain special characters or begin with numbers can be specified as a host_specific_name inside single quotes. FR:\’00\’ is valid, FR:\’00\test.kl’ is valid, FR:\’00’\test.kl is invalid because the host_specific_name must be last.

KCL commands that have <prog_name> as part of the command syntax will use the default program if none is specified. KCL commands that have <file_name> as part of the command syntax will use the default program as the file name if none is specified.

C.1 ABORT 命令

Syntax: ABORT < (**prog_name**) | ALL) > <FORCE>

where:

prog_name：任何KAREL或TP程序的名称

ALL：中止所有正在运行或暂停的任务

FORCE：即使设置了 NOABORT 属性也会中止任务。 FORCE 只适用于 ABORT prog_name; FORCE 不适用于 ABORT ALL

Purpose: 中止指定的运行或暂停的任务。 如果没有指定 prog_ name，则使用默认程序。

当前程序语句的执行在任务中止之前完成，除了当前动作 DELAY，WAIT 或 READ 语句被取消之外。

示例: KCL> ABORT test_prog FORCE

KCL> ABORT ALL

C.2 APPEND FILE 命令

Syntax: APPEND FILE **input_file_spec** TO **output_ file_spec**

where:

input_file_spec：有效的文件规范

output_file_spec：有效的文件规范

Purpose: 将指定输入文件的内容追加到指定输出文件的末尾。 **input_file_spec** 和 **output_file_spec** 必须同时包含文件名和文件类型。

示例: KCL> APPEND FILE flpy:test.kl TO productn.kl

KCL> APPEND FILE test.kl TO productn.kl

C.3 APPEND NODE 命令

Syntax: APPEND NODE <[**prog_name**]> **var_name**

where:

prog_name : the name of any KAREL or TP program

var_name : the name of any variable of type PATH

Purpose: Appends one node to the end of the specified PATH variable previously loaded in RAM. The appended node value is uninitialized and the index number is one more than the last node index. Execute the KCL> SAVE VARS command to make the change permanent.

Examples: KCL> APPEND NODE [test_prog]weld_pth

KCL> APPEND NODE weld_pth

C.4 CHDIR command

Syntax: CHDIR < **device_name** >\< **path_name** >\ or CD < **device_name** >\< **path_nam e** >

where:

device_name : a specified device

path_name : a subdirectory previously created on the memory card device using the mkdir command. When the chdir command is used to change to a subdirectory, the entire path will be displayed on the teach pendant screen as mc:\new_dir\new_file.

The double dot (..) can be used to represent the directory one level above the current directory.

Purpose: Changes the default device. If a **device_name** is not specified, displays the default device.

Examples: KCL> CHDIR rd:\

KCL> CD

KCL> CD mc:\a

KCL> CD ..

C.5 CLEAR ALL command

Syntax: CLEAR ALL <YES>

where:

YES : confirmation is not prompted

Purpose: Clears all KAREL and teach pendant programs and variable data from memory. All cleared programs and variables (if they were saved with the KCL> SAVE VARS command) can be reloaded into memory using the KCL> LOAD command.

Examples: KCL> CLEAR ALL

Are you sure? YES

KCL> CLEAR ALL Y

C.6 CLEAR BREAK CONDITION command

Syntax: CLEAR BREAK CONDITION < **prog_name** > (**brk_pnt_no** | ALL)

where:

prog_name : the name of any KAREL program in memory

brk_pnt_no : a particular condition break point

ALL : clears all condition break points

Purpose: Clears specified condition break point(s) from the specified or default program.

A condition break point only affects the program in which it is set.

Examples: KCL> CLEAR BREAK CONDITION test_prog 3

KCL> CLEAR BREAK COND ALL

C.7 CLEAR BREAK PROGRAM command

Syntax: CLEAR BREAK PROGRAM < **prog_name** > (**brk_pnt_no** | ALL)

where:

prog_name : the name of any KAREL program in memory

brk_pnt_no : a particular program break point

ALL : clears all break points

Purpose: Clears specified break point(s) from the specified or default program.

A break point only affects the program in which it is set.

Examples: KCL> CLEAR BREAK PROGRAM test_prog 3

KCL> CLEAR BREAK PROG ALL

C.8 CLEAR DICT command

Syntax: CLEAR DICT **dict_name** <(**lang_name** | ALL)>

where:

dict_name : the name of any dictionary to be cleared

lang_name : the name of the language. The available choices are ENGLISH, JAPANESE, FRENCH, GERMAN, SPANISH or DEFAULT.

ALL : clears the dictionary from all languages

Purpose: Clears a dictionary from the specified language or from all languages. If no language is specified, it is cleared from the DEFAULT language only.

Examples: KCL> CLEAR DICT tpsy ENGLISH

KCL> CLEAR DICT tpsy

C.9 CLEAR PROGRAM command

Syntax: CLEAR PROGRAM < **prog_name** > <YES>

where:

prog_name : the name of any KAREL or teach pendant program in memory

YES : confirmation is not prompted

Purpose: Clears the program data from memory for the specified or default program.

Examples: KCL> CLEAR PROGRAM test_prog

Are you sure? YES

KCL> CLEAR PROG test_prog Y

C.10 CLEAR VARS command

Syntax: CLEAR VARS < **prog_name** > <YES>

where:

prog_name : the name of any KAREL or teach pendant program with variables

YES : confirmation is not prompted

Purpose: Clears the variable and type data associated with the specified or default program from memory.

Variables and types that are referenced by a loaded program are not cleared.

Examples: KCL> CLEAR VARS test_prog

Are you sure? YES

KCL> CLEAR VARS test_prog Y

C.11 COMPRESS DICT command

Syntax: COMPRESS DICT **file_name**

where:

file_name : the file name of the user dictionary you want to compress.

Purpose: Compresses a dictionary file from the default storage device, using the specified dictionary name. The file type of the user dictionary must be ".UTX". The compressed dictionary file will have the same file name as the user dictionary, and be of type ".TX".

See Also: Chapter 10 *DICTIONARIES AND FORMS*

Example: KCL> COMPRESS DICT tphcmneg

C.12 COMPRESS FORM command

Syntax: COMPRESS FORM < **file_name** >

where:

file_name : the file name of the form you want to compress.

Purpose: Compresses a form file from the default storage device using the specified form name. The file type of the form must be ".FTX". A compressed dictionary file and variable file will be created. The compressed dictionary file will have the same file name as the form file and be of type ".TX". The variable file will have a four character file name, that is extracted from the form file name, and be of type ".VR". If no form file name is specified, the name "FORM" is used.

See Also: Chapter 10 *DICTIONARIES AND FORMS*

Examples: KCL> COMPRESS FORM

KCL> COMPRESS FORM mnpalteg

C.13 CONTINUE command

Syntax: CONTINUE <(**prog_name**) | ALL)>

where:

prog_name : the name of any KAREL or teach pendant program which is a task

ALL : continues all paused tasks

Purpose: Continues program execution of the specified task that has been paused by a hold, pause, or test run operation. If the program is aborted, the program execution is started at the first executable line.

When a task is paused, the CYCLE START button on the operator panel has the same effect as the KCL> CONTINUE command.

CONTINUE is a motion command; therefore, the device from which it is issued must have motion control.

See Also: Refer to the $RMT_MASTER description in the *FANUC Robotics Software Reference Manual* for more information about assigning motion control to a remote device.

Examples: KCL> CONTINUE test_prog

KCL> CONT ALL

C.14 COPY FILE command

Syntax: COPY <FILE> **from_file_spec** TO **to_file_spec** <OVERWRITE>

where:

from_file_spec : a valid file specification

to_file_spec : a valid file specification

OVERWRITE : specifies copy over (overwrite) an existing file

Purpose: Copies the contents of one file to another with overwrite option. Allows file transfers between different devices and between the controller and a host system.

The wildcard character (*) can be used to replace **from_file_spec** 's entire file name, the first part of the file name, the last part of the file name, or both the first and last parts of the file name. The file

type can also use the wildcard in the same manner. The wildcard character in the **to_file_spec** can only replace the entire file name or the entire file type.

Examples: KCL> COPY flpy:\test.kl TO rdu:newtest.kl

KCL> COPY mc:\test_dir\test.kl TO mc:\test_dir\newtest.kl

KCL> COPY FILE flpy:*.kl TO rd:*.kl

KCL> COPY *.* TO fr:

KCL> COPY FILE *.kl TO rd:*.bak OVERWRITE

KCL> COPY FILE flpy:*main*.kl TO rd:* OV

KCL> COPY mdb:*.tp TO mc:

C.15 CREATE VARIABLE command

Syntax: CREATE VARIABLE <[**prog_name**]> **var_name <IN (CMOS | DRAM)> : data_type**

where:

prog_name : the name of any KAREL or TP program

var_name:data_type : a valid variable name and data type

Purpose: Allows you to declare a variable that will be associated with the specified or default program. You must specify a valid identifier for the **var_name** and a valid **data_type** .

Only one variable can be declared with the CREATE VAR command. You must enter the KCL> SAVE VARS command to save the declared variable with the program variable data. Use the KCL> SET VARIABLE command to assign a value to a variable.

The following data types are valid (user types are also supported): ARRAY OF BYTE, JOINTPOS, ARRAY OF SHORT, JOINTPOS1 to JOINTPOS9, BOOLEAN, POSITION, REAL, CONFIG, VECTOR, FILE, XYZWPR, XYZWPREXT, INTEGER

You can create multi-dimensional arrays of the above type. A maximum of 3 dimensions may be specified. Paths may only be created from a user defined type.

By default, the variable will be created in temporary memory (in DRAM), and must be recreated every power up. The value will always be reset to uninitialized.

If IN CMOS is specified the variable will be created in permanent memory. The variable's value will be recovered every time the controller is turned on.

See Also: SET VARIABLE command

Examples: KCL> CREATE VAR [test_prog]count IN CMOS: INTEGER

KCL> CREATE VAR vec:ARRAY[3,2,4] OF VECTOR

C.16 DELETE FILE command

Syntax: DELETE FILE **file_spec** <YES>

where:

file_spec : a valid file specification

YES : confirmation is not prompted

Purpose: Deletes the specified file from the specified storage device. The wildcard character (*) can be used to replace the entire file name, the first part of the file name, the last part of the file name, or both the first and last parts of the file name. The file type can also use the wildcard in the same manner.

Examples: KCL> DELETE FILE testprog.pc

Are you sure? YES

KCL> DELETE FILE rd:\testprog.pc YES

KCL> DELETE FILE rd:*.* Y

C.17 DELETE NODE command

Syntax: DELETE NODE <[**prog_name**]> **var_name [node_index**]

where:

prog_name : the name of any KAREL or TP program

var_name : the name of any variable of type PATH

[**node_index**] : a node in the path

Purpose: Deletes the specified node from the specified PATH variable. The PATH variable must be loaded in memory. Enter the KCL> SAVE VARS command to make the change permanent.

Examples: KCL> DELETE NODE [test_prog]weld_pth[4]

KCL> DELETE NODE weld_pth[3]

C.18 DELETE VARIABLE command

Syntax: DELETE VARIABLE <[**prog_name**]> **var_name**

where:

prog_name : the name of any KAREL or TP program with variables

var_name : the name of any program variable

Purpose: Deletes the specified variable from memory. A variable that is linked with loaded p-code cannot be deleted. Enter the KCL> SAVE VARS command to make the change permanent.

Examples: KCL> DELETE VARIABLE [test_prog]weld_pth

KCL> DELETE VAR weld_pth

C.19 DIRECTORY command

Syntax: DIRECTORY < **file_spec** >

where:

file_spec : a valid file specification

Purpose: Displays a list of the files that are on a storage device. If **file_spec** is not specified, directory information is displayed for all of the files stored on a specified device. The directory information displayed includes the following:

The volume name of the device (if specified when the device was initialized)

The name of the subdirectory, if available

The names and types of files currently stored on the device and the sizes of the files in bytes

The number of files, the number of bytes left, and the number of bytes total, if available

The wildcard character (*) can be used to replace the entire file name, the first part of the file name, the last part of the file name, or both the first and last parts of the file name. The file type can also use the wildcard in the same manner.

Examples: KCL> DIRECTORY rd:

KCL> DIR *.kl

KCL> DIR *SPOT*.kl

KCL> CD MC: \test_dir

Use the CD command to change to the KCL> DIR subdirectory before you use the DIR commandor KCL> DIR \test_dir*.* display the subdirectory contents without using the CD command.

C.20 DISABLE BREAK PROGRAM command

Syntax: DISABLE BREAK PROGRAM < **prog_name** > **brk_pnt_no**

where:

prog_name : the name of any KAREL or TP program in memory

brk_pnt_no : a particular program break point

Purpose: Disables the specified break point in the specified or default program.

Examples: KCL> DISABLE BREAK PROGRAM test_prog 3

KCL> DISABLE BREAK PROG 3

C.21 DISABLE CONDITION command

Syntax: DISABLE CONDITION < **prog_name** > **condition_no**

where:

prog_name : the name of any KAREL program in memory

condition_no : a particular condition

Purpose: Disables the specified condition in the specified or default program.

Examples: KCL> DISABLE CONDITION test_prog

KCL> DISABLE COND 3

C.22 DISMOUNT command

Syntax: DISMOUNT **device_name:**

where:

device_name : device to be dismounted

Purpose: Dismounts a mounted storage device and indicates to the controller that a storage device is no longer available for reading or writing data.

Example: KCL> DISMOUNT rd:

C.23 EDIT command

Syntax: EDIT < **file_spec** >

where:

file_spec : a valid file specification

Purpose: Provides an ASCII text editor which can be used for editing dictionary files, command files and KAREL source files.

If **file_spec** is not specified, the default program name is used as the file name and the default file type is .KL (KAREL source code).

If a previous editing session exists, then **file_spec** is ignored and the editing session is resumed.

Examples: KCL> EDIT startup.cf

KCL> ED

C.24 ENABLE BREAK PROGRAM

Syntax: ENABLE BREAK PROGRAM < **prog_name** > **brk_pnt_no**

where:

prog_name : the name of any KAREL or TP program in memory

brk_pnt_no : a particular program break point

Purpose: Enables the specified break point in the specified or default program.

Examples: KCL> ENABLE BREAK PROGRAM test_prog 3

KCL> ENABLE BREAK PROG 3

C.25 ENABLE CONDITION command

Syntax: ENABLE CONDITION < **prog_name** > **condition_no**

where:

prog_name : the name of any KAREL program in memory

condition_no : a particular condition

Purpose: Enables the specified condition in the specified or default program.

Examples: KCL> ENABLE CONDITION test_prog

KCL> ENABLE COND 3

C.26 FORMAT command

Syntax: FORMAT **device_name:** < **volume_name** > <YES>

where:

device_name : the specified device to be initialized

volume_name : label for the device

YES : confirmation is not prompted

Purpose: Formats a specified device. A device must be formatted before storing files on it.

Examples: KCL> FORMAT rd:

Are you sure? YES

KCL> FORMAT rd: Y

C.27 HELP command

Syntax: HELP < **command_name** >

where:

command_name : a KCL command

Purpose: Displays on-line help for KCL commands. If you specify a **command_name** argument, the required syntax and a brief description of the specified command is displayed.

Examples: KCL> HELP LOAD PROG

KCL> HELP

C.28 HOLD command

Syntax: HOLD <(**prog_name** | ALL)>

where:

prog_name : the name of any KAREL or TP program

ALL : holds all executing programs

Purpose: Pauses the specified or default program that is being executed and holds motion at the current position (after a normal deceleration).

Use the KCL> CONTINUE command or the CYCLE START button on the operator panel to resume program execution.

Examples: KCL> HOLD test_prog

KCL> HO ALL

C.29 INSERT NODE command

Syntax: INSERT NODE <[**prog_name**]> **var_name [node_index**]

where:

prog_name : the name of any KAREL or TP program

var_name : the name of any variable of type PATH

[node_index] : a node in the path

Purpose: Inserts a node in front of the specified node in the PATH variable. The PATH variable must be loaded in memory.

The inserted node index number is the **node_index** you specify and the inserted node value is uninitialized. The index numbers for subsequent nodes are incremented by one. You must enter the KCL> SAVE VARS command to make the change permanent.

Examples: KCL> INSERT NODE [test_prog]weld_pth[2]

KCL> INSERT NODE weld_pth[3]

C.30 LOAD ALL command

Syntax: LOAD ALL < **file_name** > <CONVERT>

where:

file_name : a valid file name

CONVERT : converts variables to system definition

Purpose: Loads a p-code and variable file from the default storage device and default directory into memory using the specified or default file name. The file types for the p-code and variable files are assumed to be ".PC" and ".VR" respectively.

If **file_name** is not specified, the default program is used. If the default has not been set, then the message, "Default program name not set," will be displayed.

Examples: KCL> LOAD ALL test_prog

KCL> LOAD ALL

C.31 LOAD DICT command

Syntax: LOAD DICT **file_name dict_name** < **lang_name** >

where:

file_name : the name of the file to be loaded

dict_name : the name of any dictionary to be loaded. The name will be truncated to 4 characters.

lang_name : a particular language. The available choices are ENGLISH, JAPANESE, FRENCH, GERMAN, SPANISH or DEFAULT.

Purpose: Loads a dictionary file from the default storage device and default directory into memory using the specified file name. The file type is assumed to be ".TX."

See Also: Chapter 10 *DICTIONARIES AND FORMS*

Examples: KCL> LOAD DICT tpaleg tpal FRENCH

KCL> LOAD DICT tpaleg tpal

C.32 LOAD FORM command

Syntax: LOAD FORM < **form_name** >

where:

form_name : the name of the form to be loaded

Purpose: Loads the specified form, from the default storage device, into memory. A form consists of a compressed dictionary file and a variable file. If no name is specified, 'FORM.TX' and 'FORM.VR' are loaded.

If the specified **form_name** is greater than four characters, the first two characters are not used for the dictionary name or the variable file name.

See Also: For more information on creating and using forms, refer to Chapter 10 *DICTIONARIES AND FORMS*

Example: KCL> LOAD FORM

Loading FORM.TX with dictionary name FORM

Loading FORM.VR

KCL> LOAD FORM tpexameg

Loading TPEXAMEG.TX with dictionary name EXAM

Loading EXAM.VR

C.33 LOAD MASTER command

Syntax: LOAD MASTER < **file_name** > <CONVERT>

where:

file_name : a valid file name

CONVERT : converts variables to system definition

Purpose: Loads a mastering data file from the default storage device and default directory into memory using the specified or default file name. The file type is assumed to be ".SV."

If **file_name** is not specified, the default file name, "SYSMAST.SV," is used.

Example: KCL> LOAD MASTER

C.34 LOAD PROGRAM command

Syntax: LOAD PROGRAM < **file_name** >

where:

file_name : a valid file name

Purpose: Loads a p-code file from the default storage device and default directory into memory using the specified or default file name. The file type is assumed to be ‘‘.PC.’’

If **file_spec** is not specified, the default program is used. If the default has not been set, then the message, ‘‘Default program name not set,’’ will be displayed.

The following note applies to R-30iB controllers:

Note The KAREL option must be installed on the robot controller in order to load KAREL programs.

Examples: KCL> LOAD PROGRAM test_prog

KCL> LOAD PROG

C.35 LOAD SERVO command

Syntax: LOAD SERVO < **file_name** > <CONVERT>

where:

file_name : a valid file name

CONVERT : converts variables to system definition

Purpose: Loads a servo parameter file from the default storage device and default directory into memory using the specified or default file name. The file type is assumed to be ‘‘.SV.’’

If **file_name** is not specified, the default file name, ‘‘SYSSERVO.SV,’’ is used.

Example: KCL> LOAD SERVO

C.36 LOAD SYSTEM command

Syntax: LOAD SYSTEM < **file_name** > <CONVERT>

where:

file_name : a valid file name

CONVERT : converts variables to system definition

Purpose: Loads the specified system variable file into memory, assigning values to all of the saved system variables. The default storage device and default directory are used with the specified or default file name. The file type is assumed to be ".SV."

If **file_name** is not specified, the default file name, "SYSVARS.SV," is used.

Examples: KCL> LOAD SYSTEM awdef

KCL> LOAD SYSTEM CONVERT

The following rules are applicable for system variables:

- If an array system variable that is not referenced by a program already exists when a .SV file is loaded, the size in the .SV file is used and the contents are loaded. No errors are posted.
- If an array system variable that is referenced by a program already exists when a .SV file with a LARGER size is loaded, the size in the .SV file is ignored, and NONE of the array values are loaded. The following errors are posted; " *var_name* memory not updated", "Array len creation mismatch".
- If an array system variable that is referenced by a program already exists when a .SV file with a SMALLER size is loaded, the size in the .SV file is ignored but ALL the array values are loaded. No errors are posted.
- If a .SV file with a different type definition is loaded, the .SV file will stop loading and detect the error. The following errors are posted; "Create type - *var_name* failed", "Duplicate creation mismatch".
- If a .SV. file with a different type definition is loaded, but the CONVERT option is specified, it tries to load as much as it can. For example, the controller has a SCR_T type which has the field $NEW but not the field $OLD. When an old .SV file is loaded that has $OLD but not $NEW, the load procedure creates the SCR_T type based on what is in the .SV file and posts a "Create type - var_name failed", "Duplicate creation mismatch" error. It then creates the type SCR_! which has the field $OLD but not the field $NEW. It then does a field by field copy of all of the old valid fields into the new type. Therefore, since there is not $NEW information in the old type that field is not updated and the $OLD information is discarded. Any fields whose types don't match are discarded from the loaded type. So if a field was changed from integer to real, the integer field in the loaded data would be discarded. Any fields that are arrays will follow the same rules as array system variables.

C.37 LOAD TP command

Syntax: LOAD TP < **file_name** > <OVERWRITE>

where:

file_name : a valid file name

OVERWRITE : If specified, may overwrite a previously loaded TP program with the same name

Purpose: Loads a TP program from the default storage device and default directory into memory using the specified or default file name. The file type is assumed to be ‘‘.TP.’’

If **file_name** is not specified, the default program is used. If the default has not been set, then the message, ‘‘Default program name not set,’’ will be displayed.

Examples: KCL> LOAD TP testprog

KCL> LOAD TP

C.38 LOAD VARS command

Syntax: LOAD VARS < **file_name** > <CONVERT>

where:

file_name : a valid file name

CONVERT : converts variables to system definition

Purpose: Loads the specified or default variable data file from the default storage device and directory into memory. The file type is assumed to be ‘‘.VR.’’

If **file_name** is not specified, the default program is used. If the default has not been set then the message, ‘‘Default program name not set,’’ will be displayed.

Examples: KCL> LOAD VARS test_prog

KCL> LOAD VARS

The following rules are applicable for array variables:

- If an array variable that is not referenced by a program already exists when a .VR file is loaded, the size in the .VR file is used and the contents are loaded. No errors are posted.
- If an array variable already exists when a program is loaded, the size in the .PC file is ignored and the program is loaded anyway. The following errors are posted: " *var_name* PC array length ignored", and "Array len creation mismatch".
- If an array variable that is referenced by a program already exists when a .VR file with a LARGER size is loaded, the size in the .VR file is ignored and NONE of the array values are loaded. The following errors are posted; " *var_name* memory not updated," "Array len creation mismatch."
- If an array variable that is referenced by a program already exists when a .VR file with a SMALLER size is loaded, the size in the .VR file is ignored but ALL the array values are loaded. The following errors are posted; " *var_name* array length updated," "Array len creation mismatch."

The following rules are applicable for user-defined types in KAREL programs:

- Once a type is created it can never be changed, regardless of whether a program references it or not. If all the variables referencing the type are deleted, the type will also be deleted. A new version can then be loaded.
- If a type already exists when a program with a different type definition is loaded, the .PC file will not be loaded. The following errors are posted; "Create type - *var_name* failed," "Duplicate creation mismatch."
- If a type already exists when a .VR file with a different type definition is loaded, the .VR file will stop loading when it detects the error. The following errors are posted; "Create type - *var_name* failed," "Duplicate creation mismatch".

C.39 LOGOUT command

Syntax: LOGOUT

Purpose: Logs the current user from the KCL device out of the system. The password level reverts to the OPERATOR level. If passwords are not enabled, an error message will be displayed by KCL such as, "No user currently logged in".

Example: KCL>LOGOUT

(The alarm message: "Logout (SAM) SETUP from KCL")

KCL Username>

C.40 MKDIR command

Syntax: MKDIR **<device_name>\path_name**

where:

device_name : a valid storage device

path_name : a subdirectory previously created on the memory card device using the mkdir command.

Purpose: MKDIR creates a subdirectory on the memory card (MC:) device. FANUC Robotics recommends you nest subdirectories only to 8 levels.

Example: KCL> MKDIR mc:\test_dir

KCL> MKDIR mc:\prog_dir\tpnx_dir

C.41 MOUNT command

Syntax: MOUNT **device_name**

where:

device_name : a valid storage device

Purpose: MOUNT indicates to the controller that a storage device is available for reading or writing data.

A device must be formatted with the KCL> FORMAT command before it can be mounted successfully.

Example: KCL> MOUNT rd:

C.42 MOVE FILE command

Syntax: MOVE <FILE> **file_spec**

where:

file_spec : a valid file specification.

Purpose: Moves the specified file from one memory file device to another. The file should exist on the FROM or RAM disks. If file_spec is a file on the FROM disk, the file is moved to the RAM disk, and vice versa.

The wildcard character (*) can be used to replace the entire file name, the first part of the file name, the last part of the file name, or both the first and last parts of the file name. The file type can also use the wildcard in the same manner. If file_spec specifies multiple files, then they are all moved to the other disk.

Examples: KCL> MOVE FILE fr:*.kl

KCL> MOVE rd:*.*

C.43 PAUSE command

Syntax: PAUSE <(**prog_name** | ALL)> <FORCE>

where:

prog_name : the name of any KAREL or TP program which is a task

ALL : pauses all running tasks

FORCE : pauses the task even if the NOPAUSE attribute is set

Purpose: Pauses the specified running task. If **prog_name** is not specified, the default program is used.

Execution of the current motion segment and the current program statement is completed before the task is paused.

Condition handlers remain active. If the condition handler action is NOPAUSE and the condition is satisfied, task execution resumes.

If the statement is a WAIT FOR and the wait condition is satisfied while the task is paused, the statement following the WAIT FOR is executed immediately when the task is resumed.

If the statement is a DELAY, timing will continue while the task is paused. If the delay time is finished while the task is paused, the statement following the DELAY is immediately executed when the task is resumed. If the statement is a READ, it will accept input even though the task is paused.

The KCL> CONTINUE command resumes execution of a paused task. When a task is paused, the CYCLE START button on the operator panel has the same effect as the KCL> CONTINUE command.

Examples: KCL> PAUSE test_prog FORCE

KCL> PAUSE ALL

C.44 PURGE command

Syntax: PURGE **device_name**

where:

device_name : the name of the memory file device to be purged

Purpose: Purges the specified memory file device by freeing any used blocks that are no longer needed. The device should be set to "FR:" for FROM disk, "RD:" for RAM disk, or "MF:" for both disks.

The purge operation is only necessary when the device does not have enough memory to perform an operation. The purge operation will erase file blocks that were previously used, but are no longer needed. These are called garbage blocks. The FROM disk can contain many garbage blocks if files are deleted or overwritten. The RAM disk will not normally contain garbage blocks, but they may occur when power is removed during a file copy.

The device must be mounted and no files can be open on the device or an error will be displayed.

Examples: KCL> PURGE fr:

KCL> PURGE mf:

C.45 PRINT command

Syntax: PRINT **file_spec**

where:

file_spec : a valid file specification

Purpose: Allows you to print the contents of an ASCII file to the default device.

Example: KCL> PRINT testprog.kl

C.46 RECORD command

Syntax: RECORD <[**prog_name**]> **var_name**

where:

prog_name : the name of any KAREL or TP program

var_name : the name of any POSITION, XYZWPR, or JOINTPOS variable

Purpose: Records the position of the TCP and/or auxiliary or extended axes. The robot must be calibrated before the RECORD command is issued. The variable can be a system variable or a program variable that exists in memory. The position is recorded relative to the user frame of reference.

You must enter the KCL> SAVE command to permanently assign the recorded position. The Record function key, F3, under the teach pendant TEACH menu also allows you to record positions.

Example: KCL> RECORD [paint_prog]start_pos

KCL> RECORD $GROUP[1].$uframe

C.47 RENAME FILE command

Syntax: RENAME FILE **old_file_spec** TO **new_file_ spec**

where:

old_file_spec : a valid file specification

new_file_spec : a valid file specification

Purpose: Changes the **old_file_spec** to the **new_file_spec** . The file will no longer exist under the **old_file_spec** . The **old_file_spec** and the **new_file_spec** must include both the file name and the file type. The same file type must be used in both file_specs but they cannot be the same file.

Use the KCL> COPY FILE command to change the device name of a file.

Examples: KCL> RENAME FILE test.kl TO productn.kl

KCL> RENAME FILE mycmd.cf TO yourcmd.cf

C.48 RENAME VARIABLE command

Syntax: RENAME VARIABLE <[**prog_name**]> **old_var_ name new_var_name**

where:

prog_name : the name of any KAREL or TP program

old_var_name : the name of any program variable

new_var_name : a valid program variable name

Purpose: Changes the **old_var_name** to the **new_var_name** in the program specified with the **old_var_name** . The variable will no longer exist under the **old_var_name** . The variable must exist in memory under the **old_var_name** in the specified program.

The **new_var_name** cannot already exist in memory. The variable still belongs to the same program. You cannot specify a **prog_name** with the **new_var_name.**

You must enter the KCL> SAVE VARS command to make the change permanent.

Examples: KCL> RENAME VARIABLE [test_prog]count part_count

KCL> RENAME VAR count part_count

C.49 RENAME VARS command

Syntax: RENAME VARS **old_prog_name new_prog_name**

where:

old_prog_name : the name of any KAREL or TP program

new_prog_name : the name of any KAREL or TP program

Purpose: Changes the name of the variable data associated with the **old_prog_name** to the **new_prog_name** . The variable data will no longer exist under the **old_prog_name** .

Before you use the RENAME VARS command, the variable data must exist in memory under the **old_prog_name** . Variable data cannot already exist in memory under the **new_prog_name** .

The command does not rename the program. To rename a KAREL program, use the KCL> RENAME FILE to rename the .KL file, edit the program name in the .KL file, translate the program, and load the new C file. To rename a TP program, use the SELECT menu.

You must enter the KCL> SAVE VARS command to make the change permanent.

Example: KCL> RENAME VARS test_1 test_2

Use this sequence of KCL commands to copy the variable data of one program (**prog_1**) into a variable file that is then used by another program (**prog_2**):

LOAD VARS **prog_1**

RENAME VARS **prog_1 prog_2**

SAVE VARS **prog_2**

LOAD ALL **prog_2**

The effect of this sequence of commands cannot be accomplished with the KCL> COPY FILE command.

The name of the program to which the variable data belongs is stored in the variable file. The KCL> COPY FILE command does not change that stored program name, so the data cannot be used with another program.

C.50 RESET command

Syntax: RESET

Purpose: Enables servo power after an error condition has shut off servo power, provided the cause of the error has been cleared. The command also clears the message line on the CRT/KB display. The error message remains displayed if the error condition still exists.

The RESET command has no effect on a program that is being executed. It has the same effect as the FAULT RESET button on the operator panel and the RESET function key on the teach pendant RESET screen.

Example: KCL> RESET

C.51 RMDIR command

Syntax: RMDIR **<device_name>\path_name**

where:

device_name : a valid storage device

path_name : a subdirectory previously created on the memory card device using the mkdir command.

Purpose: RMDIR deletes a subdirectory on the memory card (MC:) device. The directory must be empty before it can be deleted.

Example: KCL> RMDIR mc:\test_dir

KCL> RMDIR mc:\test_dir\prog_dir

C.52 RUN command

Syntax: RUN < **prog_name** >

where:

prog_name :the name of any KAREL or TP program

Purpose: Executes the specified program. The program must be loaded in memory. If no program is specified the default program is run. If uninitialized variables are encountered, program execution is paused.

Execution begins at the first executable line. RUN is a motion command; therefore, the device from which it is issued must have motion control. If a RUN command is issued in a command file, it is executed as a NOWAIT command. Therefore, the statement following the RUN command will be executed immediately after the RUN command is issued without waiting for the program, specified by the RUN command, to end.

See Also: Refer to the $RMT_MASTER description in the *FANUC Robotics Software Reference Manual* for more information about assigning motion control to a remote device.

Example: KCL> RUN test_prog

C.53 RUNCF command

Syntax: RUNCF **input_file_spec** < **output_file_spec** >

where:

input_file_spec : a valid file specification

output_file_spec : a valid file specification

Purpose: Executes the KCL command procedure that is stored in the specified input file and displays the output to the specified output file. The input file type is assumed to be .CF. The output file type is assumed to be .LS if no file type is supplied.

If **output_file_spec** is not specified, the output will be displayed to the KCL output window.

The RUNCF command can be nested within command files up to four levels. Use **%INCLUDE input_file_spec** to include another .CF file into the command procedure. RUNCF command itself is not allowed inside a command procedure.

If the command file contains motion commands, the device from which the RUNCF command is issued must have motion control.

See Also: Refer to Section 13.4 , ‘‘Command Procedures,’’ for more information

Examples: KCL> RUNCF startup output

KCL> RUNCF startup

C.54 SAVE MASTER command

Syntax: SAVE MASTER < **file_name** >

where:

file_name : a valid file name

Purpose: Saves the mastering data file from the default storage device and default directory into memory using the specified or default file name. The file type will be ‘‘.SV.’’

If **file_name** is not specified, the default file name, ‘‘SYSMAST.SV,’’ is used.

Example: KCL> SAVE MASTER

C.55 SAVE SERVO command

Syntax: SAVE SERVO < **file_name** >

where:

file_name :a valid file name

Purpose: Saves the servo parameters into the default storage device using the specified or default file name. The file type will be ‘‘.SV.’’

If **file_name** is not specified, the default file name, ‘‘SYSSERVO.SV,’’ is used.

Example: KCL> SAVE SERVO

C.56 SAVE SYSTEM command

Syntax: SAVE SYSTEM < **file_name** >

where:

file_name :a valid file name

Purpose: Saves the system variable values into the default storage device and default directory using the specified system variable file (.SV). If you do not specify a **file_spec** the default name, ‘‘SYSVARS.SV,’’is used. For example:

SAVE SYSTEM **file_1**

In this case, the system variable data is saved in a variable file called **file_1.SV** .

SAVE SYSTEM

In this case, the system variable data is saved in a system variable file ‘‘SYSVARS.SV.’’

Examples: KCL> SAVE SYSTEM file_1

KCL> SAVE SYSTEM

C.57 SAVE TP command

Syntax: SAVE TP < **file_name** > <= **prog_name** >

where:

file_name : a valid file name

prog_name : the name of any TP program

Purpose: Saves the specified TP program to the specified file (.TP). If you do not specify a **file_name** or a **prog_name** , the default program name is used. If only a **file_name** is specified, that name will also be used for **prog_name** . For example:

SAVE TP **file_1**

In this case, the TP program **file_1** is saved in a file called **file_1.TP** .

SAVE TP = **prog_1**

In this case, the TP program **prog_1** is saved in a file whose name is the default program name.

If you specify a program name, it must be preceded by an equal sign (=).

Examples: KCL> SAVE TP file_1 = prog_1

KCL> SAVE TP file_1

KCL> SAVE TP = prog_1

KCL> SAVE TP

C.58 SAVE VARS command

Syntax: SAVE VARS < **file_name** > <= **prog_name** >

where:

file_name : a valid file name

prog_name : the name of any KAREL or TP program

Purpose: Saves variable data from the specified program, including the currently assigned values, to the specified variable file (.VR). If you do not specify a **file_name** or a **prog_name** , the default program name is used. If only a **file_name** is specified, that name will also be used for **prog_name** . For example:

SAVE VARS **file_1**

In this case, the variable data for the program **file_1** is saved in a variable file called **file_1.VR.**

SAVE VARS = **prog_** 1

In this case, the variable data for **prog_** 1 is saved in a variable file whose name is the default program name.

If you specify a program name, it must be preceded by an equal sign (=).

Any variable data that is not saved is lost when an initial start of the controller is performed.

Examples: KCL> SAVE VARS file_1 = prog_1

KCL> SAVE VARS file_1

KCL> SAVE VARS = prog_1

KCL> SAVE VARS

C.59 SET BREAK CONDITION command

Syntax: SET BREAK CONDITION < **prog_name** > **condition_no**

where:

prog_name : the name of any running or paused KAREL program

condition_no : a particular condition

Purpose: Allows you to set a break point on the specified condition in the specified program or default program. The specified condition must already exist so the program must be running or paused. When the break point is triggered, a message will be posted to the error log and the break point will be cleared.

Examples: KCL> SET BREAK CONDITION test_prog 1

KCL> SET BREAK COND 2

C.60 SET BREAK PROGRAM command

Syntax: SET BREAK PROGRAM < **prog_name** > **brk_pnt_no line_no** <(PAUSE | DISPLAY | TRACE ON | TRACE OFF)>

where:

prog_name : the name of any KAREL or TP program in memory

brk_pnt_no : a particular program break point

line_no : a line number

PAUSE : task is paused when break point is executed

DISPLAY : message is displayed on the teach pendant USER menu when break point is executed

TRACE ON : trace is enabled when break point is executed

TRACE OFF : trace is disabled when break point is executed

Purpose: Allows you to set a break point at a specified line in the specified or default program. The specified line must be an executable line of source code. Break points will be executed before the specified line in the program. By default the task will pause when the break point is executed. DISPLAY, TRACE ON, and TRACE OFF will not pause task execution.

Break points are local only to the program in which the break points were set. For example, break point #1 can exist among one or more loaded programs with each at a unique line number. If you specify an existing break point number, the existing break point is cleared and a new one is set in the specified program at the specified line.

Break points in a program are cleared if the program is cleared from memory. You also use the KCL> CLEAR BREAK PROGRAM command to clear break points from memory.

Use the KCL> CONTINUE command or the operator panel CYCLE START button to resume execution of a paused program.

Examples: KCL> SET BREAK PROGRAM test_prog 1 22 DISPLAY

KCL> SET BREAK PROG 3 30

C.61 SET CLOCK command

Syntax: SET CLOCK **'dd-mmm-yy hh:mm'**

where:

The date is specified using two numeric characters for the day, a three letter abbreviation for the month, and two numeric characters for the year; for example, 01-JAN- 00.

The time is specified using two numeric characters for the hour and two numeric characters for the minutes; for example, 12:45.

Purpose: Sets the date and time of the internal controller clock.

The date and time are included in directory and translator listings.

See Also: SHOW CLOCK command

Example: KCL> SET CLOCK '02-JAN-xx 21:45'

C.62 SET DEFAULT command

Syntax: SET DEFAULT **prog_name**

where:

prog_name : the name of any KAREL or TP program

Purpose: Sets the default program name to be used as an argument default for program and file names. The default program name can also be set at the teach pendant.

See Also: Section 13.1.1 , "Default Program"

Examples: KCL> SET DEFAULT test_prog

KCL> SET DEF test_prog

C.63 SET GROUP command

Syntax: SET GROUP **group_no**

where:

group_no : a valid group number

Purpose: Sets the default group number to use in other commands.

Example: KCL> SET GROUP 1

C.64 SET LANGUAGE command

Syntax: SET LANGUAGE **lang_name**

where:

lang_name : a particular language. The available choices are ENGLISH, JAPANESE, FRENCH, GERMAN, SPANISH or DEFAULT.

Purpose: Sets the $LANGUAGE system variable which determines the language to use.

Example: KCL> SET LANG ENGLISH

C.65 SET LOCAL VARIABLE command

Syntax: SET LOCAL VARIABLE **var_name** <IN **rout_name** > <FROM **prog_name** > < **task_name** > = **value** <{, **value** }>

where:

var_name :a local variable or parameter name

rout_name :the name of any KAREL routine

prog_name :the name of any KAREL program

task_name : the name of any KAREL task

value : new value for variable

Purpose: Assigns the specified value to the specified local variable or routine parameter. You can assign constant values or variable values, but the value must be of the data type that has been declared for the variable.

Please use the HELP SET VAR command for more information on assigning data types.

If the IN clause is omitted, the routine at the top of the stack is assumed. If the FROM clause is omitted, the default program is assumed. If the **task_name** is omitted, the stack of the KCL default task is searched.

Note The file RD: **prog_name.rs** is required to obtain local variable information.

Example: See SHOW LOCAL VARIABLE command.

See Also: SHOW LOCAL VARIABLE and TRANSLATE command.

C.66 SET PORT command

Syntax: SET PORT **port_name [index] = value**

where:

port_name[index] : a valid I/O port **value** : a new value for the port

Purpose: Assigns the specified value to a specified input or output port. SET PORT can be used with either physical or simulated output ports, but only with simulated input ports.

The valid ports are:

DIN, DOUT, RDO, OPOUT, TPOUT, WDI, WDO (BOOLEAN)-AIN, AOUT, GIN, GOUT (INTEGER)

See Also: SIMULATE, UNSIMULATE command, Chapter 14 *INPUT/OUTPUT SYSTEM* , application-specific FANUC Robotics Setup and Operations Manual.

Example: KCL> SET PORT DOUT [1] = ON

KCL> SET PORT GOUT [2] = 255

KCL> SET PORT AIN [1] = 1000

C.67 SET TASK command

Syntax: SET TASK <[**prog_name**]> **attr_name** = **value**

where:

prog_name : the name of any KAREL or TP program which is a task

attr_name : PRIORITY or TRACELEN

value : new integer value for attribute

Purpose: Sets the specified task attribute. PRIORITY sets the task priority. The lower the number, the higher the priority. 1 to 89 is lower than motion, but higher than the user interface. 90 to 99 is lower than the user interface. The default is 50. TRACELEN sets the trace buffer length. The default is 10 lines.

C.68 SET TRACE command

Syntax: SET TRACE (OFF | ON) <[**prog_name**]>

where:

prog_name : the name of any KAREL or TP program loaded in memory

Purpose: Turns the trace function ON or OFF (default is OFF). The program statement currently being executed and its line number are stored in a buffer when TRACE is ON. TRACE should only be set to ON during debugging operations because it slows program execution. To see the trace data, SHOW TRACE command must be used.

See Also: SHOW TRACE command

C.69 SET VARIABLE command

Syntax: SET VARIABLE <[**prog_name**]> **var_name** = **value** <{, **value** }>

where:

prog_name : the name of any KAREL or TP program

var_name : a valid program variable

value : new value for variable or a program or system variable

Purpose: Assigns the specified value to the specified variable. You can assign constant values or variable values, but the value must be of the data type that has been declared for the variable.

You can assign values to system variables with KCL write access, to program variables, or to standard and user-defined variables and fields. You can assign only one ARRAY element. Use brackets ([]) after the variable name to specify an element.

Certain data types like positions and vectors might have more than one value specified.

KCL> SET VAR position_var = 0,0,0,0,0,0

The SET VARIABLE command displays the previous value of the specified variable followed by the value which you have just assigned, providing you with an opportunity to check the assignment. The DATA key on the teach pendant also allows you to assign values to variables.

When you use SET VARIABLE to define a position you can use one of the following formats:

KCL> SET VARIABLE var_name.X = value

KCL> SET VARIABLE var_name.Y = value

KCL> SET VARIABLE var_name.Z = value

KCL> SET VARIABLE var_name.W = value

KCL> SET VARIABLE var_name = value

where X,Y,Z,W,P, and R specify the location and orientation, c_str is a string value representing configuration in terms of joint placement and turn numbers. Refer to Section 8.1, ‘‘Positional Data.’’ For example, to set X=200.0, W=60.0 and the turn numbers for axes 4 and 6 to 1 and 0 you would type the following lines:

KCL> SET VARIABLE var_name.X = 200

KCL> SET VARIABLE var_name.W = 60

KCL> SET VARIABLE var_name.C = ’1,0’

You must enter the KCL>SAVE VARS command to make the changes permanent.

See Also: Section 2.3 , ‘‘Data Types’’

Examples: KCL> SET VARIABLE [prog1] scale = $MCR.$GENOVERRIDE

KCL> SET VAR weld_pgm.angle = 45.0

KCL> SET VAR v[2,1,3].r = -0.897

KCL> SET VAR part_array[2] = part_array[1]

KCL> SET VAR weld_pos.x = 50.0

KCL> SET VAR pth_b[3].nodepos = pth_a[3].nodepos

C.70 SET VERIFY command

Syntax: SET VERIFY (ON | OFF)

Purpose: This turns the display of KCL commands ON or OFF during execution of a KCL command procedure (default is ON, meaning each command is displayed as it is executed). Only the RUNCF command is displayed when VERIFY is OFF.

C.71 SHOW BREAK command

Syntax: SHOW BREAK < **prog_name** >

where:

prog_name : the name of any KAREL or TP program in memory

Purpose: Displays a list of program break points for the specified or default program. The following information is displayed for each break point:

- Break point number
- Line number of the break point in the program

Examples: KCL> SHOW BREAK test_prog

KCL> SH BREAK

C.72 SHOW BUILTINS command

Syntax: SHOW BUILTINS

Purpose: Displays all the softpart built-ins that are loaded on the controller.

Example: KCL> SHOW BUILTINS

C.73 SHOW CONDITION command

Syntax: SHOW CONDITION < **prog_name** > < **condition_no** >

where:

prog_name : the name of any running or paused KAREL program

condition_no : a particular condition

Purpose: Displays the specified condition handler or a list of condition handlers for the specified or default program. Also displays enabled/disabled status and whether a break point is set. Condition handlers only exist when a program is running or paused.

Examples: KCL> SHOW CONDITION test_prog

KCL> SH COND

C.74 SHOW CLOCK command

Syntax: SHOW CLOCK

Purpose: Displays the current date and time of the controller clock.

See Also: SET CLOCK command

Example: KCL> SHOW CLOCK

C.75 SHOW CURPOS command

Syntax: SHOW CURPOS

Purpose: Displays the position of the TCP relative to the current user frame of reference with an x, y, and z location in millimeters; w, p, and r orientation in degrees; and the current configuration string. Be sure the robot is calibrated.

Example: KCL> SHOW CURPOS

C.76 SHOW DEFAULT command

Syntax: SHOW DEFAULT

Purpose: Shows the current default program name.

Example: KCL> SHOW DEFAULT

C.77 SHOW DEVICE command

Syntax: SHOW DEVICE **device_name:**

where:

device_name : device to be shown

Purpose: Shows the status of the device.

Example: KCL> SHOW DEVICE rd:

C.78 SHOW DICTS command

Syntax: SHOW DICTS

Purpose: Shows the dictionaries loaded in the system for the language specified in the system variable $LANGUAGE.

Example: KCL> SHOW DICTS

C.79 SHOW GROUP command

Syntax: SHOW GROUP

Purpose: Shows the default group number.

Example: KCL> SHOW GROUP

C.80 SHOW HISTORY command

Syntax: SHOW HISTORY

Purpose: Shows the nesting information of the routine calls. To display the source lines of KAREL programs, the .KL programs must exist on the RAM disk.

Example: KCL> SHOW HIST

C.81 SHOW LANG command

Syntax: SHOW LANG

Purpose: Shows the language specified in the system variable $LANGUAGE.

Example: KCL> SHOW LANG

C.82 SHOW LANGS command

Syntax: SHOW LANGS

Purpose: Shows all language currently available in the system.

Example: KCL> SHOW LANGS

C.83 SHOW LOCAL VARIABLE command

Syntax: SHOW LOCAL VARIABLE **var_name** <(HEXADECIMAL | BINARY)> <IN **rout_name** > <FROM **prog_name** > < **task_name** >

where:

var_name : a local variable or parameter name

rout_name : the name of any KAREL routine

prog_name : the name of any KAREL program

task_name : the name of any KAREL task

Purpose: Displays the name, type, and value of the specified local variable or routine parameter. Use brackets ([ ]) after the variable name to specify a specific ARRAY element. If you do not specify a specific element the entire variable is displayed.

If the IN clause is omitted, the routine at the top of the stack is assumed. If the FROM clause is omitted, the default program is assumed. If the **task_name** is omitted, the stack of the KCL default task is searched.

Note The file RD: **prog_name.rs** is required to obtain local variable information.

Example: Generate a .rs file from the KAREL translator.

KCL> TRANS testprog RS

Copy the .rs file to the RD device.

This is done automatically when you load the program from the KCL.

KCL> SET DEF testprog

KCL> LOAD PROG

Copied testprog.rs to RD:testprog.rs

To show local variables, the program must be running, paused, or aborted in the routine specified.

KCL> RUN

KCL> SHOW LOCAL VARS

KCL> SHOW LOCAL VARS IN testprog VALUES

KCL> SHOW LOCAL VAR var_1 IN rout_1 FROM testprog testtask

KCL> SHOW LOCAL VAR param_1

To set local variables, the program must be paused.

KCL> pause

KCL> set local var int_var = 12345

KCL> set local var strparam = "This is a string parameter"

See Also: TRANSLATE command.

C.84 SHOW LOCAL VARS command

Syntax: SHOW LOCAL VARS <VALUES> <IN **rout_name** > <FROM **prog_name** > < **task_name** >

where:

VALUES: :specifies values should be displayed

rout_name :the name of any KAREL routine

prog_name :the name of any KAREL program

task_name :the name of any KAREL task

Purpose: Displays a list including the name, type, and if specified, the current value of each local variable and each routine parameter.

If the IN clause is omitted, the routine at the top of the stack is assumed. If the FROM clause is omitted, the default program is assumed. If the **task_name** is omitted, the stack of the KCL default task is searched

Note The file RD: **prog_name.rs** is required to obtain local variable information.

Example: See SHOW LOCAL VARIABLE command.

See Also: TRANSLATE command and SHOW LOCAL VARIABLE.

C.85 SHOW MEMORY command

Syntax: SHOW MEMORY

Purpose: Displays current memory status. The command displays the following status information for memory and lists each memory pool separately:

Total number of bytes in the pool

Available number of bytes in the pool

Example: KCL> SHOW MEMORY

C.86 SHOW PROGRAM command

Syntax: SHOW PROGRAM < **prog_name** >

where:

prog_name : the name of any KAREL or TP program in memory

Purpose: Displays the status information of the specified or default program being executed.

Example: KCL> SHOW PROGRAM test_prog

KCL> SH PROG

C.87 SHOW PROGRAMS command

Syntax: SHOW PROGRAMS

Purpose: Shows a list of programs and variable data that are currently loaded in memory.

Examples: KCL> SHOW PROGRAMS

KCL> SH PROGS

C.88 SHOW SYSTEM command

Syntax: SHOW SYSTEM < **data_type** > <VALUES>

where:

data_type : any valid KAREL data type

Purpose: Displays a list including the name, type, and if specified, the current value of each system variable. If you specify a **data_type** , only the system variables of that type are listed.

See Also: SHOW VARIABLE command

Examples: KCL> SHOW SYSTEM REAL VALUES

KCL> SH SYS

C.89 SHOW TASK command

Syntax: SHOW TASK < **prog_name** >

where:

prog_name : the name of any KAREL or TP program which is a task

Purpose: Displays the task control data for the specified task. If **prog_name** is not specified, the default program is used.

Examples: KCL> SHOW TASK test_prog

KCL> SH TASK

C.90 SHOW TASKS command

Syntax: SHOW TASKS

Purpose: Displays the status of all known tasks running KAREL programs or TP programs.

You may see extra tasks running that are not yours. If the teach pendant is displaying a menu that was written using KAREL, such as Program Adjustment or Setup Passwords, you will see the status for this task also.

Examples: KCL> SHOW TASKS

C.91 SHOW TRACE command

Syntax: SHOW TRACE < **prog_name** >

where:

prog_name : the name of any KAREL or TP program which is a task

Purpose: Shows all the program statements and line numbers that have been executed since TRACE has been turned on.

The number of lines that are shown depends on the trace buffer length, which can be set with the SET_TASK command or the SET_TSK_ATTR built-in routine. To display the source lines of KAREL programs, the .KL files must exist on the RAM disk.

See Also: SET TRACE command

Example: KCL> SHOW TRACE

C.92 SHOW TYPES command

Syntax: SHOW TYPES < **prog_name** > < **FIELDS** >

where:

prog_name : the name of any KAREL or TP program

FIELDS : specifies fields should be displayed

Purpose: Displays a list including the name, type, and if specified, the fields of each user-defined type in the specified or default program. The actual array dimensions and string sizes are not shown.

See Also: SHOW VARS command, SHOW VARIABLE command

Examples: KCL> SHOW TYPES test_prog FIELDS

KCL> SH TYPES

C.93 SHOW VARIABLE command

Syntax: SHOW VARIABLE <[**prog_name**]> **var_name** <(HEXADECIMAL | BINARY)>

where:

prog_name : the name of any KAREL or TP program

var_name : a valid program variable

Purpose: Displays the name, type, and value of the specified variable.

You can display the values of system variables that allow KCL read access or the values of program variables. Use brackets ([]) after the variable name to specify a specific ARRAY element. If you do not specify a specific element the entire variable is displayed.

See Also: SHOW VARS command, SHOW SYSTEM command

Examples: KCL> SHOW VARIABLE $UTOOL

KCL> SH VAR [test_prog]group_mask HEX

KCL> SH VAR [test_prog]group_mask BINARY

KCL> SH VAR weld_pth[3]

C.94 SHOW VARS command

Syntax: SHOW VARS < **prog_name** > <VALUES>

where:

prog_name : the name of any KAREL or TP program

VALUES : specifies values should be displayed

Purpose: Displays a list including the name, type and, if specified, the current value of each variable in the specified or default program.

See Also: SHOW VARIABLE command, SHOW SYSTEM command, SHOW TYPES command

Example: KCL> SHOW VARS test_prog VALUES

KCL> SH VARS

C.95 SHOW data_type command

Syntax: SHOW **data_type** < **prog_name** > <VALUES>

where:

data_type : any valid KAREL data type

prog_name : the name of any KAREL or TP program

VALUES : specifies values should be displayed

Purpose: Displays a list of variables in the specified or default program (**prog_name**) of the specified data type (**data_type**). The list includes the name, type, and if specified, the current value of each variable.

See Also: SHOW VARS command, SHOW VARIABLE command

Examples: KCL> SHOW REAL test_prog VALUES

KCL> SH INTEGER

C.96 SIMULATE command

Syntax: SIMULATE **port_name[index]** < **= value** >

where:

port_name[index] : a valid I/O port

value : a new value for the port

Purpose: Simulating I/O allows you to test a program that uses I/O. Simulating I/O does not actually send output signals or receive input signals.



Warning

Depending on how signals are used, simulating signals might alter program execution. Do not simulate signals that are set up for safety checks. If you do, you could injure personnel or damage equipment.

When simulating a port value, you can specify its initial simulated value or allow the initial value to be the same as the physical port value. If no value is specified, the current physical port value is used.

The valid ports are:

DIN, DOUT, WDI, WDO (BOOLEAN)AIN, AOUT, GIN, GOUT (INTEGER)

See Also: UNSIMULATE command

Examples: KCL> SIMULATE DIN[17]

KCL> SIM DIN[1] = ON

KCL> SIM AIN[1] = 100

C.97 SKIP command

Syntax: SKIP < **prog_name** >

where:

prog_name : the name of any KAREL or TP program which is a task

Purpose: Skips execution of the current statement in the specified task. If **prog_name** is not specified, the default program is used. It has no effect when a task is running or when the system is in a READY state.

Entire motion statements are skipped with this command. You cannot skip single motion segments. The KCL> CONTINUE command resumes execution of the paused task with the statement following the last skipped statement. END statements cannot be skipped.

If you skip the last RETURN statement in a function routine, there is no way to return the value of the function to the calling program. Therefore, when executing the END statement of the routine, the task will abort.

If you skip into a FOR loop, you have skipped the statement that initializes the loop counter. When the ENDFOR statement is executed the program will try to remove the loop counter from the stack. If the FOR loop was nested in another FOR loop, the loop counter for the previous FOR loop will be removed from the stack, causing potentially invalid results. If the FOR loop was not nested, a stack underflow error will occur, causing the task to abort.

READ, MOVE, DELAY, WAIT FOR, and PULSE statements can be paused after they have begun execution. In these cases, when the task is resumed, execution of the paused statement must be finished before subsequent statements are executed. Subsequent skipped statements will not be executed. In particular, READ and WAIT FOR statements often require user intervention, such as entering data, before statement execution is completed.

Step mode operation and step mode type have no effect on the KCL> SKIP command.

Examples: KCL> SKIP test_prog

KCL> SKIP

C.98 STEP OFF command

Syntax: STEP OFF

Purpose: Disables single stepping for the program in which it was enabled.

Example: KCL> STEP OFF

C.99 STEP ON command

Syntax: STEP ON < **prog_name** >

where:

prog_name : the name of any KAREL or TP program which is a task

Purpose: Enables single stepping for the specified or default program.

Examples: KCL> STEP ON test_prog

KCL> STEP ON

C.100 TRANSLATE command

Syntax: TRANSLATE < **file_spec** > <DISPLAY> <LIST> <RS>

where:

file_spec : a valid file specification

DISPLAY : display source during translation

LIST : create listing file

RS : create routine stack (.rs) file for local var access

Purpose: Translates KAREL source code (.KL type files) into p-code (.PC type files), which can be loaded into memory and executed.

Translation of a program can be canceled using the CANCEL COMMAND key, CTRL-C, or CTRL-Y on the CRT/KB.

Examples: KCL> TRANSLATE testprog DISPLAY LIST

KCL> TRAN

C.101 TYPE command

Syntax: TYPE **file_spec**

where:

file_spec : a valid file specification

Purpose: This command allows you to display the contents of the specified ASCII file on the CRT/KB. You can specify any type of ASCII file.

Examples: KCL> TYPE rd:testprog.kl

KCL> TYPE testprog.kl

C.102 UNSIMULATE command

Syntax: UNSIMULATE (**port_name[index]** | **ALL**)

where:

port_name[index] : a valid I/O port

ALL : all simulated I/O ports

Purpose: Discontinues simulation of the specified input or output port. When a port is unsimulated, the physical value replaces the simulated value.



Warning

Depending on how signals are used, unsimulating signals might alter program execution or activate peripheral equipment. Do not unsimulate a signal unless you are sure of the result. If you do, you could injure personnel or damage equipment.

If you specify ALL instead of a particular port, simulation on all the simulated ports is discontinued.

The valid ports are:

DIN, DOUT, WDI, WDOAIN, AOUT, GIN, GOUT

See Also: SIMULATE command

Examples: KCL> UNSIMULATE DIN[17]

KCL> UNSIM ALL

C.103 WAIT command

Syntax: WAIT < **prog_name** > (DONE | PAUSE)

where:

prog_name : the name of any KAREL or TP program which is a task

DONE : specifies that the command procedure wait until execution of the current task is completed or aborted

PAUSE : specifies that the command procedure wait until execution of the current task is paused, completed, or aborted.

Purpose: Defers execution of the commands that follow the KCL> WAIT command in a command procedure until a task pauses or completes execution.

The command procedure waits until the condition specified with the DONE or PAUSE argument is met.

See Also: Section 13.4 , "Command Procedures"

Example: The following is an example of an executable command procedure:

```
> SET DEF testprog
> LOAD ALL
> RUN -- execute program
> WAIT PAUSE
> SHOW CURPOS -- display position of TCP when program pauses
> CONTINUE
> WAIT DONE
> CLEAR ALL YES -- clear after execution
```

附录D.

字符代码

目录

附录D 字符代码 D–1
D.1 字符代码 D–2

D.1 CHARACTER CODES

This appendix lists the ASCII numeric decimal codes and their corresponding ASCII, Multinational, graphic, and European characters as implemented on the KAREL system. The ASCII character set is the default character set for the KAREL system. Use the CHR Built-In Function, in Appendix A, to access the Multinational and Graphics character sets.

Table D–1. ASCII Character Codes

Decimal Code	Character Value	Decimal Code	Character Value	Decimal Code	Character Value	Decimal Code	Character Value
000	(NUL)	032	SP	064	@	096	‘
001	(SOH)	033	!	065	A	097	a
002	(STX)	034	"	066	B	098	b
003	(ETX)	035	#	067	C	099	c
004	(EOT)	036	$	068	D	100	d
005	(ENQ)	037	%	069	E	101	e
006	(ACK)	038	&	070	F	102	f
007	(BEL)	039	’	071	G	103	g
008	(BS)	040	(	072	H	104	h
009	(HT)	041	)	073	I	105	i
010	(LF)	042	*	074	J	106	j
011	(VT)	043	+	075	K	107	k
012	(FF)	044	’	076	L	108	l
013	(CR)	045	-	077	M	109	m
014	(SO)	046	.	078	N	110	n
015	(SI)	047	/	079	O	111	o
016	(DLE)	048	0	080	P	112	p
017	(DC1)	049	1	081	Q	113	q
018	(DC2)	050	2	082	R	114	r
019	(DC3)	051	3	083	S	115	s
020	(DC4)	052	4	084	T	116	t
021	(NAK)	053	5	085	U	117	u
022	(SYN)	054	6	086	V	118	v
023	(ETB)	055	7	087	W	119	w

Table D–1. ASCII Character Codes (Cont'd)

Decimal Code	Character Value	Decimal Code	Character Value	Decimal Code	Character Value	Decimal Code	Character Value
024	(CAN)	056	8	088	X	120	x
025	(EM)	057	9	089	Y	121	y
026	(SUB)	058	:	090	Z	122	z
027	(ESC)	059	;	091	[	123	{
028	(FS)	060	<	092	\	124	|
029	(GS)	061	=	093	]	125	}
030	(RS)	062	>	094	^	126	~
031	(US)	063	?	095	—	127	(DEL)

Table D–2. Special ASCII Character Codes

Decimal Code	Character Value	Decimal Code	Character Value
128	Clear window	154	Turn Multinational mode on
129	Clear to end of line	155 48	Foreground color black
130	Clear to end of window	155 49	Foreground color red
131	Set cursor position	155 50	Foreground color green
132	Carriage return	155 51	Foreground color yellow
133	Line feed	155 52	Foreground color blue
134	Reverse line feed	155 53	Foreground color magenta
135	Carriage return & line feed	155 54	Foreground color cyan
136	Back Space	155 55	Foreground color white

Table D–2. Special ASCII Character Codes (Cont'd)

Decimal Code	Character Value	Decimal Code	Character Value
137	Home cursor in window	155 127	Foreground color default
138	Blink video attribute	156 48	Background color black
139	Reverse video attribute	156 49	Background color red
140	Bold video attribute	156 50	Background color green
141	Underline video attribute	156 51	Background color yellow
142	Wide video size	156 52	Background color blue
143	Normal video attribute	156 53	Background color magenta
146	Turn Graphics mode on	156 54	Background color cyan
147	Turn ASCII mode on	156 55	Background color white
148	High/wide video size	156 127	Background color default
153	Normal video size		

Table D–3. Multinational Character Codes

Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value
000		032		064	À	096	à
001		033	¡	065	Á	097	á
002		034	©	066	Â	098	â
003		035	£	067	Ã	099	ã
004	(IND)	036		068	Ä	100	ä
005	(NEL)	037	¥	069	Å	101	å
006	(SSA)	038		070	Æ	102	æ
007	(ESA)	039	§	071	Ç	103	ç

Table D–3. Multinational Character Codes (Cont'd)

Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value
008	(HTS)	040	¤	072	È	104	è
009	(HTJ)	041	©	073	É	105	é
010	(VTS)	042	ª	074	Ê	106	ê
011	(PLD)	043	«	075	Ë	107	ë
012	(PLU)	044		076	Ì	108	ì
013	(RI)	045		077	Í	109	í
014	(SS2)	046		078	Î	110	î
015	(SS3)	047		079	Ï	111	ï
016	(DCS)	048	○	080		112	
017	(PU1)	049	±	081	Ñ	113	ñ
018	(PU2)	050	²	082	Ò	114	ò
019	(STS)	051	³	083	Ó	115	ó
020	(CCH)	052		084	Ô	116	ô
021	(MW)	053	µ	085	Õ	117	õ
022	(SPA)	054	¶	086	Ö	118	ö
023	(EPA)	055	•	087	Œ	119	œ
024		056		088	Ø	120	ø
025		057	¹	089	Ù	121	ù
026		058		090	Ú	122	ú
027	(CSI)	059	»	091	Û	123	û
028	(ST)	060	¼	092	Ü	124	ü
029	(OSC)	061	½	093	Y	125	ÿ
030	(PM)	062		094		126	
031	(APC)	063	¿	095	ß	127	

Table D–4. Graphics Character Codes (not available in R-30iB)

Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value
000	(NUL)	032	SP	064	@	096	♦
001	(SOH)	033	!	065	A	097	
002	(STX)	034	"	066	B	098	HT
003	(ETX)	035	#	067	C	099	FF
004	(EOT)	036	$	068	D	100	CR
005	(ENQ)	037	%	069	E	101	LF
006	(ACK)	038	&	070	F	102	○
007	(BEL)	039	’	071	G	103	±
008	(BS)	040	(	072	H	104	NL
009	(HT)	041	)	073	I	105	VT
010	(LF)	042	*	074	J	106	┘
011	(VT)	043	+	075	K	107	┐
012	(FF)	044	’	076	L	108	┌
013	(CR)	045	-	077	M	109	└
014	(SO)	046	.	078	N	110	+
015	(SI)	047	/	079	O	111	-
016	(DLE)	048	0	080	P	112	-
017	(DC1)	049	1	081	Q	113	-
018	(DC2)	050	2	082	R	114	-
019	(DC3)	051	3	083	S	115	_
020	(DC4)	052	4	084	T	116	├

Table D–4. Graphics Character Codes (not available in R-30iB) (Cont'd)

Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value	Decimal Codes	Character Value
021	(NAK)	053	5	085	U	117	┣
022	(SYN)	054	6	086	V	118	┻
023	(ETB)	055	7	087	W	119	┳
024	(CAN)	056	8	088	X	120	|
025	(EM)	057	9	089	Y	121	≤
026	(SUB)	058	:	090	Z	122	≥
027	(ESC)	059	;	091	[	123	Π
028	(FS)	060	<	092		124	≠
029	(GS)	061	=	093	]	125	£
030	(RS)	062	>	094	^	126	•
031	(US)	063	?	095	(blank)	127	(DEL)

Table D–5. Teach Pendant Input Codes

Code	Value	Code	Value
0	48	174	USER KEY 2
1	49	175	USER KEY 3
2	50	176	USER KEY 4
3	51	177	USER KEY 5
4	52	178	USER KEY 6
5	53	185	FWD
6	54	186	BWD
7	55	187	COORD

Table D–5. Teach Pendant Input Codes (Cont'd)

Code	Value	Code	Value
8	56	188	+X
9	57	189	+Y
128	PREV	190	+Z
129	F1	191	+X rotation
131	F2	192	+Y rotation
132	F3	193	+Z rotation
133	F4	194	-X
134	F5	195	-Y
135	NEXT	196	-Z
143	SELECT	197	-X rotation
144	MENUS	198	-Y rotation
145	EDIT	199	-Z rotation
146	DATA	210	USER KEY
147	FCTN	212	Up arrow
148	ITEM	213	Down arrow
149	+%	214	Right arrow
150	-%	215	Left arrow
151	HOLD	147	DEADMAN switch, left
152	STEP	248	DEADMAN switch, right
153	RESET	249	ON/OFF switch
173	USER KEY 1	250	EMERGENCY STOP

Table D–6. European Character Codes

Code	Value	Code	Value	Code	Value
192	A‘	213	O~	234	e^
193	A‘	214	O:	235	e:
194	A^	215	OE	236	i‘
195	A~	216	O/	237	i‘
196	A:	217	U‘	238	i^
197	Ao	218	U‘	239	i:

Table D–6. European Character Codes (Cont'd)

Code	Value	Code	Value	Code	Value
198	AE	219	U^	240	
199	CC	220	U:	241	n~
200	E‘	221	Y:	242	o‘
201	E‘	222		243	o‘
202	E^	223	Bb	244	o^
203	E:	224	a‘	245	o~
204	I‘	225	a‘	246	o:
205	I‘	226	a^	247	oe
206	I^	227	a~	248	
207	I:	228	a:	249	u‘
208		229	ao	250	u‘
209	N~	230	ae	251	u^
210	O‘	231		252	u:
211	O‘	232	e‘	253	y:
212	O^	233	e‘	254	

A^ = A with ^ on top

A‘ = A with ‘ on top

Ao = A with o on top

A~ = A with ~ on top

A: = A with .. on top

AE = A and E run together

OE = A and E run together

Bb = Beta

Table D–7. Graphics Characters

Decimal Value	ASCII Character	Graphic Character
97	a	solid box
102	f	diamond
103	g	plus/minus
106	j	lower-right box corner
107	k	upper-right box corner
108	l	upper-left box corner
109	m	lower-left box corner
110	n	intersection lines
111	o	pixel row 1 horizontal line
112	p	pixel row 2 horizontal line
113	q	pixel row 3 horizontal line
114	r	pixel row 4 horizontal line
115	s	pixel row 5 horizontal line
116	t	T from right
117	u	T from left
119	v	T from above
119	w	T from below
120	x	Vertical Line
121	y	Less than or equal
122	z	Greater than or equal
123	{	Pi
124	|	Not equal
125	}	British pound symbol

附录E.

语法图

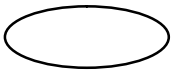
目录

附录E 语法图 E–1

KAREL syntax diagrams use the following symbols:

- Rectangle 

  A rectangle encloses elements that are defined in another syntax diagram or in accompanying text.

- Oval 

  An oval encloses KAREL reserved words that are entered exactly as shown.

- Circle 

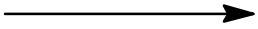
  A circle encloses special characters that are entered exactly as shown.

- Dot 

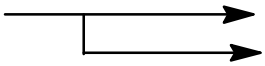
  A dot indicates a mandatory line end (; or ENTER key) before the next syntax element.

- Caret ^

  A caret indicates an optional line end.

- Arrows 

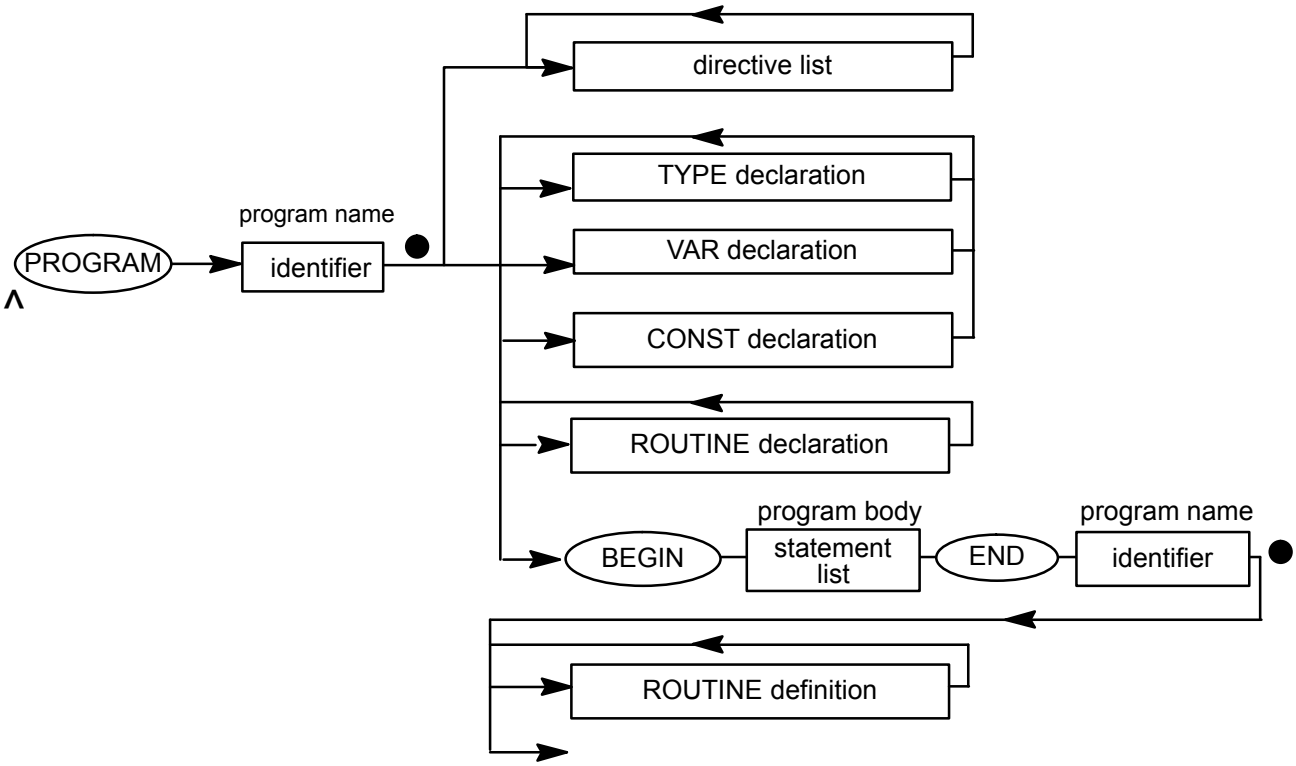
  Arrows indicate allowed paths and the correct sequence in a diagram.

- Branch 

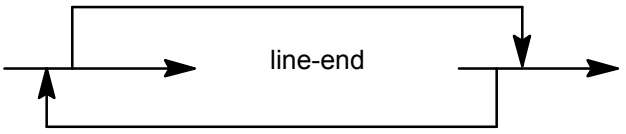
  Branches indicate optional paths or sequences.

Figure E–1.

PROGRAM□ □module definition



^ -- 0 or more line-ends



● --newline

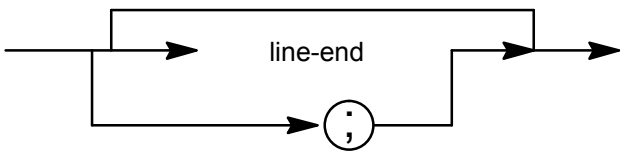


Figure E–2.

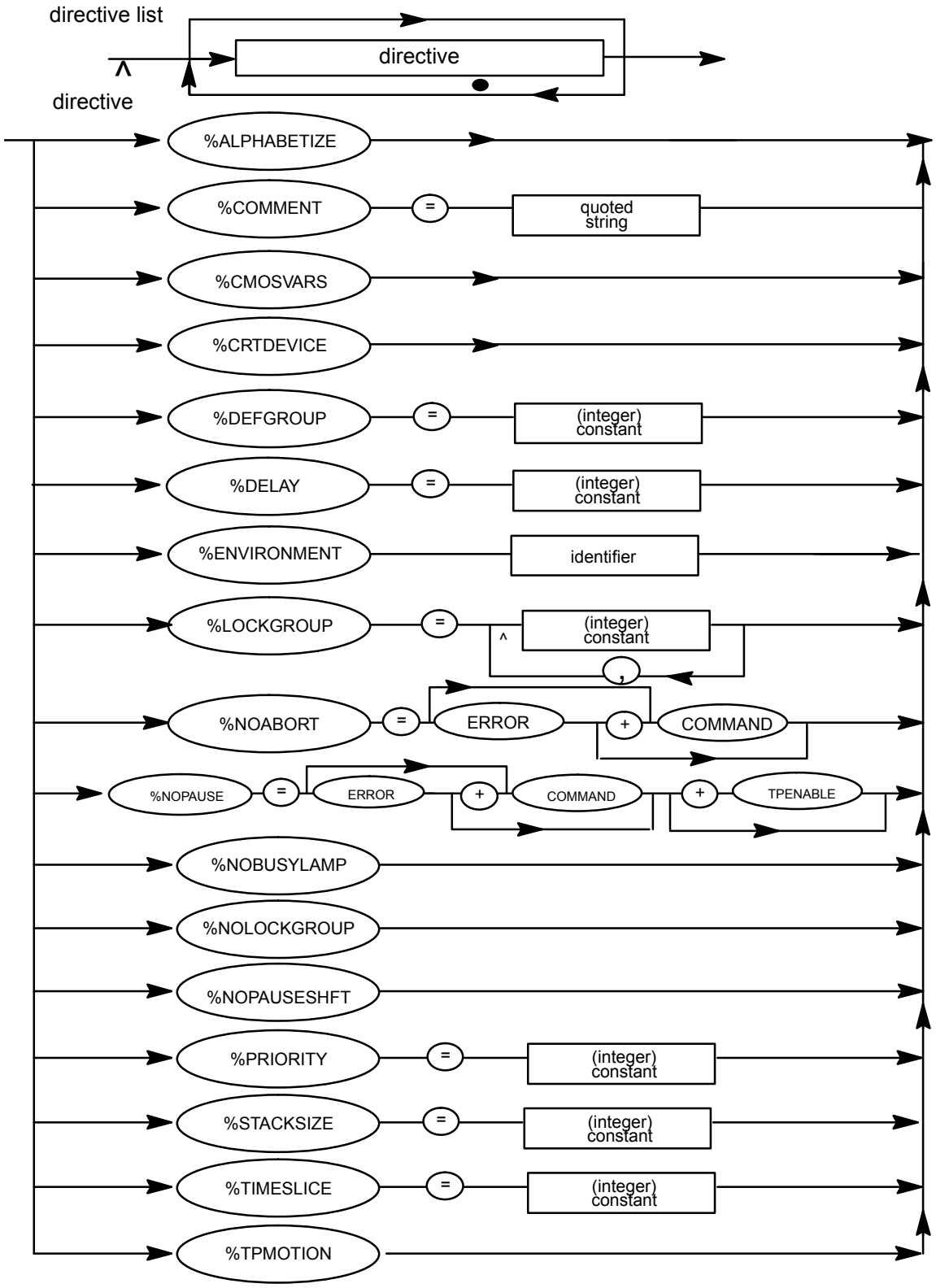

directive list
directive
directive
%ALPHABETIZE
%COMMENT
=
quoted string
%CMOSVARS
%CRTDEVICE
%DEFGROUP
=
(integer) constant
%DELAY
=
(integer) constant
%ENVIRONMENT
identifier
%LOCKGROUP
=
^
(integer) constant
,
%NOABORT
=
ERROR
+
COMMAND
%NOPAUSE
=
ERROR
+
COMMAND
+
TPENABLE
%NOBUSYLAMP
%NOLOCKGROUP
%NOPAUSESHFT
%PRIORITY
=
(integer) constant
%STACKSIZE
=
(integer) constant
%TIMESLICE
=
(integer) constant
%TPMOTION

Figure E–3.

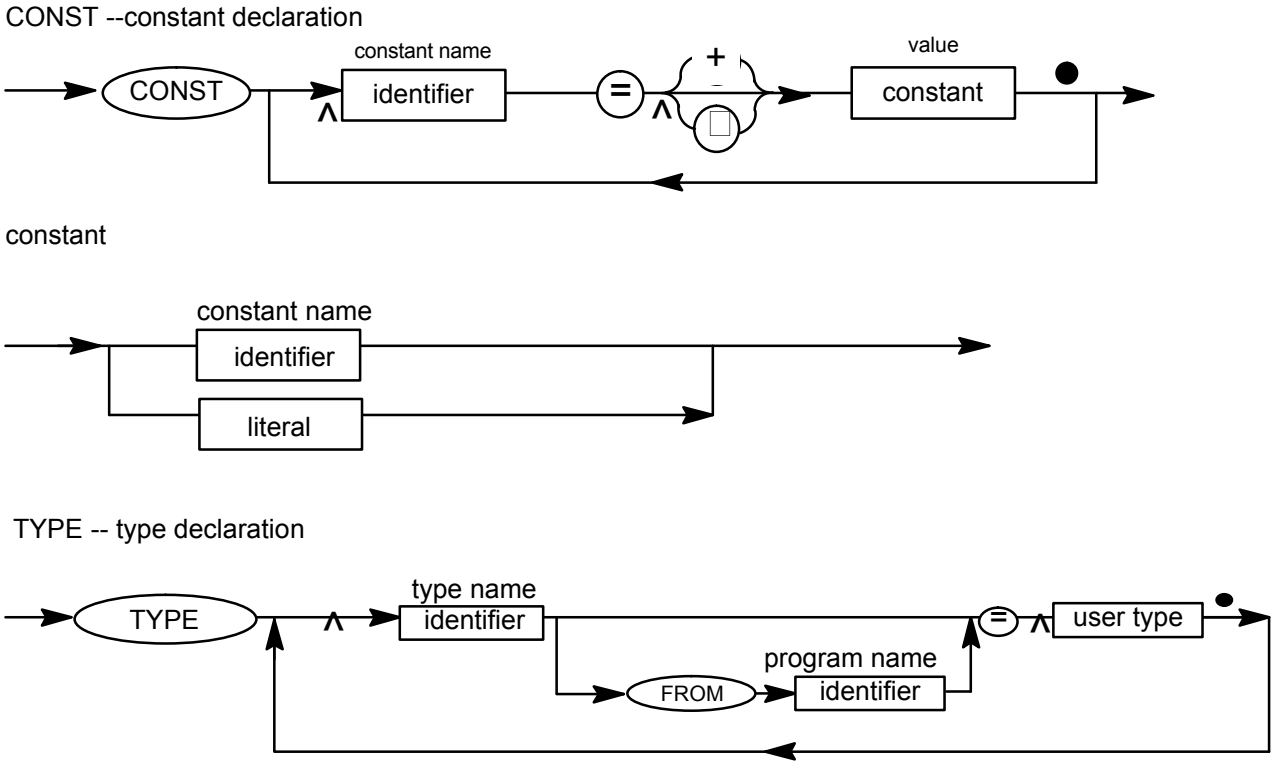
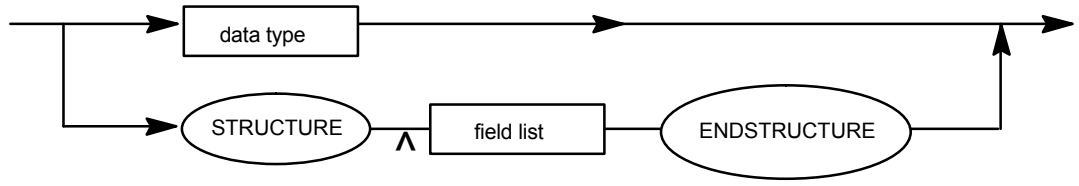
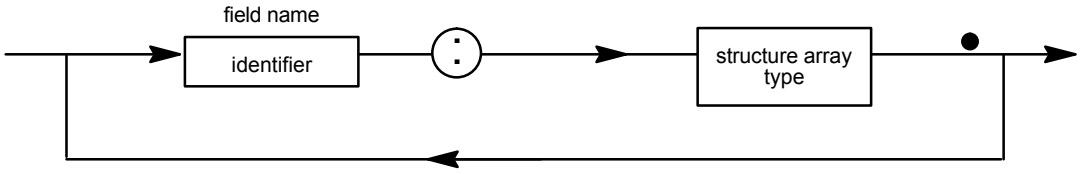

CONST --constant declaration
constant name
value
CONST
identifier
=
+
-
constant
constant
constant name
identifier
literal
TYPE -- type declaration
type name
TYPE
identifier
=
user type
program name
FROM
identifier

Figure E–4.

user type



field list



structure array type

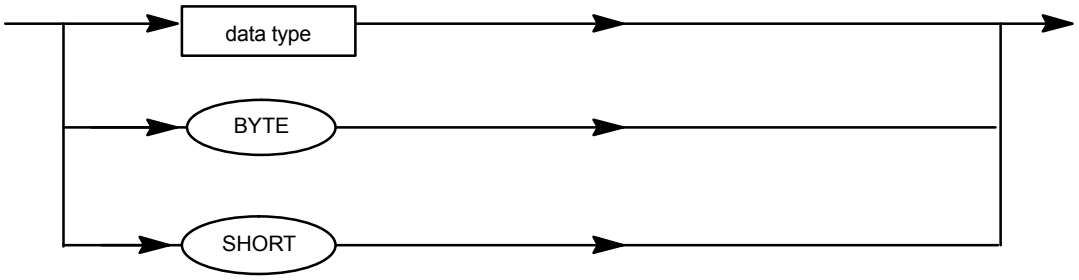


Figure E–5.

data type

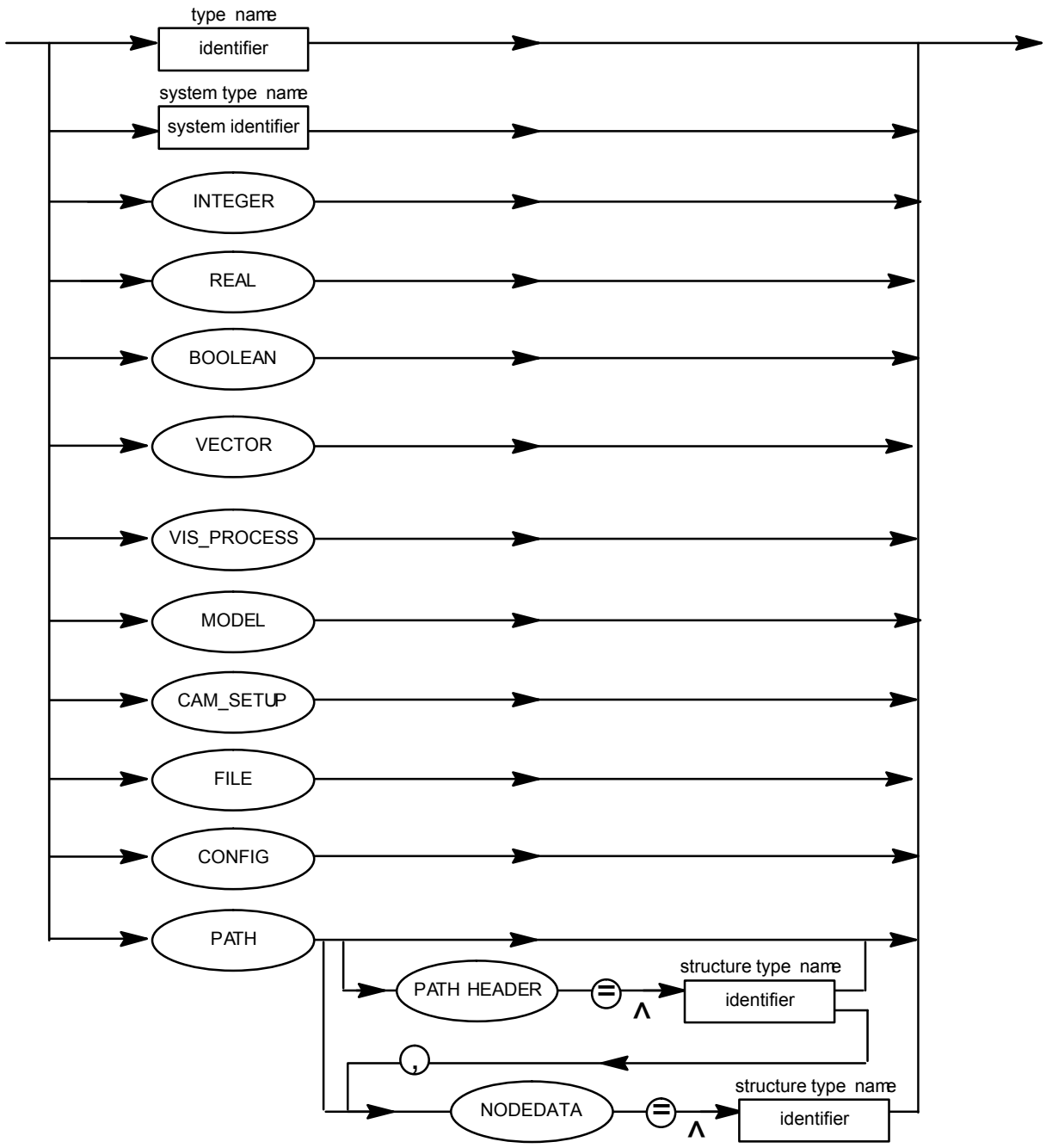


Figure E–6.

data type continued

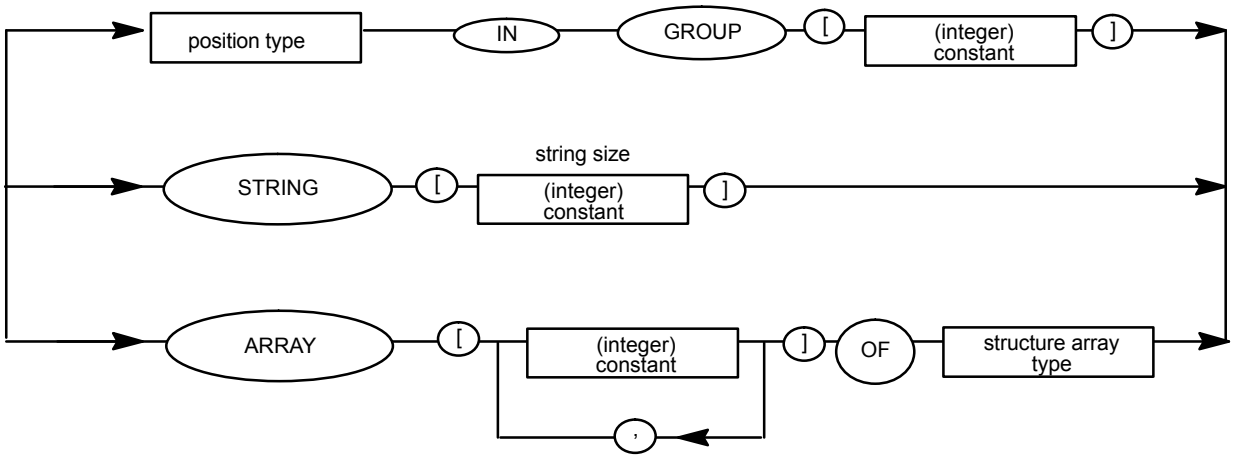


Figure E–7.

position type

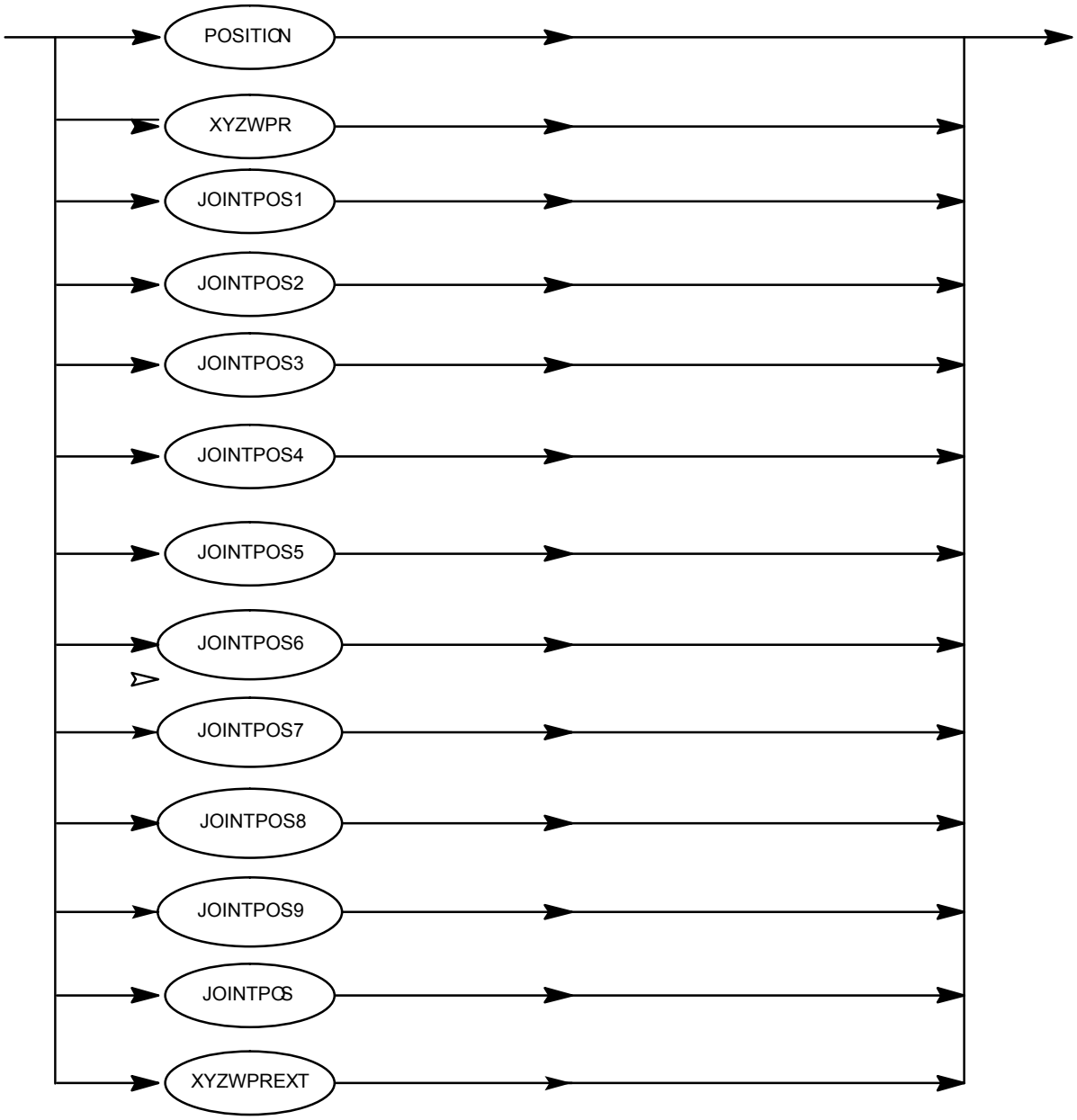
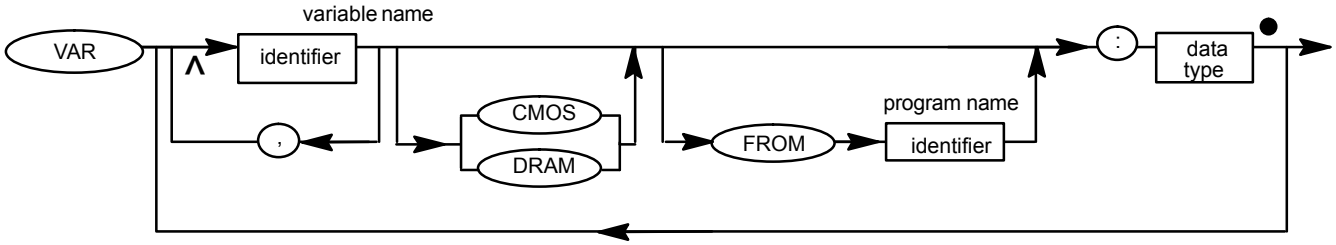


Figure E–8.

VAR -- variable declaration



ROUTINE -- routine declaration

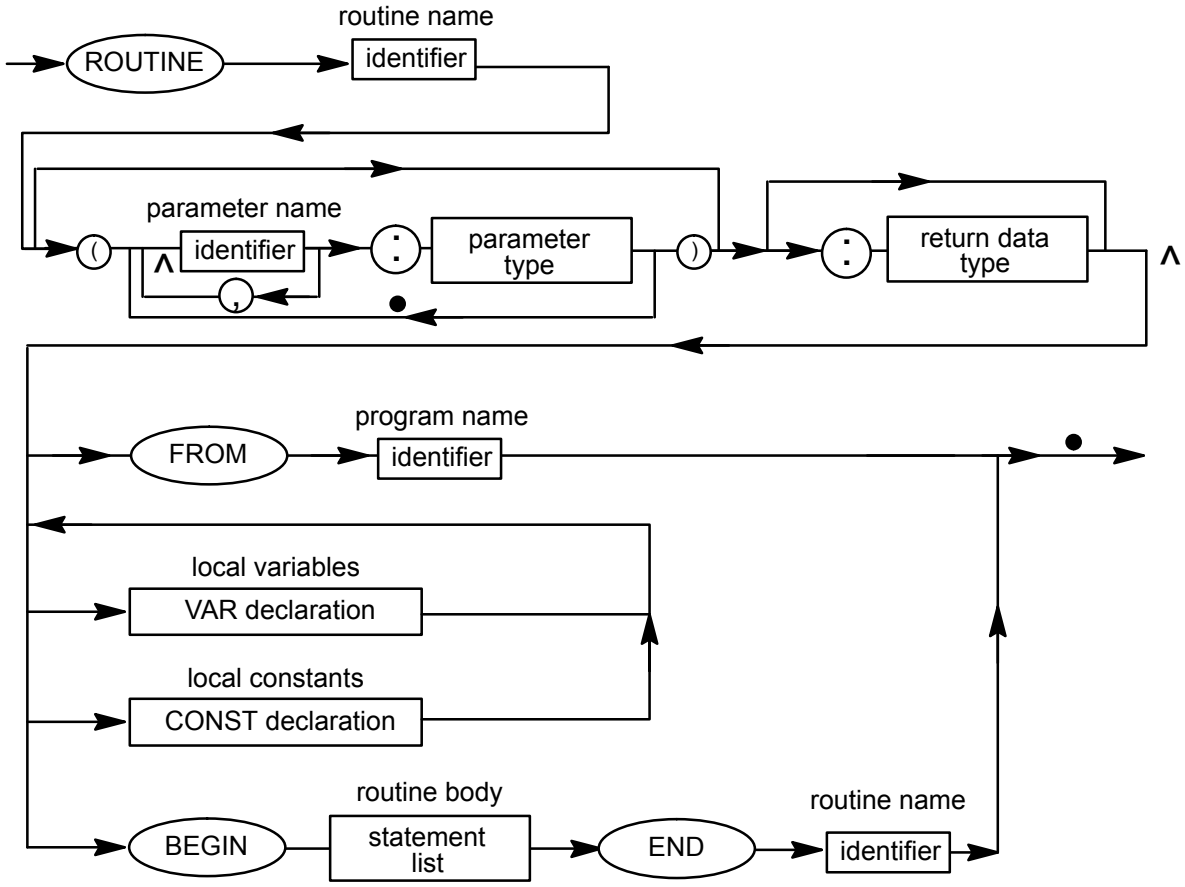


Figure E–9.

return data type

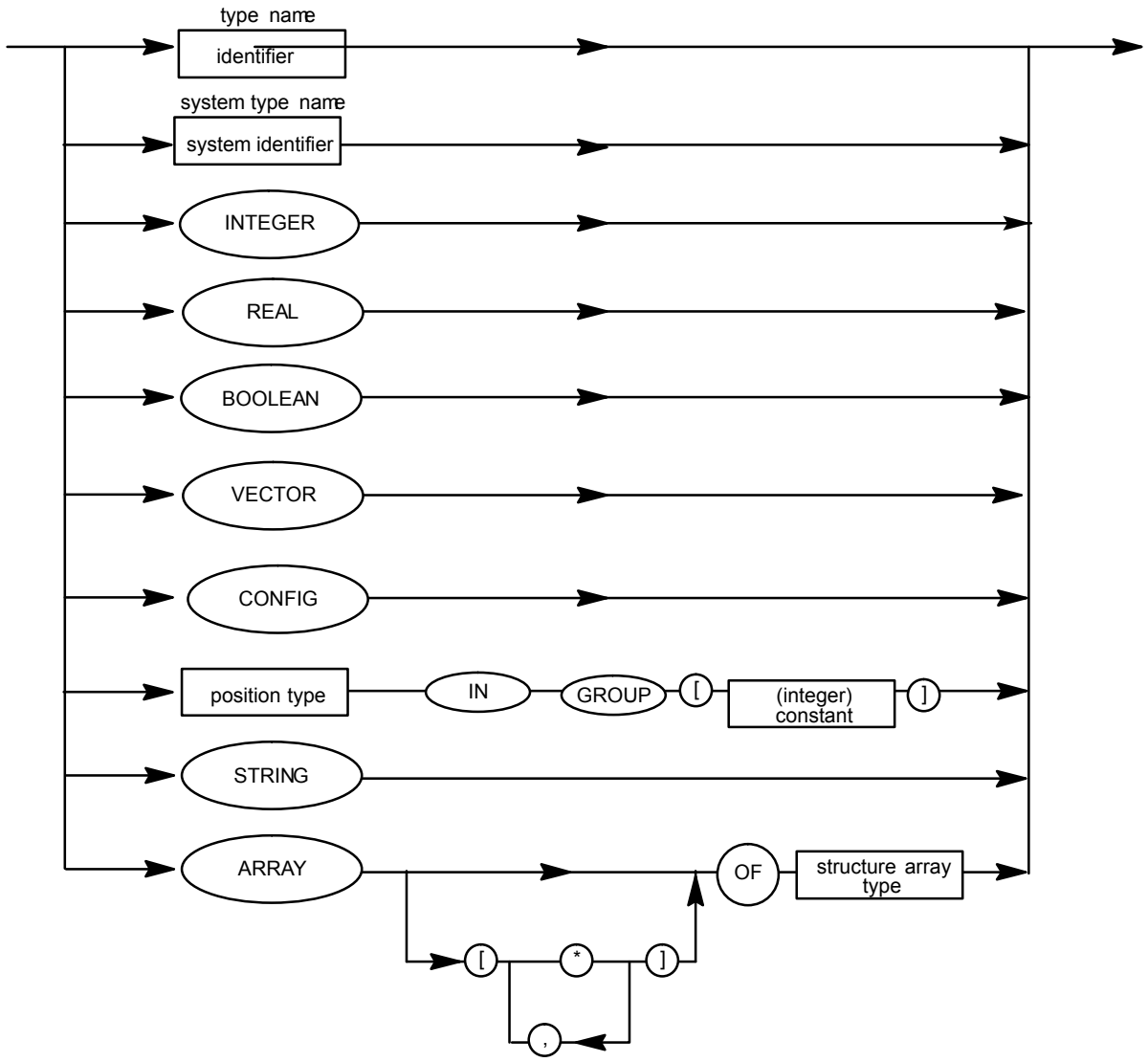


Figure E–10.

parameter type

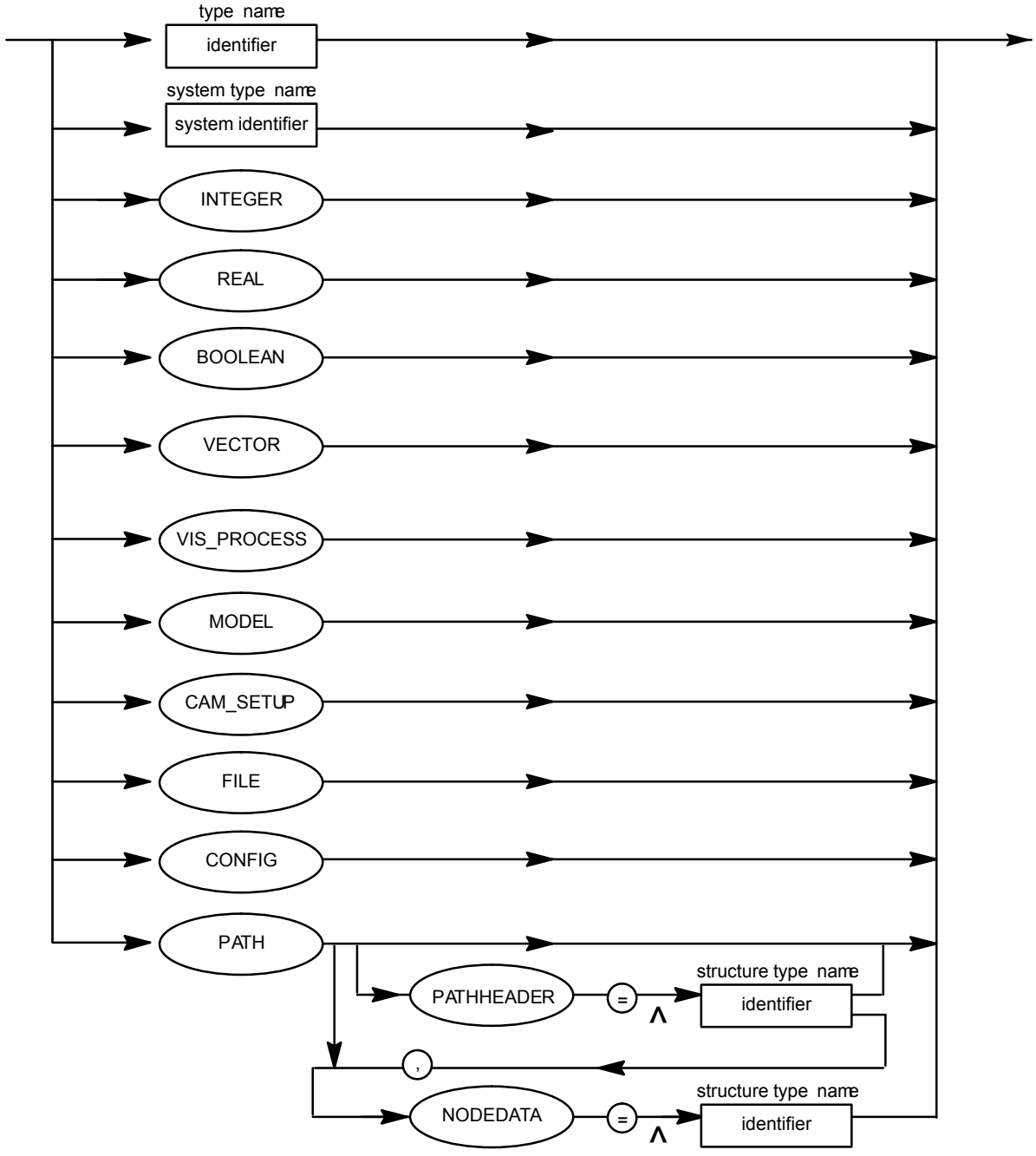


Figure E–11.

parameter type continued

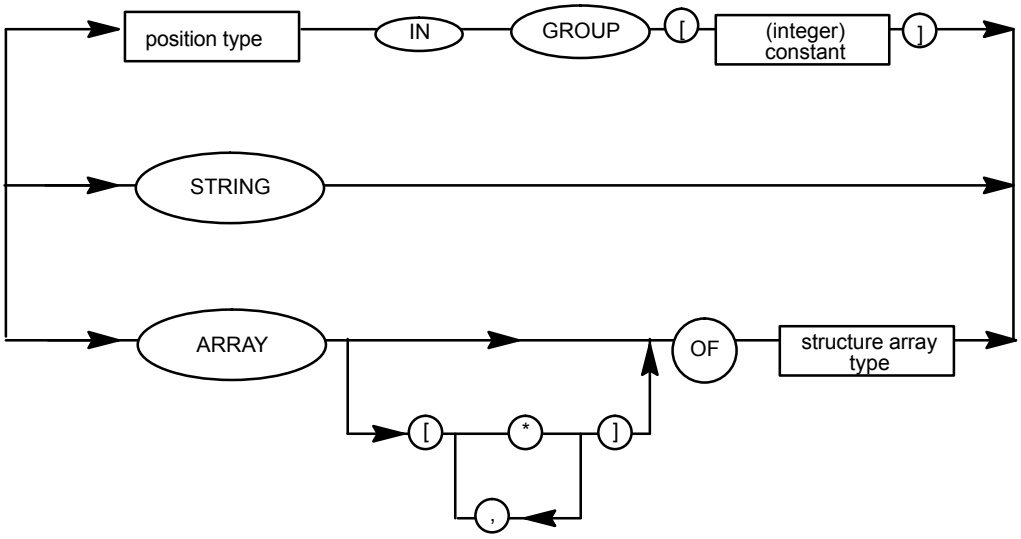


Figure E–12.

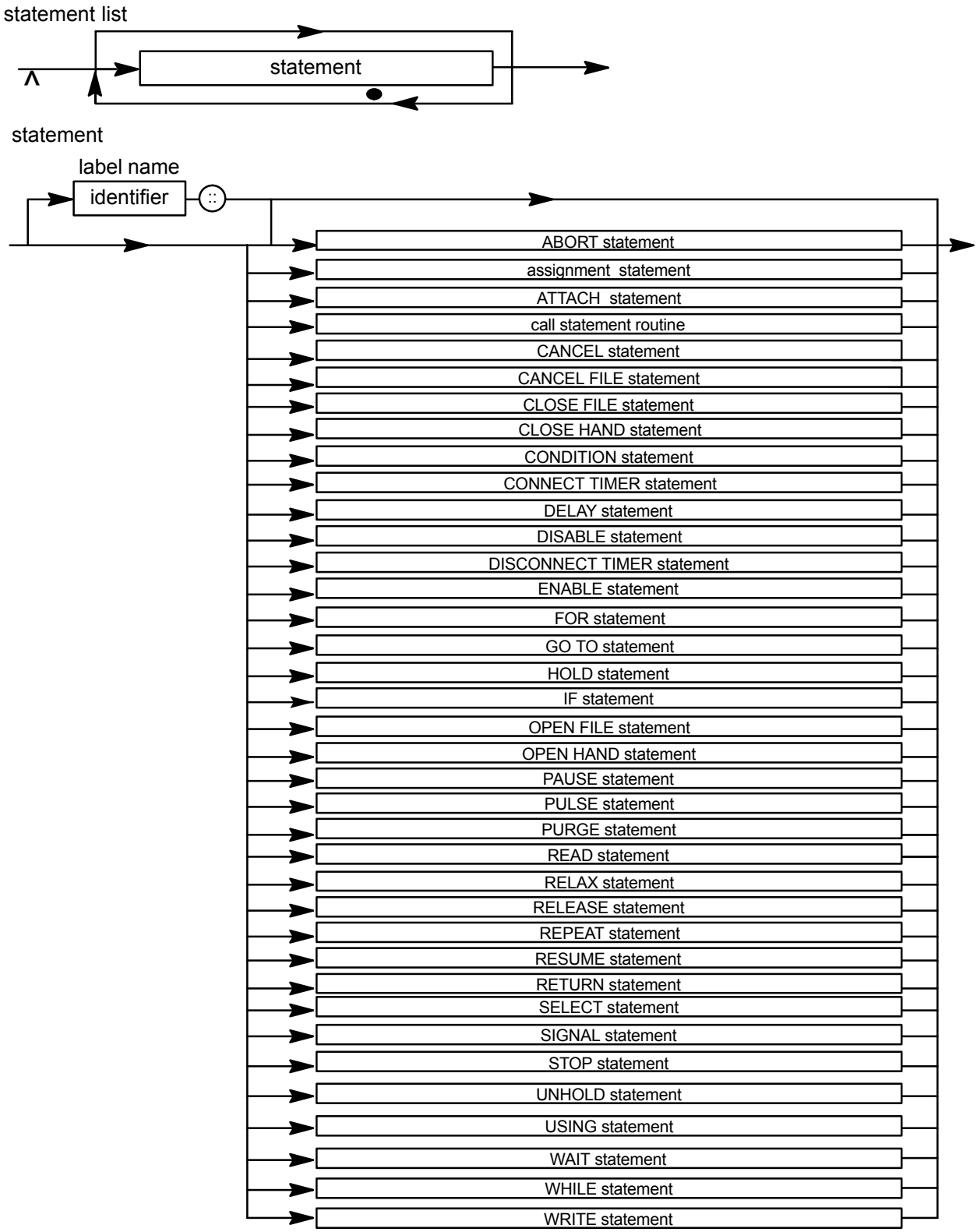

statement list
statement
statement
label name
identifier
::
ABORT statement
assignment statement
ATTACH statement
call statement routine
CANCEL statement
CANCEL FILE statement
CLOSE FILE statement
CLOSE HAND statement
CONDITION statement
CONNECT TIMER statement
DELAY statement
DISABLE statement
DISCONNECT TIMER statement
ENABLE statement
FOR statement
GO TO statement
HOLD statement
IF statement
OPEN FILE statement
OPEN HAND statement
PAUSE statement
PULSE statement
PURGE statement
READ statement
RELAX statement
RELEASE statement
REPEAT statement
RESUME statement
RETURN statement
SELECT statement
SIGNAL statement
STOP statement
UNHOLD statement
USING statement
WAIT statement
WHILE statement
WRITE statement

Figure E–13.

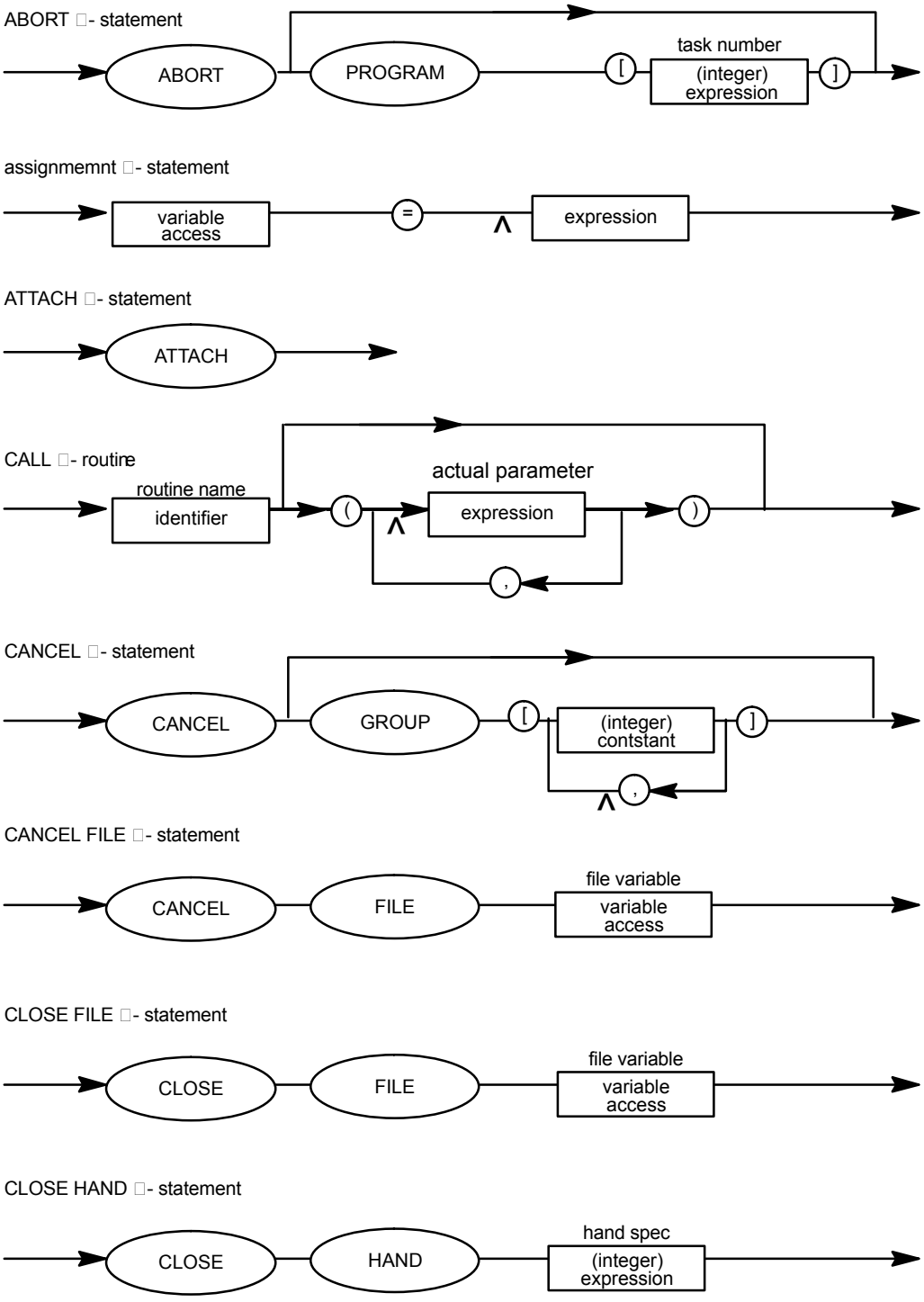
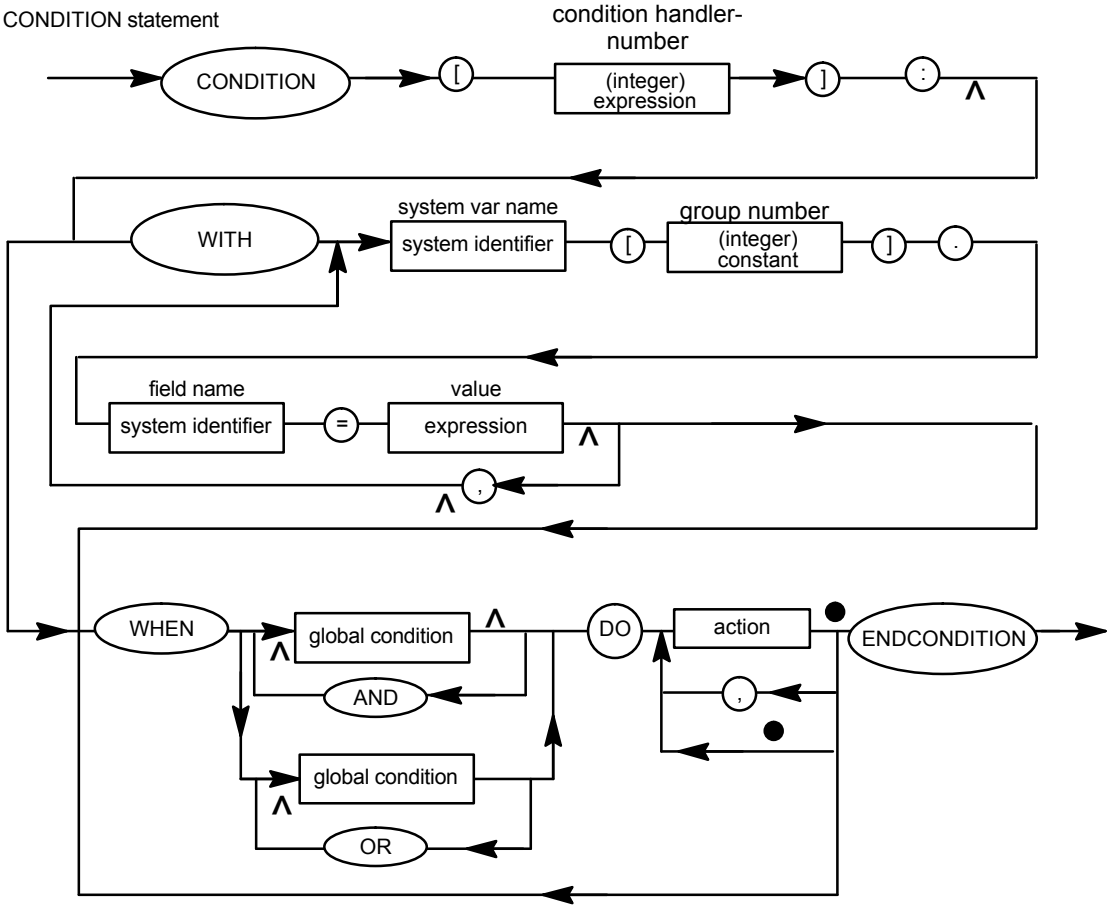

ABORT □- statement
ABORT
PROGRAM
task number
[
(integer)
expression
]
assignmemnt □- statement
variable
access
=
expression
ATTACH □- statement
ATTACH
CALL □- routine
routine name
identifier
(
actual parameter
expression
)
,
CANCEL □- statement
CANCEL
GROUP
[
(integer)
contstant
]
,
CANCEL FILE □- statement
CANCEL
FILE
file variable
variable
access
CLOSE FILE □- statement
CLOSE
FILE
file variable
variable
access
CLOSE HAND □- statement
CLOSE
HAND
hand spec
(integer)
expression

Figure E–14.

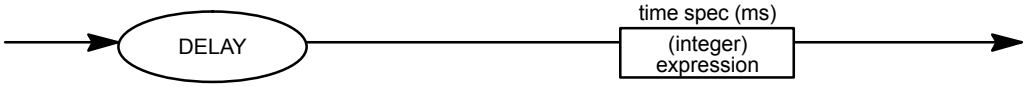
CONDITION statement



CONNECT TIMER □- statement



DELAY □- statement



DISABLE □- statement

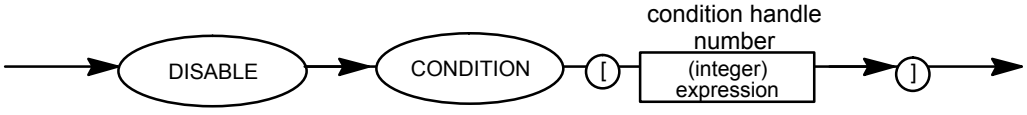
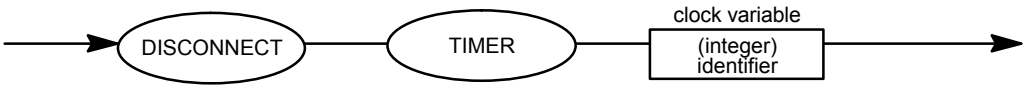
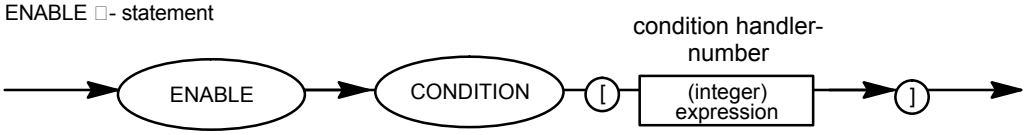


Figure E–15.

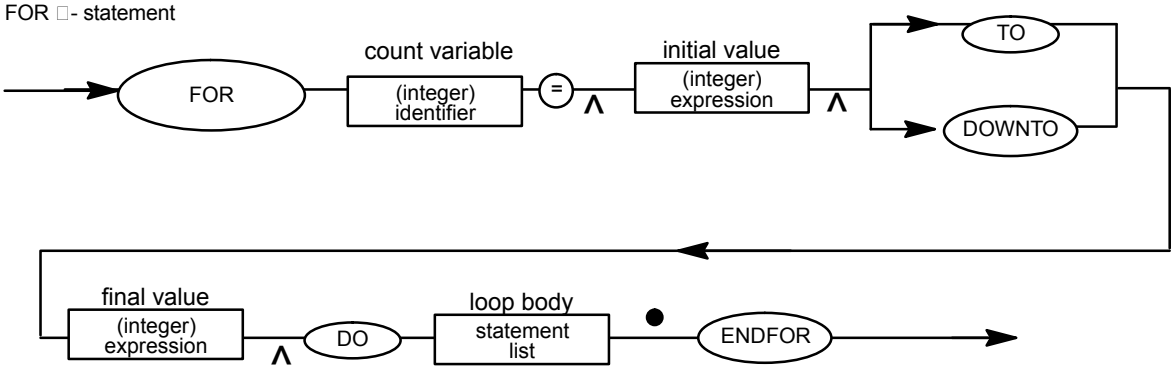
DISCONNECT TIMER □- statement



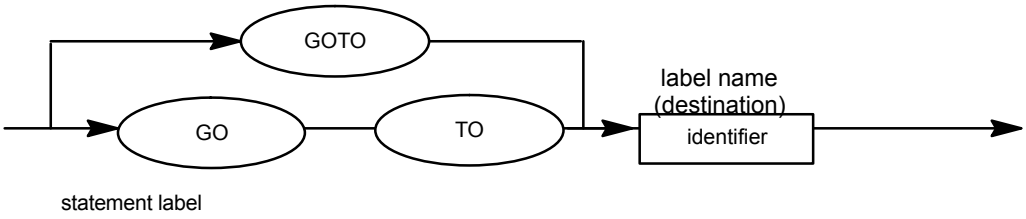
ENABLE □- statement



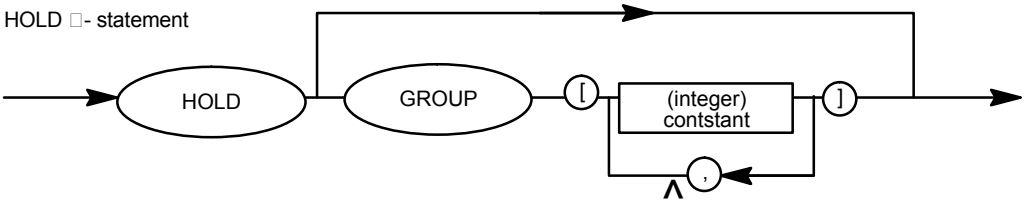
FOR □- statement



GO TO □- statement



HOLD □- statement



IF THEN □- statement

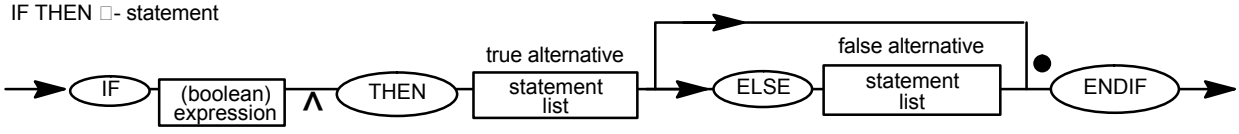


Figure E–16.

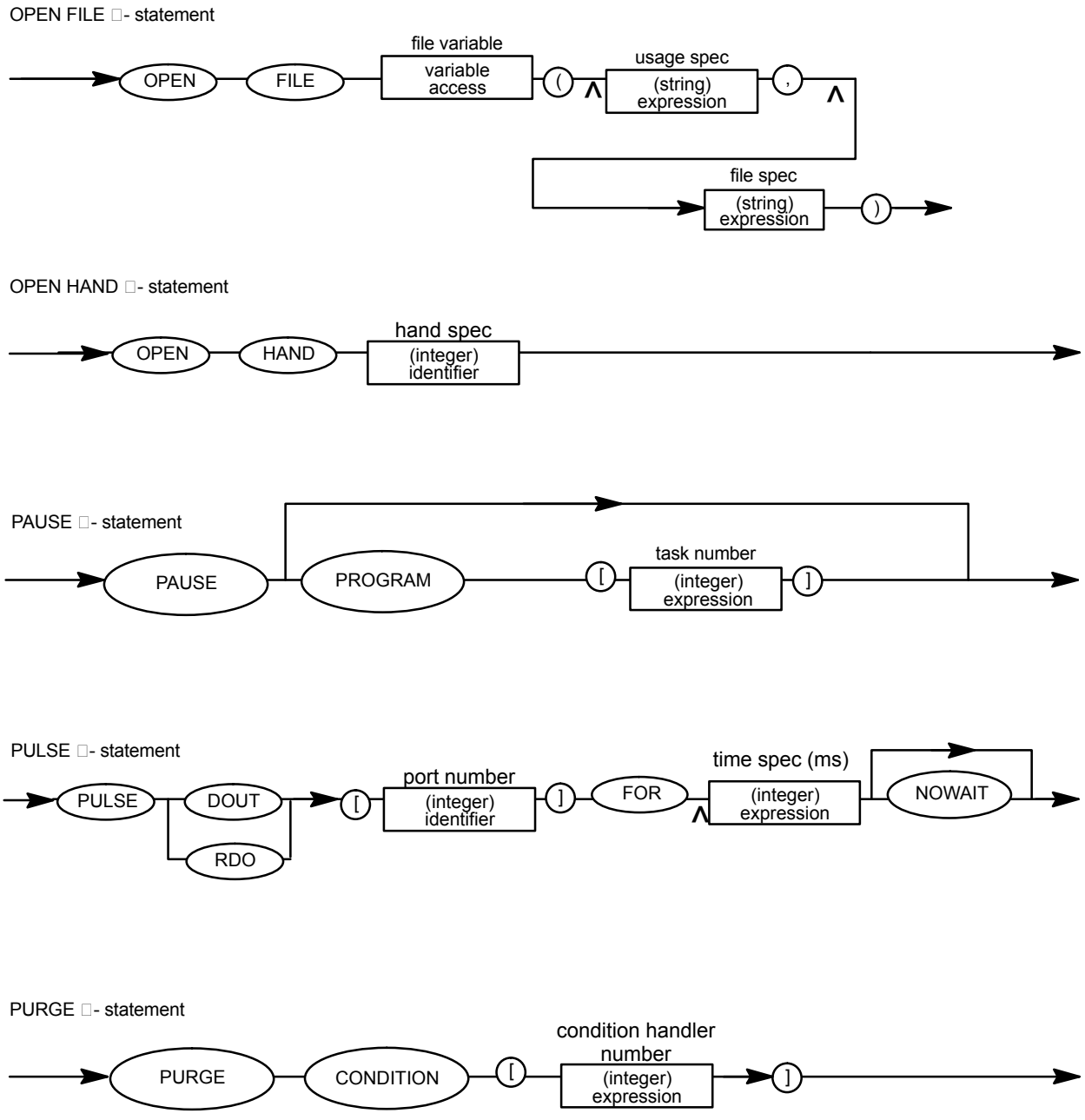
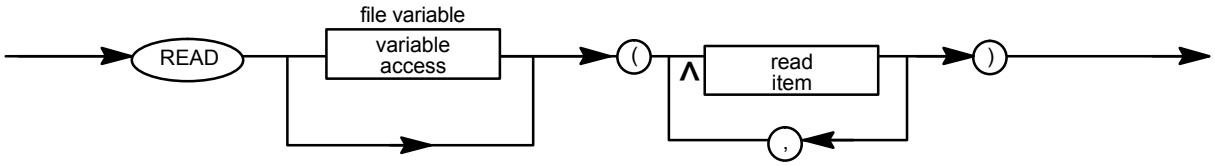
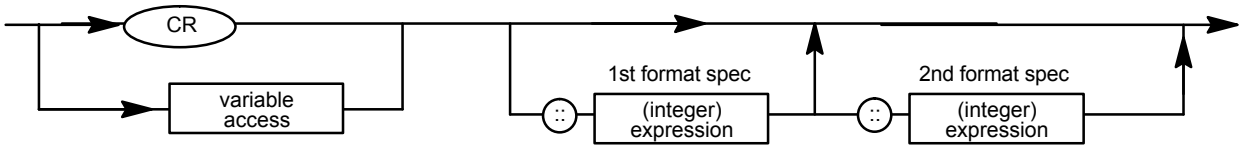

OPEN FILE □- statement
file variable
variable access
usage spec
(string) expression
OPEN
FILE
(
,
file spec
(string) expression
)
OPEN HAND □- statement
hand spec
(integer) identifier
OPEN
HAND
PAUSE □- statement
task number
(integer) expression
PAUSE
PROGRAM
[
]
PULSE □- statement
port number
(integer) identifier
time spec (ms)
(integer) expression
PULSE
DOUT
RDO
FOR
NOWAIT
PURGE □- statement
condition handler number
(integer) expression
PURGE
CONDITION

Figure E–17.

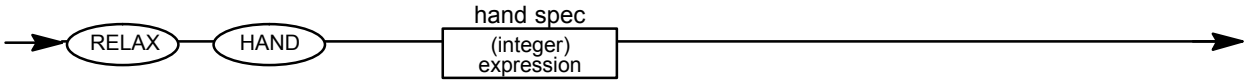
READ □- statement



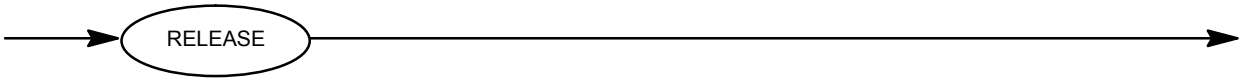
read item □- statement



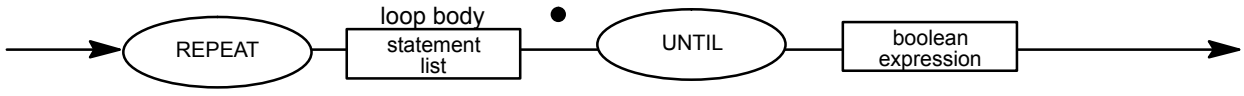
RELAX □- statement



RELEASE □- statement



REPEAT □- statement



RESUME □- statement

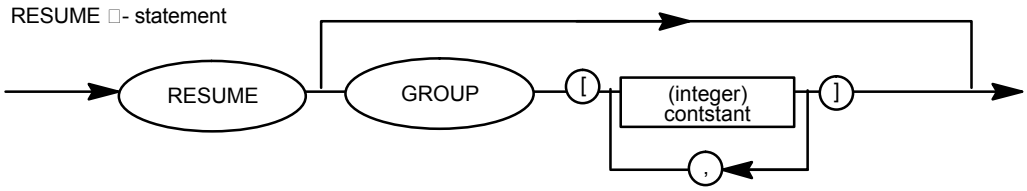


Figure E–18.

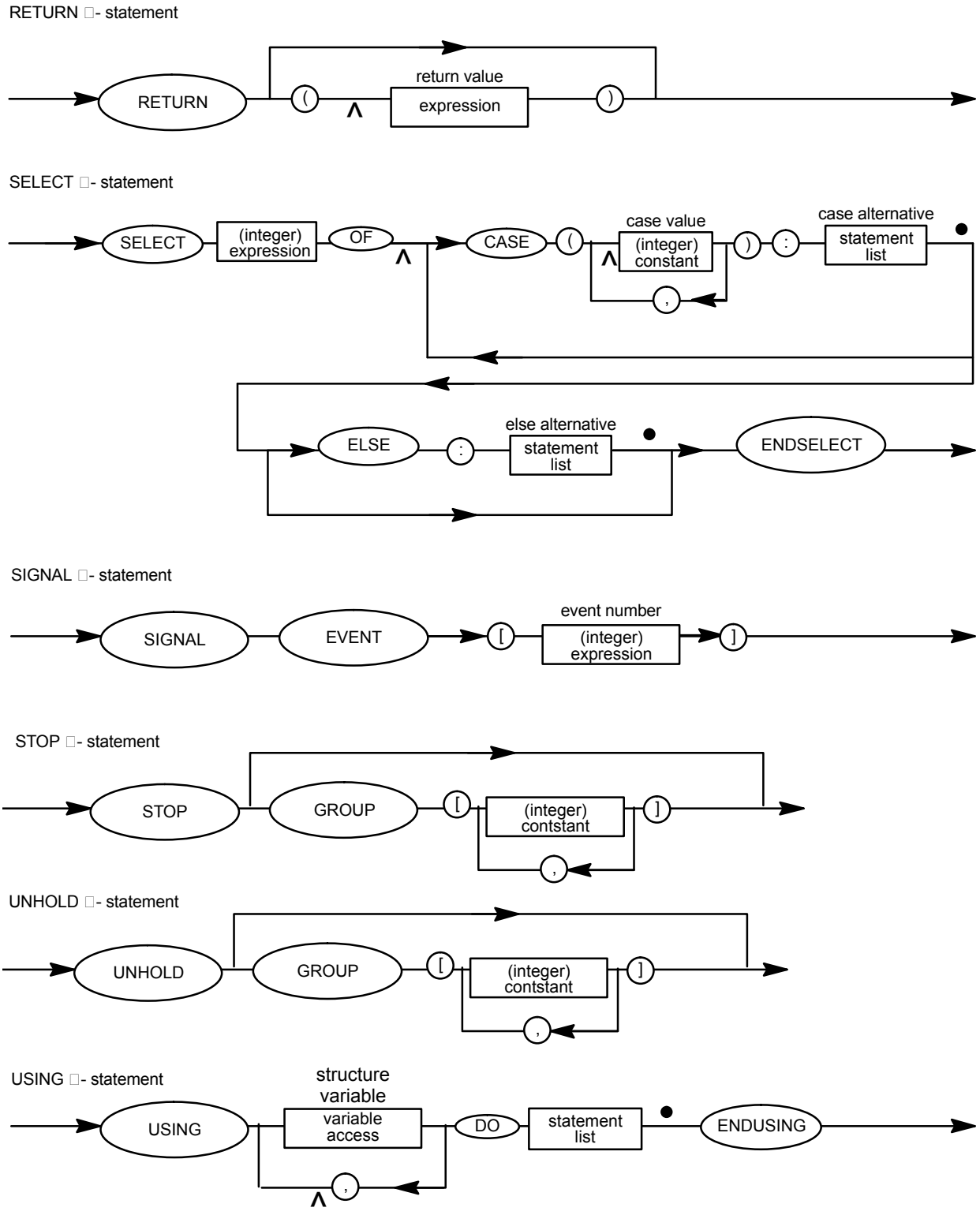
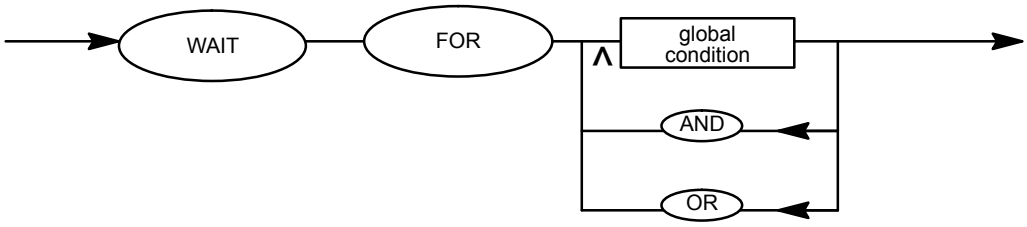
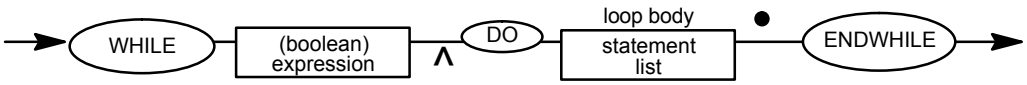

RETURN □- statement
RETURN
(
return value
expression
)
SELECT □- statement
SELECT
(integer) expression
OF
CASE
(
case value
(integer) constant
,
)
:
case alternative
statement list
ELSE
:
else alternative
statement list
ENDSELECT
SIGNAL □- statement
SIGNAL
EVENT
[
event number
(integer) expression
]
STOP □- statement
STOP
GROUP
[
(integer) contstant
,
]
UNHOLD □- statement
UNHOLD
GROUP
[
(integer) contstant
,
]
USING □- statement
USING
structure variable
variable access
,
DO
statement list
ENDUSING

Figure E–19.

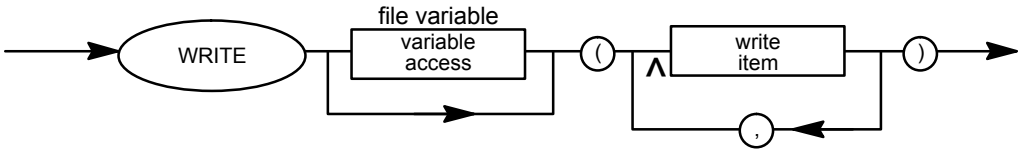
WAIT □- statement



WHILE □- statement



WRITE □- statement



write item

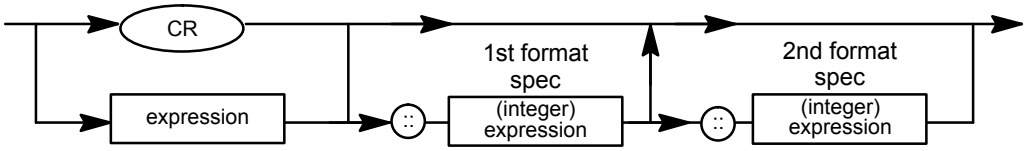


Figure E–20.

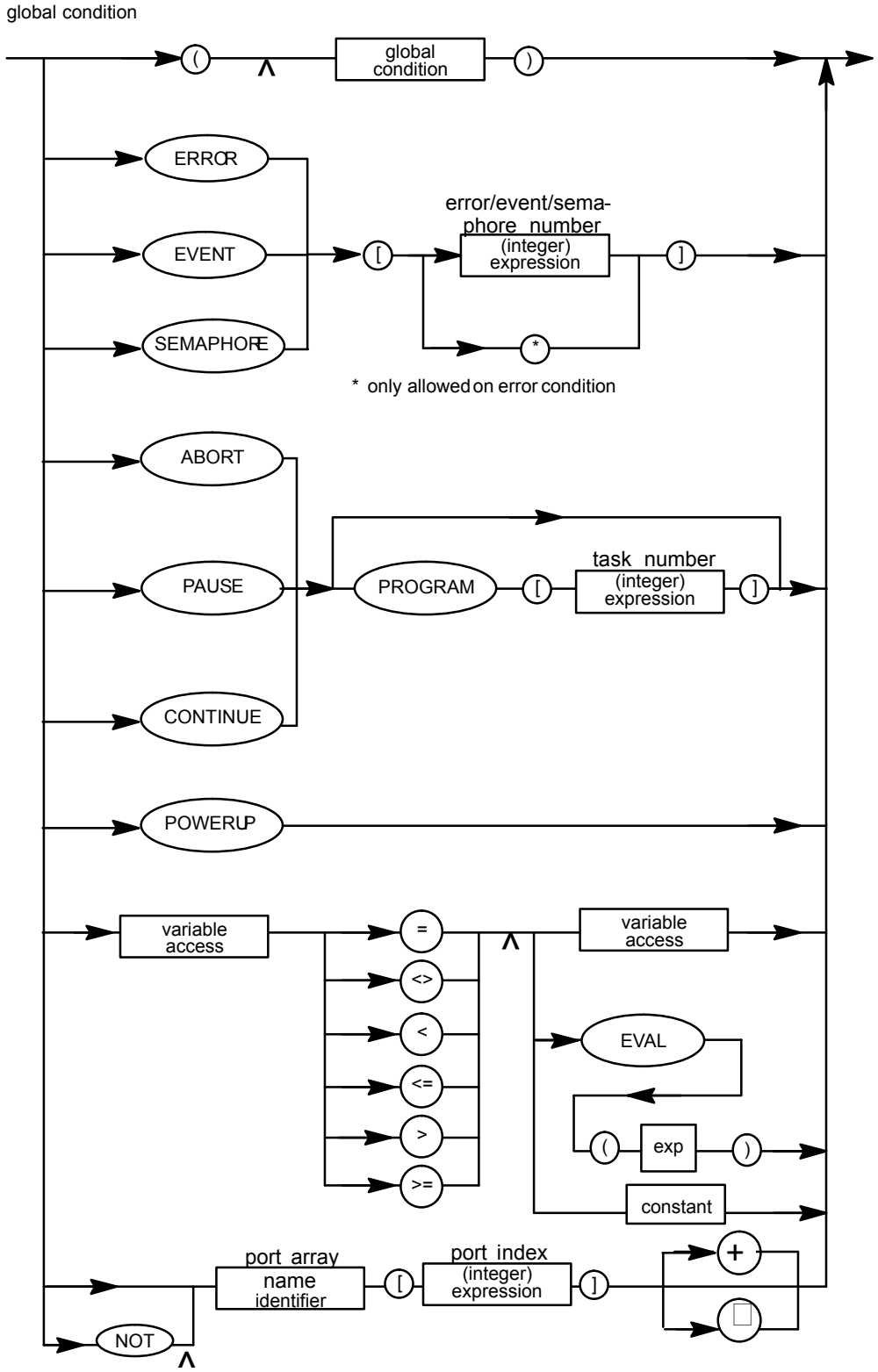

global condition
(
global condition
)
ERROR
EVENT
SEMAPHORE
error/event/sema-phore number
[
(integer) expression
]
*
* only allowed on error condition
ABORT
PAUSE
CONTINUE
PROGRAM
task number
[
(integer) expression
]
POWERUP
variable access
=
<>
<
<=
>
>=
variable access
EVAL
(
exp
)
constant
NOT
port array name identifier
[
port index (integer) expression
]
+
-

Figure E–21.

condition handler action

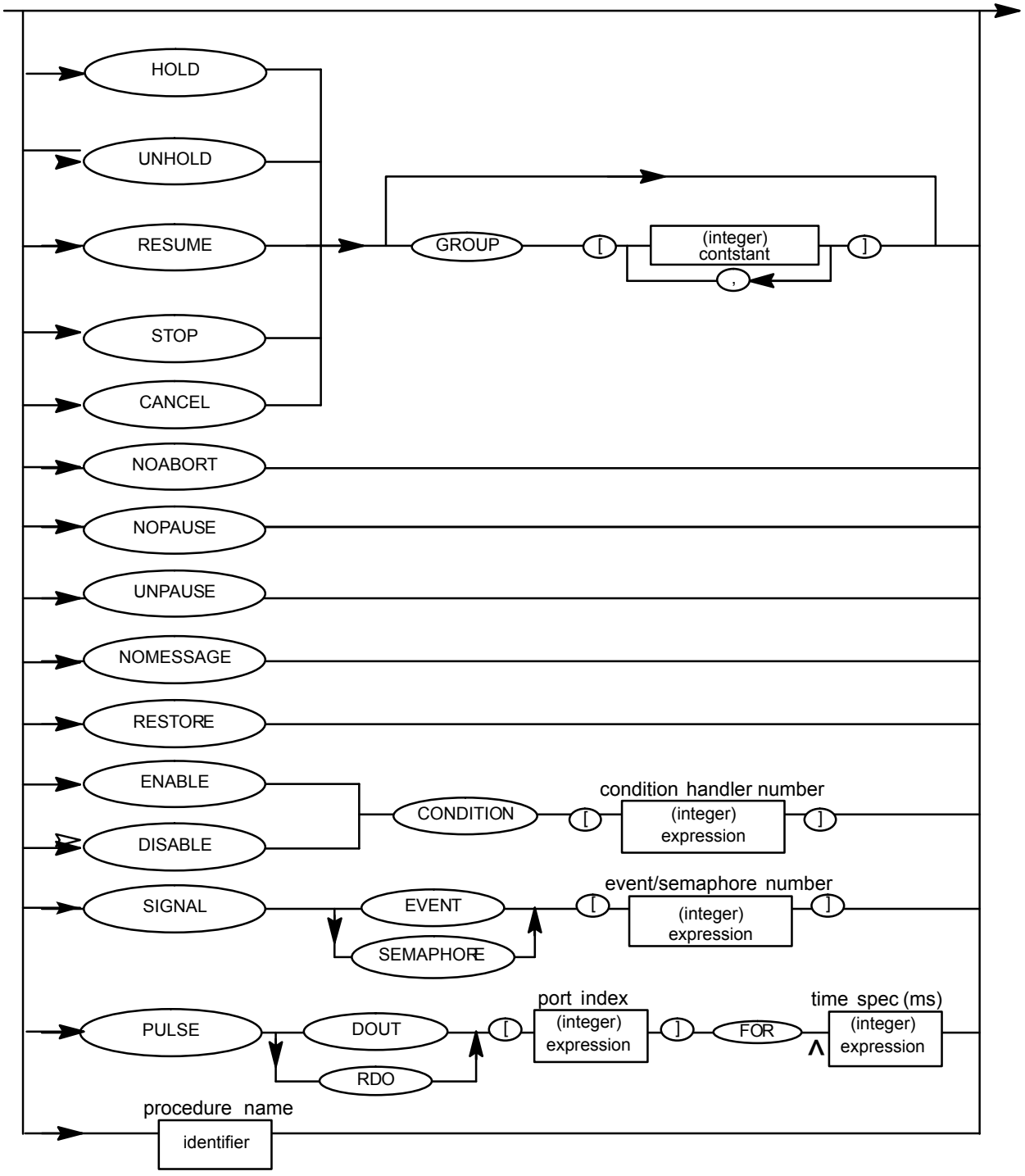


Figure E–22.

condition handler action continued

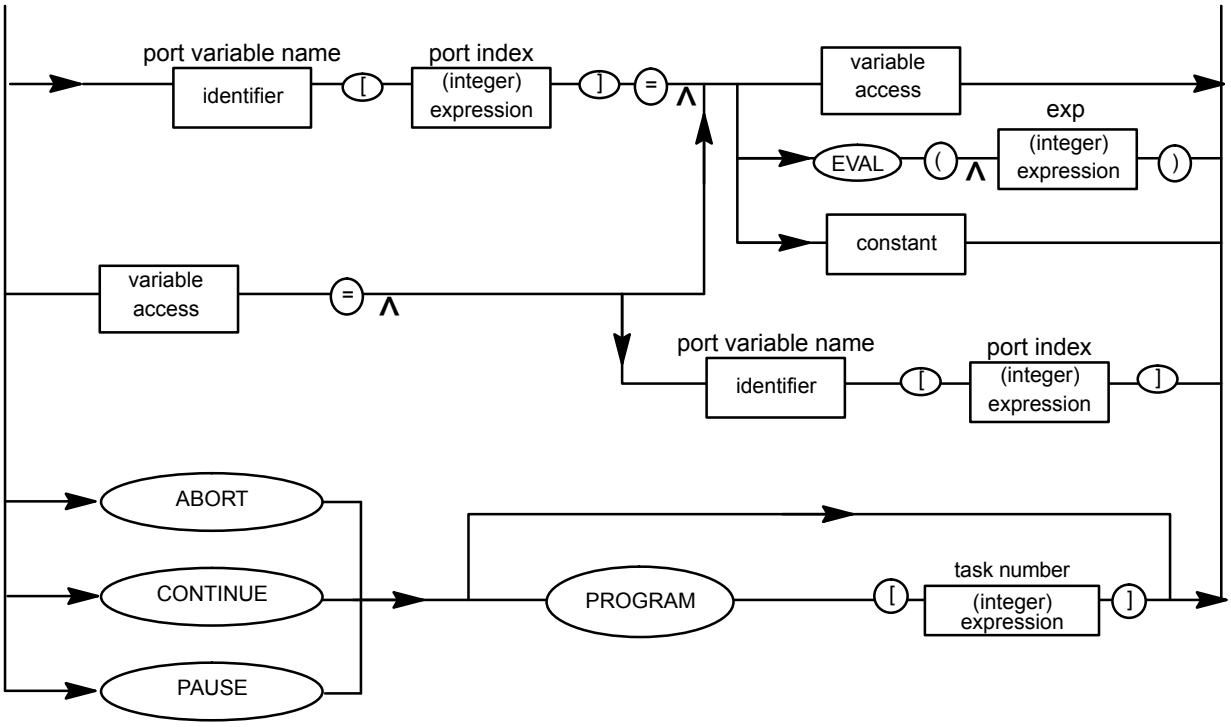
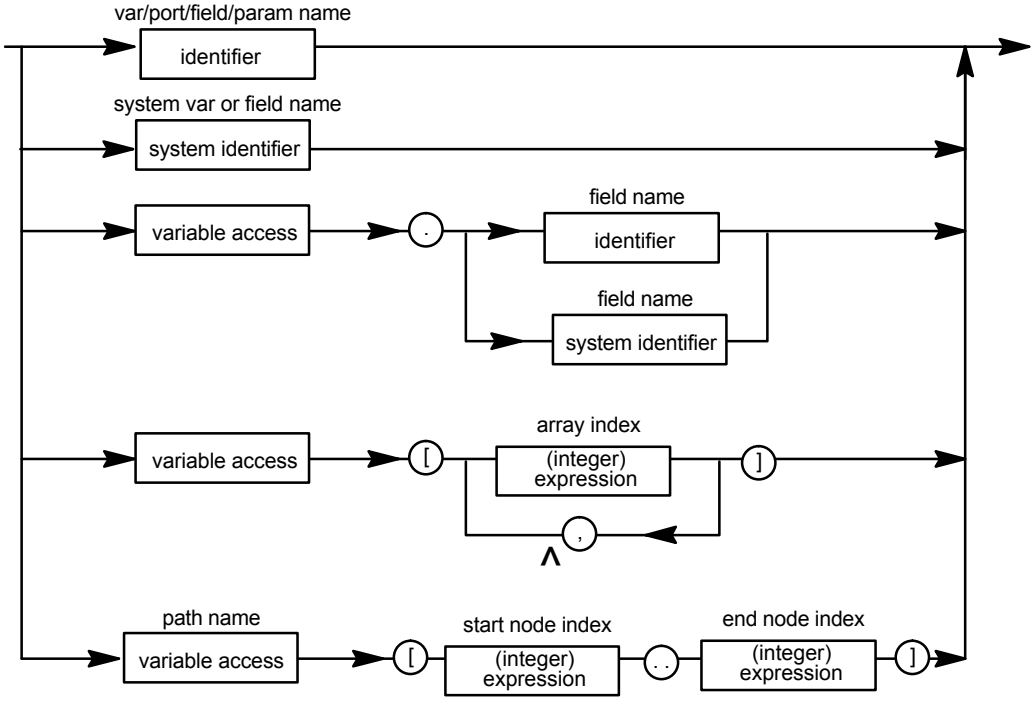


Figure E–23.

variable access



expression

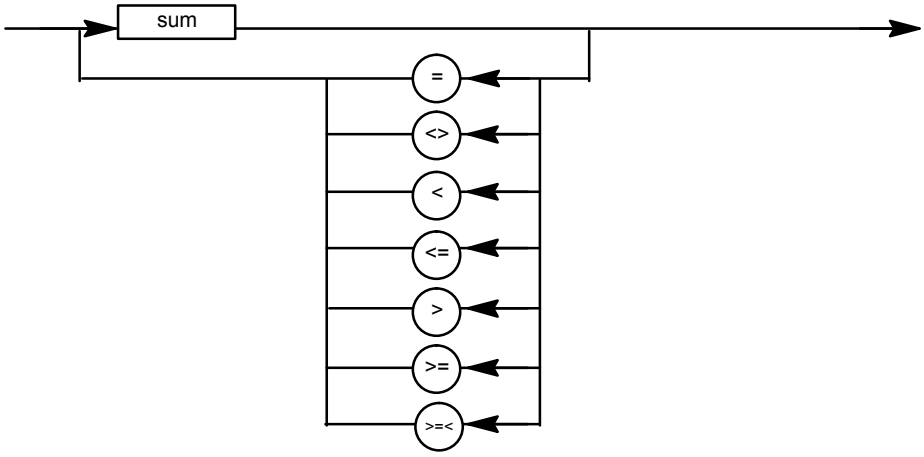
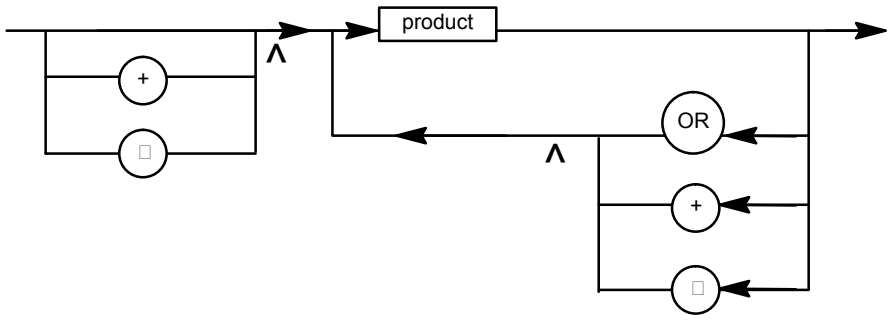
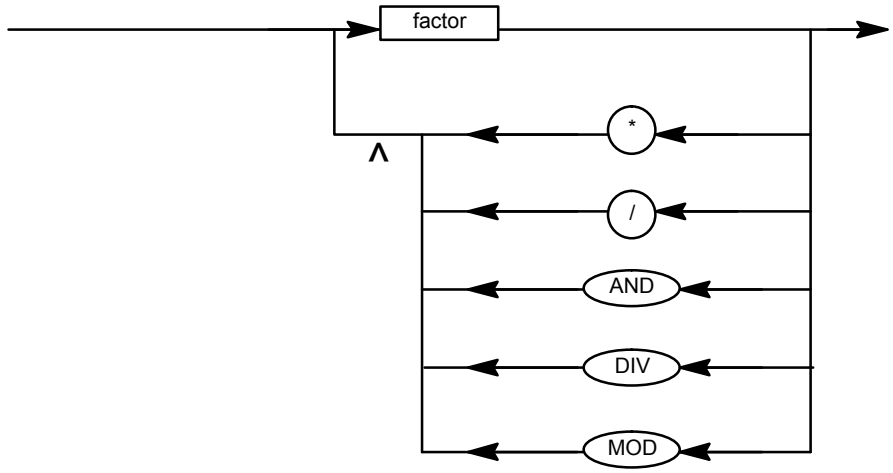


Figure E–24.

sum



product



factor

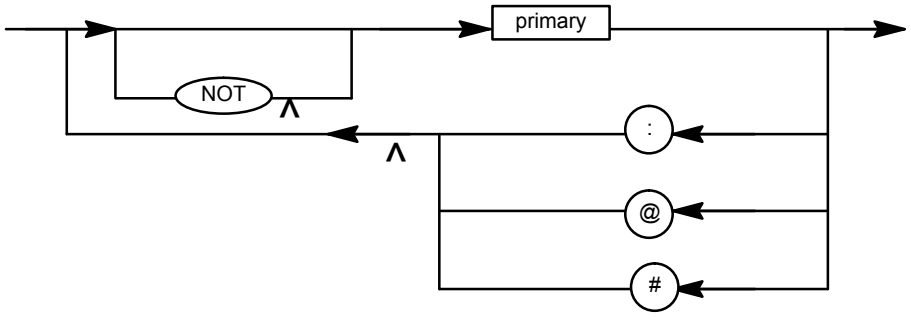
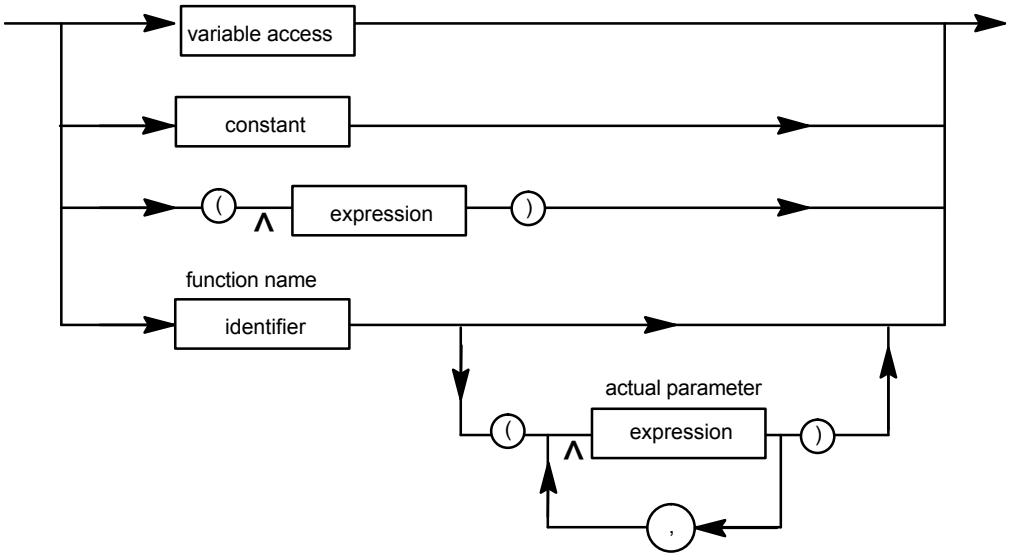
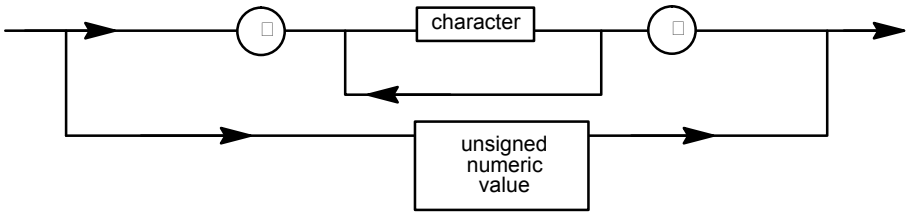


Figure E–25.

primary



literal



术语表

A

abort

Abnormal termination of a computer program caused by hardware or software malfunction or operator cancellation.

absolute pulse code system

A positional information system for servomotors that relies on battery-backed RAM to store encoder pulse counts when the robot is turned off. This system is calibrated when it is turned on.

A/D value

An analog to digital-value. Converts a multilevel analog electrical system pattern into a digital bit.

AI

Analog input.

AO

Analog output.

alarm

The difference in value between actual response and desired response in the performance of a controlled machine, system or process. Alarm=Error.

algorithm

A fixed step-by-step procedure for accomplishing a given result.

alphanumeric

Data that are both alphabetical and numeric.

AMPS

Amperage amount.

analog

The representation of numerical quantities by measurable quantities such as length, voltage or resistance. Also refers to analog type I/O blocks and distinguishes them from discrete I/O blocks. Numerical data that can vary continuously, for example, voltage levels that can vary within the range of -10 to +10 volts.

AND

An operation that places two contacts or groups of contacts in series. All contacts in series control the resulting status and also mathematical operator.

ANSI

American National Standard Institute, the U.S. government organization with responsibility for the development and announcement of technical data standards.

APC

See absolute pulse code system.

APC motor

See servomotor.

application program

The set of instructions that defines the specific intended tasks of robots and robot systems to make them reprogrammable and multifunctional. You can initiate and change these programs.

arm

A robot component consisting of an interconnecting set of links and powered joints that move and support the wrist socket and end effector.

articulated arm

A robot arm constructed to simulate the human arm, consisting of a series of rotary motions and joints, each powered by a motor.

ASCII

Abbreviation for American Standard Code for Information Interchange. An 8-level code (7 bits plus 1 parity bit) commonly used for the exchange of data.

automatic mode

The robot state in which automatic operation can be initiated.

automatic operation

The time during which robots are performing programmed tasks through unattended program execution.

axis

1. A straight line about which a robot joint rotates or moves. 2. One of the reference lines or a coordinate system. 3. A single joint on the robot arm.

B

backplane

A group of connectors mounted at the back of a controller rack to which printed circuit boards are mated.

BAR

A unit of pressure equal to 100,000 pascals.

barrier

A means of physically separating persons from the restricted work envelope; any physical boundary to a hazard or electrical device/component.

battery low alarm

A programmable value (in engineering units) against which the analog input signal automatically is compared on Genius I/O blocks. A fault is indicated if the input value is equal to or less than the low alarm value.

baud

A unit of transmission speed equal to the number of code elements (bits) per second.

big-endian

The adjectives big-endian and little-endian refer to which bytes are most significant in multi-byte data types and describe the order in which a sequence of bytes is stored in a computer's memory. In a big-endian system, the most significant value in the sequence is stored at the lowest storage address (i.e., first). In a little-endian system, the least significant value in the sequence is stored first.

binary

A numbering system that uses only 0 and 1.

bit

Contraction of binary digit. 1. The smallest unit of information in the binary numbering system, represented by a 0 or 1. 2. The smallest division of a programmable controller word.

bps

Bits per second.

buffer

A storage area in the computer where data is held temporarily until the computer can process it.

bus

A channel along which data can be sent.

bus controller

A Genius bus interface board for a programmable controller.

bus scan

One complete communications cycle on the serial bus.

Bus Switching Module

A device that switches a block cluster to one bus or the other of a dual bus.

byte

A sequence of binary digits that can be used to store a value from 0 to 255 and usually operated upon as a unit. Consists of eight bits used to store two numeric or one alpha character.

C

calibration

The process whereby the joint angle of each axis is calculated from a known reference point.

Cartesian coordinate system

A coordinate system whose axes (x, y, and z) are three intersecting perpendicular straight lines. The origin is the intersection of the axes.

Cartesian coordinates

A set of three numbers that defines the location of a point within a rectilinear coordinate system and consisting of three perpendicular axes (x, y, z).

cathode ray tube

A device, like a television set, for displaying information.

central processing unit

The main computer component that is made up of a control section and an arithmetic-logic section. The other basic units of a computer system are input/output units and primary storage.

channel

The device along which data flow between the input/output units of a computer and primary storage.

character

One of a set of elements that can be arranged in ordered groups to express information. Each character has two forms: 1. a man-intelligible form, the graphic, including the decimal digits 0-9, the letters A-Z, punctuation marks, and other formatting and control symbols; 2. a computer intelligible form, the code, consisting of a group of binary digits (bits).

circular

A MOTYPE option in which the robot tool center point moves in an arc defined by three points. These points can be positions or path nodes.

clear

To replace information in a storage unit by zero (or blank, in some machines).

closed loop

A control system that uses feedback. An open loop control system does not use feedback.

C-MOS RAM

Complementary metal-oxide semiconductor random-access memory. A read/write memory in which the basic memory cell is a pair of MOS (metal-oxide semiconductor) transistors. It is an implementation of S-RAM that has very low power consumption, but might be less dense than other S-RAM implementations.

coaxial cable

A transmission line in which one conductor is centered inside and insulated from an outer metal tube that serves as the second conductor. Also known as coax, coaxial line, coaxial transmission line, concentric cable, concentric line, concentric transmission line.

component

An inclusive term used to identify a raw material, ingredient, part or subassembly that goes into a higher level of assembly, compound or other item.

computer

A device capable of accepting information, applying prescribed processes to the information, and supplying the results of these processes.

configuration

The joint positions of a robot and turn number of wrist that describe the robot at a specified position. Configuration is designated by a STRING value and is included in positional data.

continuous path

A trajectory control system that enables the robot arm to move at a constant tip velocity through a series of predefined locations. A rounding effect of the path is required as the tip tries to pass through these locations.

continuous process control

The use of transducers (sensors) to monitor a process and make automatic changes in operations through the design of appropriate feedback control loops. While such devices historically have been mechanical or electromechanical, microcomputers and centralized control is now used, as well.

continuous production

A production system in which the productive equipment is organized and sequenced according to the steps involved to produce the product. Denotes that material flow is continuous during the production process. The routing of the jobs is fixed and set-ups are seldom changed.

controlled stop

A controlled stop controls robot deceleration until it stops. When a safety stop input such as a safety fence signal is opened, the robot decelerates in a controlled manner and then stops. After the robot stops, the Motor Control Contactor opens and drive power is removed.

controller

A hardware unit that contains the power supply, operator controls, control circuitry, and memory that directs the operation and motion of the robot and communications with external devices. See control unit.

controller memory

A medium in which data are retained. Primary storage refers to the internal area where the data and program instructions are stored for active use, as opposed to auxiliary or external storage (magnetic tape, disk, diskette, and so forth.)

control, open-loop

An operation where the computer applies control directly to the process without manual intervention.

control unit

The portion of a computer that directs the automatic operation of the computer, interprets computer instructions, and initiates the proper signals to the other computer circuits to execute instructions.

coordinate system

See Cartesian coordinate system.

CPU

See central processing unit.

CRT

See cathode ray tube.

cps (viscosity)

Centipoises per second.

CRT/KB

Cathode ray tube/keyboard. An optional interface device for the robot system. The CRT/KB is used for some robot operations and for entering programs. It can be a remote device that attaches to the robot via a cable.

cycle

1. A sequence of operations that is repeated regularly. The time it takes for one such sequence to occur. 2. The interval of time during which a system or process, such as seasonal demand or a manufacturing operation, periodically returns to similar initial conditions. 3. The interval of time during which an event or set of events is completed. In production control, a cycle is the length of time between the release of a manufacturing order and shipment to the customer or inventory.

cycle time

1. In industrial engineering, the time between completion of two discrete units of production. 2. In materials management, the length of time from when material enters a production facility until it exits. See throughput.

cursor

An indicator on a teach pendant or CRT display screen at which command entry or editing occurs. The indicator can be a highlighted field or an arrow (> or ^).

cylindrical

Type of work envelope that has two linear major axes and one rotational major axis. Robotic device that has a predominantly cylindrical work envelope due to its design. Typically has fewer than 6 joints and typically has only 1 linear axis.

D

D/A converter

A digital-to-analog converter. A device that transforms digital data into analog data.

D/A value

A digital-to-analog value. Converts a digital bit pattern into a multilevel analog electrical system.

daisy chain

A means of connecting devices (readers, printers, etc.) to a central processor by party-line input/output buses that join these devices by male and female connectors. The last female connector is shorted by a suitable line termination.

daisy chain configuration

A communications link formed by daisy chain connection of twisted pair wire.

data

A collection of facts, numeric and alphabetical characters, or any representation of information that is suitable for communication and processing.

data base

A data file philosophy designed to establish the independence of computer program from data files. Redundancy is minimized and data elements can be added to, or deleted from, the file designs without changing the existing computer programs.

DC

Abbreviation for direct current.

DEADMAN switch

A control switch on the teach pendant that is used to enable servo power. Pressing the DEADMAN switch while the teach pendant is on activates servo power and releases the robot brakes; releasing the switch deactivates servo power and applies the robot brakes.

debugging

The process of detecting, locating and removing mistakes from a computer program, or manufacturing control system. See diagnostic routine.

deceleration tolerance

The specification of the percentage of deceleration that must be completed before a motion is considered finished and another motion can begin.

default

The value, display, function or program automatically selected if you have not specified a choice.

deviation

Usually, the absolute difference between a number and the mean of a set of numbers, or between a forecast value and the actual data.

device

Any type of control hardware, such as an emergency-stop button, selector switch, control pendant, relay, solenoid valve, or sensor.

diagnostic routine

A test program used to detect and identify hardware/software malfunctions in the controller and its associated I/O equipment. See debugging.

diagnostics

Information that permits the identification and evaluation of robot and peripheral device conditions.

digital

A description of any data that is expressed in numerical format. Also, having the states On and Off only.

digital control

The use of a digital computer to perform processing and control tasks in a manner that is more accurate and less expensive than an analog control system.

digital signal

A single point control signal sent to or from the controller. The signal represents one of two states: ON (TRUE, 1. or OFF (FALSE, 0).

directory

A listing of the files stored on a device.

discrete

Consisting of individual, distinct entities such as bits, characters, circuits, or circuit components. Also refers to ON/OFF type I/O blocks.

disk

A secondary memory device in which information is stored on a magnetically sensitive, rotating disk.

disk memory

A non-programmable, bulk-storage, random-access memory consisting of a magnetized coating on one or both sides of a rotating thin circular plate.

drive power

The energy source or sources for the robot servomotors that produce motion.

DRAM

Dynamic Random Access Memory. A read/write memory in which the basic memory cell is a capacitor. DRAM (or D-RAM) tends to have a higher density than SRAM (or S-RAM). Due to the support circuitry required, and power consumption needs, it is generally impractical to use. A battery can be used to retain the content upon loss of power.

E

edit

1. A software mode that allows creation or alteration of a program. 2. To modify the form or format of data, for example, to insert or delete characters.

emergency stop

The operation of a circuit using hardware-based components that overrides all other robot controls, removes drive power from the actuators, and causes all moving parts of to stop. The operator panel and teach pendant are each equipped with EMERGENCY STOP buttons.

enabling device

A manually operated device that, when continuously activated, permits motion. Releasing the device stops the motion of the robot and associated equipment that might present a hazard.

encoder

1. A device within the robot that sends the controller information about where the robot is. 2. A transducer used to convert position data into electrical signals. The robot system uses an incremental optical encoder to provide position feedback for each joint. Velocity data is computed from the encoder signals and used as an additional feedback signal to assure servo stability.

end effector

An accessory device or tool specifically designed for attachment to the robot wrist or tool mounting plate to enable the robot to perform its intended tasks. Examples include gripper, spot weld gun, arc weld gun, spray paint gun, etc.

end-of-arm tooling

Any of a number of tools, such as welding guns, torches, bells, paint spraying devices, attached to the faceplate of the robot wrist. Also called end effector or EOAT.

engineering units

Units of measure as applied to a process variable, for example, psi, Degrees F., etc.

envelope, maximum

The volume of space encompassing the maximum designed movements of all robot parts including the end effector, workpiece, and attachments.

EOAT

See end of arm tooling, tool.

EPROM

Erasable Programmable Read Only Memory. Semiconductor memory that can be erased and reprogrammed. A non-volatile storage memory.

error

The difference in value between actual response and desired response in the performance of a controlled machine, system or process. Alarm=Error.

error message

A numbered message, displayed on the CRT/KB and teach pendant, that indicates a system problem or warns of a potential problem.

Ethernet

A Local Area Network (LAN) bus-oriented, hardware technology that is used to connect computers, printers, terminal concentrators (servers), and many other devices together. It consists of a master cable and connection devices at each machine on the cable that allow the various devices to "talk" to each other. Software that can access the Ethernet and cooperate with machines connected to the cable is necessary. Ethernets come in varieties such as baseband and broadband and can run on different media, such as coax, twisted pair and fiber. Ethernet is a trademark of Xerox Corporation.

execute

To perform a specific operation, such as one that would be accomplished through processing one statement or command, a series of statements or commands, or a complete program or command procedure.

extended axis

An optional, servo-controlled axis that provides extended reach capability for a robot, including in-booth rail, single- or double-link arm, also used to control motion of positioning devices.

F

faceplate

The tool mounting plate of the robot.

feedback

1. The signal or data fed back to a commanding unit from a controlled machine or process to denote its response to the command signal. The signal representing the difference between actual response and desired response that is used by the commanding unit to improve performance of the controlled machine or process. 2. The flow of information back into the control system so that actual performance can be compared with planned performance, for instance in a servo system.

field

A specified area of a record used for a particular category of data. 2. A group of related items that occupy the same space on a CRT/KB screen or teach pendant LCD screen. Field name is the name of the field; field items are the members of the group.

field devices

User-supplied devices that provide information to the PLC (inputs: push buttons, limit switches, relay contacts, an so forth) or perform PLC tasks (outputs: motor starters, solenoids, indicator lights, and so forth.)

file

1. An organized collection of records that can be stored or retrieved by name. 2. The storage device on which these records are kept, such as bubble memory or disk.

filter

A device to suppress interference that would appear as noise.

Flash File Storage

A portion of FROM memory that functions as a separate storage device. Any file can be stored on the FROM disk.

Flash ROM

Flash Read Only Memory. Flash ROM is not battery-backed memory but it is non-volatile. All data in Flash ROM is saved even after you turn off and turn on the robot.

flow chart

A systems analysis tool to graphically show a procedure in which symbols are used to represent operations, data, flow, and equipment. See block diagram, process chart.

flow control

A specific production control system that is based primarily on setting production rates and feeding work into production to meet the planned rates, then following it through production to make sure that it is moving. This concept is most successful in repetitive production.

format

To set up or prepare a memory card or floppy disk (not supported with version 7.20 and later) so it can be used to store data in a specific system.

FR

See Flash ROM.

F-ROM

See Flash ROM.

FROM disk

See Flash ROM.

G

general override stat

A percentage value that governs the maximum robot jog speed and program run speed.

Genius I/O bus

The serial bus that provides communications between blocks, controllers, and other devices in the system especially with respect to GE FANUC Genius I/O.

gripper

The "hand" of a robot that picks up, holds and releases the part or object being handled. Sometimes referred to as a manipulator. See EOAT, tool.

group signal

An input/output signal that has a variable number of digital signals, recognized and taken as a group.

gun

See applicator.

H

Hand Model.

Used in Interference Checking, the Hand Model is the set of virtual model elements (spheres and cylinders) that are used to represent the location and shape of the end of arm tooling with respect to the robot's faceplate.

hardware

1. In data processing, the mechanical, magnetic, electrical and electronic devices of which a computer, controller, robot, or panel is built. 2. In manufacturing, relatively standard items such as nuts, bolts, washers, clips, and so forth.

hard-wire

To connect electric components with solid metallic wires.

hard-wired

1. Having a fixed wired program or control system built in by the manufacturer and not subject to change by programming. 2. Interconnection of electrical and electronic devices directly through physical wiring.

hazardous motion

Unintended or unexpected robot motion that can cause injury.

hexadecimal

A numbering system having 16 as the base and represented by the digits 0 through 9, and A through F.

hold

A smoothly decelerated stopping of all robot movement and a pause of program execution. Power is maintained on the robot and program execution generally can be resumed from a hold.

HTML.

Hypertext Markup Language. A markup language that is used to create hypertext and hypermedia documents incorporating text, graphics, sound, video, and hyperlinks.

http.

Hypertext transfer protocol. The protocol used to transfer HTML files between web servers.

I

impedance

A measure of the total opposition to current flow in an electrical circuit.

incremental encoder system

A positional information system for servomotors that requires calibrating the robot by moving it to a known reference position (indicated by limit switches) each time the robot is turned on or calibration is lost due to an error condition.

index

An integer used to specify the location of information within a table or program.

index register

A memory device containing an index.

industrial robot

A reprogrammable multifunctional manipulator designed to move material, parts, tools, or specialized devices through variable programmed motions in order to perform a variety of tasks.

industrial robot system

A system that includes industrial robots, end effectors, any equipment devices and sensors required for the robot to perform its tasks, as well as communication interfaces for interlocking, sequencing, or monitoring the robot.

information

The meaning derived from data that have been arranged and displayed in a way that they relate to that which is already known. See data.

initialize

1. Setting all variable areas of a computer program or routine to their desired initial status, generally done the first time the code is executed during each run. 2. A program or hardware circuit that returns a program a system, or hardware device to an original state. See startup, initial.

input

The data supplied from an external device to a computer for processing. The device used to accomplish this transfer of data.

input device

A device such as a terminal keyboard that, through mechanical or electrical action, converts data from the form in which it has been received into electronic signals that can be interpreted by the CPU or programmable controller. Examples are limit switches, push buttons, pressure switches, digital encoders, and analog devices.

input processing time

The time required for input data to reach the microprocessor.

input/output

Information or signals transferred between devices, discreet electrical signals for external control.

input/output control

A technique for controlling capacity where the actual output from a work center is compared with the planned output developed by CRP. The input is also monitored to see if it corresponds with plans so that work centers will not be expected to generate output when jobs are not available to work on.

integrated circuit

A solid-state micro-circuit contained entirely within a chip of semiconductor material, generally silicon. Also called chip.

interactive

Refers to applications where you communicate with a computer program via a terminal by entering data and receiving responses from the computer.

interface

1. A concept that involves the specifications of the inter-connection between two equipments having different functions. 2. To connects a PLC with the application device, communications channel, and peripherals through various modules and cables. 3. The method or equipment used to communicate between devices.

interference zone

An area that falls within the work envelope of a robot, in which there is the potential for the robot motion to coincide with the motion of another robot or machine, and for a collision to occur.

interlock

An arrangement whereby the operation of one control or mechanism brings about, or prevents, the operations of another.

interrupt

A break in the normal flow of a system or program that occurs in a way that the flow can be resumed from that point at a later time. Interrupts are initiated by two types of signals: 1. signals originating within the computer system to synchronize the operation of the computer system with the outside

world; 2. signals originating exterior to the computer system to synchronize the operation of the computer system with the outside world.

I/O

Abbreviation for input/output or input/output control.

I/O block

A microprocessor-based, configurable, rugged solid state device to which field I/O devices are attached.

I/O electrical isolation

A method of separating field wiring from logic level circuitry. This is typically done through optical isolation devices.

I/O module

A printed circuit assembly that is the interface between user devices and the Series Six PLC.

I/O scan

A method by which the CPU monitors all inputs and controls all outputs within a prescribed time. A period during which each device on the bus is given a turn to send information and listen to all of the broadcast data on the bus.

ISO

The International Standards Organization that establishes the ISO interface standards.

isolation

1. The ability of a logic circuit having more than one inputs to ensure that each input signal is not affected by any of the others. 2. A method of separating field wiring circuitry from logic level circuitry, typically done optically.

item

1. A category displayed on the teach pendant on a menu. 2. A set of adjacent digits, bits, or characters that is treated as a unit and conveys a single unit of information. 3. Any unique manufactured or purchased part or assembly: end product, assembly, subassembly, component, or raw material.

J

jog coordinate systems

Coordinate systems that help you to move the robot more effectively for a specific application. These systems include JOINT, WORLD, TOOL, and USER.

JOG FRAME

A jog coordinate system you define to make the robot jog the best way possible for a specific application. This can be different from world coordinate frame.

jogging

Pressing special keys on the teach pendant to move the robot.

jog speed

Is a percentage of the maximum speed at which you can jog the robot.

joint

1. A single axis of rotation. There are up to six joints in a robot arm (P-155 swing arm has 8). 2. A jog coordinate system in which one axis is moved at a time.

JOINT

A motion type in which the robot moves the appropriate combination of axes independently to reach a point most efficiently. (Point to point, non-linear motion).

joint interpolated motion

A method of coordinating the movement of the joints so all joints arrive at the desired location at the same time. This method of servo control produces a predictable path regardless of speed and results in the fastest cycle time for a particular move. Also called joint motion.

K

K

Abbreviation for kilo, or exactly 1024 in computer jargon. Related to 1024 words of memory.

KAREL

The programming language developed for robots by the FANUC Robotics America, Inc.

L

label

An ordered set of characters used to symbolically identify an instruction, a program, a quantity, or a data area.

LCD

See liquid crystal display.

lead time

The span of time needed to perform an activity. In the production and inventory control context, this activity is normally the procurement of materials and/or products either from an outside supplier or from one's own manufacturing facility. Components of lead time can include order preparation time, queue time, move or transportation time, receiving and inspection time.

LED

See Light Emitting Diode.

LED display

An alphanumeric display that consists of an array of LEDs.

Light Emitting Diode

A solid-state device that lights to indicate a signal on electronic equipment.

limiting device

A device that restricts the work envelope by stopping or causing to stop all robot motion and that is independent of the control program and the application programs.

limit switch

A switch that is actuated by some part or motion of a machine or equipment to alter the electrical circuit associated with it. It can be used for position detection.

linear

A motion type in which the appropriate combination of axes move in order to move the robot TCP in a straight line while maintaining tool center point orientation.

liquid crystal display

A digital display on the teach pendant that consists of two sheets of glass separated by a sealed-in, normally transparent, liquid crystal material. Abbreviated LCD.

little-endian

The adjectives big-endian and little-endian refer to which bytes are most significant in multi-byte data types and describe the order in which a sequence of bytes is stored in a computer's memory. In a big-endian system, the most significant value in the sequence is stored at the lowest storage address (i.e., first). In a little-endian system, the least significant value in the sequence is stored first.

load

1. The weight (force) applied to the end of the robot arm. 2. A device intentionally placed in a circuit or connected to a machine or apparatus to absorb power and convert it into the desired useful form. 3. To copy programs or data into memory storage.

location

1. A storage position in memory uniquely specified by an address. 2. The coordinates of an object used in describing its x, y, and z position in a Cartesian coordinate system.

lockout/tagout

The placement of a lock and/or tag on the energy isolating device (power disconnecting device) in the off or open position. This indicates that the energy isolating device or the equipment being controlled will not be operated until the lock/tag is removed.

log

A record of values and/or action for a given function.

logic

A fixed set of responses (outputs) to various external conditions (inputs). Also referred to as the program.

loop

The repeated execution of a series of instructions for a fixed number of times, or until interrupted by the operator.

M

mA

See milliampere.

machine language

A language written in a series of bits that are understandable by, and therefore instruct, a computer. This is a "first level" computer language, as compared to a "second level" assembly language, or a "third level" compiler language.

machine lock

A test run option that allows the operator to run a program without having the robot move.

macro

A source language instruction from which many machine-language instructions can be generated.

magnetic disk

A metal or plastic floppy disk (not supported on version 7.10 and later) that looks like a phonograph record whose surface can store data in the form of magnetized spots.

magnetic disk storage

A storage device or system consisting of magnetically coated metal disks.

magnetic tape

Plastic tape, like that used in tape recorder, on which data is stored in the form of magnetized spots.

maintenance

Keeping the robots and system in their proper operating condition.

MC

See memory card.

mechanical unit

The robot arm, including auxiliary axis, and hood/deck and door openers.

medium

plural **media** . The physical substance upon which data is recorded, such as a memory card (or floppy disk which is not supported on version 7.10 and later).

memory

A device or media used to store information in a form that can be retrieved and is understood by the computer or controller hardware. Memory on the controller includes C-MOS RAM, Flash ROM and D-RAM.

memory card

A C-MOS RAM memory card or a flash disk-based PC card.

menu

A list of options displayed on the teach pendant screen.

message

A group of words, variable in length, transporting an item of information.

microprocessor

A single integrated circuit that contains the arithmetic, logic, register, control and memory elements of a computer.

microsecond

One millionth (0.000001) of a second

milliampere

One one-thousandth of an ampere. Abbreviated mA.

millisecond

One thousandth of a second. Abbreviated msec.

module

A distinct and identifiable unit of computer program for such purposes as compiling, loading, and linkage editing. It is eventually combined with other units to form a complete program.

motion type

A feature that allows you to select how you want the robot to move from one point to the next. MOTYPES include joint, linear, and circular.

mode

1. One of several alternative conditions or methods of operation of a device. 2. The most common or frequent value in a group of values.

N

network

1. The interconnection of a number of devices by data communication facilities. "Local networking" is the communications network internal to a robot."Global networking" is the ability to provide communications connections outside of the robot's internal system. 2. Connection of geographically separated computers and/or terminals over communications lines. The control of transmission is managed by a standard protocol.

non-volatile memory

Memory capable of retaining its stored information when power is turned off.

O

Obstacle Model.
Used in Interference Checking, the Obstacle Model is the set of virtual model elements (spheres, cylinders, and planes) that are used to represent the shape and the location of a given obstacle in space.

off-line
Equipment or devices that are not directly connected to a communications line.

off-line operations
Data processing operations that are handled outside of the regular computer program. For example, the computer might generate a report off-line while the computer was doing another job.

off-line programming
The development of programs on a computer system that is independent of the "on-board" control of the robot. The resulting programs can be copied into the robot controller memory.

offset
The count value output from a A/D converter resulting from a zero input analog voltage. Used to correct subsequent non-zero measurements also incremental position or frame adjustment value.

on-line
A term to describe equipment or devices that are connected to the communications line.

on-line processing
A data processing approach where transactions are entered into the computer directly, as they occur.

operating system
Lowest level system monitor program.

operating work envelope
The portion of the restricted work envelope that is actually used by the robot while it is performing its programmed motion. This includes the maximum the end-effector, the workpiece, and the robot itself.

operator
A person designated to start, monitor, and stop the intended productive operation of a robot or robot system.

operator box
A control panel that is separate from the robot and is designed as part of the robot system. It consists of the buttons, switches, and indicator lights needed to operate the system.

operator panel
A control panel designed as part of the robot system and consisting of the buttons, switches, and indicator lights needed to operate the system.

optional features
Additional capabilities available at a cost above the base price.

OR

An operation that places two contacts or groups of contacts in parallel. Any of the contacts can control the resultant status, also a mathematical operation.

orientation

The attitude of an object in space. Commonly described by three angles: rotation about x (w), rotation about y (p), and rotation about z (r).

origin

The point in a Cartesian coordinate system where axes intersect; the reference point that defines the location of a frame.

OT

See overtravel.

output

Information that is transferred from the CPU for control of external devices or processes.

output device

A device, such as starter motors, solenoids, that receive data from the programmable controller.

output module

An I/O module that converts logic levels within the CPU to a usable output signal for controlling a machine or process .

outputs

Signals, typically on or off, that controls external devices based upon commands from the CPU.

override

See general override.

overtravel

A condition that occurs when the motion of a robot axis exceeds its prescribed limits.

overwrite

To replace the contents of one file with the contents of another file when copying.

P

parity

The anticipated state, odd or even, of a set of binary digits.

parity bit

A binary digit added to an array of bits to make the sum of all bits always odd or always even.

parity check

A check that tests whether the number of ones (or zeros) in an array of binary digits is odd or even.

parity error

A condition that occurs when a computed parity check does not agree with the parity bit.

part

A material item that is used as a component and is not an assembly or subassembly.

pascal

A unit of pressure in the meter-kilogram-second system equivalent to one newton per square meter.

path

1. A variable type available in the KAREL system that consists of a list of positions. Each node includes positional information and associated data. 2. The trajectory followed by the TCP in a move.

PCB

See printed circuit board.

PC Interface

The PC Interface software uses Ethernet connections to provide file transfer protocol (FTP) functions, PC send macros, telnet interface, TCP/IP interface web server functions, and host communications.

pendant

See teach pendant.

PLC

See programmable logic controller or cell controller.

PMC

The programmable machine controller (PMC) functions provide a ladder logic programming environment to create PMC functions. This provides the capability to use the robot I/O system to run PLC programs in the background of normal robot operations. This function can be used to control bulk supply systems, fixed automation that is part of the robot workcell, or other devices that would normally require basic PLC controls.

printed circuit board

A flat board whose front contains slots for integrated circuit chips and connections for a variety of electronic components, and whose back is printed with electrically conductive pathways between the components.

production mode

See automatic mode.

program

1. A plan for the solution of a problem. A complete program includes plans for the transcription of data, coding for the computer, and plans for the absorption of the results into the system. 2. A sequence of instructions to be executed by the computer or controller to control a robot/robot system. 3. To furnish a computer with a code of instructions. 4. To teach a robot system a specific set of movements and instructions to do a task.

programmable controller

See programmable logic controller or cell controller.

programmable logic controller

A solid-state industrial control device that receives inputs from user-supplied control devices, such as switches and sensors, implements them in a precise pattern determined by ladder diagram-based programs stored in the user memory, and provides outputs for control of processes or user-supplied devices such as relays and motor starters.

Program ToolBox

The Program ToolBox software provides programming utilities such as mirror image and flip wrist editing capabilities.

protocol

A set of hardware and software interfaces in a terminal or computer that allows it to transmit over a communications network, and that collectively forms a communications language.

psi

Pounds per square inch.

Q

queue.

1. Waiting lines resulting from temporary delays in providing service. 2. The amount of time a job waits at a work center before set-up or work is performed on the job. See also job queue.

R

RAM

See Random Access Memory.

random access

A term that describes files that do not have to be searched sequentially to find a particular record but can be addressed directly.

Random Access Memory

1. Volatile, solid-state memory used for storage of programs and locations; battery backup is required. 2. The working memory of the controller. Programs and variable data must be loaded into RAM before the program can execute or the data can be accessed by the program.

range

1. A characterization of a variable or function. All the values that a function can possess. 2. In statistics, the spread in a series of observations. 3. A programmable voltage or current spectrum of values to which input or output analog signals can be limited.

RI

Robot input.

RO

Robot output.

read

To copy, usually from one form of storage to another, particularly from external or secondary storage to internal storage. To sense the meaning of arrangements of hardware. To sense the presence of information on a recording medium.

Read Only Memory

A digital memory containing a fixed pattern of bits that you cannot alter.

record

To store the current set or sets of information on a storage device.

recovery

The restoration of normal processing after a hardware or software malfunction through detailed procedures for file backup, file restoration, and transaction logging.

register

1. A special section of primary storage in a computer where data is held while it is being worked on.
2. A memory device capable of containing one or more computer bits or words.

remote/local

A device connection to a given computer, with remote devices being attached over communications lines and local devices attached directly to a computer channel; in a network, the computer can be a remote device to the CPU controlling the network.

repair

To restore robots and robot systems to operating condition after damage, malfunction, or wear.

repeatability

The closeness of agreement among the number of consecutive movements made by the robot arm to a specific point.

reset

To return a register or storage location to zero or to a specified initial condition.

restricted work envelope

That portion of the work envelope to which a robot is restricted by limiting devices that establish limits that will not be exceeded in the event of any reasonably foreseeable failure of the robot or its controls. The maximum distance the robot can travel after the limited device is actuated defines the restricted work envelope of the robot.

RIA

Robotic Industries Association Subcommittee of the American National Standards Institute, Inc.

robot

A reprogrammable multifunctional manipulator designed to move material, parts, tools, or specialized devices, through variable programmed motions for the performance of a variety of tasks.

Robot Model.

Used in Interference Checking, the Robot Model is the set of virtual model elements (sphere and cylinders) that are used to represent the location and shape of the robot arm with respect to the robot's base. Generally, the structure of a six axes robot can be accurately modeled as a series of cylinders and spheres. Each model element represents a link or part of the robot arm.

ROM

See Read Only Memory.

routine

1. A list of coded instructions in a program. 2. A series of computer instructions that performs a specific task and can be executed as often as needed during program execution.

S

saving data.

Storing program data in Flash ROM, to a floppy disk (not supported on version 7.10 and later), or memory card.

scfm

Standard cubic feet per minute.

scratch start

Allows you to enable and disable the automatic recovery function.

sensor

A device that responds to physical stimuli, such as heat, light, sound pressure, magnetism, or motion, and transmits the resulting signal or data for providing a measurement, operating a control or both. Also a device that is used to measure or adjust differences in voltage in order to control sophisticated machinery dynamically.

serial communication

A method of data transfer within a PLC whereby the bits are handled sequentially rather than simultaneously as in parallel transmission.

serial interface

A method of data transmission that permits transmitting a single bit at a time through a single line. Used where high speed input is not necessary.

Server Side Include (SSI)

A method of calling or "including" code into a web page.

servomotor

An electric motor that is controlled to produce precision motion. Also called a "smart" motor.

SI

System input.

signal

The event, phenomenon, or electrical quantity that conveys information from one point to another.

significant bit

A bit that contributes to the precision of a number. These are counted starting with the bit that contributes the most value, of "most significant bit", and ending with the bit that contributes the least value, or "least significant bit".

singulating

Separating parts into a single layer.

slip sheet

A sheet of material placed between certain layers of a unit load. Also known as tier sheet.

SO

System output.

specific gravity

The ratio of a mass of solid or liquid to the mass of an equal volume of water at 45C. You must know the specific gravity of the dispensing material to perform volume signal calibration. The specific gravity of a dispensing material is listed on the MSDS for that material.

SRAM

A read/write memory in which the basic memory cell is a transistor. SRAM (or S-RAM) tends to have a lower density than DRAM. A battery can be used to retain the content upon loss of power.

slpm

Standard liters per minute.

Standard Operator Panel (SOP).

A panel that is made up of buttons, keyswitches, and connector ports.

state

The on or off condition of current to and from and input or output device.

statement

See instruction.

storage device

Any device that can accept, retain, and read back one or more times. The available storage devices are SRAM, Flash ROM (FROM or F-ROM), floppy disks (not available on version 7.10 and later), memory cards, or a USB memory stick.

system variable

An element that stores data used by the controller to indicate such things as robot specifications, application requirements, and the current status of the system.

T

Tare

The difference between the gross weight of an object and its contents, and the object itself. The weight of an object without its contents.

TCP

See tool center point.

teaching

Generating and storing a series of positional data points effected by moving the robot arm through a path of intended motions.

teach mode

1. The mode of operation in which a robot is instructed in its motions, usually by guiding it through these motions using a teach pendant. 2. The generation and storage of positional data. Positional data can be taught using the teach pendant to move the robot through a series of positions and recording those positions for use by an application program.

teach pendant

1. A hand-held device used to instruct a robot, specifying the character and types of motions it is to undertake. Also known as teach box, teach gun. 2. A portable device, consisting of an LCD display and a keypad, that serves as a user interface to the KAREL system and attaches to the operator box or operator panel via a cable. The teach pendant is used for robot operations such as jogging the robot, teaching and recording positions, and testing and debugging programs.

telemetry

The method of transmission of measurements made by an instrument or a sensor to a remote location.

termination type

Feature that controls the blending of robot motion between segments.

tool

A term used loosely to define something mounted on the end of the robot arm, for example, a hand, gripper, or an arc welding torch.

tool center point

1. The location on the end-effector or tool of a robot hand whose position and orientation define the coordinates of the controlled object. 2. Reference point for position control, that is, the point on the tool that is used to teach positions. Abbreviated TCP.

TOOL Frame

The Cartesian coordinate system that has the position of the TCP as its origin to stet. The z-axis of the tool frame indicates the approach vector for the tool.

TP.

See teach pendant.

transducer

A device for converting energy from one form to another.

U

UOP

See user operator panel.

URL

Universal Resource Locator. A standard addressing scheme used to locate or reference files on web servers.

USB memory stick

The controller USB memory stick interface supports a USB 1.1 interface. The USB Organization specifies standards for USB 1.1 and 2.0. Most memory stick devices conform to the USB 2.0 specification for operation and electrical standards. USB 2.0 devices as defined by the USB Specification must be backward compatible with USB 1.1 devices. However, FANUC Robotics does not support any security or encryption features on USB memory sticks. The controller supports most widely-available USB Flash memory sticks from 32MB up to 1GB in size.

USER Frame

The Cartesian coordinate system that you can define for a specific application. The default value of the User Frame is the World Frame. All positional data is recorded relative to User Frame.

User Operator Panel

User-supplied control device used in place of or in parallel with the operator panel or operator box supplied with the controller. Abbreviated UOP .

V

variable

A quantity that can assume any of a given set of values.

variance

The difference between the expected (or planned) and the actual, also statistics definitions.

vision system

A device that collects data and forms an image that can be interpreted by a robot computer to determine the position or to “see” an object.

volatile memory

Memory that will lose the information stored in it if power is removed from the memory circuit device.

W

web server

An application that allows you to access files on the robot using a standard web browser.

warning device

An audible or visible device used to alert personnel to potential safety hazards.

work envelope

The volume of space that encloses the maximum designed reach of the robot manipulator including the end effector, the workpiece, and the robot itself. The work envelope can be reduced or restricted by limiting devices. The maximum distance the robot can travel after the limit device is actuated is considered the basis for defining the restricted work envelope.

write

To deliver data to a medium such as storage.

Index

D

DOSFILE_INF
    built-in procedure, A–127
DRAM, 1–8

E

Ethernet Device, 9–7, 9–11

F

F-ROM Disk, 9–2
file system, 9–2
filtered memory device, 9–7
flash file storage disk, 1–9
Flash File Storage disk (FR), 9–7, 9–9
Flash Rom (F-ROM), 1–8
FMD device*, see* filtered memory device

I

*i*RVision, A–345, A–347 to A–348, A–350 to A–351, A–353 to A–354, A–362 to A–363

J

J957 option, 9–9

M

memory, 1–8
memory card, 9–2
memory card (MC), 9–7
memory device, 9–7, 9–11
    filtered, 9–7
memory device backup, 9–11
memory device binary, 9–7
memory stick
    USB, 9–3

P

personal computer, 9–2

R

R545
    order number, 9–12
R632
    order number, 1–2
R648
    order number, 11–2
RAM Disk, 9–2
RAM Disk (RD), 9–7, 9–10

S

semaphore, 15–9
socket messaging
    configuring, 11–3
    KAREL, 11–9
    network performance, 11–10
    overview, 11–2
    programming examples, 11–11
    system requirements, 11–2
SRAM, 1–8 to 1–9
storage
    file system, 9–2
storage devices
    overview, 9–7

U

USB memory stick, 9–3
USB Memory Stick Device (UD1), 9–7 to 9–8
USB Memory Stick Device (UT1), 9–7

V

V_CAM_CALIB

built-in procedure, A–345
V_GET_OFFSET
built-in procedure, A–347
V_GET_PASSFL
built-in procedure, A–348
V_GET_QUEUE
built-in procedure, A–350
V_INIT_QUEUE
built-in procedure, A–350
V_RALC_QUEUE
built-in procedure, A–351
V_RUN_FIND
built-in procedure, A–351
V_SET_REF
built-in procedure, A–353
V_START_VTRK
built-in procedure, A–354
V_STOP_VTRK
built-in procedure, A–354
VAR_INFO
built-in procedure, A–355
VAR_LIST
built-in procedure, A–357
VECTOR
data type, A–360
VOL_SPACE
built-in procedure, A–361
VREG_FND_POS
built-in procedure, A–362
VREG_OFFSET
built-in procedure, A–363